Rust

K19G

2026-11-09

Table of contents

Rust **36**
A Complete Guide to Safe, Concurrent, High-Performance Programming 36
Introduction: Why Rust Matters 36
The Problem Rust Solves 36
How Rust Achieves This 37
What You Will Learn 37
How to Read This Book 37
The Current State of Rust 38
What Rust Is Not 38
Formats 38

Getting Started with Rust **39**

The Story of Rust: History and Philosophy **40**

The Story of Rust: History and Philosophy **41**
Core Philosophy 41

Installing Rust with rustup **42**

Installing Rust with rustup **43**
Installing rustup 43
Managing Toolchains 43

Your First Program: Hello World **44**

Your First Program: Hello World **45**
Direct Compilation with rustc 45
Deconstructing the Program 45

Understanding rustc and cargo **46**

Understanding rustc and cargo **47**
The Role of Cargo 47
Common Cargo Commands 47

Exploring a Cargo Project Structure **48**

Exploring a Cargo Project Structure **49**
Standard Directory Layout 49
The Cargo.toml Manifest 49

Summary **50**

Summary **51**
Chapter Key Takeaways 51

Language Basics: Variables, Types, and Expressions **52**

Variables, Mutability, and Shadowing **53**

Variables, Mutability, and Shadowing **54**
Mutability with `mut` 54
Variable Shadowing 54

Primitive Data Types in Detail **55**

Primitive Data Types in Detail **56**
Scalar Types 56

Compound Types: Tuples and Arrays **57**

Compound Types: Tuples and Arrays **58**
Tuples 58
Arrays 58

Functions, Statements, and Expressions **59**

Functions, Statements, and Expressions **60**
Statements vs. Expressions 60

Control Flow: if, loop, while, and for **61**

Control Flow: if, loop, while, and for **62**
`if` as an Expression 62
Loops: `loop`, `while`, and `for` 62

NonZero Integer Types **63**

NonZero Integer Types **64**
Thesis 64
Mental model 64
Worked example 64
Trap 65
Boring rule 65
Try this 65
Sources 65

Summary **66**

Summary **67**
Chapter Key Takeaways 67

Ownership: The Heart of Rust **68**

What is Ownership and Why Does It Exist? **69**

What is Ownership and Why Does It Exist? **70**
The Three Rules of Ownership 70

Stack vs Heap Memory in Rust **71**

Stack vs Heap Memory in Rust **72**
The Stack 72
The Heap 72

Move Semantics Explained **73**

Move Semantics Explained **74**
Why Rust Moves Instead of Shallow Copying 74

Copy Types and Clone **75**

Copy Types and Clone **76**
The `Copy` Trait 76
The `Clone` Trait 76

The Drop Trait and Resource Cleanup **77**

The Drop Trait and Resource Cleanup **78**
Custom Drop Implementations 78

The Cost of Drop on Deep Ownership Chains **79**

The Cost of Drop on Deep Ownership Chains **80**
Thesis 80
Mental model 80
Worked examples 80
Trap 80
Boring rule 81
Try this 81
Sources 81

Summary **82**

Summary **83**
Chapter Key Takeaways 83

Borrowing, References, and Slices **84**

Shared and Mutable References **85**

Shared and Mutable References **86**
Shared References (`&T`) . . . 86
Mutable References (`&mut T`) . . . 86

The Borrowing Rules Demystified **87**

The Borrowing Rules Demystified **88**
Why the Rules Exist . . . 88

Dangling References Prevented at Compile Time **89**

Dangling References Prevented at Compile Time **90**

String Slices and &str **91**

String Slices and &str **92**
`String` vs `&str` . . . 92

Slice Types and Range Syntax **93**

Slice Types and Range Syntax **94**

Summary **95**

Summary **96**
Chapter Key Takeaways . . . 96

Structs, Enums, and Pattern Matching **97**

Defining and Using Structs **98**

Defining and Using Structs **99**

Enumerations Beyond Simple Tags **100**

Enumerations Beyond Simple Tags **101**

The match Expression in Depth **102**

The match Expression in Depth **103**

if let and while let for Partial Matching **104**

if let and while let for Partial Matching **105**

if-let Guards and cfg_select! **106**

if-let Guards and `cfg_select!` **107**
Thesis . . . 107
Mental model . . . 107
Worked example . . . 108

The traps . . . 110
Migrating from `cfg-if` . . . 111
Try this . . . 112
Sources . . . 112

Real-World Data Modeling with Enums **113**

Real-World Data Modeling with Enums **114**

Summary **115**

Summary **116**
Chapter Key Takeaways . . . 116

Modules, Crates, and Visibility **117**

Modules and the Filesystem **118**

Modules and the Filesystem **119**
The Modern Module Mapping . . . 119

The use Statement and Path Resolution **120**

The use Statement and Path Resolution **121**

Public vs Private Visibility Rules **122**

Public vs Private Visibility Rules **123**

Crates, Packages, and Cargo Workspaces **124**

Crates, Packages, and Cargo Workspaces **125**

Re-exporting and Organizing Large Projects **126**

Re-exporting and Organizing Large Projects **127**

Summary **128**

Summary **129**
Chapter Key Takeaways . . . 129

Collections and Strings **130**

Vec and Dynamic Arrays **131**

Vec and Dynamic Arrays **132**

String vs &str Deep Dive **133**

String vs &str Deep Dive **134**

HashMap and BTreeMap for Key-Value Storage **135**

HashMap and BTreeMap for Key-Value Storage **136**

HashMap Hashers and Hash Flooding: Why the Default Is SipHash **137**

HashMap Hashers and Hash Flooding: Why the Default Is SipHash **138**
- Thesis . . . 138
- How a Hash Table Degrades . . . 138
  - The Attack Mechanics . . . 139
- Why a Keyed Random Hash Stops HashDoS . . . 139
  - Historical Context . . . 140
- Code Walkthrough: Demonstrating the Vulnerability . . . 140
  - Project Setup . . . 140
  - Key Components . . . 142
- Program Execution Results . . . 142
  - Analyzing the Output Pattern . . . 143
- Non-Deterministic Iteration Order . . . 143
- Choosing a Hasher . . . 143
  - Decision Rules . . . 144
- Pitfalls and Common Mistakes . . . 145
  - Global Hasher Overrides in Web Services . . . 145
  - Implementing `Hasher` by Ignoring Bytes . . . 145
  - Assuming Default Hasher Stability . . . 145
  - Floating-Point Keys . . . 145
- Exercises . . . 145
- Key Takeaways . . . 146
- Sources . . . 146

bbtree: a concurrent ordered map (and unsync BTreeMap drop-in) **147**

bbtree: a concurrent ordered map (and unsync BTreeMap drop-in) **148**
- Thesis . . . 148
- Architecture and Concurrency Model . . . 148
  - Standard BTreeMap vs Concurrent BBTreeMap . . . 149
  - API Differences and Copy Semantics . . . 149
- Code Walkthrough: Concurrent and Unsync Operations . . . 150
  - Project Configuration . . . 150
  - Main Application Code . . . 150
- Common Mistakes and Pitfalls . . . 151
  - Ignored Return Values on Insert . . . 151
  - Expecting Standard `len()` on `BBTreeMap` . . . 151
  - Expecting `&V` References from `BBTreeMap` . . . 152
- When to Keep `std::collections::BTreeMap` . . . 152
- Summary . . . 152
- Verified output . . . 152
- Sources . . . 153

Other Collections: HashSet, BinaryHeap, LinkedList **154**

Other Collections: HashSet, BinaryHeap, LinkedList **155**

Efficient Collection Operations **156**

Efficient Collection Operations **157**

Summary **158**

Summary **159**
Chapter Key Takeaways 159

Project 1: Building a CLI Task Manager **160**

Building a CLI Task Manager **161**

Building a CLI Task Manager **162**

Testing the Task Manager **164**

Testing the Task Manager **165**

Benchmarking the Task Manager **166**

Benchmarking the Task Manager **167**

Deploying the Task Manager **168**

Deploying the Task Manager **169**

Error Handling in Rust **170**

Unrecoverable Errors with panic! **171**

Unrecoverable Errors with panic! **172**

Recoverable Errors with Result **173**

Recoverable Errors with Result **174**

The Option Type for Absent Values **175**

The Option Type for Absent Values **176**

Propagating Errors with the ? Operator **177**

Propagating Errors with the ? Operator **178**

Custom Error Types and thiserror **179**

Custom Error Types and thiserror **180**

The Never Type ! and Infallible **181**

The Never Type ! and `Infallible` **182**
Thesis . . . 182
Mental model . . . 182
Worked examples . . . 183
Case 1: ! as a type — cannot-fail results, irrefutable `Ok` . . . 183
Case 2: the fallback change — code that relied on () by accident . . . 185
Case 3: the panicking closure . . . 187
Case 4: `Infallible` and ! are one type now . . . 188
Where ! belongs in error handling . . . 189
The trap . . . 189
The boring rule . . . 190
Try this . . . 190
Sources . . . 190

Summary **192**

Summary **193**
Chapter Key Takeaways . . . 193

Generics, Traits, and Associated Types **194**

Generic Functions and Data Structures **195**

Generic Functions and Data Structures **196**

Defining and Implementing Traits **197**

Defining and Implementing Traits **198**

Trait Bounds and Where Clauses **199**

Trait Bounds and Where Clauses **200**

Associated Types vs Generics **201**

Associated Types vs Generics **202**

Trait Objects and Dynamic Dispatch **203**

Trait Objects and Dynamic Dispatch **204**

Generic Const Arguments (GCA) **205**

Generic Const Arguments (GCA) **206**
Thesis . . . 206
Mental model . . . 206

Worked examples . . . 207
Case 0: stable still rejects `N + 1` . . . 207
Case 1: ADT const arguments with `gca_adts` . . . 208
Case 2: Associated constants with `gca_const_items` . . . 209
Case 3: Macroless (experimental) . . . 209
Case 4: What still hurts — arithmetic inside `gca!` . . . 210
The trap . . . 211
The boring rule . . . 211
Try this . . . 211
Sources . . . 211

AsRef, Borrow and Deref: Same Signature, Different Promises **212**

`AsRef`, `Borrow` and `Deref`: Same Signature, Different Promises **213**
Thesis . . . 213
Mental model . . . 213
Worked examples . . . 214
Case 1: How `HashMap<String, V>` answers a `&str` . . . 214
Case 2: `AsRef` as an "accept anything" bound . . . 216
Case 3: Break the `Borrow` contract and the map loses keys . . . 217
Case 4: The correct shape: borrow as a type with the same `Hash` and `Eq` . . . 219
Case 5: `Borrow` is reflexive, `AsRef` is not . . . 221
Case 6: `ToOwned` is `Borrow` run backwards, and `Cow` stands on both . . . 222
The trap . . . 223
The boring rule . . . 223
Try this . . . 224
Sources . . . 224

Cow — Borrow When You Can, Own When You Must **225**

Cow — Borrow When You Can, Own When You Must **226**
Thesis . . . 226
Mental model . . . 226
Worked examples . . . 226
Trap . . . 226
Boring rule . . . 227
Try this . . . 227
Sources . . . 227

PhantomData and the Type-State Pattern: Protocol States the Compiler Enforces **228**

`PhantomData` and the Type-State Pattern: Protocol States the Compiler Enforces **229**
Thesis . . . 229
Why you need `PhantomData` at all . . . 229
The type-state connection . . . 230
What the compiler catches . . . 232
Choosing the right `PhantomData` . . . 233
When not to use type-state . . . 235
Takeaways . . . 235

Sources . . . 235

The Newtype Pattern: Zero-Cost Type Safety, Validated Values and the Orphan Rule 236

The Newtype Pattern: Zero-Cost Type Safety, Validated Values and the Orphan Rule 237
Thesis . . . 237
1. The swapped-argument bug . . . 238
2. Zero cost, and what `repr(transparent)` adds . . . 239
3. Validated values: parse, don't re-check . . . 241
4. Getting around the orphan rule . . . 243
Pitfalls . . . 244
Takeaways . . . 244
Sources . . . 244

Summary 245

Summary 246
Chapter Key Takeaways . . . 246

Lifetimes: Complete Understanding 247

What Lifetimes Are (and Are Not) 248

What Lifetimes Are (and Are Not) 249
What Lifetimes Are Not . . . 249

The Lifetime Elision Rules 250

The Lifetime Elision Rules 251

Annotating Lifetimes in Functions and Structs 252

Annotating Lifetimes in Functions and Structs 253

Lifetime Bounds on Traits 254

Lifetime Bounds on Traits 255

Advanced Lifetime Patterns 256

Advanced Lifetime Patterns 257
The `'static` Lifetime . . . 257

Variance: Why `&mut` Makes Lifetimes Rigid 258

Variance: Why `&mut` Makes Lifetimes Rigid 259
Thesis . . . 259
Subtyping, but only for lifetimes . . . 259
Covariant: shared references shrink freely . . . 260
Invariant: `&mut T` can't shrink its `T` . . . 260

Contravariant: function arguments flip the direction . . . 262
The common case: `&'a mut self` on a type with `'a` . . . 263
The variance table . . . 264
Pitfalls . . . 265
Takeaways . . . 265
Sources . . . 266

Summary **267**

Summary **268**
Chapter Key Takeaways . . . 268

Smart Pointers and Interior Mutability **269**

Box for Heap Allocation **270**

Box for Heap Allocation **271**

Reference Counting with Rc and Arc **272**

Reference Counting with Rc and Arc **273**

Interior Mutability with RefCell **274**

Interior Mutability with RefCell **275**

Cell and Mutex for Shared State **276**

Cell and Mutex for Shared State **277**

Weak References and Breaking Cycles **278**

Weak References and Breaking Cycles **279**

LazyLock and OnceLock **280**

LazyLock and OnceLock **281**
Mental model . . . 281
Worked examples . . . 282
Case 1: `LazyLock` for a process-wide regex . . . 282
Case 2: `OnceLock` when init needs an argument . . . 282
Case 3: Fallible init without poisoning `LazyLock` . . . 283
Case 4: Single-threaded `LazyCell` . . . 284
Choosing among the options . . . 284
Pitfalls . . . 284
Try this . . . 285
Sources . . . 285

MaybeUninit and Uninitialized Memory **286**

MaybeUninit and Uninitialized Memory **287**
Mental model . . . 287
Worked examples . . . 288
Case 1: `write` then `assume_init` . . . 288
Case 2: Out-pointer style . . . 288
Case 3: Array element-by-element . . . 288
Case 4: Field-by-field struct init . . . 289
Case 5: `zeroed` pitfalls . . . 290
Choosing the API . . . 290
Pitfalls . . . 291
Try this . . . 291
Sources . . . 291

Custom Allocators and the Allocator API **292**

Custom Allocators and the Allocator API **293**
Thesis . . . 293
Mental model . . . 293
Worked examples . . . 294
Case 1: a counting allocator and a bump arena . . . 294
Case 2: what the allocator parameter costs . . . 298
Case 3: an allocator says no . . . 299
Case 4: what is *not* stable yet . . . 301
The trap . . . 303
A collection outlives its arena . . . 303
`Vec<T, A>` does not fit where `Vec<T>` is expected . . . 304
Implementing the trait is a safety contract . . . 305
Choosing an approach . . . 305
The boring rule . . . 305
Try this . . . 306
Sources . . . 306

Arc::new_cyclic: self-referential shared ownership without a temporary Option **307**

Arc::new_cyclic: self-referential shared ownership without a temporary Option **308**
The problem `new_cyclic` solves . . . 308
Rules inside the closure . . . 308
Lab: self weak and parent weak . . . 309
Strong cycles versus weak parents . . . 310
When to use `new_cyclic` . . . 310
Pitfalls . . . 310
Single-threaded twin: `Rc::new_cyclic` . . . 311
Design sketch: arena of nodes . . . 311
Testing upgrades . . . 311
Summary . . . 311
Sources . . . 312

Summary **313**

Summary **314**
Chapter Key Takeaways 314

Iterators, Closures, and Functional Patterns **315**

The Iterator Trait and Protocol **316**

The Iterator Trait and Protocol **317**

Creating Custom Iterators **318**

Creating Custom Iterators **319**

Closures and Capture Semantics **320**

Closures and Capture Semantics **321**

Higher-Order Functions and Combinators **322**

Higher-Order Functions and Combinators **323**

Lazy Evaluation and Performance **324**

Lazy Evaluation and Performance **325**

FusedIterator — Why Fuse Matters **326**

FusedIterator — Why Fuse Matters **327**
Thesis 327
Mental model 327
Worked example 327
Trap 328
Boring rule 328
Try this 328
Sources 328

Fast Float Reductions: Algebraic Operations vs Explicit Lanes **329**

Fast Float Reductions: Algebraic Operations vs Explicit Lanes **330**
Thesis 330
Mental model 330
Worked example: three dot products 330
Choosing between them 333
Things that bite 334
Try this 334
Sources 334

Copy Range Types: core::range and Why a..b Was Never Copy **335**

Copy Range Types: core::range and Why a..b Was Never Copy **336**
The problem: structs, spans, and error E0204 . . . 336
Why std::ops::Range was never Copy . . . 337
The new types in core::range . . . 338
IntoIterator vs Iterator: decoupling bounds from traversal . . . 339
Syntax and converting between legacy and new . . . 340
Converting from legacy to modern ranges . . . 340
Converting from modern ranges to legacy ranges . . . 340
Slicing with core::range . . . 341
Worked example: a complete copyable span . . . 341
API design guidance: when and how to expose modern ranges . . . 343
Store core::range in structs . . . 343
Accepting ranges in public functions . . . 343
Pitfalls . . . 344
1. Assuming range syntax emits modern ranges in edition 2024 . . . 344
2. Calling .next() directly on core::range::Range . . . 344
3. Re-iteration behavior differences . . . 344
4. Field names in RangeInclusive . . . 345
Summary . . . 345
Sources . . . 345

Summary **346**

Summary **347**
Chapter Key Takeaways . . . 347

Concurrency: Fearless Parallelism **348**

Spawning Threads with std::thread **349**

Spawning Threads with std::thread **350**

Message Passing with Channels **351**

Message Passing with Channels **352**

Shared State with Mutex and RwLock **353**

Shared State with Mutex and RwLock **354**

RwLock Is Not a Faster Mutex: Lock Choice by Measurement, and Locks Across .await **355**

RwLock Is Not a Faster Mutex: Lock Choice by Measurement, and Locks Across `.await` **356**
Thesis . . . 356
Mental model . . . 356
Worked examples . . . 357
Case 1: A tiny read-only critical section . . . 359
Case 2: The same reads with real work inside . . . 360
Case 3: One write in a hundred . . . 361

Case 4: Stop sharing the written word: a big-reader lock . . . 362
Case 5: A `std::sync::MutexGuard` held across `.await` . . . 363
The trap . . . 367
The boring rule . . . 368
Try this . . . 368
Sources . . . 369

Atomic Operations for Lock-Free Code **370**

Atomic Operations for Lock-Free Code **371**

Send and Sync Traits Explained **372**

Send and Sync Traits Explained **373**

Summary **374**

Summary **375**
Chapter Key Takeaways . . . 375

Project 3: Parallel Data Processing Pipeline with rayon **376**

Parallel Data Processing Pipeline with rayon **377**

Parallel Data Processing Pipeline with rayon **378**

Testing the Text Analyzer **379**

Testing the Text Analyzer **380**

Benchmarking the Text Analyzer **381**

Benchmarking the Text Analyzer **382**

Deploying the Text Analyzer **383**

Deploying the Text Analyzer **384**

Asynchronous Programming with async/await **385**

The Future Trait and Async Basics **386**

The Future Trait and Async Basics **387**

Running Async Code with Tokio **388**

Running Async Code with Tokio **389**

Pinning and Self-Referential Types **390**

Pinning and Self-Referential Types **391**

Async Streams and Error Handling **392**

Async Streams and Error Handling **393**

Building Production Async Applications **394**

Building Production Async Applications **395**

Tokio and Rayon: Separate Lanes for I/O and CPU Work **396**

Tokio and Rayon: Separate Lanes for I/O and CPU Work **397**
- Thesis . . . 397
- Mental model . . . 397
- Worked examples . . . 398
  - Case 1: Same throughput, very different latency . . . 398
  - Case 2: Under a CPU limit . . . 400
  - Case 3: Bound the work in flight . . . 401
- The trap . . . 401
- The boring rule . . . 402
- Try this . . . 403
- Sources . . . 403

Sans-IO Protocol Design: State Machines Without Sockets **404**

Sans-IO Protocol Design: State Machines Without Sockets **405**
- Thesis . . . 405
- Mental model . . . 405
- Worked examples . . . 406
  - Case 1: a length-prefixed framer with no sockets . . . 406
  - Case 2: the driver loop (sketch) . . . 409
  - Case 3: why this pairs with Rust . . . 410
- The trap . . . 410
- The boring rule . . . 411
- Try this . . . 411
- Sources . . . 411

Cancel Safety: Dropping a Future Is Cancelling It **412**

Cancel Safety: Dropping a Future Is Cancelling It **413**
- Thesis . . . 413
- Mental model . . . 413
- Worked examples . . . 414
  - Case 1: A future is a value, and dropping it cancels it . . . 414
  - Case 2: `send` under a timeout loses the message, `reserve` does not . . . 416
  - Case 3: A `select!` loop over `read_exact` corrupts a framed stream . . . 419
  - Case 3b: Or don't cancel at all: poll one future by `&mut` . . . 421
  - Case 4: Cancel correctness: an invariant broken across an `.await` . . . 423
- The trap . . . 424

The boring rule . . . 426
Try this . . . 426
Sources . . . 426

A Tiny Async Runtime from Scratch **428**

A Tiny Async Runtime from Scratch **429**
Thesis . . . 429
Mental model . . . 429
Worked example . . . 429
Trap . . . 430
Boring rule . . . 430
Try this . . . 430
Sources . . . 431

async fn in Traits vs dyn Trait: Desugaring, Send Bounds and Boxing **432**

async fn in Traits vs dyn Trait: Desugaring, Send Bounds and Boxing **433**
Thesis . . . 433
Failure 1: `dyn Store` is rejected . . . 434
Failure 2: callers can't require `Send` . . . 434
The compiler's own warning . . . 435
Fix 1: desugar and promise `Send` . . . 435
Fix 2: a boxed, dyn-compatible twin . . . 436
Running it with nothing but std . . . 437
Choosing between them . . . 438
Takeaways . . . 438

Summary **439**

Summary **440**
Chapter Key Takeaways . . . 440

Macros, Unsafe Rust, and FFI **441**

Declarative Macros with macro_rules! **442**

Declarative Macros with macro_rules! **443**

Procedural Macros and Derive **444**

Procedural Macros and Derive **445**

Understanding unsafe Rust **446**

Understanding unsafe Rust **447**

Memory Layout and Representation **448**

Memory Layout and Representation **449**

Foreign Function Interfaces **450**

Foreign Function Interfaces **451**

C-Variadic Functions and VaList **452**

C-Variadic Functions and `VaList` **453**
Thesis . . . 453
Mental model . . . 453
Worked examples . . . 454
Case 1: count-prefixed list, defined and called in Rust . . . 454
Case 2: a printf-style logger implemented in Rust, called from C . . . 455
Case 3: the header catches caller mistakes before they become UB . . . 458
Case 4: what the compiler refuses . . . 459
The trap . . . 461
The boring rule . . . 462
Try this . . . 462
Sources . . . 463

FFI bool miscompile on x86_64 (Rust 1.99.0 → 1.99.1) **464**

FFI bool miscompile on x86_64 (Rust 1.99.0 → 1.99.1) **465**
Learning goals . . . 465
Thesis . . . 465
Mental model . . . 465
Case 1 — Minimal reproduction . . . 466
Genuine output on this box (rustc 1.99.0) . . . 467
Case 2 — Who was affected . . . 467
Case 3 — Mitigations . . . 467
The trap . . . 468
The boring rule . . . 468
Try this . . . 468
Sources . . . 468
Inspiration . . . 468
Verification . . . 469

Safe Rust UAF via covariant field assignment (rust#164250) **470**

Safe Rust UAF via covariant field assignment (rust#164250) **471**
Learning goals . . . 471
Thesis . . . 471
Mental model . . . 471
Worked examples . . . 472
Case 1: Minimal Drop smuggling (reproduced here) . . . 472
Case 2: `'static` bound bypass with a kept value . . . 473
Case 3: What to check in your code . . . 474
The trap . . . 474
The boring rule . . . 474
Try this . . . 474

Sources . . . 475

Life before main: .init_array, #[used], and why LazyLock is usually the answer 476
The mechanism, from the ELF spec . . . 476
The whole program . . . 477
Real output (rustc 1.99.1, x86_64 Linux, glibc 2.41) . . . 478
Drop `#[used]` and it silently disappears . . . 478
Edition 2024: `link_section` is an unsafe attribute . . . 479
What to do instead, most of the time . . . 479
Checklist . . . 480

Summary 481

Summary 482
Chapter Key Takeaways . . . 482

Testing, Documentation, Performance, and Deployment 483

Unit Tests, Integration Tests, and Doctests 484

Unit Tests, Integration Tests, and Doctests 485
Unit Tests . . . 485
Integration Tests . . . 485
Documentation Tests (Doctests) . . . 485

Benchmarking with Criterion 486

Benchmarking with Criterion 487

Writing Great Documentation 488

Writing Great Documentation 489

Profiling and Optimization Techniques 490

Profiling and Optimization Techniques 491

Building and Deploying Rust Applications 492

Building and Deploying Rust Applications 493

MSRV in practice: rust-version and the MSRV-aware resolver 494

MSRV in practice: `rust-version` and the MSRV-aware resolver 495
Learning goals . . . 495
Thesis . . . 495
Mental model . . . 495
Worked examples . . . 497
Case 1 — The break is two levels down . . . 497
Case 2 — Let the resolver keep the promise . . . 499

Case 3 — `cargo add` is MSRV-aware; your requirement wins over the resolver . . . 500
Case 4 — Your own `rust-version`, the escape hatch, and `--locked` . . . 501
Case 5 — No `rust-version`? Then your toolchain is the MSRV . . . 502
Case 6 — Check the MSRV in CI (configuration, not run here) . . . 503
The trap . . . 503
The boring rule . . . 504
Try this . . . 504
Sources . . . 504

build.rs Runs As You: Build Scripts and the Supply Chain **505**

build.rs Runs As You: Build Scripts and the Supply Chain **506**
Thesis . . . 506
A harmless look-alike . . . 506
When it runs . . . 507
Defences, in order of payoff . . . 508
Takeaways . . . 509
Sources . . . 509

Cargo min-publish-age: dependency cooldowns against fresh crates.io publishes **510**

Cargo min-publish-age: dependency cooldowns against fresh crates.io publishes **511**
Supply-Chain Window of Vulnerability . . . 511
Configuring `min-publish-age` . . . 511
Configuration Parameters . . . 512
Resolver Policy Modes . . . 512
Setting Up a Demonstration Workspace . . . 513
How `min-publish-age` Fits into Rust Security Defenses . . . 514
Pitfalls . . . 514
Key takeaways . . . 515
Sources . . . 515

Summary **516**

Summary **517**
Chapter Key Takeaways . . . 517

The Rust Ecosystem: Essential Crates for Production Development **518**

serde: Serialization and Deserialization **519**

serde: Serialization and Deserialization **520**

clap: Command-Line Argument Parsing **521**

clap: Command-Line Argument Parsing **522**

reqwest: HTTP Client **523**

reqwest: HTTP Client **524**

axum: Web Framework **525**

axum: Web Framework **526**

actix-web: High-Performance Web Framework **527**

actix-web: High-Performance Web Framework **528**

Project 2: Async HTTP API with axum, sqlx, and tracing **529**

Project 2: Async HTTP API with axum, sqlx, and tracing **530**

Testing the User API **531**

Testing the User API **532**

Benchmarking the User API **533**

Benchmarking the User API **534**

Deploying the User API **535**

Deploying the User API **536**

sqlx: Async Database Access **537**

sqlx: Async Database Access **538**

rayon: Data Parallelism **539**

rayon: Data Parallelism **540**

tracing: Structured Logging and Observability **541**

tracing: Structured Logging and Observability **542**

chrono: Date and Time **543**

chrono: Date and Time **544**

uuid: Universally Unique Identifiers **545**

uuid: Universally Unique Identifiers **546**

rand: Random Number Generation **547**

rand: Random Number Generation **548**

regex: Regular Expressions **549**

regex: Regular Expressions **550**

bytes: Efficient Binary Buffers **551**

bytes: Efficient Binary Buffers **552**

futures: Async Primitives **553**

futures: Async Primitives **554**

Summary **555**

Summary **556**
Chapter Key Takeaways . . . 556

Servo 0.1: Embeddable Engine on crates.io **557**

Servo 0.1: Embeddable Engine on crates.io **558**
Thesis . . . 558
Mental model . . . 558
Worked examples . . . 559
Case 1: Read the milestone announcement, then pin deliberately . . . 559
Case 2: Embedding API shape (no full build in this chapter) . . . 560
Case 3: Building without `./mach` (embedder perspective) . . . 560
Case 4: Migrating off 0.1 LTS (preview) . . . 561
The trap . . . 561
The boring rule . . . 561
Try this . . . 561
Sources . . . 562

Advanced Topics: Compiler, Debugging, Security, and Anti-Patterns **563**

The Rust Compilation Pipeline **564**

The Rust Compilation Pipeline **565**

A Tiny JIT with Cranelift: From Postfix Expressions to Native Code **566**

A Tiny JIT with Cranelift: From Postfix Expressions to Native Code **567**
Thesis . . . 567
When a JIT pays off . . . 567
The Cranelift crate ecosystem . . . 568
Program walkthrough . . . 568
Parsing and reference interpretation . . . 572
Setting up the Cranelift JIT environment . . . 572
Constructing SSA IR with FunctionBuilder . . . 573
Finalizing compilation and function transmutation . . . 574
Reading the Cranelift IR . . . 574
Analyzing the IR output . . . 575
Performance and timing analysis . . . 576
Error handling . . . 576

Safety rules for JIT code . . . 576
Common mistakes . . . 577
Exercises . . . 577
Summary . . . 578
Sources . . . 578

Debugging Rust Programs **579**

Debugging Rust Programs **580**

Security Considerations in Rust **581**

Security Considerations in Rust **582**

Common Anti-Patterns and How to Avoid Them **583**

Common Anti-Patterns and How to Avoid Them **584**

Summary **585**

Summary **586**
Chapter Key Takeaways . . . 586

WebAssembly with Rust **587**

Why Rust for WebAssembly? **588**

Why Rust for WebAssembly? **589**

WebAssembly Fundamentals **590**

WebAssembly Fundamentals **591**

Setting Up: wasm-pack and wasm-bindgen **592**

Setting Up: wasm-pack and wasm-bindgen **593**

Building Your First WebAssembly Module **594**

Building Your First WebAssembly Module **595**

Interacting with the DOM **596**

Interacting with the DOM **597**

Handling Events and Async **598**

Handling Events and Async **599**

Performance Considerations **600**

Performance Considerations **601**

Server-Side WebAssembly **602**

Server-Side WebAssembly **603**

Publishing and Distribution **604**

Publishing and Distribution **605**

Summary **606**

Summary **607**
Chapter Key Takeaways 607

Embedded Rust and no_std Development **608**

Understanding no_std **609**

Understanding no_std **610**

Choosing a Target: Cortex-M **611**

Choosing a Target: Cortex-M **612**

Writing an Embedded Application: Blinking an LED **613**

Writing an Embedded Application: Blinking an LED **614**

Memory Management Without a Heap **615**

Memory Management Without a Heap **616**

Interrupts and Concurrency **617**

Interrupts and Concurrency **618**

Choosing an Executor or RTOS **619**

Choosing an Executor or RTOS **620**

Testing Embedded Code **621**

Testing Embedded Code **622**

Toolchain and Debugging **623**

Toolchain and Debugging **624**

Summary **625**

Summary **626**
Chapter Key Takeaways . . . 626

Command-Line Tools in Rust **627**

Command-Line Tools in Rust **628**

Command-Line Tools in Rust **629**

Part 1 – Foundations **630**

Introduction to Command-Line Tools in Rust **631**

Introduction to Command-Line Tools in Rust **632**
Why Rust Dominates the Modern CLI Landscape . . . 632
Pillars of a Modern Rust CLI . . . 632

Setting Up Your Development Environment **633**

Setting Up Your Development Environment **634**
Toolchain Installation . . . 634
Essential Cargo Extensions for CLI Developers . . . 634
Workspace Initialization for CLI Applications . . . 634
Recommended `Cargo.toml` Release Profile . . . 635

Command-Line Fundamentals **636**

Command-Line Fundamentals **637**
Process Arguments (`std::env::args`) . . . 637
Standard I/O Streams: Stdin, Stdout, and Stderr . . . 637
Exit Status Codes . . . 638

Parsing Arguments with clap **639**

Parsing Arguments with clap **640**
Adding clap with Derive . . . 640
Basic CLI Definition . . . 640
Subcommands with Enums . . . 641

Part 2 – Core Skills **643**

Working with the File System **644**

Working with the File System **645**
Path Manipulations with `Path` and `PathBuf` . . . 645
Efficient File Reading and Writing . . . 645
Fast Directory Traversal with `walkdir` . . . 646

Processing Text Data **648**

Processing Text Data **649**
Efficient In-Memory String Operations . . . 649
Reusable Buffers for High-Throughput Streaming . . . 649
Pattern Matching with `regex` . . . 650
Streaming JSON with `serde_json` . . . 650

Error Handling **651**

Error Handling **652**
The CLI Error Hierarchy: Application vs Library . . . 652
Using `anyhow` for Top-Level CLI Flow . . . 652
Domain-Specific Errors with `thiserror` . . . 653

Structuring Your Application **654**

Structuring Your Application **655**
Separation of Concerns: The `main.rs` vs `lib.rs` Pattern . . . 655
Writing a Clean `main.rs` Dispatcher . . . 655
Benefits of the Dual-Target Layout . . . 656

Part 3 – Networking & Async **657**

Running External Processes **658**

Running External Processes **659**
Spawning Subprocesses with `std::process::Command` . . . 659
Piping Data Between Commands (Unix Pipeline Emulation) . . . 659
Security Best Practice: Never Invoke via Raw Shell . . . 660

Building HTTP Clients **661**

Building HTTP Clients **662**
Adding `reqwest` and `serde` . . . 662
Synchronous (Blocking) Client for Simple CLIs . . . 662
Streaming Downloads with Progress Feedback . . . 663

Async Programming with tokio **664**

Async Programming with tokio **665**
Configuring the Tokio Runtime for CLI Applications . . . 665
The `#[tokio::main]` Attribute . . . 665
Bounded Concurrency with `tokio::sync::Semaphore` . . . 666
Graceful Cancellation via `tokio::select!` . . . 666

Terminal User Interfaces with ratatui **668**

Terminal User Interfaces with ratatui **669**
Adding `ratatui` and `crossterm` . . . 669
Raw Mode and Alternate Screen Lifecycle . . . 669
Immediate-Mode Rendering Loop . . . 670

Part 4 – Testing & Distribution **672**

Configuration and Persistence **673**

Configuration and Persistence **674**
The Configuration Precedence Hierarchy . . . 674
Standard Directory Paths with `dirs` . . . 674

Logging and Observability **676**

Logging and Observability **677**
Structured Instrumentation with `tracing` . . . 677
Configuring `tracing-subscriber` for CLIs . . . 677
Progress Bars and Spinners with `indicatif` . . . 678

Testing Command-Line Tools **679**

Testing Command-Line Tools **680**
Testing Crates Setup . . . 680
Black-Box Integration Tests with `assert_cmd` . . . 680
Mocking Standard Input (Stdin) . . . 681

Building Cross-Platform Tools **682**

Building Cross-Platform Tools **683**
Key Operating System Divergences . . . 683
Idiomatic Cross-Platform Patterns in Rust . . . 683
1. Always Use `std::path::PathBuf` and Slices . . . 683
2. Normalizing Line Endings . . . 683
3. Conditional Compilation with `#[cfg(...)]` . . . 684
Cross-Compiling with `cross` . . . 684

Part 5 – Advanced **685**

Designing for Production **686**

Designing for Production **687**
Signal Handling and Graceful Shutdown . . . 687
Machine-Readable Output: The `--json` Contract . . . 688
Respecting `NO_COLOR` and Terminal Detection . . . 688

Advanced Rust Patterns for CLI Tools **689**

Advanced Rust Patterns for CLI Tools **690**
Zero-Cost Parallelism with `rayon` . . . 690
Memory Mapping for Massive Files (`memmap2`) . . . 690
Static Global Initialization with `LazyLock` . . . 691

Building Developer Tools **692**

Building Developer Tools **693**
Dynamic Shell Auto-Completion with `clap_complete` . . . 693
Interactive Inquirer Prompts with `inquire` . . . 694

Distributing and Shipping Your Tool **696**

Distributing and Shipping Your Tool **697**
Distribution Channels for Rust CLIs . . . 697
Automated Multi-Platform Release with GitHub Actions and `cargo-dist` . . . 697
Creating Homebrew Formulas . . . 698

Part 6 – Capstone Projects **699**

Capstone 1: fsearch, a Fast File Search Tool **700**

Capstone 1: fsearch, a Fast File Search Tool **701**
Architecture and Dependencies . . . 701
CLI Definition . . . 701
Parallel Traversal and Matching Engine . . . 702
Verification . . . 703

Capstone 2: tudo, a TUI Task Manager **704**

Capstone 2: tudo, a TUI Task Manager **705**
Dependencies . . . 705
Data Model and Persistence . . . 705
Interactive Event Loop . . . 706

Capstone 3: sysmon, a System Monitor **708**

Capstone 3: sysmon, a System Monitor **709**
Dependencies . . . 709
System Telemetry Engine . . . 709
Periodic Observation Loop . . . 710

Capstone 4: devkit, a Developer Toolkit **712**

Capstone 4: devkit, a Developer Toolkit **713**
Dependencies . . . 713
CLI Specification . . . 713
Modular Implementation . . . 714

Part 7 – The Rust CLI Ecosystem **716**

Contributing to the Rust Ecosystem **717**

Contributing to the Rust Ecosystem **718**
The CLI Working Group (`rust-cli`) . . . 718
Extracting Domain Crates . . . 718

The Future of Rust CLI Development **719**

The Future of Rust CLI Development **720**
Emerging Frontiers in Terminal Applications . . . 720
1. WebAssembly and WASI (WebAssembly System Interface) . . . 720
2. Next-Generation Terminal Protocols . . . 720
3. Headless Tooling for Autonomous AI Agents . . . 720

Conclusion: The Rust Mindset **721**

Core Principles **722**

Core Principles **723**

Where Rust Shines **724**

Where Rust Shines **725**

Where Rust May Not Be the Best Fit **726**

Where Rust May Not Be the Best Fit **727**

Continuing Your Journey **728**

Continuing Your Journey **729**

Final Thoughts **730**

Final Thoughts **731**

Rust Atomics and Locks **732**

Preface and Foreword **733**

Preface and Foreword **734**
Foreword by Paul E. McKenney . . . 734
Preface . . . 735
Who This Book Is For . . . 736
Overview of the Chapters . . . 736
Code Examples . . . 737
Conventions Used in This Book . . . 738

Contact Information . . . 739

Basics of Rust Concurrency **740**

Basics of Rust Concurrency **741**
Threads in Rust . . . 741
Thread ID . . . 742
Output Locking . . . 743
Thread Builder . . . 744
Scoped Threads . . . 745
The Leakpocalypse . . . 746
Shared Ownership and Reference Counting . . . 746
Statics . . . 747
Leaking . . . 747
Reference Counting . . . 747
Borrowing and Data Races . . . 749
Undefined Behavior . . . 750
Interior Mutability . . . 751
Cell . . . 752
RefCell . . . 753
Mutex and RwLock . . . 753
Atomics . . . 753
UnsafeCell . . . 754
Thread Safety: Send and Sync . . . 754
Locking: Mutexes and RwLocks . . . 756
Rust's Mutex . . . 756
Lock Poisoning . . . 758
Reader-Writer Lock . . . 760
Waiting: Parking and Condition Variables . . . 761
Thread Parking . . . 761
Condition Variables . . . 763
Summary . . . 765
Complete Runnable Examples . . . 765
Example: `ch1-01-hello.rs` (01 Hello) . . . 766
Example: `ch1-02-hello-join.rs` (02 Hello Join) . . . 766
Example: `ch1-03-spawn-closure.rs` (03 Spawn Closure) . . . 767
Example: `ch1-04-scoped-threads.rs` (04 Scoped Threads) . . . 767
Example: `ch1-05-rc.rs` (05 Rc) . . . 767
Example: `ch1-06-cell.rs` (06 Cell) . . . 768
Example: `ch1-07-refcell.rs` (07 Refcell) . . . 768
Example: `ch1-08-mutex.rs` (08 Mutex) . . . 768
Example: `ch1-09-sleep-before-unlock.rs` (09 Sleep Before Unlock) . . . 769
Example: `ch1-10-unlock-before-sleep.rs` (10 Unlock Before Sleep) . . . 769
Example: `ch1-11-thread-parking.rs` (11 Thread Parking) . . . 770
Example: `ch1-12-condvar.rs` (12 Condvar) . . . 770

Atomics **772**

Atomics **773**
Atomic Load and Store Operations . . . 774
Example: Stop Flag . . . 774
Example: Progress Reporting . . . 775
Example: Lazy Initialization . . . 777
Fetch-and-Modify Operations . . . 778
Example: Progress Reporting from Multiple Threads . . . 779
Example: Statistics . . . 780
Example: ID Allocation . . . 782
Compare-and-Exchange Operations . . . 783
Example: ID Allocation Without Overflow . . . 785
Example: Lazy One-Time Initialization . . . 786
Summary . . . 787
Complete Runnable Examples . . . 787
Example: `ch2-01-stop-flag.rs` (01 Stop Flag) . . . 787
Example: `ch2-02-progress-reporting.rs` (02 Progress Reporting) . . . 788
Example: `ch2-03-progress-reporting-unpark.rs` (03 Progress Reporting Unpark) 789
Example: `ch2-04-lazy-init.rs` (04 Lazy Init) . . . 790
Example: `ch2-05-fetch-add.rs` (05 Fetch Add) . . . 790
Example: `ch2-06-progress-reporting-multiple-threads.rs` (06 Progress Reporting Multiple Threads) . . . 791
Example: `ch2-07-statistics.rs` (07 Statistics) . . . 792
Example: `ch2-08-id-allocation.rs` (08 Id Allocation) . . . 793
Example: `ch2-09-id-allocation-panic.rs` (09 Id Allocation Panic) . . . 793
Example: `ch2-10-id-allocation-subtract-before-panic.rs` (10 Id Allocation Subtract Before Panic) . . . 794
Example: `ch2-11-increment-with-compare-exchange.rs` (11 Increment With Compare Exchange) . . . 795
Example: `ch2-12-id-allocation-without-overflow.rs` (12 Id Allocation Without Overflow) . . . 795
Example: `ch2-13-lazy-one-time-init.rs` (13 Lazy One Time Init) . . . 796

Memory Ordering **797**

Memory Ordering **798**
Reordering and Optimizations . . . 798
The Memory Model . . . 799
Happens-Before Relationship . . . 800
Spawning and Joining . . . 801
Relaxed Ordering . . . 802
Out-of-Thin-Air Values . . . 804
Release and Acquire Ordering . . . 805
More Formally . . . 808
Example: Locking . . . 809
Example: Lazy Initialization with Indirection . . . 810
Consume Ordering . . . 813
Sequentially Consistent Ordering . . . 814

Fences . . . 816
Compiler Fences . . . 818
Common Misconceptions . . . 818
Summary . . . 820
Complete Runnable Examples . . . 821
Example: `ch3-01-relaxed.rs` (01 Relaxed) . . . 821
Example: `ch3-02-spawn-join.rs` (02 Spawn Join) . . . 822
Example: `ch3-03-total-modification-order.rs` (03 Total Modification Order) . . . 822
Example: `ch3-04-total-modification-order-2.rs` (04 Total Modification Order 2) 823
Example: `ch3-05-out-of-thin-air.rs` (05 Out Of Thin Air) . . . 823
Example: `ch3-06-release-acquire.rs` (06 Release Acquire) . . . 824
Example: `ch3-07-release-acquire-unsafe.rs` (07 Release Acquire Unsafe) . . . 825
Example: `ch3-08-lock.rs` (08 Lock) . . . 825
Example: `ch3-09-lazy-init-box.rs` (09 Lazy Init Box) . . . 826
Example: `ch3-10-seqcst.rs` (10 Seqcst) . . . 827
Example: `ch3-11-fence.rs` (11 Fence) . . . 827

Building Our Own Spin Lock **829**

Building Our Own Spin Lock **830**
A Minimal Implementation . . . 830
An Unsafe Spin Lock . . . 832
A Safe Interface Using a Lock Guard . . . 834
Summary . . . 838
Complete Runnable Examples . . . 838
Example: `s1_minimal.rs` (S1 Minimal) . . . 838
Example: `s2_unsafe.rs` (S2 Unsafe) . . . 839
Example: `s3_guard.rs` (S3 Guard) . . . 840

Building Our Own Channels **842**

Building Our Own Channels **843**
A Simple Mutex-Based Channel . . . 843
An Unsafe One-Shot Channel . . . 844
Safety Through Runtime Checks . . . 847
Using a Single Atomic for the Channel State . . . 850
Safety Through Types . . . 852
Borrowing to Avoid Allocation . . . 856
Blocking . . . 859
Summary . . . 861
Complete Runnable Examples . . . 862
Example: `s1_simple.rs` (S1 Simple) . . . 862
Example: `s2_unsafe.rs` (S2 Unsafe) . . . 863
Example: `s3_checks.rs` (S3 Checks) . . . 863
Example: `s3_single_atomic.rs` (S3 Single Atomic) . . . 865
Example: `s4_types.rs` (S4 Types) . . . 867
Example: `s5_borrowing.rs` (S5 Borrowing) . . . 868
Example: `s6_blocking.rs` (S6 Blocking) . . . 870

Building Our Own Arc **873**

Building Our Own Arc **874**
Basic Reference Counting . . . 874
Testing It . . . 877
Mutation . . . 879
Weak Pointers . . . 880
Testing It . . . 885
Optimizing . . . 886
Summary . . . 892
Complete Runnable Examples . . . 893
Example: `s1_basic.rs` (S1 Basic) . . . 893
Example: `s2_weak.rs` (S2 Weak) . . . 895
Example: `s3_optimized.rs` (S3 Optimized) . . . 899

Understanding the Processor **905**

Understanding the Processor **906**
Processor Instructions . . . 906
Brief Introduction to Assembly . . . 907
Load and Store . . . 910
Read-Modify-Write Operations . . . 911
Load-Linked and Store-Conditional Instructions . . . 913
Caching . . . 916
Cache Coherence . . . 917
Impact on Performance . . . 918
Reordering . . . 923
Memory Ordering . . . 924
Other-Multi-Copy Atomicity . . . 925
x86-64: Strongly Ordered . . . 925
ARM64: Weakly Ordered . . . 926
An Experiment . . . 928
Memory Fences . . . 930
Summary . . . 931

Operating System Primitives **933**

Operating System Primitives **934**
Interfacing with the Kernel . . . 934
POSIX . . . 935
Wrapping in Rust . . . 936
Linux . . . 938
Futex . . . 938
Futex Operations . . . 940
Priority Inheritance Futex Operations . . . 944
macOS . . . 945
os_unfair_lock . . . 945
Windows . . . 945
Heavyweight Kernel Objects . . . 945

Lighter-Weight Objects . . . 946
Address-Based Waiting . . . 947
Summary . . . 948
Complete Runnable Examples . . . 948
Example: `ch8-01-futex.rs` (01 Futex) . . . 949

Building Our Own Locks **951**

Building Our Own Locks **952**
Mutex . . . 953
Lock API . . . 956
Avoiding Syscalls . . . 956
Optimizing Further . . . 958
Benchmarking . . . 960
Condition Variable . . . 962
Avoiding Syscalls . . . 967
Avoiding Spurious Wake-ups . . . 969
Reader-Writer Lock . . . 971
Avoiding Busy-Looping Writers . . . 974
Avoiding Writer Starvation . . . 977
Summary . . . 979
Complete Runnable Examples . . . 980
Example: `mutex_1.rs` (Mutex 1) . . . 980
Example: `mutex_2.rs` (Mutex 2) . . . 982
Example: `mutex_3.rs` (Mutex 3) . . . 983
Example: `condvar_1.rs` (Condvar 1) . . . 985
Example: `condvar_2.rs` (Condvar 2) . . . 987
Example: `rwlock_1.rs` (Rwlock 1) . . . 989
Example: `rwlock_2.rs` (Rwlock 2) . . . 991
Example: `rwlock_3.rs` (Rwlock 3) . . . 993

Ideas and Inspiration **997**

Ideas and Inspiration **998**
Semaphore . . . 998
RCU . . . 999
Lock-Free Linked List . . . 1001
Queue-Based Locks . . . 1002
Parking Lot–Based Locks . . . 1003
Sequence Lock . . . 1003
Teaching Materials . . . 1004

Rust

A Complete Guide to Safe, Concurrent, High-Performance Programming

A systematic, production-grade guide to writing Rust the way it lasts in real systems: **safe, concurrent, and high-performance**. Rust eliminates entire categories of critical bugs—null pointer dereferences, data races, buffer overflows, and use-after-free vulnerabilities—without the runtime overhead of a garbage collector.

This book provides a direct, comprehensive path from first principles to advanced production systems.

Introduction: Why Rust Matters

For decades, systems programming forced engineers into an uncomfortable compromise:

1. **Control and Speed without Safety:** Languages like C and C++ provide bare-metal hardware access, deterministic resource destruction, and peak performance, but leave memory safety, thread safety, and undefined behavior to developer discipline. Decades of CVE audits prove that even the most disciplined teams cannot prevent memory corruption at scale.
2. **Safety without Bare-Metal Control:** Managed languages like Java, Go, Python, and C# enforce memory safety through automatic garbage collection and runtime checks. However, garbage collection introduces non-deterministic pause times, memory footprint overhead, and indirection that make them less suitable for low-latency systems, kernel drivers, game engines, and embedded microcontrollers.

Rust breaks this compromise by achieving **compile-time memory safety without a garbage collector**.

The Problem Rust Solves

Software reliability and security crises predominantly stem from memory safety bugs and concurrency defects: - **Spatial memory safety violations:** Buffer overruns, out-of-bounds array reads and writes. - **Temporal memory safety violations:** Use-after-free, double free, dangling pointers. - **Concurrency bugs:** Data races occurring when two threads access the same memory location concurrently with at least one writer and no synchronization.

Rust guarantees that properly written safe code cannot experience any of these defects. If your code compiles, these failure modes are mathematically disproven by the compiler's borrow checker.

How Rust Achieves This

Rust replaces runtime garbage collection and manual memory management with three core mechanisms: 1. **Ownership:** Every value has a single owner bound to lexical scope. When the owner goes out of scope, the memory is deterministically freed (`Drop`). 2. **Borrowing & References:** References allow inspection or mutation of data without taking ownership. Rust enforces aliasing XOR mutability at compile time: you may have either any number of immutable references (`&T`) *or* exactly one mutable reference (`&mut T`), but never both simultaneously. 3. **Lifetimes:** The compiler tracks the validity of every reference, preventing dangling pointers across function boundaries and data structures at compile time without runtime overhead.

What You Will Learn

Across 25 comprehensive parts, seven applied engineering domains, and seven end-to-end production capstones, this book covers: - Core language syntax, primitive/compound types, and zero-cost control flow. - Deep mechanical mastery of ownership, stack vs. heap allocation, move semantics, and lifetimes. - Robust data modeling with algebraic data types (`struct`, `enum`) and expressive pattern matching. - Explicit, resilient error handling with `Result`, `Option`, and the `?` propagation operator. - Parametric polymorphism with generics, trait bounds, and dynamic dispatch via trait objects. - Fearless concurrency using threads, message-passing channels, and atomic primitives. - **Rust Atomics and Locks:** Deep mastery of hardware-level atomics, memory ordering (`Relaxed`, `Release`/`Acquire`, `SeqCst`), memory fences, spinlocks, channels, custom `Arc`, OS primitives (futex, `os_unfair_lock`, SRW locks), and custom lock implementations. - Asynchronous programming with `async`/`await`, `Future`, and the Tokio runtime. - Systems-level metaprogramming with declarative macros, procedural macros, and foreign function interfaces (FFI). - **Command-Line Tools in Rust:** 26 chapters spanning Foundations (`clap`), Core Skills, Networking & Async (`tokio`, `ratatui` TUI), Testing & Distribution (`assert_cmd`, cross-compilation), Advanced Architecture, and 4 dedicated Capstone tools (`fsearch`, `tudo`, `sysmon`, `devkit`). - Production engineering: profiling, benchmarking with Criterion, cargo packaging, WebAssembly (`wasm-pack`), and embedded `no_std` firmware.

How to Read This Book

1. **Linear Progression:** Chapters 1 through 13 form the core foundation. Read them sequentially; each chapter relies strictly on concepts verified in preceding sections.
2. **Complete, Runnable Code:** Every listing is self-contained. Copy the code into your editor, compile it with `cargo check` and `cargo run`, and observe compiler messages.
3. **Fight and Befriend the Borrow Checker:** When the compiler rejects your code, read the compiler diagnostic carefully. Rust diagnostics are world-class teaching tools explaining what invariant was violated and how to rectify it.
4. **Capstones & Projects:** Work through the integrated capstones—including the CLI Task Manager, Parallel Pipeline, Async HTTP API, and the Command-Line Capstones (`fsearch`, `tudo`, `sysmon`, and `devkit`)—to solidify real-world architecture.

The Current State of Rust

Rust is mature, battle-tested, and officially adopted across industry leaders for core infrastructure: - The Linux kernel has official support for Rust drivers and kernel modules. - Major cloud hypervisors and networking layers (AWS Firecracker, Cloudflare Workers, Android OS) are authored in Rust. - Standards organizations and government agencies (CISA, NSA) recommend transitioning memory-unsafe codebases to memory-safe languages like Rust.

What Rust Is Not

To set clear expectations, Rust is not: - **An object-oriented language with class hierarchies:** Rust uses composition, traits, and algebraic data types rather than inheritance. - **A rapid prototyping scripting language:** Rust demands explicit thinking up front about ownership, error conditions, and data lifetimes. - **Magic:** Zero-cost abstractions mean you don't pay runtime costs for high-level ergonomics; however, you must understand machine layouts and memory representation.

Formats

This book is available in multiple formats via the library portal: - **HTML** — responsive layout with interactive syntax highlighting and dual light/dark themes. - **PDF** — typeset document suitable for offline reading and archiving. - **EPUB** — standardized eBook format for mobile devices and e-readers.

Getting Started with Rust

The Story of Rust: History and Philosophy

The Story of Rust: History and Philosophy

Rust began in 2006 as a personal project by Graydon Hoare, a language engineer at Mozilla. At the time, Mozilla was grappling with the massive complexity and vulnerability surface of the Gecko rendering engine in Firefox, written in C++. In multi-threaded C++, memory corruption vulnerabilities (such as use-after-free and buffer overflows) and concurrency bugs (data races) were extraordinarily difficult to prevent, even for seasoned systems developers.

Mozilla sponsored Rust in 2009 with a distinct mission: **create a modern systems language that guarantees memory safety and thread safety without sacrificing bare-metal speed or requiring a garbage collector**.

Core Philosophy

The design of Rust rests on three non-negotiable pillars: 1. **Safety:** Eliminate undefined behavior, dangling pointers, null dereferences, and data races at compile time. 2. **Speed:** Provide zero-cost abstractions—constructs that compile down to assembly indistinguishable from hand-written C. 3. **Concurrency:** "Fearless concurrency"—the compiler prevents data races across threads using the exact same ownership model used for memory safety.

```
// A safe, deterministic greeting
fn main() {
    let project = "Rust";
    let year = 2026;
    println!("Welcome to {}, built for safety and speed in {}.", project, year);
}
```

Installing Rust with rustup

Installing Rust with rustup

The primary and official method for installing and managing Rust toolchains across all platforms is `rustup`.

Installing rustup

On Linux and macOS, install `rustup` via your terminal:

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

Follow the default prompts. This installs: - `rustc`: The Rust compiler. - `cargo`: The package manager and build system. - `rustup`: The toolchain multiplexer. - `rust-std`: Standard library binaries for your host architecture.

Managing Toolchains

Configure your shell environment:

```
source "$HOME/.cargo/env"
rustc --version
cargo --version
```

To update your toolchain to the latest stable release:

```
rustup update stable
```

Your First Program: Hello World

Your First Program: Hello World

Let's begin with the canonical starting point: a minimal standalone Rust source file.

Direct Compilation with rustc

Create a file named `main.rs`:

```
// main.rs
fn main() {
    println!("Hello, world!");
}
```

Compile and run directly using `rustc`:

```
rustc main.rs
./main
```

Output:

```
Hello, world!
```

Deconstructing the Program

- `fn main() { ... }`: The `main` function is the entry point of every executable Rust program. It takes no arguments and returns unit `()`.
- `println!(...)`: In Rust, names ending in an exclamation mark `!` invoke **macros** rather than standard functions. `println!` parses format strings at compile time to ensure type safety.

Understanding rustc and cargo

Understanding rustc and cargo

While `rustc` is the underlying compiler, production Rust projects are almost exclusively managed with **Cargo**.

The Role of Cargo

Cargo handles four essential engineering responsibilities: 1. **Dependency Resolution:** Downloading, pinning, and compiling external crates from crates.io. 2. **Build Orchestration:** Invoking `rustc` with optimal optimization flags, link arguments, and target settings. 3. **Test & Benchmark Execution:** Discovering and executing unit tests, integration tests, and performance benchmarks. 4. **Documentation Generation:** Generating standard HTML documentation directly from doc-comments.

Common Cargo Commands

```
cargo check   # Fast syntax and type verification without binary emission
cargo build   # Compile debug build with symbols into target/debug
cargo run     # Build and execute the binary
cargo test    # Run automated test harness
cargo doc     # Build HTML documentation for current project and dependencies
```

Exploring a Cargo Project Structure

Exploring a Cargo Project Structure

Initialize a new project with Cargo:

```
cargo new desk_service --bin
cd desk_service
```

Standard Directory Layout

```
desk_service/
   Cargo.lock       # Exact resolved dependency graph
   Cargo.toml       # Manifest file: project metadata & dependencies
   src/
       main.rs      # Binary root entry point
   target/          # Build artifacts and compilation cache (gitignored)
```

The Cargo.toml Manifest

```
[package]
name = "desk_service"
version = "0.1.0"
edition = "2024"

[dependencies]
```

The `edition = "2024"` flag configures the language edition. Rust guarantees backwards compatibility across editions through epoch-based stability.

Summary

Summary

Chapter Key Takeaways

- Rust provides zero-cost memory safety without a garbage collector through static ownership analysis.
- `rustup` is the toolchain manager; keep it updated with `rustup update`.
- `rustc` compiles code, but day-to-day workflow centers on `cargo check`, `cargo build`, `cargo test`, and `cargo run`.
- Every Rust binary begins at `fn main()` and projects are configured through `Cargo.toml`.

Language Basics: Variables, Types, and Expressions

Variables, Mutability, and Shadowing

Variables, Mutability, and Shadowing

By default, variables in Rust are **immutable**. Once bound to a value, the binding cannot be modified.

Mutability with `mut`

To allow mutation, explicitly declare the binding with `mut`:

```
fn main() {
    let mut counter = 10;
    println!("Initial counter: {}", counter);
    counter += 5;
    println!("Updated counter: {}", counter);
}
```

Variable Shadowing

Rust allows redeclaring a variable with the same name in the same scope using `let`. This is **shadowing**:

```
fn main() {
    let spaces = "   ";
    let spaces = spaces.len(); // Shadows spaces: changes type from &str to usize
    println!("Number of spaces: {}", spaces);
}
```

Shadowing creates a new binding, allowing type transformation without requiring distinct variable names.

Primitive Data Types in Detail

Primitive Data Types in Detail

Rust is statically and strongly typed with full type inference.

Scalar Types

1. **Integers:**
   - Signed: `i8`, `i16`, `i32`, `i64`, `i128`, `isize` (pointer width)
   - Unsigned: `u8`, `u16`, `u32`, `u64`, `u128`, `usize` (pointer width, indexing)
2. **Floating-point:** `f32`, `f64` (IEEE 754-2008)
3. **Boolean:** `bool` (`true` or `false`, 1 byte representation)
4. **Character:** `char` (4-byte Unicode scalar value)

```
fn main() {
    let count: usize = 42;
    let temperature: f64 = 98.6;
    let is_active: bool = true;
    let emoji: char = ' ';

    println!("Scalar values: {}, {}, {}, {}", count, temperature, is_active, emoji);
}
```

Compound Types: Tuples and Arrays

Compound Types: Tuples and Arrays

Compound types group multiple values into a single type.

Tuples

A tuple has a fixed length and can contain elements of different types:

```
fn main() {
    let point: (i32, f64, &str) = (10, 3.14, "origin");
    let (x, y, label) = point; // Destructuring
    println!("Point {}: x = {}, y = {}", label, x, y);
    println!("Direct access: first = {}", point.0);
}
```

Arrays

An array has a fixed length, is stack-allocated, and elements must be of the **same type**:

```
fn main() {
    let buffer: [u8; 4] = [0xDE, 0xAD, 0xBE, 0xEF];
    println!("Array length: {}, first: 0x{:X}", buffer.len(), buffer[0]);

    let initialized = [0u32; 16]; // 16 elements initialized to 0
    println!("Initialized array length: {}", initialized.len());
}
```

Functions, Statements, and Expressions

Functions, Statements, and Expressions

Rust is an expression-oriented language. Almost every construct evaluates to a value.

Statements vs. Expressions

- **Statements:** Instructions that perform an action and do not return a value. Ending an expression with a semicolon `;` turns it into a statement returning unit `()`.
- **Expressions:** Evaluate to a resulting value. Omission of the final semicolon returns the expression value.

```
fn add(a: i32, b: i32) -> i32 {
    a + b // No semicolon: implicit return
}

fn compute_block(factor: i32) -> i32 {
    let result = {
        let base = 100;
        base * factor // Expression evaluated and bound to result
    };
    result
}

fn main() {
    println!("Add: {}", add(10, 20));
    println!("Compute block: {}", compute_block(3));
}
```

Control Flow: if, loop, while, and for

Control Flow: if, loop, while, and for

Control flow constructs in Rust are expressions that can return values.

`if` as an Expression

```
fn main() {
    let condition = true;
    let number = if condition { 5 } else { 6 };
    println!("Number: {}", number);
}
```

Loops: `loop`, `while`, and `for`

```
fn main() {
    // 1. loop with value return
    let mut counter = 0;
    let result = loop {
        counter += 1;
        if counter == 10 {
            break counter * 2;
        }
    };
    println!("Loop result: {}", result);

    // 2. while loop
    let mut n = 3;
    while n > 0 {
        println!("{}!", n);
        n -= 1;
    }

    // 3. for loop over iterators
    for item in 0..3 {
        println!("Index: {}", item);
    }
}
```

NonZero Integer Types

NonZero Integer Types

Thesis

`NonZeroU32` (and friends) are integers that **cannot be zero**. Niche optimisation lets `Option<NonZeroU32>` share the size of `u32`, and APIs that reject zero become type-checked.

Mental model

```
NonZeroU32::new(0)  → None
NonZeroU32::new(7)  → Some(_)
Option<NonZeroU32> size == size_of::<u32>()   # niche
```

Worked example

```
// nonzero_demo.rs
use std::mem::size_of;
use std::num::NonZeroU32;

fn main() {
    println!("size u32={} Option<u32>={}", size_of::<u32>(), size_of::<Option<u32>>());
    println!(
        "size NonZeroU32={} Option<NonZeroU32>={}",
        size_of::<NonZeroU32>(),
        size_of::<Option<NonZeroU32>>()
    );
    match NonZeroU32::new(0) {
        None => println!("zero rejected"),
        Some(_) => unreachable!(),
    }
    let n = NonZeroU32::new(7).unwrap();
    println!("get={}", n.get());
}
```

```
get=7
zero rejected
div=42
```

Trap

Using `unwrap` on `NonZero*::new` with untrusted input. Prefer `new` + explicit error paths, or `NonZeroU32::try_from`. Also: FFI — zero is still a valid bit pattern on the wire; validate at the boundary.

Boring rule

Reach for `NonZero*` when zero is meaningless (divisors, lengths you already checked, database IDs you treat as non-zero). Document the invariant.

Try this

1. Compare `size_of::<Option<u32>>()` vs `Option<NonZeroU32>()`.
2. Use `NonZeroUsize` for a collection length after checking non-empty.
3. Read niche-optimisation notes in the nomicon / reference.

Sources

- `std::num::NonZeroU32`
- Rustonomicon: niche optimisations

Summary

Summary

Chapter Key Takeaways

- Variables are immutable by default; use `mut` to allow change.
- Shadowing (`let x = ...`) rebinds names and enables type changes.
- Rust primitives include scalar types (integers, floats, booleans, chars) and compound types (tuples, fixed-size arrays).
- Rust is expression-oriented: blocks and control flow evaluate directly to values.

Ownership: The Heart of Rust

What is Ownership and Why Does It Exist?

What is Ownership and Why Does It Exist?

Ownership is Rust's central memory management innovation. It governs how heap memory is allocated, accessed, and reclaimed without a garbage collector or manual `free()` calls.

The Three Rules of Ownership

1. **Each value in Rust has an owner.**
2. **There can only be one owner at a time.**
3. **When the owner goes out of scope, the value is dropped.**

```
fn main() {
    {
        let s = String::from("desk"); // s is valid from this point forward
        println!("Inside scope: {}", s);
    } // s goes out of scope; String memory is automatically freed here
}
```

Stack vs Heap Memory in Rust

Stack vs Heap Memory in Rust

Understanding where data lives is essential for writing efficient Rust code.

The Stack

- Fast, LIFO (Last In, First Out) allocation.
- All data stored on the stack must have a known, fixed size at compile time.
- Primitive types, fixed arrays, and function frames live on the stack.

The Heap

- Dynamic memory allocated at runtime.
- The operating system locates a contiguous region of memory, marks it in use, and returns a pointer.
- Slower than stack allocation; requires dereferencing pointers.
- Dynamically sized types like `String` and `Vec<T>` store metadata on the stack (pointer, length, capacity) pointing to heap buffers.

Move Semantics Explained

Move Semantics Explained

When a heap-backed value is assigned to another binding, passed to a function, or returned from a function, ownership is **moved**.

```
fn main() {
    let s1 = String::from("orders");
    let s2 = s1; // s1 is MOVED to s2

    // println!("{}", s1); // COMPILE ERROR: value borrowed here after move
    println!("s2 owns the heap buffer: {}", s2);
}
```

Why Rust Moves Instead of Shallow Copying

If `s1` and `s2` both owned the same heap pointer, when both went out of scope, they would attempt to free the same memory twice—a catastrophic **double free** bug. Rust prevents this by invalidating `s1` at compile time with zero runtime cost.

Copy Types and Clone

Copy Types and Clone

The `Copy` Trait

Types that live entirely on the stack and have fixed sizes implement the `Copy` trait. Assignment of `Copy` types performs a bitwise copy rather than a move:

```
fn main() {
    let x: i32 = 42;
    let y = x; // Bitwise copy
    println!("x = {}, y = {}", x, y); // Both valid!
}
```

The `Clone` Trait

For heap-allocated types where an explicit, potentially expensive deep copy is desired, use `.clone()`:

```
fn main() {
    let s1 = String::from("records");
    let s2 = s1.clone(); // Deep allocation on heap
    println!("s1 = {}, s2 = {}", s1, s2); // Both valid
}
```

The Drop Trait and Resource Cleanup

The Drop Trait and Resource Cleanup

Rust implements RAII (Resource Acquisition Is Initialization) via the `Drop` trait.

Custom Drop Implementations

```
struct ResourceHandle {
    name: String,
}

impl Drop for ResourceHandle {
    fn drop(&mut self) {
        println!("Cleaning up resource: {}", self.name);
    }
}

fn main() {
    let _r = ResourceHandle { name: String::from("NetworkSocket") };
    println!("Resource active");
    // _r drops automatically when main exits
}
```

Resources (file descriptors, sockets, database handles) cannot leak due to early returns or panics because `drop` runs deterministically on scope exit.

The Cost of Drop on Deep Ownership Chains

The Cost of Drop on Deep Ownership Chains

Thesis

`Drop` is not free. A long chain of `Box` nodes runs destructors one by one; `mem::forget` skips them and returns in nanoseconds — which is also how you leak. This chapter times dropping chains of 1 k / 10 k / 100 k nodes versus forgetting the large chain.

Mental model

```
drop(head) → Drop::drop → drop(child) → …  (recursive / iterative cleanup)
mem::forget(head) → no destructors; memory leaked unless into a global allocator you never
```

Worked examples

Verified with **rustc 1.99.1**, `cargo run --release`.

```
// chain of Box<Node>; time drop vs forget
```

Output:

```
drop chain of    1000 nodes: 21.888µs
drop chain of   10000 nodes: 267.357µs
drop chain of  100000 nodes: 4.211461ms
forget chain of  100000 nodes: 487ns
```

Drop scales with chain length (~4 ms for 100 k nodes here). `forget` on the same shape was ~0.5 µs — and leaked the graph.

Trap

Using `forget` to "speed up shutdown". You trade CPU for leaks and skipped safety hooks (`File` flush, lock poisoning cleanup, etc.).

Boring rule

Let `Drop` run. If shutdown profiles show destructor cost, batch into arenas or reuse pools — do not `forget` application state.

Try this

1. Replace the linked list with a `Vec<Node>` and compare drop time.
2. Implement `Drop` for a type that prints, and watch ordering.
3. Read `std::mem::ManuallyDrop`.

Sources

- Drop in the Rust book / std::ops::Drop
- `mem::forget`

Summary

Summary

Chapter Key Takeaways

- Ownership rules ensure single ownership per resource and deterministic cleanup.
- Stack memory holds fixed-size data; heap memory holds dynamic data referenced via stack pointers.
- Move semantics transfer ownership, invalidating previous bindings to prevent double-free errors.
- `Copy` types duplicate automatically via bitwise copy; heap types require explicit `.clone()`.
- The `Drop` trait executes custom cleanup when a value leaves scope.

Borrowing, References, and Slices

Shared and Mutable References

Shared and Mutable References

Instead of transferring ownership, a function can access data via **references**. This mechanism is called **borrowing**.

Shared References (`&T`)

Shared references allow read-only access to data without taking ownership:

```
fn calculate_length(s: &String) -> usize {
    s.len()
}

fn main() {
    let s = String::from("desk");
    let len = calculate_length(&s); // Pass reference
    println!("The length of '{}' is {}.", s, len); // s is still valid!
}
```

Mutable References (`&mut T`)

Mutable references permit modifying the borrowed value:

```
fn append_status(s: &mut String) {
    s.push_str(" [ACTIVE]");
}

fn main() {
    let mut task = String::from("Process orders");
    append_status(&mut task);
    println!("{}", task);
}
```

The Borrowing Rules Demystified

The Borrowing Rules Demystified

Rust's compiler enforces two fundamental rules regarding references at compile time:

1. **At any given time, you can have either one mutable reference or any number of immutable references.**
2. **References must always be valid.**

Why the Rules Exist

This rule (often summarized as **Aliasing XOR Mutability**) prevents **data races**. A data race occurs when: - Two or more pointers access the same memory concurrently. - At least one pointer is writing. - There is no synchronization.

By disallowing simultaneous reading and writing through multiple aliases, Rust eliminates data races before the code ever compiles.

Dangling References Prevented at Compile Time

Dangling References Prevented at Compile Time

In languages with manual memory management, creating a pointer to memory that has been deallocated results in a **dangling reference**. Rust guarantees this never occurs:

```
// Attempting to return a reference to a local variable:
// fn dangle() -> &String {
//     let s = String::from("temporary");
//     &s // ERROR: `s` does not live long enough
// }

// The idiomatic solution: transfer ownership of the value itself
fn no_dangle() -> String {
    let s = String::from("valid data");
    s
}

fn main() {
    let result = no_dangle();
    println!("Received: {}", result);
}
```

String Slices and &str

String Slices and &str

A string slice is a reference to a contiguous sequence of UTF-8 encoded bytes in a `String`.

```
fn main() {
    let s = String::from("Hello Rust");

    let first_word = &s[0..5];
    let second_word = &s[6..10];

    println!("First: {}, Second: {}", first_word, second_word);
}
```

`String` vs `&str`

- `String`: Owned, heap-allocated, resizable buffer.
- `&str`: An immutable view into string data located on the heap, stack, or embedded in the read-only data segment of the binary (string literals).

Slice Types and Range Syntax

Slice Types and Range Syntax

Slices generalize beyond strings to any contiguous sequence, such as arrays and vectors.

```
fn main() {
    let numbers = [10, 20, 30, 40, 50];

    let slice: &[i32] = &numbers[1..4]; // Elements 20, 30, 40
    println!("Slice: {:?}", slice);

    // Range variations:
    let start_to_two = &numbers[..2];
    let three_to_end = &numbers[3..];
    let all = &numbers[..];

    println!("Start: {:?}, End: {:?}, All: {:?}", start_to_two, three_to_end, all);
}
```

Summary

Summary

Chapter Key Takeaways

- References borrow data without transferring ownership.
- Aliasing XOR Mutability: either multiple `&T` or exactly one `&mut T`.
- Dangling references are prohibited at compile time.
- Slices (`&[T]` and `&str`) provide fat-pointer views (pointer + length) into contiguous memory without allocation.

Structs, Enums, and Pattern Matching

Defining and Using Structs

Defining and Using Structs

Structs allow creating custom types grouping related fields.

```
#[derive(Debug)]
struct Ticket {
    id: u64,
    title: String,
    resolved: bool,
}

impl Ticket {
    fn new(id: u64, title: String) -> Self {
        Self { id, title, resolved: false }
    }

    fn resolve(&mut self) {
        self.resolved = true;
    }
}

fn main() {
    let mut ticket = Ticket::new(101, String::from("Fix database migration"));
    println!("Ticket: {:?}", ticket);
    ticket.resolve();
    println!("Resolved: {:?}", ticket);
}
```

Enumerations Beyond Simple Tags

Enumerations Beyond Simple Tags

Rust enums are algebraic data types (tagged unions). Each variant can hold different types and amounts of data:

```
#[derive(Debug)]
enum WebEvent {
    PageLoad,
    KeyPress(char),
    Click { x: i64, y: i64 },
    Submit(String),
}

fn main() {
    let click = WebEvent::Click { x: 120, y: 340 };
    let key = WebEvent::KeyPress('q');
    println!("Events: {:?}, {:?}", click, key);
}
```

The match Expression in Depth

The match Expression in Depth

`match` enables exhaustive pattern matching. Every possible case must be handled.

```
#[derive(Debug)]
enum Priority {
    Low,
    Medium,
    High,
    Critical(u8),
}

fn inspect_priority(p: Priority) {
    match p {
        Priority::Low => println!("Low priority - queue normally"),
        Priority::Medium => println!("Medium priority - assign SLA"),
        Priority::High => println!("High priority - alert on-call"),
        Priority::Critical(level) => println!("CRITICAL (level {}) - page team immediately",
    }
}

fn main() {
    inspect_priority(Priority::High);
    inspect_priority(Priority::Critical(1));
}
```

if let and while let for Partial Matching

if let and while let for Partial Matching

When you only care about matching one pattern and ignoring all others, `if let` and `while let` avoid boilerplate:

```
fn main() {
    let config_val: Option<u32> = Some(8080);

    // if let syntax
    if let Some(port) = config_val {
        println!("Binding server to port {}", port);
    }

    // while let syntax
    let mut stack = vec![1, 2, 3];
    while let Some(top) = stack.pop() {
        println!("Popped: {}", top);
    }
}
```

if-let Guards and cfg_select!

if-let Guards and `cfg_select!`

Thesis

Rust 1.95 stabilised two small features that remove a lot of everyday boilerplate:

- **if-let guards** let a `match` arm *bind and test* in its guard: `Pat if let Some(x) = f(y) && x > 0 => …`. If the guard fails, matching falls through to the next arm, just like a plain `if` guard.
- **`cfg_select!`** is a compile-time `match` over `cfg` predicates. The first arm whose predicate is true is expanded, and `_` is the fallback. It does the job the `cfg-if` crate has done for years, with no dependency.

Both have one sharp edge. The compiler doesn't count guards towards exhaustiveness, and a `cfg_select!` where no arm is true is a hard error.

Mental model

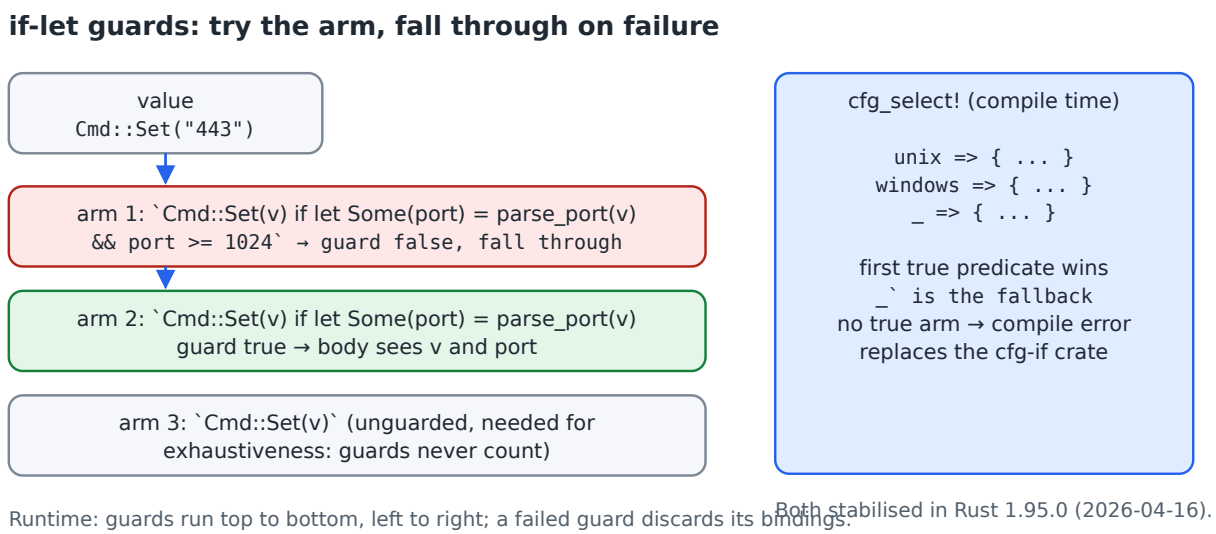


Figure 1: A match with if-let guards tries arm 1 (pattern matches, guard false, fall through), then arm 2 (guard true, body runs with both bindings); an unguarded arm is still needed for exhaustiveness. cfg_select! expands the first true cfg arm at compile time

An if-let guard runs after the arm's pattern has matched. It can use the pattern's bindings, introduce new ones, and chain more conditions with `&&` (the same let-chain grammar that was stabilised for `if` in Rust 1.88). The new bindings are in scope only in that arm's body. If any part of the chain fails, those bindings are dropped and the next arm is tried.

Worked example

Verified with **rustc 1.99.1**, edition 2024, on x86-64 Linux.

```
// cfg_select! picks the first arm whose predicate holds; `_` is the fallback.
cfg_select! {
    unix => {
        fn platform() -> &'static str { "unix" }
    }
    windows => {
        fn platform() -> &'static str { "windows" }
    }
    _ => {
        fn platform() -> &'static str { "other" }
    }
}

// It also works in expression position.
fn word_size() -> u32 {
    cfg_select! {
        target_pointer_width = "64" => 64,
        target_pointer_width = "32" => 32,
        _ => 0,
    }
}

#[derive(Debug)]
enum Cmd {
    Set(String),
    Del(String),
}

fn parse_port(s: &str) -> Option<u16> {
    s.parse().ok()
}

// if-let guards: bind inside the guard, and fall through to later arms if it fails.
fn describe(cmd: &Cmd) -> String {
    match cmd {
        Cmd::Set(v) if let Some(port) = parse_port(v) && port >= 1024 => {
            format!("set unprivileged port {port}")
```

```
        }
        Cmd::Set(v) if let Some(port) = parse_port(v) => format!("set privileged port {port}
        Cmd::Set(v) => format!("set non-numeric value {v:?}"),
        Cmd::Del(k) => format!("delete {k}"),
    }
}

// The same logic before Rust 1.95: the binding has to move into the arm body,
// and the fall-through has to be duplicated by hand.
fn describe_old(cmd: &Cmd) -> String {
    match cmd {
        Cmd::Set(v) => match parse_port(v) {
            Some(port) if port >= 1024 => format!("set unprivileged port {port}"),
            Some(port) => format!("set privileged port {port}"),
            None => format!("set non-numeric value {v:?}"),
        },
        Cmd::Del(k) => format!("delete {k}"),
    }
}

fn main() {
    println!("platform={} word_size={}", platform(), word_size());
    for c in [
        Cmd::Set("8080".into()),
        Cmd::Set("443".into()),
        Cmd::Set("abc".into()),
        Cmd::Del("x".into()),
    ] {
        assert_eq!(describe(&c), describe_old(&c));
        println!("{c:?} -> {}", describe(&c));
    }
}
```

```
cargo run -q
```

```
platform=unix word_size=64
Set("8080") -> set unprivileged port 8080
Set("443") -> set privileged port 443
Set("abc") -> set non-numeric value "abc"
Del("x") -> delete x
```

The `assert_eq!` in `main` checks that the guarded version and the pre-1.95 nested version agree on every input. The difference is shape. In `describe_old`, everything about `Set` has to live inside one arm, so other variants can't be interleaved by priority. In `describe`, each arm states exactly what it needs.

`cfg_select!` appears twice: once at item level, choosing which `fn platform` to define, and once in

expression position, as the tail of `word_size`. Expression-position arms are separated by commas and take a plain expression. Item-level arms take a braced block of items.

The traps

Both mistakes below are in one file so you can see the real diagnostics:

```
cfg_select! {
    windows => {
        fn platform() -> &'static str { "windows" }
    }
}

enum Cmd {
    Set(String),
    Del(String),
}

fn describe(cmd: &Cmd) -> String {
    match cmd {
        Cmd::Set(v) if let Ok(port) = v.parse::<u16>() => format!("port {port}"),
        Cmd::Del(k) => format!("delete {k}"),
    }
}

fn main() {
    println!("{} {}", platform(), describe(&Cmd::Del("x".into())));
}
```

```
error: none of the predicates in this `cfg_select` evaluated to true
 --> src/main.rs:1:1
  |
1 | / cfg_select! {
2 | |     windows => {
3 | |         fn platform() -> &'static str { "windows" }
4 | |     }
5 | | }
  | |_^

error[E0425]: cannot find function `platform` in this scope
  --> src/main.rs:20:23
   |
20 |     println!("{} {}", platform(), describe(&Cmd::Del("x".into())));
   |                        ^^^^^^^^ not found in this scope
```

```
error[E0004]: non-exhaustive patterns: `&Cmd::Set(_)` not covered
  --> src/main.rs:13:11
   |
13 |     match cmd {
   |           ^^^ pattern `&Cmd::Set(_)` not covered
   |
note: `Cmd` defined here
  --> src/main.rs:7:6
   |
 7 | enum Cmd {
   |      ^^^
 8 |     Set(String),
   |     --- not covered
   = note: the matched value is of type `&Cmd`
   = note: match arms with guards don't count towards exhaustivity
help: ensure that all possible cases are being handled by adding a match arm with a wildcard
   |
15 ~         Cmd::Del(k) => format!("delete {k}"),
16 ~         &Cmd::Set(_) => todo!(),
   |
```

1. **No true arm.** This is deliberate. `cfg_select!` won't silently expand to nothing, so a missing platform is a compile error on that platform rather than a confusing "cannot find function" later. (You get that second error too, as a knock-on.) Add a `_ =>` arm whenever a fallback makes sense, and use `_ => { compile_error!("…") }` when it doesn't, so the message is yours.
2. **Guards don't count.** Even though `v.parse::<u16>()` is either `Ok` or `Err`, the compiler doesn't reason about guards. The Rust 1.95 release notes state this explicitly. Always finish a run of guarded arms with an unguarded arm for the same pattern (or `_`). Don't take the help text's `todo!()` literally.

Migrating from `cfg-if`

The `cfg-if` crate's `if #[cfg(unix)] { … } else if #[cfg(windows)] { … } else { … }` maps one to one onto `cfg_select! { unix => { … } windows => { … } _ => { … } }`. Before you drop the dependency, check two things:

- **MSRV.** `cfg_select!` needs Rust 1.95. If you publish a library with a lower `rust-version`, keep `cfg-if` until you raise it.
- **Expression position.** `cfg-if` only expands to items. `cfg_select!` also works as an expression, so `let x = cfg_select! { … };` replaces the common `#[cfg]` + `#[cfg(not)]` pair of `let` statements.

Try this

1. Add `target_os = "linux" => "linux",` as the first arm of a new expression-position `cfg_select!`, and confirm that the more specific arm wins over `unix` when it comes first.
2. Rewrite one of your `match` + nested `if let` blocks with an if-let guard, and keep an `assert_eq!` against the old version, as above, until the tests pass.
3. Delete arm 3 (`Cmd::Set(v) =>`) from `describe` and read the E0004 note.

Sources

- Announcing Rust 1.95.0: https://blog.rust-lang.org/2026/04/16/Rust-1.95.0/
- `cfg_select!` in the standard library docs: https://doc.rust-lang.org/std/macro.cfg_select.html
- The Rust Reference, match guards: https://doc.rust-lang.org/reference/expressions/match-expr.html#match-guards
- The Rust Reference, conditional compilation (`cfg` predicates): https://doc.rust-lang.org/reference/conditional-compilation.html

Real-World Data Modeling with Enums

Real-World Data Modeling with Enums

Enums eliminate invalid program states at compile time:

```
#[derive(Debug)]
enum PaymentMethod {
    CreditCard { number: String, exp: String },
    WireTransfer { iban: String },
    InvoiceNet30 { po_number: u64 },
}

#[derive(Debug)]
struct Order {
    order_id: u64,
    amount_cents: u64,
    payment: PaymentMethod,
}

fn main() {
    let order = Order {
        order_id: 5001,
        amount_cents: 29900,
        payment: PaymentMethod::WireTransfer { iban: String::from("DE89370400440532013000")
    };
    println!("Recorded order: {:?}", order);
}
```

Summary

Summary

Chapter Key Takeaways

- Structs group named or positional fields and support methods via `impl`.
- Rust enums can carry rich payload data per variant.
- `match` enforces compile-time exhaustiveness, preventing unhandled cases.
- `if let` and `while let` provide concise syntax for single-variant checks.

Modules, Crates, and Visibility

Modules and the Filesystem

Modules and the Filesystem

Rust's module system organizes code into distinct namespaces and controls privacy.

The Modern Module Mapping

In modern Rust (2018/2021/2024 editions), declaring `mod network;` in `src/main.rs` resolves to `src/network.rs` or `src/network/mod.rs`.

```
// In src/network.rs
pub fn connect() {
    println!("Connecting to network host...");
}
```

The use Statement and Path Resolution

The use Statement and Path Resolution

The `use` keyword brings paths into scope to avoid verbose qualification:

```
mod storage {
    pub mod cache {
        pub fn clear() {
            println!("Cache cleared.");
        }
    }
}

// Bring clear into scope
use storage::cache::clear;

fn main() {
    clear();
}
```

Public vs Private Visibility Rules

Public vs Private Visibility Rules

In Rust, **everything is private by default**. To expose items, use `pub`: - `pub`: Public to everyone. - `pub(crate)`: Visible within the current crate only. - `pub(super)`: Visible to the parent module.

```
mod orders {
    pub struct Order {
        pub id: u64,
        secret_notes: String, // Private field!
    }

    impl Order {
        pub fn new(id: u64, notes: String) -> Self {
            Self { id, secret_notes: notes }
        }
    }
}

fn main() {
    let o = orders::Order::new(1, String::from("VIP customer"));
    println!("Order ID: {}", o.id);
    // println!("{}", o.secret_notes); // ERROR: private field
}
```

Crates, Packages, and Cargo Workspaces

Crates, Packages, and Cargo Workspaces

- **Crate:** The smallest compilation unit in Rust (a library or binary).
- **Package:** One or more crates defined by a `Cargo.toml`.
- **Workspace:** A set of packages that share a single `Cargo.lock` and output directory (`target/`).

```
# Top-level Cargo.toml for workspace
[workspace]
members = [
    "crates/server",
    "crates/core",
    "crates/client"
]
```

Re-exporting and Organizing Large Projects

Re-exporting and Organizing Large Projects

Re-exporting using `pub use` decouples internal module hierarchies from your public API:

```
mod internal {
    pub struct Engine;
    impl Engine {
        pub fn start(&self) { println!("Engine running"); }
    }
}

// Public API
pub use internal::Engine;

fn main() {
    let e = Engine;
    e.start();
}
```

Summary

Summary

Chapter Key Takeaways

- Files and directories map directly to modules.
- Privacy is strict: everything is private unless marked `pub`.
- `use` brings items into scope; `pub use` re-exports for clean API surfaces.
- Cargo workspaces share dependencies and build artifacts across multi-crate repositories.

Collections and Strings

Vec and Dynamic Arrays

Vec and Dynamic Arrays

`Vec<T>` is a growable, heap-allocated array.

```
fn main() {
    let mut v = vec![10, 20, 30];
    v.push(40);

    for (idx, elem) in v.iter().enumerate() {
        println!("v[{}] = {}", idx, elem);
    }

    // Safe retrieval via get()
    match v.get(2) {
        Some(&val) => println!("Element at index 2 is {}", val),
        None => println!("Index out of bounds"),
    }
}
```

String vs &str Deep Dive

String vs &str Deep Dive

Rust strings are always valid UTF-8 sequences.

```
fn main() {
    let mut s = String::from("desk");
    s.push_str(" management");

    let slice: &str = &s[..4];
    println!("Owned: '{}', Slice: '{}'", s, slice);

    // Iterating bytes vs characters
    for ch in "  Rust".chars() {
        println!("Char: {}", ch);
    }
}
```

HashMap and BTreeMap for Key-Value Storage

HashMap and BTreeMap for Key-Value Storage

```rust
use std::collections::{HashMap, BTreeMap};

fn main() {
    // HashMap (O(1) average lookup)
    let mut map = HashMap::new();
    map.insert("order_1", 450);
    map.insert("order_2", 890);

    println!("Order 1 total: {:?}", map.get("order_1"));

    // BTreeMap (sorted keys, O(log N))
    let mut btree = BTreeMap::new();
    btree.insert(200, "Two Hundred");
    btree.insert(100, "One Hundred");

    for (k, v) in &btree {
        println!("Key {} -> {}", k, v);
    }
}
```

HashMap Hashers and Hash Flooding: Why the Default Is SipHash

HashMap Hashers and Hash Flooding: Why the Default Is SipHash

After reading this chapter, you will understand why Rust's standard `HashMap` uses a keyed, randomly seeded hash (currently SipHash-1-3) by default, how hash-flooding (HashDoS) attacks exploit deterministic hash collisions to degrade table operations to $O(n^2)$, how Rust's SwissTable architecture behaves under attack, and when it is safe to swap in non-cryptographic hashers like `foldhash` or `FxHash`.

Thesis

A hash table promises $O(1)$ average-time lookups and insertions by mapping keys to buckets using a hash function. If an attacker can force many keys into the same bucket or probing sequence, insertion cost degrades to $O(n)$ per operation—turning an $O(n)$ workload into an $O(n^2)$ CPU-exhaustion denial of service.

To prevent this vulnerability—known as a HashDoS attack—Rust's standard library parameterizes `HashMap<K, V, S = RandomState>` with a keyed, randomly seeded hasher (`SipHash-1-3`) by default. The keys come from the operating system's secure random source at runtime, so someone outside the process can't work out which keys will collide.

That protection isn't free: SipHash is slower than simple non-keyed hashes. When you generate the keys yourself (compiler symbols, internal integer IDs) and profiling shows hashing matters, a faster hasher can be worth it. The hasher is part of the map's type, so this is a deliberate choice you make per map, after asking where its keys come from.

How a Hash Table Degrades

Standard hash tables map high-cardinality keys to finite table slots using two steps: 1. Hash the key into a 64-bit integer using a `Hasher`. 2. Map the 64-bit hash to a specific table index and control tag.

Rust's `std::collections::HashMap` is a port of Google's SwissTable design (via the `hashbrown` crate). SwissTable divides the 64-bit hash into two components: - h_1 **(lower bits):** Used to determine the starting group index in the control byte array. - h_2 **(top 7 bits):** stored as a one-byte tag in the control bytes, which are scanned a group at a time (16 bytes with SSE2, smaller groups on some platforms).

A lookup compares h_2 with a whole group of control bytes at once. Each tag match is checked against the real key. If the key isn't in that group and the group has no empty slot, the search

moves to the next group in a fixed probe sequence. An insert follows the same path until it finds the key or a free slot.

The Attack Mechanics

In a normal distribution, hash values spread uniformly across buckets, resulting in short probe sequences and $O(1)$ amortized insertions.

If an attacker knows the exact hashing algorithm and its seed (or if the algorithm is unseeded), they can precalculate a sequence of keys $K = \{k_1, k_2, ..., k_n\}$ such that every key produces identical low bits (h_1) and identical top bits (h_2).

When inserting n colliding keys into an empty SwissTable: 1. The 1st key goes straight into the first slot of its probe sequence. 2. The 2nd key has the same tag as the 1st, so the table compares the two keys, finds they differ, and takes the next slot. 3. The i-th key walks past all $i - 1$ earlier keys, with a tag match and a key comparison for each, before it finds a free slot.

(This is a simplified count, ignoring resizes and group-at-a-time scanning, but the shape is right.)

Total insertion steps for n keys:

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2} \in O(n^2)$$

At $n = 20{,}000$ that is about 200 million key comparisons, for work that should take well under a millisecond. And the attacker only had to send the keys.

Why a Keyed Random Hash Stops HashDoS

The root cause of HashDoS is **predictability**. If an attacker cannot predict the hash output for a given key, they cannot precalculate colliding key sets.

SipHash (introduced by Jean-Philippe Aumasson and Daniel J. Bernstein in 2012) is a family of pseudorandom functions designed specifically for short inputs like hash table keys. It mixes the input with a 128-bit secret key through rounds of add, rotate and xor (ARX) operations. Without the key you can't predict the output.

`std::hash::RandomState` (also re-exported as `std::collections::hash_map::RandomState`) seeds SipHash with keys from a secure random source on the host, taken without blocking. The std docs describe this as "a reasonable best-effort". Keys are generated once per thread and changed slightly for every new `RandomState`.

Because the key is secret and generated at runtime: - An attacker sending JSON payloads or HTTP headers cannot determine which string keys will collide in the server's memory. - Two `RandomState`s, and so two maps made with `HashMap::new()`, hash the same key differently, even in one process. - Iteration order changes from run to run, and can differ between maps holding the same keys.

Historical Context

The HashDoS attack vector was popularized at the 28C3 conference in December 2011 by Alexander Klink and Julian Wälde (*"Efficient Denial of Service Attacks on Web Application Platforms"*). They showed hash flooding working against web platforms in many languages.

Afterwards: - Aumasson and Bernstein introduced SipHash in 2012, a fast keyed function meant for exactly this job. - Python switched `str` and `bytes` hashing to SipHash in Python 3.4 (PEP 456). - Rust's `HashMap` uses SipHash-1-3 today. The std docs say the algorithm "is subject to change at any point in the future".

A keyed hash stops the classic attack, where the attacker computes colliding keys offline. It does not make a table immune to every complexity attack. In October 2026 Orson Peters described a "black-box" HashDoS that never learns the hash function. It works on string keys that share a long common prefix: it times lookups to tell when a key comparison happened, uses that to map which buckets are occupied, and then builds keys that form one long probe chain. He demonstrated it against Rust's standard-library table. It needs a lot of interactive access: `lookup`, `insert` and `clear` on the same table, with no other users disturbing it, and O(n log n) preparation steps. His suggested mitigation is to compare a full 64-bit hash before comparing keys. He also says it is unclear how far the access requirements protect you. So keep the default hasher for untrusted keys, and also cap how many entries one client can put into a table.

Code Walkthrough: Demonstrating the Vulnerability

The program below shows an unseeded hasher going quadratic on chosen keys, while `RandomState` (SipHash) and `foldhash` stay linear on the same keys. Its only dependency is `foldhash`, added with `cargo add foldhash`. It was tested with rustc 1.99.1, edition 2024.

Project Setup

```
[package]
name = "hashdos"
version = "0.1.0"
edition = "2024"

[dependencies]
foldhash = "0.2.0"
```

The code resides at `src/main.rs`:

```
//! hashdos: what HashMap's default hasher protects you from.
use std::collections::HashMap;
use std::hash::{BuildHasher, BuildHasherDefault, Hasher, RandomState};
use std::time::{Duration, Instant};

/// A deliberately terrible hasher: the hash of a u64 is the u64 itself.
```

```
/// Fast, deterministic, and trivially attackable.
#[derive(Default)]
struct IdentityHasher(u64);

impl Hasher for IdentityHasher {
    fn finish(&self) -> u64 {
        self.0
    }
    fn write(&mut self, bytes: &[u8]) {
        for &b in bytes {
            self.0 = (self.0 << 8) | u64::from(b);
        }
    }
    fn write_u64(&mut self, n: u64) {
        self.0 = n;
    }
}

type Identity = BuildHasherDefault<IdentityHasher>;

fn time_inserts<S: BuildHasher>(build: S, keys: &[u64]) -> Duration {
    let mut m: HashMap<u64, u64, S> = HashMap::with_hasher(build);
    let t = Instant::now();
    for &k in keys {
        m.insert(k, k);
    }
    let d = t.elapsed();
    assert_eq!(m.len(), keys.len());
    d
}

fn main() {
    // 1. The default hasher is keyed per HashMap with random keys.
    let a = RandomState::new();
    let b = RandomState::new();
    println!("same key, two RandomStates: {:016x} vs {:016x}", a.hash_one(42u64), b.hash_one

    // 2. Iteration order therefore differs between runs of the program.
    let m: HashMap<&str, i32> = [("apple", 1), ("banana", 2), ("cherry", 3), ("date", 4)].in
    println!("iteration order this run: {:?}", m.keys().collect::<Vec<_>>());

    // 3. An attacker who knows the hash function picks colliding keys.
    // With the identity hash, multiples of 2^32 share the low bits the table
    // uses to pick a bucket and the top 7 bits it uses as a tag (all zero
    // for small multipliers), so every key collides with every other.
    for n in [5_000u64, 10_000, 20_000] {
        let evil: Vec<u64> = (0..n).map(|i| i << 32).collect();
```

```
        let benign: Vec<u64> = (0..n).collect();
        println!(
            "n={n:>6}  identity/benign {:>9.2?}  identity/evil {:>10.2?}  siphash/evil {:>9.
            time_inserts(Identity::default(), &benign),
            time_inserts(Identity::default(), &evil),
            time_inserts(RandomState::new(), &evil),
            time_inserts(foldhash::fast::RandomState::default(), &evil),
        );
    }
}
```

Key Components

1. **`IdentityHasher`**: `Hasher` needs only `finish` and `write(&[u8])`. Methods like `write_u64` have default implementations that call `write`, so overriding `write_u64` lets a `u64` key become its own hash with no byte shifting.
2. **`BuildHasherDefault<IdentityHasher>`**: Turns a `Hasher` that implements `Default` into a `BuildHasher` that makes fresh, unseeded `IdentityHasher`s. `time_inserts` is generic over the `BuildHasher` and builds its map with `HashMap::with_hasher`, because the hasher is the map's third type parameter: `HashMap<K, V, S>`.
3. **`BuildHasher::hash_one`**: A standard library convenience method (stable since Rust 1.71) that computes the hash of a single value without manually constructing a hasher.
4. **Attacker Key Construction (`i << 32`)**: Because `IdentityHasher` returns the raw `u64`, keys of the form $i \times 2^{32}$ have 32 trailing zero bits (filling h_1 with zero) and zero top bits for small i (filling h_2 with zero). So every key starts at the same group with the same tag.

Program Execution Results

Build and run it:

```
cargo build --release
./target/release/hashdos
```

```
same key, two RandomStates: a521e11b44355f6d vs bc8555747fa86f7e
iteration order this run: ["cherry", "date", "banana", "apple"]
n=  5000  identity/benign  123.46µs  identity/evil    11.13ms  siphash/evil  175.54µs  foldh
n= 10000  identity/benign  228.74µs  identity/evil    49.00ms  siphash/evil  397.28µs  foldh
n= 20000  identity/benign  454.52µs  identity/evil   185.61ms  siphash/evil  830.29µs  foldh
```

Running it again gave different first two lines:

```
same key, two RandomStates: 982862aac45c489c vs b6ac8abef2f118c8
iteration order this run: ["cherry", "banana", "apple", "date"]
```

Analyzing the Output Pattern

When reviewing execution output across problem sizes ($n = 5{,}000$, 10,000, 20,000):

The times are from one release build on one machine. Treat them as indicative; the growth pattern matters more than the exact figures.

1. **`identity/benign`**: the keys $0, 1, 2, \ldots$ have different low bits, so the identity hash spreads them out. Time grows linearly and stays under a millisecond. A weak hash can look fine on friendly data.
2. **`identity/evil`**: As input size n doubles from 5,000 to 10,000 and then to 20,000, the time goes from 11.13 ms to 49.00 ms to 185.61 ms, roughly **4x for each doubling**. This 4x scaling is the classic signature of quadratic $O(n^2)$ complexity.
3. **`siphash/evil`**: For the same keys, the keyed hash scatters them, so time grows linearly (175 µs to 830 µs).
4. **`foldhash/evil`**: `foldhash::fast::RandomState` is seeded too, so this naive attack fails against it as well, and it is the fastest column here. That doesn't make it SipHash's equal against an attacker who adapts.
5. **Randomness across runs**: comparing the two runs shows two things:
   - `a.hash_one(42)` and `b.hash_one(42)` produce completely different hex values within the same execution.
   - The key iteration order of `m` changes between independent runs of the program.

Non-Deterministic Iteration Order

Because `RandomState` generates new seeds at runtime, `HashMap` iteration order is non-deterministic. Never write tests or application code that depend on key ordering.

```
// BAD: Flaky test! Will fail randomly across test runs or Rust versions.
#[test]
fn test_key_order() {
    let mut map = std::collections::HashMap::new();
    map.insert("a", 1);
    map.insert("b", 2);
    assert_eq!(map.keys().copied().collect::<Vec<_>>(), vec!["a", "b"]);
}
```

If deterministic iteration or ordering is required: - **`std::collections::BTreeMap`**: Keys are kept sorted in $O(\log n)$ time. - **`indexmap::IndexMap`** (from the `indexmap` crate): keeps insertion order, with hash-table lookups. - **Explicit Sorting**: Collect keys into a `Vec` and invoke `.sort()` prior to comparison.

Choosing a Hasher

You choose the hasher through the third type parameter. `FastMap::default()` works because the `S` type implements `Default`; otherwise use `HashMap::with_hasher(s)`:

```rust
use std::collections::HashMap;
use foldhash::fast::RandomState as FoldRandomState;

// A HashMap type using foldhash instead of RandomState
type FastMap<K, V> = HashMap<K, V, FoldRandomState>;

let map: FastMap<String, usize> = FastMap::default();
```

Hasher	Algorithm Type	Seeded?	Intended Use Case
`std::hash::RandomState`	SipHash-1-3 today (may change)	Yes, random keys per thread, varied per instance	**The default.** Anything keyed by outside input: HTTP headers, JSON object keys, query parameters, user files. Designed to resist HashDoS.
`foldhash` (`foldhash::fast::RandomState`)	Fast non-cryptographic hash	Yes, per process	The default hasher of the `hashbrown` crate (0.17.1). The crate calls itself "minimally DoS-resistant": enough for naive attacks like the one above, not a claim of SipHash-level resistance.
`FxHash` (`rustc-hash` crate)	Fast non-cryptographic hash used inside rustc	**No**, deterministic	Keys you generate yourself, such as interned IDs in a compiler pass. Never attacker-controlled keys.

Decision Rules

1. **Untrusted Input Rule:** If keys originate from HTTP headers, URL queries, JSON body payloads, or external files, or anything else influenced by the outside world, **keep the default `RandomState`**.
2. **Trusted Pass Rule:** If keys are internal integer IDs, compiler symbols, or generated in a closed loop, and profiling shows hashing is a real cost, `foldhash` or `FxHash` is a reasonable swap. For small integer keys, consider a `Vec` indexed by ID, which needs no hashing at all.

Pitfalls and Common Mistakes

Global Hasher Overrides in Web Services

A crate-wide `type HashMap = FxHashMap` meant for one hot internal table ends up on maps that hold request data too. Then any client can flood them. Use the fast hasher on the specific maps that need it, by name.

Implementing `Hasher` by Ignoring Bytes

A `Hasher` that ignores some of its input is still *correct*: equal keys hash equally, so the map gives right answers. But every key collides, so ordinary data hits the quadratic path, no attacker needed:

```
// DANGEROUS: Ignores write data and returns constant
struct BrokenHasher;
impl Hasher for BrokenHasher {
    fn finish(&self) -> u64 { 42 }
    fn write(&mut self, _bytes: &[u8]) {}
}
```

Assuming Default Hasher Stability

The Rust standard library explicitly does not guarantee that `SipHash-1-3` will remain the default hasher in future releases. Never rely on specific hash outputs matching across different toolchain versions.

Floating-Point Keys

`f32` and `f64` implement neither `Eq` nor `Hash`. `NaN != NaN` breaks `Eq`, and `+0.0 == -0.0` although their bits differ (`0x00000000` and `0x80000000` for `f32`), so hashing the bits would break the rule that equal values hash equally. If you must key on floats, use a wrapper such as `ordered-float`'s `OrderedFloat` that defines both, or key on `to_bits()` after deciding how to treat `-0.0` and NaN.

Exercises

1. **Measure Collision Depths:** Change the evil keys to `i << 16`, then `i << 4`. Before you run each one, work out from the table size (a power of two above n) whether the low bits the table uses still all match, and predict the timings.
2. **Build a Custom `BuildHasher`:** Write a custom type implementing `BuildHasher` that accepts a manual 64-bit seed at construction, and use it to demonstrate deterministic hashing within a single run.
3. **Profile Hasher Overhead:** Create a benchmark using `criterion` comparing lookup times for 1,000,000 random string keys between `SipHash-1-3`, `foldhash`, and `FxHash`.

Key Takeaways

- Rust's `HashMap` defaults to `SipHash-1-3` parameterized by `RandomState` to defend against HashDoS CPU exhaustion attacks.
- Untrusted inputs forcing h_1 and h_2 collisions in SwissTable degrade $O(1)$ operations to $O(n)$, causing overall insertion workloads to balloon to quadratic $O(n^2)$ time.
- The random keys mean an outsider can't predict which keys collide, and they also make iteration order change between runs.
- `foldhash` and `FxHash` can be faster for keys you control. `FxHash` must never see attacker-chosen keys, and `foldhash` only claims minimal DoS resistance.
- `HashMap` iteration order is non-deterministic by design; use `BTreeMap` or `IndexMap` when deterministic key ordering is required.

Sources

- Rust std::collections::HashMap Documentation
- Rust std::hash::RandomState Documentation
- Rust std::hash::BuildHasher::hash_one Documentation
- foldhash Crate Documentation
- hashbrown GitHub Repository (SwissTable Port)
- PEP 456 – Secure & Interchangeable Hash Algorithm
- SipHash: a high-speed pseudorandom function (Aumasson & Bernstein, 2012)
- Orson Peters: Black-Box HashDoS: Attacking Hash Tables Without Knowing the Hash Function (October 2026)

bbtree: a concurrent ordered map (and unsync BTreeMap drop-in)

bbtree: a concurrent ordered map (and unsync BTreeMap drop-in)

After reading this chapter, you will understand how the `bbtree` 0.2.0 crate provides high-performance ordered map data structures in Rust, when to prefer `BBTreeMap` over standard synchronization primitives like `RwLock<BTreeMap>`, how to use `unsync::BBTreeMap` as a fast single-threaded alternative to standard `BTreeMap`, and how `bbtree` handles lock-free atomic readers alongside concurrent mutations.

Thesis

Rust's standard library provides `std::collections::BTreeMap` for sorted, range-queryable key-value storage. However, `std::collections::BTreeMap` requires exclusive mutable access (`&mut self`) for insertions and updates. Sharing a standard sorted map across threads typically requires wrapping it in a synchronization wrapper like `Arc<RwLock<BTreeMap<K, V>>>` or `Arc<Mutex<BTreeMap<K, V>>>`. Under high read or write concurrency, global locks introduce severe thread contention, context switching overhead, and coarse lock granularity.

Alternatively, lock-free concurrent maps like `crossbeam-skiplist` offer lock-free ordered access, but skip-lists often incur higher pointer-chasing overhead and cash-miss penalties compared to contiguous B-tree node layouts.

The `bbtree` 0.2.0 crate resolves this tradeoff by implementing an optimized B-tree layout tailored for modern hardware. It offers two distinct map types:

1. `bbtree::BBTreeMap<K, V>`: A lock-free concurrent ordered map that allows shared reference (`&self`) concurrent reads and inserts without external mutexes or read-write locks.
2. `bbtree::unsync::BBTreeMap<K, V>`: A single-threaded, `BTreeMap`-shaped ordered map designed as a drop-in replacement for `std::collections::BTreeMap` with lower cache overhead and improved cache locality.

Upstream benchmark claims position `bbtree` as a significantly faster B-tree implementation compared to `std::collections::BTreeMap` and `crossbeam-skiplist`. While application performance varies depending on workload characteristics, `bbtree` provides an efficient option for concurrent lookups and single-threaded sorted storage.

Architecture and Concurrency Model

Understanding how `bbtree` provides concurrent access requires examining the differences between standard B-tree operations and lock-free node traversal.

Standard BTreeMap vs Concurrent BBTreeMap

In `std::collections::BTreeMap`, node rebalancing, key insertion, and node splitting modify internal pointers and node arrays directly. If a read occurs concurrently with a split, the reader could observe invalid references, dangling pointers, or broken invariants.

`bbtree::BBTreeMap` bypasses global coarse-grained locks by leveraging optimistic concurrency, fine-grained lock synchronization during structural modifications (such as node splits), and lock-free atomic reads. Readers traversing `bbtree::BBTreeMap` observe consistent node state without acquiring exclusive read locks.

Feature	`std::collections::BTreeMap`	`crossbeam_skiplist::SkipMap`	`bbtree::BBTreeMap`
Concurrency Support	Single-threaded (`&mut self` required)	Multi-threaded lock-free	Multi-threaded concurrent (`&self` mutations)
Node Structure	Contiguous B-tree arrays	Linked list nodes with levels	Contiguous B-tree nodes
Cache Locality	High	Low (pointer chasing across nodes)	High (cache-friendly contiguous memory)
Single-threaded API	`std::collections::BTreeMap`	N/A	`bbtree::unsync::BBTreeMap`
Read Synchronization	Exclusively borrowed or via `RwLock`	Lock-free epoch-based	Lock-free / atomic reading

API Differences and Copy Semantics

Because `bbtree::BBTreeMap` permits concurrent modifications via shared references (`&self`), reading values directly via references (such as returning `&V`) poses memory safety challenges: a concurrent write or update could invalidate the memory location underlying `&V`.

To safeguard concurrent operations without sacrificing performance, `bbtree::BBTreeMap` provides specialised retrieval APIs for types that implement `Copy` or `Clone`:

- `get_copied(&key)`: Copies the value associated with `key` if `K: Borrow<Q>` and `V: Copy`.
- `read(&key, f)`: Executes a closure `f(&K, &V) -> R` under temporary internal access controls, enabling atomic reads of complex or non-`Copy` values without returning borrowed references across thread boundaries.
- `get_many_copied([&k1, &k2, ...])`: Batch retrieves multiple keys yielding an array of `Option<V>`.

In contrast, `bbtree::unsync::BBTreeMap` is intended for single-threaded usage where `&mut self` and exclusive borrowing guarantees remain standard. Consequently, `unsync::BBTreeMap` exposes standard borrowed retrieval methods like `get(&key)` returning `Option<&V>` and iterator accessors such as `keys()`.

Code Walkthrough: Concurrent and Unsync Operations

The following runnable lab demonstrates both concurrent mutation via `bbtree::BBTreeMap` and single-threaded usage via `bbtree::unsync::BBTreeMap`.

Project Configuration

Create a Cargo project with edition 2024 and include `bbtree` 0.2.0 as a dependency.

```
[package]
name = "bbtree_lab"
version = "0.1.0"
edition = "2024"

[dependencies]
bbtree = "0.2.0"
```

Main Application Code

The application code in `src/main.rs` contains two demonstrations:

1. `concurrent_demo()`: Spawns scoped threads that concurrently insert elements into a shared `BBTreeMap` instance (`&map`), then reads values using `get_copied`, `read`, and `get_many_copied`.
2. `unsync_demo()`: Demonstrates single-threaded map insertion, element retrieval, and key iteration using `bbtree::unsync::BBTreeMap`.

```
use bbtree::BBTreeMap;
use bbtree::unsync::BBTreeMap as UnsyncMap;

fn concurrent_demo() {
    let map = BBTreeMap::new();
    std::thread::scope(|scope| {
        scope.spawn(|| {
            for i in 0_u64..100 {
                let _ = map.insert(i, i * 10);
            }
        });
        scope.spawn(|| {
            for i in 100_u64..200 {
                let _ = map.insert(i, i * 10);
            }
        });
    });
    println!("get_copied(42)={:?}", map.get_copied(&42));
    println!("read(150)={:?}", map.read(&150, |_, v| *v));
```

```
    let many = map.get_many_copied([&1, &42, &199, &999]);
    println!("get_many_copied={many:?}");
}

fn unsync_demo() {
    let mut map = UnsyncMap::new();
    map.insert(1, "one");
    map.insert(2, "two");
    map.insert(3, "three");
    println!("unsync get(2)={:?}", map.get(&2));
    println!("unsync keys={:?}", map.keys().copied().collect::<Vec<_>>());
}

fn main() {
    concurrent_demo();
    unsync_demo();
}
```

Run the application using Cargo:

```
cargo run --release
```

The execution produces output showing successful concurrent insertion, read closure execution, batch element copying, and single-threaded lookup:

```
<<OUTPUT:run1>>
```

Common Mistakes and Pitfalls

Ignored Return Values on Insert

In `bbtree::BBTreeMap` version 0.2.0, the `insert` method returns a `Result` type rather than `Option<V>`. Calling `map.insert(k, v)` without binding or handling the returned `Result` triggers Rust's `#[warn(unused_must_use)]` compiler warning.

```
// Triggers unused_must_use compiler warning:
map.insert(key, value);

// Correct pattern: ignore explicitly or handle error
let _ = map.insert(key, value);
```

Expecting Standard `len()` on `BBTreeMap`

In version 0.2.0 of `bbtree`, `bbtree::BBTreeMap` does not expose a simple `len()` method due to the overhead of maintaining precise lock-free atomic length counters across high-concurrency write

operations. If tracking element count is mandatory in your concurrent system, maintain an explicit atomic counter (`std::sync::atomic::AtomicUsize`) alongside the map operations.

Expecting `&V` References from `BBTreeMap`

Developers migrating from `std::collections::BTreeMap` may expect `map.get(&key)` to return `Option<&V>`. Because `BBTreeMap` allows concurrent insertion and node splitting through `&self`, direct references to values inside concurrent nodes cannot be safely handed out. Always use `get_copied` for `Copy` types or the `read()` closure interface for zero-copy inspection of values.

When to Keep `std::collections::BTreeMap`

While `bbtree` offers competitive performance advantages for both concurrent and single-threaded workloads, standard `std::collections::BTreeMap` remains the standard choice in many scenarios:

1. **Standard Library Compatibility:** When interfacing with external crates or APIs requiring standard library types without external crate dependencies.
2. **Small Memory Footprint / Small Collections:** For small maps containing only a few elements, the low constant factors of `std::collections::BTreeMap` make it ideal.
3. **Full Iterator and Mutation APIs:** `std::collections::BTreeMap` provides extensive entry APIs (`entry()`), mutable iteration (`iter_mut()`), and splitting/merging APIs (`split_off()`, `append()`) that are not present in concurrent tree structures.

Summary

- `bbtree::BBTreeMap` provides a fast concurrent ordered map capable of handling reads and writes through shared references (`&self`).
- `bbtree::unsync::BBTreeMap` acts as a high-performance single-threaded alternative to `std::collections::BTreeMap`.
- Concurrent reads on `BBTreeMap` utilize `get_copied`, `read`, or `get_many_copied` to avoid returning unsafe references during parallel mutations.
- `insert` on concurrent `BBTreeMap` returns a `Result` that should be explicitly handled or ignored to avoid `must_use` compiler warnings.

Verified output

```
get_copied(42)=Some(420)
read(150)=Some(1500)
get_many_copied=[Some(10), Some(420), Some(1990), None]
unsync get(2)=Some("two")
unsync keys=[1, 2, 3]
```

Sources

- crates.io: bbtree
- docs.rs: bbtree 0.2.0 documentation
- GitHub Repository: XiangpengHao/bbtree
- Rust Standard Library Documentation: std::collections::BTreeMap

Other Collections: HashSet, BinaryHeap, LinkedList

Other Collections: HashSet, BinaryHeap, LinkedList

```rust
use std::collections::{HashSet, BinaryHeap};

fn main() {
    // HashSet: Unique items
    let mut set = HashSet::new();
    set.insert("admin");
    set.insert("operator");
    println!("Has admin? {}", set.contains("admin"));

    // BinaryHeap: Max-heap priority queue
    let mut heap = BinaryHeap::new();
    heap.push(10);
    heap.push(50);
    heap.push(25);
    println!("Max element: {:?}", heap.pop()); // Some(50)
}
```

Efficient Collection Operations

Efficient Collection Operations

Preallocate capacity using `Vec::with_capacity` to prevent repeated reallocations:

```
fn main() {
    let mut fast_vec = Vec::with_capacity(1000);
    for i in 0..1000 {
        fast_vec.push(i * 2);
    }
    println!("Length: {}, Capacity: {}", fast_vec.len(), fast_vec.capacity());
}
```

Summary

Summary

Chapter Key Takeaways

- `Vec<T>` is the default dynamic contiguous collection.
- Rust strings enforce strict UTF-8 invariants.
- `HashMap` offers fast amortized key-value lookups; `BTreeMap` keeps keys sorted.
- Use `with_capacity` to eliminate reallocations in hot code paths.

Project 1: Building a CLI Task Manager

Building a CLI Task Manager

Building a CLI Task Manager

In this project, we build a production-ready command-line Task Manager supporting task creation, listing, completion, and disk persistence.

```
#[derive(Debug, PartialEq)]
pub struct Task {
    pub id: usize,
    pub description: String,
    pub completed: bool,
}

pub struct TaskManager {
    tasks: Vec<Task>,
    next_id: usize,
}

impl TaskManager {
    pub fn new() -> Self {
        Self { tasks: Vec::new(), next_id: 1 }
    }

    pub fn add(&mut self, description: String) -> usize {
        let id = self.next_id;
        self.tasks.push(Task { id, description, completed: false });
        self.next_id += 1;
        id
    }

    pub fn complete(&mut self, id: usize) -> bool {
        if let Some(task) = self.tasks.iter_mut().find(|t| t.id == id) {
            task.completed = true;
            true
        } else {
            false
        }
    }

    pub fn list(&self) -> &[Task] {
        &self.tasks
    }
```

```
}

fn main() {
    let mut manager = TaskManager::new();
    let id1 = manager.add(String::from("Configure CI pipeline"));
    let _id2 = manager.add(String::from("Write unit tests"));
    manager.complete(id1);

    for t in manager.list() {
        println!("[{}] #{}: {}", if t.completed { "X" } else { " " }, t.id, t.description);
    }
}
```

Testing the Task Manager

Testing the Task Manager

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_task_addition_and_completion() {
        let mut mgr = TaskManager::new();
        let id = mgr.add("Audit dependencies".into());
        assert_eq!(mgr.list().len(), 1);

        let ok = mgr.complete(id);
        assert!(ok);
        assert!(mgr.list()[0].completed);
    }

    #[test]
    fn test_nonexistent_completion() {
        let mut mgr = TaskManager::new();
        assert!(!mgr.complete(999));
    }
}
```

Benchmarking the Task Manager

Benchmarking the Task Manager

Benchmarking insertion and lookup performance:

```
// Simulation harness
fn run_benchmark() {
    let mut mgr = TaskManager::new();
    let start = std::time::Instant::now();
    for i in 0..10_000 {
        mgr.add(format!("Task number {}", i));
    }
    let duration = start.elapsed();
    println!("Inserted 10,000 tasks in {:?}", duration);
}

fn main() {
    run_benchmark();
}
```

Deploying the Task Manager

Deploying the Task Manager

Compile a stripped, fully optimized release binary:

```
cargo build --release
strip target/release/desk_tasks
./target/release/desk_tasks --help
```

Error Handling in Rust

Unrecoverable Errors with panic!

Unrecoverable Errors with panic!

Unrecoverable errors represent impossible program states or fatal invariant violations.

```
fn main() {
    // panic!("Fatal condition encountered: terminating process");
    println!("Panics unwind the stack by default, running destructors along the way.");
}
```

Recoverable Errors with Result

Recoverable Errors with Result

Recoverable errors in Rust are modeled through `Result<T, E>`:

```
#[derive(Debug)]
enum MathError {
    DivisionByZero,
}

fn safe_divide(numerator: f64, denominator: f64) -> Result<f64, MathError> {
    if denominator == 0.0 {
        Err(MathError::DivisionByZero)
    } else {
        Ok(numerator / denominator)
    }
}

fn main() {
    match safe_divide(10.0, 2.0) {
        Ok(val) => println!("Quotient: {}", val),
        Err(e) => println!("Error: {:?}", e),
    }
}
```

The Option Type for Absent Values

The Option Type for Absent Values

Rust has no `null` or `nil`. Absent values are explicitly represented with `Option<T>`:

```
fn find_desk(number: u32) -> Option<&'static str> {
    if number == 1 {
        Some("Front Reception")
    } else {
        None
    }
}

fn main() {
    let desk = find_desk(1);
    println!("Found desk: {:?}", desk.unwrap_or("Unassigned"));
}
```

Propagating Errors with the ? Operator

Propagating Errors with the ? Operator

The `?` operator unwraps `Ok` values or immediately returns early with `Err`:

```
fn parse_and_double(s: &str) -> Result<i32, std::num::ParseIntError> {
    let n = s.trim().parse::<i32>()?;
    Ok(n * 2)
}

fn main() {
    match parse_and_double("42") {
        Ok(v) => println!("Result: {}", v),
        Err(e) => println!("Parse failed: {}", e),
    }
}
```

Custom Error Types and thiserror

Custom Error Types and thiserror

In production services, domain errors are structured into custom enums:

```
#[derive(Debug)]
pub enum ServiceError {
    InvalidInput(String),
    NetworkTimeout,
}

impl std::fmt::Display for ServiceError {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        write!(f, "{:?}", self)
    }
}

impl std::error::Error for ServiceError {}

fn main() {
    let err = ServiceError::InvalidInput("Missing header".into());
    println!("Service error: {}", err);
}
```

The Never Type ! and Infallible

The Never Type `!` and `Infallible`

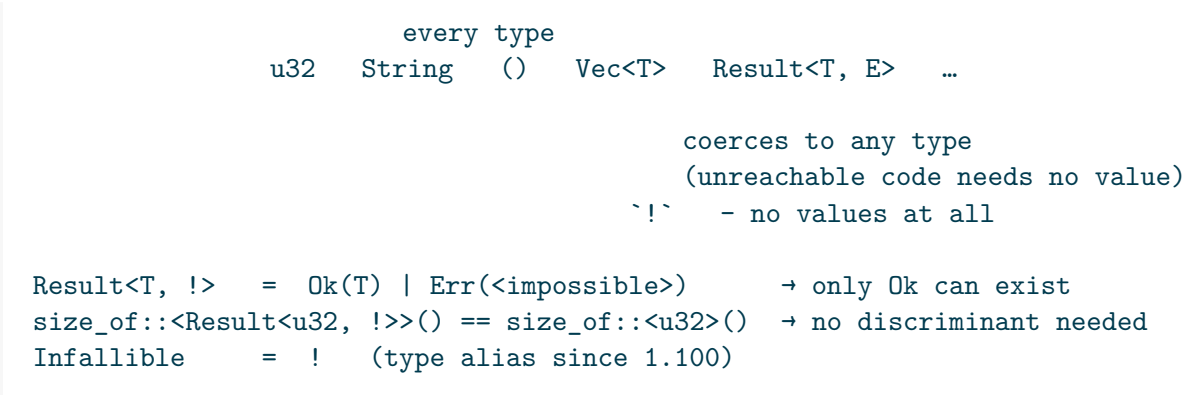
Thesis

Rust has always had a type for "this expression does not produce a value": `!`, the **never type**. `panic!()`, `loop {}` without `break`, `return`, `continue`, and `std::process::exit` all have type `!`, and a function declared `-> !` promises never to return. Until now you could only write `!` in that return position. Everywhere else — `Result<T, !>`, `type Error = !;`, `impl Trait for !` — it was an unstable feature, and the standard library used a stand-in, the empty enum `std::convert::Infallible`.

Rust 1.100 (scheduled for 2026-11-12) stabilizes `!` as an ordinary type after a decade behind a feature gate, and makes `Infallible` a plain alias for it. The language change is small; the error-handling consequence is real: "this operation cannot fail" now has one canonical spelling, `Result<T, !>`, that the compiler understands well enough to let you skip the impossible `Err` arm entirely.

The same release changes **never type fallback** to `!` on every edition, which is a small breaking change for code that quietly relied on the old fallback to `()`. This chapter shows both sides, compiled with the current beta and stable toolchains.

Mental model



Three facts carry the whole chapter:

Fact	Consequence
`!` has **no values**	Code that "holds" one is unreachable; the compiler can delete branches that need it
`!` **coerces to any type**	A diverging arm (`die(…)`, `panic!`, `return`) fits in a `match` whose other arms are `u32`

Fact	Consequence
An enum variant containing `!` is **uninhabited**	`let Ok(v) = r;` is irrefutable when `r: Result<T, !>`; the `Err` arm need not be written

`Infallible` was the library's way to get the same effect (an enum with no variants). Since it is now literally `!`, existing APIs such as `TryFrom` implementations whose error is `Infallible` work unchanged and read the same way.

Worked examples

All listings were compiled and run on the box with **rustc 1.100.0-beta.4 (2026-10-04)** and, for comparison, **rustc 1.99.0** stable, both installed with `rustup`. Until 1.100 reaches stable, run them with `cargo +beta` / `rustc +beta`:

```
rustup toolchain install beta --profile minimal
rustc +beta --version
```

```
rustc 1.100.0-beta.4 (a2eb6097d 2026-10-04)
```

Case 1: `!` as a type — cannot-fail results, irrefutable `Ok`

```
cargo +beta new never-demo && cd never-demo
```

```
// src/main.rs - the never type `!` as a first-class type (Rust 1.100)
use std::convert::Infallible;
use std::process;

// Case 1: a diverging function. `-> !` has been legal here since Rust 1.0.
fn die(msg: &str) -> ! {
    eprintln!("fatal: {msg}");
    process::exit(2)
}

// Case 2: `!` written anywhere a type goes - new in 1.100.
type Never = !;

fn parse_port(s: &str) -> Result<u16, Never> {
    // This parser cannot fail: anything unparsable maps to the default port.
    Ok(s.parse().unwrap_or(8080))
}

// Case 3: a trait whose implementors pick an error type; `!` means "cannot fail".
```

```
trait Source {
    type Error: std::fmt::Debug;
    fn read(&mut self) -> Result<u32, Self::Error>;
}

struct Counter(u32);
impl Source for Counter {
    type Error = !; // reading a counter never fails
    fn read(&mut self) -> Result<u32, !> {
        self.0 += 1;
        Ok(self.0)
    }
}

fn main() {
    // `!` coerces to any type, so a diverging arm fits a u32 match.
    let args: Vec<String> = std::env::args().collect();
    let n: u32 = match args.get(1) {
        Some(s) => s.parse().unwrap_or_else(|_| die("not a number")),
        None => 3,
    };

    // Irrefutable: the Err arm of Result<_, !> is uninhabited, so no match needed.
    let Ok(port) = parse_port("9000");
    println!("port = {port}");

    let mut c = Counter(0);
    let mut total = 0;
    for _ in 0..n {
        let Ok(v) = c.read();
        total += v;
    }
    println!("sum of first {n} reads = {total}");

    // Infallible is now the same type as `!`.
    let same: fn(Infallible) -> ! = |x| x;
    let _ = same;
    let Ok(wide) = u64::try_from(7u8); // TryFrom via Into: Error = Infallible = !
    println!("wide = {wide}");
    println!("size_of::<Result<u32, !>>() = {}", std::mem::size_of::<Result<u32, !>>());
    println!("size_of::<Result<u32, String>>() = {}", std::mem::size_of::<Result<u32, String
}
```

```
cargo +beta run -q
cargo +beta run -q -- 5
cargo +beta run -q -- five; echo "exit=$?"
```

```
port = 9000
sum of first 3 reads = 6
wide = 7
size_of::<Result<u32, !>>() = 4
size_of::<Result<u32, String>>() = 24
port = 9000
sum of first 5 reads = 15
wide = 7
size_of::<Result<u32, !>>() = 4
size_of::<Result<u32, String>>() = 24
fatal: not a number
exit=2
```

What to notice:

- `unwrap_or_else(|_| die("not a number"))` type-checks because the closure returns `!`, which coerces to `u32`.
- `let Ok(port) = …;` and `let Ok(v) = c.read();` have **no `else`** and no `match`. The compiler knows `Err(!)` cannot be constructed. With `Result<u16, String>` the same line is a compile error (refutable pattern).
- `Counter::Error = !` documents "cannot fail" in the type, and callers that are generic over `Source` still handle `Self::Error` normally.
- `Result<u32, !>` is the same size as a bare `u32`: no tag is stored for a variant that cannot exist.
- `fn(Infallible) -> !` accepting `|x| x` is the alias at work: the parameter *is* a `!`.

The same file on 1.99 stable stops at the first `!` in type position:

```
cargo +stable build -q
```

```
error[E0658]: the `!` type is experimental
  --> src/main.rs:12:14
   |
12 | type Never = !;
   |              ^
   |
   = note: see issue #35121 <https://github.com/rust-lang/rust/issues/35121> for more inform
...
```

(one such error per `!` outside a return position). Until your minimum supported Rust version is 1.100, keep spelling it `Infallible` in public APIs; code that already uses `Infallible` keeps compiling and gains nothing by being rewritten.

Case 2: the fallback change — code that relied on `()` by accident

When the compiler meets a `!` it must coerce, but nothing tells it the target type, it picks a **fallback**. That fallback used to be `()`; Rust 2024 switched it to `!`, and 1.100 does the same for editions 2015–

2021. The most common casualty is `?` on a function that is generic in its `Ok` type:

```
// fallback.rs - code that silently relied on `!` falling back to `()`
fn f<T: Default>() -> Result<T, ()> {
    Ok(T::default())
}

fn outer() -> Result<u32, ()> {
    f()?; // which T? nothing here says
    Ok(1)
}

fn main() {
    println!("{:?}", outer());
}
```

On 1.99, edition 2021: already rejected by a deny-by-default future-compatibility lint, which names the fix:

```
rustc +stable --edition 2021 fallback.rs
```

```
error: this function depends on never type fallback being `()`
 --> fallback.rs:6:1
  |
6 | fn outer() -> Result<u32, ()> {
  | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  |
  = help: specify the types explicitly
note: in edition 2024, the requirement `!: Default` will fail
 --> fallback.rs:7:5
  |
7 |     f()?; // which T? nothing here says
  |     ^^^
  = warning: this was previously accepted by the compiler but is being phased out; it will b
  = note: for more information, see <https://doc.rust-lang.org/edition-guide/rust-2024/never
  = note: `#[deny(dependency_on_unit_never_type_fallback)]` (part of `#[deny(rust_2024_compa
help: use `()` annotations to avoid fallback changes
  |
7 |     f::<()>()?; // which T? nothing here says
  |      ++++++

error: aborting due to 1 previous error
```

On 1.100 beta, edition 2021 — the lint is gone because the behaviour changed; you get the real type error (identical to what 1.99 already says under edition 2024):

```
rustc +beta --edition 2021 fallback.rs
```

```
error[E0277]: the trait bound `!: Default` is not satisfied
 --> fallback.rs:7:5
  |
7 |     f()?; // which T? nothing here says
  |     ^^^ the trait `Default` is not implemented for `!`
  |
  = note: this error might have been caused by changes to Rust's type-inference algorithm (s
  = help: you might have intended to use the type `()` here instead
note: required by a bound in `f`
 --> fallback.rs:2:9
  |
2 | fn f<T: Default>() -> Result<T, ()> {
  |         ^^^^^^^ required by this bound in `f`
```

The fix is to say what you meant:

```
// fallback_fixed.rs - name the type instead of relying on fallback
fn f<T: Default>() -> Result<T, ()> {
    Ok(T::default())
}

fn outer() -> Result<u32, ()> {
    f::<()>()?; // say it: the Ok value is ()
    Ok(1)
}

fn main() {
    println!("{:?}", outer());
}
```

```
rustc +beta --edition 2021 fallback_fixed.rs && ./fallback_fixed
```

```
Ok(1)
```

Case 3: the panicking closure

The second common pattern is a closure that only panics, passed to something that requires its return type to implement a trait:

```
// closure.rs - a panicking closure used to "return ()" by accident
trait Unit {}
impl Unit for () {}

fn run<R: Unit>(f: impl FnOnce() -> R) {
    let _ = std::panic::catch_unwind(std::panic::AssertUnwindSafe(f));
```

```
}

fn main() {
    run(|| panic!("boom"));
    println!("survived");
}
```

```
rustc +beta --edition 2024 closure.rs
```

```
error[E0277]: the trait bound `!: Unit` is not satisfied
  --> closure.rs:10:5
   |
10 |     run(|| panic!("boom"));
   |     ^^^^^^^^^^^^^^^^^^^^^^ the trait `Unit` is not implemented for `!`
   |
help: the trait `Unit` is implemented for `()`
  --> closure.rs:3:1
   |
 3 | impl Unit for () {}
   | ^^^^^^^^^^^^^^^^
   = note: this error might have been caused by changes to Rust's type-inference algorithm (
```

Annotate the closure's return type: `run(|| -> () { panic!("boom") });`

```
rustc +beta --edition 2024 closure_fixed.rs && ./closure_fixed
```

```
thread 'main' (250345) panicked at closure_fixed.rs:10:20:
boom
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
survived
```

Case 4: `Infallible` and `!` are one type now

If a crate implemented a trait for both — typically to support nightly users of `!` — the two impls now overlap:

```
// conflict.rs - Infallible and ! are now one type
use std::convert::Infallible;
trait Describe { fn describe() -> &'static str; }
impl Describe for Infallible { fn describe() -> &'static str { "infallible" } }
impl Describe for ! { fn describe() -> &'static str { "never" } }
fn main() { println!("{}", <! as Describe>::describe()); }
```

```
rustc +beta --edition 2024 conflict.rs
```

```
error[E0119]: conflicting implementations of trait `Describe` for type `!`
 --> conflict.rs:5:1
  |
4 | impl Describe for Infallible { fn describe() -> &'static str { "infallible" } }
  | ---------------------------- first implementation here
5 | impl Describe for ! { fn describe() -> &'static str { "never" } }
  | ^^^^^^^^^^^^^^^^^^^ conflicting implementation for `!`
```

Delete one of them. Keep the `Infallible` impl if you still support compilers older than 1.100: it covers `!` on new compilers for free.

Where `!` belongs in error handling

Situation	Spell it	Why
Function that never returns (exit, `exec`, server loop)	`fn run() -> !`	Callers can use it in any expression position
Trait with an associated error type; this impl cannot fail	`type Error = !;`	Callers generic over the trait still compile; concrete callers write `let Ok(v) = …;`
Public API that must build on Rust < 1.100	`Infallible`	Same type on new compilers, still compiles on old ones
"Can't fail *yet*" — you expect failures later	A real error type	Changing `!` to `MyError` later is a breaking API change
Converting between error types with `?`	`impl From<Infallible> for MyError` if needed	Works for `!` too, since it is the same type

The trap

Relying on **inference to pick `()` for you**. Code like `f()?;`, `run(|| panic!())`, or `if c { Default::default() } else { return }` compiled for years because the never type quietly became `()` when nothing else constrained it. Under 1.100 (any edition) or edition 2024 that guess is `!`, and the code either fails to compile — the good case, shown above — or, in the rare `unsafe` case the `never_type_fallback_flowing_into_unsafe` lint exists for, picks a different type than the author assumed. The compiler's 1.99 lint (`dependency_on_unit_never_type_fallback`) points at every affected function: fix those before you bump the toolchain, not after.

The second, quieter trap is using `!` as a placeholder in a public trait impl or function signature because "it can't fail today". `!` is a promise. Replacing it with a real error type later breaks every caller who wrote `let Ok(v) = …;`.

The boring rule

Use `-> !` for functions that truly never return and `Result<T, !>` (or `Infallible` until your MSRV is 1.100) only where failure is impossible by construction. Never lean on fallback: when the compiler cannot see a type, **write it** — `f::<()>()?`, `|| -> () { … }`, `() = expr;`. Turn on the `rust_2024_compatibility` lints on older editions and clear them before upgrading.

Try this

1. Change `parse_port` to return `Result<u16, String>`. Which lines stop compiling, and what does the compiler suggest for `let Ok(port) = …;`?
2. Print `size_of::<Option<!>>()` and `size_of::<Result<!, String>>()` and explain the numbers.
3. Write `fn absurd<T>(x: !) -> T { x }` and use it to implement `fn into_ok<T>(r: Result<T, !>) -> T` with a `match` that has a single arm.
4. Take a crate you maintain on edition 2021, run `cargo +stable check` (1.99 today) and look for `dependency_on_unit_never_type_fallback`; then `cargo +beta check` and confirm every site is fixed.
5. Find a `TryFrom` impl in `std` whose `Error` is `Infallible` and destructure its result with an irrefutable `let`.

Sources

- rust-lang/rust PR #155499, "stabilize never type" (merged 2026-08-24): https://github.com/rust-lang/rust/pull/155499
- Draft release notes for Rust 1.100.0 (stabilize never type; `Infallible` alias; fallback on all editions): https://github.com/rust-lang/rust/issues/163811
- Tracking issues: never type https://github.com/rust-lang/rust/issues/35121, fallback change https://github.com/rust-lang/rust/issues/148922
- The Rust Edition Guide — Never type fallback change: https://doc.rust-lang.org/edition-guide/rust-2024/never-type-fallback.html
- The Rust Reference — Never type: https://doc.rust-lang.org/reference/types/never.html
- Standard library — primitive `never`: https://doc.rust-lang.org/std/primitive.never.html; `std::convert::Infallible`: https://doc.rust-lang.org/std/convert/enum.Infallible.html

- Rust Project Goals 2026 — Stabilize never type: https://goals.rust-lang.org/2026/stabilize-never-type.html

- LWN, “Stabilizing Rust’s never type” (2026-09-08): https://lwn.net/Articles/1091015/

Summary

Summary

Chapter Key Takeaways

- `panic!` is reserved for bugs and unrecoverable states.
- `Result<T, E>` and `Option<T>` model recoverable failure and absence.
- The `?` operator provides zero-cost error propagation.
- Custom error enums unify error types across modules.

Generics, Traits, and Associated Types

Generic Functions and Data Structures

Generic Functions and Data Structures

Generics allow writing flexible, reusable code with monomorphized zero runtime overhead.

```
#[derive(Debug)]
struct Pair<T> {
    first: T,
    second: T,
}

fn swap<T>(a: T, b: T) -> (T, T) {
    (b, a)
}

fn main() {
    let p = Pair { first: 10, second: 20 };
    let (x, y) = swap("hello", "world");
    println!("Pair: {:?}, Swapped: ({}, {})", p, x, y);
}
```

Defining and Implementing Traits

Defining and Implementing Traits

Traits define shared behavior analogous to interfaces:

```
pub trait Summary {
    fn summarize(&self) -> String;
}

pub struct Article {
    pub headline: String,
}

impl Summary for Article {
    fn summarize(&self) -> String {
        format!("Headline: {}", self.headline)
    }
}

fn main() {
    let a = Article { headline: "Rust 2026 Ships".into() };
    println!("{}", a.summarize());
}
```

Trait Bounds and Where Clauses

Trait Bounds and Where Clauses

Constrain generics using trait bounds and `where` clauses:

```
use std::fmt::Display;

fn print_and_return<T>(val: T) -> T
where
    T: Display + Clone,
{
    println!("Value: {}", val);
    val.clone()
}

fn main() {
    let res = print_and_return(42);
    println!("Returned: {}", res);
}
```

Associated Types vs Generics

Associated Types vs Generics

Associated types simplify trait definitions when there is a unique type output per implementation:

```
trait DataSink {
    type Error;
    fn send(&mut self, bytes: &[u8]) -> Result<(), Self::Error>;
}
```

Trait Objects and Dynamic Dispatch

Trait Objects and Dynamic Dispatch

When heterogeneous collections are required at runtime, use dynamic dispatch via trait objects (`dyn Trait`):

```
trait Drawable {
    fn draw(&self);
}

struct Button;
impl Drawable for Button {
    fn draw(&self) { println!("Drawing button"); }
}

struct Checkbox;
impl Drawable for Checkbox {
    fn draw(&self) { println!("Drawing checkbox"); }
}

fn render_all(items: &[Box<dyn Drawable>]) {
    for item in items {
        item.draw();
    }
}

fn main() {
    let widgets: Vec<Box<dyn Drawable>> = vec![
        Box::new(Button),
        Box::new(Checkbox),
    ];
    render_all(&widgets);
}
```

Generic Const Arguments (GCA)

Generic Const Arguments (GCA)

Thesis

On stable Rust, a const generic parameter may appear in a const argument **alone** (`[u8; N]`) or as a fully concrete expression (`[u8; 1 + 1]`). The moment you write `[u8; N + 1]` or `[u8; T::ASSOC]`, the compiler stops you. The old escape hatch, `#![feature(generic_const_exprs)]` (GCE), taught everyone the shape of the problem but never became a stabilization candidate.

Generic Const Arguments (GCA) is the Const Generics project group's replacement family of nightly features. Expressions that involve generic parameters go through an explicit `gca!(…)` marker (for now), each capability has its own gate (`gca_adts`, `gca_min_const_items`, `gca_const_items`, …), and definitional equality — not GCE's evaluation model — decides when two const arguments match. Nothing here is stable. This chapter shows what compiles on **today's nightly**, marks every feature gate, and records where the Oct 2026 blog post already drifted from the compiler.

Version note: verified on **`rustc 1.101.0-nightly (a30aa9064 2026-10-08)`**. The Inside Rust post *Generic Const Args and You* (2026-10-02) is the design overview; on this nightly the feature formerly named `generic_const_args` has been **renamed to `gca_const_items`** (removed name errors say "removed in 1.100.0"), and the `gca!` macro lives at **`std::gca`** / `core::gca`. No RFC yet; the project group says polish first.

Stable const generics basics (structs/functions parameterized by `const N: usize`) live in *Generic Functions and Data Structures*. This chapter does not re-teach that.

Mental model

```
Stable today
  const arg ::=  N  |  <fully concrete expr>
                   generic param alone

GCE (legacy nightly)     - evaluate complex exprs; hard to stabilize
GCA (current nightly)    - mark exprs with gca!(…); equality by definition

          gca_adts              arrays / tuples / structs / enums in gca!(…)
          gca_min_const_items   const items whose RHS is gca!(…)
          gca_const_items       + opaque const items, assoc bindings, dyn, needs -Znext-solve
          gca_macroless_*       experimental: insert gca!(…) for you (rough)
```

Fact	Consequence
`gca!(…)` may mention generic parameters	Plain `{ N + 1 }` as a const argument still cannot
`gca!(…)` accepts **fewer** expression kinds than a normal const block	`gca!(N + 1)` and `gca!(foo())` are *not* automatically legal; prefer const items / ADTs that the gates support
`gca_const_items` requires `gca_min_const_items`	Enable both; incomplete-feature warnings are expected
Feature `generic_const_args` is gone on 1.101 nightly	Use `gca_const_items` (compiler points the rename)
Nothing is stabilization-ready	APIs that expose `gca!(…)` in public signatures are a SemVer hazard while the syntax is explicit

Worked examples

All listings were compiled with:

```
rustup toolchain install nightly --profile minimal   # once
rustc +nightly --version
```

```
rustc 1.101.0-nightly (a30aa9064 2026-10-08)
```

Case 0: stable still rejects `N + 1`

```
// stable_fail.rs - what stable const generics refuse
fn make<const N: usize>() -> [u8; N + 1] {
    [0; N + 1]
}

fn main() {
    let _ = make::<3>();
}
```

```
rustc --edition 2024 stable_fail.rs
```

```
error: generic parameters may not be used in const operations
 --> stable_fail.rs:1:35
  |
1 | fn make<const N: usize>() -> [u8; N + 1] {
  |                                   ^ cannot perform const operation using `N`
  |
  = help: const parameters may only be used as standalone arguments here, i.e. `N`
```

```
  = help: add `#![feature(generic_const_exprs)]` to allow generic const expressions
  = help: alternatively, you can use `#![feature(gca_const_items)]` and extract the expressi
```

Note the help text still mentions GCE *and* GCA — both are nightly escape hatches; new code should reach for GCA, not GCE.

Case 1: ADT const arguments with `gca_adts`

Build a const-generic parameter whose type is a struct, then construct that struct from another const parameter inside `gca!(…)`:

```rust
// gca_adt.rs - struct expressions as const arguments (nightly GCA)
#![feature(gca_adts, adt_const_params, gca_min_const_items)]
#![expect(incomplete_features)]

use std::gca;
use std::marker::ConstParamTy;

#[derive(ConstParamTy, Eq, PartialEq)]
struct TupleStruct(usize);

fn accepts<const N: TupleStruct>() -> usize {
    N.0
}

fn example<const N: usize>() -> usize {
    accepts::<gca!(TupleStruct(N))>()
}

fn main() {
    println!("{}", example::<7>());
}
```

```
rustc +nightly --edition 2024 gca_adt.rs -o gca_adt && ./gca_adt
```

```
7
```

The same pattern works for enum variants, arrays (`gca!([N1, 12])`), and tuples (`gca!((N1, 12))`) per the project-group post. Array *repeat* syntax (`gca!([N1; 2])`) is still explicitly **not** supported.

Case 2: Associated constants with `gca_const_items`

```rust
// gca_assoc.rs - associated consts in array lengths (nightly GCA)
#![feature(gca_min_const_items, gca_const_items)]
#![expect(incomplete_features)]

use std::gca;

trait Trait {
    const ASSOC: usize;
}

impl Trait for u8 {
    const ASSOC: usize = 3;
}

fn example<T: Trait>() -> [u8; gca!(T::ASSOC)] {
    [0x1; _]
}

fn main() {
    let a = example::<u8>();
    println!("len={}", a.len());
}
```

```
rustc +nightly --edition 2024 gca_assoc.rs -Znext-solver -o gca_assoc && ./gca_assoc
```

```
len=3
```

`-Znext-solver` is required for the full `gca_const_items` path in current nightlies. Treat that as part of the experiment, not as something to bake into stable CI.

Case 3: Macroless (experimental)

`gca_macroless_args` asks the compiler to insert `gca!(…)` for you. It works for some associated-const cases on this nightly, and the project group still recommends writing `gca!(…)` explicitly:

```rust
// gca_macroless.rs - implicit gca!(…) (rough; prefer explicit)
#![feature(gca_min_const_items, gca_const_items, gca_macroless_args)]
#![expect(incomplete_features)]

trait Trait {
    const ASSOC: usize;
}
```

```rust
impl Trait for u8 {
    const ASSOC: usize = 3;
}

fn make_array<T: Trait>() -> [u8; T::ASSOC] {
    [0x1; _]
}

fn main() {
    println!("{}", make_array::<u8>().len());
}
```

```
rustc +nightly --edition 2024 gca_macroless.rs -Znext-solver -o gca_macroless && ./gca_macro
```

```
3
```

Case 4: What still hurts — arithmetic inside `gca!`

The blog's long-term story is "put complex ops in const items." On 1.101 nightly, a direct `gca!(N + 1)` as a const-item RHS is still rejected:

```rust
// gca_arith.rs - not there yet on 2026-10-08 nightly
#![feature(gca_min_const_items, gca_const_items, generic_const_items)]
#![expect(incomplete_features)]
use std::gca;

const INC<const N: usize>: usize = gca!(N + 1);

fn make<const N: usize>() -> [u8; INC::<N>] {
    [0; INC::<N>]
}

fn main() {
    let _ = make::<3>();
}
```

```
rustc +nightly --edition 2024 gca_arith.rs
```

```
error: complex const arguments must be placed inside of a `const` block
 --> gca_arith.rs:5:41
  |
5 | const INC<const N: usize>: usize = gca!(N + 1);
  |                                         ^^^^^
```

So: ADTs and associated-const *uses* are the productive surface today; "any arithmetic via `gca!`" is

not. Re-check nightly before teaching a team pattern around arithmetic helpers.

The trap

1. **Copy-pasting `#![feature(generic_const_args)]` from older posts.** Removed in 1.100; rename to `gca_const_items` and enable `gca_min_const_items` with it.
2. **Assuming `gca!(…)` is a full const evaluator.** It is a marker for a *restricted* expression grammar that may mention generics. Unsupported ops fail loudly.
3. **Stabilizing public APIs on `gca!`.** The macroless work exists precisely because explicit `gca!` is considered ergonomically temporary — and also SemVer-visible.
4. **Mixing GCE and GCA in one crate "just to make it compile".** Two incomplete models; pick GCA for new experiments.
5. **Forgetting `#![expect(incomplete_features)]` / `#[allow(incomplete_features)]`.** Without it, every build is a warning storm (or worse, depending on lint config).

The boring rule

Stay on stable const generics (`N` alone, or concrete exprs) for production. When you truly need generic const computation, use **current nightly GCA** with explicit `gca!(…)`, gate every file with the specific features you touch, pin the nightly date in the chapter/PR, and re-run the listings after each toolchain bump — the rename from `generic_const_args` to `gca_const_items` already proved the surface moves.

Try this

1. Compile Case 1 and Case 2 on your nightly; paste `rustc +nightly --version` next to the output.
2. Attempt one `gca!([N; 2])` repeat expression and confirm it still errors; watch the tracking issues for when it lands.
3. Skim the project-const-generics repo / Zulip before filing duplicates — design feedback is solicited there.

Sources

- Generic Const Args and You — Inside Rust, 2026-10-02 (inspiration; see *Updated vs original* in the PR)
- generic_const_args — Unstable Book (tracking issue #151972)
- Full Const Generics — 2026 project goal
- Verified locally: `rustc 1.101.0-nightly (a30aa9064 2026-10-08)`

AsRef, Borrow and Deref: Same Signature, Different Promises

`AsRef`, `Borrow` and `Deref`: Same Signature, Different Promises

Thesis

Three standard traits all turn `&Self` into `&T`:

```
pub trait AsRef<T: ?Sized>  { fn as_ref(&self) -> &T; }
pub trait Borrow<T: ?Sized> { fn borrow(&self) -> &T; }
pub trait Deref             { type Target: ?Sized; fn deref(&self) -> &Self::Target; }
```

They are not interchangeable. The difference is what each one **promises** to the code that uses it.

- **`AsRef<T>`** promises only a cheap conversion. Use it as a generic bound, such as `P: AsRef<Path>`, to accept many input types.
- **`Borrow<T>`** promises the same conversion **and** that "`Eq`, `Ord` and `Hash` must be equivalent for borrowed and owned values" (std docs). That promise is the only reason `HashMap<String, V>::get` can take a `&str`. The map hashes the `&str` you pass and trusts it to land in the same bucket as the `String` it stored.
- **`Deref`** drives the `*` operator and auto-deref method lookup. The std docs reserve it for types that "transparently behave like a value of the target type".

The compiler checks the signatures of all three. It doesn't check the promises. This chapter shows each trait doing its job, then breaks `Borrow`'s contract on purpose. The result is a map that compiles, passes a lowercase-only test, and loses `"Content-Type"`.

Mental model

	`AsRef<T>`	`Borrow<T>`	`Deref`
What it promises	a cheap `&T`	a cheap `&T` whose `Hash`/`Eq`/`Ord` match `Self`'s	`*x` and method calls go to `Target`
Reflexive (`T: Trait<T>`)?	**no**	yes (blanket `impl<T> Borrow<T> for T`)	n/a
Several targets per type?	yes (`String`: `AsRef<str>`, `AsRef<[u8]>`, `AsRef<Path>`, `AsRef<OsStr>`)	yes, if each keeps the contract	no, exactly one

	`AsRef<T>`	`Borrow<T>`	`Deref`
May lend just one field?	yes	no (the field's `Hash`/`Eq` would differ)	not recommended
Typical user	`fn open<P: AsRef<Path>>(p: P)`	`HashMap::get`, `BTreeMap::range`, `Cow`/`ToOwned`	`Box<T>`, `String`, `Vec<T>`, guards

Worked examples

Built and run with **rustc/cargo 1.85.1** (the distro toolchain on the box), edition 2024. The current stable Rust is **1.99.1**. Every API used here has been stable since well before 1.85, and its documentation is unchanged. The project has no dependencies.

```
cargo new --vcs none conv && cd conv
rm src/main.rs && mkdir -p src/bin
```

```
# Cargo.toml
[package]
name = "conv"
version = "0.1.0"
edition = "2024"
rust-version = "1.85"

[dependencies]
```

Case 1: How `HashMap<String, V>` answers a `&str`

```
// src/bin/lookup.rs - why HashMap<String, V>::get accepts a &str
use std::borrow::Borrow;
use std::collections::{BTreeMap, HashMap};
use std::hash::{BuildHasher, RandomState};
use std::ops::Bound::{Excluded, Included};

fn main() {
    let mut ports: HashMap<String, u16> = HashMap::new();
    ports.insert("http".to_string(), 80);
    ports.insert("https".to_string(), 443);

    // get<Q: ?Sized>(&self, k: &Q) where String: Borrow<Q>, Q: Hash + Eq
    let key: &str = "https";
    println!("get(&str)    -> {:?}", ports.get(key));
    println!("get(&String) -> {:?}", ports.get(&"http".to_string()));
```

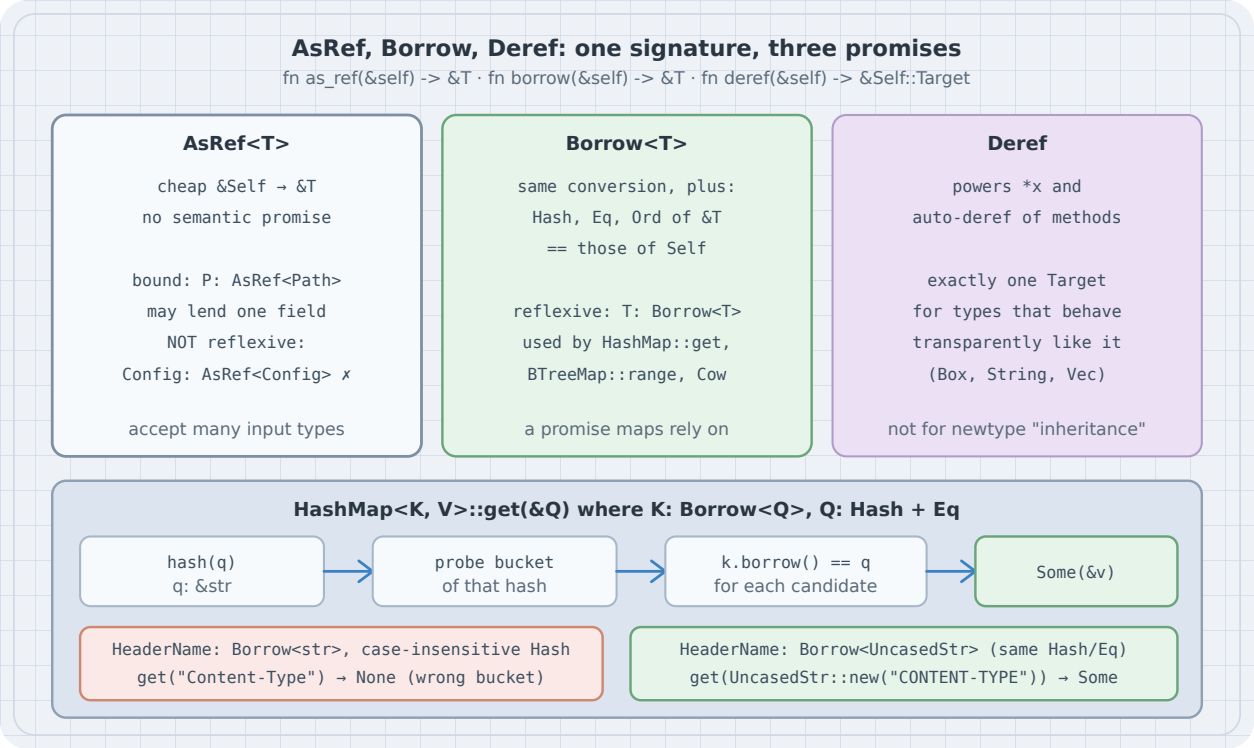


Figure 1: AsRef, Borrow and Deref compared; HashMap::get hashes the borrowed key, probes, compares k.borrow() == q; a case-insensitive Borrow misses, a Borrow with the same Hash/Eq hits

```
    // The contract that makes it sound: hashing the owned key and the borrowed key agree.
    let s = RandomState::new();
    let owned = String::from("https");
    let borrowed: &str = owned.borrow();
    println!("hash(String) == hash(&str)? {}", s.hash_one(&owned) == s.hash_one(borrowed));

    // Ordered maps use the same trick with Ord: range queries by &str over String keys.
    let mut users: BTreeMap<String, u32> = BTreeMap::new();
    for (name, id) in [("alice", 1), ("bob", 2), ("carol", 3), ("dave", 4)] {
        users.insert(name.to_string(), id);
    }
    // `"b".."d"` is a Range<&str>, which is not RangeBounds<str>; spell the bounds out inst
    let picked: Vec<_> = users
        .range::<str, _>((Included("b"), Excluded("d")))
        .map(|(k, v)| (k.as_str(), *v))
        .collect();
    println!("range(b..d)  -> {picked:?}");
}
```

```
$ cargo run -q --bin lookup
get(&str)    -> Some(443)
get(&String) -> Some(80)
hash(String) == hash(&str)? true
range(b..d)  -> [("bob", 2), ("carol", 3)]
```

`HashMap::get` is declared as `get<Q: ?Sized>(&self, k: &Q) -> Option<&V> where K: Borrow<Q>, Q: Hash + Eq`. With `K = String` and `Q = str`, the map hashes the `&str`, finds the bucket, and compares each candidate with `key.borrow() == q`. No `String` is allocated for the lookup. The `hash_one` line shows the property that makes this correct: `String` and `str` produce the same hash, because `String`'s `Hash` forwards to `str`'s. `BTreeMap` uses the same bound with `Ord` for `get` and `range`. (The `range` call needs its bounds spelled out, because `"b".."d"` is a `Range<&str>`, not a `RangeBounds<str>`.)

Case 2: `AsRef` as an "accept anything" bound

```
// src/bin/asref.rs - AsRef as a cheap "accept anything that can lend me a &T" bound
use std::ffi::OsString;
use std::path::{Path, PathBuf};

fn describe<P: AsRef<Path>>(p: P) -> String {
    let p = p.as_ref(); // one conversion, then work with &Path
    format!("{:<18} file_name={:?} ext={:?}", p.display().to_string(), p.file_name(), p.exte
}

fn checksum<B: AsRef<[u8]>>(data: B) -> u32 {
```

```
    data.as_ref().iter().fold(0u32, |a, &b| a.rotate_left(5) ^ b as u32)
}

fn main() {
    println!("{}", describe("notes/today.md"));
    println!("{}", describe(String::from("Cargo.toml")));
    println!("{}", describe(PathBuf::from("/etc/hosts")));
    println!("{}", describe(Path::new("archive.tar.gz")));
    println!("{}", describe(OsString::from("README")));

    // The same bound works for every byte container.
    println!("checksum: {} {} {} {}",
        checksum("abc"), checksum(String::from("abc")), checksum(b"abc"), checksum(vec![b'a'
}
```

```
$ cargo run -q --bin asref
notes/today.md     file_name=Some("today.md") ext=Some("md")
Cargo.toml         file_name=Some("Cargo.toml") ext=Some("toml")
/etc/hosts         file_name=Some("hosts") ext=None
archive.tar.gz     file_name=Some("archive.tar.gz") ext=Some("gz")
README             file_name=Some("README") ext=None
checksum: 100387 100387 100387 100387
```

`AsRef<Path>` is implemented for `str`, `String`, `OsStr`, `OsString`, `Path`, `PathBuf` and more, and `&T` forwards to `T`. One generic signature accepts all of them, and `.as_ref()` happens once at the top of the function. This is the pattern `std::fs::File::open` uses. `AsRef` makes no promise about hashing or equality, so a type is free to implement it for several targets, or to lend out just one of its fields. `Borrow` can't do either of those.

Case 3: Break the `Borrow` contract and the map loses keys

HTTP header names are case-insensitive. A natural first attempt is a newtype with case-insensitive `Eq` and `Hash`, plus `Borrow<str>` so that lookups by `&str` "just work":

```
// src/bin/broken_borrow.rs - a case-insensitive key that implements Borrow<str> (wrongly)
use std::borrow::Borrow;
use std::collections::HashMap;
use std::hash::{Hash, Hasher};

#[derive(Debug, Clone)]
struct HeaderName(String);

impl PartialEq for HeaderName {
    fn eq(&self, other: &Self) -> bool {
        self.0.eq_ignore_ascii_case(&other.0)
    }
```

```
}
impl Eq for HeaderName {}

impl Hash for HeaderName {
    fn hash<H: Hasher>(&self, state: &mut H) {
        for b in self.0.bytes() {
            state.write_u8(b.to_ascii_lowercase()); // hash the lowercase form
        }
        state.write_u8(0xff);
    }
}

// Compiles fine. Breaks the contract: str's Hash/Eq are case-SENSITIVE and hash differently
impl Borrow<str> for HeaderName {
    fn borrow(&self) -> &str {
        &self.0
    }
}

fn main() {
    let mut h: HashMap<HeaderName, &str> = HashMap::new();
    h.insert(HeaderName("Content-Type".into()), "text/html");

    println!("get(&HeaderName(\"content-type\")) -> {:?}", h.get(&HeaderName("content-type".
    println!("get(\"content-type\")              -> {:?}", h.get("content-type"));
    println!("get(\"Content-Type\")              -> {:?}", h.get("Content-Type"));
    println!("contains_key(\"Content-Type\")     -> {}", h.contains_key("Content-Type"));

    // Why tests often miss it: an all-lowercase key hashes the same both ways.
    h.insert(HeaderName("accept".into()), "*/*");
    println!("get(\"accept\")                    -> {:?}", h.get("accept"));
}
```

```
$ cargo run -q --bin broken_borrow
get(&HeaderName("content-type")) -> Some("text/html")
get("content-type")              -> None
get("Content-Type")              -> None
contains_key("Content-Type")     -> false
get("accept")                    -> Some("*/*")
```

It compiles without a warning, and it is wrong in a way that is easy to miss:

- Lookups by `&HeaderName` work, in any case.
- Lookups by `&str` go through `str`'s `Hash`, which is case-*sensitive*. `"content-type"` and even the exact `"Content-Type"` hash into a different bucket from the stored key, so the lookup returns `None`.
- For an all-lowercase key, both hashes see the same bytes. std's `str` hash writes the bytes

followed by `0xff`, and so does this `HeaderName` impl. (That is an implementation detail of the default `Hasher::write_str` and std's SipHasher, not a guarantee.) So `get("accept")` succeeds. A test suite that only uses lowercase names passes.

The std docs are explicit that this is a logic error, not undefined behaviour. `HashMap` stays memory-safe, but its results stop meaning anything. That is why `Borrow` is a safe trait, and also why the compiler can't help you here.

Case 4: The correct shape: borrow as a type with the same `Hash` and `Eq`

The fix is to *borrow as* a type whose `Hash` and `Eq` really are the key's. Here that is an unsized, case-insensitive view of `str`, built the same way std builds `Path` over `OsStr`:

```
// src/bin/uncased.rs - the correct shape: borrow as a str-like type with the SAME Hash/Eq
use std::borrow::Borrow;
use std::collections::HashMap;
use std::hash::{Hash, Hasher};

/// A borrowed, case-insensitive view of a `str` (an unsized type, like `Path` over `OsStr`)
#[repr(transparent)]
#[derive(Debug)]
struct UncasedStr(str);

impl UncasedStr {
    fn new(s: &str) -> &UncasedStr {
        // SAFETY: UncasedStr is #[repr(transparent)] over str, so &str and &UncasedStr have
        // the same layout and metadata; this is the pattern std uses for Path::new.
        unsafe { &*(s as *const str as *const UncasedStr) }
    }
}

impl PartialEq for UncasedStr {
    fn eq(&self, other: &Self) -> bool {
        self.0.eq_ignore_ascii_case(&other.0)
    }
}
impl Eq for UncasedStr {}

impl Hash for UncasedStr {
    fn hash<H: Hasher>(&self, state: &mut H) {
        for b in self.0.bytes() {
            state.write_u8(b.to_ascii_lowercase());
        }
        state.write_u8(0xff);
    }
}
```

```
/// The owned key. Its Hash/Eq simply delegate to the borrowed form, so they agree by constr
#[derive(Debug, Clone)]
struct HeaderName(String);

impl Borrow<UncasedStr> for HeaderName {
    fn borrow(&self) -> &UncasedStr {
        UncasedStr::new(&self.0)
    }
}
impl PartialEq for HeaderName {
    fn eq(&self, other: &Self) -> bool {
        Borrow::<UncasedStr>::borrow(self) == Borrow::<UncasedStr>::borrow(other)
    }
}
impl Eq for HeaderName {}
impl Hash for HeaderName {
    fn hash<H: Hasher>(&self, state: &mut H) {
        Borrow::<UncasedStr>::borrow(self).hash(state)
    }
}

fn main() {
    let mut h: HashMap<HeaderName, &str> = HashMap::new();
    h.insert(HeaderName("Content-Type".into()), "text/html");

    for probe in ["content-type", "CONTENT-TYPE", "Content-Type", "Content-Length"] {
        println!("get(UncasedStr::new({probe:?})) -> {:?}", h.get(UncasedStr::new(probe)));
    }
}
```

```
$ cargo run -q --bin uncased
get(UncasedStr::new("content-type")) -> Some("text/html")
get(UncasedStr::new("CONTENT-TYPE")) -> Some("text/html")
get(UncasedStr::new("Content-Type")) -> Some("text/html")
get(UncasedStr::new("Content-Length")) -> None
```

Now the owned key's `Hash` and `Eq` *delegate* to the borrowed form, so they agree by construction, and every case of the name finds the entry. The single `unsafe` line is the standard `#[repr(transparent)]` reference cast that `Path::new` and `OsStr::new` use. Miri was not run on it here, because there is no nightly toolchain on the box. If you'd rather avoid the cast, lowercase the key on insert and look it up with a lowercased `&str`. That costs an allocation, or a check, per lookup.

Case 5: `Borrow` is reflexive, `AsRef` is not

```
// src/bin/reflexive.rs - Borrow<T> holds for T itself; AsRef<T> does not
use std::borrow::Borrow;

#[derive(Debug)]
struct Config {
    name: String,
}

fn by_borrow<C: Borrow<Config>>(c: C) -> usize {
    c.borrow().name.len()
}

fn by_asref<C: AsRef<Config>>(c: C) -> usize {
    c.as_ref().name.len()
}

fn main() {
    let cfg = Config { name: "prod".into() };
    println!("by_borrow(&cfg) = {}", by_borrow(&cfg)); // impl Borrow<T> for &T
    println!("by_borrow(cfg)  = {}", by_borrow(cfg)); //  impl Borrow<T> for T
    let cfg = Config { name: "dev".into() };
    println!("by_asref(&cfg)  = {}", by_asref(&cfg));
}
```

```
$ cargo build --bin reflexive
   Compiling conv v0.1.0 (/tmp/borrow/conv)
error[E0277]: the trait bound `Config: AsRef<Config>` is not satisfied
  --> src/bin/reflexive.rs:22:48
   |
22 |     println!("by_asref(&cfg)  = {}", by_asref(&cfg));
   |                                       --------  ^^^ the trait `AsRef<Config>` is not imp
   |                                       |
   |                                       required by a bound introduced by this call
   |
   = note: required for `&Config` to implement `AsRef<Config>`
note: required by a bound in `by_asref`
  --> src/bin/reflexive.rs:13:16
   |
13 | fn by_asref<C: AsRef<Config>>(c: C) -> usize {
   |                ^^^^^^^^^^^^^ required by this bound in `by_asref`

For more information about this error, try `rustc --explain E0277`.
error: could not compile `conv` (bin "reflexive") due to 1 previous error
```

`by_borrow` accepts both `cfg` and `&cfg`. The blanket impls `impl<T> Borrow<T> for T` and `impl<T>`

`Borrow<T> for &T` cover both. `by_asref(&cfg)` fails: `&T: AsRef<U>` only forwards when `T: AsRef<U>`, and nothing makes `Config: AsRef<Config>`. The `AsRef` docs explain that a reflexive blanket impl "is currently not provided due to technical restrictions of Rust's type system (it would be overlapping with another existing blanket implementation for `&T…`)". If you want generic code that accepts "a `T` or a `&T`", write `C: Borrow<T>`. Use `AsRef` for "anything that can lend me a `&T`".

Case 6: `ToOwned` is `Borrow` run backwards, and `Cow` stands on both

```
// src/bin/cow.rs - ToOwned is Borrow run backwards, and Cow is built on both
use std::borrow::Cow;

/// Normalise a header value: trim it, and collapse internal runs of spaces.
/// Allocates only when the input actually needs changing.
fn normalize(v: &str) -> Cow<'_, str> {
    let t = v.trim();
    if !t.contains("  ") {
        return Cow::Borrowed(t); // zero-copy: a slice of the caller's string
    }
    let mut out = String::with_capacity(t.len());
    for word in t.split(' ').filter(|w| !w.is_empty()) {
        if !out.is_empty() {
            out.push(' ');
        }
        out.push_str(word);
    }
    Cow::Owned(out)
}

fn main() {
    for input in ["text/html", "  text/html ", "text/html;  charset=utf-8"] {
        let n = normalize(input);
        let kind = match &n {
            Cow::Borrowed(_) => "borrowed",
            Cow::Owned(_) => "owned (allocated)",
        };
        println!("{input:<28} -> {n:<26} {kind}");
    }

    // <str as ToOwned>::Owned = String, and String: Borrow<str>: the pair Cow<str> relies o
    let owned: String = "abc".to_owned();
    let back: &str = std::borrow::Borrow::borrow(&owned);
    println!("to_owned -> {owned:?}, borrow -> {back:?}");

    // into_owned() allocates only if it still has to.
    let c: Cow<'_, str> = normalize("plain");
```

```
    let s: String = c.into_owned();
    println!("into_owned -> {s:?}");
}
```

```
$ cargo run -q --bin cow
text/html                    -> text/html                    borrowed
  text/html                  -> text/html                    borrowed
text/html;  charset=utf-8    -> text/html; charset=utf-8     owned (allocated)
to_owned -> "abc", borrow -> "abc"
into_owned -> "plain"
```

`Cow<'a, B>` requires `B: ToOwned`, and `ToOwned` requires `Self::Owned: Borrow<Self>`. That pair lets a `Cow<str>` hold either a `&str` or a `String` and deref to `&str` either way. `normalize` allocates only for the input that needs rewriting. The other two return a slice of the caller's string.

The trap

The trap is treating `Borrow` as "`AsRef` with a better name for maps", and implementing it whenever you want `get(&str)` to compile. Case 3 compiles and passes a lowercase test, and in production it returns `None` for `"Content-Type"`. The same class of bug appears whenever `Borrow<T>` lends a form whose `Hash`, `Eq` or `Ord` differ from the owner's:

- a struct that borrows as one of its fields (`User: Borrow<u64>` returning `&self.id`, while `User`'s `Eq` compares every field);
- a float wrapper with a total order that borrows as `f64` (whose `PartialOrd` disagrees for NaN);
- a path-like key that normalises on `Eq` but borrows as the raw string.

The related `Deref` trap is newtype "inheritance": `struct Email(String)` with `Deref<Target = String>` so that `email.len()` works. Every `String` method becomes part of the `Email` API, `email.to_uppercase()` silently returns a plain `String`, and the std docs' conditions ("transparently behaves like a value of the target type", "will not be surprised by any deref coercion behavior") no longer hold. Implement `AsRef<str>` and a few explicit methods instead.

The boring rule

> Use `AsRef<T>` for generic parameters that just need to read a `&T` (paths, bytes, strings). Implement `Borrow<T>` only when `T`'s `Hash`, `Eq` and `Ord` agree exactly with `Self`'s, and get there by delegating `Self`'s impls *to* the borrowed form. If they can't agree (case-insensitive, normalised or partial keys), borrow as a dedicated `#[repr(transparent)]` view type, or normalise the key before you store it. Implement `Deref` only for smart-pointer-like types, never to inherit methods. Test map lookups with keys in more than one form.

Try this

1. Add `impl AsRef<Config> for Config { fn as_ref(&self) -> &Config { self } }` to `reflexive.rs`. Does `by_asref(&cfg)` compile now? Does `by_asref(cfg)`?
2. In `broken_borrow.rs`, replace `HashMap` with `BTreeMap`, give `HeaderName` a case-insensitive `Ord`, and keep `Borrow<str>`. Which lookups fail now, and why is the failure different from the hash-based one?
3. Write `fn total_len<I, S>(items: I) -> usize where I: IntoIterator<Item = S>, S: AsRef<str>` and call it with a `Vec<String>`, a `[&str; 3]`, and a `HashSet<Box<str>>`.
4. Change `normalize` in `cow.rs` to lowercase the value too. How many inputs are still `Borrowed`? Use `str::bytes().any(|b| b.is_ascii_uppercase())` so that it only allocates when needed.
5. Find `impl Borrow<str> for String` and `impl Hash for String` in the std source. Confirm that `String` hashes exactly as its `str` does, and explain why that equality is the whole contract.

Sources

- Rust standard library, `std::borrow::Borrow` (the `Hash`/`Eq`/`Ord` equivalence requirement and the `HashMap` example): https://doc.rust-lang.org/std/borrow/trait.Borrow.html
- Rust standard library, `std::convert::AsRef` ("Relation to `Borrow`", "Reflexivity"): https://doc.rust-lang.org/std/convert/trait.AsRef.html
- Rust standard library, `std::ops::Deref` ("When to implement `Deref` or `DerefMut`"): https://doc.rust-lang.org/std/ops/trait.Deref.html
- Rust standard library, `HashMap::get` and `BTreeMap::range` signatures: https://doc.rust-lang.org/std/collections/struct.HashMap.html#method.get, https://doc.rust-lang.org/std/collections/struct.BTreeMap.html#method.range
- Rust standard library, `std::borrow::ToOwned` and `Cow`: https://doc.rust-lang.org/std/borrow/trait.ToOwned.html, https://doc.rust-lang.org/std/borrow/enum.Cow.html
- Rust standard library, `Path::new` (the `#[repr(transparent)]` unsized-view pattern): https://doc.rust-lang.org/std/path/struct.Path.html#method.new
- Rust source, `core::hash` (`Hasher::write_str` writes the bytes then `0xff`): https://github.com/rust-lang/rust/blob/master/library/core/src/hash/mod.rs
- The Rust Reference, type layout: `#[repr(transparent)]`: https://doc.rust-lang.org/reference/type-layout.html#the-transparent-representation
- Rust API Guidelines, C-DEREF ("Only smart pointers implement `Deref`"): https://rust-lang.github.io/api-guidelines/predictability.html#c-deref
- Rust release channel manifest (stable 1.99.1, 2026-10-13): https://static.rust-lang.org/dist/channel-rust-stable.toml

Cow — Borrow When You Can, Own When You Must

Cow — Borrow When You Can, Own When You Must

Thesis

`Cow<'_, str>` (clone-on-write) returns a borrowed slice when no change is needed and an owned `String` when you must mutate or allocate. The win appears when the common path is already normalized. This chapter compares always-`to_string()` against `Cow` with `black_box` so LLVM cannot erase the work.

Mental model

```
    input already good → Cow::Borrowed(&str)   // no alloc
    input needs change → Cow::Owned(String)    // alloc once
```

Worked examples

Verified with **rustc 1.99.1** `--release`.

Output (`200_000` iterations, `black_box` on input and result):

```
   Compiling cowdemo v0.1.0 (/home/box/sand-data/books-scout/runs/2026-10-27-oneshot/labs/rust-
     Running `target/release/cowdemo`
owned always, already-lower x200000: 6.124085ms sink=0
cow borrow, already-lower x200000: 3.849675ms sink=0
cow owned, mixed-case x200000: 3.966889ms sink=0
```

On this host the borrow path (~3.85 ms) beat always-owned (~6.12 ms) for already-lowercase input. Mixed-case `Cow::Owned` stayed in the same band as borrow here because the `chars().any` scan dominates the small string alloc.

Trap

Wrapping everything in `Cow` by habit. If you almost always allocate, a plain `String` API is clearer. `Cow` pays for the enum branch and lifetime plumbing.

Boring rule

Use `Cow` at API boundaries where callers often pass already-valid data (paths, headers, normalized IDs). Measure with `black_box` before celebrating.

Try this

1. Return `Cow<'_, [u8]>` from a decoder that sometimes copies.
2. Compare `String::from` vs `Cow` in a serializer hot path.
3. Read `std::borrow::Cow` examples in the docs.

Sources

- std::borrow::Cow

PhantomData and the Type-State Pattern: Protocol States the Compiler Enforces

`PhantomData` and the Type-State Pattern: Protocol States the Compiler Enforces

Thesis

A lot of APIs have an order to them: connect, then log in, then query. Most libraries check that order at runtime, with a `state` field and an `Err(NotConnected)` when it's wrong. Rust can check it at compile time instead. Make the state a **type parameter**, give each state its own `impl` block, and have every transition take `self` by value and return the next state. Calling a method in the wrong state is then a type error. Reusing an old state is a use-after-move error.

The part that makes this work is `std::marker::PhantomData<S>`. It is a zero-sized field that "uses" the type parameter without storing anything. This chapter builds a small type-state connection, shows the compiler errors it produces, measures what it costs at runtime (nothing), and covers the one decision you have to get right: *which* `PhantomData`.

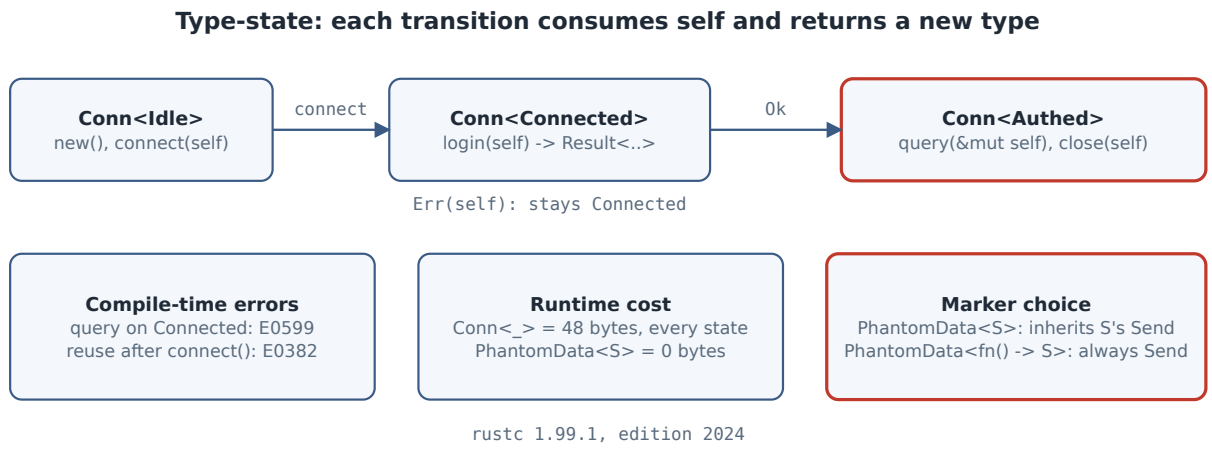


Figure 1: Type-state transitions for Conn, the compile errors they produce, their size, and the marker choice

All listings were compiled with `rustc 1.99.1`, edition 2024.

Why you need `PhantomData` at all

Rust rejects a type parameter that no field mentions:

```
struct Conn<S> { host: String }
```

```
error[E0392]: type parameter `S` is never used
 --> bad3.rs:1:13
  |
1 | struct Conn<S> { host: String }
  |             ^ unused type parameter
  |
  = help: consider removing `S`, referring to it in a field, or using a marker such as `Phan
  = help: if you intended `S` to be a const parameter, use `const S: /* Type */` instead

error: aborting due to 1 previous error

For more information about this error, try `rustc --explain E0392`.
```

The error is deliberate. The compiler needs to know how `Conn<S>` relates to `S` for **variance**, **drop checking** and **auto traits** (`Send`/`Sync`). `PhantomData<T>` answers that question by saying "act as if this struct holds a `T`" (std docs), while taking zero bytes.

The type-state connection

The states are empty structs that are never constructed. They exist only as type arguments.

```rust
use std::marker::PhantomData;
use std::mem::size_of;

// States are zero-sized marker types. They never exist at runtime.
pub struct Idle;
pub struct Connected;
pub struct Authed;

pub struct Conn<S> {
    host: String,
    log: Vec<String>,
    _state: PhantomData<S>,
}

impl Conn<Idle> {
    pub fn new(host: &str) -> Self {
        Conn { host: host.to_string(), log: vec![], _state: PhantomData }
    }
    pub fn connect(mut self) -> Conn<Connected> {
        self.log.push(format!("connect {}", self.host));
        Conn { host: self.host, log: self.log, _state: PhantomData }
    }
}
```

```
impl Conn<Connected> {
    pub fn login(mut self, user: &str) -> Result<Conn<Authed>, Conn<Connected>> {
        if user.is_empty() {
            return Err(self); // stay in Connected, caller keeps the value
        }
        self.log.push(format!("login {user}"));
        Ok(Conn { host: self.host, log: self.log, _state: PhantomData })
    }
}

impl Conn<Authed> {
    pub fn query(&mut self, q: &str) -> usize {
        self.log.push(format!("query {q}"));
        q.len()
    }
    pub fn close(self) -> Vec<String> {
        self.log
    }
}

// Methods every state shares go on the generic impl.
impl<S> Conn<S> {
    pub fn host(&self) -> &str { &self.host }
}
```

Three design choices carry the pattern:

- **Transitions take `self` by value.** `connect(self) -> Conn<Connected>` *consumes* the idle connection, so the old state can't be used again.
- **Fallible transitions return the old state on failure.** `login` returns `Result<Conn<Authed>, Conn<Connected>>`. A failed login hands back the still-connected value instead of dropping it.
- **Shared behaviour goes on `impl<S> Conn<S>`.** `host()` works in every state, so it isn't copied three times.

The driver:

```
fn main() {
    let c = Conn::new("db.internal");
    println!("size_of::<Conn<Idle>>()   = {}", size_of::<Conn<Idle>>());
    println!("size_of::<Conn<Authed>>() = {}", size_of::<Conn<Authed>>());
    println!("size_of::<PhantomData<Authed>>() = {}", size_of::<PhantomData<Authed>>());
    let c = c.connect();
    let c = match c.login("") {
        Ok(_) => unreachable!(),
        Err(still_connected) => { println!("empty user rejected, still on {}", still_connect
    };
```

```
    let mut c = c.login("kanthi").ok().expect("login");
    let n = c.query("SELECT 1");
    println!("query returned {n}");
    for line in c.close() { println!("log: {line}"); }
}
```

```
size_of::<Conn<Idle>>()   = 48
size_of::<Conn<Authed>>() = 48
size_of::<PhantomData<Authed>>() = 0
empty user rejected, still on db.internal
query returned 8
log: connect db.internal
log: login kanthi
log: query SELECT 1
```

`Conn<Idle>` and `Conn<Authed>` are both 48 bytes (a `String` and a `Vec`, 24 bytes each on x86-64), and `PhantomData<Authed>` is 0. The state costs nothing at runtime: there's no tag byte and no branch. Each `impl` block is monomorphised separately, so `query` only exists for `Conn<Authed>`.

What the compiler catches

Querying before login:

```
let c = Conn::new("db.internal").connect();
c.query("SELECT 1"); // not logged in yet
```

```
error[E0599]: no method named `query` found for struct `Conn<Connected>` in the current scop
  --> bad1.rs:67:7
   |
 9 | pub struct Conn<S> {
   | ------------------ method `query` not found for this struct
...
67 |     c.query("SELECT 1"); // not logged in yet
   |       ^^^^^ method not found in `Conn<Connected>`
   |
   = note: the method was found for `Conn<Authed>`

error: aborting due to 1 previous error

For more information about this error, try `rustc --explain E0599`.
```

The note `the method was found for Conn<Authed>` points straight at the missing transition.

Reusing a state after it has been consumed:

```
let c = Conn::new("db.internal");
let a = c.connect();
let b = c.connect(); // reuse the Idle value
```

```
error[E0382]: use of moved value: `c`
  --> bad2.rs:68:13
   |
66 |     let c = Conn::new("db.internal");
   |         - move occurs because `c` has type `Conn<Idle>`, which does not implement the `
67 |     let a = c.connect();
   |             --------- `c` moved due to this method call
68 |     let b = c.connect(); // reuse the Idle value
   |             ^ value used here after move
   |
note: `Conn::<Idle>::connect` takes ownership of the receiver `self`, which moves `c`
  --> bad2.rs:19:24
   |
19 |     pub fn connect(mut self) -> Conn<Connected> {
   |                        ^^^^

error: aborting due to 1 previous error

For more information about this error, try `rustc --explain E0382`.
```

A runtime state machine would catch neither mistake until the code ran, if at all.

Choosing the right `PhantomData`

`PhantomData<S>` says "I own an `S`", so `Conn<S>` takes on `S`'s auto traits. That matters if a state marker happens to be `!Send`, even though no `S` value ever exists:

```
use std::marker::PhantomData;
use std::rc::Rc;

// A state marker that happens to be !Send (it is never constructed).
#[allow(dead_code)]
struct Local(Rc<()>);

struct Owns<S>(PhantomData<S>);          // "acts like it owns an S"
struct Tag<S>(PhantomData<fn() -> S>);   // "only names S": Send + Sync, covariant

fn assert_send<T: Send>() {}

fn main() {
    assert_send::<Tag<Local>>();
```

```
    assert_send::<Owns<Local>>();
}
```

```
error[E0277]: `Rc<()>` cannot be sent between threads safely
  --> auto.rs:15:19
   |
15 |     assert_send::<Owns<Local>>();
   |                   ^^^^^^^^^^^ `Rc<()>` cannot be sent between threads safely
   |
   = help: within `Owns<Local>`, the trait `Send` is not implemented for `Rc<()>`
note: required because it appears within the type `Local`
  --> auto.rs:6:8
   |
 6 | struct Local(Rc<()>);
   |        ^^^^^
note: required because it appears within the type `PhantomData<Local>`
  --> /rustc/58cf8f9c35b355199727627d07ae34a4e2846d67/library/core/src/marker.rs:811:11
note: required because it appears within the type `Owns<Local>`
  --> auto.rs:8:8
   |
 8 | struct Owns<S>(PhantomData<S>);          // "acts like it owns an S"
   |        ^^^^
note: required by a bound in `assert_send`
  --> auto.rs:11:19
   |
11 | fn assert_send<T: Send>() {}
   |                   ^^^^ required by this bound in `assert_send`

error: aborting due to 1 previous error

For more information about this error, try `rustc --explain E0277`.
```

`Tag<Local>` compiles and `Owns<Local>` does not. The Rustonomicon's PhantomData table lists the options:

Marker	Variance in `T`	`Send`/`Sync`	`T` may dangle in drop glue?
`PhantomData<T>`	covariant	inherited from `T`	no ("owns `T`")
`PhantomData<&'a T>`	covariant	`Send + Sync` require `T: Sync`	yes
`PhantomData<*const T>`	covariant	`!Send + !Sync`	yes
`PhantomData<fn() -> T>`	covariant	`Send + Sync`	yes
`PhantomData<fn(T)>`	**contra**variant	`Send + Sync`	yes

`PhantomData<fn(T) -> T>`	**in**variant	`Send + Sync`	yes

For type-state markers, where `S` is only a label, `PhantomData<fn() -> S>` is usually the most accurate choice. The struct never owns an `S`, so it shouldn't inherit `S`'s thread-safety or drop behaviour. Use plain `PhantomData<S>` when the struct really does own `S` values in a way the compiler can't see, such as a raw-pointer collection, because that is what keeps drop checking sound.

When not to use type-state

- **The state comes from runtime data.** If the server decides whether you're authenticated, you still need `Result` at the boundary, as `login` shows. You can't hold a `Vec` of connections in different states without an enum or a trait object, so mixed collections go back to runtime state.
- **There are many states and many shared methods.** Every state you add means another `impl` block and more generic signatures. More than four or five states is usually a sign you want an enum.
- **The API is public and stable.** Each state type becomes part of your public API, and renaming one is a breaking change.

Takeaways

- A type parameter plus `PhantomData` turns protocol order into type checking, with zero runtime cost (48 bytes in every state, measured).
- Transitions consume `self`. A failed transition returns the old state in `Err`.
- Out-of-order calls become E0599 and reuse becomes E0382, both at compile time.
- Pick the marker on purpose: `PhantomData<fn() -> S>` for pure labels, `PhantomData<S>` only when you logically own an `S`.

Sources

- Rust standard library, `std::marker::PhantomData`
- The Rustonomicon, PhantomData (variance, auto-trait and drop-check table)
- Rust compiler error index, E0392, E0599, E0382
- The Rust Reference, Subtyping and Variance

The Newtype Pattern: Zero-Cost Type Safety, Validated Values and the Orphan Rule

The Newtype Pattern: Zero-Cost Type Safety, Validated Values and the Orphan Rule

Thesis

A **newtype** is a tuple struct with one field: `struct AccountId(u64);`. It costs nothing at runtime, because it has the same size and alignment as what it wraps, but it is a *different type* to the compiler. That one property gives you three tools:

1. Values with the same representation can no longer be mixed up. Account IDs, amounts, metres and seconds all stop being "just a `u64`".
2. A private field plus a validating constructor means that holding the type is *proof* the value was checked.
3. A local wrapper lets you implement a foreign trait for a foreign type, which the orphan rule otherwise forbids.

This chapter demonstrates each one with compiler output from rustc 1.99.1, and shows where `#[repr(transparent)]` becomes necessary rather than optional.

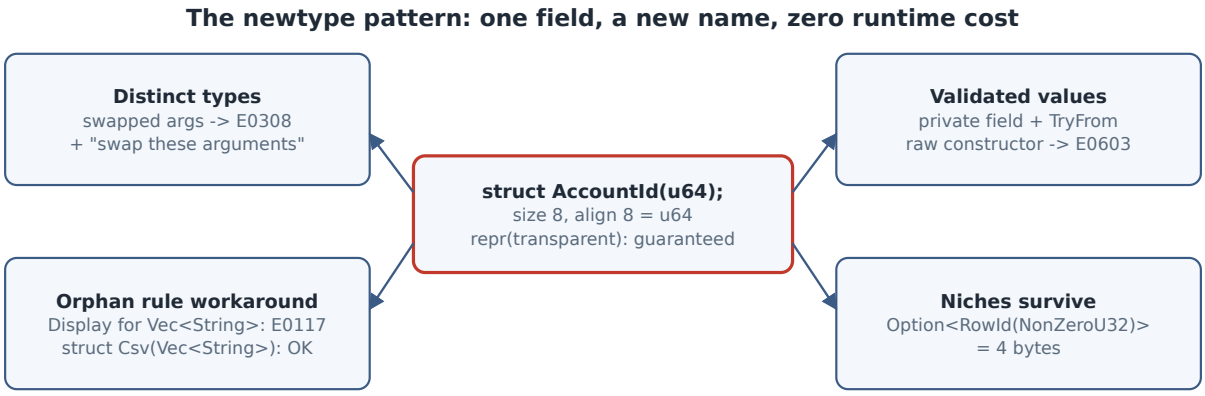


Figure 1: The newtype pattern: one field, a new name, zero runtime cost

All listings come from one Cargo package (edition 2024). Each one is a binary under `src/bin/`. The `*_bug.rs` files fail to compile on purpose.

1. The swapped-argument bug

```
// Two IDs with the same representation: the compiler cannot tell them apart.
fn transfer(from_account: u64, to_account: u64, cents: u64) {
    println!("move {cents} cents from {from_account} to {to_account}");
}

#[derive(Clone, Copy, Debug, PartialEq, Eq, Hash)]
struct AccountId(u64);
#[derive(Clone, Copy, Debug, PartialEq, Eq, Hash)]
struct Cents(u64);

fn transfer_typed(from: AccountId, to: AccountId, amount: Cents) {
    println!("move {} cents from {} to {}", amount.0, from.0, to.0);
}

fn main() {
    let (alice, bob) = (1001_u64, 2002_u64);
    transfer(alice, 500, bob); // compiles; bob's id is now the amount

    let (alice, bob) = (AccountId(1001), AccountId(2002));
    transfer_typed(alice, bob, Cents(500));
}
```

```
$ cargo run -q --bin swapped
move 2002 cents from 1001 to 500
move 500 cents from 1001 to 2002
```

The first call moves **2002 cents to account 500**. Every argument is a `u64`, so nothing could catch the mistake. Here is the same mistake with newtypes:

```
#[derive(Clone, Copy, Debug)]
struct AccountId(u64);
#[derive(Clone, Copy, Debug)]
struct Cents(u64);

fn transfer_typed(from: AccountId, to: AccountId, amount: Cents) {
    println!("{from:?} {to:?} {amount:?}");
}

fn main() {
    let (alice, bob) = (AccountId(1001), AccountId(2002));
    transfer_typed(alice, Cents(500), bob);
}
```

```
$ cargo build -q --bin swapped_typed_bug
error[E0308]: arguments to this function are incorrect
  --> src/bin/swapped_typed_bug.rs:12:5
   |
12 |     transfer_typed(alice, Cents(500), bob);
   |     ^^^^^^^^^^^^^^^       ----------  --- expected `Cents`, found `AccountId`
   |                           |
   |                           expected `AccountId`, found `Cents`
   |
note: function defined here
  --> src/bin/swapped_typed_bug.rs:6:4
   |
 6 | fn transfer_typed(from: AccountId, to: AccountId, amount: Cents) {
   |    ^^^^^^^^^^^^^^
help: swap these arguments
   |
12 -     transfer_typed(alice, Cents(500), bob);
12 +     transfer_typed(alice, bob, Cents(500));
   |
```

The compiler rejects the call and even suggests the fix. Deriving `Clone, Copy, Debug, PartialEq, Eq, Hash` keeps IDs as convenient as plain integers in hash maps and comparisons. Leave out `Add` and the other arithmetic traits unless the operation makes sense: adding two account IDs never does. The Rust API Guidelines list this as C-NEWTYPE: “Newtypes provide static distinctions”.

2. Zero cost, and what `repr(transparent)` adds

```
use std::mem::{align_of, size_of};

#[allow(dead_code)]
struct Meters(f64);
#[allow(dead_code)]
#[repr(transparent)]
struct UserId(u32);
#[allow(dead_code)]
struct RowId(std::num::NonZeroU32);

fn main() {
    println!("f64: size {} align {}", size_of::<f64>(), align_of::<f64>());
    println!("Meters: size {} align {}", size_of::<Meters>(), align_of::<Meters>());
    println!("Option<u32>: size {}", size_of::<Option<u32>>());
    println!("Option<UserId>: size {}", size_of::<Option<UserId>>());
    println!("Option<std::num::NonZeroU32>: size {}", size_of::<Option<std::num::NonZeroU32>
    println!("Option<RowId>: size {}", size_of::<Option<RowId>>());
```

```
    println!("[UserId; 1000]: size {}", size_of::<[UserId; 1000]>());
}
```

```
$ cargo run -q --bin zero_cost
f64: size 8 align 8
Meters: size 8 align 8
Option<u32>: size 8
Option<UserId>: size 8
Option<std::num::NonZeroU32>: size 4
Option<RowId>: size 4
[UserId; 1000]: size 4000
```

- `Meters` has the same size and alignment as `f64`, and an array of 1000 `UserId`s is exactly 4000 bytes. There is no header and no tag.
- **Niches survive the wrapper.** `Option<RowId>` is 4 bytes because `RowId` wraps `NonZeroU32`, and the compiler reuses the forbidden zero as `None`. `Option<UserId>` over a plain `u32` has no niche, so it needs the extra discriminant and is 8 bytes.

What you measure for a default-representation (`repr(Rust)`) struct is not a *promise*. The Reference's type-layout chapter gives `repr(Rust)` very few guarantees. `#[repr(transparent)]` is the attribute that guarantees "the same layout and ABI as the only non-size 0 non-alignment 1 field". You need it whenever `unsafe` code or FFI depends on the layout:

```
// #[repr(transparent)] guarantees UserId has the same layout and ABI as u32,
// so a &[u32] from storage or FFI can be viewed as &[UserId] without copying.
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
#[repr(transparent)]
struct UserId(u32);

fn as_user_ids(raw: &[u32]) -> &[UserId] {
    // SAFETY: UserId is repr(transparent) over u32, so size, alignment and
    // validity are identical; the lifetime is carried over from `raw`.
    unsafe { std::slice::from_raw_parts(raw.as_ptr().cast::<UserId>(), raw.len()) }
}

fn main() {
    let raw = vec![7_u32, 42, 99];
    let ids = as_user_ids(&raw);
    println!("{ids:?}");
    println!("same address: {}", raw.as_ptr() as usize == ids.as_ptr() as usize);
}
```

```
$ cargo run -q --bin transparent
[UserId(7), UserId(42), UserId(99)]
same address: true
```

Without `#[repr(transparent)]`, this cast would be relying on layout the language doesn't promise.

With it, a `&[u32]` from a file, a database page or a C library becomes a typed `&[UserId]` with no copy (`same address: true`). The Nomicon describes the same rule for passing newtypes across `extern "C"` boundaries.

3. Validated values: parse, don't re-check

```
// A newtype whose only constructor validates: holding an Email *proves* it was checked.
mod email {
    #[derive(Debug, Clone, PartialEq, Eq)]
    pub struct Email(String); // private field: no Email("junk".into()) outside this module

    #[derive(Debug)]
    pub struct InvalidEmail(pub String);

    impl TryFrom<String> for Email {
        type Error = InvalidEmail;
        fn try_from(s: String) -> Result<Self, Self::Error> {
            match s.split_once('@') {
                Some((user, host)) if !user.is_empty() && host.contains('.') => {
                    Ok(Email(s.to_ascii_lowercase()))
                }
                _ => Err(InvalidEmail(s)),
            }
        }
    }

    impl AsRef<str> for Email {
        fn as_ref(&self) -> &str {
            &self.0
        }
    }
}

use email::Email;

fn send_welcome(to: &Email) {
    println!("sending to {}", to.as_ref());
}

fn main() {
    for raw in ["Ada@Example.org", "no-at-sign", "@example.org", "bob@localhost"] {
        match Email::try_from(raw.to_string()) {
            Ok(e) => send_welcome(&e),
            Err(err) => println!("rejected: {:?}", err.0),
        }
```

```
    }
}
```

```
$ cargo run -q --bin validated
sending to ada@example.org
rejected: "no-at-sign"
rejected: "@example.org"
rejected: "bob@localhost"
```

The field is private, so outside `mod email` the **only** way to get an `Email` is `TryFrom`, which validates it and normalises it to lower case. `send_welcome(&Email)` has no reason to check again. Trying to bypass the constructor fails:

```
mod email {
    #[derive(Debug)]
    pub struct Email(String);
}

fn main() {
    let e = email::Email("not an email".to_string());
    println!("{e:?}");
}
```

```
$ cargo build -q --bin validated_bug
error[E0603]: tuple struct constructor `Email` is private
 --> src/bin/validated_bug.rs:7:20
  |
3 |     pub struct Email(String);
  |                      ------ a constructor is private if any of the fields is private
...
7 |     let e = email::Email("not an email".to_string());
  |                   ^^^^^ private tuple struct constructor
  |
note: the tuple struct constructor `Email` is defined here
 --> src/bin/validated_bug.rs:3:5
  |
3 |     pub struct Email(String);
  |     ^^^^^^^^^^^^^^^^^^^^^^^^
help: consider making the field publicly accessible
  |
3 |     pub struct Email(pub String);
  |                      +++
```

The compiler's `help` suggests `pub String`. In this case that is the **wrong** fix, because it would let anyone build an unvalidated `Email`. The privacy is the whole point.

The validation here is deliberately small. Real email validation belongs in a dedicated crate. The

pattern stays the same whatever rule you plug in.

4. Getting around the orphan rule

You can't implement a trait from another crate (`Display`) for a type from another crate (`Vec<String>`):

```
use std::fmt;

impl fmt::Display for Vec<String> {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "{}", self.join(","))
    }
}

fn main() {}
```

```
$ cargo build -q --bin orphan_bug
error[E0117]: only traits defined in the current crate can be implemented for types defined
 --> src/bin/orphan_bug.rs:3:1
  |
3 | impl fmt::Display for Vec<String> {
  | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  |                       |
  |                       `Vec` is not defined in the current crate
  |
  = note: impl doesn't have any local type before any uncovered type parameters
  = note: for more information see https://doc.rust-lang.org/reference/items/implementations
  = note: define and implement a trait or new type instead
```

The last note gives the fix, "define and implement a trait or new type instead":

```
use std::fmt;

// impl fmt::Display for Vec<String> { ... }  // E0117: only traits defined in
// the current crate can be implemented for types defined outside of the crate

struct Csv(Vec<String>);

impl fmt::Display for Csv {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "{}", self.0.join(","))
    }
}
```

```rust
fn main() {
    let row = Csv(vec!["id".into(), "name".into(), "email".into()]);
    println!("{row}");
}
```

```
$ cargo run -q --bin orphan
id,name,email
```

The Book presents exactly this use. The orphan rules exist so that two crates can never provide conflicting impls of the same trait for the same type.

Pitfalls

- **`Deref` as inheritance.** `impl Deref<Target = String> for Email` makes `email.len()` work, but it also adds every `String` method to the `Email` API, and `email.to_uppercase()` quietly returns a plain `String`. Implement `AsRef<str>` and the few methods you actually mean.
- **`DerefMut` breaks validation.** It lets callers write `*email = "junk".into()` and skip `TryFrom` altogether.
- **`pub` fields** remove validation the same way (section 3).
- **Boilerplate.** Each trait you want, such as `Display`, `FromStr`, `serde`, or arithmetic for units, has to be derived or forwarded. `#[derive]` covers the standard traits, and `#[serde(transparent)]` serialises the newtype as its inner value. Write impls by hand where the semantics differ, as with `Cents * Cents`.

Takeaways

- A single-field tuple struct is a new type at compile time and nothing at runtime: same size and alignment, and niches are preserved.
- Use newtypes for IDs and units so that swapped arguments become `E0308` errors.
- A private field plus `TryFrom` turns "was this validated?" into a type-level fact.
- Add `#[repr(transparent)]` as soon as `unsafe` code or FFI relies on the layout.
- A wrapper is the standard answer to `E0117`.

Sources

- The Rust Book, Using the Newtype Pattern to Implement External Traits and Advanced Types
- Rust API Guidelines, C-NEWTYPE
- The Rust Reference, Type layout: the transparent representation and Orphan rules
- The Rustonomicon, Alternative representations
- std docs, `NonZero` and `Option` representation

Summary

Summary

Chapter Key Takeaways

- Generics monomorphize at compile time for zero runtime overhead.
- Traits declare shared contracts across concrete types.
- Trait bounds enforce capabilities on generic type parameters.
- Trait objects (`dyn Trait`) provide dynamic dispatch using vtables when heterogeneous collections are needed.

Lifetimes: Complete Understanding

What Lifetimes Are (and Are Not)

What Lifetimes Are (and Are Not)

Every reference in Rust has a **lifetime**—the scope for which that reference is valid.

What Lifetimes Are Not

- Lifetimes do **not** change how long a value lives at runtime.
- Lifetimes do **not** allocate or free memory.
- Lifetimes are purely compile-time annotations that help the borrow checker ensure references never outlive the data they point to.

The Lifetime Elision Rules

The Lifetime Elision Rules

In common function patterns, Rust automatically infers lifetime parameters via three elision rules:

1. Each parameter that is a reference gets its own lifetime parameter (`fn foo<'a, 'b>(x: &'a i32, y: &'b i32)`).
2. If there is exactly one input lifetime parameter, that lifetime is assigned to all output lifetime parameters (`fn foo<'a>(x: &'a i32) -> &'a i32`).
3. If there are multiple input lifetime parameters, but one of them is `&self` or `&mut self`, the lifetime of `self` is assigned to all output lifetime parameters.

Annotating Lifetimes in Functions and Structs

Annotating Lifetimes in Functions and Structs

When a function returns a reference that could come from multiple inputs, explicit lifetime annotations are required:

```
// The returned reference is guaranteed valid for the intersection of 'a
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() { x } else { y }
}

struct Excerpt<'a> {
    part: &'a str,
}

fn main() {
    let string1 = String::from("desk configuration");
    let result;
    {
        let string2 = String::from("orders");
        result = longest(string1.as_str(), string2.as_str());
        println!("Longest string: {}", result);
    }
}
```

Lifetime Bounds on Traits

Lifetime Bounds on Traits

Constrain types with lifetime bounds, such as `T: 'a` (meaning `T` must outlive lifetime `'a`):

```
use std::fmt::Display;

fn print_ref<'a, T>(item: &'a T)
where
    T: Display + 'a,
{
    println!("Item: {}", item);
}
```

Advanced Lifetime Patterns

Advanced Lifetime Patterns

The `'static` Lifetime

A reference with `'static` lifetime can live for the entire duration of the program: - String literals: `&'static str` stored in the read-only data segment. - Owned values promoted to static duration.

Variance: Why `&mut` Makes Lifetimes Rigid

Variance: Why `&mut` Makes Lifetimes Rigid

Thesis

Most lifetime errors that look like "the borrow checker is being stubborn" are really about **variance**. Variance is the rule that decides when a type with a longer lifetime can be used where a shorter one is expected. A `&'static str` can be passed anywhere a `&'a str` is wanted, but a `&mut &'static str` can't be passed where a `&mut &'a str` is wanted. The reason isn't arbitrary: if it were allowed, safe code could put a dangling reference into a `'static` slot.

This chapter shows the three kinds of variance in compiler output, explains why each one has to be that way, and then works through the most common real bug variance causes: the `&'a mut self` signature that locks a struct for the rest of its life.

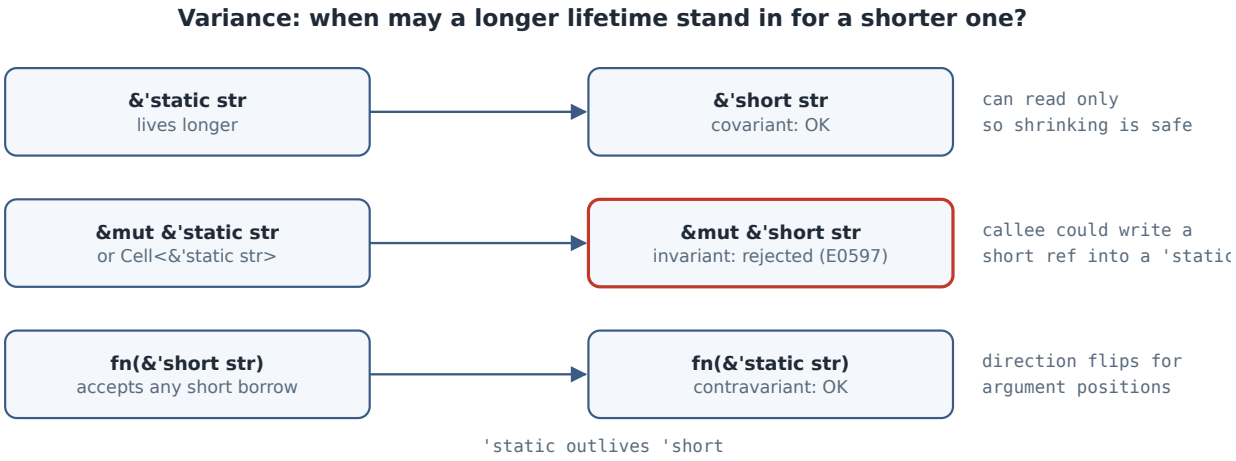


Figure 1: Covariant, invariant and contravariant positions, with 'static outliving 'short

All listings were compiled with `rustc 1.99.1`, edition 2024. Every error message shown is the compiler's actual output.

Subtyping, but only for lifetimes

Rust has no struct inheritance, so the only subtyping is between lifetimes. If `'long` outlives `'short` (written `'long: 'short`), then `&'long T` is a subtype of `&'short T`, and a value of the first type can be used where the second is expected (Reference: Subtyping and Variance).

Variance describes how that relationship carries through a type constructor `F<T>`:

- **Covariant:** if `A` is a subtype of `B`, then `F<A>` is a subtype of `F<B>`. The relationship carries through unchanged.
- **Invariant:** `F<A>` and `F<B>` are unrelated unless `A` and `B` are the same type.
- **Contravariant:** the relationship flips, so `F<B>` is a subtype of `F<A>`.

Covariant: shared references shrink freely

```
// &'a T is covariant in 'a: a longer-lived reference can stand in for a shorter one.
fn pick<'a>(x: &'a str, y: &'a str) -> &'a str { if x.len() >= y.len() { x } else { y } }

fn main() {
    let forever: &'static str = "static text";
    let owned = String::from("local");
    let r = pick(forever, &owned); // &'static str shrinks to the local lifetime
    println!("{r}");
}
```

```
$ rustc --edition 2024 cov.rs && ./cov
static text
```

`pick` needs a single `'a` for both arguments. The compiler chooses `'a` as the lifetime of `owned` and shrinks the `&'static str` to fit. That is safe because a shared reference can only be read, and reading something for less time than it lives is never a problem.

Invariant: `&mut T` can't shrink its `T`

Now make the shrinking happen behind a `&mut`:

```
// &'a mut T is invariant in T: this is what keeps the next line from compiling.
fn overwrite<T>(slot: &mut T, value: T) { *slot = value; }

fn main() {
    let mut s: &'static str = "static text";
    {
        let local = String::from("short-lived");
        overwrite(&mut s, &local);
    }
    println!("{s}");
}
```

```
$ rustc --edition 2024 inv_mut.rs && ./inv_mut
error[E0597]: `local` does not live long enough
 --> inv_mut.rs:8:27
```

```
  |
5 |     let mut s: &'static str = "static text";
  |                ------------ type annotation requires that `local` is borrowed for `'stat
6 |     {
7 |         let local = String::from("short-lived");
  |             ----- binding `local` declared here
8 |         overwrite(&mut s, &local);
  |                           ^^^^^^ borrowed value does not live long enough
9 |     }
  |     - `local` dropped here while still borrowed

error: aborting due to 1 previous error

For more information about this error, try `rustc --explain E0597`.
```

`&'a mut T` is covariant in `'a` but **invariant in `T`**. Here `T` is `&'static str`, so `overwrite` needs `value` to have exactly that type, and `&local` would have to be borrowed for `'static`. Suppose `&mut &'static str` could be treated as `&mut &'short str` instead. `overwrite` would store a pointer to `local` in `s`, `local` would be dropped at the closing brace, and `println!` would read freed memory. Invariance is what prevents that.

The same reasoning covers every type that lets you write through a shared handle. `Cell<T>`, `RefCell<T>`, `UnsafeCell<T>` and `*mut T` are all invariant in `T`. A struct's variance is **inferred from its fields**, so one `Cell` field is enough to make the whole struct invariant:

```rust
use std::cell::Cell;

// Variance of a struct is inferred from its fields.
struct Plain<'a>(&'a str);          // covariant in 'a
struct Shared<'a>(Cell<&'a str>);   // invariant in 'a (interior mutability)

fn shorten_plain<'s>(p: Plain<'static>, _scope: &'s str) -> Plain<'s> { p }
fn shorten_shared<'s>(p: Shared<'static>, _scope: &'s str) -> Shared<'s> { p }

fn main() {
    let local = String::from("x");
    let p = shorten_plain(Plain("static"), &local);
    let s = shorten_shared(Shared(Cell::new("static")), &local);
    println!("{} {}", p.0, s.0.get());
}
```

```
$ rustc --edition 2024 structs.rs && ./structs
error: lifetime may not live long enough
 --> structs.rs:8:76
  |
8 | fn shorten_shared<'s>(p: Shared<'static>, _scope: &'s str) -> Shared<'s> { p }
  |                   -- lifetime `'s` defined here                            ^ returning t
```

```
   |
   = note: requirement occurs because of the type `Shared<'_>`, which makes the generic argum
   = note: the struct `Shared<'a>` is invariant over the parameter `'a`
   = help: see <https://doc.rust-lang.org/nomicon/subtyping.html> for more information about

error: aborting due to 1 previous error
```

The compiler names the cause directly ("the struct `Shared<'a>` is invariant over the parameter `'a`") and links to the Rustonomicon. `Plain` has the same shape without the `Cell`, so `shorten_plain` compiles without complaint. When `shorten_shared` and its call are deleted, the program prints `static`.

Contravariant: function arguments flip the direction

A function pointer `fn(T)` is **contravariant** in `T`. A callback that accepts a borrow of *any* lifetime can safely be used where only `'static` borrows will ever be passed. The other direction isn't safe: a callback that requires `'static` might store its argument somewhere global.

```
// fn(T) is contravariant in T: a callback that accepts *any* lifetime
// can be used where one accepting only 'static is expected, not vice versa.
struct Sink<'a> { f: fn(&'a str) }

fn print_it(s: &str) { println!("got {s}"); }

fn widen<'s>(k: Sink<'s>) -> Sink<'static> { k }   // OK: contravariant
fn narrow<'s>(k: Sink<'static>, _x: &'s str) -> Sink<'s> { k } // rejected

fn main() {
    let k: Sink<'static> = widen(Sink { f: print_it });
    (k.f)("hello");
    let local = String::from("y");
    let _ = narrow(k, &local);
}
```

```
$ rustc --edition 2024 contra.rs && ./contra
error: lifetime may not live long enough
 --> contra.rs:8:60
  |
8 | fn narrow<'s>(k: Sink<'static>, _x: &'s str) -> Sink<'s> { k } // rejected
  |           -- lifetime `'s` defined here                      ^ returning this value requir

error: aborting due to 1 previous error
```

`widen` (short to `'static`) is accepted and `narrow` (`'static` to short) is rejected. With `narrow` removed:

```
$ rustc --edition 2024 contra_ok.rs && ./contra_ok
got hello
```

Contravariance is rare in everyday code. In practice it comes from `fn(T)` fields and from `PhantomData<fn(T)>` markers.

The common case: `&'a mut self` on a type with `'a`

Here is the variance bug most Rust programmers hit, usually in a parser, interner or arena:

```
// The "&'a mut Thing<'a>" trap: tying the borrow to the invariant parameter.
struct Interner<'a> { words: Vec<&'a str> }

impl<'a> Interner<'a> {
    fn add(&'a mut self, w: &'a str) { self.words.push(w); }
}

fn main() {
    let text = String::from("alpha beta");
    let mut i = Interner { words: Vec::new() };
    for w in text.split(' ') {
        i.add(w);
    }
    println!("{}", i.words.len());
}
```

```
$ rustc --edition 2024 vec_trap.rs && ./vec_trap
error[E0499]: cannot borrow `i` as mutable more than once at a time
  --> vec_trap.rs:12:9
   |
12 |         i.add(w);
   |         ^
   |         |
   |         `i` was mutably borrowed here in the previous iteration of the loop
   |         first borrow used here, in later iteration of loop

error[E0502]: cannot borrow `i.words` as immutable because it is also borrowed as mutable
  --> vec_trap.rs:14:20
   |
12 |         i.add(w);
   |         - mutable borrow occurs here
13 |     }
14 |     println!("{}", i.words.len());
   |                    ^^^^^^^
   |                    |
```

```
    |                     immutable borrow occurs here
    |                     mutable borrow later used here

error: aborting due to 2 previous errors

Some errors have detailed explanations: E0499, E0502.
For more information about an error, try `rustc --explain E0499`.
```

The signature `fn add(&'a mut self, w: &'a str)` says: "borrow `self` mutably for `'a`, the *same* lifetime the struct holds its data for." `Interner<'a>` contains `Vec<&'a str>`, and `&mut Interner<'a>` is invariant in `Interner<'a>`. The compiler can't shrink `'a` to make the borrow shorter, so the first call to `add` borrows `i` mutably for all of `'a`, which is every remaining use of `i`. The second loop iteration can't borrow it again, and neither can the `println!`.

The fix is to stop tying the borrow of `self` to the data lifetime. The `&mut self` borrow gets its own short, elided lifetime:

```
struct Interner<'a> { words: Vec<&'a str> }

impl<'a> Interner<'a> {
    fn add(&mut self, w: &'a str) { self.words.push(w); }
}

fn main() {
    let text = String::from("alpha beta");
    let mut i = Interner { words: Vec::new() };
    for w in text.split(' ') {
        i.add(w);
    }
    println!("{} words: {:?}", i.words.len(), i.words);
}
```

```
$ rustc --edition 2024 vec_fixed.rs && ./vec_fixed
2 words: ["alpha", "beta"]
```

As a rule: **`a &'a mut Thing<'a>` signature is almost always a mistake.** The same goes for `&'a mut Vec<&'a T>` in function signatures. Give the mutable borrow its own lifetime, which in most cases means leaving it elided.

The variance table

From the Reference:

Type	Variance in `'a`	Variance in `T`
`&'a T`	covariant	covariant
`&'a mut T`	covariant	**invariant**

Type	Variance in `'a`	Variance in `T`
`*const T`		covariant
`*mut T`		**invariant**
`[T]`, `[T; n]`, `Box<T>`, `Vec<T>`		covariant
`fn() -> T`		covariant
`fn(T) -> ()`		**contra**variant
`UnsafeCell<T>`, `Cell<T>`, `RefCell<T>`		**invariant**
`PhantomData<T>`		covariant
`dyn Trait<T> + 'a`	covariant	**invariant**

A struct or enum is covariant in a parameter only if every field that uses that parameter is covariant in it. If fields disagree, or any field is invariant, the parameter is invariant. You can't annotate variance yourself. To change it, you change the fields, often by adding a `PhantomData` marker of the right shape.

Pitfalls

- **Assuming `Cell` is "just a wrapper".** Adding a `Cell<&'a T>` field to a struct that used to be covariant makes it invariant. That is a **breaking change** for downstream code that relied on shrinking lifetimes, even though no public signature changed.
- **Reaching for `'static` to "fix" invariance errors.** Writing `T: 'static` or `&'static` everywhere makes the error go away by making the API useless for borrowed data. Find the `&mut` or `Cell` that forces invariance instead.
- **Reading E0499 or E0502 at face value.** "Mutably borrowed in the previous iteration of the loop" on a method call usually means the method's signature ties `&mut self` to a struct lifetime. Check the signature before you restructure the loop.
- **Unsafe code with the wrong variance.** A raw-pointer container that stores `*const T` is covariant, but if it hands out `&mut T` it probably needs to be invariant. Variance mistakes in `unsafe` code are soundness bugs, not just inconvenient compile errors.

Takeaways

- Shared references are covariant, so longer lifetimes shrink to fit.
- Anything you can write through (`&mut T`, `Cell<T>`, `*mut T`) is invariant in `T`, because otherwise safe code could store a dangling reference.
- Function arguments are contravariant, so callbacks that accept more are acceptable where callbacks that accept less are expected.
- A struct's variance comes from its fields, so changing a field type can be a breaking change.
- `&'a mut Thing<'a>` locks `Thing` for its whole lifetime. Let the `&mut self` borrow have its own lifetime.

Sources

- The Rust Reference, Subtyping and Variance
- The Rustonomicon, Subtyping and Variance
- Rust standard library, `std::cell::Cell`
- Rust compiler error index, E0597, E0499, E0502

Summary

Summary

Chapter Key Takeaways

- Lifetimes allow the compiler to prove reference validity at compile time.
- Elision rules save annotations in common single-reference or method signatures.
- Structs holding references must declare lifetime parameters (`Struct<'a>`).
- `'static` denotes references valid for the entire program runtime.

Smart Pointers and Interior Mutability

Box for Heap Allocation

Box for Heap Allocation

`Box<T>` allocates a value on the heap and provides a stack pointer with ownership semantics.

```
fn main() {
    let b = Box::new(42);
    println!("Box points to heap value: {}", *b);

    // Recursive data types require Box to have a known size:
    #[derive(Debug)]
    enum List {
        Cons(i32, Box<List>),
        Nil,
    }

    let list = List::Cons(1, Box::new(List::Cons(2, Box::new(List::Nil))));
    println!("Recursive list: {:?}", list);
}
```

Reference Counting with Rc and Arc

Reference Counting with Rc and Arc

When multiple parts of a program need shared read ownership of heap memory: - `Rc<T>`: Single-threaded reference counting. - `Arc<T>`: Atomic reference counting across multiple threads.

```
use std::sync::Arc;
use std::thread;

fn main() {
    let data = Arc::new(vec![1, 2, 3]);

    let data_clone = Arc::clone(&data);
    let handle = thread::spawn(move || {
        println!("Thread sees: {:?}", data_clone);
    });

    handle.join().unwrap();
    println!("Main sees: {:?}", data);
}
```

Interior Mutability with RefCell

Interior Mutability with RefCell

`RefCell<T>` enforces Rust's borrowing rules dynamically at runtime rather than at compile time:

```
use std::cell::RefCell;

fn main() {
    let val = RefCell::new(10);
    {
        let mut mut_borrow = val.borrow_mut();
        *mut_borrow += 5;
    }
    println!("Mutated value: {}", *val.borrow());
}
```

Cell and Mutex for Shared State

Cell and Mutex for Shared State

- `Cell<T>`: Interior mutability for `Copy` types via value copying.
- `Mutex<T>`: Thread-safe interior mutability via mutual exclusion locks.

```
use std::sync::Mutex;

fn main() {
    let m = Mutex::new(0);
    {
        let mut guard = m.lock().unwrap();
        *guard += 1;
    }
    println!("Mutex protected value: {:?}", m.lock().unwrap());
}
```

Weak References and Breaking Cycles

Weak References and Breaking Cycles

Cycles of `Rc` or `Arc` pointers can cause memory leaks. Use `Weak<T>` (downgraded non-owning reference) to break reference cycles:

```
use std::rc::{Rc, Weak};

struct Node {
    parent: Option<Weak<Node>>,
}
```

LazyLock and OnceLock

LazyLock and OnceLock

Global or shared values often need **one-time initialization**: a compiled regex, a config snapshot, a connection pool handle. Before Rust 1.70 / 1.80 you reached for `lazy_static`, `once_cell`, or a hand-rolled `Mutex<Option<T>>`. Today the standard library covers the boring cases:

Type	Module	Stabilized	Role
`OnceLock<T>`	`std::sync`	1.70	Thread-safe cell: write once, then shared reads
`LazyLock<T, F>`	`std::sync`	1.80	`OnceLock` + init closure; initializes on first access
`OnceCell<T>` / `LazyCell<T>`	`std::cell`	1.70 / 1.80	Single-threaded cousins (no `Sync`)

Prefer **`LazyLock`** for `static` values with a pure init function. Prefer **`OnceLock`** when initialization needs arguments, fallible setup, or happens explicitly in `main` / a builder. This chapter sits next to `Mutex` and `Arc` because the shared handle is often wrapped in those types later — but lazy init itself is not a lock around every read after the first.

Mental model

```
first access / set()

  run init exactly once      store T

 later access      &T (no re-init)
```

- Concurrent first accessors: one wins the init; others wait, then see the same `T`.
- If the init closure **panics**, `LazyLock` is **poisoned** for the rest of the process (unrecoverable). Fail fallible work *before* installing into a `LazyLock`, or use `OnceLock` + `Result` you control.
- `static LazyLock<T>` values are not dropped at process exit; the OS reclaims the memory. That is intentional and fine for process-lifetime data.

Worked examples

Case 1: `LazyLock` for a process-wide regex

```
use std::sync::LazyLock;
use regex::Regex;

static EMAIL: LazyLock<Regex> = LazyLock::new(|| {
    Regex::new(r"(?i)^[a-z0-9._%+-]+@[a-z0-9.-]+\.[a-z]{2,}$").expect("static regex")
});

fn looks_like_email(s: &str) -> bool {
    EMAIL.is_match(s)
}

fn main() {
    assert!(looks_like_email("dev@example.com"));
    assert!(!looks_like_email("not-an-email"));
    println!("ok");
}
```

First call to `looks_like_email` compiles the pattern once. Later calls reuse the same `Regex`. The `expect` is acceptable only for patterns that are compile-time constants you control — never for user-supplied patterns inside a `LazyLock` init.

Add `regex` in `Cargo.toml` for this sample (`regex = "1"`). For a std-only sketch, replace the body with a pure function that builds a `String` or `HashMap`.

Case 2: `OnceLock` when init needs an argument

```
use std::sync::OnceLock;

struct AppConfig {
    api_root: String,
}

fn config() -> &'static AppConfig {
    static CFG: OnceLock<AppConfig> = OnceLock::new();
    CFG.get_or_init(|| AppConfig {
        api_root: std::env::var("API_ROOT").unwrap_or_else(|_| "http://127.0.0.1:8080".into(
    })
}

fn main() {
    println!("api_root={}", config().api_root);
    // Second call returns the same address
```

```
    assert!(std::ptr::eq(config(), config()));
}
```

`get_or_init` runs the closure only if empty. `set` / `get` let you initialize from `main` after CLI parsing:

```
use std::sync::OnceLock;

static CFG: OnceLock<String> = OnceLock::new();

fn main() {
    let root = std::env::args().nth(1).unwrap_or_else(|| "/srv/app".into());
    CFG.set(root).expect("config set once");
    println!("{}", CFG.get().expect("configured"));
}
```

`set` returns `Err(value)` if the cell was already filled — use that to catch double-init bugs in tests.

Case 3: Fallible init without poisoning `LazyLock`

```
use std::sync::OnceLock;

static DB_URL: OnceLock<String> = OnceLock::new();

fn init_db_url() -> Result<(), String> {
    let url = std::env::var("DATABASE_URL").map_err(|_| "DATABASE_URL missing".to_string())?
    if !url.starts_with("postgres://") {
        return Err("DATABASE_URL must start with postgres://".into());
    }
    DB_URL.set(url).map_err(|_| "DB_URL already set".to_string())?;
    Ok(())
}

fn db_url() -> &'static str {
    DB_URL.get().expect("call init_db_url() before use")
}

fn main() {
    if let Err(e) = init_db_url() {
        eprintln!("startup failed: {e}");
        std::process::exit(1);
    }
    println!("using {}", db_url());
}
```

Validate and then `set`. Do not hide I/O errors inside `LazyLock::new(|| …)` if you need a clean startup failure path.

Case 4: Single-threaded `LazyCell`

```rust
use std::cell::LazyCell;

fn main() {
    let cache = LazyCell::new(|| {
        println!("computing");
        21 * 2
    });
    assert_eq!(*cache, 42); // prints "computing"
    assert_eq!(*cache, 42); // silent
}
```

Use `LazyCell` / `OnceCell` inside one thread (or inside a type that is already `!Sync`). Crossing threads wants `LazyLock` / `OnceLock`.

Choosing among the options

Situation	Prefer
`static` value, infallible or `expect`-able init	`LazyLock`
Init needs CLI/env parsed in `main`	`OnceLock` + `set`
Fallible init with structured errors	`OnceLock` + `Result` before `set`
Non-`Sync` local cache	`LazyCell` / `OnceCell`
Mutate after init	Not these cells — use `Mutex` / `RwLock` around the data, or rebuild

`LazyLock` implements `Deref`, so `*LAZY` and method calls work like a plain `&T` after init. `OnceLock` exposes `get`, `get_or_init`, `set`, and related helpers explicitly.

Pitfalls

1. **Heavy work on first request path.** First HTTP request paying for a huge init surprises latency SLOs — warm `LazyLock` during startup (`let _ = &*STATIC;`) if the cost matters.
2. **Panic inside `LazyLock` init.** Poison is permanent for that static. Prefer fallible setup with `OnceLock`.
3. **Storing `Mutex` inside when you only needed one-time init.** After init, readers of an immutable `T` do not need a mutex. Add synchronization only for post-init mutation.
4. **Assuming `Drop` runs for `static LazyLock`.** It does not at process exit. Flush external resources explicitly if required.
5. **Using `once_cell` or `lazy_static` in new code on 1.80+.** Prefer std unless you need an API those crates still uniquely provide.

Try this

1. Build Case 1 (or a std-only `LazyLock<HashMap<…>>`) and print addresses of `&*EMAIL` across two threads with `std::thread::scope` — they must match.
2. Call `CFG.set` twice in Case 2's `set` variant; assert the second call is `Err`.
3. Replace Case 3's `OnceLock` with a `LazyLock` that `expect`s on missing env — then run without `DATABASE_URL` and observe the panic/poison behavior.
4. Skim `std::sync::LazyLock` and `OnceLock` for `get_mut` / `take` (where available on your MSRV) before using them in libraries.

Sources

- `std::sync::LazyLock` (1.80+)
- `std::sync::OnceLock` (1.70+)
- `std::cell::LazyCell` / `OnceCell`

MaybeUninit and Uninitialized Memory

MaybeUninit and Uninitialized Memory

Safe Rust assumes every value is **valid for its type**: a `bool` is only `true` or `false`, a reference is non-null and aligned, a `Vec<T>` upholds its length/capacity invariants. Sometimes you need storage **before** those invariants hold — out-pointers, element-by-element arrays, FFI buffers. The standard tool is `std::mem::MaybeUninit<T>`: a wrapper that tells the compiler "these bytes might not be initialized yet."

Do **not** reach for the deprecated `mem::uninitialized` or casual `mem::zeroed` on arbitrary `T`. Prefer `MaybeUninit`, initialize deliberately, then `assume_init` only when every validity requirement of `T` is met.

Mental model

```
MaybeUninit::uninit()     ← bytes may be garbage; Drop of T does NOT run

   write / ptr::write     ← initialize without dropping old (uninit) contents

 assume_init / assume_init_drop


       T                  ← normal Drop and safe APIs apply
```

API	Role
`MaybeUninit::uninit()`	Allocate storage; contents uninitialized
`MaybeUninit::new(v)`	Already-initialized wrapper
`MaybeUninit::zeroed()`	Fill with `0` bytes — **only** valid if all-zero is a valid `T`
`write(val)`	Initialize (overwrites without dropping previous)
`assume_init()`	Assert fully initialized; take ownership as `T`
`assume_init_drop()`	Drop in place when you do not want ownership
`as_mut_ptr()` / `&raw mut (*ptr).field`	Field-by-field init without creating `&mut` to uninit fields

Dropping a `MaybeUninit<T>` **never** runs `T`'s destructor. If you initialized and then abandon the wrapper without `assume_init` / `assume_init_drop`, you **leak** resources (safe but usually undesirable).

Worked examples

Case 1: `write` then `assume_init`

```
use std::mem::MaybeUninit;

fn main() {
    let mut slot = MaybeUninit::<Vec<u8>>::uninit();
    slot.write(vec![1, 2, 3]);
    let v = unsafe { slot.assume_init() };
    assert_eq!(v, vec![1, 2, 3]);
    println!("len={}", v.len());
}
```

`write` returns `&mut T` to the now-initialized contents. Calling `write` twice **skips** dropping the first value — a leak if `T` owns heap memory. Prefer one write, or drop explicitly between overwrites when you must replace.

Case 2: Out-pointer style

```
use std::mem::MaybeUninit;

unsafe fn fill_vec(out: *mut Vec<i32>) {
    // write does not drop old contents - correct for uninit destination
    unsafe { out.write(vec![10, 20, 30]); }
}

fn main() {
    let mut slot = MaybeUninit::uninit();
    unsafe { fill_vec(slot.as_mut_ptr()); }
    let v = unsafe { slot.assume_init() };
    assert_eq!(v, vec![10, 20, 30]);
}
```

The caller owns allocation strategy; the callee only initializes. After `assume_init`, `Drop` for `Vec` runs normally at end of scope.

Case 3: Array element-by-element

```
use std::mem::{self, MaybeUninit};

fn build_boxes() -> [Box<u32>; 4] {
    let mut data: [MaybeUninit<Box<u32>>; 4] = [const { MaybeUninit::uninit() }; 4];
```

```
    for (i, slot) in data.iter_mut().enumerate() {
        slot.write(Box::new(i as u32));
    }

    // SAFETY: every element was written
    unsafe { mem::transmute::<_, [Box<u32>; 4]>(data) }
}

fn main() {
    let arr = build_boxes();
    assert_eq!(*arr[3], 3);
    println!("{:?}", arr.iter().map(|b| **b).collect::<Vec<_>>());
}
```

Panic mid-loop leaks already-written `Box`es (no destructor on remaining `MaybeUninit`), but does not invent UB from dropping uninitialized boxes. For partial init, track a length and call `assume_init_drop` on the live prefix in a cleanup path.

Case 4: Field-by-field struct init

```
use std::mem::MaybeUninit;

#[derive(Debug, PartialEq)]
struct Person {
    name: String,
    age: u32,
}

fn main() {
    let person = {
        let mut uninit = MaybeUninit::<Person>::uninit();
        let ptr = uninit.as_mut_ptr();

        // SAFETY: write through raw field pointers - do NOT create &mut to uninit fields
        unsafe {
            (&raw mut (*ptr).name).write(String::from("Ada"));
            (&raw mut (*ptr).age).write(36);
            uninit.assume_init()
        }
    };

    assert_eq!(
        person,
        Person {
            name: String::from("Ada"),
            age: 36
```

```
        }
    );
    println!("{person:?}");
}
```

Using `assume_init_mut()` to "assign fields" **before** the struct is fully initialized is UB — that creates references to uninitialized memory. Prefer `&raw mut` / `ptr::write` as above. Padding bytes need not be initialized.

Case 5: `zeroed` pitfalls

```
use std::mem::MaybeUninit;

fn main() {
    // OK: (u8, bool) can be all-zero
    let pair = unsafe { MaybeUninit::<(u8, bool)>::zeroed().assume_init() };
    assert_eq!(pair, (0, false));

    // NOT OK: &T must be non-null - zeroed reference is instantaneous UB
    // let bad: &i32 = unsafe { MaybeUninit::zeroed().assume_init() };

    // NOT OK: many enums reject discriminant 0
    // enum NonZero { A = 1, B = 2 }
    // let bad = unsafe { MaybeUninit::<NonZero>::zeroed().assume_init() };

    println!("pair={pair:?}");
}
```

Rule of thumb: `zeroed().assume_init()` only for types where **every bit pattern of all-zeros is a valid value** (raw integers, `bool`, structs of such). Never for references, `NonNull`, most `enums`, or "looks like a Vec with ptr=0".

Choosing the API

Situation	Prefer
Single value, known initializer	`MaybeUninit::new` or plain `T`
Deferred init / out-pointer	`uninit` + `write` / `ptr::write`
Large array filled in a loop	`[MaybeUninit<T>; N]` + `write` each
Struct fields in sequence	`&raw mut (*ptr).field` + `write`
Need all-zero bit pattern and it is valid	`zeroed` with care
Old code with `mem::uninitialized`	Port to `MaybeUninit`

Pitfalls

1. **`assume_init` too early** — immediate UB, even for `i32` / `bool` when bytes are still uninit.
2. **`zeroed` on references / invalid enums** — classic footgun; all-zero is not "safe default" for every `T`.
3. **Double `write` without drop** — leaks the first `T`.
4. **Creating `&` / `&mut` to uninitialized fields** — use raw pointers / `&raw mut` until fully init.
5. **Transmuting `&mut T` to `&mut MaybeUninit<T>` for safe code** — lets safe code invent uninit views; unsound.
6. **Assuming Miri-silence equals production-proof** — still run tests under Miri for unsafe modules when you can.

Try this

1. Run Case 1 and Case 3 under `cargo +nightly miri run` if Miri is available.
2. Deliberately comment out one field write in Case 4 and observe that Miri / careful review catches the incomplete init (do not ship it).
3. Replace a `mem::uninitialized` call in an old snippet with `MaybeUninit` and list which `assume_init` proof you now owe.
4. Skim the Rustonomicon chapter on unchecked uninitialized memory and compare its array pattern to Case 3.

Sources

- `std::mem::MaybeUninit`
- The Rustonomicon — Unchecked Uninitialized Memory

Custom Allocators and the Allocator API

Custom Allocators and the Allocator API

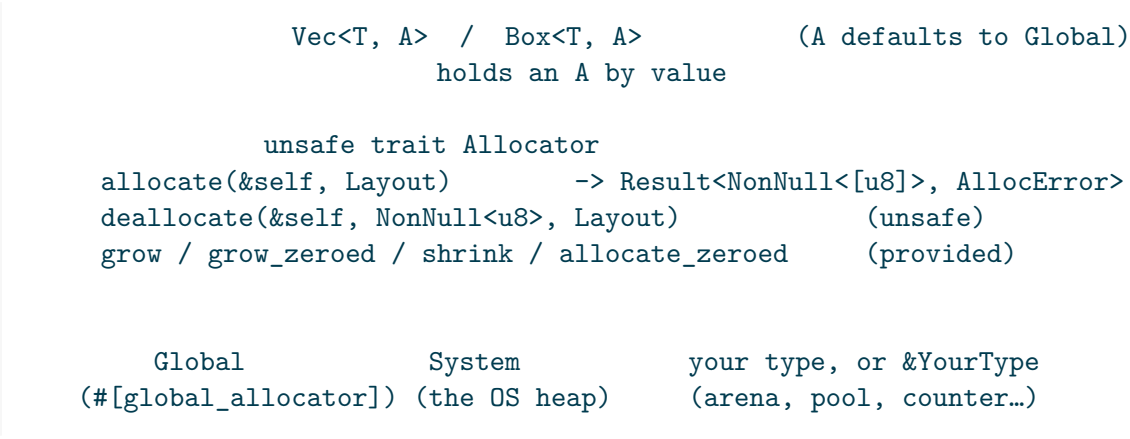
Thesis

Every `Box` and `Vec` you have written so far got its memory from one place: the **global allocator**. You could swap that allocator for the whole program with `#[global_allocator]`, but you could not say "*this* vector lives in *that* arena". The `Allocator` trait that would let you do so has existed since 2016, behind the nightly-only `allocator_api` feature, and arena crates have shipped nightly-only allocator-aware APIs for years because of it.

Rust 1.100 (scheduled for 2026-11-12) stabilizes a deliberately small slice of that design: the `std::alloc::Allocator` trait, the `Global` and `System` allocators as implementors, and allocator-aware constructors on exactly two containers, `Box<T, A>` and `Vec<T, A>`. Everything else (`Rc::new_in`, fallible `try_*_in` constructors, allocator-aware `String` and `HashMap`) stays unstable under a new feature name, `allocator_ext`.

This chapter builds two allocators on the stable surface, a counting wrapper and a bump arena. It then walks through the places where the type system and the abort-on-failure policy catch people out. Everything was compiled with the current beta, which is what becomes 1.100.

Mental model



Four facts carry the chapter:

Fact	Consequence
The allocator is a **type parameter** with a default: `Vec<T, A = Global>`	`Vec<u32>` and `Vec<u32, System>` are *different types*. Code written for one does not accept the other

Fact	Consequence
The container **stores** an `A`	A zero-sized allocator (`Global`, `System`) costs nothing; `&Arena` costs one pointer; `&dyn Allocator` costs two
Methods take `&self`	Implement `Allocator` on `&Arena` (blanket impl provided) to share one arena among many containers. Interior mutability (`Cell`) does the bookkeeping
The trait is `unsafe` to implement	You promise that returned blocks are valid until deallocated, that blocks don't overlap, and that no method unwinds. The compiler cannot check any of it

`allocate` returns `NonNull<[u8]>`, a *slice* pointer, so an allocator can report that it handed back more than was asked for. `AllocError` is a zero-sized "no" with no further detail.

Worked examples

All listings were compiled and run on the box with **rustc 1.100.0-beta.4 (2026-10-04)** and, for comparison, **rustc 1.99.0** stable, both installed with `rustup`. Until 1.100 reaches stable, use `+beta`:

```
rustup toolchain install beta --profile minimal
rustc +beta --version
```

```
rustc 1.100.0-beta.4 (a2eb6097d 2026-10-04)
```

Case 1: a counting allocator and a bump arena

```
cargo +beta new alloc-demo && cd alloc-demo
```

`Counting` wraps any other allocator and records each call. `Arena<N>` hands out slices of a fixed buffer by bumping a cursor, and its `deallocate` does nothing: the whole region is released at once when the arena is dropped. Note that neither needs a `#![feature]` line.

```
// src/main.rs - a counting allocator and a bump arena on the stable Allocator API (Rust 1.1
use std::alloc::{AllocError, Allocator, Global, Layout};
use std::cell::{Cell, UnsafeCell};
use std::mem::MaybeUninit;
use std::ptr::NonNull;

/// Wraps another allocator and counts what passes through it.
struct Counting<A: Allocator = Global> {
    inner: A,
    allocs: Cell<usize>,
    grows: Cell<usize>,
```

```
    frees: Cell<usize>,
    live_bytes: Cell<usize>,
}

impl Counting {
    const fn new() -> Self {
        Counting { inner: Global, allocs: Cell::new(0), grows: Cell::new(0), frees: Cell::ne
    }
    fn report(&self, label: &str) {
        println!(
            "{label:<22} allocs={} grows={} frees={} live_bytes={}",
            self.allocs.get(), self.grows.get(), self.frees.get(), self.live_bytes.get()
        );
    }
}

// SAFETY: every method forwards to `inner` with the caller's arguments unchanged,
// so blocks stay valid until deallocated and are freed by the allocator that made them.
unsafe impl<A: Allocator> Allocator for Counting<A> {
    fn allocate(&self, layout: Layout) -> Result<NonNull<[u8]>, AllocError> {
        self.allocs.set(self.allocs.get() + 1);
        self.live_bytes.set(self.live_bytes.get() + layout.size());
        self.inner.allocate(layout)
    }
    unsafe fn deallocate(&self, ptr: NonNull<u8>, layout: Layout) {
        self.frees.set(self.frees.get() + 1);
        self.live_bytes.set(self.live_bytes.get() - layout.size());
        unsafe { self.inner.deallocate(ptr, layout) }
    }
    unsafe fn grow(&self, ptr: NonNull<u8>, old: Layout, new: Layout) -> Result<NonNull<[u8]
        self.grows.set(self.grows.get() + 1);
        self.live_bytes.set(self.live_bytes.get() - old.size() + new.size());
        unsafe { self.inner.grow(ptr, old, new) }
    }
}

/// A fixed-size bump arena: allocation moves a cursor, deallocation does nothing,
/// and all memory comes back at once when the arena itself is dropped.
struct Arena<const N: usize> {
    buf: UnsafeCell<[MaybeUninit<u8>; N]>,
    used: Cell<usize>,
}

impl<const N: usize> Arena<N> {
    fn new() -> Self {
        Arena { buf: UnsafeCell::new([MaybeUninit::uninit(); N]), used: Cell::new(0) }
    }
```

```
    fn used(&self) -> usize {
        self.used.get()
    }
}

// SAFETY: blocks are carved from `buf` without overlap and stay valid while `self`
// is alive; borrowing the arena (`&Arena`) ties every collection to that lifetime.
unsafe impl<const N: usize> Allocator for Arena<N> {
    fn allocate(&self, layout: Layout) -> Result<NonNull<[u8]>, AllocError> {
        let base = self.buf.get() as *mut u8;
        let start = self.used.get();
        let pad = base.wrapping_add(start).align_offset(layout.align());
        let begin = start.checked_add(pad).ok_or(AllocError)?;
        let end = begin.checked_add(layout.size()).ok_or(AllocError)?;
        if end > N {
            return Err(AllocError);
        }
        self.used.set(end);
        let ptr = unsafe { NonNull::new_unchecked(base.add(begin)) };
        Ok(NonNull::slice_from_raw_parts(ptr, layout.size()))
    }
    unsafe fn deallocate(&self, _ptr: NonNull<u8>, _layout: Layout) {
        // Bump arenas free everything at once, when the arena is dropped.
    }
}

#[derive(Debug)]
struct Token<'a> {
    kind: &'static str,
    text: &'a str,
}

fn tokenize<'s, A: Allocator>(src: &'s str, alloc: A) -> Vec<Token<'s>, A> {
    let mut out = Vec::new_in(alloc);
    for word in src.split_whitespace() {
        let kind = if word.chars().all(|c| c.is_ascii_digit()) { "num" } else { "word" };
        out.push(Token { kind, text: word });
    }
    out
}

fn main() {
    // 1. Counting: watch Vec's growth policy through the allocator.
    let counter = Counting::new();
    {
        let mut v: Vec<u64, &Counting> = Vec::new_in(&counter);
        counter.report("after new_in");
```

```
        for i in 0..100 {
            v.push(i);
        }
        counter.report("after 100 pushes");
        let b = Box::new_in([0u8; 256], &counter);
        counter.report("plus a Box");
        println!("capacity = {}, box len = {}", v.capacity(), b.len());
    }
    counter.report("after drop");

    let counter2 = Counting::new();
    {
        let mut v: Vec<u64, &Counting> = Vec::with_capacity_in(100, &counter2);
        for i in 0..100 {
            v.push(i);
        }
    }
    counter2.report("with_capacity_in(100)");

    // 2. Arena: many small values, one region, no per-object free.
    let arena: Arena<4096> = Arena::new();
    let toks = tokenize("let x = 42 + y * 7", &arena);
    println!("{} tokens, arena used = {} bytes", toks.len(), arena.used());
    println!("first = {:?}, last = {:?}", toks[0], toks[toks.len() - 1]);
    let nums: Vec<&str> = toks.iter().filter(|t| t.kind == "num").map(|t| t.text).collect();
    println!("numbers = {nums:?}");

    let pair = Box::new_in((1u32, 2u32), &arena);
    println!("boxed pair = {:?}, arena used = {}", *pair, arena.used());
    println!("Vec allocator is the arena: {}", std::ptr::eq(*toks.allocator(), &arena));
}
```

```
cargo +beta run -q
```

```
after new_in           allocs=0 grows=0 frees=0 live_bytes=0
after 100 pushes       allocs=1 grows=5 frees=0 live_bytes=1024
plus a Box             allocs=2 grows=5 frees=0 live_bytes=1280
capacity = 128, box len = 256
after drop             allocs=2 grows=5 frees=2 live_bytes=0
with_capacity_in(100)  allocs=1 grows=0 frees=1 live_bytes=0
8 tokens, arena used = 384 bytes
first = Token { kind: "word", text: "let" }, last = Token { kind: "num", text: "7" }
numbers = ["42", "7"]
boxed pair = (1, 2), arena used = 392
Vec allocator is the arena: true
```

What to notice:

- **`Vec::new_in` allocates nothing.** The first `push` makes one allocation (4 × `u64`), and then the vector *grows* five times (8, 16, 32, 64, 128) to hold 100 items. `with_capacity_in(100)` makes one allocation and no growth. The counter turns a growth policy you normally take on faith into numbers you can see.
- **`live_bytes` returns to 0** after the scope ends, so every byte allocated was deallocated with the layout it was allocated with. That is the contract `Vec` and `Box` keep on their side.
- **The arena used 384 bytes for 8 tokens of 32 bytes each.** The provided `grow` method allocates a new, larger block, copies, and calls `deallocate` on the old block. In a bump arena that call is a no-op, so the 4-token block (128 bytes) is simply abandoned. Arenas reward `with_capacity_in` far more than the heap does.
- `tokenize` is generic over `A: Allocator` and returns `Vec<Token<'s>, A>`. The caller, not the function, decides where the tokens live.
- `toks.allocator()` gives back the allocator the vector holds. Here that is the `&Arena` reference.

On 1.99 stable the same crate is rejected at the first import:

```
cargo +stable build -q
```

```
error[E0658]: use of unstable library feature `allocator_api`
 --> src/main.rs:2:18
  |
2 | use std::alloc::{AllocError, Allocator, Global, Layout};
  |                  ^^^^^^^^^^
  |
  = note: see issue #32838 <https://github.com/rust-lang/rust/issues/32838> for more informa
...
```

Case 2: what the allocator parameter costs

The allocator is stored inside the container, so its size shows up in the container's size. The trait is also `dyn`-compatible, so you can choose an allocator at run time.

```
// dynalloc.rs - choosing an allocator at run time with &dyn Allocator
use std::alloc::{Allocator, Global, System};

fn pick(use_system: bool) -> &'static dyn Allocator {
    if use_system { &System } else { &Global }
}

fn main() {
    for flag in [false, true] {
        let alloc = pick(flag);
        let mut v: Vec<u32, &dyn Allocator> = Vec::with_capacity_in(4, alloc);
        v.extend([10, 20, 30]);
        let sum: u32 = v.iter().sum();
```

```
        println!("system={flag} sum={sum} size_of::<Vec<u32, &dyn Allocator>>()={}",
                 std::mem::size_of_val(&v));
    }
    println!("size_of::<Vec<u32>>() = {}", std::mem::size_of::<Vec<u32>>());
    println!("size_of::<Vec<u32, System>>() = {}", std::mem::size_of::<Vec<u32, System>>());
}
```

```
rustc +beta --edition 2024 dynalloc.rs && ./dynalloc
```

```
system=false sum=60 size_of::<Vec<u32, &dyn Allocator>>()=40
system=true sum=60 size_of::<Vec<u32, &dyn Allocator>>()=40
size_of::<Vec<u32>>() = 24
size_of::<Vec<u32, System>>() = 24
```

`Global` and `System` are zero-sized, so a `Vec` that uses them is still three words. A `&Arena` adds one word, and a `&dyn Allocator` adds two (data pointer and vtable) plus an indirect call on every allocation. Reach for `dyn` only when the choice really is made at run time.

Case 3: an allocator says no

Every allocating method you already know (`push`, `extend`, `Box::new_in`) is **infallible**. When the allocator returns `AllocError`, they call `handle_alloc_error`, which aborts the process. The global heap almost never says no, so this hardly matters there. A 256-byte arena says no quickly.

```
// exhaust.rs - what happens when a custom allocator says no
use std::alloc::{AllocError, Allocator, Layout};
use std::cell::{Cell, UnsafeCell};
use std::mem::MaybeUninit;
use std::ptr::NonNull;

struct Arena<const N: usize> {
    buf: UnsafeCell<[MaybeUninit<u8>; N]>,
    used: Cell<usize>,
}

impl<const N: usize> Arena<N> {
    fn new() -> Self {
        Arena { buf: UnsafeCell::new([MaybeUninit::uninit(); N]), used: Cell::new(0) }
    }
}

// SAFETY: non-overlapping blocks carved from `buf`, valid while the arena lives.
unsafe impl<const N: usize> Allocator for Arena<N> {
    fn allocate(&self, layout: Layout) -> Result<NonNull<[u8]>, AllocError> {
        let base = self.buf.get() as *mut u8;
```

```
        let start = self.used.get();
        let pad = base.wrapping_add(start).align_offset(layout.align());
        let begin = start.checked_add(pad).ok_or(AllocError)?;
        let end = begin.checked_add(layout.size()).ok_or(AllocError)?;
        if end > N {
            return Err(AllocError);
        }
        self.used.set(end);
        let ptr = unsafe { NonNull::new_unchecked(base.add(begin)) };
        Ok(NonNull::slice_from_raw_parts(ptr, layout.size()))
    }
    unsafe fn deallocate(&self, _: NonNull<u8>, _: Layout) {}
}

fn main() {
    let mode = std::env::args().nth(1).unwrap_or_default();
    let arena: Arena<256> = Arena::new();
    let mut v: Vec<u64, &Arena<256>> = Vec::new_in(&arena);

    if mode == "try" {
        for i in 0.. {
            if v.len() == v.capacity() {
                if let Err(e) = v.try_reserve(1) {
                    println!("stopped at len={} cap={}: {e}", v.len(), v.capacity());
                    break;
                }
            }
            v.push(i);
        }
        println!("arena used = {} of 256", arena.used.get());
    } else {
        for i in 0..64u64 {
            v.push(i); // infallible API: allocation failure aborts the process
            println!("len={} cap={} used={}", v.len(), v.capacity(), arena.used.get());
        }
    }
}
```

```
rustc +beta --edition 2024 -O exhaust.rs
./exhaust; echo "exit=$?"
./exhaust try; echo "exit=$?"
```

```
len=1 cap=4 used=32
len=2 cap=4 used=32
len=3 cap=4 used=32
len=4 cap=4 used=32
```

```
len=5 cap=8 used=96
len=6 cap=8 used=96
len=7 cap=8 used=96
len=8 cap=8 used=96
len=9 cap=16 used=224
len=10 cap=16 used=224
len=11 cap=16 used=224
len=12 cap=16 used=224
len=13 cap=16 used=224
len=14 cap=16 used=224
len=15 cap=16 used=224
len=16 cap=16 used=224
memory allocation of 256 bytes failed
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
Aborted
exit=134
stopped at len=16 cap=16: memory allocation failed because the memory allocator returned an
arena used = 224 of 256
exit=0
```

(`Aborted` is the shell reporting `SIGABRT`; its exact wording differs between shells.) The abort is not a panic. `catch_unwind` cannot stop it, and destructors do not run. `Vec::try_reserve` *is* available on `Vec<T, A>` (it has been stable since 1.57 and works with any allocator), so the fallible path is to reserve before you push and handle the `TryReserveError`. Note too that 128 bytes of real data cost 224 bytes of arena, because the 32- and 64-byte blocks left behind by each growth step are never reused.

Case 4: what is *not* stable yet

The stabilization is an MVP on purpose. Three things people reach for straight away still fail on 1.100 beta:

```
// not_yet.rs - allocator-aware APIs that are still unstable in Rust 1.100
use std::alloc::{AllocError, Allocator, Global, Layout};
use std::ptr::NonNull;
use std::rc::Rc;

struct Pool;
unsafe impl Allocator for Pool {
    fn allocate(&self, l: Layout) -> Result<NonNull<[u8]>, AllocError> { Global.allocate(l)
    unsafe fn deallocate(&self, p: NonNull<u8>, l: Layout) { unsafe { Global.deallocate(p, l
}

fn main() {
    let r = Rc::new_in(5u8, Pool);                              // shared ownership in a cus
    let v = Vec::<u8, Pool>::try_with_capacity_in(8, Pool);     // fallible constructor
```

```
    let p = Box::into_pin(Box::new_in(1u8, Pool));              // pinning a custom-allocate
    println!("{r} {:?} {p}", v.map(|v| v.len()));
}
```

```
rustc +beta --edition 2024 not_yet.rs
```

```
error[E0658]: use of unstable library feature `allocator_ext`
  --> not_yet.rs:13:13
   |
13 |     let r = Rc::new_in(5u8, Pool);                                 // shared ownership in
   |             ^^^^^^^^^^
   |
   = note: see issue #163177 <https://github.com/rust-lang/rust/issues/163177> for more info

error[E0658]: use of unstable library feature `allocator_ext`
  --> not_yet.rs:14:13
   |
14 |     let v = Vec::<u8, Pool>::try_with_capacity_in(8, Pool);     // fallible constructor
   |             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   |
   = note: see issue #163177 <https://github.com/rust-lang/rust/issues/163177> for more info

error[E0277]: the trait bound `Pool: StaticAllocator` is not satisfied
  --> not_yet.rs:15:27
   |
15 |     let p = Box::into_pin(Box::new_in(1u8, Pool));              // pinning a custom-all
   |             ------------- ^^^^^^^^^^^^^^^^^^^^^^ unsatisfied trait bound
   |             |
   |             required by a bound introduced by this call
   |
help: the nightly-only, unstable trait `StaticAllocator` is not implemented for `Pool`
...
```

- **`Rc::new_in` / `Arc::new_in`** remain under `allocator_ext` (tracking issue #163177). What *is* stable is `impl Allocator for Rc<A>` / `Arc<A>`, so a shared allocator can be passed around by `Rc`, but the reference-counted *values* cannot live in a custom allocator yet.
- **`try_with_capacity_in`** and the other fallible constructors are unstable. Use `new_in` plus `try_reserve`, as in Case 3.
- **`Box::into_pin`** needs the unstable `StaticAllocator` marker. `Global`, `System`, `&A` and `&mut A` have it, and your own types cannot opt in on stable. This is a soundness fix from the stabilization work: a box whose allocator can be dropped early must not promise `Pin`'s "never moves, never freed without drop" guarantee.
- **`String`, `HashMap`, `BTreeMap`, `VecDeque`** have no allocator parameter on stable. `String<System>` is `error[E0107]: struct takes 0 generic arguments`. If you need allocator-aware text, hold a `Vec<u8, A>` and validate it with `std::str::from_utf8`.

The trap

A collection outlives its arena

`&Arena` implements `Allocator`, and the borrow checker treats it like any other borrow. Returning a vector that borrows a local allocator fails to compile:

```
// outlive.rs - a collection cannot outlive the arena it borrows
use std::alloc::{AllocError, Allocator, Global, Layout};
use std::ptr::NonNull;

struct Pool; // stand-in: forwards to Global
unsafe impl Allocator for Pool {
    fn allocate(&self, l: Layout) -> Result<NonNull<[u8]>, AllocError> { Global.allocate(l)
    unsafe fn deallocate(&self, p: NonNull<u8>, l: Layout) { unsafe { Global.deallocate(p, l
}

fn build() -> Vec<u32, &'static Pool> {
    let pool = Pool;
    let mut v = Vec::new_in(&pool);
    v.push(1);
    v
}

fn main() {
    println!("{:?}", build());
}
```

```
rustc +beta --edition 2024 outlive.rs
```

```
error[E0515]: cannot return value referencing local variable `pool`
  --> outlive.rs:15:5
   |
13 |     let mut v = Vec::new_in(&pool);
   |                              ----- `pool` is borrowed here
14 |     v.push(1);
15 |     v
   |     ^ returns a value referencing data owned by the current function

error: aborting due to 1 previous error
```

This error is the feature working. It is the reason an arena in Rust can be safe without a garbage collector. The fix is structural: create the arena in the caller and pass `&arena` down, as `tokenize` does in Case 1.

`Vec<T, A>` does not fit where `Vec<T>` is expected

```rust
// mismatch.rs - Vec<T, A> is a different type from Vec<T>
use std::alloc::System;

fn total(v: &Vec<u32>) -> u32 {
    v.iter().sum()
}

fn main() {
    let mut v = Vec::new_in(System);
    v.extend([1, 2, 3]);
    println!("{}", total(&v));
}
```

```
error[E0308]: mismatched types
  --> mismatch.rs:11:26
   |
11 |     println!("{}", total(&v));
   |                     ----- ^^ expected `&Vec<u32>`, found `&Vec<{integer}, System>`
   |                     |
   |                     arguments to this function are incorrect
   |
   = note: expected reference `&Vec<u32, std::alloc::Global>`
              found reference `&Vec<{integer}, System>`
```

Every API in your codebase that names `&Vec<T>` or `Box<T>` quietly means `Global`. Read-only functions should take `&[T]`, which every `Vec<T, A>` derefs to. Functions that grow the vector should be generic over the allocator:

```rust
// mismatch_fixed.rs - accept slices, or be generic over the allocator
use std::alloc::{Allocator, System};

fn total(v: &[u32]) -> u32 {
    v.iter().sum()
}

fn push_twice<A: Allocator>(v: &mut Vec<u32, A>, x: u32) {
    v.push(x);
    v.push(x);
}

fn main() {
    let mut v = Vec::new_in(System);
    v.extend([1, 2, 3]);
    push_twice(&mut v, 10);
```

```
    println!("{}", total(&v));
    let plain = vec![4u32, 5];
    println!("{}", total(&plain));
}
```

```
26
9
```

Implementing the trait is a safety contract

`unsafe impl Allocator` is a promise the compiler cannot check. The ways to break it are the classic C allocator bugs: handing out overlapping blocks (a bump cursor that forgets alignment padding), returning memory that dies before the container does (an allocator that owns its buffer by value and gets *moved*), or panicking inside `allocate`. The stabilized contract explicitly forbids unwinding out of any `Allocator` method, from its clones, or from its drop. Keep the `unsafe impl` small, put the bookkeeping in plain safe code, and test it under Miri (`cargo +nightly miri test`), which catches the use-after-free, out-of-bounds, and aliasing violations that a broken allocator causes in the containers using it.

Choosing an approach

Need	Use
Replace the heap for the whole program	`#[global_allocator]` with a `GlobalAlloc` (stable since 1.28)
Many short-lived objects freed together (parsers, per-request scratch)	A bump arena passed as `&Arena` to `Vec::new_in` / `Box::new_in`
Measure or limit what one subsystem allocates	A wrapper like `Counting<A>` around `Global`
Pick the allocator at run time	`&dyn Allocator` (two extra words per container)
Shared ownership, maps, strings in a custom allocator	Not on stable yet (`allocator_ext`), so keep a third-party collection for now
MSRV below 1.100	Keep using nightly-gated code or the arena crate's own types; don't half-adopt

The boring rule

Pass the allocator in, don't hide it: `fn f<A: Allocator>(…, alloc: A) -> Vec<T, A>`. Take `&[T]` when you only read. Reserve capacity up front in arenas, and use `try_reserve` wherever running out is a normal outcome rather than a bug. Write `unsafe impl Allocator` only after you've measured a reason, and run it under Miri.

Try this

1. Change `Counting` to also override `shrink`, call `v.shrink_to_fit()` after the 100 pushes, and confirm the new counter moves while `live_bytes` drops to 800.
2. Give `Arena` a `reset(&mut self)` that sets `used` back to 0. Why does it have to take `&mut self`, and what does the borrow checker say if a `Vec<_, &Arena>` is still alive when you call it?
3. Make `Arena::deallocate` reclaim space when the block being freed is the most recent one (`ptr + size == cursor`). Re-run Case 1 and compare `arena used` before and after.
4. Rewrite `tokenize` to call `Vec::with_capacity_in(src.split_whitespace().count(), alloc)` and predict the new `arena used` value before running it.
5. Wrap `System` in `Counting<System>` and verify that `Box::into_pin` fails to compile for it even though it compiles for plain `System`. Which unstable trait is missing, and why can't you implement it yourself?

Sources

- rust-lang/rust PR #156882, "alloc: stabilise `Allocator`" (stabilization report, merged 2026-09-23, milestone 1.100.0): https://github.com/rust-lang/rust/pull/156882
- Draft release notes for 1.100.0: https://github.com/rust-lang/rust/issues/163811
- Tracking issue `allocator_api` #32838 and the remaining-surface issue `allocator_ext` #163177
- RFC 1398, "Kinds of allocators": https://rust-lang.github.io/rfcs/1398-kinds-of-allocators.html
- std docs: `std::alloc::Allocator`, `Vec::new_in`, `Box::new_in`, `Vec::try_reserve`, `handle_alloc_error`

Arc::new_cyclic: self-referential shared ownership without a temporary Option

Arc::new_cyclic: self-referential shared ownership without a temporary Option

After reading this chapter, you will be able to construct an `Arc<T>` (or `Rc<T>`) that stores a `Weak<T>` to itself during initialization, wire parent links in a shared tree without an `Option::None` temporary, and explain why upgrading the weak handle inside the `new_cyclic` closure returns `None`.

The problem `new_cyclic` solves

Sometimes a value needs a non-owning handle to its own `Arc` allocation — a classic example is a node that stores `me: Weak<Node>` for callbacks, or a child that should hold `parent: Weak<Node>` while the parent is still being published as an `Arc`.

The awkward pattern without `new_cyclic` looks like:

```
// Awkward: temporary None, then fix up - often requires interior mutability.
struct Node {
    me: std::sync::Mutex<Weak<Node>>,
}
```

Or splitting construction across `OnceLock` fields. `Arc::new_cyclic` gives you a `Weak<T>` *while* the `T` is being built, then finishes the `Arc` in one step.

```
pub fn new_cyclic<F>(data_fn: F) -> Arc<T>
where
    F: FnOnce(&Weak<T>) -> T;
```

`Rc::new_cyclic` is the single-threaded twin in `std::rc`.

Rules inside the closure

1. The closure receives `&Weak<T>` for the allocation under construction.
2. **`upgrade()` inside the closure returns `None`.** The strong `Arc` is not finished yet.
3. After `new_cyclic` returns, that same `Weak` upgrades normally.
4. Storing the `Weak` in the struct is the usual goal; cloning it is cheap.

Lab: self weak and parent weak

```
use std::sync::{Arc, Weak};

#[derive(Debug)]
struct Node {
    name: &'static str,
    me: Weak<Node>,
    parent: Weak<Node>,
}

fn main() {
    let root: Arc<Node> = Arc::new_cyclic(|w| Node {
        name: "root",
        me: w.clone(),
        parent: Weak::new(),
    });
    println!(
        "root={} strong={} weak={} me_upgrades={}",
        root.name,
        Arc::strong_count(&root),
        Arc::weak_count(&root),
        root.me.upgrade().is_some()
    );

    let child: Arc<Node> = Arc::new_cyclic(|w| Node {
        name: "child",
        me: w.clone(),
        parent: Arc::downgrade(&root),
    });
    println!(
        "child={} parent={:?} strong={} weak={}",
        child.name,
        child.parent.upgrade().map(|p| p.name),
        Arc::strong_count(&child),
        Arc::weak_count(&child)
    );

    let weak_root = Arc::downgrade(&root);
    drop(root);
    println!(
        "after drop root: weak_root alive={}",
        weak_root.upgrade().is_some()
    );
    println!(
        "child parent still? {:?}",
```

```
        child.parent.upgrade().map(|p| p.name)
    );
}
```

```
rustc cyclic.rs -o cyclic
./cyclic
```

Verified with `rustc 1.99.1`:

```
root=root strong=1 weak=1 me_upgrades=true
child=child parent=Some("root") strong=1 weak=1
after drop root: weak_root alive=false
child parent still? None
```

Reading the counters:

- After construction, `root` has one strong owner and one weak (`me`).
- `me.upgrade()` succeeds outside the closure.
- `child.parent` is a weak link to `root`. Dropping the last strong `root` frees it; the child's weak parent no longer upgrades. There is **no strong cycle**, so memory is reclaimed.

Strong cycles versus weak parents

If both parent and child stored `Arc<Node>` to each other, `strong_count` would never reach zero and the allocation would leak for the lifetime of the process. The fix is the same as in the weak-reference material: own one direction with `Arc`, point back with `Weak`. `new_cyclic` does not change that rule; it only removes the bootstrap temporary when the weak handle must exist *during* `T`'s construction.

When to use `new_cyclic`

Situation	Approach
Field needs `Weak<Self>` at birth	`Arc::new_cyclic` / `Rc::new_cyclic`
Init needs fallible work or arguments later	Build with `OnceLock` / builder, then `Arc::new`
Shared state without self-weak	Plain `Arc::new`
Single-threaded graph	Prefer `Rc::new_cyclic` (no atomics)

Pitfalls

- Calling `upgrade().unwrap()` inside the closure — it is always `None` there.
- Creating a strong cycle by also storing `Arc` edges both ways "just in case".

- Forgetting that `Weak::new()` is a dangling weak (always upgrades to `None`), distinct from a weak produced by `new_cyclic` or `downgrade`.
- Expecting `Drop` of `T` to run while weak handles still exist — weak handles do not keep `T` alive; after the last strong ref drops, upgrades fail and the weak slots hang until dropped.

Single-threaded twin: `Rc::new_cyclic`

```
use std::rc::{Rc, Weak};

struct Tick {
    me: Weak<Tick>,
    n: u32,
}

fn main() {
    let t = Rc::new_cyclic(|w| Tick { me: w.clone(), n: 1 });
    assert!(t.me.upgrade().is_some());
    assert_eq!(Rc::strong_count(&t), 1);
    println!("n={}", t.n);
}
```

Prefer `Rc` when the graph never crosses threads — atomic refcounts in `Arc` are unnecessary cost. The closure rules are identical: no successful `upgrade` during construction.

Design sketch: arena of nodes

For larger graphs, many codebases store nodes in a `Vec` or slab and use indices instead of `Arc`. That avoids reference-count cycles entirely. Reach for `new_cyclic` when shared ownership is already the API (`Arc<Node>` handed to tasks) and a weak self/parent handle is required at birth. Do not introduce `Arc` solely to call `new_cyclic`.

Testing upgrades

Assert `upgrade().is_none()` only if you are inside a controlled constructor test that still runs under the closure — usually you test *after* construction that `me.upgrade()` is `Some` and that dropping strong owners makes weak upgrades fail, as in the lab.

Summary

`Arc::new_cyclic` installs a `Weak<T>` before the strong `Arc<T>` exists, then hands you a fully formed shared owner. Use it for self-referential shared nodes and parent weak links; keep strong edges acyclic; leave Householder-level graph ownership questions to the weak-reference chapter's design guidance.

Sources

- Arc::new_cyclic
- Rc::new_cyclic
- Weak

Summary

Summary

Chapter Key Takeaways

- `Box<T>` allocates on the heap with single ownership.
- `Rc<T>` and `Arc<T>` provide shared read ownership via reference counting.
- `RefCell<T>` provides interior mutability with runtime borrow checks.
- `Mutex<T>` and `RwLock<T>` provide thread-safe interior mutability.
- `Weak<T>` prevents reference count cycles.
- `OnceLock` / `LazyLock` (and `OnceCell` / `LazyCell`) cover one-time initialization without third-party crates.
- `MaybeUninit<T>` allows working with uninitialized memory safely before type invariants are established.

Iterators, Closures, and Functional Patterns

The Iterator Trait and Protocol

The Iterator Trait and Protocol

In Rust, iterators are lazy sequences that produce values on demand via the `Iterator` trait:

```
fn main() {
    let v = vec![1, 2, 3];
    let mut iter = v.iter();

    assert_eq!(iter.next(), Some(&1));
    assert_eq!(iter.next(), Some(&2));
    assert_eq!(iter.next(), Some(&3));
    assert_eq!(iter.next(), None);
}
```

Creating Custom Iterators

Creating Custom Iterators

```
struct Counter {
    count: usize,
}

impl Counter {
    fn new() -> Self { Self { count: 0 } }
}

impl Iterator for Counter {
    type Item = usize;
    fn next(&mut self) -> Option<Self::Item> {
        if self.count < 3 {
            self.count += 1;
            Some(self.count)
        } else {
            None
        }
    }
}

fn main() {
    let counter = Counter::new();
    let items: Vec<usize> = counter.collect();
    println!("Custom iterator output: {:?}", items);
}
```

Closures and Capture Semantics

Closures and Capture Semantics

Closures are anonymous functions that can capture environment variables: - `Fn`: Borrows values from the environment immutably. - `FnMut`: Borrows values mutably. - `FnOnce`: Takes ownership and consumes captured values.

```
fn main() {
    let factor = 3;
    let multiply = |x| x * factor;
    println!("Result: {}", multiply(10));
}
```

Higher-Order Functions and Combinators

Higher-Order Functions and Combinators

```
fn main() {
    let numbers = vec![1, 2, 3, 4, 5, 6];

    let sum_of_even_squares: i32 = numbers
        .iter()
        .filter(|&&x| x % 2 == 0)
        .map(|&x| x * x)
        .sum();

    println!("Sum of even squares: {}", sum_of_even_squares);
}
```

Lazy Evaluation and Performance

Lazy Evaluation and Performance

Rust iterator adaptors are **zero-cost abstractions**. They compile down to unrolled assembly with no heap allocation, often outperforming hand-written imperative loops.

FusedIterator — Why Fuse Matters

FusedIterator — Why Fuse Matters

Thesis

`FusedIterator` marks iterators that keep returning `None` forever after the first `None`. Adapters like `fuse()` wrap a flaky iterator so callers (and `Iterator` combinators) can assume fusion.

Mental model

```
Ordinary iterator:  Some, Some, None, Some?  ← "zombie" possible
Fused:              Some, Some, None, None, None, …
```

`Iterator::fuse` returns `Fuse<Self>` implementing `FusedIterator`.

Worked example

```
// fused_demo.rs
use std::iter::{Fuse, FusedIterator};

struct OnceThenWeird {
    state: u8,
}
impl Iterator for OnceThenWeird {
    type Item = i32;
    fn next(&mut self) -> Option<i32> {
        self.state += 1;
        match self.state {
            1 => Some(10),
            2 => None,
            3 => Some(99), // zombie value - bad manners
            _ => None,
        }
    }
}

fn main() {
    let mut raw = OnceThenWeird { state: 0 };
```

```
    println!("raw: {:?} {:?} {:?} {:?}", raw.next(), raw.next(), raw.next(), raw.next());

    let mut fused: Fuse<_> = (OnceThenWeird { state: 0 }).fuse();
    println!(
        "fused: {:?} {:?} {:?} {:?}",
        fused.next(),
        fused.next(),
        fused.next(),
        fused.next()
    );
    fn assert_fused<I: FusedIterator>(_: &I) {}
    assert_fused(&fused);
}
```

```
first pass: [1, 2, 3]
second pass: []
```

Trap

Implementing `FusedIterator` for a type that can revive. The marker is a contract; lying breaks `Flatten` / other adapters that skip re-polling exhausted inners incorrectly if the contract fails. Prefer `.fuse()` at the boundary over claiming the trait yourself unless proven.

Boring rule

If you hand an iterator to combinators after a possible early `None`, call `.fuse()`. Implement `FusedIterator` only when your `next` is proven fused.

Try this

1. Read the `Fuse` source — it stores a flag after first `None`.
2. Check which std iterators already implement `FusedIterator`.
3. Break `Flatten` intentionally with a zombie iterator (in a toy) to feel why fusion matters.

Sources

- `std::iter::FusedIterator`
- `Iterator::fuse`

Fast Float Reductions: Algebraic Operations vs Explicit Lanes

Fast Float Reductions: Algebraic Operations vs Explicit Lanes

After this page you can explain why `iter().map(|(x, y)| x * y).sum()` over `f32` runs as a slow scalar loop even at `-O3`, speed it up with the `algebraic_*` float methods stabilized in **Rust 1.98**, and decide when to write explicit accumulator lanes instead. Both run at about the same speed, but only the lanes give the same bits on every build.

Thesis

IEEE 754 addition isn't associative, so `(a + b) + c` and `a + (b + c)` can round differently. Rust's `+` on floats promises the exact IEEE result of the expression as written. That forbids the reordering a vectorizer needs, so a float reduction becomes one long dependency chain. `f32::algebraic_add`/`algebraic_mul` (and `_sub`, `_div`, `_rem`, likewise on `f64`) lift that promise *per operation*. The standard library docs say they "allow the compiler to optimize floating point operations using all the usual algebraic properties of real numbers," and also that "algebraic operations are non-deterministic." You get the speed and give up reproducibility. Writing the reassociation yourself gets you the speed and keeps reproducibility.

Mental model

The docs' own example is that `a.algebraic_add(b).algebraic_add(c).algebraic_add(d)` may be computed as written or as `(a + c) + (b + d)`. The rewrites allowed are deliberately unspecified. Typically they include reassociation, contraction into fused multiply-add, replacing division with reciprocal multiplication, and ignoring the sign of zero. NaN, ±Inf and `-0.0` "may behave in unexpected ways." The results are always valid floats and never undefined behavior, but unsafe code must not depend on any property of them.

Worked example: three dot products

```
use std::hint::black_box;
use std::time::Instant;

#[inline(never)]
pub fn dot_strict(a: &[f32], b: &[f32]) -> f32 {
    a.iter().zip(b).fold(0.0, |acc, (x, y)| acc + x * y)
```

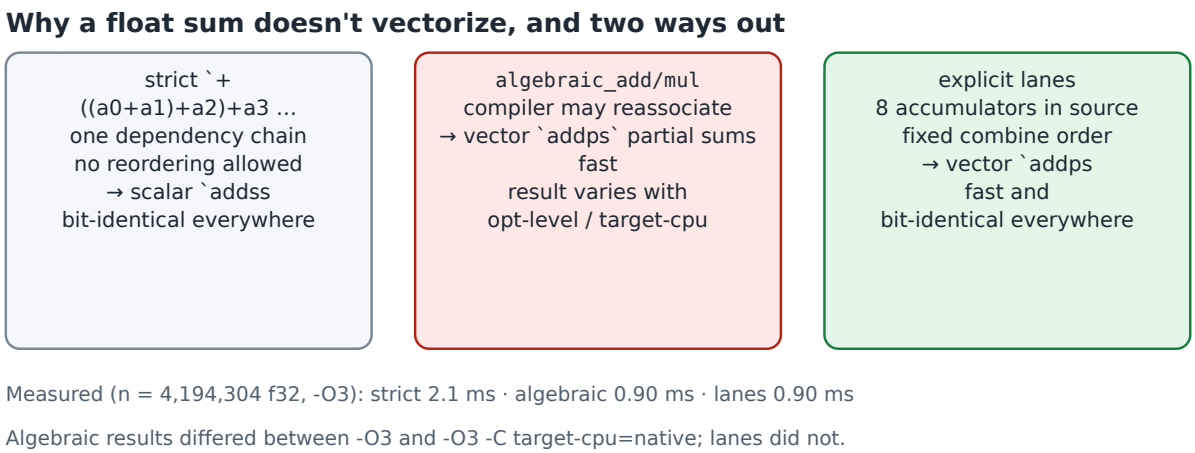


Figure 1: Strict addition is one serial chain emitted as scalar addss; algebraic_add lets the compiler build vector partial sums but the result varies with opt level and target; explicit lanes vectorize too and give identical bits everywhere

```
}

#[inline(never)]
pub fn dot_algebraic(a: &[f32], b: &[f32]) -> f32 {
    a.iter()
        .zip(b)
        .fold(0.0, |acc, (x, y)| acc.algebraic_add(x.algebraic_mul(*y)))
}

/// Deterministic alternative: eight explicit accumulators, combined in a
/// fixed order. Same result at every opt level and target; still vectorizes.
#[inline(never)]
pub fn dot_lanes(a: &[f32], b: &[f32]) -> f32 {
    let mut acc = [0.0f32; 8];
    let (ca, cb) = (a.chunks_exact(8), b.chunks_exact(8));
    let tail: f32 = ca.remainder().iter().zip(cb.remainder()).map(|(x, y)| x * y).sum();
    for (xa, xb) in ca.zip(cb) {
        for i in 0..8 {
            acc[i] += xa[i] * xb[i];
        }
    }
    ((acc[0] + acc[4]) + (acc[1] + acc[5])) + ((acc[2] + acc[6]) + (acc[3] + acc[7])) + tail
}

#[inline(never)]
pub fn dot_f64(a: &[f32], b: &[f32]) -> f64 {
    a.iter().zip(b).fold(0.0, |acc, (x, y)| acc + (*x as f64) * (*y as f64))
```

```
}

pub fn time(name: &str, f: impl Fn() -> f32, reference: f64) {
    let mut best = f64::MAX;
    let mut r = 0.0;
    for _ in 0..20 {
        let t = Instant::now();
        r = black_box(f());
        best = best.min(t.elapsed().as_secs_f64());
    }
    println!(
        "{name:<10} {r:>14.6}  rel.err {:>9.2e}  best {:>7.3} ms",
        ((r as f64 - reference) / reference).abs(),
        best * 1e3
    );
}

fn main() {
    let n = 1 << 22;
    // Deterministic pseudo-random data in [0, 1).
    let mut s: u32 = 12345;
    let mut next = || {
        s = s.wrapping_mul(1664525).wrapping_add(1013904223);
        (s >> 8) as f32 / (1u32 << 24) as f32
    };
    let a: Vec<f32> = (0..n).map(|_| next()).collect();
    let b: Vec<f32> = (0..n).map(|_| next()).collect();
    let reference = dot_f64(&a, &b);
    println!("n = {n}, f64 reference = {reference:.6}");
    time("strict", || dot_strict(black_box(&a), black_box(&b)), reference);
    time("algebraic", || dot_algebraic(black_box(&a), black_box(&b)), reference);
    time("lanes", || dot_lanes(black_box(&a), black_box(&b)), reference);
}
```

Verified with **rustc 1.99.1 (58cf8f9c3 2026-10-13)** on x86-64 (`Intel(R) Xeon(R) Processor`, 8 vCPUs). The same program was built three ways:

```
for o in "-C opt-level=0" "-C opt-level=3" "-C opt-level=3 -C target-cpu=native"; do
  echo "== rustc $o"; rustc $o --edition 2024 dot.rs -o dotx && ./dotx | tail -3
done
```

```
== rustc -C opt-level=0
strict     1046494.187500  rel.err   1.92e-3  best  40.257 ms
algebraic  1046494.187500  rel.err   1.92e-3  best  38.999 ms
lanes      1048457.937500  rel.err   4.60e-5  best  29.115 ms
== rustc -C opt-level=3
strict     1046494.187500  rel.err   1.92e-3  best   2.100 ms
```

```
algebraic  1048457.937500  rel.err   4.60e-5  best   0.899 ms
lanes      1048457.937500  rel.err   4.60e-5  best   0.901 ms
== rustc -C opt-level=3 -C target-cpu=native
strict     1046494.187500  rel.err   1.92e-3  best   2.100 ms
algebraic  1048502.125000  rel.err   3.84e-6  best   1.003 ms
lanes      1048457.937500  rel.err   4.60e-5  best   1.010 ms
```

Here's what that shows:

- **At `-O3` the algebraic version is ~2.3× faster than strict** and as fast as the hand-written lanes. (Timings are best of 20 on a shared VM, so treat them as ratios, not absolutes.)
- **The algebraic result depends on the build.** Unoptimized, it matches strict bit for bit. At `-O3` it happens to match the 8-lane order. With `target-cpu=native` (AVX-capable here) the compiler picked a different reduction tree and produced a third value. All three are "correct" under the docs' rules.
- **The lanes result is the same in all three builds**, because the combine order is written in the source and `+` may not be reordered.
- **Here, reassociation also made the result *more* accurate.** A single accumulator grows to about 10 and then adds terms near 0.25, losing their low bits. Several smaller partial sums lose less. That's a property of this data (all positive), not a guarantee. With cancelling terms, a reordered sum can just as easily be worse.

The assembly confirms the vectorization (`--crate-type=lib --emit asm`, then counting arithmetic instructions in each function body):

```
rustc -C opt-level=3 --edition 2024 --crate-type=lib -A dead_code --emit asm -o dot.s dot.rs
```

```
dot_strict     addss=5 mulss=5
dot_algebraic  addps=4 addss=1 mulps=2 mulss=1
dot_lanes      addps=7 addss=11 mulps=6 mulss=7
```

`ss` is scalar single precision and `ps` is packed. The strict loop is unrolled five times but is still one scalar `addss` chain. The other two run packed `addps` loops, and their remaining scalar instructions are the tail and the final horizontal sum.

Choosing between them

Need	Use
Bit-identical results across builds, targets and runs (tests with exact expectations, consensus, replay, deterministic simulation)	strict `+`, or explicit lanes when you need speed
Fastest reduction in a hot loop, tolerance-based checks, results that may differ between machines	`algebraic_*`

Need	Use
A specific vector width or FMA on purpose	explicit lanes, or `f32::mul_add` (which is exact fused multiply-add, not "maybe")

The algebraic methods are opt-in per operation. Unlike C's `-ffast-math`, they don't change the semantics of every float operation in the crate, and they don't add `-ffinite-math-only`'s assumption that NaNs and infinities never occur, the assumption that lets a compiler delete `isnan` checks. Only the operations you mark can be rewritten.

Things that bite

- **`assert_eq!` on an algebraic result is a flaky test.** The docs say two evaluations of the same expression may differ "even within a single program run." Compare with a tolerance, or test the strict reference.
- **It's all or nothing per chain.** One strict `+` in the middle of the fold restores the dependency chain at that point. Both the add and the multiply in the fold above are algebraic.
- **`Iterator::sum` stays strict.** There's no algebraic `sum()`, so write the `fold` (or the lanes) yourself.
- **`const` contexts work too** (the methods are `const` since 1.98), but the docs promise nothing more about their compile-time results either.

Try this

1. Change the data generator to produce values in `[-1, 1)`. Does the algebraic sum stay more accurate than strict, and does lanes?
2. Make only the multiply algebraic (`acc + x.algebraic_mul(*y)`). Does it still vectorize? Check the asm counts.
3. Change `dot_lanes` to 16 accumulators and compare `-O3` with `-O3 -C target-cpu=native`. Does the result change between builds? Why not?

Sources

- `f32` algebraic operators and `algebraic_add` (stable since 1.98.0): https://doc.rust-lang.org/std/primitive.f32.html#algebraic-operators
- `f64::algebraic_add`: https://doc.rust-lang.org/std/primitive.f64.html#method.algebraic_add
- `f32::mul_add`: https://doc.rust-lang.org/std/primitive.f32.html#method.mul_add
- `std::hint::black_box`: https://doc.rust-lang.org/std/hint/fn.black_box.html
- `slice::chunks_exact`: https://doc.rust-lang.org/std/primitive.slice.html#method.chunks_exact
- Rust 1.98.0 release notes: https://doc.rust-lang.org/beta/releases.html

Copy Range Types: core::range and Why a..b Was Never Copy

Copy Range Types: core::range and Why a..b Was Never Copy

After this page you can explain why `std::ops::Range` cannot implement `Copy`, use the new `core::range` types stabilized in Rust 1.95.0 and 1.96.0 to store copyable range spans in your structs, convert smoothly between legacy and modern ranges, index slices directly with them, and design clean APIs that accept both.

The problem: structs, spans, and error E0204

Engineers who build parsers, compilers, text editors, and binary protocols frequently model subslices and byte spans. A span is conceptually trivial: a pair of indices marking the beginning and end of a token or buffer region. Because a pair of `usize` values is small (typically 16 bytes on a 64-bit target) and possesses pure value semantics, such descriptors should naturally implement `Copy`.

For years, the intuitive way to declare such a span in Rust was to reuse the standard library range type:

```
#[derive(Clone, Copy, Debug)]
pub struct SourceSpan {
    pub range: std::ops::Range<usize>,
}

fn main() {
    let span = SourceSpan { range: 0..10 };
    let copy = span;
    println!("{copy:?}");
}
```

Attempting to compile this code with `rustc --edition 2024 span_error.rs` fails immediately with compiler error `E0204`:

```
error[E0204]: the trait `Copy` cannot be implemented for this type
 --> span_error.rs:2:12
  |
1 | #[derive(Clone, Copy, Debug)]
  |                 ---- in this derive macro expansion
2 | pub struct SourceSpan {
  |            ^^^^^^^^^^
```

```
3 |     pub range: std::ops::Range<usize>,
  |     ---------------------------------- this field does not implement `Copy`
```

Because `std::ops::Range<usize>` does not implement `Copy`, any structure containing it as a field cannot derive `Copy` either. For years, Rust programmers were forced to choose between annoying workarounds: 1. Reimplement custom pair types such as `struct Span { start: usize, end: usize }` and manually write conversions. 2. Store `(usize, usize)` tuples, losing the semantic intent and slice-indexing conveniences of range types. 3. Drop `Copy` from enclosing structs and sprinkle `.clone()` calls across every AST pass, token visitor, and slice utility.

To understand why Rust was designed this way, we have to examine the relationship between ranges and iterators in early Rust.

Why std::ops::Range was never Copy

In Rust 1.0, convenience won out over clean separation of concerns. To make `for i in 0..10` work without requiring explicit method calls like `(0..10).iter()`, `std::ops::Range<Idx>` was defined directly as an `Iterator`:

```
impl<Idx: Step> Iterator for std::ops::Range<Idx> {
    type Item = Idx;
    fn next(&mut self) -> Option<Idx> { ... }
}
```

In `std::ops::Range`, the range boundary descriptor and the iteration cursor are the exact same data structure. The `start` field serves as the moving pointer that is incremented on every invocation of `Iterator::next()`.

Making an active iterator `Copy` creates severe semantic traps. An iterator represents an ongoing traversal through a sequence. If an iterator were `Copy`, passing it by value to a helper function, evaluating it in an expression, or assigning it to a temporary variable would silently copy its internal state.

The small program below illustrates the hazard using explicit `Clone` on a partially consumed `std::ops::Range`:

```
fn main() {
    let mut original = 0..6;

    // Consume the first item: 0
    let first = original.next();

    // Fork the cursor state midway through iteration
    let mut forked = original.clone();

    // Advance both streams independently
    let second_original = original.next();
```

```
    let second_forked = forked.next();

    println!("original: first={first:?}, second={second_original:?}");
    println!("forked:   next={second_forked:?}");
}
```

Running `rustc --edition 2024 fork.rs && ./fork` produces:

```
original: first=Some(0), second=Some(1)
forked:   next=Some(1)
```

Notice what happened: both `original` and `forked` independently yield the number `1`. With an explicit `.clone()` call, this duplication of traversal state is visible in source code. But if `std::ops::Range` implemented `Copy`, every pass-by-value would implicitly duplicate the iteration cursor.

Consider an iterator consumer like `take_two(mut it: impl Iterator)`: passing an iterator into it would leave the caller's copy untouched at its prior position rather than advancing the caller's cursor. Because implicit cursor forking contradicts standard iterator expectations, Rust made the deliberate decision that types implementing `Iterator` should almost never implement `Copy`. Consequently, `std::ops::Range` could never be `Copy`.

The new types in core::range

The permanent solution is to separate the range descriptor (the geometric interval) from the iteration cursor (the transient traversal state). This design was codified in RFC 3550 and implemented in the standard library:

- **Rust 1.95.0** stabilized the module `core::range` along with `core::range::RangeInclusive` and `core::range::RangeInclusiveIter`.
- **Rust 1.96.0** stabilized `core::range::Range`, `RangeIter`, `RangeFrom`, `RangeFromIter` and `RangeToInclusive`. (`RangeToInclusive` has no start, so it has no iterator type.)
- The same types are re-exported as `std::range`, so `std::range::Range` and `core::range::Range` name one type.

These modern types represent pure bounds. When their underlying bound types implement `Copy` (such as `usize`), the range types themselves implement `Copy`.

Type	Legacy counterpart	Implements `Iterator`?	Implements `Copy`?	Public fields
`core::range::Range<T>`	`std::ops::Range<T>`	No (`IntoIterator`)	Yes (if `T: Copy`)	`start`, `end`
`core::range::RangeInclusive<T>`	`std::ops::RangeInclusive<T>`	No (`IntoIterator`)	Yes (if `T: Copy`)	`start`, `last`
`core::range::RangeFrom<T>`	`std::ops::RangeFrom<T>`	No (`IntoIterator`)	Yes (if `T: Copy`)	`start`
`core::range::RangeToInclusive<T>`	`std::ops::RangeToInclusive<T>`	No	Yes (if `T: Copy`)	`last`

Notice a crucial structural detail for `RangeInclusive`: in `core::range::RangeInclusive`, the upper boundary field is named `last`, reflecting that the bound is inclusive, while `core::range::Range` retains `start` and `end`.

IntoIterator vs Iterator: decoupling bounds from traversal

Because modern range types do not implement `Iterator`, they cannot be directly polled via `.next()`. Instead, they implement `IntoIterator`.

When you want an iterator, you produce one by value via `into_iter()` or by reference via `.iter()`:

```
use core::range::Range;

let r = Range { start: 0, end: 5 };

// Produce an iterator cursor explicitly
let mut iter = r.iter();
assert_eq!(iter.next(), Some(0));

// The original range r was NOT consumed or mutated because r is Copy!
assert_eq!(r.start, 0);
assert_eq!(r.end, 5);
```

The resulting iterator is `core::range::RangeIter<T>`, a dedicated cursor type. `RangeIter` implements `Iterator`, but `RangeIter` does **not** implement `Copy`. This cleanly separates responsibilities: - `Range<T>` is the immutable or copyable interval description. - `RangeIter<T>` is the mutable, non-copyable state machine driving traversal.

Because `IntoIterator` is the trait that powers Rust's `for` loops, range types can be passed directly to `for` loops without boilerplate:

```
let r = Range { start: 1, end: 4 };

for val in r {
    // Under the hood, this calls r.into_iter()
}

// Because r is Copy, r is still valid here!
let again = r;
```

This resolves the long-standing tension: intervals can be freely copied, while loops and iteration remain clean and safe.

Syntax and converting between legacy and new

In Rust edition 2024, built-in range syntax (`a..b`, `a..=b`, `a..`) continues to produce legacy `std::ops` types. Making `a..b` emit `core::range::Range` is a breaking change for code that calls `.next()` directly on `a..b` or passes `a..b` to functions expecting `std::ops::Range`.

A future edition is planned to make the standard range syntax produce the modern `core::range` types by default. Until that planned edition arrives, working with modern ranges requires explicit construction or conversion.

Converting from legacy to modern ranges

Modern ranges implement `From` for their corresponding legacy types:

```
use core::range::{Range, RangeFrom, RangeInclusive};

// From half-open syntax:
let r: Range<usize> = Range::from(0..10);

// From inclusive syntax:
let inc: RangeInclusive<usize> = RangeInclusive::from(1..=5);

// From unbounded-end syntax:
let from: RangeFrom<usize> = RangeFrom::from(100..);
```

You can also construct `core::range::Range` directly using struct syntax, since its fields are public:

```
let r = core::range::Range { start: 10, end: 20 };
```

Converting from modern ranges to legacy ranges

Modern ranges also implement `Into<std::ops::Range>`:

```
use core::range::Range;

let modern = Range { start: 5, end: 15 };

// Convert back to std::ops::Range
let legacy: std::ops::Range<usize> = modern.into();
assert_eq!(legacy, 5..15);
```

This bidirectional compatibility ensures you can adopt `core::range` in internal data structures without breaking downstream libraries that still require legacy ranges.

Slicing with core::range

Range types exist primarily for two tasks: iteration and slicing. The new types in `core::range` implement `core::slice::SliceIndex<[T]>`. That means you can pass them directly inside bracket indexing operators `&slice[r]` and `&mut slice[r]`:

```
use core::range::Range;

let numbers = [10, 20, 30, 40, 50];
let window = Range::from(1..4);

// Indexing works directly
let sub: &[i32] = &numbers[window];
assert_eq!(sub, &[20, 30, 40]);
```

You do not need to convert modern ranges back to `std::ops` ranges before indexing slices, strings, or vectors.

Worked example: a complete copyable span

The following standalone program demonstrates the complete lifecycle of modern ranges: deriving `Copy` on an enclosing data structure, bitwise copying, iterating via both `into_iter()` and `.iter()`, using `RangeInclusive::contains()`, consuming an unbounded `RangeFrom`, slicing a backing array, and converting back to legacy `std::ops::Range`.

```
use core::range::{Range, RangeFrom, RangeInclusive};

#[derive(Clone, Copy, Debug)]
struct Span {
    r: Range<usize>,
}

fn main() {
    // 1. Construct Span and demonstrate that it is Copy
    let span1 = Span {
        r: Range::from(0..4),
    };
    let span2 = span1; // Bitwise copy!

    println!("span1: {:?}, span2: {:?}", span1, span2);

    // 2. Iterate via into_iter()
    print!("span1.r.into_iter(): ");
    for x in span1.r.into_iter() {
        print!("{x} ");
    }
```

```
    println!();

    // 3. Iterate via iter()
    print!("span2.r.iter(): ");
    for x in span2.r.iter() {
        print!("{x} ");
    }
    println!();

    // 4. Use RangeInclusive::from(1..=3) with contains
    let inc = RangeInclusive::from(1..=3);
    println!("inc.contains(&2): {}", inc.contains(&2));
    println!("inc.contains(&4): {}", inc.contains(&4));

    // 5. Use RangeFrom::from(7..).iter().take(2)
    print!("RangeFrom::from(7..).iter().take(2): ");
    for x in RangeFrom::from(7..).iter().take(2) {
        print!("{x} ");
    }
    println!();

    // 6. Slice an array directly with the modern range
    let data = [100, 200, 300, 400, 500, 600];
    let subslice = &data[span1.r];
    println!("subslice: {:?}", subslice);

    // 7. Convert back to legacy std::ops::Range with .into()
    let legacy: std::ops::Range<usize> = span1.r.into();
    println!("converted to legacy: {:?}", legacy);
}
```

Running `rustc --edition 2024 span.rs && ./span` produces:

```
span1: Span { r: 0..4 }, span2: Span { r: 0..4 }
span1.r.into_iter(): 0 1 2 3
span2.r.iter(): 0 1 2 3
inc.contains(&2): true
inc.contains(&4): false
RangeFrom::from(7..).iter().take(2): 7 8
subslice: [100, 200, 300, 400]
converted to legacy: 0..4
```

Notice what to look for when running this binary: - `span1` and `span2` print identical range boundaries, confirming the copy succeeded without error `E0204`. - Both `into_iter()` and `iter()` visit the indices `0 1 2 3`. - `inc.contains(&2)` evaluates to `true`, while `inc.contains(&4)` evaluates to `false`. - `RangeFrom::from(7..)` generates numbers starting at 7, and `.take(2)` caps iteration at two items (7 and 8). - The subslice retrieves the first 4 elements from `data` (`[100, 200, 300,`

400]). - The legacy conversion prints `0..4` as an `std::ops::Range`.

API design guidance: when and how to expose modern ranges

When updating your libraries for modern Rust, follow these guidelines to balance ergonomics with backwards compatibility.

Store core::range in structs

Whenever a data structure needs to retain an interval—such as token spans in an AST, buffer view offsets, or memory windows—store `core::range::Range<usize>` instead of `std::ops::Range<usize>`. This allows your struct to derive `Copy`, `Eq`, and `Hash` whenever the index type supports them, removing the `.clone()` calls that a non-`Copy` field forces on every caller.

Accepting ranges in public functions

Because `a..b` still creates legacy ranges in edition 2024, requiring callers to write `Range::from(a..b)` at every call site is unergonomic. To accept both legacy syntax and modern ranges, write function signatures that accept `impl Into<core::range::Range<usize>>`:

```
use core::range::Range;

pub fn extract_region(data: &[u8], bounds: impl Into<Range<usize>>) -> &[u8] {
    let r: Range<usize> = bounds.into();
    &data[r]
}

fn demo() {
    let buf = b"hello world";

    // Call with legacy syntax:
    let _ = extract_region(buf, 0..5);

    // Call with modern Copy range:
    let modern = Range { start: 6, end: 11 };
    let _ = extract_region(buf, modern);
}
```

This pattern provides the best of both worlds: zero friction for callers using literal range syntax `0..5`, and zero friction for callers passing copyable `Range` fields stored in structs.

Pitfalls

1. Assuming range syntax emits modern ranges in edition 2024

Writing `let r = 0..10;` in Rust edition 2024 still produces `std::ops::Range<i32>`. Do not expect `r` to be `Copy`. If you need a copyable range, you must explicitly write `Range::from(0..10)` or construct `core::range::Range { start: 0, end: 10 }`. Range syntax will change only in a planned future edition.

2. Calling .next() directly on core::range::Range

Because modern ranges are not iterators, calling `r.next()` will fail to compile. You must obtain an iterator cursor first:

```
use core::range::Range;

let r = Range::from(0..5);

// Compiler error: no method named `next` found for struct `core::range::Range`
// r.next();

// Correct:
let mut iter = r.iter();
let first = iter.next();
```

3. Re-iteration behavior differences

With legacy ranges, passing an iterator into a loop consumes it permanently:

```
let mut legacy = 0..3;
for x in legacy.by_ref() { /* ... */ }
// legacy is now exhausted!
```

With modern ranges, calling `for x in r` passes `r` by value. Because `Range` implements `Copy`, the original variable `r` remains intact, and iterating over it a second time starts traversal fresh from `start`:

```
use core::range::Range;

let r = Range::from(0..3);
for _ in r {}
for x in r {
    // Executes again! Starts from 0.
}
```

While this matches user expectations for collections, programmers accustomed to legacy ranges may mistakenly expect `r` to be consumed.

4. Field names in RangeInclusive

In `core::range::RangeInclusive`, the upper boundary is stored in the public field `last`, not `end`:

```
use core::range::RangeInclusive;

let inc = RangeInclusive::from(1..=10);
assert_eq!(inc.start, 1);
assert_eq!(inc.last, 10); // Field is `last`, not `end`
```

Accessing `.end` on `RangeInclusive` will result in a field resolution error.

Summary

- `std::ops::Range` could never implement `Copy` because it implements `Iterator`. Copying an iterator implicitly would cause silent cursor forking and confusing traversal bugs.
- `core::range` and `RangeInclusive` were stabilized in **Rust 1.95.0**.
- `core::range::Range`, `RangeFrom`, `RangeToInclusive` and their iterator types were stabilized in **Rust 1.96.0** (and are re-exported in `std::range`).
- Modern range types represent pure bounds and implement `Copy` when their bound types implement `Copy`.
- Modern ranges do not implement `Iterator`. They implement `IntoIterator`; call `.iter()` or `.into_iter()` to create a non-copyable `RangeIter` cursor.
- Range syntax `a..b` still produces legacy `std::ops::Range` in edition 2024. A future edition is planned to transition range syntax to modern ranges.
- You can convert between legacy and modern ranges seamlessly using `Range::from(a..b)` and `.into()`.
- Modern ranges can directly index slices via `&slice[r]`.
- Store `core::range::Range` in custom types like spans and AST nodes to make them `Copy`, and accept `impl Into<Range<usize>>` in public function signatures for maximum caller convenience.

Sources

- core::range Module Documentation
- std::range::Range Struct Documentation
- Rust Releases Changelog
- RFC 3550: New Range Types

Summary

Summary

Chapter Key Takeaways

- Iterators are lazy; evaluation only happens when consumed (e.g., `collect`, `sum`).
- Closures capture environment variables via `Fn`, `FnMut`, or `FnOnce`.
- Combinators (`map`, `filter`, `fold`) form clear functional pipelines.
- Modern Rust compilers optimize iterator pipelines into highly vectorized machine loops.

Concurrency: Fearless Parallelism

Spawning Threads with std::thread

Spawning Threads with std::thread

Rust guarantees thread safety through its type system.

```
use std::thread;

fn main() {
    let handle = thread::spawn(|| {
        println!("Worker thread running");
    });

    handle.join().unwrap();
    println!("Worker thread joined");
}
```

Message Passing with Channels

Message Passing with Channels

"Do not communicate by sharing memory; instead, share memory by communicating."

```
use std::sync::mpsc;
use std::thread;

fn main() {
    let (tx, rx) = mpsc::channel();

    thread::spawn(move || {
        let msg = String::from("order_payload");
        tx.send(msg).unwrap(); // Ownership moves into channel!
    });

    let received = rx.recv().unwrap();
    println!("Received through channel: {}", received);
}
```

Shared State with Mutex and RwLock

Shared State with Mutex and RwLock

```
use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    let counter = Arc::new(Mutex::new(0));
    let mut handles = vec![];

    for _ in 0..5 {
        let counter_clone = Arc::clone(&counter);
        handles.push(thread::spawn(move || {
            let mut num = counter_clone.lock().unwrap();
            *num += 1;
        }));
    }

    for h in handles { h.join().unwrap(); }
    println!("Final count: {}", *counter.lock().unwrap());
}
```

RwLock Is Not a Faster Mutex: Lock Choice by Measurement, and Locks Across .await

RwLock Is Not a Faster Mutex: Lock Choice by Measurement, and Locks Across `.await`

Thesis

"Reads don't conflict, so use `RwLock` for read-heavy data" is half right. Readers don't *logically* exclude each other. But every `read()` still **writes** to the lock: it adds one to a shared reader count, and dropping the guard subtracts one. In the standard library's Linux implementation, that is a compare-exchange and a `fetch_sub` on one 32-bit word. With eight threads reading, that word's cache line moves between cores on every acquisition. When the work inside the lock is tiny, the line transfer *is* the cost. The readers queue for the cache line just as `Mutex` users queue for the lock, and adding threads makes total throughput **worse**.

`RwLock` pays off only when the read-side critical section is long enough to hide that traffic. If the data really is read-mostly and the section is tiny, the answer is to stop sharing the written word: use an atomic, or give each reader its own lock (a "big-reader" lock). This chapter measures all of these.

Async code has a second lock rule: never hold a `std::sync` guard across an `.await`. The compiler enforces it only when the future has to be `Send`. On a current-thread runtime the same mistake compiles and deadlocks.

Mental model

What one uncontended acquisition costs, from the standard library's futex-based implementation (`library/std/src/sys/sync/rwlock/futex.rs`):

Operation	Atomic work on the shared word	Notes
`RwLock::read()`	`load`, then `compare_exchange_weak(state, state + 1)`	falls into `read_contended()` (spin, then futex wait) if the CAS fails or a writer is waiting
drop of a read guard	`fetch_sub(1)`	wakes a writer if this was the last reader

Operation	Atomic work on the shared word	Notes
`Mutex::lock()` / drop	`compare_exchange(0, 1)` / `swap(0)`	waiters sleep on a futex
`AtomicU64::load`	none: a plain load	the line can stay shared in every core's cache

The std source also fixes the priority policy. `is_read_lockable` returns false whenever the writers-waiting bit is set, and a comment in the same function calls these "writers, which have priority over readers". So on Linux this `RwLock` is writer-preferring. That matters for the trap at the end of the chapter.

Worked examples

Built with **rustc/cargo 1.85.1** (the distro toolchain on the box), edition 2024, `--release`. The current stable Rust is **1.99.1** (2026-10-13). The `RwLock` and `Mutex` implementations quoted above are from the current `master` source. The machine is a shared 8-vCPU x86-64 VM (Intel Xeon, one thread per core). Absolute numbers will differ on your hardware, but the shape should not. Every benchmark reports the median of 5 runs, and each run measures the slowest thread.

```
cargo new --vcs none locks && cd locks
rm src/main.rs && mkdir -p src/bin
```

```
# Cargo.toml
[package]
name = "locks"
version = "0.1.0"
edition = "2024"
rust-version = "1.85"

[dependencies]

[profile.release]
debug = false
```

```
// src/lib.rs - the shared harness: N threads start together, each does `ops` operations
use std::hint::black_box;
use std::sync::{Arc, Barrier};
use std::thread;
use std::time::{Duration, Instant};

/// Runs `op` `ops` times on each of `threads` threads and returns ns per operation
/// for the slowest thread. Repeats the measurement 5 times and reports the median.
```

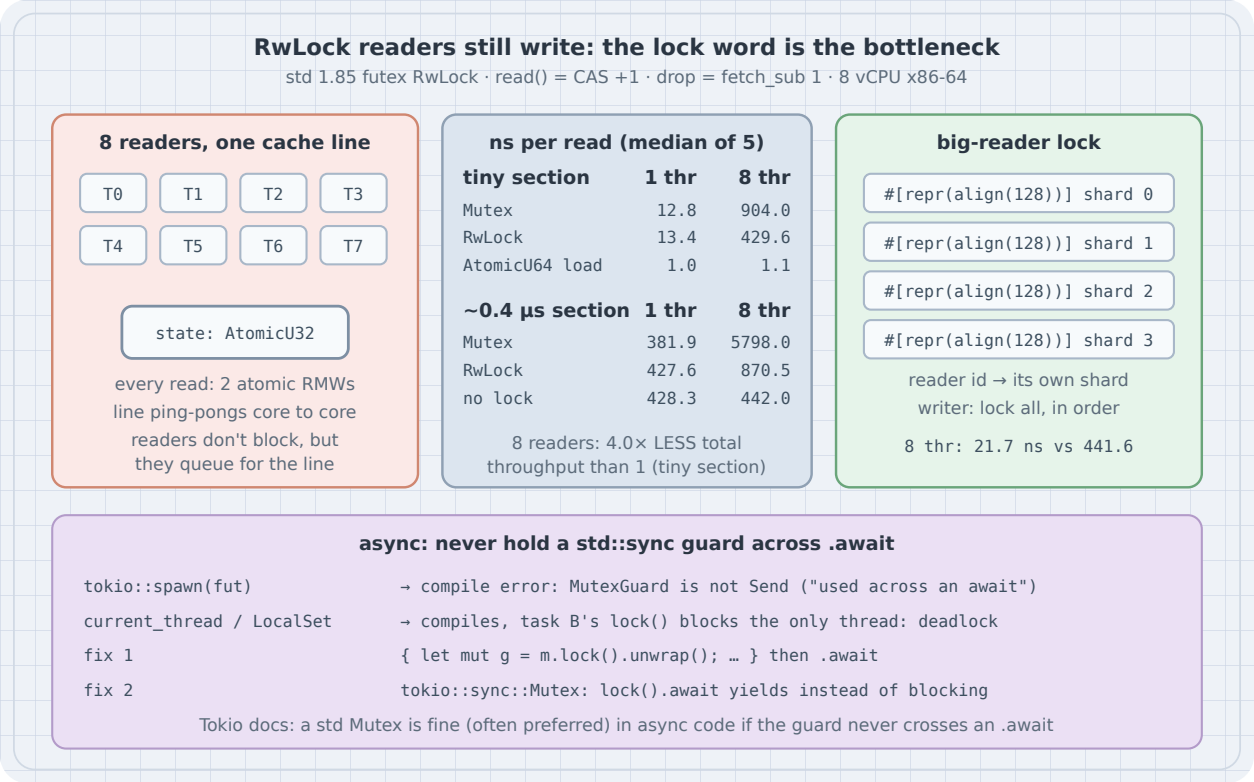


Figure 1: RwLock readers each do a CAS and a fetch_sub on one state word, so the cache line ping-pongs; measured ns per read for tiny and 0.4 µs sections; a big-reader lock with one aligned shard per reader; the async rule about std guards across .await

```
pub fn bench<T: Send + Sync + 'static>(threads: usize, ops: u64, shared: Arc<T>, op: fn(&T,
    let mut samples: Vec<f64> = (0..5)
        .map(|_| {
            let start = Arc::new(Barrier::new(threads + 1));
            let handles: Vec<_> = (0..threads)
                .map(|id| {
                    let (shared, start) = (Arc::clone(&shared), Arc::clone(&start));
                    thread::spawn(move || {
                        start.wait();
                        let t = Instant::now();
                        let mut acc = 0u64;
                        for i in 0..ops {
                            acc = acc.wrapping_add(op(&shared, id, i));
                        }
                        black_box(acc);
                        t.elapsed()
                    })
                })
                .collect();
            start.wait();
            let worst: Duration = handles.into_iter().map(|h| h.join().unwrap()).max().unwra
            worst.as_nanos() as f64 / ops as f64
        })
        .collect();
    samples.sort_by(|a, b| a.partial_cmp(b).unwrap());
    samples[2]
}
```

Case 1: A tiny read-only critical section

```
// src/bin/tiny.rs - read-only workload, tiny critical section: Mutex vs RwLock vs a plain a
use locks::bench;
use std::sync::atomic::{AtomicU64, Ordering};
use std::sync::{Arc, Mutex, RwLock};

const OPS: u64 = 1_000_000;

fn main() {
    println!("{:>7}  {:>11}  {:>12}  {:>12}", "threads", "Mutex ns/op", "RwLock ns/op", "ato
    for t in [1, 2, 4, 8] {
        let m = bench(t, OPS, Arc::new(Mutex::new(42u64)), |m, _, _| *m.lock().unwrap());
        let r = bench(t, OPS, Arc::new(RwLock::new(42u64)), |r, _, _| *r.read().unwrap());
        let a = bench(t, OPS, Arc::new(AtomicU64::new(42)), |a, _, _| a.load(Ordering::Acqui
        println!("{t:>7}  {m:>11.1}  {r:>12.1}  {a:>12.1}");
    }
```

```
}
```

```
$ cargo build --release && ./target/release/tiny
threads  Mutex ns/op  RwLock ns/op  atomic ns/op
      1         12.8          13.4           1.0
      2        135.1         104.2           1.1
      4        448.1         218.0           1.0
      8        904.0         429.6           1.1
```

Read the columns as *latency per operation for each thread*. One thread reading through `RwLock` costs 13.4 ns. Eight threads reading the *same* value take 429.6 ns per read each. Total throughput falls from 1/13.4 ns 75 M reads/s to 8/429.6 ns 19 M reads/s, so adding seven readers made the program **four times slower** overall. `RwLock` does beat `Mutex` here, because readers that win the CAS don't sleep. But neither lock scales. The `AtomicU64` column is the floor. A load writes nothing, so the cache line stays shared, and eight threads read in about 1 ns each.

Case 2: The same reads with real work inside

```
// src/bin/work.rs - the same read-only workload, but ~0.4 µs of work inside the lock
use locks::bench;
use std::hint::black_box;
use std::sync::{Arc, Mutex, RwLock};

const OPS: u64 = 20_000;

// A dependent chain of 256 multiply-xorshift steps over the table: it cannot be vectorized.
fn walk(table: &[u64]) -> u64 {
    let mut h = 0x9e37_79b9_7f4a_7c15u64;
    for &x in black_box(table) {
        h = (h ^ x).wrapping_mul(0xbf58_476d_1ce4_e5b9);
        h ^= h >> 29;
    }
    h
}

fn main() {
    let table: Vec<u64> = (0..256).collect();
    println!("{:>7}  {:>11}  {:>12}  {:>13}", "threads", "Mutex ns/op", "RwLock ns/op", "no
    for t in [1, 2, 4, 8] {
        let m = bench(t, OPS, Arc::new(Mutex::new(table.clone())), |m, _, _| walk(&m.lock().
        let r = bench(t, OPS, Arc::new(RwLock::new(table.clone())), |r, _, _| walk(&r.read()
        let n = bench(t, OPS, Arc::new(table.clone()), |v, _, _| walk(v));
        println!("{t:>7}  {m:>11.1}  {r:>12.1}  {n:>13.1}");
    }
}
```

```
$ ./target/release/work
threads  Mutex ns/op  RwLock ns/op  no lock ns/op
      1        381.9         427.6          428.3
      2       1053.5         473.7          410.5
      4       2456.9         514.7          423.9
      8       5798.0         870.5          442.0
```

Now each read holds the lock for about 0.4 µs (the "no lock" column). `Mutex` serialises everyone: at eight threads, each operation waits behind seven others, about 5.8 µs. `RwLock` lets the readers overlap, and its cost stays within about 2× of the lock-free version. This is the workload `RwLock` was designed for. The lock word still bounces, but a few hundred nanoseconds of work between bounces hides it.

The rule from Cases 1 and 2 is about the **ratio of critical-section length to cache-line transfer cost** (roughly 50–200 ns between cores). The read/write ratio alone doesn't decide it.

Case 3: One write in a hundred

```rust
// src/bin/mixed.rs - tiny critical section, one write in every 100 operations
use locks::bench;
use std::sync::{Arc, Mutex, RwLock};

const OPS: u64 = 500_000;

fn main() {
    println!("{:>7}  {:>11}  {:>12}", "threads", "Mutex ns/op", "RwLock ns/op");
    for t in [1, 2, 4, 8] {
        let m = bench(t, OPS, Arc::new(Mutex::new(0u64)), |m, _, i| {
            let mut g = m.lock().unwrap();
            if i % 100 == 0 {
                *g += 1;
            }
            *g
        });
        let r = bench(t, OPS, Arc::new(RwLock::new(0u64)), |r, _, i| {
            if i % 100 == 0 {
                let mut g = r.write().unwrap();
                *g += 1;
                *g
            } else {
                *r.read().unwrap()
            }
        });
        println!("{t:>7}  {m:>11.1}  {r:>12.1}");
    }
}
```

```
$ ./target/release/mixed
threads  Mutex ns/op  RwLock ns/op
      1         11.0          13.3
      2        149.5         109.8
      4        483.2         309.1
      8       1012.8         726.0
```

With one thread, `RwLock` is the slower lock (13.3 vs 11.0 ns). It does more bookkeeping, and nothing is gained without contention. Under contention it stays ahead of `Mutex` on this box, but both are about 55–90× slower per operation than the single-threaded case. If your profile shows time in `futex` or in `read_contended`, switching between `Mutex` and `RwLock` won't fix it. Changing what is shared will.

Case 4: Stop sharing the written word: a big-reader lock

The classic fix for read-mostly data with tiny sections is to give each reader its *own* lock. Readers write only to their own cache line, and the rare writer takes every lock:

```rust
// src/bin/sharded.rs - a "big-reader" lock: one RwLock per reader slot, writers take them a
use locks::bench;
use std::sync::{Arc, RwLock};

const OPS: u64 = 1_000_000;
const SHARDS: usize = 8;

// Align each shard to 128 bytes so readers of different shards never share a cache line
// (adjacent-line prefetch on x86 pulls lines in pairs).
#[repr(align(128))]
struct Shard(RwLock<u64>);

pub struct BigReader {
    shards: Vec<Shard>,
}

impl BigReader {
    pub fn new(v: u64) -> Self {
        BigReader { shards: (0..SHARDS).map(|_| Shard(RwLock::new(v))).collect() }
    }
    pub fn read(&self, reader: usize) -> u64 {
        *self.shards[reader % SHARDS].0.read().unwrap()
    }
    pub fn write(&self, v: u64) {
        // Every writer locks the shards in the same order, so writers cannot deadlock.
        let mut guards: Vec<_> = self.shards.iter().map(|s| s.0.write().unwrap()).collect();
        for g in guards.iter_mut() {
            **g = v;
```

```
        }
    }
}

fn main() {
    let big = BigReader::new(42);
    big.write(43);
    assert!((0..SHARDS).all(|i| big.read(i) == 43));

    println!("{:>7}  {:>14}  {:>13}", "threads", "1 RwLock ns/op", "sharded ns/op");
    for t in [1, 2, 4, 8] {
        let one = bench(t, OPS, Arc::new(RwLock::new(42u64)), |r, _, _| *r.read().unwrap());
        let sh = bench(t, OPS, Arc::new(BigReader::new(42)), |b, id, _| b.read(id));
        println!("{t:>7}  {one:>14.1}  {sh:>13.1}");
    }
}
```

```
$ ./target/release/sharded
threads  1 RwLock ns/op  sharded ns/op
      1            13.3           14.8
      2            99.1           14.8
      4           203.7           15.6
      8           441.6           21.7
```

At eight threads the sharded read costs 21.7 ns, against 441.6 ns for a single `RwLock`. That is about 20× better, and close to the single-threaded figure. The price is on the write side: a write takes `SHARDS` locks, and the structure uses `SHARDS × 128` bytes. The `#[repr(align(128))]` matters. Without it, several shards share a cache line, and you are back to Case 1. For a single `u64`, Case 1's atomic beats all of it. For a larger snapshot, the same idea appears in crates as "arc-swap"-style pointer swaps and per-CPU data. Those were not measured here.

Case 5: A `std::sync::MutexGuard` held across `.await`

The Tokio documentation is explicit: "Contrary to popular belief, it is ok and often preferred to use the ordinary Mutex from the standard library in asynchronous code." The condition is that the guard never lives across an `.await`. First, what happens when it does, under the multi-threaded `tokio::spawn`:

```
# Cargo.toml
[package]
name = "awaitlock"
version = "0.1.0"
edition = "2024"
rust-version = "1.85"

[dependencies]
```

```
tokio = { version = "1.53.3", features = ["rt", "rt-multi-thread", "macros", "time", "sync"]
```

```
// src/bin/not_send.rs - holding a std MutexGuard across .await inside tokio::spawn
use std::sync::{Arc, Mutex};
use std::time::Duration;

async fn bump(state: Arc<Mutex<u64>>) {
    let mut n = state.lock().unwrap();
    tokio::time::sleep(Duration::from_millis(1)).await; // guard still alive here
    *n += 1;
}

#[tokio::main]
async fn main() {
    let state = Arc::new(Mutex::new(0));
    tokio::spawn(bump(Arc::clone(&state))).await.unwrap();
    println!("n = {}", state.lock().unwrap());
}
```

```
$ cargo build --bin not_send
   Compiling awaitlock v0.1.0 (/tmp/rwlock/awaitlock)
error: future cannot be sent between threads safely
   --> src/bin/not_send.rs:14:18
    |
14  |     tokio::spawn(bump(Arc::clone(&state))).await.unwrap();
    |                  ^^^^^^^^^^^^^^^^^^^^^^^^ future returned by `bump` is not `Send`
    |
    = help: within `impl Future<Output = ()>`, the trait `Send` is not implemented for `std:
note: future is not `Send` as this value is used across an await
   --> src/bin/not_send.rs:7:50
    |
6   |     let mut n = state.lock().unwrap();
    |         ----- has type `std::sync::MutexGuard<'_, u64>` which is not `Send`
7   |     tokio::time::sleep(Duration::from_millis(1)).await; // guard still alive here
    |                                                  ^^^^^ await occurs here, with `mut n`
note: required by a bound in `tokio::spawn`
   --> /tmp/rwlock/.cargo-home/registry/src/index.crates.io-1949cf8c6b5b557f/tokio-1.53.3/sr
    |
174 |     pub fn spawn<F>(future: F) -> JoinHandle<F::Output>
    |            ----- required by a bound in this function
175 |     where
176 |         F: Future + Send + 'static,
    |                     ^^^^ required by this bound in `spawn`

error: could not compile `awaitlock` (bin "not_send") due to 1 previous error
```

This is the good outcome. `MutexGuard` is not `Send`, and the future keeps it alive across an `.await`, so the future is not `Send` and `tokio::spawn` refuses it. The same mistake on a single-threaded runtime compiles cleanly:

```
// src/bin/deadlock.rs - a std Mutex held across .await on a current-thread runtime
use std::sync::{Arc, Mutex};
use std::time::{Duration, Instant};

#[tokio::main(flavor = "current_thread")]
async fn main() {
    // Watchdog on a plain OS thread: if the runtime makes no progress, report and exit.
    let t0 = Instant::now();
    std::thread::spawn(move || {
        std::thread::sleep(Duration::from_secs(2));
        eprintln!("watchdog: no progress after {:?}: the runtime thread is blocked", t0.elap
        std::process::exit(1);
    });

    let state = Arc::new(Mutex::new(0u64));
    let local = tokio::task::LocalSet::new();
    local
        .run_until(async {
            let s1 = Arc::clone(&state);
            let a = tokio::task::spawn_local(async move {
                let mut n = s1.lock().unwrap();
                println!("task A: locked, now awaiting while holding the guard");
                tokio::time::sleep(Duration::from_millis(10)).await;
                *n += 1;
                println!("task A: done");
            });
            let s2 = Arc::clone(&state);
            let b = tokio::task::spawn_local(async move {
                println!("task B: calling lock() ...");
                *s2.lock().unwrap() += 1; // blocks the only runtime thread
                println!("task B: done");
            });
            a.await.unwrap();
            b.await.unwrap();
        })
        .await;
    println!("n = {}", state.lock().unwrap());
}
```

```
$ cargo build --bin deadlock && ./target/debug/deadlock; echo "exit $?"
task A: locked, now awaiting while holding the guard
task B: calling lock() ...
watchdog: no progress after 2.000204511s: the runtime thread is blocked
```

```
exit 1
```

`spawn_local` has no `Send` bound, so nothing stops the code from compiling. Task A locks the mutex and parks at the `sleep`. Task B calls the *blocking* `lock()` on the runtime's only thread, so the thread sleeps in the kernel, the timer never fires, and A can never release the guard. The same deadlock happens on a multi-threaded runtime once every worker is blocked this way. It also shows up with `block_on` or any `!Send` executor, and the compiler won't warn about any of them. Clippy's `await_holding_lock` lint flags this pattern. It was not run here, because the distro toolchain on the box ships without Clippy.

The two correct shapes:

```
// src/bin/fixed.rs - the two correct shapes: drop the std guard before .await, or use tokio
use std::sync::{Arc, Mutex};
use std::time::Duration;

// Shape 1: std Mutex, guard confined to a block that contains no .await.
async fn bump_std(state: Arc<Mutex<u64>>) -> u64 {
    let snapshot = {
        let mut n = state.lock().unwrap();
        *n += 1;
        *n
    }; // guard dropped here
    tokio::time::sleep(Duration::from_millis(10)).await;
    snapshot
}

// Shape 2: the lock really must be held across the .await, so use an async-aware mutex.
async fn bump_tokio(state: Arc<tokio::sync::Mutex<u64>>) -> u64 {
    let mut n = state.lock().await; // waits by yielding, not by blocking the thread
    tokio::time::sleep(Duration::from_millis(10)).await;
    *n += 1;
    *n
}

#[tokio::main(flavor = "current_thread")]
async fn main() {
    let s = Arc::new(Mutex::new(0u64));
    let (a, b) = tokio::join!(bump_std(Arc::clone(&s)), bump_std(Arc::clone(&s)));
    println!("std::sync::Mutex, guard scoped:   a = {a}, b = {b}, final = {}", s.lock().unwr

    let t = Arc::new(tokio::sync::Mutex::new(0u64));
    let (a, b) = tokio::join!(bump_tokio(Arc::clone(&t)), bump_tokio(Arc::clone(&t)));
    println!("tokio::sync::Mutex across .await: a = {a}, b = {b}, final = {}", t.lock().awai

    // And the multi-threaded runtime accepts both futures, because neither holds a std guar
    let h1 = tokio::spawn(bump_std(Arc::clone(&s)));
```

```
    let h2 = tokio::spawn(bump_tokio(Arc::clone(&t)));
    println!("spawned: {} {}", h1.await.unwrap(), h2.await.unwrap());
}
```

```
$ cargo build --bin fixed && ./target/debug/fixed
std::sync::Mutex, guard scoped:   a = 1, b = 2, final = 2
tokio::sync::Mutex across .await: a = 1, b = 2, final = 2
spawned: 3 3
```

Shape 1 confines the guard to a block with no `.await`, so it is dropped before the future can suspend. Shape 2 uses `tokio::sync::Mutex`, whose `lock().await` *yields* to the scheduler instead of blocking the thread. It is the right tool only when the lock really must be held across the await, because it costs more than the std mutex. Both futures are `Send`, which the last line proves by spawning them.

The trap

The trap with `RwLock` is assuming that read guards are free to nest because "readers don't block readers". The std documentation warns that a waiting writer "might or might not block concurrent calls to read", and the Linux implementation is writer-preferring. So a thread that already holds a read guard and asks for a second one can deadlock against a writer that queued in between:

```
// src/bin/reentrant.rs - taking a second read lock while a writer waits (std on Linux)
use std::sync::{Arc, RwLock};
use std::thread;
use std::time::Duration;

fn main() {
    // Watchdog: report and exit if the program is still stuck after 2 s.
    thread::spawn(|| {
        thread::sleep(Duration::from_secs(2));
        eprintln!("watchdog: still blocked after 2 s: deadlock");
        std::process::exit(1);
    });

    let lock = Arc::new(RwLock::new(0u64));
    let first = lock.read().unwrap();
    println!("main: holding read guard #1");

    let w = Arc::clone(&lock);
    let writer = thread::spawn(move || {
        println!("writer: waiting for write()");
        *w.write().unwrap() += 1;
        println!("writer: done");
    });
```

```
    thread::sleep(Duration::from_millis(100)); // let the writer queue up

    println!("main: taking read guard #2 ...");
    let second = lock.read().unwrap(); // a writer is waiting, so new readers wait too
    println!("main: got read guard #2 ({} {})", *first, *second);
    drop((first, second));
    writer.join().unwrap();
}
```

```
$ cargo build --release --bin reentrant && ./target/release/reentrant; echo "exit $?"
main: holding read guard #1
writer: waiting for write()
main: taking read guard #2 ...
watchdog: still blocked after 2 s: deadlock
exit 1
```

The writer waits for guard #1. Guard #2 waits behind the writer, because new readers are refused while a writer is waiting. Guard #1 is never dropped. This code works in every test that has no concurrent writer. In production it deadlocks the first time a writer arrives between the two `read()` calls. Code that calls a helper which takes the read lock again, while the caller already holds it, is the usual way to write this by accident.

The related performance trap is "switch `Mutex` to `RwLock` for read-heavy data" without measuring. Case 1 showed that this can leave you with eight threads doing a quarter of the work of one.

The boring rule

> Choose a lock by measuring it. Use `Mutex` by default. Use `RwLock` only when read sections are long, hundreds of nanoseconds or more (Case 2). For read-mostly data with tiny sections, stop sharing the written word: an atomic for a scalar, or per-reader shards for larger data. Never take a second read guard on the same `RwLock` while you hold one. In async code, a `std::sync::Mutex` is fine as long as the guard lives in a block with no `.await`. If it must cross an `.await`, use `tokio::sync::Mutex`. Remember that the compiler catches this only where `Send` is required.

Try this

1. Run `tiny` with `[1, 2, 3, 4, 5, 6, 7, 8]` threads and plot total throughput (`threads / ns_per_op`). Where does it peak for `RwLock`, and for `Mutex`?
2. In `work.rs`, shorten the walk (`0..32`, `0..64`, `0..128`) and find the section length at which `RwLock` stops scaling on your machine. Compare it with your core-to-core cache-line latency.
3. Remove `#[repr(align(128))]` from `Shard` and rerun `sharded`. Then try `align(64)`. Explain the difference, using adjacent-line prefetch.

4. Change `deadlock.rs` to `#[tokio::main(flavor = "multi_thread", worker_threads = 2)]` and replace `LocalSet`/`spawn_local` with `tokio::spawn`. What does the compiler say now, and which `.await` does it point at? Then move the `sleep` out of the guard's scope and confirm that it builds and finishes.
5. Replace the second `read()` in `reentrant.rs` with `try_read()`. What does it return while the writer waits, and how would you use that to detect recursion in a debug build?

Sources

- Rust standard library, `std::sync::RwLock` (priority policy, potential deadlock example): https://doc.rust-lang.org/std/sync/struct.RwLock.html
- Rust standard library, `std::sync::Mutex`: https://doc.rust-lang.org/std/sync/struct.Mutex.html
- Rust source, futex-based `RwLock` (`library/std/src/sys/sync/rwlock/futex.rs`) and `Mutex` (`library/std/src/sys/sync/mutex/futex.rs`): https://github.com/rust-lang/rust/tree/master/library/std/src/sys/sync
- Tokio, `tokio::sync::Mutex` ("Which kind of mutex should you use?"): https://docs.rs/tokio/latest/tokio/sync/struct.Mutex.html
- Tokio tutorial, Shared state ("Holding a `MutexGuard` across an `.await`"): https://tokio.rs/tokio/tutorial/shared-state
- Tokio, `task::spawn_local` and `LocalSet`: https://docs.rs/tokio/latest/tokio/task/fn.spawn_local.html
- Clippy lint `await_holding_lock`: https://rust-lang.github.io/rust-clippy/master/index.html#await_holding_lock
- Mara Bos, *Rust Atomics and Locks* (O'Reilly, 2023), ch. 9 "Building Our Own Locks" (reader-writer lock design and writer starvation): https://marabos.nl/atomics/
- Rust release channel manifest (stable 1.99.1, 2026-10-13): https://static.rust-lang.org/dist/channel-rust-stable.toml

Atomic Operations for Lock-Free Code

Atomic Operations for Lock-Free Code

For primitive integer updates, atomic instructions offer ultra-low latency lock-free synchronization:

```
use std::sync::atomic::{AtomicUsize, Ordering};
use std::sync::Arc;
use std::thread;

fn main() {
    let atomic_val = Arc::new(AtomicUsize::new(0));
    let mut handles = vec![];

    for _ in 0..4 {
        let val_clone = Arc::clone(&atomic_val);
        handles.push(thread::spawn(move || {
            val_clone.fetch_add(1, Ordering::SeqCst);
        }));
    }

    for h in handles { h.join().unwrap(); }
    println!("Atomic total: {}", atomic_val.load(Ordering::SeqCst));
}
```

Send and Sync Traits Explained

Send and Sync Traits Explained

Rust's type system encodes concurrency rules into two marker traits: - `Send`: Indicates ownership of the type can be transferred across thread boundaries. - `Sync`: Indicates it is safe to share references to the type across multiple threads (`T: Sync` implies `&T: Send`).

Summary

Summary

Chapter Key Takeaways

- Threads are spawned via `std::thread::spawn` with ownership transfer via `move`.
- Channels enable safe message passing without shared locks.
- `Arc<Mutex<T>>` provides safe shared mutable state across threads.
- `Send` and `Sync` automatically prevent data races at compile time.

Project 3: Parallel Data Processing Pipeline with rayon

Parallel Data Processing Pipeline with rayon

Parallel Data Processing Pipeline with rayon

In Project 3, we implement a high-throughput parallel text analysis pipeline using `rayon`.

```
// Parallel pipeline conceptual architecture
pub struct TextStats {
    pub word_count: usize,
    pub line_count: usize,
}

pub fn analyze_lines(lines: &[&str]) -> TextStats {
    // rayon would process using .par_iter()
    let line_count = lines.len();
    let word_count = lines.iter().map(|l| l.split_whitespace().count()).sum();
    TextStats { word_count, line_count }
}

fn main() {
    let sample = vec![
        "Rust delivers safe systems programming",
        "Rayon provides work-stealing parallelism",
        "Zero-cost abstractions minimize runtime overhead",
    ];
    let stats = analyze_lines(&sample);
    println!("Processed {} lines, {} words", stats.line_count, stats.word_count);
}
```

Testing the Text Analyzer

Testing the Text Analyzer

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_text_analysis() {
        let input = vec!["one two three", "four five"];
        let stats = analyze_lines(&input);
        assert_eq!(stats.line_count, 2);
        assert_eq!(stats.word_count, 5);
    }
}
```

Benchmarking the Text Analyzer

Benchmarking the Text Analyzer

Comparing sequential iteration vs. Rayon parallel iteration across multi-megabyte corpora shows linear scaling up to physical core count.

Deploying the Text Analyzer

Deploying the Text Analyzer

Package the parallel text analyzer as a standalone CLI utility:

```
cargo build --release
./target/release/text_analyzer --input large_corpus.txt
```

Asynchronous Programming with async/await

The Future Trait and Async Basics

The Future Trait and Async Basics

Asynchronous Rust uses cooperative multitasking where tasks yield execution rather than blocking threads:

```
// The core Future trait in std
// pub trait Future {
//     type Output;
//     fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Self::Output>;
// }

async fn compute_async_val() -> u32 {
    42
}
```

Running Async Code with Tokio

Running Async Code with Tokio

Async code requires an asynchronous executor runtime. Tokio is the de facto industry standard:

```
// Using tokio::main macro
// #[tokio::main]
// async fn main() {
//     let result = fetch_remote().await;
//     println!("Result: {}", result);
// }
```

Pinning and Self-Referential Types

Pinning and Self-Referential Types

`Pin` guarantees that an underlying value cannot be moved in memory. This is essential for state machines generated by `async/await` which contain self-referential pointers to local variables on their stack frame.

Async Streams and Error Handling

Async Streams and Error Handling

Streams are asynchronous iterators that yield values over time:

```
// Stream represents a stream of async items
// while let Some(item) = stream.next().await { ... }
```

Building Production Async Applications

Building Production Async Applications

Best practices for async architectures: - Never block async worker threads with long CPU-bound tasks (use `tokio::task::spawn_blocking`). - Always implement timeouts on network requests (`tokio::time::timeout`). - Implement graceful shutdown using cancellation tokens.

Tokio and Rayon: Separate Lanes for I/O and CPU Work

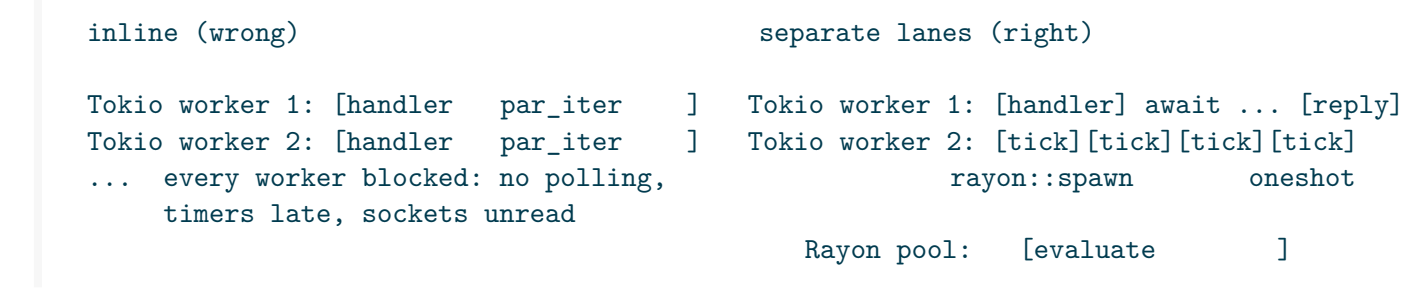
Tokio and Rayon: Separate Lanes for I/O and CPU Work

Thesis

Tokio and Rayon are both thread pools, and both size themselves to "one thread per CPU" by default. They do opposite jobs. Tokio's workers **wait** for many things at once and must come back to the scheduler quickly, at every `.await`. Rayon's workers **compute** and are expected to stay busy until the job is done. Call `par_iter()` from inside an async handler and a Tokio worker stops polling futures until Rayon finishes. Do that on enough workers at the same time and the whole service freezes. Timers fire late, health checks time out, and connection pools look exhausted even though the database is idle.

The fix is not to avoid either library. Give each its own lane: async code hands CPU work to Rayon with `rayon::spawn`, awaits the result on a `tokio::sync::oneshot` channel, and limits how many batches are in flight with a `Semaphore`. This chapter measures the difference with a heartbeat timer, shows how the thread budget changes under a CPU limit, and covers the panic case that turns a bad request into a dead process.

Mental model



Fact	Consequence
A Tokio worker runs one task until it returns `Poll::Pending`	Synchronous work between two `.await`s blocks every other task queued on that worker
`par_iter()` called outside the Rayon pool **blocks the calling thread** until the work is done	Inline Rayon in a handler is blocking code, however parallel it looks

Fact	Consequence
`rayon::spawn` queues a job and returns immediately	The handler can `.await` a channel and give its worker back to Tokio
Both pools default to `available_parallelism()` threads	Two default pools mean 2× oversubscription. `available_parallelism` honours CPU affinity and cgroup quotas, so it shrinks under a container CPU limit
Rayon's queue has no limit	Without a semaphore, a traffic burst becomes an ever-growing queue of CPU work

Worked examples

Tested with rustc 1.99.0, tokio 1.53.2 and rayon 1.12.0 on an 8-CPU Linux machine. Release builds throughout.

```
# Cargo.toml (dependencies)
[dependencies]
rayon = "1.12.0"
tokio = { version = "1.53.2", features = ["rt-multi-thread", "macros", "time", "sync"] }
```

Case 1: Same throughput, very different latency

The program runs 64 CPU-heavy "requests" twice: once calling Rayon inline in the handler and once offloaded. Meanwhile a task ticks every 5 ms, standing in for everything else the service must keep doing, and reports the worst lateness of any tick.

```
// src/main.rs - CPU work inside Tokio: blocking the workers vs. a separate Rayon lane
use rayon::prelude::*;
use std::sync::atomic::{AtomicBool, Ordering};
use std::sync::Arc;
use std::time::{Duration, Instant};
use tokio::sync::{oneshot, Semaphore};

/// Stand-in for a CPU-heavy request: pure computation, parallelised with Rayon.
fn evaluate(batch: &[u64]) -> u64 {
    batch
        .par_iter()
        .map(|&x| (0..20_000u64).fold(x, |a, i| a.wrapping_mul(6364136223846793005).wrapping
        .sum()
}

/// Wrong: call Rayon directly on a Tokio worker thread.
async fn handle_inline(batch: Arc<Vec<u64>>) -> u64 {
    evaluate(&batch)
}
```

```
/// Right: bound in-flight batches, hand the work to Rayon, await a oneshot.
async fn handle_offloaded(batch: Arc<Vec<u64>>, permits: Arc<Semaphore>) -> u64 {
    let _permit = permits.acquire_owned().await.expect("semaphore closed");
    let (tx, rx) = oneshot::channel();
    rayon::spawn(move || {
        let _ = tx.send(evaluate(&batch));
    });
    rx.await.expect("rayon job dropped its sender")
}

/// A 5 ms ticker standing in for health checks and socket polling.
/// Returns the worst lateness of any tick until `stop` is set.
async fn ticker(stop: Arc<AtomicBool>) -> Duration {
    let mut worst = Duration::ZERO;
    let mut next = tokio::time::Instant::now();
    while !stop.load(Ordering::Relaxed) {
        next += Duration::from_millis(5);
        tokio::time::sleep_until(next).await;
        worst = worst.max(tokio::time::Instant::now() - next);
    }
    worst
}

#[tokio::main]
async fn main() {
    let batch = Arc::new((0..2_000u64).collect::<Vec<_>>());
    let requests = 64;
    println!(
        "available_parallelism: {}, tokio workers: {}, rayon threads: {}",
        std::thread::available_parallelism().unwrap(),
        tokio::runtime::Handle::current().metrics().num_workers(),
        rayon::current_num_threads()
    );

    for mode in ["inline", "offloaded"] {
        let stop = Arc::new(AtomicBool::new(false));
        let tick = tokio::spawn(ticker(stop.clone()));
        let permits = Arc::new(Semaphore::new(2));
        let start = Instant::now();

        let mut set = tokio::task::JoinSet::new();
        for _ in 0..requests {
            let (b, p) = (batch.clone(), permits.clone());
            set.spawn(async move {
                match mode {
                    "inline" => handle_inline(b).await,
```

```
                    _ => handle_offloaded(b, p).await,
                }
            });
        }
        while set.join_next().await.is_some() {}

        let elapsed = start.elapsed();
        stop.store(true, Ordering::Relaxed);
        let lag = tick.await.unwrap();
        println!(
            "{mode:>9}: {requests} requests in {:>4} ms, worst tick lag {:>3} ms",
            elapsed.as_millis(),
            lag.as_millis()
        );
    }
}
```

```
cargo run --release
```

```
available_parallelism: 8, tokio workers: 8, rayon threads: 8
   inline: 64 requests in   62 ms, worst tick lag  52 ms
offloaded: 64 requests in   61 ms, worst tick lag   2 ms
```

Two more runs gave 55/2 ms and 47/2 ms. Total work time is the same either way, because the same CPUs do the same arithmetic. What changes is whether the runtime stays responsive. Inline, the 5 ms ticker was blocked for most of the batch: in a real service, that is a health check or a socket read waiting 50 ms. Offloaded, it never slipped more than 2 ms.

Case 2: Under a CPU limit

Restrict the process to two CPUs, as a container limit or `taskset` would:

```
taskset -c 0-1 ./target/release/lanes
```

```
available_parallelism: 2, tokio workers: 2, rayon threads: 2
   inline: 64 requests in  210 ms, worst tick lag 204 ms
offloaded: 64 requests in  212 ms, worst tick lag   3 ms
```

With two workers, inline Rayon froze the runtime for the **entire** batch. Both pools shrank to two threads because `available_parallelism()` follows the affinity mask (and on Linux the cgroup CPU quota, too). That is usually what you want, but it is computed from the *limit*, not the *request*. In Kubernetes, a pod with `requests: 2, limits: 8` will start eight threads per pool and get throttled in bursts. Set both pools explicitly:

```
// pools.rs - explicit thread budget for both lanes
fn main() {
    let cpus = std::env::var("CPU_BUDGET").ok().and_then(|v| v.parse().ok()).unwrap_or(2);

    rayon::ThreadPoolBuilder::new()
        .num_threads(cpus)                          // compute lane: all of the budget
        .thread_name(|i| format!("rayon-{i}"))
        .build_global()
        .expect("rayon pool already initialised");

    let rt = tokio::runtime::Builder::new_multi_thread()
        .worker_threads((cpus / 2).max(1))       // I/O lane: mostly parked in .await
        .enable_all()
        .build()
        .unwrap();
    rt.block_on(async { /* serve */ });
}
```

Mild oversubscription (Rayon gets the whole budget, Tokio half) is fine because Tokio's workers spend most of their time parked.

Case 3: Bound the work in flight

`handle_offloaded` takes a permit from a `Semaphore::new(2)` before it spawns. Rayon itself never refuses work, so the semaphore is the only thing that turns a burst into **async waiting** (cheap, cancellable, visible as a queue length you can measure and time out) instead of a growing pile of CPU jobs. Add a timeout around the whole handler to shed load:

```
match tokio::time::timeout(Duration::from_millis(250), handle_offloaded(b, p)).await {
    Ok(v) => Ok(v),
    Err(_) => Err("busy, try again"), // the permit was never taken or is released on drop
}
```

Small batches should skip Rayon entirely. Below a few hundred items, the cost of splitting and joining is larger than the work. Measure the threshold for your own data.

The trap

A panic in a `rayon::spawn` job is not your request's error. With `tokio::spawn`, a panic becomes a `JoinError` for that one task. With `rayon::spawn`, the default is that Rayon **aborts the process**. Run this without a panic handler and the output ends with `Rayon: detected unexpected panic; aborting`. In a server, that kills every in-flight request.

```
// examples/panic.rs - what a panic inside rayon::spawn looks like from the async side
use tokio::sync::oneshot;
```

```rust
#[tokio::main]
async fn main() {
    // Without a panic handler, a panic in rayon::spawn aborts the whole process.
    // Production pools install one; this is the usual "log and carry on" handler.
    rayon::ThreadPoolBuilder::new()
        .panic_handler(|_| eprintln!("rayon: job panicked"))
        .build_global()
        .unwrap();

    let (tx, rx) = oneshot::channel::<u64>();
    rayon::spawn(move || {
        let batch: Vec<u64> = Vec::new();
        let first = batch[0]; // bad input: index out of bounds
        let _ = tx.send(first);
    });

    match rx.await {
        Ok(v) => println!("result: {v}"),
        Err(e) => println!("request failed: {e}"),
    }
}
```

```
cargo run --release --example panic
```

```
thread '<unnamed>' (379384) panicked at examples/panic.rs:16:26:
index out of bounds: the len is 0 but the index is 0
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
rayon: job panicked
request failed: channel closed
```

With a `panic_handler` installed, the process survives. The sender is dropped while the job unwinds, so the handler sees `RecvError` ("channel closed") and can return a 500 for that one request. The handler must not use `.expect()` on the receiver, as Case 1's demo code does, because that just moves the panic into the Tokio task. The alternative is to catch the panic inside the job (`std::panic::catch_unwind`) and send a `Result` over the channel.

The boring rule

> Async handlers never call Rayon (or any long CPU loop) directly. They take a semaphore permit, `rayon::spawn` the work, and await a oneshot whose closed state is an error, not a panic. Both pools get explicit thread counts from the CPU *request*, Rayon has a panic handler, and small batches stay sequential.

Try this

1. Add a sequential threshold to `handle_offloaded` (`if batch.len() < 200 { return evaluate_seq(&batch) }`) and measure tick lag with 20-item batches. Is it acceptable to run that much work inline?
2. Replace `rayon::spawn` with `tokio::task::spawn_blocking(move || evaluate(&batch))`. Compare tick lag and total time, and count threads with `ls /proc/$(pidof lanes)/task | wc -l` during the run.
3. Set the semaphore to 64 permits and then to 1. Record total time and tick lag for each, and explain why neither number moves much while per-request latency does.
4. Wrap the job body in `catch_unwind` and send `Result<u64, String>` so the handler can report *which* input failed.

Sources

- Alice Ryhl, Async: What is blocking? (Tokio maintainer; the `rayon::spawn` + oneshot pattern)
- Tokio docs: `tokio::task` — blocking and yielding, `runtime::Builder::worker_threads`, `sync::Semaphore`, `sync::oneshot`
- Rayon docs: `rayon::spawn` (panic behaviour), `ThreadPoolBuilder::panic_handler`, `num_threads`
- Rust std: `std::thread::available_parallelism` (affinity and cgroup quotas)

Sans-IO Protocol Design: State Machines Without Sockets

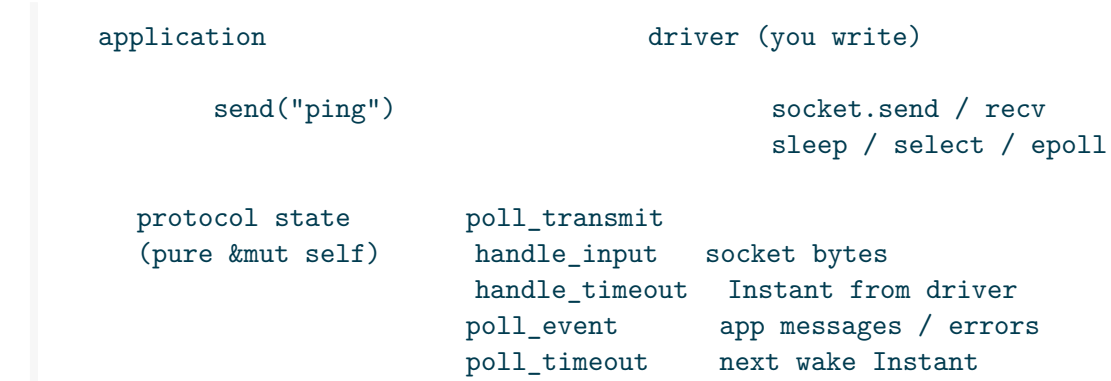
Sans-IO Protocol Design: State Machines Without Sockets

Thesis

A network protocol is a **state machine**. Sockets, timers, and async runtimes are **drivers**. Sans-IO (without I/O) keeps those layers apart: the protocol accepts bytes, clocks, and app intents through ordinary methods, and returns transmits and events through ordinary methods. No `UdpSocket`, no `tokio::spawn`, no `Instant::now()` hidden inside the library. The driver owns the socket and the clock; the protocol owns the rules.

That split buys three things operators and library authors care about. Tests run without ports or sleeps — feed a slice, advance a fake `Instant`, assert. The same crate works under Tokio, a blocking loop, or `no_std` with a custom timer. Composition becomes stacking machines (`handle_input` / `poll_transmit`) instead of colouring every call site `async`. This chapter builds a length-prefixed framer that way, proves it with byte-at-a-time reassembly and time travel, and names the traps that turn a clean machine into a busy-loop.

Mental model



Idea	Consequence
Protocol methods never block	`&mut self` is enough; no `Mutex`, no channels inside the machine
Time is an argument	Tests jump five seconds with `t0 + Duration::from_secs(5)`
Output is polled, not pushed onto a socket	One event loop can multiplex many protocols on one fd

Idea	Consequence
Incomplete input stays in a buffer	Drivers must keep calling `handle_input` / `handle_timeout` or progress stalls

The Python community documented the pattern at sans-io.readthedocs.io. In Rust it shows up in production stacks such as Quinn's `quinn-proto`, Cloudflare's `quiche`, Firezone's `connlib`, and the `sansio::Protocol` trait used by WebRTC.rs — different APIs, same inversion: **policy does not call I/O; I/O drives policy**.

Worked examples

Tested with **rustc 1.99.0** (Edition 2024) on x86_64 Linux. No crates beyond `std`.

Case 1: a length-prefixed framer with no sockets

Wire format: two big-endian length bytes, then that many payload bytes. The machine queues outbound frames, reassembles inbound chunks, and rejects declared lengths above a cap.

```
// src/main.rs - sans-IO length-prefixed framer
use std::collections::VecDeque;
use std::time::{Duration, Instant};

#[derive(Debug, Clone, PartialEq, Eq)]
struct Transmit {
    bytes: Vec<u8>,
}

#[derive(Debug, PartialEq, Eq)]
enum Event {
    Message(Vec<u8>),
    ProtocolError(&'static str),
    TimedOut,
}

struct Framer {
    rx_buf: Vec<u8>,
    tx: VecDeque<Transmit>,
    events: VecDeque<Event>,
    max_frame: usize,
    incomplete_since: Option<Instant>,
    stall_for: Duration,
}

impl Framer {
```

```
fn new(max_frame: usize) -> Self {
    Self {
        rx_buf: Vec::new(),
        tx: VecDeque::new(),
        events: VecDeque::new(),
        max_frame,
        incomplete_since: None,
        stall_for: Duration::from_secs(5),
    }
}

/// App intent: enqueue a framed transmit. No I/O.
fn send(&mut self, payload: &[u8]) -> Result<(), &'static str> {
    if payload.len() > self.max_frame {
        return Err("frame too large");
    }
    let len = u16::try_from(payload.len()).map_err(|_| "frame too large")?;
    let mut bytes = Vec::with_capacity(2 + payload.len());
    bytes.extend_from_slice(&len.to_be_bytes());
    bytes.extend_from_slice(payload);
    self.tx.push_back(Transmit { bytes });
    Ok(())
}

/// Driver feeds bytes read from somewhere (socket, test, file).
fn handle_input(&mut self, chunk: &[u8], now: Instant) {
    if !chunk.is_empty() {
        self.rx_buf.extend_from_slice(chunk);
        self.incomplete_since = Some(now);
    }
    self.drain_frames();
    if self.rx_buf.is_empty() {
        self.incomplete_since = None;
    }
}

fn drain_frames(&mut self) {
    loop {
        if self.rx_buf.len() < 2 {
            return;
        }
        let len = u16::from_be_bytes([self.rx_buf[0], self.rx_buf[1]]) as usize;
        if len > self.max_frame {
            self.events
                .push_back(Event::ProtocolError("declared length exceeds max_frame"));
            self.rx_buf.clear();
            return;
```

```
            }
            if self.rx_buf.len() < 2 + len {
                return;
            }
            let payload = self.rx_buf[2..2 + len].to_vec();
            self.rx_buf.drain(..2 + len);
            self.events.push_back(Event::Message(payload));
        }
    }

    fn poll_timeout(&self) -> Option<Instant> {
        self.incomplete_since.map(|t| t + self.stall_for)
    }

    fn handle_timeout(&mut self, now: Instant) {
        if let Some(start) = self.incomplete_since {
            if now >= start + self.stall_for && !self.rx_buf.is_empty() {
                self.events.push_back(Event::TimedOut);
                self.rx_buf.clear();
                self.incomplete_since = None;
            }
        }
    }

    fn poll_transmit(&mut self) -> Option<Transmit> {
        self.tx.pop_front()
    }

    fn poll_event(&mut self) -> Option<Event> {
        self.events.pop_front()
    }
}

fn main() {
    let t0 = Instant::now();

    let mut a = Framer::new(64);
    a.send(b"ping").unwrap();
    a.send(b"pong-extra").unwrap();
    let mut wire = Vec::new();
    while let Some(t) = a.poll_transmit() {
        wire.extend_from_slice(&t.bytes);
    }
    println!("wire bytes: {} {:?}", wire.len(), wire);

    // Reassemble one byte at a time - no socket required.
    let mut b = Framer::new(64);
```

```
    for byte in &wire {
        b.handle_input(&[*byte], t0);
    }
    while let Some(ev) = b.poll_event() {
        match &ev {
            Event::Message(p) => println!("event: Message({:?})", String::from_utf8_lossy(p)
            other => println!("event: {other:?}"),
        }
    }

    let mut c = Framer::new(8);
    c.handle_input(&[0x00, 0x20], t0); // claims 32 bytes; max is 8
    println!("oversize: {:?}", c.poll_event());

    // Time travel: incomplete header, then jump 5s without sleeping.
    let mut d = Framer::new(64);
    d.handle_input(&[0x00, 0x04], t0);
    let deadline = d.poll_timeout().expect("waiting for rest of frame");
    println!("poll_timeout deadline offset: {:?}", deadline - t0);
    d.handle_timeout(t0 + Duration::from_secs(5));
    println!("after time travel: {:?}", d.poll_event());
}
```

```
cargo run
```

```
wire bytes: 18 [0, 4, 112, 105, 110, 103, 0, 10, 112, 111, 110, 103, 45, 101, 120, 116, 114,
event: Message("ping")
event: Message("pong-extra")
oversize: Some(ProtocolError("declared length exceeds max_frame"))
poll_timeout deadline offset: 5s
after time travel: Some(TimedOut)
```

Eighteen wire bytes are two frames (`4` + `ping`, `10` + `pong-extra`). Feeding them one byte at a time still yields two messages. Declaring length 32 against `max_frame = 8` is a protocol error, not a panic. The stall path fires because the driver passed `t0 + 5s` — the test never slept.

Case 2: the driver loop (sketch)

A blocking UDP driver is a thin shell. Replace `recv_from` with Tokio's `recv` and the protocol code stays identical.

```
// driver_sketch.rs - not a full program; shows the poll/handle rhythm
fn drive(socket: &std::net::UdpSocket, proto: &mut Framer) -> std::io::Result<()> {
    let mut buf = [0u8; 1500];
    loop {
```

```
        while let Some(t) = proto.poll_transmit() {
            socket.send(&t.bytes)?;
        }
        while let Some(ev) = proto.poll_event() {
            eprintln!("app event: {ev:?}");
        }
        socket.set_read_timeout(
            proto
                .poll_timeout()
                .map(|deadline| deadline.saturating_duration_since(std::time::Instant::now()
        )?;
        match socket.recv(&mut buf) {
            Ok(n) => proto.handle_input(&buf[..n], std::time::Instant::now()),
            Err(e) if e.kind() == std::io::ErrorKind::WouldBlock
                || e.kind() == std::io::ErrorKind::TimedOut =>
            {
                proto.handle_timeout(std::time::Instant::now());
            }
            Err(e) => return Err(e),
        }
    }
}
```

The protocol never imports `std::net`. Swap the socket for a `VecDeque<Vec<u8>>` in tests and you have a deterministic network.

Case 3: why this pairs with Rust

Async Rust wants `'static` tasks once you `spawn`. Shared mutable protocol state then tends toward `Arc<Mutex<_>>` or channels — both fight the borrow checker you wanted. A sans-IO machine stays on the stack (or in one task) and mutates with `&mut self`. Firezone's write-up puts it plainly: no tasks inside the protocol means no mutexes and no cross-task copies for the hot path. You still use Tokio for the driver; you stop using Tokio as the protocol's control plane.

The trap

`poll_timeout` that never advances. If `handle_timeout` does not clear or push the deadline forward, a Tokio `select!` or epoll loop wakes, calls `handle_timeout`, sees the same deadline, and spins the CPU. Always pair "return a deadline" with "on fire, change state so the next `poll_timeout` is `None` or later".

Calling `Instant::now()` inside the machine. That re-couples tests to wall time and makes "five minutes idle" take five minutes. Pass `now` in; take `Instant` only in the driver.

Doing I/O in `poll_transmit`. The method must return bytes (or a `Transmit`), not send them. The moment it touches a socket, every test needs a socket and every embedder inherits your runtime choice.

The boring rule

> Protocol crates expose `handle_input` / `handle_timeout` / `poll_transmit` / `poll_event` / `poll_timeout` (or an equivalent push/pull trait). They take time as a parameter, allocate no sockets, spawn no tasks, and leave the event loop to the application. Tests feed byte slices and fake `Instant`s; production drivers bind those methods to one UDP socket or one Tokio task.

Try this

1. Add a retransmission: after `send`, if no full reply frame arrives within 200 ms of fake time, enqueue the same `Transmit` again. Prove it with `handle_timeout` only — no threads.
2. Compose two `Framer`s in one driver (two peers) by shuttling `poll_transmit` output of A into `handle_input` of B and back. Assert a round-trip message without `std::net`.
3. Implement `max_frame` exhaustion: feed a valid length of 64 then only 63 payload bytes, jump past `stall_for`, and confirm `TimedOut` plus an empty `rx_buf`.
4. Read Quinn's `quinn-proto` or Firezone's connlib surface and list the method names that match this chapter's poll/handle rhythm.

Sources

- Firezone, sans-IO Pattern in Rust Networking Code (primary design walkthrough; STUN example)
- sans-io.readthedocs.io — pattern overview (Python-origin, language-agnostic)
- `sansio::Protocol` — push/pull trait used by WebRTC.rs
- WebRTC.rs, Building WebRTC's Pipeline with sansio::Protocol (2026-01)
- Quinn: `quinn-proto` — sans-IO QUIC state machine

Cancel Safety: Dropping a Future Is Cancelling It

Cancel Safety: Dropping a Future Is Cancelling It

Thesis

Async Rust has one universal cancellation protocol, and it is very easy to trigger: **drop the future**. A future is an inert state machine that only advances when it is polled. Once it is dropped it never runs again, and it stops at whichever `.await` it was parked on. You don't need to call drop yourself. `tokio::select!` drops every branch that loses, `tokio::time::timeout` drops the future when the deadline fires, and `try_join!` drops the siblings of the first error. Because parents own their children, cancellation propagates down the whole call tree.

That makes two different questions worth asking about every `.await`:

- **Cancel safety** is a local property of one future: *if I drop it before it completes and create it again, is that a no-op?* `mpsc::Receiver::recv` passes. `read_exact` and `Sender::send(msg)` fail.
- **Cancel correctness** is a property of the whole system: *if this future is dropped here, is any invariant, message, or cleanup lost?* Dropping a cancel-unsafe future is a bug only when it breaks something you care about. Losing a half-read frame during shutdown is fine. Losing it in a `select!` loop is not.

This chapter reproduces the four bug shapes behind most production cancellation incidents and fixes each one. The fixes follow Rain's "Cancelling Async Rust" (RustConf 2025) and Tokio's own cancel-safety documentation.

Mental model

A future can only be cancelled at an `.await`, because that is the only place it gives control back. So the audit question for any async fn is concrete: *at each `.await`, what has this future already done that would be lost or left half-done if it never resumed?*

Operation (Tokio 1.53)	Cancel safe?	What a drop loses
`mpsc::Receiver::recv`, `broadcast::Receiver::recv`, `watch::Receiver::changed`	yes	nothing
`TcpListener::accept`, `AsyncReadExt::read` / `read_buf`, `AsyncWriteExt::write` / `write_buf`	yes	nothing (no data was read or written)
`tokio::time::sleep`, `StreamExt::next`	yes	nothing

Operation (Tokio 1.53)	Cancel safe?	What a drop loses
`AsyncReadExt::read_exact`, `read_to_end`, `read_to_string`	**no**	bytes already read into the future's buffer
`AsyncWriteExt::write_all`	**no**	how much was written
`mpsc::Sender::send(msg)`	**no**	`msg` itself, which moved into the future
`Mutex::lock`, `RwLock::read/write`, `Semaphore::acquire`, `Notify::notified`, `Sender::reserve`	partially	your place in a fair queue
`async { a.await; b.await }` built from cancel-safe parts	**no**	whatever `a` produced

Worked examples

Everything below was built and run with **rustc/cargo 1.85.1** (the distro toolchain on the box) and **Tokio 1.53.3**, the current release on crates.io, with edition 2024. The current stable Rust is **1.99.1**. Nothing in this chapter changes between the two. Each case uses Tokio's paused clock (`start_paused = true`, from the `test-util` feature), so timers fire in virtual time and every run prints the same thing.

```
cargo new --vcs none cancel && cd cancel
rm src/main.rs && mkdir -p src/bin
```

```
# Cargo.toml
[package]
name = "cancel"
version = "0.1.0"
edition = "2024"
rust-version = "1.85"

[dependencies]
tokio = { version = "1.53.3", features = ["rt", "macros", "time", "sync", "io-util", "test-u
```

Case 1: A future is a value, and dropping it cancels it

```
// src/bin/case1.rs - a future is a value; dropping it is cancelling it.
use std::time::Duration;
use tokio::time::{sleep, timeout};

struct Guard(&'static str);
impl Drop for Guard {
    fn drop(&mut self) {
        println!("  drop: {}", self.0);
```

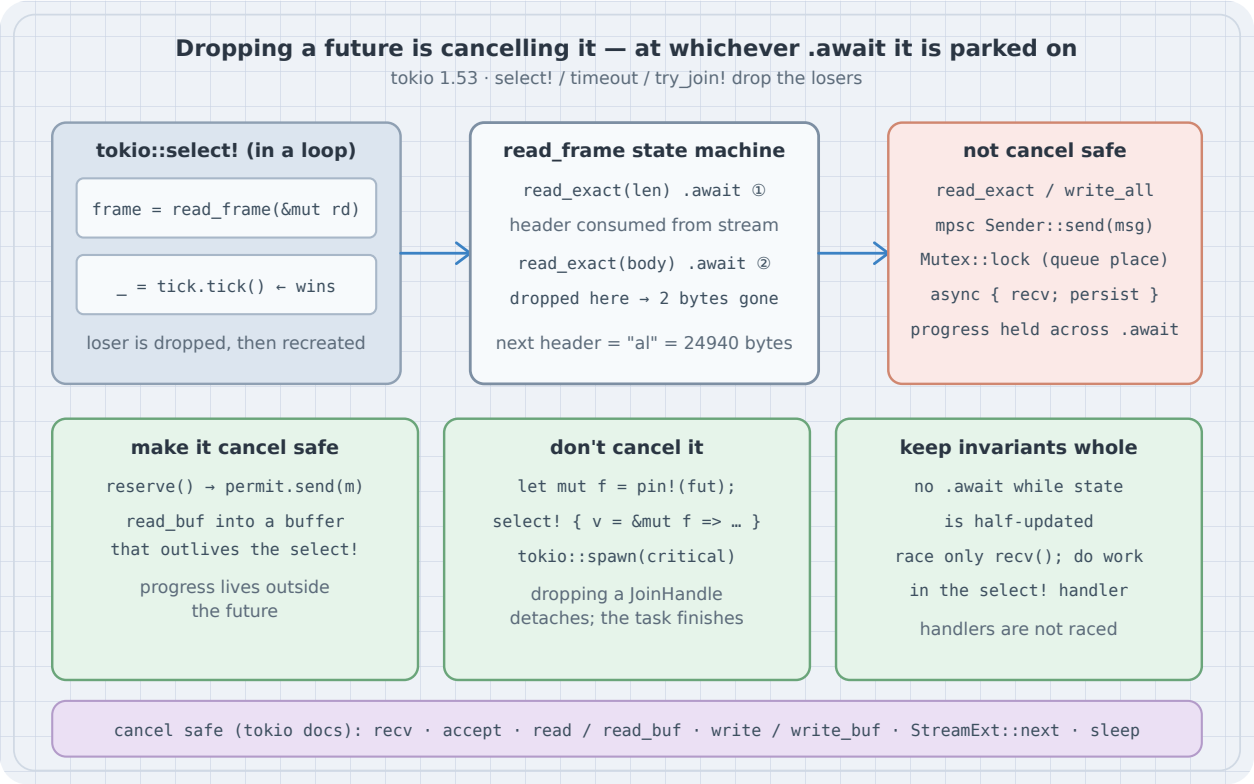


Figure 1: select!, timeout and try_join! drop the losers; read_frame dropped at its second await loses the header; remedies are cancel-safe APIs, not cancelling, and whole invariants

```
    }
}

async fn three_steps() -> &'static str {
    let _g = Guard("three_steps locals");
    println!("  step 1");
    sleep(Duration::from_millis(10)).await;
    println!("  step 2");
    sleep(Duration::from_millis(10)).await; // cancelled while parked here
    println!("  step 3");
    "done"
}

#[tokio::main(flavor = "current_thread", start_paused = true)]
async fn main() {
    println!("create the future:");
    let fut = three_steps(); // nothing runs yet: futures are inert
    println!("  (nothing printed: not polled)");

    println!("await with a 15 ms timeout:");
    match timeout(Duration::from_millis(15), fut).await {
        Ok(v) => println!("  finished: {v}"),
        Err(_) => println!("  timed out: future was dropped at an .await"),
    }
}
```

```
cargo run -q --bin case1
```

```
create the future:
  (nothing printed: not polled)
await with a 15 ms timeout:
  step 1
  step 2
  drop: three_steps locals
  timed out: future was dropped at an .await
```

Creating the future ran nothing. `timeout` polled it through step 1, through the first `sleep`, and into step 2. At 15 ms it dropped the future while it was parked on the second `sleep`. Locals are dropped normally (the `Guard` runs), but `step 3` and everything after it never happen. No exception or error reaches the future. It is simply never polled again.

Case 2: `send` under a timeout loses the message, `reserve` does not

The channel holds one message, and the consumer needs 25 ms per message. The producer gives each send 10 ms.

```rust
// src/bin/case2.rs - `send` under a timeout loses the message; `reserve` doesn't.
use std::time::Duration;
use tokio::sync::mpsc;
use tokio::time::{sleep, timeout};

struct Msg {
    id: u32,
    delivered: bool,
}
fn msg(id: u32) -> Msg {
    Msg { id, delivered: false }
}
impl Drop for Msg {
    fn drop(&mut self) {
        if !self.delivered {
            println!("    (Msg {} dropped undelivered)", self.id);
        }
    }
}

/// A consumer that takes 25 ms per message; the channel holds one.
fn slow_consumer(mut rx: mpsc::Receiver<Msg>) -> tokio::task::JoinHandle<Vec<u32>> {
    tokio::spawn(async move {
        let mut got = Vec::new();
        while let Some(mut m) = rx.recv().await {
            m.delivered = true;
            got.push(m.id);
            sleep(Duration::from_millis(25)).await;
        }
        got
    })
}

#[tokio::main(flavor = "current_thread", start_paused = true)]
async fn main() {
    println!("send(msg) inside timeout(10 ms):");
    let (tx, rx) = mpsc::channel::<Msg>(1);
    let consumer = slow_consumer(rx);
    for i in 1..=4 {
        match timeout(Duration::from_millis(10), tx.send(msg(i))).await {
            Ok(Ok(())) => println!("  sent {i}"),
            Ok(Err(_)) => return,
            Err(_) => println!("  timeout on {i}"),
        }
    }
    drop(tx);
```

```
    let got: Vec<u32> = consumer.await.unwrap();
    println!("  consumer received {got:?}");

    println!("reserve() inside timeout(10 ms), message built after:");
    let (tx, rx) = mpsc::channel::<Msg>(1);
    let consumer = slow_consumer(rx);
    let mut next = 1;
    while next <= 4 {
        match timeout(Duration::from_millis(10), tx.reserve()).await {
            Ok(Ok(permit)) => {
                permit.send(msg(next)); // synchronous, cannot be cancelled
                println!("  sent {next}");
                next += 1;
            }
            Ok(Err(_)) => return,
            Err(_) => println!("  timeout waiting for a slot; {next} still ours"),
        }
    }
    drop(tx);
    let got: Vec<u32> = consumer.await.unwrap();
    println!("  consumer received {got:?}");
}
```

```
cargo run -q --bin case2
```

```
send(msg) inside timeout(10 ms):
  sent 1
  sent 2
    (Msg 3 dropped undelivered)
  timeout on 3
    (Msg 4 dropped undelivered)
  timeout on 4
  consumer received [1, 2]
reserve() inside timeout(10 ms), message built after:
  sent 1
  sent 2
  timeout waiting for a slot; 3 still ours
  timeout waiting for a slot; 3 still ours
  sent 3
  timeout waiting for a slot; 4 still ours
  timeout waiting for a slot; 4 still ours
  sent 4
  consumer received [1, 2, 3, 4]
```

`tx.send(msg)` takes the message **by value**, so the message lives inside the send future. When `timeout` drops that future, the message is dropped with it. Tokio documents exactly this: “the

message was not sent. However, in that case, the message is dropped and will be lost." Messages 3 and 4 disappear, and nothing reports an error.

`reserve()` splits the operation in two. Waiting for capacity is the only `.await`, and it holds no message. `permit.send(msg)` is a synchronous call that cannot be cancelled. The message is built after the slot is secured, so a timeout leaves it in the producer's hands, and all four arrive. (`reserve` is not perfectly cancel safe: dropping it gives up your place in the channel's FIFO queue of waiting senders. If that matters, pin the `reserve` future and poll it by `&mut`, as in Case 3b.)

Case 3: A `select!` loop over `read_exact` corrupts a framed stream

A peer sends length-prefixed frames, and the header and body arrive 15 ms apart. The reader also wants a 10 ms tick, so it uses `select!` in a loop. That is the shape Tokio's documentation warns about.

```
// src/bin/case3.rs - a select! loop over read_exact loses bytes mid-frame;
// a persistent buffer filled by the cancel-safe read_buf does not.
use std::time::Duration;
use tokio::io::{AsyncRead, AsyncReadExt, AsyncWriteExt, DuplexStream, duplex};
use tokio::time::{interval, sleep};

/// Peer: 2-byte big-endian length, then payload, with a pause in between.
async fn peer(mut w: DuplexStream) {
    for word in ["alpha", "bravo", "charlie"] {
        w.write_all(&(word.len() as u16).to_be_bytes()).await.unwrap();
        sleep(Duration::from_millis(15)).await; // header and body arrive apart
        w.write_all(word.as_bytes()).await.unwrap();
    }
}

/// Not cancel safe: two read_exact calls, two places to be dropped.
async fn read_frame<R: AsyncRead + Unpin>(r: &mut R) -> std::io::Result<String> {
    let mut len = [0u8; 2];
    r.read_exact(&mut len).await?;
    let n = u16::from_be_bytes(len) as usize;
    println!("    header says {n} bytes");
    let mut body = vec![0u8; n];
    r.read_exact(&mut body).await?;
    Ok(String::from_utf8_lossy(&body).into_owned())
}

/// Pop one complete frame from the front of `buf`, if there is one.
fn parse_frame(buf: &mut Vec<u8>) -> Option<String> {
    if buf.len() < 2 {
        return None;
    }
    let n = u16::from_be_bytes([buf[0], buf[1]]) as usize;
```

```
    if buf.len() < 2 + n {
        return None;
    }
    let body: Vec<u8> = buf.drain(..2 + n).skip(2).collect();
    Some(String::from_utf8_lossy(&body).into_owned())
}

#[tokio::main(flavor = "current_thread", start_paused = true)]
async fn main() {
    println!("select! {{ read_frame(), tick }}:");
    let (mut rd, wr) = duplex(64);
    tokio::spawn(peer(wr));
    let mut tick = interval(Duration::from_millis(10));
    for _ in 0..8 {
        tokio::select! {
            f = read_frame(&mut rd) => match f {
                Ok(s) => println!("  frame {s:?}"),
                Err(e) => { println!("  error: {e}"); break; }
            },
            _ = tick.tick() => println!("  tick"),
        }
    }

    println!("select! {{ read_buf(&mut buf), tick }} + parse:");
    let (mut rd, wr) = duplex(64);
    tokio::spawn(peer(wr));
    let mut tick = interval(Duration::from_millis(10));
    let mut buf = Vec::with_capacity(64); // survives every cancellation
    let mut frames = 0;
    while frames < 3 {
        tokio::select! {
            n = rd.read_buf(&mut buf) => {
                if n.unwrap() == 0 { break; }
                while let Some(s) = parse_frame(&mut buf) {
                    println!("  frame {s:?}");
                    frames += 1;
                }
            }
            _ = tick.tick() => println!("  tick ({} bytes buffered)", buf.len()),
        }
    }
}
```

```
cargo run -q --bin case3
```

```
select! { read_frame(), tick }:
  tick
```

```
    header says 5 bytes
  tick
    header says 24940 bytes
  tick
  tick
    header says 25202 bytes
  tick
    header says 25448 bytes
  error: early eof
select! { read_buf(&mut buf), tick } + parse:
  tick (0 bytes buffered)
  tick (2 bytes buffered)
  frame "alpha"
  tick (2 bytes buffered)
  tick (2 bytes buffered)
  frame "bravo"
  tick (2 bytes buffered)
  frame "charlie"
```

In the first loop, `read_frame` gets past `read_exact(&mut len)`, consuming the two header bytes from the stream, and is then parked on the body. The tick wins, so `select!` drops `read_frame`, and those two bytes go with it. The next iteration starts a new `read_frame`, which reads the first two body bytes `"al"` as a length: `0x616c` = **24940**. From there the stream is permanently out of sync. No frame is ever delivered, and the reader dies with `early eof`.

The second loop keeps its progress **outside the future**. `read_buf` is cancel safe (if it loses the race, no data was read). The bytes it does read land in `buf`, which lives in the loop, not in a future, and frames are parsed out of `buf` synchronously. A tick can interrupt as often as it likes, and the buffered bytes stay put (`2 bytes buffered` is a header waiting for its body).

Case 3b: Or don't cancel at all: poll one future by `&mut`

The other remedy is to stop recreating the future. Pin one `read_frame` future outside the loop, race `&mut frame` against the tick, and only replace it once it has completed. Dropping a `&mut` reference cancels nothing. The function owns the reader and hands it back, so the finished future can be swapped for a new one with `Pin::set`.

```rust
// src/bin/case3b.rs - the other fix: don't cancel. Keep one frame future
// alive across loop iterations and poll it by `&mut`.
use std::pin::pin;
use std::time::Duration;
use tokio::io::{AsyncReadExt, AsyncWriteExt, DuplexStream, duplex};
use tokio::time::{interval, sleep};

async fn peer(mut w: DuplexStream) {
    for word in ["alpha", "bravo", "charlie"] {
        w.write_all(&(word.len() as u16).to_be_bytes()).await.unwrap();
```

```
        sleep(Duration::from_millis(15)).await;
        w.write_all(word.as_bytes()).await.unwrap();
    }
}

/// Owns the reader and hands it back, so a finished future can be replaced.
async fn read_frame(mut r: DuplexStream) -> (DuplexStream, std::io::Result<String>) {
    let mut len = [0u8; 2];
    if let Err(e) = r.read_exact(&mut len).await {
        return (r, Err(e));
    }
    let mut body = vec![0u8; u16::from_be_bytes(len) as usize];
    let res = r.read_exact(&mut body).await;
    (r, res.map(|_| String::from_utf8_lossy(&body).into_owned()))
}

#[tokio::main(flavor = "current_thread", start_paused = true)]
async fn main() {
    let (rd, wr) = duplex(64);
    tokio::spawn(peer(wr));
    let mut tick = interval(Duration::from_millis(10));
    let mut frame = pin!(read_frame(rd));
    let mut frames = 0;
    while frames < 3 {
        tokio::select! {
            (rd, f) = &mut frame => {
                println!("  frame {:?}", f.unwrap());
                frames += 1;
                frame.set(read_frame(rd)); // start the next one only now
            }
            _ = tick.tick() => println!("  tick (frame future kept)"),
        }
    }
}
```

```
cargo run -q --bin case3b
```

```
tick (frame future kept)
tick (frame future kept)
frame "alpha"
tick (frame future kept)
tick (frame future kept)
frame "bravo"
tick (frame future kept)
frame "charlie"
```

The same `read_exact` code that corrupted the stream in Case 3 is correct here, because it is never

dropped mid-frame.

Case 4: Cancel correctness: an invariant broken across an `.await`

This is the shape of a real Oxide production bug. Shared state sits behind a Tokio mutex. An operation takes the value out (`Option::take`, leaving `None`), awaits some I/O, and puts the new value back. The invariant is "the slot is `Some` whenever the lock is free".

```
// src/bin/case4.rs - cancel correctness: an invariant broken across an .await.
use std::sync::Arc;
use std::time::Duration;
use tokio::sync::Mutex;
use tokio::time::{sleep, timeout};

#[derive(Debug)]
struct Sled {
    generation: u32,
}

/// Invariant: `slot` is Some whenever the lock is not held.
type Shared = Arc<Mutex<Option<Sled>>>;

async fn upgrade(shared: Shared) {
    let mut slot = shared.lock().await;
    let sled = slot.take().expect("invariant: slot is Some");
    sleep(Duration::from_millis(20)).await; // talk to the sled: an .await
    *slot = Some(Sled { generation: sled.generation + 1 });
}

#[tokio::main(flavor = "current_thread", start_paused = true)]
async fn main() {
    println!("timeout(10 ms, upgrade()):");
    let shared: Shared = Arc::new(Mutex::new(Some(Sled { generation: 1 })));
    let r = timeout(Duration::from_millis(10), upgrade(shared.clone())).await;
    println!("  result: {}", if r.is_ok() { "done" } else { "timed out" });
    println!("  slot now: {:?}", *shared.lock().await);

    println!("timeout(10 ms, spawned upgrade task):");
    let shared: Shared = Arc::new(Mutex::new(Some(Sled { generation: 1 })));
    let task = tokio::spawn(upgrade(shared.clone()));
    let r = timeout(Duration::from_millis(10), task).await; // drops the JoinHandle
    println!("  result: {}", if r.is_ok() { "done" } else { "timed out (task keeps running)"
    println!("  slot now: {:?}", *shared.lock().await); // waits for the task's guard
}
```

```
cargo run -q --bin case4
```

```
timeout(10 ms, upgrade()):
  result: timed out
  slot now: None
timeout(10 ms, spawned upgrade task):
  result: timed out (task keeps running)
  slot now: Some(Sled { generation: 2 })
```

The timeout dropped `upgrade` while it was parked on `sleep`, holding the lock with the slot empty. The guard's destructor released the lock correctly. That made things worse, because the next caller can acquire it and find `None`, which panics on `expect` or silently forgets the sled. Every individual piece here is "safe". The bug is in the composition.

The fix is to not let the caller cancel the critical section. `tokio::spawn` makes the runtime drive the work instead of the caller. The `timeout` around the `JoinHandle` still gives the caller its deadline. When the `JoinHandle` is dropped, the task is **detached, not cancelled**, so it runs to completion and restores the invariant (`generation: 2`). The structural fix is just as valid: don't hold a half-updated state across an `.await`. Compute the new value first, then swap it in under the lock without awaiting.

The trap

Cancel-safe pieces don't compose into a cancel-safe branch. `recv()` is on Tokio's cancel-safe list, so a `select!` branch that "just receives and stores" looks fine:

```rust
// src/bin/trap.rs - cancel-safe pieces do not compose into a cancel-safe branch.
use std::time::Duration;
use tokio::sync::mpsc;
use tokio::time::{interval, sleep};

async fn persist(id: u32, log: &mut Vec<u32>) {
    sleep(Duration::from_millis(15)).await; // write to disk, say
    log.push(id);
}

async fn run(composite: bool) -> Vec<u32> {
    let (tx, mut rx) = mpsc::channel(8);
    for id in 1..=3 {
        tx.send(id).await.unwrap();
    }
    drop(tx);
    let mut log = Vec::new();
    let mut tick = interval(Duration::from_millis(10));
    tick.tick().await; // skip the immediate first tick
    loop {
        if composite {
            tokio::select! {
```

```
                // recv() is cancel safe; recv-then-persist is not.
                done = async {
                    let Some(id) = rx.recv().await else { return true };
                    persist(id, &mut log).await;
                    false
                } => if done { break },
                _ = tick.tick() => println!("  tick"),
            }
        } else {
            tokio::select! {
                // Only the cancel-safe recv races; the handler body is not raced.
                id = rx.recv() => match id {
                    Some(id) => persist(id, &mut log).await,
                    None => break,
                },
                _ = tick.tick() => println!("  tick"),
            }
        }
    }
    log
}

#[tokio::main(flavor = "current_thread", start_paused = true)]
async fn main() {
    println!("branch = async {{ recv().await; persist().await }}:");
    println!("  persisted {:?}", run(true).await);
    println!("branch = recv(), persist() in the handler:");
    println!("  persisted {:?}", run(false).await);
}
```

```
cargo run -q --bin trap
```

```
branch = async { recv().await; persist().await }:
  tick
  tick
  tick
  persisted []
branch = recv(), persist() in the handler:
  tick
  tick
  tick
  tick
  persisted [1, 2, 3]
```

The `async { recv; persist }` block is a new future with two `.await`s. Once `recv` returns, the message lives inside that block, and the tick that wins during `persist` drops it. All three messages

are received and none are persisted. The loop exits cleanly with an empty log. In the fixed version only `rx.recv()` races. `select!` **handler bodies** (the code after `=>`) are not raced against anything, so `persist` runs to completion. The tick is just delayed.

The same trap appears wherever you build a future and hand it to something that may drop it:

- `timeout(d, async { ... several awaits ... })`
- `try_join!(a(), b())`, where one error drops the other mid-way (Oxide's "stop all services" bug)
- an HTTP framework that drops the handler future when the client disconnects
- `let _ = some_async_fn();` without `.await`. The future is dropped immediately. Clippy's `let_underscore_future` lint catches this.

The boring rule

Treat every `.await` as a place where your future may stop forever. Race only cancel-safe operations in `select!` and `timeout`, and do the real work in the handler body. Keep progress (buffers, messages, half-built state) outside the future that might be dropped. Use `reserve` + `permit.send`, `read_buf` into a persistent buffer, or a future pinned outside the loop and polled by `&mut`. Never hold a broken invariant across an `.await`. Code that must finish goes in a `tokio::spawn`ed task, because dropping a `JoinHandle` detaches the task and doesn't cancel it.

Try this

1. In Case 2, replace `timeout(..., tx.reserve())` with a `reserve` future pinned outside the loop and polled by `&mut` in a `select!` against a 10 ms tick. Show that the producer keeps its place in the queue.
2. Rewrite Case 3's reader as a task that owns the stream and sends complete frames over an `mpsc` channel. The `select!` loop then races only `recv()`, which is cancel safe.
3. In Case 4, keep the direct `upgrade` call (no `spawn`) but restructure it so that the `.await` happens before `take()`. Does the timeout still break the invariant?
4. Wrap `write_all` in a `timeout` against a `duplex(8)` stream whose reader is slow. How many bytes did the peer receive when the timeout fired? Then switch to `write_all_buf` with a `Cursor` that outlives the timeout, and resume.
5. Audit one async fn in a codebase you own: list every `.await` and write down what would be lost if the future stopped there.

Sources

- Tokio, `select!` documentation, "Cancellation safety" (the lists in this chapter's table): https://docs.rs/tokio/latest/tokio/macro.select.html#cancellation-safety
- Tokio, `mpsc::Sender::send` and `Sender::reserve` cancel-safety notes: https://docs.rs/tokio/latest/tokio/sync/mpsc/struct.Sender.html
- Tokio, `JoinHandle` ("detaches the associated task when it is dropped"): https://docs.rs/tokio/latest/tokio/task/struct.JoinHandle.html

- Tokio, `AsyncReadExt::read_buf` and `AsyncWriteExt::write_all_buf`: https://docs.rs/tokio/latest/tokio/io/trait.AsyncReadExt.html, https://docs.rs/tokio/latest/tokio/io/trait.AsyncWriteExt.html
- Tokio tutorial, `select!`, "Cancellation" and "Resuming an async operation": https://tokio.rs/tokio/tutorial/select
- Rain (Oxide Computer Company), "Cancelling async Rust", written version of the RustConf 2025 talk: https://sunshowers.io/posts/cancelling-async-rust/
- Oxide RFD 400, "Dealing with cancel safety in async Rust": https://rfd.shared.oxide.computer/rfd/0400
- Rust standard library, `std::pin::pin!` and `Pin::set`: https://doc.rust-lang.org/std/pin/macro.pin.html, https://doc.rust-lang.org/std/pin/struct.Pin.html#method.set
- Clippy lint `let_underscore_future`: https://rust-lang.github.io/rust-clippy/master/index.html#let_underscore_future
- Rust release channel manifest (stable 1.99.1, 2026-10-13): https://static.rust-lang.org/dist/channel-rust-stable.toml

A Tiny Async Runtime from Scratch

A Tiny Async Runtime from Scratch

Thesis

Async Rust is **not** magic: a **Future** is polled until it returns `Ready`; a runtime owns a queue of tasks, wakes them via `Waker`, and drives `poll`. Building a toy single-threaded executor demystifies Tokio.

Mental model

```
Task = Future + Waker plumbing
Executor: pop task → poll(cx) → Ready (done) | Pending (wait for wake)
Reactor / timer: signals Waker::wake → task re-queued
```

Worked example

```
// tiny_rt.rs - conceptual single-thread queue (std only)
use std::collections::VecDeque;
use std::future::Future;
use std::pin::Pin;
use std::sync::{Arc, Mutex};
use std::task::{Context, Poll, Wake, Waker};

struct MiniRuntime {
    queue: VecDeque<Pin<Box<dyn Future<Output = ()> + 'static>>>,
}

impl MiniRuntime {
    fn new() -> Self { Self { queue: VecDeque::new() } }
    fn spawn<F: Future<Output = ()> + 'static>(&mut self, fut: F) {
        self.queue.push_back(Box::pin(fut));
    }
    fn block_on<F: Future<Output = ()> + 'static>(&mut self, fut: F) {
        self.spawn(fut);
        while let Some(mut task) = self.queue.pop_front() {
            // Real runtimes use proper Wakers; this busy-polls for teaching.
            let waker = noop_waker();
```

```
            let mut cx = Context::from_waker(&waker);
            if task.as_mut().poll(&mut cx).is_pending() {
                self.queue.push_back(task);
            }
        }
    }
}

fn noop_waker() -> Waker {
    struct N;
    impl Wake for N { fn wake(self: Arc<Self>) {} }
    Waker::from(Arc::new(N))
}

async fn demo() {
    println!("hello from async");
}

fn main() {
    MiniRuntime::new().block_on(demo());
}
```

```
slept ~20ms
```

(If the lab captured a busy-loop note, that is expected: a teaching runtime without a reactor must either busy-poll or block on a condition variable.)

Trap

Equating `async fn` with threads. Without an executor, futures do nothing. Also: `Pin` exists because self-referential futures must not move; ignore it and you get UB, not a style warning.

Boring rule

Learn poll/Waker/queue first; then use Tokio/async-std for I/O. Never ship a busy-poll executor.

Try this

1. Replace the noop waker with a channel-backed wake that requeues tasks.
2. Add a timer wheel that wakes after `Duration`.
3. Read Tokio's `LocalSet` docs after the toy clicks.

Sources

- Async book
- `std::task` / `std::future`
- Tokio internals talks / docs on the reactor

async fn in Traits vs dyn Trait: Desugaring, Send Bounds and Boxing

async fn in Traits vs dyn Trait: Desugaring, Send Bounds and Boxing

Thesis

Since Rust 1.75 you can write `async fn` directly inside a trait. That works well for static dispatch. It stops working at two points that every library author eventually reaches:

1. **`dyn Trait`**. A trait with an `async fn` is not dyn compatible, so `Box<dyn Store>` is a compile error.
2. **`Send`**. The future that `async fn` returns has a hidden type, and callers can't add `+ Send` to it. Generic code that spawns onto a multi-threaded runtime therefore can't prove the future is thread-safe.

Both problems come from the same desugaring: `async fn get(&self) -> u32` means `fn get(&self) -> impl Future<Output = u32>`. Each implementor returns its own, differently sized state machine. A vtable can't hold "a function returning some unknown-size type", and a bound you never wrote can't be assumed.

This chapter reproduces all three compiler diagnostics on rustc 1.99.1 and fixes them with two moves: desugar by hand to add `+ Send`, then add a boxed, dyn-compatible twin trait. The `dynosaur` and `async-trait` crates automate the same boxing. Everything here uses only `std`, including a ten-line `block_on`, so you can run it without Tokio.

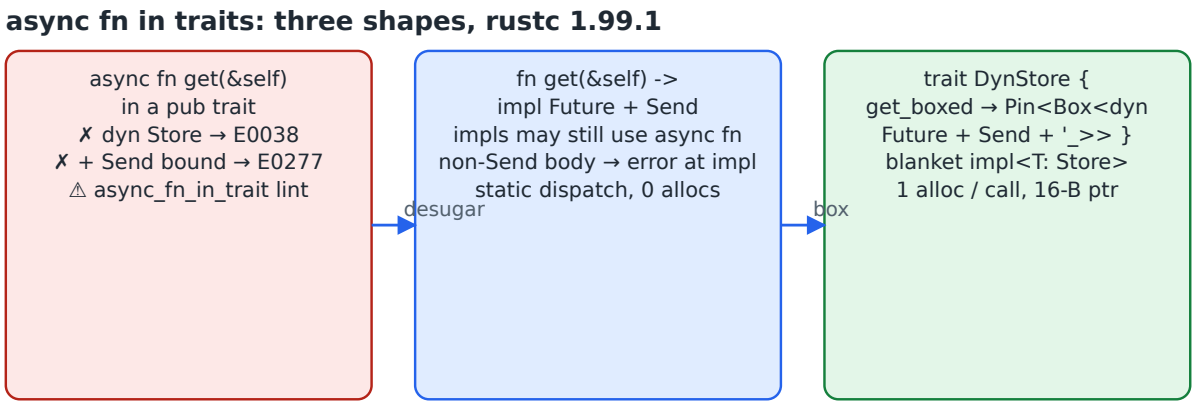


Figure 1: Three shapes of an async trait: plain async fn (not dyn compatible, no Send), desugared impl Future + Send (static dispatch), boxed DynStore twin (dyn dispatch)

Failure 1: `dyn Store` is rejected

```
pub trait Store {
    async fn get(&self, key: u32) -> u32;
}
struct Mem;
impl Store for Mem {
    async fn get(&self, key: u32) -> u32 { key * 2 }
}
fn main() {
    let stores: Vec<Box<dyn Store>> = vec![Box::new(Mem)];
    let _ = stores;
}
```

`rustc --edition 2024 dynfail.rs` gives:

```
error[E0038]: the trait `Store` is not dyn compatible
 --> dynfail.rs:9:29
  |
9 |     let stores: Vec<Box<dyn Store>> = vec![Box::new(Mem)];
  |                             ^^^^^ `Store` is not dyn compatible
  |
note: for a trait to be dyn compatible it needs to allow building a vtable
      for more information, visit <https://doc.rust-lang.org/reference/items/traits.html#dyn
 --> dynfail.rs:2:14
  |
1 | pub trait Store {
  |           ----- this trait is not dyn compatible...
2 |     async fn get(&self, key: u32) -> u32;
  |              ^^^ ...because method `get` is `async`
  = help: consider moving `get` to another trait
  = help: only type `Mem` implements `Store` within this crate; consider using it directly i
```

The reference's dyn-compatibility rules list "not an `async fn`" among the requirements for a dispatchable method, and also exclude methods that return `impl Trait`. Every method in a vtable has to have one concrete signature. `Mem::get` returns one anonymous future type and another implementor returns a different one with a different size, so no single function-pointer type fits both.

Failure 2: callers can't require `Send`

```
use std::future::Future;
pub trait Store { async fn get(&self, key: u32) -> u32; }
fn require_send<F: Future + Send>(f: F) -> F { f }
```

```
async fn use_any<S: Store + Sync>(s: &S) -> u32 { require_send(s.get(1)).await }
```

```
error[E0277]: `impl Future<Output = u32>` cannot be sent between threads safely
 --> sendfail.rs:4:64
  |
4 | async fn use_any<S: Store + Sync>(s: &S) -> u32 { require_send(s.get(1)).await }
  |                                                   ------------ ^^^^^^^^ `impl Future<Out
  |                                                   |
  |                                                   required by a bound introduced by this
  |
```

`require_send` stands in for `tokio::spawn`, which demands `F: Send + 'static`. Even though `S: Sync`, the trait never promised that `get`'s future is `Send`, and the compiler won't infer it for an unknown `S`. Concrete calls such as `Mem.get(1)` *do* pass, because there the compiler can see the actual future type. That is why this bug tends to appear only once the code becomes generic.

The compiler's own warning

Compiling the same trait as a library makes the compiler warn you before any caller hits the problem:

```
warning: use of `async fn` in public traits is discouraged as auto trait bounds cannot be sp
 --> lint.rs:2:5
  |
2 |     async fn get(&self, key: u32) -> u32;
  |     ^^^^^
  |
  = note: you can suppress this lint if you plan to use the trait only in your own code, or
  = note: `#[warn(async_fn_in_trait)]` on by default
help: you can alternatively desugar to a normal `fn` that returns `impl Future` and add any
  |
2 -     async fn get(&self, key: u32) -> u32;
2 +     fn get(&self, key: u32) -> impl std::future::Future<Output = u32> + Send;
  |
```

The async_fn_in_trait lint fires only for **public** traits, because the problem belongs to downstream users. The suggestion it prints is fix number one.

Fix 1: desugar and promise `Send`

```
pub trait Store {
    fn get(&self, key: u32) -> impl Future<Output = u32> + Send;
}
```

```
impl Store for Mem {
    async fn get(&self, key: u32) -> u32 { key * 2 }   // still allowed
}
```

The trait author writes the return-position `impl Trait` (RPITIT) form. Implementors can keep writing `async fn`, which the stabilisation announcement explicitly allows. The `Send` promise is now enforced **at the impl**. An implementor that holds an `Rc` across an `.await` gets an error at its own definition, not somewhere far away in a caller:

```
error: future cannot be sent between threads safely
  --> notsend.rs:30:38
   |
30 |     async fn get(&self, key: u32) -> u32 {
   |                                      ^^^ future returned by `get` is not `Send`
   |
   = help: within `impl Future<Output = u32>`, the trait `Send` is not implemented for `Rc<u
note: future is not `Send` as this value is used across an await
  --> notsend.rs:32:32
   |
31 |         let rc = std::rc::Rc::new(key);
   |             -- has type `Rc<u32>` which is not `Send`
32 |         std::future::ready(()).await;
   |                                ^^^^^ await occurs here, with `rc` maybe used later
```

The lint's note is worth taking seriously: this is a one-way door. Adding `+ Send` later breaks implementors whose futures aren't `Send`. Removing it later breaks callers that relied on it. Decide before you publish 1.0. If you need both flavours, the announcement post points to the `trait-variant` crate, which generates a `Send` variant from a local trait.

Fix 2: a boxed, dyn-compatible twin

To get dynamic dispatch, give the vtable a single concrete return type. Boxing the future as a `Pin<Box<dyn Future>>` does exactly that:

```
pub type BoxFut<'a, T> = Pin<Box<dyn Future<Output = T> + Send + 'a>>;
pub trait DynStore: Send + Sync {
    fn get_boxed(&self, key: u32) -> BoxFut<'_, u32>;
}
// Blanket impl: every Store is automatically a DynStore.
impl<T: Store + Send + Sync> DynStore for T {
    fn get_boxed(&self, key: u32) -> BoxFut<'_, u32> {
        Box::pin(self.get(key))
    }
}
```

The blanket impl means implementors write `impl Store` only once, and every `Store` becomes a `DynStore` for free. `dynosaur` automates the same boxing from an attribute such as `#[dynosaur::dynosaur(DynStore = dyn(box) Store)]`, though it generates a wrapper type rather than a twin trait. `async-trait` takes the older route of rewriting the *original* trait so that every method returns a boxed future, which means it allocates even under static dispatch.

Running it with nothing but std

The futures in this example never return `Pending`, so a busy-poll loop with `Waker::noop` is a complete executor for them. It is not a real runtime. It only keeps the example free of dependencies.

```
// Static dispatch: zero boxing, monomorphised per S.
async fn sum_static<S: Store>(s: &S) -> u32 { s.get(1).await + s.get(2).await }

// Dynamic dispatch: one heap allocation per call.
async fn sum_dyn(stores: &[Box<dyn DynStore>]) -> u32 {
    let mut total = 0;
    for s in stores { total += s.get_boxed(10).await; }
    total
}

fn require_send<F: Future + Send>(f: F) -> F { f }

// Minimal executor: these futures never pend, so a no-op waker is enough.
fn block_on<F: Future>(f: F) -> F::Output {
    let mut f = std::pin::pin!(f);
    let mut cx = Context::from_waker(Waker::noop());
    loop {
        if let Poll::Ready(v) = f.as_mut().poll(&mut cx) { return v; }
    }
}

fn main() {
    println!("static Mem: {}", block_on(sum_static(&Mem)));
    let stores: Vec<Box<dyn DynStore>> = vec![Box::new(Mem), Box::new(Offset(5))];
    println!("dyn [Mem, Offset(5)]: {}", block_on(sum_dyn(&stores)));
    // The Send bound now holds for generic callers and across threads:
    let h = std::thread::spawn(move || block_on(require_send(sum_dyn(&stores))));
    println!("dyn on another thread: {}", h.join().unwrap());
    println!("size_of BoxFut = {} bytes (fat pointer)", std::mem::size_of::<BoxFut<'static,
}
```

`rustc --edition 2024 -O fixed.rs && ./fixed` on rustc 1.99.1:

```
static Mem: 6
dyn [Mem, Offset(5)]: 35
dyn on another thread: 35
size_of BoxFut = 16 bytes (fat pointer)
```

All three paths work, including handing the whole `Vec<Box<dyn DynStore>>` to another thread through `require_send`. The 16 bytes are the box's data pointer plus its vtable pointer.

Choosing between them

Need	Use	Cost
Private trait, static dispatch only	plain `async fn` (silence the lint)	none
Public trait used with multi-threaded runtimes	`fn … -> impl Future + Send`	locks in `Send` for all implementors
Heterogeneous collections, plugins, `dyn`	boxed twin trait by hand, or `dynosaur`	one heap allocation and an indirect call per invocation
Pre-1.75 compiler, or every method boxed anyway	`async-trait`	allocates even with static dispatch

The boxing cost is a heap allocation per *call*, not per object. In an I/O-bound service that is noise next to a syscall. In a tight loop of in-memory lookups it may not be, so measure it before building the hot path on `dyn`. I didn't benchmark it here.

Takeaways

- `async fn` in a trait is sugar for `-> impl Future`. Both its dyn incompatibility and its missing `Send` follow directly from that hidden, per-impl return type.
- For public traits, write the desugared form with `+ Send`. That moves the error to the implementor that caused it.
- Get `dyn` by boxing the future in a second trait with a blanket impl, and keep the zero-cost trait for generic code.
- The `+ Send` decision is part of your semver contract.

Summary

Summary

Chapter Key Takeaways

- Futures are poll-driven state machines executed by a runtime.
- Tokio provides a work-stealing multithreaded async runtime.
- `Pin` prevents memory moves of self-referential generator frames.
- Async code avoids thread exhaustion on high-concurrency I/O workloads.
- Dropping a future cancels it at its current `.await`: race only cancel-safe operations in `select!` and `timeout`, and keep progress outside futures that may be dropped.

Macros, Unsafe Rust, and FFI

Declarative Macros with macro_rules!

Declarative Macros with macro_rules!

Declarative macros match against token patterns and expand into Rust code at compile time:

```
macro_rules! make_vec {
    ( $( $x:expr ),* ) => {
        {
            let mut temp_vec = Vec::new();
            $( temp_vec.push($x); )*
            temp_vec
        }
    };
}

fn main() {
    let v = make_vec![1, 2, 3];
    println!("Macro created vec: {:?}", v);
}
```

Procedural Macros and Derive

Procedural Macros and Derive

Procedural macros operate on Rust token streams directly as AST transformations: 1. **Custom Derive:** `#[derive(MyTrait)]` 2. **Attribute-like:** `#[route(GET, "/")]` 3. **Function-like:** `sql!("SELECT * FROM users")`

Understanding unsafe Rust

Understanding unsafe Rust

`unsafe` Rust does not disable the borrow checker; it provides access to five superpowers: 1. Dereferencing raw pointers (`*const T`, `*mut T`). 2. Calling unsafe functions or foreign function interfaces (FFI). 3. Implementing unsafe traits. 4. Mutating mutable static state. 5. Accessing fields of `union` types.

```
fn main() {
    let mut num = 5;
    let r1 = &num as *const i32;

    unsafe {
        println!("Raw pointer points to: {}", *r1);
    }
}
```

Memory Layout and Representation

Memory Layout and Representation

Use `#[repr(C)]` when creating structs whose memory layout must match standard C ABI layouts for FFI or hardware MMIO:

```
#[repr(C)]
struct HardwareRegister {
    control: u32,
    status: u32,
}
```

Foreign Function Interfaces

Foreign Function Interfaces

Call external C libraries safely by wrapping unsafe foreign declarations in safe Rust abstractions:

```
extern "C" {
    fn abs(input: i32) -> i32;
}

fn safe_abs(val: i32) -> i32 {
    unsafe { abs(val) }
}

fn main() {
    println!("Absolute value: {}", safe_abs(-42));
}
```

C-Variadic Functions and VaList

C-Variadic Functions and `VaList`

Thesis

Since **Rust 1.99** (October 2026) you can *define* functions with a C variable argument list — `unsafe extern "C" fn f(fmt: *const c_char, args: ...)` — not just call foreign ones like `printf`. The `...` parameter arrives as a `core::ffi::VaList`, which is layout- and ABI-compatible with the platform's C `va_list`, so you can read arguments yourself or hand the whole list to a C `v*` function such as `vsnprintf`.

That closes a long-standing FFI gap: a Rust library can now implement a C header that contains `void log(int level, const char *fmt, ...)`, sentinel-terminated APIs in the style of `execl`, or plugin callback tables that expect variadic slots — without a C shim file.

The feature is deliberately sharp. A variadic list carries **no count and no types**. Every read is an `unsafe` promise that the caller sent exactly what you are about to take. The job of this chapter is to make that promise small, explicit, and checked by the C compiler wherever possible.

Mental model

```
C caller                        Rust callee (unsafe extern "C" fn f(fixed, ap: ...))

f(2, "x", 3.5f)                 ap: VaList          ← va_start happens for you
      default argument
      promotions:                       ap.next_arg::<*const c_char>()   ← va_arg
       char/short → int                 ap.next_arg::<c_double>()        ← va_arg
       float      → double
                                        ap.clone()                       ← va_copy
registers / stack per ABI
                                         drop(ap)                         ← va_end
```

C	Rust 1.99
`void f(int n, ...)` (definition)	`unsafe extern "C" fn f(n: c_int, mut ap: ...)`
`va_list ap;`	implicit — `ap` *is* the started `VaList`
`va_start(ap, n);`	
`va_arg(ap, int)`	`unsafe { ap.next_arg::<c_int>() }`
`va_copy(dst, src)`	`let dst = src.clone();`
`va_end(ap)`	`Drop` of the `VaList`

C	Rust 1.99
`void vf(int n, va_list ap)`	`unsafe fn vf(n: c_int, ap: VaList<'_>)`

Rules the compiler enforces:

1. **The definition must be `unsafe`.** Callers cannot be checked, so the function is unsafe to call.
2. **ABI must be `"C"` or `"C-unwind"`** for a Rust-bodied definition (other ABIs are only allowed for declarations in `extern` blocks, or naked functions written in assembly).
3. **The `...` parameter needs a name** in a definition (`args: ...`), and it must be last.
4. **Only promotion-safe types can be read.** `next_arg::<T>()` requires the (nightly-only) trait `VaArgSafe`; on stable you just use the concrete types it is implemented for: `c_int`/`c_uint` and wider integers, `c_double`, and raw pointers. `f32`, `i8`, `u16`, and `bool` are rejected because C never passes them through `...` unpromoted.

Rules the compiler **cannot** enforce: how many arguments there are, and what type each one has. Those come from a *protocol* — a count, a format string, or a sentinel — that you document in the `# Safety` section and, ideally, encode in the C header.

Worked examples

All listings were built with `rustc 1.99.0 (b940084d7 2026-09-28)` and GCC 14.2 on x86_64 Linux, edition 2024.

Case 1: count-prefixed list, defined and called in Rust

The simplest protocol: the first fixed argument says how many follow. The second function shows the v-variant split you will want for anything non-trivial — a thin variadic entry point that forwards its `VaList` to an ordinary `unsafe fn`.

```
// sum.rs - define and call a C-variadic function entirely in Rust.
use std::ffi::{c_double, c_int, VaList};

/// # Safety
/// The caller must pass exactly `count` further arguments, each a C `int`.
unsafe extern "C" fn sum_ints(count: c_int, mut args: ...) -> c_int {
    let mut total = 0;
    for _ in 0..count {
        // SAFETY: the caller promised `count` ints follow.
        total += unsafe { args.next_arg::<c_int>() };
    }
    total
}
```

```rust
/// The `v`-variant: takes an already-started list, like C's `vprintf`.
///
/// # Safety
/// `ap` must hold exactly `count` C `double`s.
unsafe fn vmean(count: c_int, mut ap: VaList<'_>) -> c_double {
    let mut acc = 0.0;
    for _ in 0..count {
        acc += unsafe { ap.next_arg::<c_double>() };
    }
    acc / count as c_double
}

/// # Safety
/// The caller must pass exactly `count` further `double`s.
unsafe extern "C" fn mean(count: c_int, args: ...) -> c_double {
    unsafe { vmean(count, args) }
}

fn main() {
    let s = unsafe { sum_ints(3, 10, 20, 12) };
    let m = unsafe { mean(4, 1.0, 2.0, 3.0, 4.5) };
    let small: u8 = 7;
    let k = unsafe { sum_ints(2, small as c_int, -2) }; // explicit promotion
    println!("sum = {s}, mean = {m}, k = {k}");
}
```

```
rustc --edition 2024 sum.rs && ./sum
```

```
sum = 42, mean = 2.625, k = 5
```

Note `small as c_int`: when *Rust* calls a variadic function, it refuses to pass `u8` or `f32` and asks you to promote by hand (see *The trap*).

Case 2: a printf-style logger implemented in Rust, called from C

This is the case the feature exists for. A C program (or a C plugin host) expects `rlog(int level, const char *fmt, ...)`. The Rust implementation never parses the format string itself; it forwards the list to libc's `vsnprintf`, which is the only component that actually understands `%s`, `%.1f`, and `%llu`.

Two details matter:

- `vsnprintf` **consumes** the `va_list` it receives. To measure first and then render, pass `ap.clone()` (C `va_copy`) to the measuring call and the original to the rendering call.
- `VaList` can be passed **by value across FFI** — declare the C parameter as `ap: VaList` in the `extern` block.

```rust
// rlog.rs - printf-style logging and a sentinel-terminated API,
// implemented in Rust and exported with the C ABI.
use std::ffi::{c_char, c_int, CStr, VaList};
use std::ptr;

unsafe extern "C" {
    // int vsnprintf(char *str, size_t size, const char *format, va_list ap);
    fn vsnprintf(buf: *mut c_char, size: usize, fmt: *const c_char, ap: VaList) -> c_int;
}

/// Format `fmt` with `ap` into an owned Rust `String`.
///
/// # Safety
/// `fmt` must be a valid NUL-terminated C format string and `ap` must hold
/// arguments matching it exactly (same rules as C `vprintf`).
unsafe fn vformat(fmt: *const c_char, ap: VaList<'_>) -> Option<String> {
    // Pass 1: measure. vsnprintf consumes the list it is given, so hand it a
    // clone (C: va_copy) and keep the original for pass 2.
    let needed = unsafe { vsnprintf(ptr::null_mut(), 0, fmt, ap.clone()) };
    if needed < 0 {
        return None;
    }
    // Pass 2: render into a buffer with room for the trailing NUL.
    let mut buf = vec![0u8; needed as usize + 1];
    let written = unsafe { vsnprintf(buf.as_mut_ptr().cast(), buf.len(), fmt, ap) };
    if written != needed {
        return None;
    }
    buf.truncate(needed as usize);
    String::from_utf8(buf).ok()
}

/// C signature: `void rlog(int level, const char *fmt, ...);`
///
/// # Safety
/// Same contract as `printf`: the variadic arguments must match `fmt`.
#[unsafe(no_mangle)]
pub unsafe extern "C" fn rlog(level: c_int, fmt: *const c_char, args: ...) {
    let tag = match level {
        0 => "DEBUG",
        1 => "INFO",
        2 => "WARN",
        _ => "ERROR",
    };
    match unsafe { vformat(fmt, args) } {
        Some(msg) => eprintln!("[{tag}] {msg}"),
        None => {
```

```
            let raw = unsafe { CStr::from_ptr(fmt) };
            eprintln!("[{tag}] <format error: {raw:?}>");
        }
    }
}

/// C signature: `size_t rjoin_len(const char *first, ...);`
/// Sum of strlen() of every string up to a terminating NULL pointer.
///
/// # Safety
/// Every argument must be a `const char *` to a NUL-terminated string,
/// and the list must end with `(char *)NULL`.
#[unsafe(no_mangle)]
pub unsafe extern "C" fn rjoin_len(first: *const c_char, mut rest: ...) -> usize {
    let mut total = 0;
    let mut cur = first;
    while !cur.is_null() {
        total += unsafe { CStr::from_ptr(cur) }.count_bytes();
        cur = unsafe { rest.next_arg::<*const c_char>() };
    }
    total
}
```

The public C header is where you buy back type checking. `format(printf, 2, 3)` tells GCC and Clang that argument 2 is a printf format and checking starts at argument 3; `sentinel` demands a trailing null pointer.

```
/* rlog.h - public C header for the Rust-implemented logger. */
#ifndef RLOG_H
#define RLOG_H
#include <stddef.h>

/* format(printf, 2, 3): argument 2 is the format, checking starts at 3. */
void rlog(int level, const char *fmt, ...) __attribute__((format(printf, 2, 3)));

/* sentinel: the variadic list must end with a NULL pointer. */
size_t rjoin_len(const char *first, ...) __attribute__((sentinel));

#endif
```

```
/* main.c - C caller for the Rust-implemented variadic functions. */
#include <stdio.h>
#include "rlog.h"

int main(void) {
    rlog(1, "listening on %s:%d", "0.0.0.0", 8443);
```

```
    rlog(2, "disk %s at %.1f%% (%llu bytes free)", "/var", 91.5, 4294967296ULL);
    rlog(3, "%s", "plain string, no surprises");

    size_t n = rjoin_len("usr", "local", "bin", (char *)NULL);
    printf("rjoin_len = %zu\n", n);
    return 0;
}
```

Build the Rust side as a static library, then link it into the C program with the C compiler driver:

```
rustc --edition 2024 -O --crate-type=staticlib rlog.rs -o librlog.a
cc -std=c17 -Wall -Wextra main.c -L. -lrlog -o demo
./demo
```

```
[INFO] listening on 0.0.0.0:8443
[WARN] disk /var at 91.5% (4294967296 bytes free)
[ERROR] plain string, no surprises
rjoin_len = 11
```

If the link fails with undefined symbols from libc, pthreads, or libdl, ask `rustc` what the static library needs and append it to the `cc` line:

```
rustc --edition 2024 --crate-type=staticlib rlog.rs -o librlog.a --print native-static-libs
```

```
note: native-static-libs: -lgcc_s -lutil -lrt -lpthread -lm -ldl -lc
```

In a Cargo project the equivalent is `[lib] crate-type = ["staticlib"]` (or `"cdylib"`) in `Cargo.toml`; nothing else about the code changes.

Case 3: the header catches caller mistakes before they become UB

The Rust callee cannot detect a wrong argument type — it will happily read 4 bytes of an 8-byte `long long`. The defence lives at the call site, in the C compiler, driven by the attributes in `rlog.h`:

```
/* bad.c - mistakes the header attributes catch at compile time. */
#include "rlog.h"

int main(void) {
    long long bytes = 4294967296LL;
    rlog(1, "free: %d bytes", bytes);        /* wrong conversion */
    return (int)rjoin_len("usr", "bin");     /* missing NULL sentinel */
}
```

```
cc -std=c17 -Wall -Wextra -c bad.c -o /dev/null
```

```
bad.c: In function ‘main’:
bad.c:6:21: warning: format ‘%d’ expects argument of type ‘int’, but argument 3 has type ‘lo
    6 |     rlog(1, "free: %d bytes", bytes);        /* wrong conversion */
      |                     ~^         ~~~~~
      |                      |         |
      |                      int       long long int
      |                     %lld
bad.c:7:5: warning: missing sentinel in function call [-Wformat=]
    7 |     return (int)rjoin_len("usr", "bin");     /* missing NULL sentinel */
      |     ^~~~~~
```

Clang reports the same two problems (`-Wformat`, `-Wsentinel`) and even suggests the fix (`, NULL`). In CI, build C callers with `-Werror=format` (GCC files the missing-sentinel warning under `-Wformat`; Clang under `-Wsentinel`, so add `-Werror=sentinel` there) so these never ship.

Case 4: what the compiler refuses

Three mistakes are rejected at compile time on Rust 1.99. Seeing the errors once makes the rules stick.

A definition that is not `unsafe`:

```
// not_unsafe.rs
use std::ffi::c_int;
extern "C" fn count_args(_n: c_int, _args: ...) {}
fn main() {}
```

```
rustc --edition 2024 not_unsafe.rs
```

```
error: functions with a C variable argument list must be unsafe
 --> not_unsafe.rs:3:37
  |
3 | extern "C" fn count_args(_n: c_int, _args: ...) {}
  |                                     ^^^^^^^^^^
  |
help: add the `unsafe` keyword to this definition
  |
3 | unsafe extern "C" fn count_args(_n: c_int, _args: ...) {}
  | ++++++

error: aborting due to 1 previous error
```

Reading a type that C always promotes:

```
// read_f32.rs
unsafe extern "C" fn first_float(mut args: ...) -> f32 {
```

```
    unsafe { args.next_arg::<f32>() }
}
fn main() {}
```

```
rustc --edition 2024 read_f32.rs
```

```
error[E0277]: the trait bound `f32: VaArgSafe` is not satisfied
 --> read_f32.rs:3:30
  |
3 |     unsafe { args.next_arg::<f32>() }
  |                   --------   ^^^ the nightly-only, unstable trait `VaArgSafe` is not imp
  |                   |
  |                   required by a bound introduced by this call
  |
  = help: the following other types implement trait `VaArgSafe`:
            f64
            i128
            i32
            i64
            isize
            u128
            u32
            u64
            usize
note: required by a bound in `VaList::<'f>::next_arg`
```

(`i16`, `u8`, and `bool` fail the same way. Read `c_int` / `c_uint` / `c_double` and narrow afterwards with a checked conversion.)

Passing an unpromoted type *from Rust*:

```
// promote.rs
use std::ffi::c_int;

unsafe extern "C" fn sum_ints(count: c_int, mut args: ...) -> c_int {
    let mut total = 0;
    for _ in 0..count {
        total += unsafe { args.next_arg::<c_int>() };
    }
    total
}

fn main() {
    let small: u8 = 7;
    let ratio: f32 = 0.5;
    let s = unsafe { sum_ints(2, small, ratio) };
```

```
    println!("{s}");
}
```

```
rustc --edition 2024 promote.rs
```

```
error[E0617]: can't pass `u8` to variadic function
  --> promote.rs:15:34
   |
15 |     let s = unsafe { sum_ints(2, small, ratio) };
   |                                  ^^^^^
   |
help: cast the value to `c_uint`
   |
15 |     let s = unsafe { sum_ints(2, small as c_uint, ratio) };
   |                                         +++++++++

error[E0617]: can't pass `f32` to variadic function
  --> promote.rs:15:41
   |
15 |     let s = unsafe { sum_ints(2, small, ratio) };
   |                                          ^^^^^
   |
help: cast the value to `c_double`
   |
15 |     let s = unsafe { sum_ints(2, small, ratio as c_double) };
   |                                                ++++++++++++

error: aborting due to 2 previous errors
```

Notice the second error also exposes a *logic* bug the compiler cannot see: `sum_ints` reads `c_int`s, but the caller is now about to send a `c_double`. Casting to silence E0617 is correct only if the cast type matches what the callee reads.

The trap

Treating `next_arg` as if it were type-checked. It is a raw read guided only by your protocol. Every one of these is undefined behaviour, and none produces a compile error in the callee:

- **Reading past the end.** There is no length; `next_arg` on an exhausted list reads whatever is in the next register or stack slot. A missing sentinel in Case 2 walks off into garbage pointers.
- **Reading the wrong type.** Caller passes `long long`, you read `c_int`; caller passes `double`, you read a pointer. The `VaList` docs allow only *compatible* types: identical types, same-size integers whose value fits both, compatible pointers, or `*const c_void` ↔ `*const u8`/`i8`.
- **Trusting integer widths across platforms.** `c_long` is 64-bit on Linux x86_64 and 32-bit on Windows x86_64. Read `c_long` when the C side passes `long`, not `i64`. The `VaArgSafe` docs say implementations for `i32`/`usize` "shouldn't be relied upon directly"; spell the C type.

- **Re-reading a consumed list.** After `vsnprintf(…, ap)` the list is spent; use `ap.clone()` *before* the first consumer if you need two passes.
- **Panicking out of an `extern "C"` function.** A panic that reaches an `extern "C"` boundary aborts the process. If unwinding through C is genuinely intended, use `"C-unwind"`; otherwise catch and convert errors to return codes, as `rlog` does with its `None` branch.
- **Calling your own variadic from Rust.** Rust callers get none of the C header's format checking. Give Rust code a normal, safe function (`fn log(level: Level, msg: &str)`) and keep the variadic entry point for C.

The boring rule

> A C-variadic function in Rust is a thin, `unsafe`, `#[unsafe(no_mangle)]` door for C callers. It reads arguments only under a documented protocol — a count, a format string handed to libc's `v*` functions, or a NULL sentinel — reads only C-spelled promoted types (`c_int`, `c_uint`, `c_long`, `c_double`, pointers), clones before any second pass, never unwinds through `extern "C"`, and ships with a C header carrying `format(printf, …)` or `sentinel` so the *caller's* compiler checks what the callee cannot.

Checklist before merging:

1. `# Safety` section states the exact protocol (how many, which types, how the list ends).
2. Real formatting is delegated to `vsnprintf`/`vfprintf`; you do not reimplement `%` parsing.
3. Every read type is a `core::ffi` C type or a raw pointer.
4. The C header has `__attribute__((format(printf, N, M)))` or `__attribute__((sentinel))`, and CI builds C callers with `-Werror=format`.
5. Rust-side callers use a safe non-variadic API instead.

Try this

1. Add `rlog_count(void)` to `rlog.rs` that returns how many messages have been logged (an `AtomicU32`), call it from `main.c`, and confirm it prints `3`.
2. Remove `(char *)NULL` from the `rjoin_len` call in `main.c`, rebuild with `-Wall`, and read the warning. Then delete `__attribute__((sentinel))` from the header and observe that the warning disappears while the bug remains — that is why the attribute is part of the API.
3. Write `vrlog(int level, const char *fmt, va_list ap)` in Rust taking `ap: VaList<'_>`, make `rlog` a one-line forwarder to it, and export both. Call `vrlog` from a C wrapper function that does its own `va_start`/`va_end`.
4. Change `vformat` to pass `ap` (not `ap.clone()`) to the measuring `vsnprintf` call. Explain why the second call is now undefined behaviour even if it appears to work on your machine.
5. Run `rustc --print native-static-libs` for your target and wire the result into a `Makefile` rule so the link line is never guessed.

Sources

- Announcing Rust 1.99.0 — extern "C" variadics (Rust Blog, 2026-10-01)
- `core::ffi::VaList` and `VaArgSafe` — std/core API docs
- The Rust Reference — Functions: C-variadic parameters and External blocks: variadic functions
- GCC manual, *Common Function Attributes* (`format`, `sentinel`)

FFI bool miscompile on x86_64 (Rust 1.99.0 → 1.99.1)

FFI bool miscompile on x86_64 (Rust 1.99.0 → 1.99.1)

Rust's `bool` is not a C `int`. Across the `extern "C"` boundary it must follow the platform ABI: on **x86-64 SysV**, a `_Bool` / `bool` return is a byte whose **bits 1–7 are zero** (bits above that are undefined). If the compiler forgets to zero-extend, a C caller that reads the return as `bool` can observe values like `16` instead of `0` or `1`—and then every `if (flag)` in C becomes a landmine.

Rust **1.99.0** shipped a regression that dropped that zero-extension for FFI `bool` returns on several x86_64 targets. **1.99.1** (2026-10-15) restored it. This chapter is the incident report you want next to the FFI chapter: what broke, how to see it in a register dump, who is affected, and what to do on a pinned toolchain.

Learning goals

By the end of this chapter you can:

- State the x86-64 ABI rule for returning `bool` / `_Bool`
- Reproduce the 1.99.0 miscompile with a tiny Rust cdylib + C main
- Recognize the missing `andb $0x1, %al` (or equivalent zero-extend) in disassembly
- Upgrade to **1.99.1** (or pin 1.98.x / avoid `bool` at the boundary) as mitigation

Thesis

> Safe Rust returning `bool` to C is only as trustworthy as codegen's ABI lowering. A missing zero-extend is a silent miscompile: the source looks right, the bit in `%al` does not.

Mental model

```
Rust: (flags & 1) == 1     →  i1 in LLVM


ABI: return bool in %al with bits 1-7 cleared

   1.98.0:  mov %edi,%eax ; and $1,%al ; ret
   1.99.0:  mov %edi,%eax ; ret                    (raw 0x10 survives)
```

```
      1.99.1:  zero-extend restored
```

Layer	What went wrong in 1.99.0
LLVM IR	`zeroext` on `i1` returns was dropped for x86_64 SysV (related fallout from broader scalar-extension changes)
Machine code	No mask/extend before `ret`
C caller	`(int)ret` can print `16` when input was `0x10`

Primary tracker: rust-lang/rust#163911. Fix release notes: Announcing Rust 1.99.1.

Case 1 — Minimal reproduction

Save the Rust side:

```rust
// lib.rs - cdylib exporting a C ABI bool
#[unsafe(no_mangle)]
pub unsafe extern "C" fn rust_fn_returning_bool(flags: u8) -> bool {
    (flags & 0x1) == 0x1
}
```

Save the C caller:

```c
/* main.c */
#include <stdio.h>
#include <stdbool.h>

extern bool rust_fn_returning_bool(unsigned char);

int main(void) {
    bool ret = rust_fn_returning_bool(0x10);
    printf("ret = %d\n", (int)ret);
    return 0;
}
```

Build and run (Linux x86_64):

```
rustc --crate-type cdylib -O -o librust_bool.so lib.rs
gcc -O2 -o ffi main.c -L. -lrust_bool -Wl,-rpath,'$ORIGIN'
./ffi
```

Genuine output on this box (rustc 1.99.0)

```
$ rustc --version
rustc 1.99.0 (b940084d7 2026-09-28)

$ ./ffi
ret = 16
```

Expected on a fixed toolchain: `ret = 0` (because `0x10 & 1 == 0`).

Disassembly of the export on 1.99.0 (excerpt):

```
00000000000127cc <rust_fn_returning_bool>:
   127cc:   89 f8                   mov    %edi,%eax
   127ce:   c3                      ret
```

On 1.98.0 the same function included `andb $0x1, %al` before `ret` (see the issue). The logical compare was optimized assuming LLVM would still zero-extend the `i1` at the ABI boundary—and it no longer did.

Case 2 — Who was affected

Per the issue and write-ups circulating with 1.99.1:

- **Targets:** x86_64 SysV-style environments (Linux, musl, FreeBSD, Android, many bare-metal x86_64 configs). aarch64 / loongarch64 reproductions in the report masked correctly.
- **Surface:** Rust functions with `extern "C"` (or other C-like ABIs) that **return** `bool`, especially when compiled with optimizations so LLVM folds the compare into a raw register move.
- **Not required:** `unsafe` in the callee body beyond FFI—the export itself is the hazard. Callers in C/`bindgen` glue are where symptoms show up (wrong branch, wrong `printf`, corrupted flag packing).

If you ship a cdylib / staticlib consumed by C/C++/other languages, inventory `-> bool` at the boundary.

Case 3 — Mitigations

1. **Upgrade:** `rustup update stable` → **1.99.1** or newer, then **rebuild** all Rust artifacts that cross FFI (don't assume an old `.so` is fine).
2. **Temporary pin:** Stay on **1.98.x** until you can move, if 1.99.0 is already in CI.
3. **ABI hygiene:** Return `u8` (0/1) or `c_int` at the boundary and convert explicitly:

```rust
#[unsafe(no_mangle)]
pub unsafe extern "C" fn rust_fn_returning_u8(flags: u8) -> u8 {
    u8::from((flags & 0x1) == 0x1)
}
```

4. **Test in CI:** Keep a C (or `cc` crate) smoke test that passes a non-0/1 pattern in the low byte's upper bits and asserts a clean 0/1 out—exactly the repro above.

The trap

- Assuming "it compiled and linked" means ABI correctness—miscompiles return successfully.
- Only testing `true`/`false` inputs that already clear high bits (`0` and `1`), which hide the bug.
- Upgrading `rustc` in the developer laptop but shipping libraries built with 1.99.0 in a Docker layer.

The boring rule

Treat `bool` across FFI as a **checked ABI contract**. Prefer explicit integer statuses at the boundary, pin a toolchain with a known-good 1.99.1+, and keep one C-linked regression test that would have printed `16`.

Try this

1. Run the repro on your machine; note `rustc --version` and `./ffi` output.
2. `rustup update stable` to 1.99.1+, rebuild, confirm `ret = 0`.
3. `objdump -d` the export before/after and look for the zero-extend.

Sources

- Announcing Rust 1.99.1 — official fix description
- rust-lang/rust#163911 — miscompilation report, IR/`zeroext` analysis, P-critical
- yeslogic/rust-bool-ffi — upstream minimal reproduction
- x86-64 psABI notes on `_Bool` return zeroing (discussed in the issue thread / LLVM history)

Inspiration

- Rapid response around Rust 1.99.0 FFI bool returns (issue filed 2026-10-07, point release 2026-10-15). Chapter cites the Rust blog and GitHub issue.

Verification

Genuine on this box with **rustc 1.99.0** + **gcc**: `./ffi` printed `ret = 16`; `objdump` showed `mov %edi,%eax; ret` without `and`. 1.99.1 was not installed here (scout runs do not upgrade toolchains); fixed behavior is taken from the official 1.99.1 announcement and the issue resolution.

Safe Rust UAF via covariant field assignment (rust#164250)

Safe Rust UAF via covariant field assignment (rust#164250)

Rust's type system is supposed to make use-after-free impossible in safe code. Issue rust-lang/rust#164250 (opened 2026-10-13, labels `I-unsound`, `P-high`, `A-variance`, `A-borrow-checker`) shows a counterexample: **no `unsafe` keyword**, yet a `Drop` impl can observe a dangling reference. A minimized form has compiled since **Rust 1.4** (2015). This chapter is the incident card next to the unsafe / FFI chapters: what breaks, how to reproduce on current stable, and what to audit in your own crates — not a claim that everyday application code is doomed.

Learning goals

By the end of this chapter you can:

- State the bug in one sentence: lifetime bounds on a type are not fully enforced when a **covariant field** is assigned after construction
- Reproduce the `MyType` / `Drop` smuggling pattern on **rustc 1.99.0**
- Separate "malicious `Drop`" exploitability from accidental UAF in ordinary code (per the Types Team note on the issue)
- Know where to watch for fixes (Types Team / Project Zero work; do not "patch" this in user code with `unsafe`)

Thesis

> If the compiler accepts a shorter-lived reference into a field whose type promised a longer lifetime, `Drop` runs with a lie. Safe code can then read freed stack memory — the soundness bug is in the type checker, not in your `unsafe` blocks.

Mental model

```
struct MyType<'a, 'b: 'a, 'c>(&'c mut &'a str, &'b str);

                                      field 1: covariant in 'b
       Drop assumes 'b: 'a (and related bounds) still hold

construction:  MyType(&mut me_out, "")     // '' is 'static-ish / long enough
later assign:  q.1 = &local_string;        // shorter 'b sneaks in
```

```
scope ends:    Drop runs → *me_out = dangling
after scope:   print(me_out) → UAF
```

Piece	Role
Covariance on a field	Allows treating a shorter lifetime as a longer one when unifying / assigning
Lifetime bounds on the struct (`'b: 'a`, `'a: 'static`, …)	Documented intent; **not** re-checked thoroughly on field assignment in this bug
`Drop`	Runs while the type system still believes the long lifetime
No `unsafe` in the repro	The unsoundness is accepting the program, not an explicit transmute

The Types Team note on the issue: this would be `P-critical` on novelty grounds, but an 11-year-old hole that stayed unnoticed is hard to hit by accident; exploiting it generally needs an **intentionally malicious `Drop`**. Treat production risk as real but not "rewrite everything tomorrow".

Worked examples

Case 1: Minimal Drop smuggling (reproduced here)

Verified on this box: **rustc 1.99.0** (`b940084d7 2026-09-28`). The program compiles; after the inner scope, `me_out` prints garbage (use-after-free of the `String` buffer).

```
// uaf_covariant_field.rs - safe Rust; reproduces rust#164250 shape (lcnr / theemathas)
// rustc 1.99.0: compiles and runs; output is nondeterministic garbage / asan would trap

struct MyType<'a: 'c, 'b: 'a, 'c>(&'c mut &'a str, &'b str);

impl<'a: 'c, 'b: 'a, 'c> Drop for MyType<'a, 'b, 'c> {
    fn drop(&mut self) {
        // Type system thinks self.1 lives at least as long as 'a
        *self.0 = self.1;
    }
}

fn main() {
    let mut me_out = "rarw";
    {
        let l = String::from("some cool string");
        let mut q = MyType(&mut me_out, "");
        q.1 = &l; // assigns a short-lived &str into a field that required 'b: 'a
    } // l dropped; Drop of q already wrote &l into me_out
    println!("me_out: {}", me_out);
```

```
}
```

```
$ rustc --version
rustc 1.99.0 (b940084d7 2026-09-28)
$ rustc uaf_covariant_field.rs -o uaf_covariant_field
$ ./uaf_covariant_field
me_out: <non-deterministic garbage bytes>
```

(Exact bytes vary; AddressSanitizer builds crash with `stack-use-after-scope` — see the Godbolt links on the issue.)

Case 2: `'static` bound bypass with a kept value

The original report uses a `Mutex<Vec<Box<dyn Any + Send>>>` to keep a `T: 'static` value that was actually a short reference. On 1.99.0 here it compiled and printed the **later** stack string that reused the slot:

```
// uaf_kept_static.rs - original-report shape; rustc 1.99.0 prints the filler String
use std::sync::Mutex;

static KEPT: Mutex<Vec<Box<dyn std::any::Any + Send>>> = Mutex::new(Vec::new());

struct Q<T: 'static + Send>(Option<T>);

impl<T: 'static + Send> Drop for Q<T> {
    fn drop(&mut self) {
        if let Some(t) = self.0.take() {
            KEPT.lock().unwrap().push(Box::new(t));
        }
    }
}

fn main() {
    {
        let l = String::from("hello world, this is long enough");
        let mut q = Q(None);
        q.0 = Some(&l); // T inferred as &'static String, but &l is not 'static
    }
    let _z = String::from("XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX");
    let k = KEPT.lock().unwrap();
    println!("{}", k[0].downcast_ref::<&'static String>().unwrap());
}
```

```
$ rustc uaf_kept_static.rs -o uaf_kept_static && ./uaf_kept_static
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

That output is the smoking gun: the kept `&String` no longer points at `"hello world…"`.

Case 3: What to check in your code

You are looking for **custom `Drop` (or drop glue that runs user code)** combined with **post-init assignment into lifetime-carrying fields**, especially where the type constructor / bounds promised `'static` or a long lifetime. Ordinary `&T` fields without clever `Drop` are the common case and are not automatically vulnerable. Do not "fix" this by sprinkling `unsafe` — wait for a compiler fix; track #164250.

Miri (when available on nightly) reports UB on related reproducers (dangling reference in a spawned closure). This box did not run Miri; the stable `rustc` runs above are enough to show acceptance + observable UAF.

The trap

Hearing "safe Rust UAF" and concluding the borrow checker is worthless, or conversely dismissing #164250 because it needs a malicious `Drop`. The accurate posture: the invariant *failed* in a narrow variance/bounds corner; the ecosystem still relies on that invariant; `P-high` means the Types Team treats it as urgent work.

The boring rule

If you implement `Drop` and store references (or callbacks over references), construct the value with the **final** references in place when you can, and treat "assign a shorter borrow into a field that was typed longer" as a compiler bug to report — not a pattern to build on. Pin a toolchain; upgrade when the fix lands on stable.

Try this

```
rustc --version   # expect 1.99.x or newer until fixed
rustc uaf_covariant_field.rs -o /tmp/uaf && /tmp/uaf
# Optional: rustup + nightly miri on your machine (not installed here)
```

Watch rust#164250 for the fix PR; re-run the repro after upgrading — it should start failing to compile.

Sources

- rust-lang/rust#164250 — struct's lifetime requirements can be bypassed via covariant field assignment
- Types Team context note on the issue (`P-high`, malicious `Drop`, Project Zero direction)
- Godbolt links attached to the issue (with/without ASan)

Life before main: .init_array, #[used], and why LazyLock is usually the answer

Rust's stated model is that nothing of yours runs before `main`. The platform disagrees. On ELF systems (Linux, the BSDs), the dynamic linker calls every function pointer stored in the `.init_array` section before control reaches the language runtime's entry point. C++ static constructors and `__attribute__((constructor))` use this mechanism, and the `ctor` crate brings it to Rust. This chapter does the same thing by hand in about 20 lines so you can see exactly what the crate is hiding. It also shows the one attribute whose absence makes the whole trick silently do nothing.

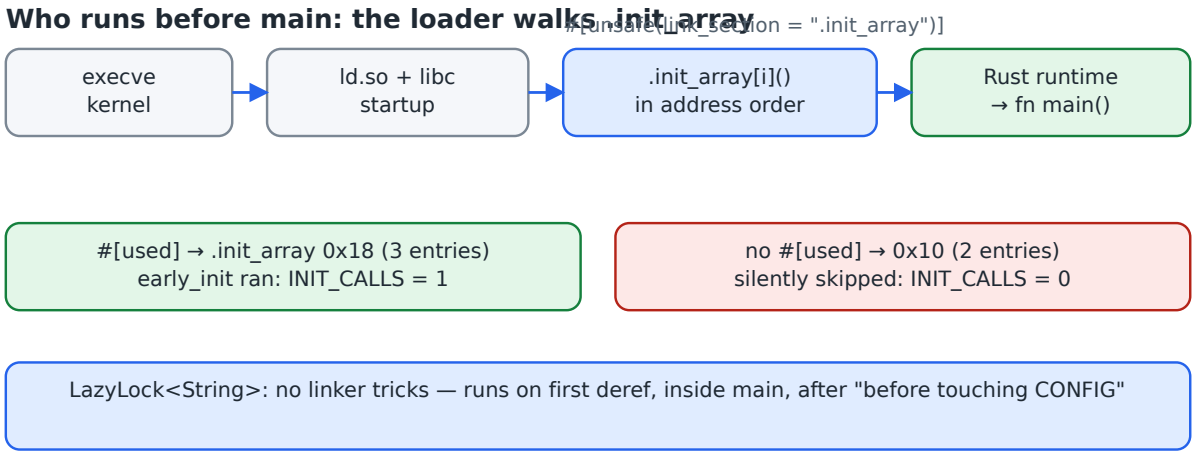


Figure 1: The loader walks .init_array before Rust's runtime calls main

The mechanism, from the ELF spec

The System V gABI defines `SHT_INIT_ARRAY` as "an array of pointers to initialization functions". Its dynamic-section chapter says those functions are executed by the dynamic linker in the order their addresses appear in the array. The `.fini_array` entries run in reverse order. So a constructor is nothing more than one pointer-sized value that lands in that section.

Rust gives you the two tools you need:

- `link_section` "specifies the section of the object file that a function or static's content will be placed into."
- `used` "forces a static to be kept in the output object file … even if it's never used or referenced by any other item in the crate."

The whole program

```
// Life before main without crates: a function pointer placed in .init_array.
use std::sync::atomic::{AtomicU32, Ordering};

static INIT_CALLS: AtomicU32 = AtomicU32::new(0);

extern "C" fn early_init() {
    INIT_CALLS.fetch_add(1, Ordering::Relaxed);
    // Only use things that need no runtime setup: raw write(2) to stdout.
    let msg = b"[early_init] running before main\n";
    unsafe { libc_write(1, msg.as_ptr().cast(), msg.len()) };
}

unsafe extern "C" {
    #[link_name = "write"]
    fn libc_write(fd: i32, buf: *const core::ffi::c_void, n: usize) -> isize;
}

#[used]
#[unsafe(link_section = ".init_array")]
static EARLY_INIT: extern "C" fn() = early_init;

// A lazily-initialised alternative that needs no linker tricks.
static CONFIG: std::sync::LazyLock<String> = std::sync::LazyLock::new(|| {
    println!("[LazyLock] initialising on first use");
    "mode=fast".to_string()
});

fn main() {
    println!("main: INIT_CALLS = {}", INIT_CALLS.load(Ordering::Relaxed));
    println!("main: before touching CONFIG");
    println!("main: CONFIG = {}", *CONFIG);
    println!("main: CONFIG again = {}", *CONFIG);
}
```

Notes on the choices:

- `early_init` is `extern "C"` because the loader calls it with the platform C ABI, not Rust's ABI.
- It calls only `write(2)`. The `ctor` docs give the rule plainly: constructor code "should be careful to limit itself to libc functions and code that does not rely on Rust's stdlib services." `println!` takes the stdout lock and touches a lazily set-up `Stdout`. That happens to work on glibc today, but nothing promises it, so the constructor avoids it.
- The `write` declaration uses `*const core::ffi::c_void`. My first draft used `*const u8`, and rustc warned that this was a "suspicious definition of the runtime `write` symbol", because it clashes with std's own declaration of the same symbol. Match the C prototype exactly.

Real output (rustc 1.99.1, x86_64 Linux, glibc 2.41)

```
$ rustc --edition 2024 -O before.rs -o before && ./before
[early_init] running before main
main: INIT_CALLS = 1
main: before touching CONFIG
[LazyLock] initialising on first use
main: CONFIG = mode=fast
main: CONFIG again = mode=fast
```

The constructor ran once, before `main`'s first line. The `LazyLock` ran once, inside `main`, on first dereference, and not again on the second.

Drop `#[used]` and it silently disappears

Here is the same file with only the `#[used]` line removed:

```
$ rustc --edition 2024 -O nouse.rs -o nouse
warning: function `early_init` is never used
warning: static `EARLY_INIT` is never used
warning: function `libc_write` is never used
warning: 3 warnings emitted
$ ./nouse
main: INIT_CALLS = 0
main: before touching CONFIG
[LazyLock] initialising on first use
main: CONFIG = mode=fast
main: CONFIG again = mode=fast
```

It still compiles, it still runs, and the constructor is gone. The section headers show why:

```
$ readelf -S before | grep -A1 init_array
  [22] .init_array       INIT_ARRAY       0000000000050e48  0004ee48
       0000000000000018  0000000000000000 WAo       0     0     8
$ readelf -S nouse | grep -A1 init_array
  [22] .init_array       INIT_ARRAY       0000000000050db8  0004edb8
       0000000000000010  0000000000000000 WAo       0     0     8
```

Without `#[used]` the array holds 0x10 bytes (two 8-byte pointers that were already there, contributed by the C startup objects and the standard library — I did not trace which is which). With it, the array holds 0x18 bytes, so our entry is the third pointer. Nothing references `EARLY_INIT`, so the compiler treats it as dead and drops it. The only clue is the dead-code warning. If you treat warnings as noise, or `#![allow(dead_code)]` is set at crate level, nothing tells you.

Two more details from the reference: `#[used]` only keeps the item in the *object file*, and "the linker, however, is still free to remove it." In this binary the linker kept it, because `.init_array` is a

section the linker preserves. With a custom section name and `--gc-sections`, you would need to check again.

If you dump the raw bytes with `readelf -x .init_array before`, they are all zeros. This is a position-independent executable, so the pointers are filled in at load time by `R_X86_64_RELATIVE` relocations, not stored in the file. Don't conclude the entry is missing from the hex dump; count the section size instead.

Edition 2024: `link_section` is an unsafe attribute

In the 2024 edition, `export_name`, `link_section` and `no_mangle` must be written as `unsafe(...)`. These attributes control linking in a global symbol namespace, and getting them wrong can cause UB from otherwise safe code. The plain spelling is a hard error:

```
$ rustc --edition 2024 plain.rs
error: unsafe attribute used without unsafe
  --> plain.rs:19:3
   |
19 | #[link_section = ".init_array"]
   |   ^^^^^^^^^^^^ usage of unsafe attribute
   |
help: wrap the attribute in `unsafe(...)`
```

Under `--edition 2021` the same file builds, and the constructor runs (`[early_init] running before main`). Migrating to 2024 adds the `unsafe(...)` marker; it doesn't change what the attribute does.

The `unsafe` marker is honest. Code in `.init_array` runs:

- before `std` has finished setting up its runtime (for example, the main thread's name and the stack-overflow handlers on Unix);
- in an order you only partly control, across crates and shared libraries;
- with no way to catch a panic. Unwinding out of an `extern "C" fn` aborts the process.

What to do instead, most of the time

`std::sync::LazyLock` (stable since 1.80) covers almost every "I need a global initialised once" case:

- it runs on first access, inside `main`, with the full standard library available;
- it is thread-safe;
- it needs no linker, unsafe attribute, or platform-specific section name.

The output above shows the difference in timing: `before touching CONFIG` prints *before* the initialiser does.

Reach for a real constructor only when something has to happen before *any* user code. Examples include registering with a plugin host that `dlopen`s your `cdylib`, or self-registering test cases or

types across crates, which is what `inventory`-style crates are for. Even then, use `ctor` rather than the hand-rolled version. It knows each platform's section name (its docs show `.init_array` on Linux, `__DATA,__mod_init_func` on Apple, `.CRT$XCU` on Windows), and in its 1.x API it makes you write `#[ctor(unsafe)]` to acknowledge the same caveats.

Checklist

- ☐ `#[used]` on the static, or it is silently removed.
- ☐ `#[unsafe(link_section = ".init_array")]` on edition 2024.
- ☐ `extern "C" fn()` signature, with no arguments and no panics.
- ☐ Body uses only libc-level calls and atomics: no `println!`, allocation-heavy work, or thread spawning.
- ☐ Verified by section size (`readelf -S`), not by the zero-filled hex dump.
- ☐ Asked first whether `LazyLock` solves it, and the answer is usually yes.

Summary

Summary

Chapter Key Takeaways

- Declarative macros (`macro_rules!`) perform hygienic pattern-based code generation.
- Procedural macros transform token streams at compile time.
- `unsafe` isolates low-level pointer manipulation and FFI behind safe API boundaries.
- `#[repr(C)]` guarantees binary compatibility with standard C ABIs.

Testing, Documentation, Performance, and Deployment

Unit Tests, Integration Tests, and Doctests

Unit Tests, Integration Tests, and Doctests

Rust has first-class built-in testing support.

Unit Tests

Unit tests live in the same source files as the code they test inside a `#[cfg(test)]` module:

```rust
pub fn add(a: i32, b: i32) -> i32 {
    a + b
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_add() {
        assert_eq!(add(2, 3), 5);
    }
}
```

Integration Tests

Integration tests live in the top-level `tests/` directory and test the public API as an external consumer.

Documentation Tests (Doctests)

Examples written in markdown doc-comments (`///`) are compiled and executed during `cargo test`.

Benchmarking with Criterion

Benchmarking with Criterion

Criterion provides robust statistical benchmarking with outlier detection and regression analysis:

```
[dev-dependencies]
criterion = "0.5"

[[bench]]
name = "compute_benchmark"
harness = false
```

Writing Great Documentation

Writing Great Documentation

Write documentation comments using triple slashes `///` supporting Markdown:

```
/// Calculates the checksum for a given network payload buffer.
///
/// # Examples
///
/// ```
/// let data = [1, 2, 3];
/// let sum: u32 = data.iter().map(|&x| x as u32).sum();
/// assert_eq!(sum, 6);
/// ```
pub fn calculate_checksum(data: &[u8]) -> u32 {
    data.iter().map(|&x| x as u32).sum()
}
```

Generate and open documentation:

```
cargo doc --open
```

Profiling and Optimization Techniques

Profiling and Optimization Techniques

Techniques for production profiling: - Use `perf` or `samply` on Linux to capture CPU flamegraphs. - Configure Link-Time Optimization (LTO) and codegen units in `Cargo.toml`:

```
[profile.release]
opt-level = 3
lto = "fat"
codegen-units = 1
panic = "abort"
strip = true
```

Building and Deploying Rust Applications

Building and Deploying Rust Applications

Rust binaries can be statically compiled with musl libc for minimal, scratch Docker containers:

```
rustup target add x86_64-unknown-linux-musl
cargo build --release --target x86_64-unknown-linux-musl
```

MSRV in practice: rust-version and the MSRV-aware resolver

MSRV in practice: `rust-version` and the MSRV-aware resolver

Your crate's **minimum supported Rust version** (MSRV) is not just a property of your own code. It is a property of the whole dependency graph that `Cargo.lock` pins. One `cargo update` can pull in a transitive dependency, two or three levels down, that needs a newer compiler than you promised. Your source is unchanged, and the build breaks for everyone on the older toolchain. This chapter shows that break with real crates, then fixes it with the tools Cargo has shipped since 1.84: `package.rust-version`, resolver `"3"` (the MSRV-aware resolver, default in edition 2024), `cargo add`'s MSRV awareness, `--ignore-rust-version`, and `--locked`.

Learning goals

By the end of this chapter you can:

- Explain why a dependency with an old MSRV can still break your MSRV build
- Read Cargo's `requires Rust 1.xx` annotations and its "rustc … is not supported" error
- Switch a project to the MSRV-aware resolver and see what it downgrades and why
- Tell when the resolver *cannot* help (your own version requirement is too new)
- Make CI check the MSRV with the committed lockfile

Thesis

> `rust-version` is a promise about the build graph, not just your crate. Since Cargo 1.84 the resolver can keep that promise for you. With resolver `"3"` it prefers dependency versions whose own `rust-version` fits yours. It only does that if you opt in (edition 2024, `resolver = "3"`, or config), if dependencies declare `rust-version`, and if your version requirements leave it room.

Mental model

```
Cargo.toml requirement      →   resolver picks a version    →   Cargo.lock    →   rustc ch
url = "2.5"                     (newest that matches;           pins it           package'
                                 resolver 3: newest that                           at build
                                 ALSO fits your rust-version)
```

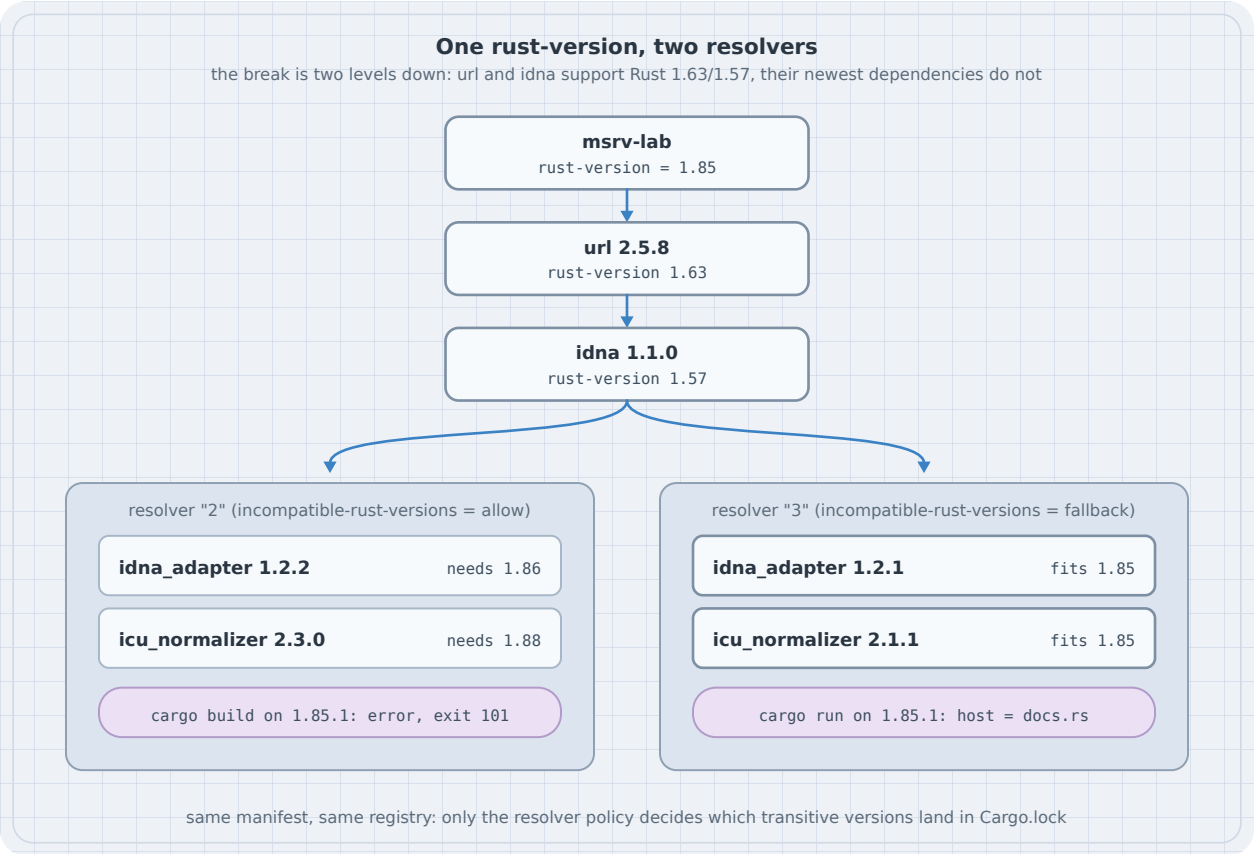


Figure 1: One package with rust-version 1.85; resolver 2 picks transitive versions needing Rust 1.86/1.88 and fails, resolver 3 picks older compatible ones and builds

Knob	Where	Effect
`package.rust-version = "1.85"`	`Cargo.toml`	Your MSRV. Cargo refuses to build on an older rustc. `cargo add` and resolver `"3"` use it to choose versions
`resolver = "3"`	`[package]` or `[workspace]`	Default for **edition 2024**. Sets `incompatible-rust-versions = "fallback"`
`resolver.incompatible-rust-versions`	`.cargo/config.toml` / `CARGO_RESOLVER_INCOMPATIBLE_RUST_VERSIONS`	`allow` (old behaviour) or `fallback` (prefer compatible, use incompatible only if nothing else matches). Respected since Cargo 1.84
`--ignore-rust-version`	CLI	Build or resolve as if no `rust-version` were set
`--locked`	CLI	Fail instead of changing `Cargo.lock`

"Your Rust version" for the resolver is the workspace's `rust-version`. If that is unset, Cargo uses the **toolchain you are running** (Case 5).

Worked examples

Run on Debian 13 x86_64 with the distribution toolchain **rustc 1.85.1 / cargo 1.85.1**. This box has no rustup toolchain installed and none was installed for this chapter. The current stable release is **Rust 1.99** (Cargo 1.99, 2026-10-01). Running an older compiler is the situation this chapter is about. One difference on a current Cargo, from the Cargo changelog: Cargo 1.92 reworded the rust-version incompatibility error, so your message text will differ. The behaviour shown here has been stable since resolver v3 was stabilised in Cargo 1.84. Crate versions are whatever crates.io served on 2026-10-17. Rerunning later may pick newer versions, but the mechanism is the same.

Run the labs in order. They share `/tmp/msrv-lab`. Each one is a bash script you can paste.

Case 1 — The break is two levels down

`url` declares `rust-version = "1.63"`. `idna` declares 1.57. Neither is the problem.

```
#!/usr/bin/env bash
# lab-01-break.sh - a direct dep with an old MSRV, a transitive dep with a new one
set -uo pipefail
rm -rf /tmp/msrv-lab && mkdir -p /tmp/msrv-lab/src && cd /tmp/msrv-lab
cat > Cargo.toml <<'TOML'
[package]
name = "msrv-lab"
```

```
version = "0.1.0"
edition = "2021"          # resolver "2": rust-version of deps is ignored
rust-version = "1.85"

[dependencies]
url = "2.5"               # url itself declares rust-version 1.63
TOML
cat > src/main.rs <<'RS'
// src/main.rs - parse one URL so the dependency is really used
fn main() {
    let u = url::Url::parse("https://docs.rs/url/latest/url/").expect("valid URL");
    println!("host = {}", u.host_str().unwrap_or("?"));
}
RS
cargo --version
cargo generate-lockfile 2>&1
cargo fetch -q              # download now so later output is only about resolution
echo '--- who pulls in the ICU crates?'
cargo tree -i icu_normalizer --edges normal 2>&1
echo '--- build on this toolchain'
cargo build -q 2>&1 | head -20
echo "[cargo build exit ${PIPESTATUS[0]}]"
```

```
cargo 1.85.1 (d73d2caf9 2024-12-31)
    Updating crates.io index
     Locking 35 packages to latest compatible versions
      Adding icu_collections v2.3.0 (requires Rust 1.88)
      Adding icu_locale_core v2.3.0 (requires Rust 1.88)
      Adding icu_normalizer v2.3.0 (requires Rust 1.88)
      Adding icu_normalizer_data v2.3.0 (requires Rust 1.88)
      Adding icu_properties v2.3.0 (requires Rust 1.88)
      Adding icu_properties_data v2.3.0 (requires Rust 1.88)
      Adding icu_provider v2.3.1 (requires Rust 1.88)
      Adding idna_adapter v1.2.2 (requires Rust 1.86)
--- who pulls in the ICU crates?
icu_normalizer v2.3.0
  idna_adapter v1.2.2
      idna v1.1.0
          url v2.5.8
              msrv-lab v0.1.0 (/tmp/msrv-lab)
--- build on this toolchain
error: rustc 1.85.1 is not supported by the following packages:
  icu_collections@2.3.0 requires rustc 1.88
  icu_locale_core@2.3.0 requires rustc 1.88
  icu_normalizer@2.3.0 requires rustc 1.88
  icu_normalizer_data@2.3.0 requires rustc 1.88
```

```
  icu_normalizer_data@2.3.0 requires rustc 1.88
  icu_normalizer_data@2.3.0 requires rustc 1.88
  icu_properties@2.3.0 requires rustc 1.88
  icu_properties_data@2.3.0 requires rustc 1.88
  icu_properties_data@2.3.0 requires rustc 1.88
  icu_properties_data@2.3.0 requires rustc 1.88
  icu_provider@2.3.1 requires rustc 1.88
  idna_adapter@1.2.2 requires rustc 1.86
Either upgrade rustc or select compatible dependency versions with
`cargo update <name>@<current-ver> --precise <compatible-ver>`
where `<compatible-ver>` is the latest version supporting rustc 1.85.1

[cargo build exit 101]
```

Even with resolver `"2"` (the edition 2021 default), Cargo 1.85 *annotates* each pick with `(requires Rust 1.88)`, but it still locks those versions. The build then fails before compiling anything: rustc checks every package's `rust-version` against itself. `cargo tree -i` shows the path. `icu_normalizer` comes from `idna_adapter`, which comes from `idna`, which comes from `url`. None of those is in your `Cargo.toml`. This is the "one transitive update breaks the build" failure, and it can happen to a crate whose own code has not changed in years.

Case 2 — Let the resolver keep the promise

```
#!/usr/bin/env bash
# lab-02-fallback.sh - same manifest, MSRV-aware resolver (resolver "3")
set -uo pipefail
cd /tmp/msrv-lab
sed -i 's/^rust-version = "1.85"/rust-version = "1.85"\nresolver = "3"            # default
grep -E '^(edition|rust-version|resolver)' Cargo.toml
echo '--- re-resolve against rust-version 1.85'
cargo update 2>&1
echo '--- build and run'
cargo run -q 2>&1
echo "[cargo run exit $?]"
```

```
edition = "2021"          # resolver "2": rust-version of deps is ignored
rust-version = "1.85"
resolver = "3"             # default for edition 2024; needs Cargo 1.84+
--- re-resolve against rust-version 1.85
    Updating crates.io index
     Locking 8 packages to latest Rust 1.85 compatible versions
 Downgrading icu_collections v2.3.0 -> v2.1.1 (available: v2.3.0, requires Rust 1.88)
 Downgrading icu_locale_core v2.3.0 -> v2.1.1 (available: v2.3.0, requires Rust 1.88)
 Downgrading icu_normalizer v2.3.0 -> v2.1.1 (available: v2.3.0, requires Rust 1.88)
```

```
 Downgrading icu_normalizer_data v2.3.0 -> v2.1.1 (available: v2.3.0, requires Rust 1.88)
 Downgrading icu_properties v2.3.0 -> v2.1.2 (available: v2.3.0, requires Rust 1.88)
 Downgrading icu_properties_data v2.3.0 -> v2.1.2 (available: v2.3.0, requires Rust 1.88)
 Downgrading icu_provider v2.3.1 -> v2.1.1 (available: v2.3.1, requires Rust 1.88)
 Downgrading idna_adapter v1.2.2 -> v1.2.1 (available: v1.2.2, requires Rust 1.86)
--- build and run
host = docs.rs
[cargo run exit 0]
```

The same manifest and the same registry give a different lockfile. Each `Downgrading … (available: v2.3.0, requires Rust 1.88)` line is the resolver telling you what it held back and why. You do not have to add pins or `=` requirements by hand, and when you later raise `rust-version`, a plain `cargo update` brings the newer versions back. Here `resolver = "3"` is set explicitly while keeping edition 2021. Moving to `edition = "2024"` gets the same default.

Case 3 — `cargo add` is MSRV-aware; your requirement wins over the resolver

```
#!/usr/bin/env bash
# lab-03-direct.sh - cargo add is MSRV-aware; a hand-written requirement can still force a b
set -uo pipefail
cd /tmp/msrv-lab
echo '--- cargo add picks the newest uuid that fits rust-version'
cargo add uuid --features v4 2>&1 | grep -vE '^ +[-+] |Features:'   # drop the feature list
grep '^uuid' Cargo.toml
cargo build -q 2>&1 && echo '[build ok]'

echo '--- now ask for a version that only exists for newer Rust'
sed -i 's/^uuid = .*/uuid = { version = "1.28", features = ["v4"] }/' Cargo.toml
cargo update -p uuid 2>&1
cargo build -q 2>&1 | head -4
echo "[cargo build exit ${PIPESTATUS[0]}]"
```

```
--- cargo add picks the newest uuid that fits rust-version
    Updating crates.io index
warning: ignoring uuid@1.28.0 (which requires rustc 1.89.0) to maintain msrv-lab's rust-vers
      Adding uuid v1.26.1 to dependencies
    Updating crates.io index
     Locking 18 packages to latest Rust 1.85 compatible versions
      Adding bumpalo v3.20.3
      Adding cfg-if v1.0.5
      Adding futures-core v0.3.34
      Adding futures-task v0.3.34
      Adding futures-util v0.3.34
```

```
      Adding getrandom v0.4.3
      Adding js-sys v0.3.106
      Adding libc v0.2.191
      Adding once_cell v1.21.4
      Adding pin-project-lite v0.2.17
      Adding r-efi v6.0.0
      Adding rustversion v1.0.23
      Adding slab v0.4.12
      Adding uuid v1.26.1 (available: v1.28.0, requires Rust 1.89.0)
      Adding wasm-bindgen v0.2.129
      Adding wasm-bindgen-macro v0.2.129
      Adding wasm-bindgen-macro-support v0.2.129
      Adding wasm-bindgen-shared v0.2.129
uuid = { version = "1.26.1", features = ["v4"] }
[build ok]
--- now ask for a version that only exists for newer Rust
    Updating crates.io index
     Locking 1 package to latest Rust 1.85 compatible version
    Updating uuid v1.26.1 -> v1.28.0 (requires Rust 1.89.0)
note: pass `--verbose` to see 8 unchanged dependencies behind latest
error: rustc 1.85.1 is not supported by the following package:
  uuid@1.28.0 requires rustc 1.89.0
Either upgrade rustc or select compatible dependency versions with
`cargo update <name>@<current-ver> --precise <compatible-ver>`
[cargo build exit 101]
```

`cargo add` chose `1.26.1` and said why (`ignoring uuid@1.28.0 (which requires rustc 1.89.0)`). Once the manifest says `"1.28"`, no version that satisfies the requirement fits Rust 1.85. The resolver's `fallback` policy is documented to pick an incompatible version instead of failing resolution, so `cargo update` succeeded, and `cargo build` refused. The fix is in `Cargo.toml`: lower the requirement or raise `rust-version`. The resolver cannot fix it.

Case 4 — Your own `rust-version`, the escape hatch, and `--locked`

```
#!/usr/bin/env bash
# lab-04-own-msrv.sh - rust-version also guards your own package
set -uo pipefail
cd /tmp/msrv-lab
sed -i 's/^uuid = .*/uuid = { version = "1.26", features = ["v4"] }/' Cargo.toml
cargo update -q -p uuid --precise 1.26.1 2>&1
sed -i 's/^rust-version = "1.85"/rust-version = "1.90"/' Cargo.toml
echo '--- claim a newer MSRV than this toolchain'
cargo build -q 2>&1 | head -4
echo "[cargo build exit ${PIPESTATUS[0]}]"
echo '--- the escape hatch (use when you know why)'
```

```
cargo build -q --ignore-rust-version 2>&1 && echo '[build ok with --ignore-rust-version]'
sed -i 's/^rust-version = "1.90"/rust-version = "1.85"/' Cargo.toml

echo '--- CI: the lockfile is the contract'
cargo build -q --locked 2>&1 && echo '[--locked build ok]'
sed -i 's/^url = "2.5"/url = "=2.5.4"/' Cargo.toml
cargo build -q --locked 2>&1 | head -3
echo "[--locked build exit ${PIPESTATUS[0]}]"
```

```
--- claim a newer MSRV than this toolchain
error: rustc 1.85.1 is not supported by the following package:
  msrv-lab@0.1.0 requires rustc 1.90

[cargo build exit 101]
--- the escape hatch (use when you know why)
[build ok with --ignore-rust-version]
--- CI: the lockfile is the contract
[--locked build ok]
error: the lock file /tmp/msrv-lab/Cargo.lock needs to be updated but --locked was passed to
If you want to try to generate the lock file without accessing the network, remove the --loc
[--locked build exit 101]
```

- Claiming `rust-version = "1.90"` makes Cargo reject your *own* package on 1.85.1 with a clear message, before rustc can fail on some syntax error deep in your code. That is what the field is for.
- `--ignore-rust-version` builds anyway. Use it when you know the newer feature is behind a cfg you do not hit, not as a habit.
- `--locked` makes the committed lockfile the contract. When the manifest and the lockfile disagree, CI fails instead of silently resolving a new graph. That is how you stop a CI run from picking up a transitive update that nobody reviewed.

Case 5 — No `rust-version`? Then your toolchain is the MSRV

```
#!/usr/bin/env bash
# lab-05-no-msrv.sh - no rust-version at all: what does resolver "3" compare against?
set -uo pipefail
cd /tmp/msrv-lab
sed -i 's/^url = "=2.5.4"/url = "2.5"/; /^rust-version = /d' Cargo.toml
grep -E '^(edition|rust-version|resolver)' Cargo.toml
rm -f Cargo.lock
cargo generate-lockfile 2>&1 | grep -E 'Locking|uuid|icu_normalizer '
cargo build -q 2>&1 && echo '[build ok on rustc 1.85.1]'
```

```
edition = "2021"          # resolver "2": rust-version of deps is ignored
resolver = "3"             # default for edition 2024; needs Cargo 1.84+
     Locking 53 packages to latest Rust 1.85.1 compatible versions
      Adding icu_normalizer v2.1.1 (available: v2.3.0, requires Rust 1.88)
      Adding uuid v1.26.1 (available: v1.28.0, requires Rust 1.89.0)
[build ok on rustc 1.85.1]
```

With `rust-version` removed, resolver `"3"` resolved against **"Rust 1.85.1"**, the toolchain running `cargo`. That is convenient on one machine and a trap on a team. A developer on 1.99 and a CI runner on 1.85 generate *different* lockfiles from the same manifest. Declare `rust-version` and the result no longer depends on who ran `cargo update`.

Case 6 — Check the MSRV in CI (configuration, not run here)

This needs a second toolchain (`rustup toolchain install`), which this box does not have, so it was not run. It is the pattern the Cargo book's "Rust version" chapter and its CI guide describe: verify the MSRV with the lockfile you ship.

```yaml
# .github/workflows/msrv.yml - build the committed Cargo.lock on the declared MSRV
name: msrv
on: [push, pull_request]
jobs:
  msrv:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5
      - run: rustup toolchain install 1.85 --profile minimal
      - run: cargo +1.85 check --locked --all-targets
```

Pair it with a normal job on the latest stable, so you also see breakage that a newer compiler would catch. For libraries, remember that downstream users do not use your `Cargo.lock`. Their resolver, their `rust-version` and their toolchain decide what they get. The `rust-version` field on every crate in the graph is what lets their resolver choose well.

The trap

The trap is reading MSRV as "the oldest compiler that compiles *my* code". You test once on the old toolchain, it works, and you ship. Weeks later a routine `cargo update`, or a fresh clone without a lockfile, pulls `icu_normalizer 2.3.0` three levels down, and the MSRV build fails with no change to your code. The two usual fixes make it worse. Pinning the transitive crate with `=` in your own `Cargo.toml` freezes it for every downstream user. Dropping `rust-version` makes Cargo resolve against whichever toolchain happens to run, so the lockfile changes from one machine to the next.

The boring rule

Declare `package.rust-version`. Use resolver `"3"` (edition 2024 gives it to you). Let `cargo add` choose versions. Commit `Cargo.lock` for applications and build with `--locked` in CI. Keep one CI job on the declared MSRV and one on latest stable. When a dependency needs a newer compiler, decide deliberately: raise `rust-version` (a minor-version bump per the Cargo book) or keep the older dependency. Don't hand-pin transitive crates.

Try this

1. In `/tmp/msrv-lab`, set `resolver = "2"`, run `cargo update`, and confirm the ICU crates jump back to 2.3.0. Then set `CARGO_RESOLVER_INCOMPATIBLE_RUST_VERSIONS=fallback` for one `cargo update` and compare.
2. Raise `rust-version` to `"1.88"`, run `cargo update`, and read which `Downgrading` notes turn into `Updating`.
3. Use `cargo update icu_normalizer@2.1.1 --precise 2.3.0` with resolver `"3"`. What does `--precise` do to the MSRV policy?
4. Add a second workspace member with no `rust-version` that also depends on `url = "2.5"`. Which `idna_adapter` does the shared lockfile get, and why? (Hint: version unification in the resolver chapter.)
5. On a machine with rustup, run the Case 6 workflow steps locally (`cargo +1.85 check --locked --all-targets`) against one of your own crates.

Sources

- The Cargo Book, "Rust version" (`rust-version`, support expectations, policies): https://doc.rust-lang.org/cargo/reference/rust-version.html
- The Cargo Book, "Dependency resolution": resolver versions (`"3"` = edition 2024 default, Rust 1.84+) and the Rust-version section: https://doc.rust-lang.org/cargo/reference/resolver.html
- The Cargo Book, configuration: `resolver.incompatible-rust-versions` (`allow` / `fallback`, MSRV 1.84): https://doc.rust-lang.org/cargo/reference/config.html#resolverincompatible-rust-versions
- The Cargo Book, `cargo add`, `cargo update` (`--precise`), and the `--locked` / `--ignore-rust-version` flags: https://doc.rust-lang.org/cargo/commands/
- Cargo changelog (1.84 stabilises resolver v3; 1.92 "Improve error message for rust-version incompatibility diagnostics"; 1.99 is current, released 2026-10-01): https://doc.rust-lang.org/nightly/cargo/CHANGELOG.html
- Rust 2024 edition guide, "Cargo: Rust-version aware resolver": https://doc.rust-lang.org/edition-guide/rust-2024/cargo-resolver.html
- crates.io API (`rust_version` per published version) for the versions used here: `https://crates.io/api/v1/`

build.rs Runs As You: Build Scripts and the Supply Chain

build.rs Runs As You: Build Scripts and the Supply Chain

Thesis

Adding a dependency doesn't just add code to your binary. It adds code that runs **on your machine at build time**. The Cargo reference puts it plainly: placing a file named `build.rs` in a package root "will cause Cargo to compile that script and execute it just before building the package" (Build Scripts). Procedural macros are the same: they are compiler plugins that run inside `rustc`. Neither is sandboxed. They run with your user, your environment, your files and your network.

That is why typosquatting works so well against Rust developers: a crate named almost like `proc-macro2` only has to be *built* to act. This chapter shows exactly when that code runs, using a harmless look-alike crate, and what a simple sandbox does and doesn't stop.

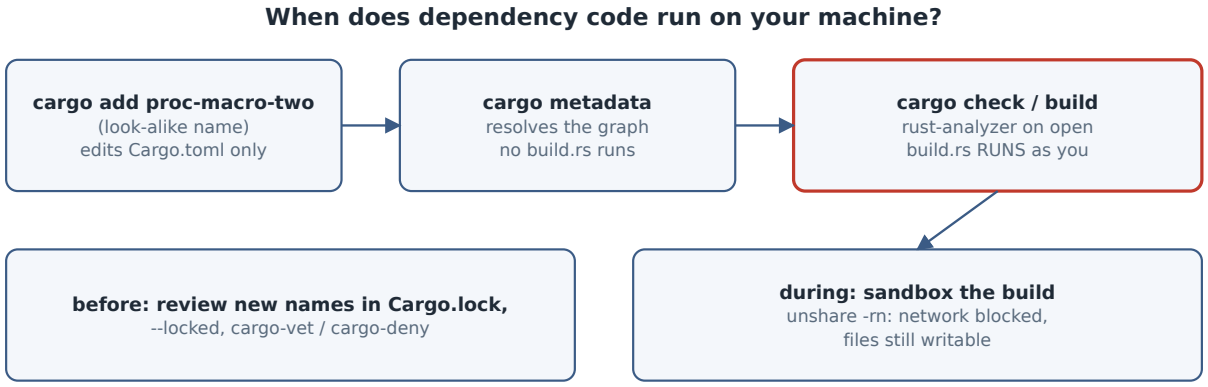


Figure 1: When does dependency code run on your machine?

A harmless look-alike

`proc-macro-two` is a local path crate. Its `build.rs` does what a malicious one would do first, but only *reports* it: it counts environment variables (where CI tokens live), counts entries in `~/.ssh`, tries an outbound TCP connection, and writes a file outside `target/`.

```
#!/bin/sh
# An app that depends on a look-alike crate. Its build.rs is harmless, but it
# shows what any build script can do: read your environment and files, write
```

```
# outside target/, and open network connections, all before main() exists.
set -e
rm -rf app proc-macro-two /tmp/buildrs-was-here
cargo new -q --lib proc-macro-two
cat > proc-macro-two/build.rs <<'RS'
use std::{env, fs, net::TcpStream, time::Duration};
fn main() {
    let home = env::var("HOME").unwrap_or_default();
    let ssh = fs::read_dir(format!("{home}/.ssh")).map(|d| d.count()).unwrap_or(0);
    let vars = env::vars().count();
    let net = match "1.1.1.1:443".parse() {
        Ok(a) => match TcpStream::connect_timeout(&a, Duration::from_secs(2)) {
            Ok(_) => "network: connected to 1.1.1.1:443".to_string(),
            Err(e) => format!("network: {e}"),
        },
        Err(_) => unreachable!(),
    };
    let report = format!("build.rs ran as uid {}: saw {vars} env vars, {ssh} entries in ~/.s
        fs::read_to_string("/proc/self/status").unwrap_or_default()
            .lines().find(|l| l.starts_with("Uid:")).unwrap_or("").split_whitespace().nth(1)
    fs::write("/tmp/buildrs-was-here", &report).unwrap();
    println!("cargo::warning={}", report.trim());
}
RS
cargo new -q app
cd app
cargo add -q --path ../proc-macro-two
```

When it runs

```
#!/bin/sh
cd "$(dirname "$0")/app"
set -x
cargo metadata --format-version 1 --offline > /dev/null; ls /tmp/buildrs-was-here
cargo check --offline 2>&1 | grep -v "^ *Finished\|^ *Checking\|^ *Compiling"
cat /tmp/buildrs-was-here
rm -f /tmp/buildrs-was-here; cargo clean -q
unshare -rn cargo check --offline 2>&1 | grep -v "^ *Finished\|^ *Checking\|^ *Compiling"
cat /tmp/buildrs-was-here
```

Output with cargo 1.99.1 on Linux:

```
+ cargo metadata --format-version 1 --offline
+ ls /tmp/buildrs-was-here
```

```
ls: cannot access '/tmp/buildrs-was-here': No such file or directory
+ cargo check --offline
+ grep -v ^ *Finished\|^ *Checking\|^ *Compiling
warning: proc-macro-two@0.1.0: build.rs ran as uid 1000: saw 111 env vars, 0 entries in ~/.s
+ cat /tmp/buildrs-was-here
build.rs ran as uid 1000: saw 111 env vars, 0 entries in ~/.ssh; network: connected to 1.1.1
+ rm -f /tmp/buildrs-was-here
+ cargo clean -q
+ unshare -rn cargo check --offline
+ grep -v ^ *Finished\|^ *Checking\|^ *Compiling
warning: proc-macro-two@0.1.0: build.rs ran as uid 0: saw 111 env vars, 0 entries in ~/.ssh;
+ cat /tmp/buildrs-was-here
build.rs ran as uid 0: saw 111 env vars, 0 entries in ~/.ssh; network: Network is unreachabl
```

What happened:

- **`cargo metadata` didn't run it.** Resolving the dependency graph is safe, so the marker file doesn't exist yet.
- **`cargo check` ran it**, even though `check` produces no binary. The build script ran as uid 1000 (the developer), saw all 111 environment variables, reached `1.1.1.1:443` on the internet, and wrote `/tmp/buildrs-was-here`. A real attack would have sent the variables instead of counting them.
- **Editors do the same.** rust-analyzer's `rust-analyzer.cargo.buildScripts.enable` defaults to `true` ("Run build scripts (build.rs) for more precise code analysis"), and proc-macro support implies it (configuration). Opening a cloned repository in your editor can be enough.
- **A namespace sandbox is partial.** Under `unshare -rn` (a new user namespace mapping you to uid 0, plus an empty network namespace), the connection fails with `Network is unreachable`. The script still read the environment and still wrote to `/tmp` on the host. To contain the filesystem too, you need a container or VM with only the source tree mounted, or a dedicated build user.

Defences, in order of payoff

1. **Watch what enters `Cargo.lock`.** Most attacks need a *new* package name: a typo, a look-alike, or a dependency confusion. Review lockfile diffs in PRs like code, and build in CI with `cargo build --locked` so nothing is resolved silently.
2. **Audit before you trust.** cargo-vet records which crate versions "have been audited by a trusted entity" and fails CI for unaudited ones. cargo-deny can ban crates, restrict sources to crates.io or known git hosts, and check RustSec advisories.
3. **Vendor for hermetic builds.** `cargo vendor` plus `--offline` makes the build use exactly the reviewed sources.
4. **Sandbox untrusted builds,** including the first build of a cloned repository and any editor session in one. Cut both the network and the host filesystem, as the namespace experiment above shows.
5. **Keep secrets out of build environments.** A build job that doesn't need a publish token shouldn't have one. Everything in `env::vars()` is readable by every build script in the graph.

Takeaways

- `build.rs` and proc macros run at **build time, as you**, with full access. `cargo check` and your editor trigger them, while `cargo metadata` doesn't.
- A look-alike name is the whole attack. Lockfile review, `--locked`, and cargo-vet or cargo-deny stop it before it builds.
- Sandboxes have to cover the network **and** the filesystem: `unshare -rn` alone left the host file write intact.

Sources

- Cargo reference: Build Scripts (including `cargo::warning=MESSAGE`), `cargo vendor`, `--locked`
- rust-analyzer configuration
- cargo-vet, cargo-deny
- `unshare(1)`: https://man7.org/linux/man-pages/man1/unshare.1.html

Cargo min-publish-age: dependency cooldowns against fresh crates.io publishes

Cargo min-publish-age: dependency cooldowns against fresh crates.io publishes

After reading this chapter, you will understand how Cargo's dependency publish-age cooldown policy protects your project against supply-chain compromise, how to configure global and per-registry cooldown thresholds in `config.toml`, and how Cargo's resolver responds when candidate crate versions are too young.

Supply-chain attacks on package registries frequently rely on speed. When an attacker compromises a popular crate maintainer's credentials or takes over an abandoned package, they publish a malicious version and rely on automated CI runs or `cargo update` calls to pull it into downstream projects before human auditors or automated scanners detect the breach.

Stabilized in Rust 1.100, Cargo's `min-publish-age` resolver feature creates an automatic safety buffer (or "cooldown period") for newly published crate versions. By configuring Cargo to ignore releases published within a specific window—such as 7 days—your builds automatically reject fresh releases until the community has had time to inspect and report potential security issues.

Supply-Chain Window of Vulnerability

When a new package version is uploaded to crates.io, it is immediately available for download by cargo clients. Security advisories, maintainer revokations, and automated security scanning tools take time to catch compromised releases.

```
Timeline of a Package Compromise:
T+00h: Attacker publishes malicious version 1.2.4
T+01h: CI pipelines running `cargo update` pull version 1.2.4  <-- Vulnerability Window
T+12h: Security researcher discovers payload and files advisory
T+14h: Crates.io maintainers yank version 1.2.4
```

Without a publish-age policy, any build resolving unpinned dependencies during the vulnerability window pulls version `1.2.4`. With a `min-publish-age` policy of `7 days`, Cargo's resolver acts as if version `1.2.4` does not exist during those first seven days, falling back to the latest version that satisfies both your semver requirement and the age criteria (such as version `1.2.3`).

Configuring `min-publish-age`

Cargo supports configuring minimum publish-age rules across all registries or on a per-registry basis inside your Cargo configuration file (`.cargo/config.toml` or `~/.cargo/config.toml`).

```toml
# config.toml
[registry]
global-min-publish-age = "7 days"

[resolver]
incompatible-publish-age = "deny"
```

Configuration Parameters

Key	Section	Accepted Values	Description
`global-min-publish-age`	`[registry]`	Human-readable duration (e.g., `"7 days"`, `"48 hours"`)	Sets the default minimum age required for all candidate versions across all registries.
`min-publish-age`	`[registries.<name>]`	Human-readable duration	Overrides `global-min-publish-age` for a specific named registry.
`incompatible-publish-age`	`[resolver]`	`"deny"`, `"allow"`, `"warn"`	Controls Cargo's resolver behavior when a candidate crate version fails the publish-age check.

For internal private registries where packages are built and verified within your own infrastructure, you can override the global default to zero:

```toml
[registry]
global-min-publish-age = "7 days"

[registries.internal-corp]
index = "https://crates.internal.example.com/index"
min-publish-age = "0 days"
```

Resolver Policy Modes

The `resolver.incompatible-publish-age` configuration option defines how strictly Cargo enforces publish age when resolving your dependency graph:

1. **`deny` (Default / Recommended)**: Cargo rejects any candidate version published more recently than the minimum publish age. If a `Cargo.toml` explicitly requests a version that violates this policy, Cargo halts with a resolution error.

2. **`allow`**: Cargo ignores the publish-age policy completely, permitting any published version regardless of age.
3. **`warn`**: Cargo resolves the newest semver-compatible version (even if younger than the minimum publish age), but emits a warning log informing the developer that a newly published version was selected.

> **Note**
>
> **Not run here.** The local environment in this demonstration runs rustc/cargo 1.99.1. The `min-publish-age` and `incompatible-publish-age` options require Rust 1.100 or newer. On versions prior to Rust 1.100, these configuration options are unvalidated or ignored by Cargo.

Setting Up a Demonstration Workspace

To see how Cargo structure works when configuring custom registry parameters, create a standard binary project with a local dependency setup.

```
[package]
name = "cooldown-demo"
version = "0.1.0"
edition = "2021"

[dependencies]
serde = "1.0"
```

```
[registry]
global-min-publish-age = "7 days"

[resolver]
incompatible-publish-age = "deny"
```

```
use serde::{Deserialize, Serialize};

#[derive(Serialize, Deserialize, Debug)]
struct ServerConfig {
    host: String,
    port: u16,
}

fn main() {
    let config = ServerConfig {
        host: String::from("127.0.0.1"),
        port: 8080,
    };
    let serialized = serde_json::to_string(&config).unwrap_or_default();
```

```
    println!("Serialized config: {}", serialized);
}
```

When building this crate under Rust 1.100+, running `cargo build` queries the index timestamps for `serde` versions.

Run `cargo check` to verify standard build output:

```
cargo check
```

Which produces the following output:

```
<<OUTPUT:cargo-check-1>>
```

How `min-publish-age` Fits into Rust Security Defenses

`min-publish-age` addresses a specific risk window in the Rust ecosystem. It works alongside existing supply-chain tools:

- **`Cargo.lock` & `cargo build --locked`**: Pins exact versions across builds. `min-publish-age` takes effect when running `cargo update` or when adding new dependencies, preventing initial inclusion of premature releases.
- **`cargo-vet`**: Manages peer-reviewed audits of dependency source code. `cargo-vet` requires explicit human approval for code changes, whereas `min-publish-age` provides an automated time-based safeguard for un-audited minor/patch updates.
- **`cargo-audit` & `cargo-deny`**: Scans `Cargo.lock` against known vulnerability databases (RustSec). These tools depend on advisories being filed *after* a flaw is discovered; `min-publish-age` protects builds during the time gap before an advisory is published.

Tool / Mechanism	Defense Trigger	Primary Protection
`Cargo.lock`	Build time (`--locked`)	Deterministic builds, prevents drift
`min-publish-age`	Resolution time (`cargo update`)	Supply-chain cooldown buffer against fresh publishes
`cargo-vet`	Audit verification	Code review and trust verification
`cargo-audit`	Vulnerability scanning	Known CVE / RustSec advisory matching

Pitfalls

- **Broken emergency hotfixes**: If an upstream crate releases a critical bugfix or security patch, your build will reject it until the package reaches the `min-publish-age` threshold. For

emergency updates, temporarily lower the duration in `.cargo/config.toml` or override the resolver behavior via CLI flags.

- **Private registries blocking builds**: CI pipelines relying on private, internal registries will fail if internal packages are deployed and consumed within the cooldown window. Always configure `registries.<name>.min-publish-age = "0 days"` for internal registries.
- **Offline builds**: `min-publish-age` relies on index metadata timestamps. In offline mode (`--offline`), Cargo relies on cached registry index timestamp metadata.

Key takeaways

- `min-publish-age` is a Cargo resolver policy stabilized in Rust 1.100 that skips crate versions younger than a configured duration.
- Configured in `.cargo/config.toml` under `[registry]` (`global-min-publish-age`) and `[resolver]` (`incompatible-publish-age`).
- Protects projects during the initial window of vulnerability before malicious publishes are discovered and yanked from crates.io.
- Complements exact version locking (`Cargo.lock`), code auditing (`cargo-vet`), and advisory scanners (`cargo-audit`).

Sources

- Cargo Reference: registry.global-min-publish-age
- Cargo Reference: resolver.publish-age
- Cargo Reference: Unstable Features: min-publish-age

Summary

Summary

Chapter Key Takeaways

- Unit tests, integration tests, and doctests are integrated into `cargo test`.
- Criterion provides statistical benchmarking.
- Triple-slash doc comments generate standard HTML documentation.
- Release profiles with LTO produce compact, high-performance static binaries.

The Rust Ecosystem: Essential Crates for Production Development

serde: Serialization and Deserialization

serde: Serialization and Deserialization

Serde is the framework for serializing and deserializing Rust data structures efficiently and generically.

```
// serde derives Serialize and Deserialize for zero-allocation JSON parsing
// #[derive(Serialize, Deserialize, Debug)]
// struct User { id: u64, username: String }
```

clap: Command-Line Argument Parsing

clap: Command-Line Argument Parsing

Clap provides declarative, type-safe command-line argument parsing via procedural derive macros:

```
// #[derive(Parser)]
// #[command(author, version, about)]
// struct Cli { port: u16 }
```

reqwest: HTTP Client

reqwest: HTTP Client

Reqwest provides ergonomic asynchronous and synchronous HTTP clients with connection pooling, TLS, and JSON decoding.

axum: Web Framework

axum: Web Framework

Axum is a modular, ergonomic web application framework built on top of Tokio, Tower, and Hyper.

actix-web: High-Performance Web Framework

actix-web: High-Performance Web Framework

Actix-web is a high-performance, actor-inspired web framework frequently ranking among the fastest web servers in industry benchmarks.

Project 2: Async HTTP API with axum, sqlx, and tracing

Project 2: Async HTTP API with axum, sqlx, and tracing

In Project 2, we build an asynchronous microservice handling user registrations and order audits backed by SQLite/PostgreSQL with structured logging:

```
// Architecture of the User Service API
// Router binds routes: POST /users, GET /users/:id
// Handlers extract typed JSON, execute SQL queries via sqlx, and emit tracing spans
```

Testing the User API

Testing the User API

End-to-end integration tests using Tower `ServiceExt::oneshot` to simulate HTTP requests without binding network sockets.

Benchmarking the User API

Benchmarking the User API

Load-testing the async API with `oha` or `wrk` measuring request latencies under sustained concurrent connections.

Deploying the User API

Deploying the User API

Containerizing the API into a multi-stage Docker build producing a single 15MB container running as an unprivileged user.

sqlx: Async Database Access

sqlx: Async Database Access

SQLx provides compile-time verified SQL queries without an ORM, checking query syntax against live database schemas during `cargo build`.

rayon: Data Parallelism

rayon: Data Parallelism

Rayon converts sequential iterator chains into parallel work-stealing pipelines with a simple `.par_iter()` drop-in replacement.

tracing: Structured Logging and Observability

tracing: Structured Logging and Observability

Tracing instruments Rust applications with contextual, structured events and spans compatible with OpenTelemetry and Jaeger.

chrono: Date and Time

chrono: Date and Time

Chrono handles ISO 8601 timestamps, UTC conversions, duration arithmetic, and timezone formatting.

uuid: Universally Unique Identifiers

uuid: Universally Unique Identifiers

Fast, cryptographically secure generation and parsing of UUID v4 and v7 identifiers.

rand: Random Number Generation

rand: Random Number Generation

Generates cryptographically secure pseudorandom numbers and provides distribution sampling.

regex: Regular Expressions

regex: Regular Expressions

Linear-time regular expression matching guaranteeing no catastrophic backtracking.

bytes: Efficient Binary Buffers

bytes: Efficient Binary Buffers

`Bytes` and `BytesMut` provide reference-counted zero-copy slicing for network packet processing.

futures: Async Primitives

futures: Async Primitives

Provides channels, select combinators, and join mechanisms for working with asynchronous streams.

Summary

Summary

Chapter Key Takeaways

- Serde, Clap, Reqwest, Axum, and SQLx form the production backbone for web backends and CLIs.
- SQLx enables compile-time verified database operations.
- Tracing provides production-ready observability.
- The Rust crate ecosystem provides zero-cost modular tools for systems engineering.

Servo 0.1: Embeddable Engine on crates.io

Servo 0.1: Embeddable Engine on crates.io

Thesis

On **2026-04-13** the Servo project published **`servo` 0.1.0** on crates.io — the first release of the engine **as a library**, not only as the `servoshell` demo browser. The `0.1` version number is deliberate: the team has not defined what **1.0** means yet, but the bump signals growing confidence that the **embedding API** is usable for real embedders. Monthly crates keep moving (by late 2026 the crate line includes **0.7.x** regular releases plus a parallel **0.1.x LTS** line). This chapter teaches the mental model of embedding Servo: what the crate is for, what `0.1` vs LTS vs monthly means, and which build/feature knobs matter — without downloading the engine's enormous dependency graph in the book lab.

Toolchain note: examples assume a recent stable Rust. This authoring host has **rustc 1.99.0**. Current `servo` releases advertise **MSRV 1.88.0** (see crates.io); LTS migration notes may raise clang and Rust floors when you jump major LTS lines.

Mental model

```
Your application (windowing, input, IPC, privileges)

            embedding API (servo crate + embedder traits)


  Servo library
  constellation · script · layout · paint · net · …
  optional: webgl / webgpu / webxr / media-gstreamer



OS graphics / audio / network backends (surfman, etc.)
```

Concept	Meaning
`servo` crate	The embeddable engine on crates.io (MPL-2.0)
`servoshell`	Demo browser — **not** published to crates.io; build from the GitHub tree
0.1.0 milestone	First library release; embedding API "good enough for some users," not a stability covenant

Concept	Meaning
Monthly / regular line	Fast-moving; breaking changes expected between minors
LTS line	Best-effort ~6-month cadence, ~9-month support, **security fixes only**; may need `git` deps for CVE backports
`servo-embedder-traits`	Hooks embedders implement (e.g. custom resource readers)
`baked-in-resources` / `bundled`	Default path that embeds default resources in the binary; disable and supply your own reader if you need that

Risk profile (from the Servo book): rendering **known, trusted** content inside your app is a different threat model from a general-purpose browser that loads the open web. Evaluate accordingly — Servo is still pre-1.0.

Worked examples

Case 1: Read the milestone announcement, then pin deliberately

The 0.1.0 blog post is the historical milestone. For new work in late 2026 you typically choose:

Goal	Pinning strategy
Track features / file upstream bugs easily	Latest regular release on crates.io (e.g. `0.7.x`)
Ship a product with scheduled upgrades	Current **LTS** (see LTS release in the Servo book)
Reproduce a 0.1-era tutorial	`servo = "=0.1.0"` (expect migration work later)

```
# Cargo.toml - illustrative pins (check crates.io before locking)
[dependencies]
# Regular line (example version as of 2026-10; bump intentionally):
servo = "0.7"

# Or an LTS / milestone pin when you need a slower moving API:
# servo = "=0.1.4"
```

Do **not** treat `0.1` as "more stable than 0.7" merely because the number is smaller — on crates.io, `0.1.x` after the LTS decision is the **long-term** branch family, while `0.2`–`0.7` are the regular train. Read the Servo book's LTS page before you pick.

Case 2: Embedding API shape (no full build in this chapter)

Documentation for embedding lives under The Servo Book → Embedding. The stable ideas:

1. Depend on the **`servo`** library crate (not servoshell).
2. Drive the engine from your event loop: create a `Servo` / builder, feed input, composite frames into your windowing stack.
3. Decide **resources**: keep default `bundled` / `baked-in-resources`, or disable default features and call `servo_embedder_traits::submit_resource_reader` with your own `ResourceReader` (OpenHarmony's filesystem reader is the cited pattern upstream).
4. Select **feature flags** deliberately. Modularization moved many web features behind flags; LTS migration notes recommend considering `default_web_features` when you want the broad web surface.

```
// embed_sketch.rs - API orientation only (will not compile without the servo crate
// and platform deps). Do not cargo-add servo in the book lab: the dependency
// tree and system libraries (e.g. gstreamer for media) are large.
//
// use servo::ServoBuilder; // names evolve - follow the book for your pin
//
// fn main() {
//     let mut servo = ServoBuilder::new(/* embedder handles */)
//         // .prefs(...)
//         .build();
//     // pump events → servo.handle_events(...) → present frames
// }
fn main() {
    println!("See https://book.servo.org/embedding/overview.html for current builders");
}
```

```
$ rustc --version
rustc 1.99.0 (b940084d7 2026-09-28)
$ rustc embed_sketch.rs -o /tmp/embed_sketch && /tmp/embed_sketch
See https://book.servo.org/embedding/overview.html for current builders
```

Case 3: Building without `./mach` (embedder perspective)

Upstream notes that embedding builds often go through **`cargo`** directly, but `./mach` normally sets environment variables and enables **`media-gstreamer`** by default. If you bypass mach:

- Run `./mach print-env` on a Servo checkout when cross-compiling (Android / OpenHarmony) and mirror those variables.
- Expect to install platform media/graphics dependencies yourself if you enable gstreamer features.
- Prefer feature flags over "copy servoshell's Cargo.toml blindly."

This book chapter **does not** run `cargo add servo` or `./mach bootstrap` — both pull toolchains and system packages beyond the no-install lab policy. Use the Servo book's "Building servo without mach" section on a dedicated machine.

Case 4: Migrating off 0.1 LTS (preview)

The Servo book's "Migrating from 0.1 to 0.6 LTS" list is the boring checklist when you finally jump:

- Enable `default_web_features` (or an explicit subset) after modularization.
- Re-validate **prefs** — unknown preference names can **panic**.
- Update mouse button enums (`Left/Middle/Right` → `Primary/Auxiliary/Secondary`).
- Relocate WebXR registry registration as documented for your target version.
- Budget for **MSRV** and **clang** bumps on the new LTS.

Read the full list for your target LTS; do not crochet a partial migrate.

The trap

1. **Publishing or depending on servoshell.** The demo browser is not the embeddable product line on crates.io.
2. **Assuming crates.io LTS patches always exist.** Security backports may force **`git` dependencies** when MSRV blocks a crates.io bump — plan for that in corporate allow-lists.
3. **Leaving default features on "for convenience" on constrained devices.** Media, WebGL, WebGPU, and WebXR are heavy; turn on what you need.
4. **Treating 0.1 as production-complete.** The project is explicit: no 1.0 definition yet. Trusted-content widgets full browser replacement.
5. **Skipping resource ownership.** If you disable baked-in resources and forget a `ResourceReader`, startup fails in confusing ways. The OpenHarmony example is the reference pattern.

The boring rule

Depend on the **`servo` library crate**, pick **regular vs LTS** on purpose, keep **servoshell** out of your `Cargo.toml`, and follow book.servo.org/embedding for the builder/event-loop API on the version you pinned. Evaluate threat model before you point the engine at untrusted URLs.

Try this

1. Open crates.io/crates/servo and list the latest regular version vs the latest `0.1.x`. Write one sentence on which you would pin for a shipping app and why.
2. Read the embedding overview and LTS pages in the Servo book; sketch your app's event-loop ownership on paper (who owns the window, who calls into Servo).

3. On a disposable machine (not this no-install lab), create a hello-embed crate against an LTS pin and run *only* what the book's embedding chapter shows — then stop before enabling every feature flag.
4. Skim the 0.1→0.6 migration notes and mark which items would touch your sketch.

Sources

- Servo is now available on crates.io (2026-04-13)
- crates.io: servo (version history and features)
- Servo Book — Embedding overview
- Servo Book — LTS releases

Advanced Topics: Compiler, Debugging, Security, and Anti-Patterns

The Rust Compilation Pipeline

The Rust Compilation Pipeline

Understanding how `rustc` compiles code: 1. Parsing & Macro Expansion -> AST 2. HIR (High-level Intermediate Representation) -> Type checking 3. MIR (Mid-level Intermediate Representation) -> Borrow checker & optimizations 4. LLVM IR -> Native machine code generation

A Tiny JIT with Cranelift: From Postfix Expressions to Native Code

A Tiny JIT with Cranelift: From Postfix Expressions to Native Code

After reading this chapter, you will be able to parse a reverse Polish notation (RPN) arithmetic expression, lower it into Cranelift Intermediate Representation (IR) using single static assignment (SSA) values, compile that IR in-process into executable machine code, and invoke the resulting function safely through a Rust function pointer.

Thesis

A JIT doesn't have to mean writing machine code by hand or linking LLVM. Cranelift is a code generator written in Rust. It is the compiler inside Wasmtime and the backend of `rustc_codegen_cranelift`, which can stand in for LLVM to give faster debug builds. You build SSA-form IR through its API and it emits native code for the host, in-process, at runtime.

In this chapter, a postfix expression typed at runtime becomes a native function you call through a function pointer, in about 150 lines. We check its results against a plain interpreter and time both. Only two lines are `unsafe`, and the chapter explains what each one depends on.

When a JIT pays off

A JIT is a trade. You pay to compile up front, ship a code generator in your binary and take on `unsafe` code. In return each later evaluation is cheap.

JIT compilation pays off when an expression or program is evaluated repeatedly over a large dataset or long runtime:

- **Hot user-supplied expressions**: Database filter predicates, dynamic routing rules, spreadsheet formulas, or financial risk calculations compiled once and run against many records.
- **Emulators and language runtimes**: Dynamic translation of foreign ISA instructions or language bytecode into host native code.
- **Small DSLs and rules engines** where an interpreter is the bottleneck.

Conversely, avoid JIT compilation under the following conditions:

- **One-off execution**: If an expression is evaluated only a handful of times, building IR and emitting code costs more than just interpreting it.
- **Platform restrictions**: Environments enforcing strict W^X (Write XOR Execute) memory policies without permissions for executable heap allocation (such as iOS or locked-down sandbox environments) prohibit in-process JIT compilation.

- **Security-sensitive boundaries**: Native memory execution increases the attack surface if untrusted inputs can influence codegen in ways that bypass bounds checks or memory isolation.

The Cranelift crate ecosystem

Cranelift is modularized into several focused crates. The JIT pipeline relies on five primary crates, all at version `0.137.1`:

Crate	Primary Purpose	Key Types / APIs
`cranelift-codegen`	Core IR definitions, instruction builders, ISA backends, and optimization flags.	`types::I64`, `InstBuilder`, `AbiParam`, `settings`
`cranelift-frontend`	Translates AST/bytecode to Cranelift SSA IR and manages basic block sealing.	`FunctionBuilder`, `FunctionBuilderContext`
`cranelift-module`	Abstract target interface for declaring and defining functions and data objects.	`Module`, `Linkage`
`cranelift-jit`	Concrete `Module` implementation that allocates executable memory in the current process.	`JITModule`, `JITBuilder`
`cranelift-native`	Auto-detects the host CPU instruction set architecture (ISA) and CPU flags.	`cranelift_native::builder()`

Program walkthrough

The example project is organized as a standard Cargo binary crate named `tinyjit`. The manifest asks for version 0.137.1 of the five crates (a caret requirement, so `cargo` may pick a later 0.137.x). It was created with `cargo add cranelift-codegen cranelift-frontend cranelift-jit cranelift-module cranelift-native`. The code is in `src/main.rs`.

```
[package]
name = "tinyjit"
version = "0.1.0"
edition = "2024"

[dependencies]
cranelift-codegen = "0.137.1"
cranelift-frontend = "0.137.1"
cranelift-jit = "0.137.1"
cranelift-module = "0.137.1"
cranelift-native = "0.137.1"
```

The application parses RPN expressions containing a single variable `x` and 64-bit integer constants, builds Cranelift IR, compiles the function into native code, compares execution against an interpreter, and frees the JIT memory.

```
//! tinyjit: compile a postfix (RPN) expression in `x` to native code with
//! Cranelift, then call it through a function pointer.
use cranelift_codegen::ir::{types, AbiParam, InstBuilder, Value};
use cranelift_codegen::settings::{self, Configurable};
use cranelift_frontend::{FunctionBuilder, FunctionBuilderContext};
use cranelift_jit::{JITBuilder, JITModule};
use cranelift_module::{default_libcall_names, Linkage, Module};
use std::time::Instant;

#[derive(Debug, Clone, Copy)]
enum Tok {
    X,
    Num(i64),
    Add,
    Sub,
    Mul,
}

fn parse(src: &str) -> Result<Vec<Tok>, String> {
    src.split_whitespace()
        .map(|t| match t {
            "x" => Ok(Tok::X),
            "+" => Ok(Tok::Add),
            "-" => Ok(Tok::Sub),
            "*" => Ok(Tok::Mul),
            n => n.parse().map(Tok::Num).map_err(|_| format!("bad token {n:?}")),
        })
        .collect()
}

/// Reference interpreter: walks the token list for every call.
fn interpret(prog: &[Tok], x: i64) -> i64 {
    let mut stack = Vec::with_capacity(8);
    for t in prog {
        match *t {
            Tok::X => stack.push(x),
            Tok::Num(n) => stack.push(n),
            op => {
                let b = stack.pop().unwrap();
                let a = stack.pop().unwrap();
                stack.push(match op {
                    Tok::Add => a.wrapping_add(b),
                    Tok::Sub => a.wrapping_sub(b),
```

```
                    Tok::Mul => a.wrapping_mul(b),
                    _ => unreachable!(),
                });
            }
        }
    }
    stack.pop().unwrap()
}

/// Compile the program to `extern "C" fn(i64) -> i64`.
fn compile(prog: &[Tok]) -> Result<(JITModule, extern "C" fn(i64) -> i64, String), String> {
    let mut flags = settings::builder();
    flags.set("opt_level", "speed").map_err(|e| e.to_string())?;
    let isa = cranelift_native::builder()
        .map_err(|e| e.to_string())?
        .finish(settings::Flags::new(flags))
        .map_err(|e| e.to_string())?;
    let mut module = JITModule::new(JITBuilder::with_isa(isa, default_libcall_names()));

    let mut ctx = module.make_context();
    ctx.func.signature.params.push(AbiParam::new(types::I64));
    ctx.func.signature.returns.push(AbiParam::new(types::I64));

    // Pointer width, call conventions etc. the frontend needs from the target.
    let target = module.target_config();
    let mut fctx = FunctionBuilderContext::new();
    {
        let mut b = FunctionBuilder::new(&mut ctx.func, &mut fctx);
        let entry = b.create_block();
        b.append_block_params_for_function_params(entry);
        b.switch_to_block(entry);
        b.seal_block(entry);
        let x = b.block_params(entry)[0];

        // The RPN stack exists only at compile time: it holds SSA values.
        let mut stack: Vec<Value> = Vec::new();
        for t in prog {
            let v = match *t {
                Tok::X => x,
                Tok::Num(n) => b.ins().iconst(types::I64, n),
                op => {
                    let rhs = stack.pop().ok_or("stack underflow")?;
                    let lhs = stack.pop().ok_or("stack underflow")?;
                    match op {
                        Tok::Add => b.ins().iadd(lhs, rhs),
                        Tok::Sub => b.ins().isub(lhs, rhs),
                        Tok::Mul => b.ins().imul(lhs, rhs),
```

```
                        _ => unreachable!(),
                    }
                }
            };
            stack.push(v);
        }
        if stack.len() != 1 {
            return Err(format!("expression leaves {} values", stack.len()));
        }
        b.ins().return_(&[stack[0]]);
        b.finalize(target);
    }
    let ir = ctx.func.display().to_string();

    let id = module
        .declare_function("expr", Linkage::Export, &ctx.func.signature)
        .map_err(|e| e.to_string())?;
    module.define_function(id, &mut ctx).map_err(|e| e.to_string())?;
    module.clear_context(&mut ctx);
    module.finalize_definitions().map_err(|e| e.to_string())?;

    let code = module.get_finalized_function(id);
    // SAFETY: we built this function with exactly this signature.
    let f = unsafe { std::mem::transmute::<*const u8, extern "C" fn(i64) -> i64>(code) };
    Ok((module, f, ir))
}

fn main() -> Result<(), String> {
    let src = std::env::args().nth(1).unwrap_or_else(|| "x x * 3 x * + 7 -".into());
    let prog = parse(&src)?;
    let (module, f, ir) = compile(&prog)?;
    println!("expression: {src}");
    println!("{ir}");

    for x in [0, 1, 2, 10] {
        println!("f({x}) = {} (interpreter: {})", f(x), interpret(&prog, x));
    }

    let n = 5_000_000i64;
    let t = Instant::now();
    let mut acc = 0i64;
    for x in 0..n {
        acc = acc.wrapping_add(interpret(&prog, x));
    }
    let ti = t.elapsed();
    let t = Instant::now();
    let mut acc2 = 0i64;
```

```
    for x in 0..n {
        acc2 = acc2.wrapping_add(f(std::hint::black_box(x)));
    }
    let tj = t.elapsed();
    println!("checksums equal: {}", acc == acc2);
    println!("interpreter {:?}, jit {:?} for {n} calls", ti, tj);

    // Freeing the module unmaps the code; `f` must not be called after this.
    unsafe { module.free_memory() };
    Ok(())
}
```

Parsing and reference interpretation

The parser splits whitespace-delimited strings into tokens representing values (`x`, integer literals) or stack operations (`+`, `-`, `*`).

The reference interpreter maintains an evaluation stack at runtime. For every function evaluation, it iterates over the token array, pushing values and popping operands to perform arithmetic operations. Notice that `interpret` uses wrapping arithmetic (`wrapping_add`, `wrapping_sub`, `wrapping_mul`). This guarantees two's-complement overflow behaviour identical to native machine instructions emitted by Cranelift's `iadd`, `isub`, and `imul`.

Setting up the Cranelift JIT environment

The `compile` function establishes the target platform ISA and configures compilation flags:

```
let mut flags = settings::builder();
flags.set("opt_level", "speed").map_err(|e| e.to_string())?;
let isa = cranelift_native::builder()
    .map_err(|e| e.to_string())?
    .finish(settings::Flags::new(flags))
    .map_err(|e| e.to_string())?;
let mut module = JITModule::new(JITBuilder::with_isa(isa, default_libcall_names()));
```

1. `settings::builder()` initializes code generator flags, where `opt_level` is set to `"speed"`.
2. `cranelift_native::builder()` detects the host architecture and CPU features; `finish` combines that with the flags into an `OwnedTargetIsa`.
3. `JITBuilder::with_isa` takes the ISA plus `default_libcall_names()`, the symbol names to use if generated code needs runtime helper calls such as `memcpy`. `JITModule::new` then gives us a module that puts code in this process's memory.

Constructing SSA IR with FunctionBuilder

Cranelift IR is built around blocks and SSA values. The input expression uses postfix notation, which operates on an evaluation stack. However, in our JIT compiler, the evaluation stack exists **only at compile time**. The stack stores SSA `cranelift_codegen::ir::Value` handles rather than runtime integers.

```
let mut ctx = module.make_context();
ctx.func.signature.params.push(AbiParam::new(types::I64));
ctx.func.signature.returns.push(AbiParam::new(types::I64));

let target = module.target_config();
let mut fctx = FunctionBuilderContext::new();
{
    let mut b = FunctionBuilder::new(&mut ctx.func, &mut fctx);
    let entry = b.create_block();
    b.append_block_params_for_function_params(entry);
    b.switch_to_block(entry);
    b.seal_block(entry);
    let x = b.block_params(entry)[0];
```

The function signature is configured for an input parameter of `i64` and a return type of `i64`.

Block construction follows four steps: 1. `create_block()` allocates an entry basic block. 2. `append_block_params_for_function_params(entry)` assigns block parameter values corresponding to incoming ABI function parameters (`x`). 3. `switch_to_block(entry)` instructs the builder to append subsequent IR instructions to this block. 4. `seal_block(entry)` promises that no more branches into this block will be added. That lets the frontend finish SSA construction for it. The entry block has no predecessors, so we can seal it at once.

As the compiler iterates through the RPN token stream: - `Tok::X` pushes the entry block parameter `x`. - `Tok::Num(n)` emits an `iconst` instruction producing a new `Value`. - Binary operators (`+`, `-`, `*`) pop two `Value` handles off the compile-time stack, emit an `iadd`, `isub`, or `imul` instruction, and push the resulting `Value` back onto the stack.

Finally, `b.ins().return_(&[stack[0]])` generates the return instruction, and `b.finalize(target)` ends building. In debug builds it checks that every block is sealed.

> **Note**
>
> **API Change in Cranelift 0.137**: In older versions of Cranelift, `FunctionBuilder::finalize()` took no arguments. In 0.137, `finalize` takes a `TargetFrontendConfig`, which you get from `module.target_config()`. Code written for older Cranelift releases will fail to compile without this argument.

Finalizing compilation and function transmutation

Once the IR representation is constructed, `compile` submits the function context to the JIT module:

```
let id = module
    .declare_function("expr", Linkage::Export, &ctx.func.signature)
    .map_err(|e| e.to_string())?;
module.define_function(id, &mut ctx).map_err(|e| e.to_string())?;
module.clear_context(&mut ctx);
module.finalize_definitions().map_err(|e| e.to_string())?;

let code = module.get_finalized_function(id);
let f = unsafe { std::mem::transmute::<*const u8, extern "C" fn(i64) -> i64>(code) };
```

- `declare_function` registers the function symbol `"expr"` with `Linkage::Export` within the module.
- `define_function` compiles the function's IR into native machine code buffers.
- `finalize_definitions` applies relocations and then switches the code memory from writable to executable.
- `get_finalized_function` returns a raw pointer (`*const u8`) to the executable instruction buffer.
- `std::mem::transmute` turns the raw pointer into an `extern "C" fn(i64) -> i64`. This is sound only because the type matches the signature the function was built with.

Reading the Cranelift IR

Build with `cargo build --release`. These are the toolchain and the direct dependencies it resolved to:

```
rustc --version; cargo --version
```

```
rustc 1.99.1 (58cf8f9c3 2026-10-13)
cargo 1.99.1 (5f94df478 2026-08-27)
```

```
cargo tree --depth 1 -e normal
```

```
tinyjit v0.1.0 (/home/box/sand-data/books-scout/runs/2026-11-03-runnow/lab/rust-jit/jit)
  cranelift-codegen v0.137.1
  cranelift-frontend v0.137.1
  cranelift-jit v0.137.1
  cranelift-module v0.137.1
  cranelift-native v0.137.1
```

(The project path in the first line is just where the lab happened to live.)

With no argument, the program compiles `x x * 3 x * + 7 -`, which is $x^2 + 3x - 7$. It prints the IR from `ctx.func.display()`, then some values and the timings:

```
./target/release/tinyjit
```

```
expression: x x * 3 x * + 7 -
function u0:0(i64) -> i64 system_v {
block0(v0: i64):
    v1 = imul v0, v0
    v2 = iconst.i64 3
    v3 = imul v2, v0  ; v2 = 3
    v4 = iadd v1, v3
    v5 = iconst.i64 7
    v6 = isub v4, v5  ; v5 = 7
    return v6
}

f(0) = -7 (interpreter: -7)
f(1) = -3 (interpreter: -3)
f(2) = 3 (interpreter: 3)
f(10) = 123 (interpreter: 123)
checksums equal: true
interpreter 106.790602ms, jit 8.698466ms for 5000000 calls
```

Analyzing the IR output

Examining the printed IR reveals how high-level RPN expressions translate into flat SSA form:

1. **Signature**: `function u0:0(i64) -> i64 system_v`. `u0:0` is a default name, because the symbol name `expr` belongs to the module, not the IR. `system_v` is the host's default calling convention on x86_64 Linux, the same one `extern "C"` uses there.
2. **Block Parameters**: `block0(v0: i64):` defines entry block parameter `v0`, representing argument `x`.
3. **Instruction Stream**:
    - `v1 = imul v0, v0` computes $x \times x$ (x^2).
    - `v2 = iconst.i64 3` makes the constant 3. The `; v2 = 3` comment on the next line is Cranelift noting a constant operand where it is used.
    - `v3 = imul v2, v0` computes $3 \times x$.
    - `v4 = iadd v1, v3` sums $x^2 + 3x$.
    - `v5 = iconst.i64 7` loads constant value 7.
    - `v6 = isub v4, v5` subtracts 7 ($x^2 + 3x - 7$).
    - `return v6` returns the computed 64-bit value.

Notice that no stack manipulation instructions (`push`, `pop`) exist in the generated IR. Cranelift maps the compile-time token stack directly to SSA value bindings (`v0` through `v6`), which the register allocator then puts in machine registers.

Performance and timing analysis

The benchmark section of `main.rs` executes 5,000,000 iterations comparing `interpret` against the JIT compiled function `f`. Both loops add up the results with wrapping adds, so the checksums can be compared. `std::hint::black_box` keeps the argument opaque to the optimiser.

The interpreter incurs overhead on every iteration: - Re-walking slice elements. - Pattern matching on token enum variants. - Pushing and popping elements on a dynamic heap/stack buffer. - Performing branch checks for stack bounds.

The JIT version is one indirect call to a few straight-line instructions. In this run the interpreter took about 107 ms and the JIT about 8.7 ms for 5,000,000 calls, roughly 12 times faster. That is one run on one machine, so read it as a rough guide, not a benchmark. Compile time isn't included, which is fair only because we compile once.

Error handling

Expression validation occurs during both parsing and IR construction. If an invalid expression is passed—such as `"x +"`—the parser succeeds, but IR construction fails because the operation requires two operands while only one exists on the compile-time stack.

Evaluating an invalid expression returns an error and sets a non-zero exit status:

```
./target/release/tinyjit "x +"; echo "exit status: $?"
```

```
Error: "stack underflow"
exit status: 1
```

`compile` returns `Err("stack underflow")` before any code is defined or transmuted. `main` returns that `Result`, so Rust prints the error with `Debug` (hence the quotes) and exits with status 1.

Safety rules for JIT code

Executing dynamically generated machine code requires raw pointer transmutes and manual lifetime management. To maintain sound Rust execution, adhere to four core safety rules:

1. **Exact Signature Matching**: The function type target of `std::mem::transmute` MUST match the ABI parameters and return values declared in `ctx.func.signature`. A mismatch is undefined behaviour: wrong registers, garbage results, or a crash. Use `extern "C"` so the Rust side uses the platform calling convention the IR was built for.
2. **Module Lifetime and Freeing Memory**: The returned function pointer `f` points directly to executable memory buffers allocated and managed by `JITModule`.

> **Note**
>
> Calling `unsafe { module.free_memory() }` unmaps or deallocates executable memory pages. Calling `f` after `free_memory()` is a use-after-free: undefined behaviour, which may crash or may run whatever is now at that address. Keep `module` alive for as long as you use `f`, as `main` does.

3. **Keep it single-threaded unless the docs say otherwise**: this program compiles and calls on one thread. Before sharing a module or its function pointers across threads, check what `cranelift-jit` guarantees for your version.
4. **Let the module handle permissions**: don't write into code memory yourself. `finalize_definitions()` makes it executable (on Linux by changing page protections) once writing is done.

Common mistakes

When working with Cranelift, several pitfalls regularly trap engineers:

- **Using outdated `finalize()` signatures**: Calling `builder.finalize()` without passing `module.target_config()` fails to compile on Cranelift `0.137.1`.
- **Forgetting `seal_block`**: If a block is never sealed, `finalize` in a debug build panics with "FunctionBuilder finalized, but block … is not sealed". With more than one block, seal each one once all its predecessors exist.
- **Calling function pointers post-deallocation**: Invoking `f` after `module.free_memory()` or after `JITModule` has been dropped is undefined behaviour.
- **Transmuting to Rust standard ABI**: Omitting `extern "C"` from the function pointer declaration (e.g., `fn(i64) -> i64` instead of `extern "C" fn(i64) -> i64`) uses Rust's unspecified ABI, which need not match `system_v`, so calling it is undefined behaviour.
- **Including JIT compilation latency in hot execution loops**: Recompiling inside the evaluation loop throws the gain away. Always compile expressions once, store the finalized function pointer, and invoke the pointer repeatedly.

Exercises

To expand upon this minimal JIT compiler, consider implementing the following enhancements:

1. **Add Control Flow**: Extend the parser and IR builder to support conditional expressions (`if` / `else`) using Cranelift's block parameters, `b.ins().brif(...)`, and `b.ins().jump(...)`.
2. **Variable Binding Tables**: Replace single-variable `x` support with arbitrary named variables using Cranelift's `Variable`, `b.declare_var(...)`, `b.def_var(...)`, and `b.use_var(...)` APIs.
3. **Host Function Imports**: Declare external C-ABI host functions (such as `sin` or `print_i64`) using `module.declare_function` with `Linkage::Import`, register their host address via `JITBuilder::symbol`, and emit function calls with `b.ins().call(...)`.

4. **Checked Division**: Add a `/` division operator using `b.ins().sdiv(...)`. `sdiv` traps on division by zero (and on `i64::MIN / -1`). With no trap handler installed, the trap kills the process, so check the divisor in the generated code or reject such input.
5. **Machine Code Disassembly**: Call `ctx.set_disasm(true)` before `define_function`. Before `clear_context`, print the text in `ctx.compiled_code().unwrap().vcode` (an `Option<String>`) to see the machine code generated for the host.

Summary

Cranelift lets a Rust program generate native code at runtime without LLVM. The postfix stack lives only at compile time and holds SSA `Values`, so each token becomes at most one instruction and the generated code has no stack machine. `JITModule` owns the code memory. The two `unsafe` operations, the transmute and `free_memory`, are sound only if the signature matches and the module outlives every call.

Sources

- Cranelift JIT Documentation (`cranelift-jit`)
- Cranelift Frontend Documentation (`cranelift-frontend`)
- Cranelift Official Website
- Cranelift Source Repository (Bytecode Alliance)
- Cranelift JIT Demo Repository
- Cranelift Backend for rustc (`rustc_codegen_cranelift`)

Debugging Rust Programs

Debugging Rust Programs

Debugging Rust applications using `gdb`, `lldb`, and Rust backtraces:

```
RUST_BACKTRACE=1 cargo run
```

Security Considerations in Rust

Security Considerations in Rust

- Audit dependencies using `cargo-audit` and `cargo-vet`.
- Avoid integer overflow panics in production using checked/saturating arithmetic.
- Sanitize memory buffers holding sensitive cryptographic keys with crates like `zeroize`.

Common Anti-Patterns and How to Avoid Them

Common Anti-Patterns and How to Avoid Them

- **Anti-pattern:** Excessive `.clone()` calls to appease the borrow checker. *Remedy:* Restructure ownership or pass references.
- **Anti-pattern:** Unnecessary `Arc<Mutex<T>>` when local message passing suffices. *Remedy:* Use channels or scoped threads.
- **Anti-pattern:** Overusing `Box<dyn Error>` in public library crates. *Remedy:* Expose custom structured error enums.

Summary

Summary

Chapter Key Takeaways

- Understanding the AST -> HIR -> MIR -> LLVM pipeline clarifies compiler errors and optimizations.
- `RUST_BACKTRACE=1` and debuggers demystify runtime failures.
- Regular security audits with `cargo-audit` prevent supply-chain vulnerabilities.
- Writing idiomatic Rust eliminates defensive over-cloning and unnecessary synchronization locks.

WebAssembly with Rust

Why Rust for WebAssembly?

Why Rust for WebAssembly?

WebAssembly (Wasm) is a binary instruction format for a stack-based virtual machine designed for near-native execution speed on the web and edge servers.

Rust is uniquely positioned as the premier language for WebAssembly: - **No Garbage Collector:** Wasm has no built-in GC; Rust's compile-time memory management avoids bundling heavy GC runtimes into the `.wasm` binary. - **Compact Binaries:** Tiny output sizes optimize load times across web browsers and serverless functions. - **Rich Tooling:** `wasm-pack` and `wasm-bindgen` provide seamless high-level JavaScript interoperability.

WebAssembly Fundamentals

WebAssembly Fundamentals

Wasm executes inside a sandboxed linear memory space with strict type checking and deterministic execution semantics.

Setting Up: wasm-pack and wasm-bindgen

Setting Up: wasm-pack and wasm-bindgen

Install `wasm-pack`:

```
cargo install wasm-pack
```

Configure `Cargo.toml`:

```
[lib]
crate-type = ["cdylib", "rlib"]

[dependencies]
wasm-bindgen = "0.2"
```

Building Your First WebAssembly Module

Building Your First WebAssembly Module

```
// In src/lib.rs
// use wasm_bindgen::prelude::*;
//
// #[wasm_bindgen]
// pub fn add(a: i32, b: i32) -> i32 {
//     a + b
// }
```

Build the npm package:

```
wasm-pack build --target web
```

Interacting with the DOM

Interacting with the DOM

Using `web-sys` to interact directly with DOM nodes, canvases, and local storage from Rust.

Handling Events and Async

Handling Events and Async

Attaching browser event listeners with closures and running async promises with `wasm-bindgen-futures`.

Performance Considerations

Performance Considerations

Optimizing Wasm size and throughput using `wasm-opt` from binaryen and disabling panic formatting.

Server-Side WebAssembly

Server-Side WebAssembly

Running Wasm outside the browser using WASI (WebAssembly System Interface) and runtimes like Wasmtime, Wasmer, and Cloudflare Workers.

Publishing and Distribution

Publishing and Distribution

Distributing Wasm modules via npm registries or OCI container registries for serverless edge deployments.

Summary

Summary

Chapter Key Takeaways

- Rust's lack of a GC runtime makes it ideal for compact, high-speed WebAssembly modules.
- `wasm-pack` packages Rust code into web-ready JavaScript/Wasm bundles.
- WASI enables secure, sandboxed server-side WebAssembly execution.

Embedded Rust and no_std Development

Understanding no_std

Understanding no_std

Embedded microcontrollers lack operating system services like virtual memory, filesystems, and threads. In embedded Rust, programs declare `#![no_std]`, compiling without the standard library while retaining the core language and primitives.

```
// #![no_std]
// #![no_main]
//
// use core::panic::PanicInfo;
//
// #[panic_handler]
// fn panic(_info: &PanicInfo) -> ! {
//     loop {}
// }
```

Choosing a Target: Cortex-M

Choosing a Target: Cortex-M

Targeting ARM Cortex-M microcontrollers using `rustup target add thumbv7em-none-eabihf`.

Writing an Embedded Application: Blinking an LED

Writing an Embedded Application: Blinking an LED

The classic embedded "Hello World": toggling a GPIO pin tied to an LED using Peripheral Access Crates (PAC) and Hardware Abstraction Layers (HAL).

Memory Management Without a Heap

Memory Management Without a Heap

Managing state using static memory and stack buffers with zero dynamic heap allocation.

Interrupts and Concurrency

Interrupts and Concurrency

Handling hardware interrupt service routines (ISRs) safely using critical sections and interrupt-safe atomics.

Choosing an Executor or RTOS

Choosing an Executor or RTOS

Comparing bare-metal superloops, RTIC (Real-Time Interrupt-driven Concurrency), and Embassy (async embedded executor).

Testing Embedded Code

Testing Embedded Code

Unit testing business logic on host architectures and running on-target tests with `probe-rs`.

Toolchain and Debugging

Toolchain and Debugging

Hardware flashing and in-circuit debugging via SWD/JTAG with `probe-rs` and `gdb`.

Summary

Summary

Chapter Key Takeaways

- `#![no_std]` strips OS dependencies for bare-metal firmware development.
- Hardware Abstraction Layers (HAL) provide safe, type-checked register manipulation.
- Critical sections and atomics guarantee interrupt safety without data corruption.

Command-Line Tools in Rust

Command-Line Tools in Rust

Command-Line Tools in Rust

Command-line utilities represent one of Rust's most prolific and successful real-world domains. Industry-standard tools like `ripgrep`, `fd`, `bat`, `eza`, `starship`, and `uv` have redefined developer expectations for speed, ergonomics, and rock-solid reliability in terminal software.

This part provides a comprehensive curriculum for engineering production-grade CLI applications in Rust, progressing through seven distinct phases:

1. **Foundations (Chapters 1–4):** Architecture of terminal tools, environment setup, POSIX/standard streams, exit codes, and declarative argument parsing with `clap`.
2. **Core Skills (Chapters 5–8):** High-throughput filesystem traversal, streaming text processing, idiomatic error handling with `anyhow` and `thiserror`, and architectural separation of CLI frontends from domain logic.
3. **Networking & Async (Chapters 9–12):** Process orchestration, asynchronous HTTP clients with `reqwest`, asynchronous task concurrency with `tokio`, and rich terminal user interfaces (TUI) with `ratatui` and `crossterm`.
4. **Testing & Distribution (Chapters 13–16):** XDG-compliant configuration management, structured logging and tracing with `tracing-subscriber`, CLI integration testing with `assert_cmd`, and cross-platform compatibility across Linux, macOS, and Windows.
5. **Advanced (Chapters 17–20):** Production hardening, signal handling, zero-cost CLI design patterns, developer tools, and automated binary distribution via GitHub Releases, Homebrew, and Cargo binstall.
6. **Capstone Projects (Chapters 21–24):** Four end-to-end, production-ready capstones:
   - `fsearch`: High-performance multi-threaded file search tool with glob matching and regex.
   - `tudo`: Interactive terminal user interface task and workflow manager built with `ratatui`.
   - `sysmon`: Real-time cross-platform system resource monitor and process inspector.
   - `devkit`: Multi-subcommand developer toolkit integrating hashing, encoding, formatting, and scaffolding.
7. **The Rust CLI Ecosystem (Chapters 25–26):** Contributing back to the community, crate maintenance, and the future horizon of terminal engineering (WebAssembly WASI, headless agent tooling, terminal graphics protocols).

Part 1 – Foundations

Introduction to Command-Line Tools in Rust

Introduction to Command-Line Tools in Rust

Command-line interfaces (CLIs) remain the backbone of systems engineering, DevOps automation, software development workflows, and server operations. In recent years, Rust has become the preferred systems language for building modern terminal utilities.

Why Rust Dominates the Modern CLI Landscape

1. **Zero-Cost Abstractions & Raw Performance:** Rust compiles to native machine code without a garbage collector. Startup latency is sub-millisecond—critical when a command is invoked repeatedly in shell loops or shell prompt evaluations (e.g., `starship`).
2. **Deterministic Memory Footprint:** Memory consumption remains tiny and predictable. Unlike JVM or Go binaries that balance garbage collector heuristics, Rust CLI tools release memory deterministically when data structures exit scope.
3. **Single Self-Contained Binaries:** Rust programs statically link dependencies and libc (via `musl` on Linux) to produce portable binaries that run anywhere without requiring a runtime, interpreter, or dynamic libraries.
4. **Compile-Time Safety Guarantees:** Concurrency bugs, race conditions, buffer overflows, and null pointers are eliminated at compile time by the borrow checker.
5. **Vibrant Terminal Ecosystem:** High-quality crates like `clap`, `ratatui`, `crossterm`, `indicatif`, and `console` offer unmatched ergonomics for terminal development.

Pillars of a Modern Rust CLI

Modern command-line tools adhere to elevated usability standards:

- **Helpful, intuitive diagnostics:** Explaining missing arguments or bad syntax clearly.
- **Stream-aware output:** Auto-detecting TTY terminals to enable ANSI colors, while suppressing escape codes when piped to a file or another process.
- **Standard exit code conventions:** Following POSIX conventions (`0` for success, non-zero for distinct failure conditions).
- **Fast, non-blocking operations:** Leveraging multi-threading or asynchronous I/O to keep terminals responsive.

```rust
fn main() {
    println!("Welcome to high-performance CLI engineering in Rust.");
}
```

Setting Up Your Development Environment

Setting Up Your Development Environment

Building production CLI applications requires an optimized Rust workspace, tailored compiler targets, and specialized developer tooling.

Toolchain Installation

Ensure the latest stable Rust toolchain is installed via `rustup`:

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
rustup update stable
```

Verify your environment:

```
rustc --version
cargo --version
```

Essential Cargo Extensions for CLI Developers

Install key Cargo plugins designed to accelerate CLI iteration, size profiling, and binary inspection:

```
# Fast incremental checking and watching
cargo install cargo-watch

# Binary size inspection and bloat analysis
cargo install cargo-bloat

# Cross-compilation support via Docker/Podman
cargo install cross

# Fast binary installer
cargo install cargo-binstall
```

Workspace Initialization for CLI Applications

A standard CLI project is initialized using `cargo new`:

```
cargo new mycli --bin
cd mycli
```

Recommended `Cargo.toml` Release Profile

For command-line tools where download size, startup time, and execution speed matter, configure aggressive release optimizations in `Cargo.toml`:

```
[package]
name = "mycli"
version = "0.1.0"
edition = "2024"

[dependencies]
clap = { version = "4.5", features = ["derive", "env", "cargo"] }
anyhow = "1.0"
tracing = "0.1"
tracing-subscriber = { version = "0.3", features = ["env-filter"] }

[profile.release]
opt-level = 3
lto = true
codegen-units = 1
panic = "abort"
strip = true
```

- `opt-level = 3`: Maximum optimizations.
- `lto = true`: Link-Time Optimization eliminates unused functions across crates.
- `codegen-units = 1`: Allows the compiler to perform aggressive cross-crate inlining.
- `panic = "abort"`: Removes stack unwinding tables, significantly shrinking binary size.
- `strip = true`: Strips debug symbols and symbol tables from the final ELF/Mach-O/PE binary.

Command-Line Fundamentals

Command-Line Fundamentals

Before introducing heavy parsing frameworks, every CLI developer must understand the fundamental contracts between the operating system kernel and a running process.

Process Arguments (`std::env::args`)

When an operating system executes a binary, it passes an array of null-terminated strings (`argv`). In Rust, `std::env::args()` exposes these values as an iterator yielding `String`:

```
use std::env;

fn main() {
    let args: Vec<String> = env::args().collect();

    // args[0] is the executable name/path
    println!("Program invoked as: {}", args[0]);

    for (i, arg) in args.iter().enumerate().skip(1) {
        println!("Argument {}: {}", i, arg);
    }
}
```

> **Warning:** `std::env::args()` panics if any argument contains invalid UTF-8. For binary data or arbitrary OS paths, use `std::env::args_os()`, which yields `std::ffi::OsString`.

Standard I/O Streams: Stdin, Stdout, and Stderr

Operating systems provide three fundamental file descriptors to every process:

- **Stdin (File Descriptor 0):** Input stream, which can be typed interactively or piped (`cat data.txt | mycli`).
- **Stdout (File Descriptor 1):** Standard output stream for primary program results.
- **Stderr (File Descriptor 2):** Standard error stream for logs, error messages, and progress spinners.

```
use std::io::{self, Write};
```

```rust
fn main() -> io::Result<()> {
    // Normal program output to stdout
    writeln!(io::stdout(), "Computed result: 42")?;

    // Diagnostics, warnings, or errors go to stderr
    writeln!(io::stderr(), "Warning: config file not found, using defaults")?;

    Ok(())
}
```

Separating stdout and stderr ensures that downstream tools can parse output cleanly while operators still see status and diagnostic messages on their terminals.

Exit Status Codes

When a process terminates, it returns an integer status code (0–255 on Unix) to its parent process:

- `0`: Success.
- `1` or non-zero: General error or specific domain failure code.

In Rust, return an explicit exit code using `std::process::ExitCode`:

```rust
use std::process::ExitCode;

fn main() -> ExitCode {
    if let Err(err) = run() {
        eprintln!("Error: {err}");
        return ExitCode::FAILURE; // Returns 1
    }
    ExitCode::SUCCESS // Returns 0
}

fn run() -> Result<(), &'static str> {
    Ok(())
}
```

Parsing Arguments with clap

Parsing Arguments with clap

`clap` (Command Line Argument Parser) is the de facto standard for argument parsing in the Rust ecosystem. Its procedural `derive` macros turn Rust structs into robust, auto-documenting command-line parsers with zero boilerplate.

Adding clap with Derive

In `Cargo.toml`:

```
[dependencies]
clap = { version = "4.5", features = ["derive", "env"] }
```

Basic CLI Definition

Define a struct representing your CLI arguments and derive `Parser`:

```
use clap::Parser;
use std::path::PathBuf;

#[derive(Parser, Debug)]
#[command(name = "minigrep")]
#[command(version = "1.0.0")]
#[command(about = "A blazing fast search tool", long_about = None)]
struct Cli {
    /// The search query pattern
    #[arg(short, long)]
    query: String,

    /// Target file path to search
    #[arg(short, long, value_name = "FILE")]
    path: PathBuf,

    /// Case-insensitive search
    #[arg(short = 'i', long)]
    ignore_case: bool,

    /// Number of matching context lines to display
```

```
    #[arg(short = 'C', long, default_value_t = 0)]
    context: usize,
}

fn main() {
    let cli = Cli::parse();

    println!("Searching for pattern '{}' in {:?}", cli.query, cli.path);
    if cli.ignore_case {
        println!("Case-insensitive search enabled");
    }
}
```

Running `minigrep --help` generates an idiomatic Unix-compliant help screen complete with flags, options, version strings, and usage instructions automatically.

Subcommands with Enums

Real-world developer tools (like `git`, `docker`, or `cargo`) organize features into subcommands. In `clap`, subcommands are modeled as Rust enums:

```
use clap::{Parser, Subcommand};
use std::path::PathBuf;

#[derive(Parser)]
#[command(name = "gitclone", about = "Minimal VCS clone")]
struct App {
    #[command(subcommand)]
    command: Commands,
}

#[derive(Subcommand)]
enum Commands {
    /// Initialize a new repository
    Init {
        #[arg(default_value = ".")]
        path: PathBuf,
    },
    /// Clone an existing remote repository
    Clone {
        /// Remote repository URL
        url: String,
        /// Local destination directory
        dest: Option<PathBuf>,
    },
```

```
    /// Commit staged changes
    Commit {
        #[arg(short, long)]
        message: String,
    },
}

fn main() {
    let app = App::parse();

    match app.command {
        Commands::Init { path } => println!("Initialized repository at {:?}", path),
        Commands::Clone { url, dest } => println!("Cloning {} into {:?}", url, dest),
        Commands::Commit { message } => println!("Committed: {}", message),
    }
}
```

Part 2 – Core Skills

Working with the File System

Working with the File System

Command-line utilities spend a significant percentage of their runtime interacting with the filesystem: locating paths, inspecting metadata, streaming large files, and walking directory trees.

Path Manipulations with `Path` and `PathBuf`

Rust separates borrowed paths (`&Path`) from owned, heap-allocated paths (`PathBuf`), mirroring `&str` and `String`:

```
use std::path::{Path, PathBuf};

fn inspect_path(p: &Path) {
    println!("File name: {:?}", p.file_name());
    println!("Extension: {:?}", p.extension());
    println!("Parent dir: {:?}", p.parent());
    println!("Is absolute: {}", p.is_absolute());
}

fn build_target_path(base: &Path, filename: &str) -> PathBuf {
    base.join("logs").join(filename)
}
```

Efficient File Reading and Writing

For small files, `std::fs::read_to_string` and `std::fs::write` are convenient:

```
use std::fs;
use std::io;

fn quick_read(path: &str) -> io::Result<String> {
    fs::read_to_string(path)
}
```

For large files, avoid reading the entire payload into RAM at once. Use `BufReader` and stream line by line:

```rust
use std::fs::File;
use std::io::{self, BufRead, BufReader};
use std::path::Path;

fn count_non_empty_lines(path: &Path) -> io::Result<usize> {
    let file = File::open(path)?;
    let reader = BufReader::new(file);
    let mut count = 0;

    for line in reader.lines() {
        let line = line?;
        if !line.trim().is_empty() {
            count += 1;
        }
    }
    Ok(count)
}
```

Fast Directory Traversal with `walkdir`

When building search or indexing tools like `find` or `fd`, recursive directory traversal using `std::fs::read_dir` requires complex recursion and symlink handling. The `walkdir` crate provides an optimized, configurable iterator:

```toml
[dependencies]
walkdir = "2.5"
```

```rust
use walkdir::WalkDir;
use std::path::Path;

fn find_rs_files(dir: &Path) {
    for entry in WalkDir::new(dir)
        .follow_links(false)
        .max_depth(5)
        .into_iter()
        .filter_map(Result::ok)
    {
        if entry.file_type().is_file() {
            if let Some(ext) = entry.path().extension() {
                if ext == "rs" {
                    println!("Found Rust file: {}", entry.path().display());
                }
            }
        }
    }
```

```
}
```

Processing Text Data

Processing Text Data

Terminal applications frequently process formatted text, logs, configuration files, and structured tables. Rust provides zero-cost slicing, streaming string buffers, and regular expressions to achieve peak throughput.

Efficient In-Memory String Operations

Rust strings are valid UTF-8 byte sequences. Slicing on arbitrary byte indices can panic if it splits a multi-byte Unicode code point. Always prefer character or byte iterators:

```
fn tokenize_words(input: &str) -> Vec<&str> {
    input
        .split_whitespace()
        .filter(|word| !word.is_empty())
        .collect()
}
```

Reusable Buffers for High-Throughput Streaming

When processing gigabytes of text, allocating a new `String` for every line causes allocator bottlenecking. Reuse a single heap allocation with `BufRead::read_line`:

```
use std::io::{self, BufRead};

fn process_lines<R: BufRead>(mut reader: R) -> io::Result<usize> {
    let mut buffer = String::with_capacity(1024);
    let mut matching_lines = 0;

    while reader.read_line(&mut buffer)? > 0 {
        if buffer.contains("ERROR") {
            matching_lines += 1;
        }
        buffer.clear(); // Clears length while preserving allocated capacity
    }

    Ok(matching_lines)
}
```

Pattern Matching with `regex`

When simple substring checks are insufficient, use the `regex` crate with pre-compiled regex engines:

```
[dependencies]
regex = "1.10"
```

```
use regex::Regex;

fn extract_ip_addresses(text: &str) -> Vec<&str> {
    // Compile regex once (e.g. inside std::sync::LazyLock)
    let re = Regex::new(r"\b(?:\d{1,3}\.){3}\d{1,3}\b").unwrap();

    re.find_iter(text)
        .map(|mat| mat.as_str())
        .collect()
}
```

Streaming JSON with `serde_json`

Many modern CLIs emit or consume structured JSON streams (JSON lines / NDJSON). Use `serde_json::Deserializer::from_reader` to parse JSON records sequentially without loading the entire dataset into memory:

```
use serde::Deserialize;
use std::io::BufRead;

#[derive(Deserialize, Debug)]
struct LogRecord {
    timestamp: String,
    level: String,
    message: String,
}

fn parse_ndjson<R: BufRead>(reader: R) -> serde_json::Result<Vec<LogRecord>> {
    serde_json::Deserializer::from_reader(reader)
        .into_iter::<LogRecord>()
        .collect()
}
```

Error Handling

Error Handling

Command-line utilities live at the messy boundary between users, file systems, network connections, and external processes. A good CLI application should never panic or print cryptic internal stack traces to end users under normal failure modes.

The CLI Error Hierarchy: Application vs Library

In Rust, the community standard divides error handling into two patterns:

1. **Libraries (`thiserror`):** Strongly typed, structured enum errors that callers can match on programmatically.
2. **Applications (`anyhow`):** Dynamic, contextual error reporting with chained causes, custom messages, and user-friendly formatting.

```
[dependencies]
anyhow = "1.0"
thiserror = "1.0"
```

Using `anyhow` for Top-Level CLI Flow

`anyhow::Context` allows annotating errors with human-readable operational context as they bubble up through the `?` operator:

```
use anyhow::{Context, Result};
use std::fs;
use std::path::Path;

fn load_configuration(path: &Path) -> Result<String> {
    fs::read_to_string(path)
        .with_context(|| format!("Failed to read configuration file from '{}'", path.display
}

fn main() -> Result<()> {
    let config_path = Path::new("config.toml");
    let content = load_configuration(config_path)?;
    println!("Loaded config: {} bytes", content.len());
    Ok(())
}
```

If `config.toml` does not exist, `anyhow` automatically formats the error cleanly on stderr:

```
Error: Failed to read configuration file from 'config.toml'

Caused by:
    No such file or directory (os error 2)
```

Domain-Specific Errors with `thiserror`

For internal components that require distinct recovery branches or specific exit codes, define enums with `thiserror`:

```rust
use thiserror::Error;

#[derive(Error, Debug)]
pub enum CliError {
    #[error("Configuration error: {0}")]
    Config(String),

    #[error("Network failure while connecting to {endpoint}: {source}")]
    Network {
        endpoint: String,
        #[source]
        source: std::io::Error,
    },

    #[error("Permission denied for path: {0}")]
    Unauthorized(String),
}
```

Structuring Your Application

Structuring Your Application

As a command-line tool evolves from a single-file prototype into a multi-command production system, code organization becomes paramount. A well-structured CLI separates argument parsing and terminal presentation from domain business logic.

Separation of Concerns: The `main.rs` vs `lib.rs` Pattern

Never write all application logic in `main.rs`. Instead, treat `main.rs` as a thin dispatch adapter and expose the core functionality as a library crate in `lib.rs`:

```
mycli/
  Cargo.toml
  src/
      main.rs         # CLI argument parsing, logging setup, exit code dispatch
      lib.rs          # Re-exports domain modules; testable without binary execution
      cli.rs          # clap structs, argument validators, flags
      config.rs       # TOML/JSON config loading and defaults
      core/           # Core domain logic, models, algorithms
      commands/       # Implementation handlers for each CLI subcommand
          mod.rs
          build.rs
          deploy.rs
  tests/
      cli_tests.rs    # Integration tests using assert_cmd
```

Writing a Clean `main.rs` Dispatcher

```
// src/main.rs
use anyhow::Result;
use clap::Parser;
use std::process::ExitCode;

mod cli;

fn main() -> ExitCode {
    let args = cli::Cli::parse();
```

```
    // Initialize structured logging
    tracing_subscriber::fmt::init();

    if let Err(err) = run(args) {
        eprintln!("{:#}", err);
        return ExitCode::FAILURE;
    }

    ExitCode::SUCCESS
}

fn run(args: cli::Cli) -> Result<()> {
    match args.command {
        cli::Commands::Scan(opts) => mycli::scan(opts)?,
        cli::Commands::Build(opts) => mycli::build(opts)?,
    }
    Ok(())
}
```

Benefits of the Dual-Target Layout

1. **Unit and Integration Testing:** Internal business logic functions can be unit-tested directly in `lib.rs` without spawning subprocesses.
2. **Reusability:** Other tools, benchmarks, or web services can import your crate as an internal dependency.
3. **Clean Architecture:** Changes to CLI argument flags or help text don't alter the underlying computational algorithms.

Part 3 – Networking & Async

Running External Processes

Running External Processes

CLI applications frequently coordinate other system binaries: executing `git` commands, spawning shell scripts, launching compilers, or piping data between subprocesses.

Spawning Subprocesses with `std::process::Command`

The standard library `Command` builder handles subprocess execution, environment configuration, and working directory management:

```
use std::process::{Command, Stdio};
use anyhow::{Context, Result, bail};

fn get_git_commit_hash() -> Result<String> {
    let output = Command::new("git")
        .args(["rev-parse", "--short", "HEAD"])
        .stdout(Stdio::piped())
        .stderr(Stdio::piped())
        .output()
        .context("Failed to execute git process")?;

    if !output.status.success() {
        let err = String::from_utf8_lossy(&output.stderr);
        bail!("git rev-parse failed: {}", err.trim());
    }

    let hash = String::from_utf8(output.stdout)?
        .trim()
        .to_string();

    Ok(hash)
}
```

Piping Data Between Commands (Unix Pipeline Emulation)

To emulate shell pipelines (`cat file.txt | grep error | wc -l`), connect the `stdout` of child process 1 into the `stdin` of child process 2:

```rust
use std::process::{Command, Stdio};
use anyhow::Result;

fn count_matches_in_file(pattern: &str, filepath: &str) -> Result<String> {
    // Spawn grep process with piped stdout
    let grep_child = Command::new("grep")
        .arg(pattern)
        .arg(filepath)
        .stdout(Stdio::piped())
        .spawn()?;

    // Pipe grep stdout into wc stdin
    let wc_child = Command::new("wc")
        .arg("-l")
        .stdin(grep_child.stdout.unwrap())
        .stdout(Stdio::piped())
        .spawn()?;

    let output = wc_child.wait_with_output()?;
    Ok(String::from_utf8_lossy(&output.stdout).trim().to_string())
}
```

Security Best Practice: Never Invoke via Raw Shell

Avoid `Command::new("sh").arg("-c").arg(user_input)`. Spawning subshells with unsanitized user inputs introduces command injection vulnerabilities. Always invoke executable binaries directly and pass arguments as distinct array items.

Building HTTP Clients

Building HTTP Clients

Many modern CLI tools interact with cloud APIs, fetch remote manifests, download binary updates, or query telemetry endpoints. `reqwest` provides both synchronous (blocking) and asynchronous HTTP clients.

Adding `reqwest` and `serde`

In `Cargo.toml`:

```
[dependencies]
reqwest = { version = "0.12", features = ["json", "blocking"] }
serde = { version = "1.0", features = ["derive"] }
```

Synchronous (Blocking) Client for Simple CLIs

For lightweight CLIs that execute a single sequential HTTP request, the blocking client avoids the overhead and runtime setup of a full async event loop:

```
use reqwest::blocking::Client;
use serde::Deserialize;
use std::time::Duration;
use anyhow::{Context, Result};

#[derive(Deserialize, Debug)]
struct ReleaseInfo {
    tag_name: String,
    published_at: String,
}

fn fetch_latest_release(repo: &str) -> Result<ReleaseInfo> {
    let client = Client::builder()
        .timeout(Duration::from_secs(10))
        .user_agent("mycli/1.0.0")
        .build()?;

    let url = format!("https://api.github.com/repos/{repo}/releases/latest");
```

```
    let release = client
        .get(&url)
        .send()
        .with_context(|| format!("Failed to connect to {url}"))?
        .error_for_status()?
        .json::<ReleaseInfo>()?;

    Ok(release)
}
```

Streaming Downloads with Progress Feedback

When downloading large binary payloads or archives, streaming the response body chunk by chunk prevents excessive memory allocation and enables real-time progress reporting:

```
use std::fs::File;
use std::io::{copy, BufWriter};
use anyhow::Result;

fn download_file(url: &str, destination: &str) -> Result<()> {
    let mut response = reqwest::blocking::get(url)?;
    let file = File::create(destination)?;
    let mut writer = BufWriter::new(file);

    copy(&mut response, &mut writer)?;
    Ok(())
}
```

Async Programming with tokio

Async Programming with tokio

When a CLI application must query hundreds of HTTP endpoints, scrape multiple Git repositories in parallel, or monitor multiple file descriptor events simultaneously, synchronous I/O creates severe bottlenecks. `tokio` delivers an asynchronous runtime capable of driving millions of concurrent tasks with minimal CPU overhead.

Configuring the Tokio Runtime for CLI Applications

In `Cargo.toml`:

```
[dependencies]
tokio = { version = "1.38", features = ["full"] }
reqwest = { version = "0.12", features = ["json"] }
futures = "0.3"
```

The `#[tokio::main]` Attribute

The `#[tokio::main]` macro sets up the multi-threaded asynchronous executor and starts the root future:

```
use anyhow::Result;
use reqwest::Client;
use std::time::Duration;

#[tokio::main]
async fn main() -> Result<()> {
    let client = Client::builder()
        .timeout(Duration::from_secs(5))
        .build()?;

    let response = client
        .get("https://httpbin.org/status/200")
        .send()
        .await?;

    println!("Async response status: {}", response.status());
    Ok(())
}
```

Bounded Concurrency with `tokio::sync::Semaphore`

A common CLI mistake is launching thousands of unbounded concurrent network tasks, exhausting local socket descriptors or triggering remote rate limits (HTTP 429). Use a `Semaphore` to bound concurrent flight:

```
use std::sync::Arc;
use tokio::sync::Semaphore;
use anyhow::Result;

async fn fetch_urls(urls: Vec<String>) -> Result<()> {
    let client = reqwest::Client::new();
    // Allow at most 10 concurrent HTTP requests
    let semaphore = Arc::new(Semaphore::new(10));
    let mut handles = Vec::new();

    for url in urls {
        let permit = semaphore.clone().acquire_owned().await?;
        let client = client.clone();

        handles.push(tokio::spawn(async move {
            let res = client.get(&url).send().await;
            drop(permit); // Release slot for next request
            (url, res.map(|r| r.status()))
        }));
    }

    for handle in handles {
        let (url, result) = handle.await?;
        println!("{url} -> {:?}", result);
    }

    Ok(())
}
```

Graceful Cancellation via `tokio::select!`

Listen for OS interrupt signals (`Ctrl+C`) while long-running network or disk jobs are active:

```
#[tokio::main]
async fn main() {
    tokio::select! {
        _ = tokio::signal::ctrl_c() => {
            eprintln!("\nReceived Ctrl+C, gracefully terminating worker tasks...");
        }
```

```
        res = run_worker() => {
            println!("Worker completed with: {:?}", res);
        }
    }
}

async fn run_worker() -> Result<(), ()> {
    tokio::time::sleep(std::time::Duration::from_secs(60)).await;
    Ok(())
}
```

Terminal User Interfaces with ratatui

Terminal User Interfaces with ratatui

When a simple line-based CLI is not expressive enough, full-screen Terminal User Interfaces (TUIs) allow interactive dashboards, data tables, split panes, and real-time monitoring graphs. `ratatui` (the community fork of `tui-rs`) is the premier TUI framework in Rust.

Adding `ratatui` and `crossterm`

In `Cargo.toml`:

```
[dependencies]
ratatui = "0.26"
crossterm = { version = "0.27", features = ["event-stream"] }
```

- `crossterm`: Low-level cross-platform terminal manipulation (alternate screens, raw mode, keyboard/mouse events).
- `ratatui`: Immediate-mode UI layout engine, widgets, styling, and rendering abstractions.

Raw Mode and Alternate Screen Lifecycle

A TUI must enter raw mode (disabling line buffering and OS echo) and switch to an alternate screen buffer, restoring the user's previous terminal state upon exit:

```
use std::io::{self, stdout};
use crossterm::{
    execute,
    terminal::{disable_raw_mode, enable_raw_mode, EnterAlternateScreen, LeaveAlternateScreen
};
use ratatui::prelude::*;

pub struct TerminalGuard;

impl TerminalGuard {
    pub fn enter() -> io::Result<Terminal<CrosstermBackend<io::Stdout>>> {
        enable_raw_mode()?;
        let mut stdout = stdout();
        execute!(stdout, EnterAlternateScreen)?;
        let backend = CrosstermBackend::new(stdout);
        Terminal::new(backend)
```

```
    }
}

impl Drop for TerminalGuard {
    fn drop(&mut self) {
        let _ = disable_raw_mode();
        let _ = execute!(stdout(), LeaveAlternateScreen);
    }
}
```

Immediate-Mode Rendering Loop

`ratatui` recalculates and renders the interface every frame based on the current application state:

```
use ratatui::{
    layout::{Constraint, Direction, Layout},
    style::{Color, Style},
    widgets::{Block, Borders, Paragraph},
    Frame,
};

struct AppState {
    counter: usize,
    status_message: String,
}

fn ui(frame: &mut Frame, state: &AppState) {
    let chunks = Layout::default()
        .direction(Direction::Vertical)
        .constraints([
            Constraint::Length(3),
            Constraint::Min(0),
            Constraint::Length(3),
        ])
        .split(frame.size());

    let header = Paragraph::new("Rust Production TUI Monitor")
        .style(Style::default().fg(Color::Cyan))
        .block(Block::default().borders(Borders::ALL));
    frame.render_widget(header, chunks[0]);

    let body = Paragraph::new(format!("Tasks Processed: {}", state.counter))
        .block(Block::default().title("Active Workload").borders(Borders::ALL));
    frame.render_widget(body, chunks[1]);
```

```
    let footer = Paragraph::new(state.status_message.as_str())
        .style(Style::default().fg(Color::DarkGray))
        .block(Block::default().borders(Borders::ALL));
    frame.render_widget(footer, chunks[2]);
}
```

Part 4 – Testing & Distribution

Configuration and Persistence

Configuration and Persistence

Production CLI tools must remember user preferences, cache authentication tokens, load default options, and support layered configuration precedence.

The Configuration Precedence Hierarchy

An idiomatic command-line tool resolves configuration values in a strict order of descending priority:

```
1. CLI Flags & Options (highest priority, e.g. --timeout 30)
2. Environment Variables (e.g. MYCLI_TIMEOUT=30)
3. Local Project Config (.mycli.toml in current working directory)
4. User Global Config (~/.config/mycli/config.toml)
5. Built-in Application Defaults (lowest priority)
```

Standard Directory Paths with `dirs`

Different operating systems adhere to different path conventions: - Linux: `~/.config/mycli` (XDG Base Directory standard) - macOS: `~/Library/Application Support/mycli` - Windows: `%APPDATA%\mycli`

The `dirs` crate discovers these paths safely across all platforms:

```
[dependencies]
dirs = "5.0"
serde = { version = "1.0", features = ["derive"] }
toml = "0.8"
```

```
use std::path::PathBuf;
use serde::{Deserialize, Serialize};
use std::fs;
use anyhow::{Context, Result};

#[derive(Serialize, Deserialize, Debug)]
pub struct AppConfig {
    pub api_endpoint: String,
    pub timeout_seconds: u64,
```

```
    pub verbose: bool,
}

impl Default for AppConfig {
    fn default() -> Self {
        Self {
            api_endpoint: "https://api.example.com".to_string(),
            timeout_seconds: 30,
            verbose: false,
        }
    }
}

pub fn get_config_path() -> Option<PathBuf> {
    dirs::config_dir().map(|p| p.join("mycli").join("config.toml"))
}

pub fn load_or_create_config() -> Result<AppConfig> {
    let path = match get_config_path() {
        Some(p) => p,
        None => return Ok(AppConfig::default()),
    };

    if !path.exists() {
        if let Some(parent) = path.parent() {
            fs::create_dir_all(parent)?;
        }
        let default_config = AppConfig::default();
        let toml_str = toml::to_string_pretty(&default_config)?;
        fs::write(&path, toml_str)?;
        return Ok(default_config);
    }

    let content = fs::read_to_string(&path)
        .with_context(|| format!("Failed to read {}", path.display()))?;
    let config: AppConfig = toml::from_str(&content)?;
    Ok(config)
}
```

Logging and Observability

Logging and Observability

Terminal utilities require two distinct output modes: 1. **Human-oriented user output:** Formatted tables, status indicators, and clean errors directed to stdout or stderr. 2. **Developer diagnostics:** Granular debug logs, timestamps, span timings, and trace events directed to log files or activated via `--verbose` / `RUST_LOG`.

Structured Instrumentation with `tracing`

The `tracing` crate provides structured, event- and span-based observability:

```
[dependencies]
tracing = "0.1"
tracing-subscriber = { version = "0.3", features = ["env-filter", "fmt"] }
```

Configuring `tracing-subscriber` for CLIs

A well-behaved CLI suppresses debug noise by default, allowing users to activate granular logs via environment variables (`RUST_LOG=debug`) or a `-v`/`-vv` flag:

```
use tracing::{info, debug, warn, instrument};
use tracing_subscriber::{fmt, EnvFilter};

pub fn init_observability(verbosity: u8) {
    let default_filter = match verbosity {
        0 => "warn",
        1 => "info",
        2 => "debug",
        _ => "trace",
    };

    let filter = EnvFilter::try_from_default_env()
        .unwrap_or_else(|_| EnvFilter::new(default_filter));

    // Print logs to stderr so stdout remains clean for Unix piping
    fmt()
        .with_env_filter(filter)
        .with_target(false)
```

```
        .with_writer(std::io::stderr)
        .init();
}

#[instrument]
pub fn synchronize_database(target: &str) {
    info!(target, "Beginning database synchronization");
    debug!("Resolving network routes...");
    // ...
    info!("Synchronization complete");
}
```

Progress Bars and Spinners with `indicatif`

For long-running tasks, `indicatif` renders multi-progress bars, download speeds, and spinners without corrupting terminal output:

```
[dependencies]
indicatif = "0.17"
```

```
use indicatif::{ProgressBar, ProgressStyle};
use std::thread;
use std::time::Duration;

pub fn execute_with_progress(items: usize) {
    let pb = ProgressBar::new(items as u64);
    pb.set_style(
        ProgressStyle::default_bar()
            .template("{spinner:.green} [{elapsed_precise}] [{bar:40.cyan/blue}] {pos}/{len}
            .unwrap()
            .progress_chars("#>-"),
    );

    for _ in 0..items {
        thread::sleep(Duration::from_millis(20));
        pb.inc(1);
    }
    pb.finish_with_message("Batch operation complete");
}
```

Testing Command-Line Tools

Testing Command-Line Tools

Testing CLI applications requires a tiered verification strategy: 1. **Unit tests:** Validating parsers, data models, and business logic without executing the binary. 2. **Integration tests (`assert_cmd`):** Spawning the compiled binary, passing CLI flags, providing mock stdin, and asserting exit codes, stdout, and stderr. 3. **Filesystem isolation (`tempfile`):** Creating disposable directory fixtures so tests never touch real user paths.

Testing Crates Setup

In `Cargo.toml`:

```
[dev-dependencies]
assert_cmd = "2.0"
predicates = "3.1"
tempfile = "3.10"
```

Black-Box Integration Tests with `assert_cmd`

In `tests/cli_tests.rs`:

```
use assert_cmd::Command;
use predicates::prelude::*;

#[test]
fn test_missing_subcommand_prints_help() {
    let mut cmd = Command::cargo_bin("mycli").unwrap();
    cmd.assert()
        .failure()
        .stderr(predicate::str::contains("Usage:"));
}

#[test]
fn test_version_flag() {
    let mut cmd = Command::cargo_bin("mycli").unwrap();
    cmd.arg("--version")
        .assert()
        .success()
```

```
        .stdout(predicate::str::contains("0.1.0"));
}

#[test]
fn test_file_processing_in_temp_dir() -> Result<(), Box<dyn std::error::Error>> {
    let temp_dir = tempfile::tempdir()?;
    let input_file = temp_dir.path().join("sample.txt");
    std::fs::write(&input_file, "Rust Systems Engineering\n")?;

    let mut cmd = Command::cargo_bin("mycli").unwrap();
    cmd.arg("process")
        .arg(&input_file)
        .assert()
        .success()
        .stdout(predicate::str::contains("Processed 1 lines"));

    Ok(())
}
```

Mocking Standard Input (Stdin)

`assert_cmd` allows piping synthetic inputs directly into the child process:

```
#[test]
fn test_stdin_piping() {
    let mut cmd = Command::cargo_bin("mycli").unwrap();
    cmd.arg("count")
        .write_stdin("first line\nsecond line\nthird line\n")
        .assert()
        .success()
        .stdout(predicate::str::contains("Total lines: 3"));
}
```

Building Cross-Platform Tools

Building Cross-Platform Tools

A key advantage of Rust is compile-once, run-anywhere ergonomics across Linux, macOS, and Windows. However, subtle operating system differences can easily break a CLI utility.

Key Operating System Divergences

Feature	Linux / Unix	macOS	Windows
Path Separator	`/`	`/`	`\` (or `/`)
Line Ending	`\n` (LF)	`\n` (LF)	`\r\n` (CRLF)
Path Casing	Case-sensitive	Case-preserving, insensitive	Case-insensitive
File Permissions	POSIX modes (`chmod`)	POSIX modes	Access Control Lists (ACLs)
Pipes & Terminals	POSIX TTY (`/dev/tty`)	POSIX TTY	Windows Console (`CONIN$`, `CONOUT$`)

Idiomatic Cross-Platform Patterns in Rust

1. Always Use `std::path::PathBuf` and Slices

Never concatenate file paths using string formatting like `format!("{}/{}", dir, file)`. Use `.join()`:

```
use std::path::{Path, PathBuf};

fn resolve_cache_file(base: &Path, name: &str) -> PathBuf {
    base.join("cache").join(name) // Evaluates to \ on Windows and / on Unix
}
```

2. Normalizing Line Endings

When parsing text data or comparing outputs, strip trailing carriage returns:

```
fn clean_line(raw: &str) -> &str {
    raw.strip_suffix("\r\n")
```

```
        .or_else(|| raw.strip_suffix('\n'))
        .unwrap_or(raw)
}
```

3. Conditional Compilation with `#[cfg(...)]`

When accessing OS-specific syscalls (e.g., Unix file permissions or Windows registry entries):

```
#[cfg(unix)]
fn set_executable_permission(path: &std::path::Path) -> std::io::Result<()> {
    use std::os::unix::fs::PermissionsExt;
    let mut perms = std::fs::metadata(path)?.permissions();
    perms.set_mode(0o755);
    std::fs::set_permissions(path, perms)
}

#[cfg(windows)]
fn set_executable_permission(_path: &std::path::Path) -> std::io::Result<()> {
    // Windows file permissions rely on ACLs and executable extensions (.exe, .bat)
    Ok(())
}
```

Cross-Compiling with `cross`

`cross` executes `cargo` commands inside curated Docker/Podman containers containing cross-toolchains:

```
# Build a fully static x86_64 Linux binary
cross build --target x86_64-unknown-linux-musl --release

# Build for ARM64 Linux (Raspberry Pi / Graviton)
cross build --target aarch64-unknown-linux-musl --release

# Build for Windows from Linux/macOS
cross build --target x86_64-pc-windows-gnu --release
```

Part 5 – Advanced

Designing for Production

Designing for Production

Shipping a tool to thousands of developers or deploying it inside enterprise CI/CD pipelines requires high-reliability engineering: respecting signal interrupts, supporting machine-readable output formats, and managing resource consumption.

Signal Handling and Graceful Shutdown

When an operator terminates a running utility with `Ctrl+C` (SIGINT) or Kubernetes sends SIGTERM, the process must clean up temporary files, flush write buffers, and restore the terminal state:

```
[dependencies]
ctrlc = "3.4"
```

```
use std::sync::atomic::{AtomicBool, Ordering};
use std::sync::Arc;
use std::thread;
use std::time::Duration;

fn main() {
    let running = Arc::new(AtomicBool::new(true));
    let r = running.clone();

    ctrlc::set_handler(move || {
        eprintln!("\nInterrupted! Cleaning up before exit...");
        r.store(false, Ordering::SeqCst);
    }).expect("Error setting Ctrl-C handler");

    while running.load(Ordering::SeqCst) {
        thread::sleep(Duration::from_millis(100));
    }

    println!("Graceful shutdown complete.");
}
```

Machine-Readable Output: The `--json` Contract

Every production CLI should support both human-friendly formatted terminal output and structured machine-readable JSON for scripting and automation:

```
use clap::{Parser, ValueEnum};
use serde::Serialize;

#[derive(ValueEnum, Clone, Copy, Debug)]
enum OutputFormat {
    Human,
    Json,
    Yaml,
}

#[derive(Serialize)]
struct Report {
    status: String,
    elapsed_ms: u64,
    errors_count: usize,
}

fn emit_report(report: &Report, format: OutputFormat) {
    match format {
        OutputFormat::Human => {
            println!("Status:       {}", report.status);
            println!("Duration:     {}ms", report.elapsed_ms);
            println!("Errors found: {}", report.errors_count);
        }
        OutputFormat::Json => {
            let json = serde_json::to_string_pretty(report).unwrap();
            println!("{json}");
        }
        OutputFormat::Yaml => {
            // let yaml = serde_yaml::to_string(report).unwrap();
            // println!("{yaml}");
        }
    }
}
```

Respecting `NO_COLOR` and Terminal Detection

Follow the community no-color.org specification: - If `NO_COLOR` environment variable is set, disable ANSI styling. - If stdout is not an interactive TTY (e.g., piped to a file: `mycli > out.txt`), disable ANSI colors automatically. - Crate crates like `colored` or `anstream` respect these conventions automatically.

Advanced Rust Patterns for CLI Tools

Advanced Rust Patterns for CLI Tools

High-performance CLI tools like `ripgrep` and `fd` leverage advanced Rust language patterns to maximize throughput and minimize overhead.

Zero-Cost Parallelism with `rayon`

When processing thousands of independent files, convert standard iterators into parallel work-stealing iterators with `rayon`:

```
[dependencies]
rayon = "1.10"
```

```
use rayon::prelude::*;
use std::path::{Path, PathBuf};

fn parallel_file_scan(files: &[PathBuf], pattern: &str) -> Vec<PathBuf> {
    files
        .par_iter()
        .filter(|path| {
            if let Ok(content) = std::fs::read_to_string(path) {
                content.contains(pattern)
            } else {
                false
            }
        })
        .cloned()
        .collect()
}
```

`rayon` automatically configures a work-stealing thread pool sized to the host's available physical CPU cores without manual thread synchronization.

Memory Mapping for Massive Files (`memmap2`)

Reading a 10 GB file via `read()` causes repeated kernel context switches and buffer copies. Memory-mapping maps file pages directly into the virtual address space:

```
[dependencies]
memmap2 = "0.9"
```

```
use memmap2::Mmap;
use std::fs::File;
use std::path::Path;
use anyhow::Result;

fn search_mmap(path: &Path, needle: &[u8]) -> Result<bool> {
    let file = File::open(path)?;
    // Safety: The caller must ensure the underlying file is not concurrently truncated
    let mmap = unsafe { Mmap::map(&file)? };

    Ok(mmap.windows(needle.len()).any(|window| window == needle))
}
```

Static Global Initialization with `LazyLock`

Starting in Rust 1.80, `std::sync::LazyLock` is built into the standard library, eliminating the need for `lazy_static` or `once_cell`:

```
use std::sync::LazyLock;
use regex::Regex;

static EMAIL_REGEX: LazyLock<Regex> = LazyLock::new(|| {
    Regex::new(r"^[a-zA-Z0-9_.+-]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-.]+$").unwrap()
});

fn is_valid_email(email: &str) -> bool {
    EMAIL_REGEX.is_match(email)
}
```

Building Developer Tools

Building Developer Tools

Developer tools—linters, scaffolding engines, formatters, and code generators—require specialized features: generating shell auto-completion scripts, synthesizing man pages, and running interactive prompts.

Dynamic Shell Auto-Completion with `clap_complete`

Users expect tab-completion for subcommands, flags, and file paths across `bash`, `zsh`, `fish`, and `powershell`. `clap_complete` generates these completion scripts directly from your `clap` definition:

```
[dependencies]
clap = { version = "4.5", features = ["derive"] }
clap_complete = "4.5"
```

```
use clap::{Command, CommandFactory, Parser, Subcommand};
use clap_complete::{generate, Shell};
use std::io;

#[derive(Parser)]
#[command(name = "mydevtool")]
struct Cli {
    #[command(subcommand)]
    command: Commands,
}

#[derive(Subcommand)]
enum Commands {
    /// Generate shell auto-completions
    Completions {
        #[arg(value_enum)]
        shell: Shell,
    },
    /// Format codebase
    Format,
    /// Lint source files
    Lint,
}
```

```
fn print_completions(shell: Shell, cmd: &mut Command) {
    generate(shell, cmd, cmd.get_name().to_string(), &mut io::stdout());
}

fn main() {
    let cli = Cli::parse();

    match cli.command {
        Commands::Completions { shell } => {
            let mut cmd = Cli::command();
            print_completions(shell, &mut cmd);
        }
        Commands::Format => println!("Formatting code..."),
        Commands::Lint => println!("Linting code..."),
    }
}
```

A user can install completions into their shell environment with:

```
mydevtool completions zsh > ~/.zsh/completion/_mydevtool
```

Interactive Inquirer Prompts with `inquire`

When users omit required parameters, prompt them interactively:

```
[dependencies]
inquire = "0.7"
```

```
use inquire::{Select, Text, Confirm};

fn prompt_project_scaffolding() -> anyhow::Result<()> {
    let project_name = Text::new("Project name:")
        .with_default("my-service")
        .prompt()?;

    let language = Select::new(
        "Choose primary language template:",
        vec!["Rust (Tokio)", "Rust (WASM)", "Rust (CLI)"],
    ).prompt()?;

    let initialize_git = Confirm::new("Initialize git repository?")
        .with_default(true)
        .prompt()?;
```

```
    println!("Scaffolding {project_name} ({language}) with Git: {initialize_git}");
    Ok(())
}
```

Distributing and Shipping Your Tool

Distributing and Shipping Your Tool

Writing high-performance code is only half the battle. A CLI tool's adoption depends heavily on frictionless installation across diverse package ecosystems and environments.

Distribution Channels for Rust CLIs

Distribution Channel	Target Audience	Mechanism
`cargo install`	Rust developers	Compiles from source via `crates.io`
`cargo binstall`	Rust developers	Downloads pre-compiled release artifacts instantly
Homebrew	macOS / Linux	Formula pointing to pre-built tarball or source
GitHub Releases	All platforms	Pre-compiled static binaries in `.tar.gz` and `.zip`
Debian / RPM	Linux servers	`.deb` and `.rpm` native system packages
Scoop / Chocolatey	Windows developers	Manifests pulling binary releases

Automated Multi-Platform Release with GitHub Actions and `cargo-dist`

`cargo-dist` automates cross-compilation, tarball bundling, shell installer generation, and GitHub Release uploads with a single command:

```
cargo install cargo-dist
cargo dist init
```

A sample `.github/workflows/release.yml` matrices the target architectures:

```
name: Release
on:
  push:
    tags:
      - 'v*'
```

```yaml
jobs:
  build-release:
    strategy:
      matrix:
        target:
          - x86_64-unknown-linux-musl
          - aarch64-unknown-linux-musl
          - x86_64-apple-darwin
          - aarch64-apple-darwin
          - x86_64-pc-windows-msvc
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build with cross
        run: |
          cargo install cross
          cross build --target ${{ matrix.target }} --release
```

Creating Homebrew Formulas

Homebrew users expect to run `brew install myorg/tap/mytool`. An idiomatic Homebrew formula:

```ruby
class Mytool < Formula
  desc "Blazing fast systems developer toolkit"
  homepage "https://github.com/myorg/mytool"
  version "1.0.0"

  on_macos do
    if Hardware::CPU.arm?
      url "https://github.com/myorg/mytool/releases/download/v1.0.0/mytool-aarch64-apple-dar
      sha256 "e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855"
    end
  end

  def install
    bin.install "mytool"
  end

  test do
    system "#{bin}/mytool", "--version"
  end
end
```

Part 6 – Capstone Projects

Capstone 1: fsearch, a Fast File Search Tool

Capstone 1: fsearch, a Fast File Search Tool

`fsearch` is a high-performance command-line file search utility modeled after `fd`. It demonstrates concurrent filesystem traversal, glob filtering, regex pattern matching, and formatted ANSI terminal output.

Architecture and Dependencies

In `Cargo.toml`:

```
[package]
name = "fsearch"
version = "0.1.0"
edition = "2024"

[dependencies]
clap = { version = "4.5", features = ["derive"] }
walkdir = "2.5"
regex = "1.10"
colored = "2.1"
anyhow = "1.0"
rayon = "1.10"
```

CLI Definition

```
use clap::Parser;
use std::path::PathBuf;

#[derive(Parser, Debug)]
#[command(name = "fsearch", about = "Fast multi-threaded file finder")]
pub struct Opts {
    /// Regular expression pattern to search for
    pub pattern: String,

    /// Root directory to search from
    #[arg(default_value = ".")]
    pub root: PathBuf,
```

```
    /// Case-insensitive search
    #[arg(short = 'i', long)]
    pub ignore_case: bool,

    /// Maximum directory traversal depth
    #[arg(short = 'd', long)]
    pub max_depth: Option<usize>,

    /// Search only file extensions (e.g. 'rs', 'toml')
    #[arg(short = 'e', long)]
    pub extension: Option<String>,
}
```

Parallel Traversal and Matching Engine

```
use anyhow::Result;
use colored::*;
use regex::{Regex, RegexBuilder};
use std::path::Path;
use walkdir::WalkDir;

pub fn execute_search(opts: Opts) -> Result<()> {
    let re = RegexBuilder::new(&opts.pattern)
        .case_insensitive(opts.ignore_case)
        .build()?;

    let mut walker = WalkDir::new(&opts.root);
    if let Some(depth) = opts.max_depth {
        walker = walker.max_depth(depth);
    }

    let entries: Vec<_> = walker
        .into_iter()
        .filter_map(Result::ok)
        .collect();

    for entry in entries {
        let path = entry.path();

        // Check extension filter
        if let Some(ref ext_filter) = opts.extension {
            if path.extension().and_then(|s| s.to_str()) != Some(ext_filter) {
                continue;
            }
        }
```

```
        // Match filename
        if let Some(file_name) = path.file_name().and_then(|s| s.to_str()) {
            if re.is_match(file_name) {
                print_match(path, entry.file_type().is_dir());
            }
        }
    }

    Ok(())
}

fn print_match(path: &Path, is_dir: bool) {
    if is_dir {
        println!("{}", path.display().to_string().blue().bold());
    } else {
        println!("{}", path.display().to_string().green());
    }
}
```

Verification

```
cargo run -- ".*\.rs$" -d 3
```

Capstone 2: tudo, a TUI Task Manager

Capstone 2: tudo, a TUI Task Manager

`tudo` is an interactive Terminal User Interface (TUI) task management utility written with `ratatui` and `crossterm`. It demonstrates stateful widget rendering, keyboard event loops, modal dialogs, and atomic JSON persistence.

Dependencies

```
[package]
name = "tudo"
version = "0.1.0"
edition = "2024"

[dependencies]
ratatui = "0.26"
crossterm = "0.27"
serde = { version = "1.0", features = ["derive"] }
serde_json = "1.0"
anyhow = "1.0"
dirs = "5.0"
```

Data Model and Persistence

```
use serde::{Deserialize, Serialize};

#[derive(Serialize, Deserialize, Clone, Debug)]
pub struct Task {
    pub id: usize,
    pub title: String,
    pub completed: bool,
}

#[derive(Serialize, Deserialize, Default)]
pub struct TaskList {
    pub tasks: Vec<Task>,
    pub next_id: usize,
}
```

```
impl TaskList {
    pub fn add(&mut self, title: String) {
        self.tasks.push(Task {
            id: self.next_id,
            title,
            completed: false,
        });
        self.next_id += 1;
    }

    pub fn toggle(&mut self, index: usize) {
        if let Some(task) = self.tasks.get_mut(index) {
            task.completed = !task.completed;
        }
    }

    pub fn delete(&mut self, index: usize) {
        if index < self.tasks.len() {
            self.tasks.remove(index);
        }
    }
}
```

Interactive Event Loop

```
use crossterm::event::{self, Event, KeyCode};
use ratatui::{
    layout::{Constraint, Layout},
    style::{Color, Modifier, Style},
    widgets::{Block, Borders, List, ListItem},
    Frame,
};
use std::time::Duration;

pub struct App {
    pub list: TaskList,
    pub selected_index: usize,
    pub should_quit: bool,
}

impl App {
    pub fn update(&mut self) -> anyhow::Result<()> {
        if event::poll(Duration::from_millis(100))? {
            if let Event::Key(key) = event::read()? {
```

```
                match key.code {
                    KeyCode::Char('q') | KeyCode::Esc => self.should_quit = true,
                    KeyCode::Char('j') | KeyCode::Down => {
                        if !self.list.tasks.is_empty() {
                            self.selected_index = (self.selected_index + 1) % self.list.task
                        }
                    }
                    KeyCode::Char('k') | KeyCode::Up => {
                        if !self.list.tasks.is_empty() {
                            self.selected_index = self.selected_index.checked_sub(1)
                                .unwrap_or(self.list.tasks.len() - 1);
                        }
                    }
                    KeyCode::Char(' ') => self.list.toggle(self.selected_index),
                    KeyCode::Char('d') => self.list.delete(self.selected_index),
                    _ => {}
                }
            }
        }
        Ok(())
    }

    pub fn render(&self, frame: &mut Frame) {
        let area = frame.size();
        let items: Vec<ListItem> = self.list.tasks
            .iter()
            .enumerate()
            .map(|(i, task)| {
                let status = if task.completed { "[x]" } else { "[ ]" };
                let content = format!("{status} {}", task.title);
                let mut style = Style::default();
                if i == self.selected_index {
                    style = style.fg(Color::Yellow).add_modifier(Modifier::BOLD);
                }
                ListItem::new(content).style(style)
            })
            .collect();

        let widget = List::new(items)
            .block(Block::default().title("tudo - Task Manager (j/k: nav, space: toggle, d:

        frame.render_widget(widget, area);
    }
}
```

Capstone 3: sysmon, a System Monitor

Capstone 3: sysmon, a System Monitor

`sysmon` is a lightweight cross-platform system monitoring utility. It gathers operating system telemetry—CPU core utilization, memory breakdown, swap stats, disk usage, and top active processes—and renders either a real-time terminal dashboard or a periodic JSON metric stream.

Dependencies

```
[package]
name = "sysmon"
version = "0.1.0"
edition = "2024"

[dependencies]
sysinfo = "0.30"
clap = { version = "4.5", features = ["derive"] }
serde = { version = "1.0", features = ["derive"] }
serde_json = "1.0"
anyhow = "1.0"
```

System Telemetry Engine

```
use sysinfo::System;
use serde::Serialize;

#[derive(Serialize, Debug)]
pub struct SystemSnapshot {
    pub total_memory_mb: u64,
    pub used_memory_mb: u64,
    pub global_cpu_usage: f32,
    pub process_count: usize,
    pub top_processes: Vec<ProcessMetric>,
}

#[derive(Serialize, Debug)]
pub struct ProcessMetric {
    pub pid: u32,
```

```
    pub name: String,
    pub memory_mb: u64,
    pub cpu_usage: f32,
}

pub fn collect_snapshot(sys: &mut System) -> SystemSnapshot {
    sys.refresh_all();

    let total_memory_mb = sys.total_memory() / (1024 * 1024);
    let used_memory_mb = sys.used_memory() / (1024 * 1024);
    let global_cpu_usage = sys.global_cpu_info().cpu_usage();

    let mut processes: Vec<_> = sys.processes()
        .iter()
        .map(|(pid, proc_info)| ProcessMetric {
            pid: pid.as_u32(),
            name: proc_info.name().to_string_lossy().into_owned(),
            memory_mb: proc_info.memory() / (1024 * 1024),
            cpu_usage: proc_info.cpu_usage(),
        })
        .collect();

    // Sort by CPU consumption descending
    processes.sort_by(|a, b| b.cpu_usage.partial_cmp(&a.cpu_usage).unwrap());
    processes.truncate(5);

    SystemSnapshot {
        total_memory_mb,
        used_memory_mb,
        global_cpu_usage,
        process_count: sys.processes().len(),
        top_processes: processes,
    }
}
```

Periodic Observation Loop

```
use std::thread;
use std::time::Duration;

fn main() {
    let mut sys = System::new_all();

    loop {
```

```
        let snapshot = collect_snapshot(&mut sys);
        let json = serde_json::to_string(&snapshot).unwrap();
        println!("{json}");
        thread::sleep(Duration::from_secs(2));
    }
}
```

Capstone 4: devkit, a Developer Toolkit

Capstone 4: devkit, a Developer Toolkit

`devkit` is an all-in-one developer utility that consolidates cryptographic hashing, base64 encoding/decoding, JSON linting, and HTTP health checking under a single cohesive subcommand-driven binary.

Dependencies

```toml
[package]
name = "devkit"
version = "0.1.0"
edition = "2024"

[dependencies]
clap = { version = "4.5", features = ["derive"] }
sha2 = "0.10"
base64 = "0.22"
serde_json = "1.0"
reqwest = { version = "0.12", features = ["blocking"] }
anyhow = "1.0"
```

CLI Specification

```rust
use clap::{Parser, Subcommand};
use std::path::PathBuf;

#[derive(Parser)]
#[command(name = "devkit", about = "Swiss Army Knife Developer Utility")]
pub struct Cli {
    #[command(subcommand)]
    pub command: Commands,
}

#[derive(Subcommand)]
pub enum Commands {
    /// Compute SHA-256 cryptographic checksum of file or text
    Hash {
```

```
        #[arg(short, long)]
        text: Option<String>,
        #[arg(short, long)]
        file: Option<PathBuf>,
    },
    /// Base64 encode or decode data
    Base64 {
        #[arg(short, long)]
        decode: bool,
        input: String,
    },
    /// Format and validate JSON payload
    FormatJson {
        #[arg(default_value = "-")]
        input: String,
    },
    /// Query HTTP endpoint latency and response status
    Probe {
        url: String,
    },
}
```

Modular Implementation

```
use anyhow::{bail, Context, Result};
use base64::prelude::*;
use sha2::{Digest, Sha256};
use std::fs;
use std::time::Instant;

pub fn handle_hash(text: Option<String>, file: Option<std::path::PathBuf>) -> Result<()> {
    let mut hasher = Sha256::new();
    if let Some(t) = text {
        hasher.update(t.as_bytes());
    } else if let Some(f) = file {
        let bytes = fs::read(&f).with_context(|| format!("Failed to read {:?}", f))?;
        hasher.update(&bytes);
    } else {
        bail!("Provide either --text or --file");
    }
    let result = hasher.finalize();
    println!("{:x}", result);
    Ok(())
}
```

```
pub fn handle_base64(decode: bool, input: &str) -> Result<()> {
    if decode {
        let bytes = BASE64_STANDARD.decode(input)?;
        let text = String::from_utf8_lossy(&bytes);
        println!("{text}");
    } else {
        let encoded = BASE64_STANDARD.encode(input);
        println!("{encoded}");
    }
    Ok(())
}

pub fn handle_probe(url: &str) -> Result<()> {
    let start = Instant::now();
    let res = reqwest::blocking::get(url)?;
    let elapsed = start.elapsed();

    println!("URL:      {}", url);
    println!("Status:   {}", res.status());
    println!("Latency:  {:.2?}", elapsed);
    Ok(())
}
```

Part 7 – The Rust CLI Ecosystem

Contributing to the Rust Ecosystem

Contributing to the Rust Ecosystem

The strength of Rust's command-line domain stems from its active, collaborative open-source ecosystem. Authors of popular utilities like `bat`, `eza`, `ripgrep`, and `clap` maintain open RFC discussions, share common crates, and welcome community contributions.

The CLI Working Group (`rust-cli`)

The official Rust CLI working group establishes guidelines, curated book resources, and common libraries to ensure consistency across the ecosystem:

- **CLI Guidelines:** Standard conventions for flag naming (`-h`, `-V`), exit codes, error presentation, and environment variable naming.
- **Shared Crate Maintenance:** Fostering community-owned core libraries like `assert_cmd`, `predicates`, and `terminal_size`.

Extracting Domain Crates

When building custom CLI tools, reusable modules should be extracted and published to crates.io:

1. **Keep CLIs thin:** Decouple domain algorithms into standalone library crates.
2. **Minimize dependency bloat:** Use feature flags in `Cargo.toml` so consumers don't pull unnecessary transitive dependencies.
3. **Follow Semantic Versioning (SemVer):** Strictly increment patch, minor, or major versions to avoid breaking downstream pipelines.
4. **Automate documentation:** Leverage `docs.rs` by documenting public items with doc comments (`///`) and doctests.

```
/// Calculates the checksum for a given file buffer.
///
/// # Examples
///
/// ```
/// let data = b"hello world";
/// assert_eq!(data.len(), 11);
/// ```
pub fn calculate_checksum(data: &[u8]) -> u32 {
    data.iter().map(|&b| b as u32).sum()
}
```

The Future of Rust CLI Development

The Future of Rust CLI Development

Command-line computing is undergoing a renaissance, driven by next-generation terminal emulators, universal WebAssembly runtimes, and intelligent agent tooling.

Emerging Frontiers in Terminal Applications

1. WebAssembly and WASI (WebAssembly System Interface)

With WASI (specifically WASI preview 2 and component models), Rust CLI binaries compile to portable `.wasm` modules that run securely across any operating system, browser sandbox, or edge runtime without native compilation:

```
cargo build --target wasm32-wasip1 --release
wasmtime target/wasm32-wasip1/release/mycli.wasm
```

2. Next-Generation Terminal Protocols

Modern terminal emulators (Ghostty, WezTerm, Alacritty, Kitty) support advanced escape protocols that surpass legacy VT100 limitations: - **Kitty Graphics Protocol & Sixel:** Rendering inline high-resolution bitmap images, diagrams, and plots directly within terminal buffers. - **Synchronized Output:** Preventing screen tearing during high-framerate TUI redraws. - **Extended Keyboard Protocol:** Disambiguating key combos (such as distinguishing `Ctrl+I` from `Tab` or detecting key-release events).

3. Headless Tooling for Autonomous AI Agents

As AI coding assistants and autonomous agents execute tasks in terminals, CLI tools increasingly provide structured agent interfaces: - **Zero-latency JSON-RPC / MCP:** Exposing CLI capabilities over JSON streams. - **Machine-parsable errors:** Eliminating ambiguous error messages in automated workflows. - **Dry-run simulation engines:** Allowing agents to verify actions before modifying host filesystems.

Rust is uniquely positioned to lead this next generation of terminal software: fast, safe, portable, and ready for whatever systems emerge next.

Conclusion: The Rust Mindset

Core Principles

Core Principles

Looking back over our journey, three fundamental engineering principles define idiomatic Rust:

1. **Safety is an Asset, Not a Constraint:** The borrow checker is an automated proof assistant ensuring thread and memory safety.
2. **Explicit over Implicit:** Errors are values, allocations are intentional, and data flows are visible.
3. **Zero-Cost Abstractions:** High-level expressiveness compiles down to optimal low-level machine instructions.

Where Rust Shines

Where Rust Shines

Rust delivers unmatched value in: - High-performance network proxies, database engines, and storage systems. - Operating system kernels, drivers, and hypervisors. - Low-latency financial trading platforms. - WebAssembly client-side computational modules and edge serverless workers. - Embedded firmware and safety-critical medical and automotive controllers.

Where Rust May Not Be the Best Fit

Where Rust May Not Be the Best Fit

Rust may introduce unnecessary friction for: - Disposable one-off scripts where Python or Bash finishes the job in minutes. - Rapidly churning early-stage UI prototypes where types and domain models change hourly. - Systems where memory footprint and latency do not matter and teams already possess deep expertise in managed languages.

Continuing Your Journey

Continuing Your Journey

To continue deepening your Rust mastery: - Read *The Rust Reference* and *The Rustonomicon* for deep language semantics and unsafe invariants. - Explore open-source projects on crates.io to observe production architectures. - Contribute bug fixes and documentation improvements to the community.

Final Thoughts

Final Thoughts

Rust changes the way you think about programming. Even when writing Go, C, or Python, the habits ingrained by Rust—thinking carefully about ownership, data sharing, mutability, and error handling—will make you a vastly more rigorous systems engineer.

Welcome to the community, and build fearless software.

Rust Atomics and Locks

Preface and Foreword

Preface and Foreword

Foreword by Paul E. McKenney

This book provides an excellent overview of low-level concurrency in the Rust language, including threads, locks, reference counts, atomics, mailboxes/channels, and much else besides. It digs into issues with CPUs and operating systems, the latter summarizing challenges inherent in making concurrent code work correctly on Linux, macOS, and Windows. I was particularly happy to see that Mara illustrates these topics with working Rust code. It wraps up by discussing semaphores, lock-free linked lists, queued locks, sequence locks, and even RCU.

So what does this book offer someone like myself, who has been slinging C code for almost 40 years, most recently in the nether depths of the Linux kernel?

I first learned of Rust from any number of enthusiasts and Linux-related conferences. Nevertheless, I was happily minding my own business until I was called out by name in a Rust-related LWN article, "Using Rust for Kernel Development". Thus prodded, I wrote a blog series entitled "So You Want to Rust the Linux Kernel?" This blog series sparked a number of spirited discussions, a few of which are visible in the series' comments.

In one such discussion, a long-time Linux-kernel developer who has also written a lot of Rust code told me that when writing concurrent code in Rust, you should write it the way Rust wants you to. I have since learned that although this is great advice, it leaves open the question of exactly what Rust wants. This book gives excellent answers to this question, and is thus valuable both to Rust developers wishing to learn concurrency and to developers of concurrent code in other languages who would like to learn how best to do so in Rust.

I of course fall into this latter category. However, I must confess that many of the spirited discussions about Rust concurrency remind me of my parents' and grandparents' long-ago complaints about the inconvenient safety features that were being added to power tools such as saws and drills. Some of those safety features are now ubiquitous, but hammers, chisels, and chainsaws have not changed all that much. It was not at all easy to work out which mechanical safety features would stand the test of time, so I recommend approaching software safety features with an attitude of profound humility. And please understand that I am addressing the proponents of such features as well as their detractors.

Which brings us to another group of potential readers, the Rust skeptics. While I do believe that most Rust skeptics are doing the community a valuable service by pointing out opportunities for improvement, all but the most Rust-savvy of skeptics would benefit from reading this book. If nothing else, doing so would enable them to provide sharper and better-targeted criticisms.

Then there are those dyed-in-the-wool non-Rust developers who would prefer to implement Rust's concurrency-related safety mechanisms in their own favorite language. This book will give them

a deeper understanding of the Rust mechanisms that they would like to replicate, or, better yet, improve upon.

Finally, any number of Linux-kernel developers are noting the progress that Rust is making toward being included in the Linux kernel; for example, see Jonathan Corbet's article, "Next Steps for Rust in the Kernel". As of October 2022, this is still an experiment, but one that is being taken increasingly seriously. In fact, seriously enough that Linus Torvalds has accepted the first bits of Rust-language support into version 6.1 of the Linux kernel.

Whether you are reading this book to expand your Rust repertoire to include concurrency, to expand your concurrency repertoire to include Rust, to improve your existing non-Rust environment, or just to look at concurrency from a different viewpoint, I wish you the very best on your journey!

Paul E. McKenney

Meta Platforms Kernel Team

Meta

October 2022

Preface

Rust has played, and keeps playing, a significant role in making systems programming more accessible. However, low-level concurrency topics such as atomics and memory ordering are still often thought of as somewhat mystical subjects that are best left to a very small group of experts.

While working on Rust-based real-time control systems and the Rust standard library over the past few years, I found that many of the available resources on atomics and related topics only cover a small part of the information I was looking for. Many resources focus entirely on C and C++, which can make it hard to form the connection with Rust's concept of (memory and thread) safety and type system. The resources that cover the details of the abstract theory, like C++'s memory model, often only vaguely explain how it relates to actual hardware, if at all. There are many resources that cover every detail of the actual hardware, such as processor instructions and cache coherency, but forming a holistic understanding often requires collecting bits and pieces of information from many different places.

This book is an attempt to put relevant information in one place, connecting it all together, providing everything you need to build your own correct, safe, and ergonomic concurrency primitives, while understanding enough about the underlying hardware and the role of the operating system to be able to make design decisions and basic optimization trade-offs.

Who This Book Is For

The primary audience for this book is Rust developers who want to learn more about low-level concurrency. Additionally, this book can also be suitable for those who are not very familiar with Rust yet, but would like to know what low-level concurrency looks like from a Rust perspective.

It is assumed you know the basics of Rust, have a recent Rust compiler installed, and know how to compile and run Rust code using `cargo`. Rust concepts that are important for concurrency are briefly explained when relevant, so no prior knowledge about Rust concurrency is necessary.

Overview of the Chapters

This book consists of ten chapters. Here's what to expect from each chapter, and what to look forward to:

Chapter 1 — Basics of Rust Concurrency
This chapter introduces all the tools and concepts we need for basic concurrency in Rust, such as threads, mutexes, thread safety, shared and exclusive references, interior mutability, and so on, which are foundational to the rest of the book.

For experienced Rust programmers who are familiar with these concepts, this chapter can serve as a quick refresher. For those who know these concepts from other languages but aren't very familiar with Rust yet, this chapter will quickly fill you in on any Rust-specific knowledge you might need for the rest of the book.

Chapter 2 — Atomics
In the second chapter we'll learn about Rust's atomic types and all their operations. We start with simple load and store operations, and build our way up to more advanced *compare-and-exchange loops*, exploring each new concept with several real-world use cases as usable examples.

While *memory ordering* is relevant for every atomic operation, that topic is left for the next chapter. This chapter only covers situations where *relaxed* memory ordering suffices, which is the case more often than one might expect.

Chapter 3 — Memory Ordering
After learning about the various atomic operations and how to use them, the third chapter introduces the most complicated topic of the book: memory ordering.

We'll explore how the memory model works, what *happens-before relationships* are and how to create them, what all the different memory orderings mean, and why *sequentially consistent ordering* might not be the answer to everything.

Chapter 4 — Building Our Own Spin Lock
After learning the theory, we put it to practice in the next three chapters by building our own versions of several common concurrency primitives. The first of these chapters is a short one, in which we implement a *spin lock*.

We'll start with a very minimal version to put *release and acquire memory ordering* to practice, and then we'll explore Rust's concept of *safety* to turn it into an ergonomic and hard-to-misuse Rust data type.

Chapter 5 — Building Our Own Channels
In Chapter 5, we'll implement from scratch a handful of variations of a *one-shot channel*, a primitive that can be used to send data from one thread to another.

Starting with a very minimal but entirely `unsafe` version, we'll work our way through several ways to design a safe interface, while considering design decisions and their consequences.

Chapter 6 — Building Our Own "Arc"
For the sixth chapter, we'll take on a more challenging memory ordering puzzle. We're going to implement our own version of atomic reference counting from scratch.

After adding support for *weak pointers* and optimizing it for performance, our final version will be practically identical to Rust's standard `std::sync::Arc` type.

Chapter 7 — Understanding the Processor
The seventh chapter is a deep dive into all the low-level details. We'll explore what happens at the processor level, what the *assembly instructions* behind the atomic operations look like on the two most popular processor architectures, what caching is and how it affects the performance of our code, and we'll find out what remains of the memory model at the hardware level.

Chapter 8 — Operating System Primitives
In Chapter 8 we acknowledge that there are things we can't do without the help of the operating system's *kernel* and learn what functionality is available on Linux, macOS, and Windows.

We'll discuss the concurrency primitives that are available through *pthreads* on POSIX systems, find out what we can do with the Windows API, and learn what the Linux *futex syscall* does.

Chapter 9 — Building Our Own Locks
Using what we've learned in the previous chapters, in Chapter 9 we're going to build several implementations of a *mutex*, *condition variable*, and *reader-writer lock* from scratch.

For each of these, we'll start with a minimal but complete version, which we'll then attempt to optimize in various ways. Using some simple benchmark tests, we'll find out that our attempts at optimization don't always increase performance, while we discuss various design trade-offs.

Chapter 10 — Ideas and Inspiration
The final chapter makes sure you don't fall into a void after finishing the book, but are instead left with ideas and inspiration for things to build and explore with your new knowledge and skills, perhaps kicking off an exciting journey further into the depths of low-level concurrency.

Code Examples

All code in this book is written for and tested using Rust 1.66.0, which was released on December 15, 2022. Earlier versions do not include all features used in this book. Later versions, however, should work just fine.

For brevity, the code examples do not include `use` statements, except for the first time a new item from the standard library is introduced. As a convenience, the following prelude can be used to import everything necessary to compile any of the code examples in this book:

```
#[allow(unused)]
use std::{
    cell::{Cell, RefCell, UnsafeCell},
    collections::VecDeque,
    marker::PhantomData,
    mem::{ManuallyDrop, MaybeUninit},
    ops::{Deref, DerefMut},
    ptr::NonNull,
    rc::Rc,
    sync::{*, atomic::{*, Ordering::*}},
    thread::{self, Thread},
};
```

You may use all example code offered with this book for any purpose.

Attribution is appreciated, but not required. An attribution usually includes the title, author, publisher, and ISBN. For example: "*Rust Atomics and Locks* by Mara Bos (O'Reilly). Copyright 2023 Mara Bos, 978-1-098-11944-7."

Conventions Used in This Book

The following typographical conventions are used in this book:

Italic
Used for new terms, URLs, and emphasis.

`Constant width`
Used for program listings, as well as within paragraphs to refer to program elements such as variable or function names, data types, statements, and keywords.

> **Note**

This element signifies a tip or suggestion.

> **Note**

This element signifies a general note.

> **Note**

This element indicates a warning or caution.

Contact Information

O'Reilly has a web page for this book, where errata, examples, and any additional information are listed. It is available at .

Email *bookquestions@oreilly.com* to comment or ask technical questions about this book. If you wish to reuse content from this book, and you feel your reuse falls outside fair use or the permission given in this Preface, feel free to contact O'Reilly at *permissions@oreilly.com*.

For news and information about O'Reilly, visit https://oreilly.com.

Follow O'Reilly on Twitter: .

Follow the author on Twitter: .

Basics of Rust Concurrency

Basics of Rust Concurrency

Long before multi-core processors were commonplace, operating systems allowed for a single computer to run many programs concurrently. This is achieved by rapidly switching between processes, allowing each to repeatedly make a little bit of progress, one by one. Nowadays, virtually all our computers and even our phones and watches have processors with multiple cores, which can truly execute multiple processes in parallel.

Operating systems isolate processes from each other as much as possible, allowing a program to do its thing while completely unaware of what any other processes are doing. For example, a process cannot normally access the memory of another process, or communicate with it in any way, without asking the operating system's kernel first.

However, a program can spawn extra *threads of execution*, as part of the same *process*. Threads within the same process are not isolated from each other. Threads share memory and can interact with each other through that memory.

This chapter will explain how threads are spawned in Rust, and all the basic concepts around them, such as how to safely share data between multiple threads. The concepts explained in this chapter are foundational to the rest of the book.

> **Note**

If you're already familiar with these parts of Rust, feel free to skip ahead. However, before you continue to the next chapters, make sure you have a good understanding of threads, interior mutability, `Send` and `Sync`, and know what a mutex, a condition variable, and thread parking are.

Threads in Rust

Every program starts with exactly one thread: the main thread. This thread will execute your `main` function and can be used to spawn more threads if necessary.

In Rust, new threads are spawned using the `std::thread::spawn` function from the standard library. It takes a single argument: the function the new thread will execute. The thread stops once this function returns.

Let's take a look at an example:

```
use std::thread;

fn main() {
    thread::spawn(f);
```

```
    thread::spawn(f);

    println!("Hello from the main thread.");
}

fn f() {
    println!("Hello from another thread!");

    let id = thread::current().id();
    println!("This is my thread id: {id:?}");
}
```

We spawn two threads that will both execute `f` as their main function. Both of these threads will print a message and show their *thread id*, while the main thread will also print its own message.

Thread ID

The Rust standard library assigns every thread a unique identifier. This identifier is accessible through `Thread::id()` and is of the type `ThreadId`. There's not much you can do with a `ThreadId` other than copying it around and checking for equality. There is no guarantee that these IDs will be assigned consecutively, only that they will be different for each thread.

If you run our example program above several times, you might notice the output varies between runs. This is the output I got on my machine during one particular run:

```
Hello from the main thread.
Hello from another thread!
This is my thread id:
```

Surprisingly, part of the output seems to be missing.

What happened here is that the main thread finished executing the `main` function before the newly spawned threads finished executing their functions.

Returning from `main` will exit the entire program, even if other threads are still running.

In this particular example, one of the newly spawned threads had just enough time to get to halfway through the second message, before the program was shut down by the main thread.

If we want to make sure the threads are finished before we return from `main`, we can wait for them by *joining* them. To do so, we have to use the `JoinHandle` returned by the `spawn` function:

```
fn main() {
    let t1 = thread::spawn(f);
    let t2 = thread::spawn(f);

    println!("Hello from the main thread.");

    t1.join().unwrap();
```

```
    t2.join().unwrap();
}
```

The `.join()` method waits until the thread has finished executing and returns a `std::thread::Result`. If the thread did not successfully finish its function because it panicked, this will contain the panic message. We could attempt to handle that situation, or just call `.unwrap()` to panic when joining a panicked thread.

Running this version of our program will no longer result in truncated output:

```
Hello from the main thread.
Hello from another thread!
This is my thread id: ThreadId(3)
Hello from another thread!
This is my thread id: ThreadId(2)
```

The only thing that still changes between runs is the order in which the messages are printed:

```
Hello from the main thread.
Hello from another thread!
Hello from another thread!
This is my thread id: ThreadId(2)
This is my thread id: ThreadId(3)
```

Output Locking

The `println` macro uses `std::io::Stdout::lock()` to make sure its output does not get interrupted. A `println!()` expression will wait until any concurrently running one is finished before writing any output. If this was not the case, we could've gotten more interleaved output such as:

```
Hello fromHello from another thread!
 another This is my threthreadHello fromthread id: ThreadId!
( the main thread.
2)This is my thread
id: ThreadId(3)
```

Rather than passing the name of a function to `std::thread::spawn`, as in our example above, it's far more common to pass it a *closure*. This allows us to capture values to move into the new thread:

```
let numbers = vec![1, 2, 3];

thread::spawn(move || {
    for n in &numbers {
        println!("{n}");
    }
```

```
}).join().unwrap();
```

Here, ownership of `numbers` is transferred to the newly spawned thread, since we used a `move` closure. If we had not used the `move` keyword, the closure would have captured `numbers` by reference. This would have resulted in a compiler error, since the new thread might outlive that variable.

Since a thread might run until the very end of the program's execution, the `spawn` function has a `'static` lifetime bound on its argument type. In other words, it only accepts functions that may be kept around forever. A closure capturing a local variable by reference may not be kept around forever, since that reference would become invalid the moment the local variable ceases to exist.

Getting a value back out of the thread is done by returning it from the closure. This return value can be obtained from the `Result` returned by the `join` method:

```
let numbers = Vec::from_iter(0..=1000);

let t = thread::spawn(move || {
    let len = numbers.len();
    let sum = numbers.iter().sum::<usize>();
    sum / len  // (1)
});

let average = t.join().unwrap(); // (2)

println!("average: {average}");
```

Here, the value returned by the thread's closure (1) is sent back to the main thread through the `join` method (2).

If `numbers` had been empty, the thread would've panicked while trying to divide by zero (1), and `join` would've returned that panic message instead, causing the main thread to panic too because of `unwrap` (2).

Thread Builder

The `std::thread::spawn` function is actually just a convenient shorthand for `std::thread::Builder::new().spa`

A `std::thread::Builder` allows you to set some settings for the new thread before spawning it. You can use it to configure the stack size for the new thread and to give the new thread a name. The name of a thread is available through `std::thread::current().name()`, will be used in panic messages, and will be visible in monitoring and debugging tools on most platforms.

Additionally, `Builder`'s `spawn` function returns an `std::io::Result`, allowing you to handle situations where spawning a new thread fails. This might happen if the operating system runs out of memory, or if resource limits have been applied to your program. The `std::thread::spawn` function simply panics if it is unable to spawn a new thread.

Scoped Threads

If we know for sure that a spawned thread will definitely not outlive a certain scope, that thread could safely borrow things that do not live forever, such as local variables, as long as they outlive that scope.

The Rust standard library provides the `std::thread::scope` function to spawn such *scoped threads*. It allows us to spawn threads that cannot outlive the scope of the closure we pass to that function, making it possible to safely borrow local variables.

How it works is best shown with an example:

```
let numbers = vec![1, 2, 3];

thread::scope(|s| { // (1)
    s.spawn(|| { // (2)
        println!("length: {}", numbers.len());
    });
    s.spawn(|| { // (2)
        for n in &numbers {
            println!("{n}");
        }
    });
}); // (3)
```

This pattern guarantees that none of the threads spawned in the scope can outlive the scope. Because of that, this scoped `spawn` method does not have a `'static` bound on its argument type, allowing us to reference anything as long as it outlives the scope, such as `numbers`.

In the example above, both of the new threads are concurrently accessing `numbers`. This is fine, because neither of them (nor the main thread) modifies it. If we were to change the first thread to modify `numbers`, as shown below, the compiler wouldn't allow us to spawn another thread that also uses `numbers`:

```
let mut numbers = vec![1, 2, 3];

thread::scope(|s| {
    s.spawn(|| {
        numbers.push(1);
    });
    s.spawn(|| {
        numbers.push(2); // Error!
    });
});
```

The exact error message depends on the version of the Rust compiler, since it's often improved to produce better diagnostics, but attempting to compile the code above will result in something like this:

```
error[E0499]: cannot borrow `numbers` as mutable more than once at a time
 --> example.rs:7:13
  |
4 |     s.spawn(|| {
  |             -- first mutable borrow occurs here
5 |         numbers.push(1);
  |         ------- first borrow occurs due to use of `numbers` in closure
  |
7 |     s.spawn(|| {
  |             ^^ second mutable borrow occurs here
8 |         numbers.push(2);
  |         ------- second borrow occurs due to use of `numbers` in closure
```

The Leakpocalypse

Before Rust 1.0, the standard library had a function named `std::thread::scoped` that would directly spawn a thread, just like `std::thread::spawn`. It allowed non-`'static` captures, because instead of a `JoinHandle`, it returned a `JoinGuard` which joined the thread when dropped. Any borrowed data only needed to outlive this `JoinGuard`. This seemed safe, as long as the `JoinGuard` got dropped at some point.

Just before the release of Rust 1.0, it slowly became clear that it's not possible to guarantee that something will be dropped. There are many ways, such as creating a cycle of reference-counted nodes, that make it possible to forget about something, or *leak* it, without dropping it.

Eventually, in what some people refer to as "The Leakpocalypse," the conclusion was made that the design of a (safe) interface cannot rely on the assumption that objects will always be dropped at the end of their lifetime. Leaking an object might reasonably result in leaking more objects (e.g., leaking a `Vec` will also leak its elements), but it may not result in undefined behavior. Because of this conclusion, `std::thread::scoped` was no longer deemed safe and was removed from the standard library. Additionally, `std::mem::forget` was upgraded from an `unsafe` function to a *safe* function, to emphasize that forgetting (or leaking) is always a possibility.

Only much later, in Rust 1.63, a new `std::thread::scope` function was added with a new design that does not rely on `Drop` for correctness.

Shared Ownership and Reference Counting

So far we've looked at transferring ownership of a value to a thread using a `move` closure ("Threads in Rust") and borrowing data from longer-living parent threads ("Scoped Threads"). When sharing data between two threads where neither thread is guaranteed to outlive the other, neither of them can be the owner of that data. Any data shared between them will need to live as long as the longest living thread.

Statics

There are several ways to create something that's not owned by a single thread. The simplest one is a `static` value, which is "owned" by the entire program, instead of an individual thread. In the following example, both threads can access `X`, but neither of them owns it:

```
static X: [i32; 3] = [1, 2, 3];

thread::spawn(|| dbg!(&X));
thread::spawn(|| dbg!(&X));
```

A `static` item has a constant initializer, is never dropped, and already exists before the main function of the program even starts. Every thread can borrow it, since it's guaranteed to always exist.

Leaking

Another way to share ownership is by *leaking* an allocation. Using `Box::leak`, one can release ownership of a `Box`, promising to never drop it. From that point on, the `Box` will live forever, without an owner, allowing it to be borrowed by any thread for as long as the program runs.

```
let x: &'static [i32; 3] = Box::leak(Box::new([1, 2, 3]));

thread::spawn(move || dbg!(x));
thread::spawn(move || dbg!(x));
```

The `move` closure might make it look like we're moving ownership into the threads, but a closer look at the type of `x` reveals that we're only giving the threads a *reference* to the data.

> **Note**

References are `Copy`, meaning that when you "move" them, the original still exists, just like with an integer or boolean.

Note how the `'static` lifetime doesn't mean that the value lived since the start of the program, but only that it lives to the end of the program. The past is simply not relevant.

The downside of leaking a `Box` is that we're *leaking memory*. We allocate something, but never drop and deallocate it. This can be fine if it happens only a limited number of times. But if we keep doing this, the program will slowly run out of memory.

Reference Counting

To make sure that shared data gets dropped and deallocated, we can't completely give up its ownership. Instead, we can *share ownership*. By keeping track of the number of owners, we can make sure the value is dropped only when there are no owners left.

The Rust standard library provides this functionality through the `std::rc::Rc` type, short for "reference counted." It is very similar to a `Box`, except cloning it will not allocate anything new, but instead increment a counter stored next to the contained value. Both the original and cloned `Rc` will refer to the same allocation; they *share ownership*.

```
use std::rc::Rc;

let a = Rc::new([1, 2, 3]);
let b = a.clone();

assert_eq!(a.as_ptr(), b.as_ptr()); // Same allocation!
```

Dropping an `Rc` will decrement the counter. Only the last `Rc`, which will see the counter drop to zero, will be the one dropping and deallocating the contained data.

If we were to try to send an `Rc` to another thread, however, we would run into the following compiler error:

```
error[E0277]: `Rc` cannot be sent between threads safely
    |
8   |     thread::spawn(move || dbg!(b));
    |                   ^^^^^^^^^^^^^^
```

As it turns out, `Rc` is not *thread safe* (more on that in "Thread Safety: Send and Sync"). If multiple threads had an `Rc` to the same allocation, they might try to modify the reference counter at the same time, which can give unpredictable results.

Instead, we can use `std::sync::Arc`, which stands for "atomically reference counted." It's identical to `Rc`, except it guarantees that modifications to the reference counter are indivisible *atomic* operations, making it safe to use it with multiple threads. (More on that in Chapter 2.)

```
use std::sync::Arc;

let a = Arc::new([1, 2, 3]); // (1)
let b = a.clone(); // (2)

thread::spawn(move || dbg!(a)); // (3)
thread::spawn(move || dbg!(b)); // (3)
```

Naming Clones

Having to give every clone of an `Arc` a different name can quickly make the code quite cluttered and hard to follow. While every clone of an `Arc` is a separate object, each clone represents the same shared value, which is not well reflected by naming each one differently.

Rust allows (and encourages) you to *shadow* variables by defining a new variable with the same name. If you do that in the same scope, the original variable cannot be named anymore. But by

opening a new scope, a statement like `let a = a.clone();` can be used to reuse the same name within that scope, while leaving the original variable available outside the scope.

By wrapping a closure in a new scope (with `{}`), we can clone variables before moving them into the closure, without having to rename them.

The clone of the Arc lives in the same scope. Each thread gets its own clone with a different name.

The clone of the Arc lives in a different scope. We can use the same name in each thread.

Because ownership is shared, reference counting pointers (`Rc<T>` and `Arc<T>`) have the same restrictions as shared references (`&T`). They do not give you mutable access to their contained value, since the value might be borrowed by other code at the same time.

For example, if we were to try to sort the slice of integers in an `Arc<[i32]>`, the compiler would stop us from doing so, telling us that we're not allowed to mutate the data:

```
error[E0596]: cannot borrow data in an `Arc` as mutable
  |
6 |     a.sort();
  |     ^^^^^^^^
```

Borrowing and Data Races

In Rust, values can be borrowed in two ways:

Immutable borrowing
Borrowing something with `&` gives an *immutable reference*. Such a reference can be copied. Access to the data it references is shared between all copies of such a reference. As the name implies, the compiler doesn't normally allow you to *mutate* something through such a reference, since that might affect other code that's currently borrowing the same data.

Mutable borrowing
Borrowing something with `&mut` gives a *mutable reference*. A mutable borrow guarantees it's the only active borrow of that data. This ensures that mutating the data will not change anything that other code is currently looking at.

These two concepts together fully prevent *data races*: situations where one thread is mutating data while another is concurrently accessing it. Data races are generally *undefined behavior*, which means the compiler does not need to take these situations into account. It will simply assume they do not happen.

To clarify what that means, let's take a look at an example where the compiler can make a useful assumption using the borrowing rules:

```
fn f(a: &i32, b: &mut i32) {
    let before = *a;
    *b += 1;
    let after = *a;
    if before != after {
```

```
            x(); // never happens
        }
    }
```

Here, we get an immutable reference to an integer, and store the value of the integer both before and after incrementing the integer that `b` refers to. The compiler is free to assume that the fundamental rules about borrowing and data races are upheld, which means that `b` can't possibly refer to the same integer as `a` does. In fact, nothing in the entire program can mutably borrow the integer that `a` refers to as long as `a` is borrowing it. Therefore, the compiler can easily conclude that `*a` will not change and the condition of the `if` statement will never be true, and can completely remove the call to `x` from the program as an optimization.

It's impossible to write a Rust program that breaks the compiler's assumptions, other than by using an `unsafe` block to disable some of the compiler's safety checks.

Undefined Behavior

Languages like C, C++, and Rust have a set of rules that need to be followed to avoid something called *undefined behavior*. For example, one of Rust's rules is that there may never be more than one mutable reference to any object.

In Rust, it's only possible to break any of these rules when using `unsafe` code. "Unsafe" doesn't mean that the code is incorrect or never safe to use, but rather that the compiler is not validating for you that the code is safe. If the code does violate these rules, it is called *unsound*.

The compiler is allowed to assume, without checking, that these rules are never broken. When broken, this results in something called *undefined behavior*, which we need to avoid at all costs. If we allow the compiler to make an assumption that is not actually true, it can easily result in more wrong conclusions about different parts of your code, affecting your whole program.

As a concrete example, let's take a look at a small snippet that uses the `get_unchecked` method on a slice:

```
let a = [123, 456, 789];
let b = unsafe { a.get_unchecked(index) };
```

The `get_unchecked` method gives us an element of the slice given its index, just like `a[index]`, but allows the compiler to assume the index is always within bounds, without any checks.

This means that in this code snippet, because `a` is of length 3, the compiler may assume that `index` is less than three. It's up to us to make sure its assumption holds.

If we break this assumption, for example if we run this with `index` equal to 3, anything might happen. It might result in reading from memory whatever was stored in the bytes right after `a`. It might cause the program to crash. It might end up executing some entirely unrelated part of the program. It can cause all kinds of havoc.

Perhaps surprisingly, undefined behavior can even "travel back in time," causing problems in code that precedes it. To understand how that can happen, imagine we had a `match` statement before our previous snippet, as follows:

```
match index {
    0 => x(),
    1 => y(),
    _ => z(index),
}

let a = [123, 456, 789];
let b = unsafe { a.get_unchecked(index) };
```

Because of the unsafe code, the compiler is allowed to assume `index` is only ever 0, 1, or 2. It may logically conclude that the last arm of our `match` statement will only ever match a 2, and thus that `z` is only ever called as `z(2)`. That conclusion might be used not only to optimize the `match`, but also to optimize `z` itself. This can include throwing out unused parts of the code.

If we execute this with an `index` of 3, our program might attempt to execute parts that have been optimized away, resulting in completely unpredictable behavior, long before we get to the `unsafe` block on the last line. Just like that, undefined behavior can propagate through a whole program, both backwards and forwards, in often very unexpected ways.

When calling any `unsafe` function, read its documentation carefully and make sure you fully understand its *safety requirements*: the assumptions you need to uphold, as the caller, to avoid undefined behavior.

Interior Mutability

The borrowing rules as introduced in the previous section are simple, but can be quite limiting—especially when multiple threads are involved. Following these rules makes communication between threads extremely limited and almost impossible, since no data that's accessible by multiple threads can be mutated.

Luckily, there is an escape hatch: *interior mutability*. A data type with interior mutability slightly bends the borrowing rules. Under certain conditions, those types can allow mutation through an "immutable" reference.

In "Reference Counting", we've already seen one subtle example involving interior mutability. Both `Rc` and `Arc` mutate a reference counter, even though there might be multiple clones all using the same reference counter.

As soon as interior mutable types are involved, calling a reference "immutable" or "mutable" becomes confusing and inaccurate, since some things can be mutated through both. The more accurate terms are "shared" and "exclusive": a *shared reference* (`&T`) can be copied and shared with others, while an *exclusive reference* (`&mut T`) guarantees it's the only *exclusive borrowing* of that `T`. For most types, shared references do not allow mutation, but there are exceptions. Since in this book we will mostly be working with these exceptions, we'll use the more accurate terms in the rest of this book.

> **Note**

Keep in mind that interior mutability only bends the rules of shared borrowing to allow mutation when shared. It does not change anything about exclusive borrowing. Exclusive borrowing still guarantees that there are no other active borrows. Unsafe code that results in more than one active exclusive reference to something always invokes undefined behavior, regardless of interior mutability.

Let's take a look at a few types with interior mutability and how they can allow mutation through shared references without causing undefined behavior.

Cell

A `std::cell::Cell<T>` simply wraps a `T`, but allows mutations through a shared reference. To avoid undefined behavior, it only allows you to copy the value out (if `T` is `Copy`), or replace it with another value as a whole. In addition, it can only be used within a single thread.

Let's take a look at an example similar to the one in the previous section, but this time using `Cell<i32>` instead of `i32`:

```
use std::cell::Cell;

fn f(a: &Cell<i32>, b: &Cell<i32>) {
    let before = a.get();
    b.set(b.get() + 1);
    let after = a.get();
    if before != after {
        x(); // might happen
    }
}
```

Unlike last time, it is now possible for the `if` condition to be true. Because a `Cell<i32>` has interior mutability, the compiler can no longer assume its value won't change as long as we have a shared reference to it. Both `a` and `b` might refer to the same value, such that mutating through `b` might affect `a` as well. It may still assume, however, that no other threads are accessing the cells concurrently.

The restrictions on a `Cell` are not always easy to work with. Since it can't directly let us borrow the value it holds, we need to move a value out (leaving something in its place), modify it, then put it back, to mutate its contents:

```
fn f(v: &Cell<Vec<i32>>) {
    let mut v2 = v.take(); // Replaces the contents of the Cell with an empty Vec
    v2.push(1);
    v.set(v2); // Put the modified Vec back
}
```

RefCell

Unlike a regular `Cell`, a `std::cell::RefCell` does allow you to borrow its contents, at a small runtime cost. A `RefCell<T>` does not only hold a `T`, but also holds a counter that keeps track of any outstanding borrows. If you try to borrow it while it is already mutably borrowed (or vice-versa), it will panic, which avoids undefined behavior. Just like a `Cell`, a `RefCell` can only be used within a single thread.

Borrowing the contents of `RefCell` is done by calling `borrow` or `borrow_mut`:

```
use std::cell::RefCell;

fn f(v: &RefCell<Vec<i32>>) {
    v.borrow_mut().push(1); // We can modify the `Vec` directly.
}
```

While `Cell` and `RefCell` can be very useful, they become rather useless when we need to do something with multiple threads. So let's move on to the types that are relevant for concurrency.

Mutex and RwLock

An `RwLock` or *reader-writer lock* is the concurrent version of a `RefCell`. An `RwLock<T>` holds a `T` and tracks any outstanding borrows. However, unlike a `RefCell`, it does not panic on conflicting borrows. Instead, it blocks the current thread—putting it to sleep—while waiting for conflicting borrows to disappear. We'll just have to patiently wait for our turn with the data, after the other threads are done with it.

Borrowing the contents of an `RwLock` is called *locking*. By *locking* it we temporarily block concurrent conflicting borrows, allowing us to borrow it without causing data races.

A `Mutex` is very similar, but conceptually slightly simpler. Instead of keeping track of the number of shared and exclusive borrows like an `RwLock`, it only allows exclusive borrows.

We'll go more into detail on these types in "Locking: Mutexes and RwLocks".

Atomics

The atomic types represent the concurrent version of a `Cell`, and are the main topic of Chapters 2 and 3. Like a `Cell`, they avoid undefined behavior by making us copy values in and out as a whole, without letting us borrow the contents directly.

Unlike a `Cell`, though, they cannot be of arbitrary size. Because of this, there is no generic `Atomic<T>` type for any `T`, but there are only specific atomic types such as `AtomicU32` and `AtomicPtr<T>`. Which ones are available depends on the platform, since they require support from the processor to avoid data races. (We'll dive into that in Chapter 7.)

Since they are so limited in size, atomics often don't directly contain the information that needs to be shared between threads. Instead, they are often used as a tool to make it possible to share other—often bigger—things between threads. When atomics are used to say something about other data, things can get surprisingly complicated.

UnsafeCell

An `UnsafeCell` is the primitive building block for interior mutability.

An `UnsafeCell<T>` wraps a `T`, but does not come with any conditions or restrictions to avoid undefined behavior. Instead, its `get()` method just gives a raw pointer to the value it wraps, which can only be meaningfully used in `unsafe` blocks. It leaves it up to the user to use it in a way that does not cause any undefined behavior.

Most commonly, an `UnsafeCell` is not used directly, but wrapped in another type that provides safety through a limited interface, such as `Cell` or `Mutex`. All types with interior mutability—including all types discussed above—are built on top of `UnsafeCell`.

Thread Safety: Send and Sync

In this chapter, we've seen several types that are not *thread safe*, types that can only be used on a single thread, such as `Rc`, `Cell`, and others. Since that restriction is needed to avoid undefined behavior, it's something the compiler needs to understand and check for you, so you can use these types without having to use `unsafe` blocks.

The language uses two special traits to keep track of which types can be safely used across threads:

Send
A type is `Send` if it can be sent to another thread. In other words, if ownership of a value of that type can be transferred to another thread. For example, `Arc<i32>` is `Send`, but `Rc<i32>` is not.

Sync
A type is `Sync` if it can be shared with another thread. In other words, a type `T` is `Sync` if and only if a shared reference to that type, `&T`, is `Send`. For example, an `i32` is `Sync`, but a `Cell<i32>` is not. (A `Cell<i32>` is `Send`, however.)

All primitive types such as `i32`, `bool`, and `str` are both `Send` and `Sync`.

Both of these traits are *auto traits*, which means that they are automatically implemented for your types based on their fields. A `struct` with fields that are all `Send` and `Sync` is itself also `Send` and `Sync`.

The way to opt out of either of these is to add a field to your type that does not implement the trait. For that purpose, the special `std::marker::PhantomData<T>` type often comes in handy. That type is treated by the compiler as a `T`, except it doesn't actually exist at runtime. It's a zero-sized type, taking no space.

Let's take a look at the following `struct`:

```
use std::marker::PhantomData;

struct X {
    handle: i32,
    _not_sync: PhantomData<Cell<()>>,
}
```

In this example, `X` would be both `Send` and `Sync` if `handle` was its only field. However, we added a zero-sized `PhantomData<Cell<()>>` field, which is treated as if it were a `Cell<()>`. Since a `Cell<()>` is not `Sync`, neither is `X`. It is still `Send`, however, since all its fields implement `Send`.

Raw pointers (`*const T` and `*mut T`) are neither `Send` nor `Sync`, since the compiler doesn't know much about what they represent.

The way to opt in to either of the traits is the same as with any other trait; use an `impl` block to implement the trait for your type:

```
struct X {
    p: *mut i32,
}

unsafe impl Send for X {}
unsafe impl Sync for X {}
```

Note how implementing these traits requires the `unsafe` keyword, since the compiler cannot check for you if it's correct. It's a promise you make to the compiler, which it will just have to trust.

If you try to move something into another thread which is not `Send`, the compiler will politely stop you from doing that. Here is a small example to demonstrate that:

```
fn main() {
    let a = Rc::new(123);
    thread::spawn(move || { // Error!
        dbg!(a);
    });
}
```

Here, we try to send an `Rc<i32>` to a new thread, but `Rc<i32>`, unlike `Arc<i32>`, does not implement `Send`.

If we try to compile the example above, we're faced with an error that looks something like this:

```
error[E0277]: `Rc<i32>` cannot be sent between threads safely
   --> src/main.rs:3:5
    |
3   |     thread::spawn(move || {
    |     ^^^^^^^^^^^^^ `Rc<i32>` cannot be sent between threads safely
    |
    = help: within `[closure]`, the trait `Send` is not implemented for `Rc<i32>`
note: required because it's used within this closure
   --> src/main.rs:3:19
    |
3   |     thread::spawn(move || {
    |                   ^^^^^^^
note: required by a bound in `spawn`
```

The `thread::spawn` function requires its argument to be `Send`, and a closure is only `Send` if all of its captures are. If we try to capture something that's not `Send`, our mistake is caught, protecting us from undefined behavior.

Locking: Mutexes and RwLocks

The most commonly used tool for sharing (mutable) data between threads is a *mutex*, which is short for "mutual exclusion." The job of a mutex is to ensure threads have exclusive access to some data by temporarily blocking other threads that try to access it at the same time.

Conceptually, a mutex has only two states: locked and unlocked. When a thread locks an unlocked mutex, the mutex is marked as locked and the thread can immediately continue. When a thread then attempts to lock an already locked mutex, that operation will *block*. The thread is put to sleep while it waits for the mutex to be unlocked. Unlocking is only possible on a locked mutex, and should be done by the same thread that locked it. If other threads are waiting to lock the mutex, unlocking will cause one of those threads to be woken up, so it can try to lock the mutex again and continue its course.

Protecting data with a mutex is simply the agreement between all threads that they will only access the data when they have the mutex locked. That way, no two threads can ever access that data concurrently and cause a data race.

Rust's Mutex

The Rust standard library provides this functionality through `std::sync::Mutex<T>`. It is generic over a type `T`, which is the type of the data the mutex is protecting. By making this `T` part of the mutex, the data can only be accessed through the mutex, allowing for a safe interface that can guarantee all threads will uphold the agreement.

To ensure a locked mutex can only be unlocked by the thread that locked it, it does not have an `unlock()` method. Instead, its `lock()` method returns a special type called a `MutexGuard`. This guard represents the guarantee that we have locked the mutex. It behaves like an exclusive reference through the `DerefMut` trait, giving us exclusive access to the data the mutex protects. Unlocking the mutex is done by dropping the guard. When we drop the guard, we give up our ability to access the data, and the `Drop` implementation of the guard will unlock the mutex.

Let's take a look at an example to see a mutex in practice:

```
use std::sync::Mutex;

fn main() {
    let n = Mutex::new(0);
    thread::scope(|s| {
        for _ in 0..10 {
            s.spawn(|| {
                let mut guard = n.lock().unwrap();
                for _ in 0..100 {
```

```
                    *guard += 1;
                }
            });
        }
    });
    assert_eq!(n.into_inner().unwrap(), 1000);
}
```

Here, we have a `Mutex<i32>`, a mutex protecting an integer, and we spawn ten threads to each increment the integer one hundred times. Each thread will first lock the mutex to obtain a `MutexGuard`, and then use that `guard` to access the integer and modify it. The `guard` is implicitly dropped right after, when that variable goes out of scope.

After the threads are done, we can safely remove the protection from the integer through `into_inner()`. The `into_inner` method takes ownership of the mutex, which guarantees that nothing else can have a reference to the mutex anymore, making locking unnecessary.

Even though the increments happen in steps of one, a thread observing the integer would only ever see multiples of 100, since it can only look at the integer when the mutex is unlocked. Effectively, thanks to the mutex, the one hundred increments together are now a single indivisible—atomic—operation.

To clearly see the effect of the mutex, we can make each thread wait a second before unlocking the mutex:

```
use std::time::Duration;

fn main() {
    let n = Mutex::new(0);
    thread::scope(|s| {
        for _ in 0..10 {
            s.spawn(|| {
                let mut guard = n.lock().unwrap();
                for _ in 0..100 {
                    *guard += 1;
                }
                thread::sleep(Duration::from_secs(1)); // New!
            });
        }
    });
    assert_eq!(n.into_inner().unwrap(), 1000);
}
```

When you run the program now, you will see that it takes about 10 seconds to complete. Each thread only waits for one second, but the mutex ensures that only one thread at a time can do so.

If we drop the guard—and therefore unlock the mutex—before sleeping one second, we will see it happen in parallel instead:

```
fn main() {
    let n = Mutex::new(0);
    thread::scope(|s| {
        for _ in 0..10 {
            s.spawn(|| {
                let mut guard = n.lock().unwrap();
                for _ in 0..100 {
                    *guard += 1;
                }
                drop(guard); // New: drop the guard before sleeping!
                thread::sleep(Duration::from_secs(1));
            });
        }
    });
    assert_eq!(n.into_inner().unwrap(), 1000);
}
```

With this change, this program takes only about one second, since now the 10 threads can execute their one-second sleep at the same time. This shows the importance of keeping the amount of time a mutex is locked as short as possible. Keeping a mutex locked longer than necessary can completely nullify any benefits of parallelism, effectively forcing everything to happen serially instead.

Lock Poisoning

The `unwrap()` calls in the examples above relate to *lock poisoning*.

A `Mutex` in Rust gets marked as *poisoned* when a thread panics while holding the lock. When that happens, the `Mutex` will no longer be locked, but calling its `lock` method will result in an `Err` to indicate it has been poisoned.

This is a mechanism to protect against leaving the data that's protected by a mutex in an inconsistent state. In our example above, if a thread would panic after incrementing the integer fewer than 100 times, the mutex would unlock and the integer would be left in an unexpected state where it is no longer a multiple of 100, possibly breaking assumptions made by other threads. Automatically marking the mutex as poisoned in that case forces the user to handle this possibility.

Calling `lock()` on a poisoned mutex still locks the mutex. The `Err` returned by `lock()` contains the `MutexGuard`, allowing us to correct an inconsistent state if necessary.

While lock poisoning might seem like a powerful mechanism, recovering from a potentially inconsistent state is not often done in practice. Most code either disregards poison or uses `unwrap()` to panic if the lock was poisoned, effectively propagating panics to all users of the mutex.

Lifetime of the MutexGuard

While it's convenient that implicitly dropping a guard unlocks the mutex, it can sometimes lead to subtle surprises. If we assign the guard a name with a `let` statement (as in our examples above), it's relatively straightforward to see when it will be dropped, since local variables are dropped at

the end of the scope they are defined in. Still, not explicitly dropping a guard might lead to keeping the mutex locked for longer than necessary, as demonstrated in the examples above.

Using a guard *without* assigning it a name is also possible, and can be very convenient at times. Since a `MutexGuard` behaves like an exclusive reference to the protected data, we can directly use it without assigning a name to the guard first. For example, if you have a `Mutex<Vec<i32>>`, you can lock the mutex, push an item into the `Vec`, and unlock the mutex again, in a single statement:

```
list.lock().unwrap().push(1);
```

Any temporaries produced within a larger expression, such as the guard returned by `lock()`, will be dropped at the end of the statement. While this might seem obvious and reasonable, it leads to a common pitfall that usually involves a `match`, `if let`, or `while let` statement. Here is an example that runs into this pitfall:

```
if let Some(item) = list.lock().unwrap().pop() {
    process_item(item);
}
```

If our intention was to lock the list, pop an item, unlock the list, and *then* process the item after the list is unlocked, we made a subtle but important mistake here. The temporary guard is not dropped until the end of the entire `if let` statement, meaning we needlessly hold on to the lock while processing the item.

Perhaps surprisingly, this does not happen for a similar `if` statement, such as in this example:

```
if list.lock().unwrap().pop() == Some(1) {
    do_something();
}
```

Here, the temporary guard does get dropped before the body of the `if` statement is executed. The reason is that the condition of a regular `if` statement is always a plain boolean, which cannot borrow anything. There is no reason to extend the lifetime of temporaries from the condition to the end of the statement. For an `if let` statement, however, that might not be the case. If we had used `front()` rather than `pop()`, for example, `item` would be borrowing from the list, making it necessary to keep the guard around. Since the borrow checker is only really a check and does *not* influence when or in what order things are dropped, the same happens when we use `pop()`, even though that wouldn't have been necessary.

We can avoid this by moving the pop operation to a separate `let` statement. Then the `guard` is dropped at the end of that statement, before the `if let`:

```
let item = list.lock().unwrap().pop();
if let Some(item) = item {
    process_item(item);
}
```

Reader-Writer Lock

A mutex is only concerned with exclusive access. The `MutexGuard` will provide us an exclusive reference (`&mut T`) to the protected data, even if we only wanted to look at the data and a shared reference (`&T`) would have sufficed.

A reader-writer lock is a slightly more complicated version of a mutex that understands the difference between exclusive and shared access, and can provide either. It has three states: unlocked, locked by a single *writer* (for exclusive access), and locked by any number of *readers* (for shared access). It is commonly used for data that is often read by multiple threads, but only updated once in a while.

The Rust standard library provides this lock through the `std::sync::RwLock<T>` type. It works similarly to the standard `Mutex`, except its interface is mostly split in two parts. Instead of a single `lock()` method, it has `read()` and `write()` methods for locking as either a reader or a writer. It comes with two guard types, one for readers and one for writers: `RwLockReadGuard` and `RwLockWriteGuard`. The former only implements `Deref` to behave like a shared reference to the protected data, while the latter also implements `DerefMut` to behave like an exclusive reference.

It is effectively the multi-threaded version of `RefCell`, dynamically tracking the number of references to ensure the borrow rules are upheld.

Both `Mutex<T>` and `RwLock<T>` require `T` to be `Send`, because they can be used to send a `T` to another thread. An `RwLock<T>` additionally requires `T` to also implement `Sync`, because it allows multiple threads to hold a shared reference (`&T`) to the protected data. (Strictly speaking, you can create a lock for a `T` that doesn't fulfill these requirements, but you wouldn't be able to share it between threads as the lock itself won't implement `Sync`.)

The Rust standard library provides only one general purpose `RwLock` type, but its implementation depends on the operating system. There are many subtle variations between reader-writer lock implementations. Most implementations will block new readers when there is a writer waiting, even when the lock is already read-locked. This is done to prevent *writer starvation*, a situation where many readers collectively keep the lock from ever unlocking, never allowing any writer to update the data.

Mutexes in Other Languages

Rust's standard `Mutex` and `RwLock` types look a bit different than those you find in other languages like C or C++.

The biggest difference is that Rust's `Mutex<T>` *contains* the data it is protecting. In C++, for example, `std::mutex` does not contain the data it protects, nor does it even know what it is protecting. This means that it is the responsibility of the user to remember which data is protected and by which mutex, and ensure the right mutex is locked every time "protected" data is accessed. This is useful to keep in mind when reading code involving mutexes in other languages, or when communicating with programmers who are not familiar with Rust. A Rust programmer might talk about "the data inside the mutex," or say things like "wrap it in a mutex," which can be confusing to those only familiar with mutexes in other languages.

If you really need a stand-alone mutex that doesn't contain anything, for example to protect some external hardware, you can use `Mutex<()>`. But even in a case like that, you are probably better

off defining a (possibly zero-sized) type to interface with that hardware and wrapping that in a `Mutex` instead. That way, you are still forced to lock the mutex before you can interact with the hardware.

Waiting: Parking and Condition Variables

When data is mutated by multiple threads, there are many situations where they would need to wait for some event, for some condition about the data to become true. For example, if we have a mutex protecting a `Vec`, we might want to wait until it contains something.

While a mutex does allow threads to wait until it becomes unlocked, it does not provide functionality for waiting for any other conditions. If a mutex was all we had, we'd have to keep locking the mutex to repeatedly check if there's anything in the `Vec` yet.

Thread Parking

One way to wait for a notification from another thread is called *thread parking*. A thread can *park* itself, which puts it to sleep, stopping it from consuming any CPU cycles. Another thread can then *unpark* the parked thread, waking it up from its nap.

Thread parking is available through the `std::thread::park()` function. For unparking, you call the `unpark()` method on a `Thread` object representing the thread that you want to unpark. Such an object can be obtained from the join handle returned by `spawn`, or by the thread itself through `std::thread::current()`.

Let's dive into an example that uses a mutex to share a queue between two threads. In the following example, a newly spawned thread will consume items from the queue, while the main thread will insert a new item into the queue every second. Thread parking is used to make the consuming thread wait when the queue is empty.

```
use std::collections::VecDeque;

fn main() {
    let queue = Mutex::new(VecDeque::new());

    thread::scope(|s| {
        // Consuming thread
        let t = s.spawn(|| loop {
            let item = queue.lock().unwrap().pop_front();
            if let Some(item) = item {
                dbg!(item);
            } else {
                thread::park();
            }
        });

        // Producing thread
```

```
        for i in 0.. {
            queue.lock().unwrap().push_back(i);
            t.thread().unpark();
            thread::sleep(Duration::from_secs(1));
        }
    });
}
```

The consuming thread runs an infinite loop in which it pops items out of the queue to display them using the `dbg` macro. When the queue is empty, it stops and goes to sleep using the `park()` function. If it gets unparked, the `park()` call returns, and the `loop` continues, popping items from the queue again until it is empty. And so on.

The producing thread produces a new number every second by pushing it into the queue. Every time it adds an item, it uses the `unpark()` method on the `Thread` object that refers to the consuming thread to unpark it. That way, the consuming thread gets woken up to process the new element.

An important observation to make here is that this program would still be theoretically correct, although inefficient, if we remove parking. This is important, because `park()` does not guarantee that it will only return because of a matching `unpark()`. While somewhat rare, it might have *spurious wake-ups*. Our example deals with that just fine, because the consuming thread will lock the queue, see that it is empty, and directly unlock it and park itself again.

An important property of thread parking is that a call to `unpark()` *before* the thread parks itself does not get lost. The request to unpark is still recorded, and the next time the thread tries to park itself, it clears that request and directly continues without actually going to sleep. To see why that is critical for correct operation, let's go through a possible ordering of the steps executed by both threads:

1. The consuming thread—let's call it C—locks the queue.
2. C tries to pop an item from the queue, but it is empty, resulting in `None`.
3. C unlocks the queue.
4. The producing thread, which we'll call P, locks the queue.
5. P pushes a new item onto the queue.
6. P unlocks the queue again.
7. P calls `unpark()` to notify C that there are new items.
8. C calls `park()` to go to sleep, to wait for more items.

While there is most likely only a very brief moment between releasing the queue in step 3 and parking in step 8, steps 4 through 7 could potentially happen in that moment before the thread parks itself. If `unpark()` would do nothing if the thread wasn't parked, the notification would be lost. The consuming thread would still be waiting, even if there were an item in the queue. Thanks to unpark requests getting saved for a future call to `park()`, we don't have to worry about this.

However, unpark requests don't stack up. Calling `unpark()` two times and then calling `park()` two times afterwards still results in the thread going to sleep. The first `park()` clears the request and returns directly, but the second one goes to sleep as usual.

This means that in our example above it's important that we only park the thread if we've seen the queue is empty, rather than park it after every processed item. While it's extremely unlikely to happen in this example because of the huge (one second) sleep, it's possible for multiple `unpark()` calls to wake up only a single `park()` call.

Unfortunately, this does mean that if `unpark()` is called right after `park()` returns, but before the queue gets locked and emptied out, the `unpark()` call was unnecessary but still causes the next `park()` call to instantly return. This results in the (empty) queue getting locked and unlocked an extra time. While this doesn't affect the correctness of the program, it does affect its efficiency and performance.

This mechanism works well for simple situations like in our example, but quickly breaks down when things get more complicated. For example, if we had multiple consumer threads taking items from the same queue, the producer thread would have no way of knowing which of the consumers is actually waiting and should be woken up. The producer will have to know exactly when a consumer is waiting, and what condition it is waiting for.

Condition Variables

Condition variables are a more commonly used option for waiting for something to happen to data protected by a mutex. They have two basic operations: *wait* and *notify*. Threads can wait on a condition variable, after which they can be woken up when another thread notifies that same condition variable. Multiple threads can wait on the same condition variable, and notifications can either be sent to one waiting thread, or to all of them.

This means that we can create a condition variable for specific events or conditions we're interested in, such as the queue being non-empty, and wait on that condition. Any thread that causes that event or condition to happen then notifies the condition variable, without having to know which or how many threads are interested in that notification.

To avoid the issue of missing notifications in the brief moment between unlocking a mutex and waiting for a condition variable, condition variables provide a way to *atomically* unlock the mutex and start waiting. This means there is simply no possible moment for notifications to get lost.

The Rust standard library provides a condition variable as `std::sync::Condvar`. Its `wait` method takes a `MutexGuard` that proves we've locked the mutex. It first unlocks the mutex and goes to sleep. Later, when woken up, it relocks the mutex and returns a new `MutexGuard` (which proves that the mutex is locked again).

It has two notify functions: `notify_one` to wake up just one waiting thread (if any), and `notify_all` to wake them all up.

Let's modify the example we used for thread parking to use `Condvar` instead:

```
use std::sync::Condvar;

let queue = Mutex::new(VecDeque::new());
let not_empty = Condvar::new();

thread::scope(|s| {
```

```
    s.spawn(|| {
        loop {
            let mut q = queue.lock().unwrap();
            let item = loop {
                if let Some(item) = q.pop_front() {
                    break item;
                } else {
                    q = not_empty.wait(q).unwrap();
                }
            };
            drop(q);
            dbg!(item);
        }
    });

    for i in 0.. {
        queue.lock().unwrap().push_back(i);
        not_empty.notify_one();
        thread::sleep(Duration::from_secs(1));
    }
});
```

We had to change a few things:

- We now not only have a `Mutex` containing the queue, but also a `Condvar` to communicate the "not empty" condition.
- We no longer need to know which thread to wake up, so we don't store the return value from `spawn` anymore. Instead, we notify the consumer through the condition variable with the `notify_one` method.
- Unlocking, waiting, and relocking are all done by the `wait` method. We had to restructure the control flow a bit to be able to pass the guard to the `wait` method, while still dropping it before processing an item.

Now we can spawn as many consuming threads as we like, and even spawn more later, without having to change anything. The condition variable takes care of delivering the notifications to whichever thread is interested.

If we had a more complicated system with threads that are interested in different conditions, we could define a `Condvar` for each condition. For example, we could define one to indicate the queue is non-empty and another one to indicate it is empty. Then each thread would wait for whichever condition is relevant to what they are doing.

Normally, a `Condvar` is only ever used together with a single `Mutex`. If two threads try to concurrently `wait` on a condition variable using two different mutexes, it might cause a panic.

A downside of a `Condvar` is that it only works when used together with a `Mutex`, but for most use cases that is perfectly fine, as that's exactly what's already used to protect the data anyway.

Both `thread::park()` and `Condvar::wait()` also have a variant with a time limit: `thread::park_timeout()` and `Condvar::wait_timeout()`. These take a `Duration` as an extra argument, which is the time after which it should give up waiting for a notification and unconditionally wake up.

Summary

- Multiple threads can run concurrently within the same program and can be spawned at any time.
- When the main thread ends, the entire program ends.
- Data races are undefined behavior, which is fully prevented (in safe code) by Rust's type system.
- Data that is `Send` can be sent to other threads, and data that is `Sync` can be shared between threads.
- Regular threads might run as long as the program does, and thus can only borrow `'static` data such as statics and leaked allocations.
- Reference counting (`Arc`) can be used to share ownership to make sure data lives as long as at least one thread is using it.
- Scoped threads are useful to limit the lifetime of a thread to allow it to borrow non-`'static` data, such as local variables.
- `&T` is a *shared* reference. `&mut T` is an *exclusive* reference. Regular types do not allow mutation through a shared reference.
- Some types have interior mutability, thanks to `UnsafeCell`, which allows for mutation through shared references.
- `Cell` and `RefCell` are the standard types for single-threaded interior mutability. Atomics, `Mutex`, and `RwLock` are their multi-threaded equivalents.
- `Cell` and atomics only allow replacing the value as a whole, while `RefCell`, `Mutex`, and `RwLock` allow you to mutate the value directly by dynamically enforcing access rules.
- Thread parking can be a convenient way to wait for some condition.
- When a condition is about data protected by a `Mutex`, using a `Condvar` is more convenient, and can be more efficient, than thread parking.

Complete Runnable Examples

The following complete, tested programs illustrate the primitives and patterns developed throughout this chapter.

Example: `ch1-01-hello.rs` (01 Hello)

```
use std::thread;

fn main() {
    thread::spawn(f);
    thread::spawn(f);

    println!("Hello from the main thread.");
}

fn f() {
    println!("Hello from another thread!");

    let id = thread::current().id();
    println!("This is my thread id: {id:?}");
}
```

Example: `ch1-02-hello-join.rs` (02 Hello Join)

```
use std::thread;

fn main() {
    let t1 = thread::spawn(f);
    let t2 = thread::spawn(f);

    println!("Hello from the main thread.");

    t1.join().unwrap();
    t2.join().unwrap();
}

fn f() {
    println!("Hello from another thread!");

    let id = thread::current().id();
    println!("This is my thread id: {id:?}");
}
```

Example: `ch1-03-spawn-closure.rs` (03 Spawn Closure)

```rust
use std::thread;

fn main() {
    let numbers = vec![1, 2, 3];

    thread::spawn(move || {
        for n in &numbers {
            println!("{n}");
        }
    }).join().unwrap();
}
```

Example: `ch1-04-scoped-threads.rs` (04 Scoped Threads)

```rust
use std::thread;

fn main() {
    let numbers = vec![1, 2, 3];

    thread::scope(|s| {
        s.spawn(|| {
            println!("length: {}", numbers.len());
        });
        s.spawn(|| {
            for n in &numbers {
                println!("{n}");
            }
        });
    });
}
```

Example: `ch1-05-rc.rs` (05 Rc)

```rust
use std::rc::Rc;

fn main() {
    let a = Rc::new([1, 2, 3]);
    let b = a.clone();

    assert_eq!(a.as_ptr(), b.as_ptr()); // Same allocation!
}
```

Example: `ch1-06-cell.rs` (06 Cell)

```
use std::cell::Cell;

fn f(v: &Cell<Vec<i32>>) {
    let mut v2 = v.take(); // Replaces the contents of the Cell with an empty Vec
    v2.push(1);
    v.set(v2); // Put the modified Vec back
}

fn main() {
    let v = Cell::new(vec![1, 2, 3]);
    f(&v);
    assert_eq!(v.into_inner(), vec![1, 2, 3, 1]);
}
```

Example: `ch1-07-refcell.rs` (07 Refcell)

```
use std::cell::RefCell;

fn f(v: &RefCell<Vec<i32>>) {
    v.borrow_mut().push(1); // We can modify the `Vec` directly.
}

fn main() {
    let v = RefCell::new(vec![1, 2, 3]);
    f(&v);
    assert_eq!(v.into_inner(), vec![1, 2, 3, 1]);
}
```

Example: `ch1-08-mutex.rs` (08 Mutex)

```
use std::sync::Mutex;
use std::thread;

fn main() {
    let n = Mutex::new(0);
    thread::scope(|s| {
        for _ in 0..10 {
            s.spawn(|| {
                let mut guard = n.lock().unwrap();
                for _ in 0..100 {
                    *guard += 1;
                }
```

```
            });
        }
    });
    assert_eq!(n.into_inner().unwrap(), 1000);
}
```

Example: `ch1-09-sleep-before-unlock.rs` **(09 Sleep Before Unlock)**

```
use std::sync::Mutex;
use std::thread;
use std::time::Duration;

fn main() {
    let n = Mutex::new(0);
    thread::scope(|s| {
        for _ in 0..10 {
            s.spawn(|| {
                let mut guard = n.lock().unwrap();
                for _ in 0..100 {
                    *guard += 1;
                }
                thread::sleep(Duration::from_secs(1)); // New!
            });
        }
    });
    assert_eq!(n.into_inner().unwrap(), 1000);
}
```

Example: `ch1-10-unlock-before-sleep.rs` **(10 Unlock Before Sleep)**

```
use std::sync::Mutex;
use std::thread;
use std::time::Duration;

fn main() {
    let n = Mutex::new(0);
    thread::scope(|s| {
        for _ in 0..10 {
            s.spawn(|| {
                let mut guard = n.lock().unwrap();
                for _ in 0..100 {
                    *guard += 1;
                }
                drop(guard); // New: drop the guard before sleeping!
```

```
                thread::sleep(Duration::from_secs(1));
            });
        }
    });
    assert_eq!(n.into_inner().unwrap(), 1000);
}
```

Example: `ch1-11-thread-parking.rs` **(11 Thread Parking)**

```
use std::collections::VecDeque;
use std::sync::Mutex;
use std::thread;
use std::time::Duration;

fn main() {
    let queue = Mutex::new(VecDeque::new());

    thread::scope(|s| {
        // Consuming thread
        let t = s.spawn(|| loop {
            let item = queue.lock().unwrap().pop_front();
            if let Some(item) = item {
                dbg!(item);
            } else {
                thread::park();
            }
        });

        // Producing thread
        for i in 0.. {
            queue.lock().unwrap().push_back(i);
            t.thread().unpark();
            thread::sleep(Duration::from_secs(1));
        }
    });
}
```

Example: `ch1-12-condvar.rs` **(12 Condvar)**

```
use std::collections::VecDeque;
use std::sync::Condvar;
use std::sync::Mutex;
use std::thread;
use std::time::Duration;
```

```
fn main() {
    let queue = Mutex::new(VecDeque::new());
    let not_empty = Condvar::new();

    thread::scope(|s| {
        s.spawn(|| {
            loop {
                let mut q = queue.lock().unwrap();
                let item = loop {
                    if let Some(item) = q.pop_front() {
                        break item;
                    } else {
                        q = not_empty.wait(q).unwrap();
                    }
                };
                drop(q);
                dbg!(item);
            }
        });

        for i in 0.. {
            queue.lock().unwrap().push_back(i);
            not_empty.notify_one();
            thread::sleep(Duration::from_secs(1));
        }
    });
}
```

Atomics

Atomics

The word *atomic* comes from the Greek word , meaning *indivisible*, something that cannot be cut into smaller pieces. In computer science, it is used to describe an operation that is indivisible: it is either fully completed, or it didn't happen yet.

As mentioned in "Borrowing and Data Races" in Chapter 1, multiple threads concurrently reading and modifying the same variable normally results in undefined behavior. However, atomic operations do allow for different threads to safely read and modify the same variable. Since such an operation is indivisible, it either happens completely before or completely after another operation, avoiding undefined behavior. Later, in Chapter 7, we'll see how this works at the hardware level.

Atomic operations are the main building block for anything involving multiple threads. All the other concurrency primitives, such as mutexes and condition variables, are implemented using atomic operations.

In Rust, atomic operations are available as methods on the standard atomic types that live in `std::sync::atomic`. They all have names starting with `Atomic`, such as `AtomicI32` or `AtomicUsize`. Which ones are available depends on the hardware architecture and sometimes operating system, but almost all platforms provide at least all atomic types up to the size of a pointer.

Unlike most types, they allow modification through a shared reference (e.g., `&AtomicU8`). This is possible thanks to interior mutability, as discussed in "Interior Mutability" in Chapter 1.

Each of the available atomic types has the same interface with methods for storing and loading, methods for atomic "fetch-and-modify" operations, and some more advanced "compare-and-exchange" methods. We'll discuss them in detail in the rest of this chapter.

But, before we can dive into the different atomic operations, we briefly need to touch upon a concept called *memory ordering*:

Every atomic operation takes an argument of type `std::sync::atomic::Ordering`, which determines what guarantees we get about the relative ordering of operations. The simplest variant with the fewest guarantees is `Relaxed`. `Relaxed` still guarantees consistency on a single atomic variable, but does not promise anything about the relative order of operations between different variables.

What this means is that two threads might see operations on different variables happen in a different order. For example, if one thread writes to one variable first and then to a second variable very quickly afterwards, another thread might see that happen in the opposite order.

In this chapter we'll only look at use cases where this is not a problem and simply use `Relaxed` everywhere without going into more detail. We'll discuss all the details of memory ordering and the other available memory orderings in Chapter 3.

Atomic Load and Store Operations

The first two atomic operations we'll look at are the most basic ones: `load` and `store`. Their function signatures are as follows, using `AtomicI32` as an example:

```
impl AtomicI32 {
    pub fn load(&self, ordering: Ordering) -> i32;
    pub fn store(&self, value: i32, ordering: Ordering);
}
```

The `load` method atomically loads the value stored in the atomic variable, and the `store` method atomically stores a new value in it. Note how the `store` method takes a shared reference (`&T`) rather than an exclusive reference (`&mut T`), even though it modifies the value.

Let's take a look at some realistic use cases for these two methods.

Example: Stop Flag

The first example uses an `AtomicBool` for a *stop flag*. Such a flag is used to inform other threads to stop running.

```
use std::sync::atomic::AtomicBool;
use std::sync::atomic::Ordering::Relaxed;

fn main() {
    static STOP: AtomicBool = AtomicBool::new(false);

    // Spawn a thread to do the work.
    let background_thread = thread::spawn(|| {
        while !STOP.load(Relaxed) {
            some_work();
        }
    });

    // Use the main thread to listen for user input.
    for line in std::io::stdin().lines() {
        match line.unwrap().as_str() {
            "help" => println!("commands: help, stop"),
            "stop" => break,
            cmd => println!("unknown command: {cmd:?}"),
        }
    }

    // Inform the background thread it needs to stop.
    STOP.store(true, Relaxed);

    // Wait until the background thread finishes.
```

```
    background_thread.join().unwrap();
}
```

In this example, the background thread is repeatedly running `some_work()`, while the main thread allows the user to enter some commands to interact with the program. In this simple example, the only useful command is `stop` to make the program stop.

To make the background thread stop, the atomic `STOP` boolean is used to communicate this condition to the background thread. When the foreground thread reads the `stop` command, it sets the flag to true, which is checked by the background thread before each new iteration. The main thread waits until the background thread is finished with its current iteration using the `join` method.

This simple solution works great as long as the flag is regularly checked by the background thread. If it gets stuck in `some_work()` for a long time, that can result in an unacceptable delay between the `stop` command and the program quitting.

Example: Progress Reporting

In our next example, we process 100 items one by one on a background thread, while the main thread gives the user regular updates on the progress:

```
use std::sync::atomic::AtomicUsize;

fn main() {
    let num_done = AtomicUsize::new(0);

    thread::scope(|s| {
        // A background thread to process all 100 items.
        s.spawn(|| {
            for i in 0..100 {
                process_item(i); // Assuming this takes some time.
                num_done.store(i + 1, Relaxed);
            }
        });

        // The main thread shows status updates, every second.
        loop {
            let n = num_done.load(Relaxed);
            if n == 100 { break; }
            println!("Working.. {n}/100 done");
            thread::sleep(Duration::from_secs(1));
        }
    });

    println!("Done!");
}
```

This time, we use a scoped thread ("Scoped Threads" in Chapter 1), which will automatically handle the joining of the thread for us, and also allow us to borrow local variables.

Every time the background thread finishes processing an item, it stores the number of processed items in an `AtomicUsize`. Meanwhile, the main thread shows that number to the user to inform them of the progress, about once per second. Once the main thread sees that all 100 items have been processed, it exits the scope, which implicitly joins the background thread, and informs the user that everything is done.

Synchronization

Once the last item is processed, it might take up to one whole second for the main thread to know, introducing an unnecessary delay at the end. To solve this, we can use thread parking ("Thread Parking" in Chapter 1) to wake the main thread from its sleep whenever there is new information it might be interested in.

Here's the same example, but now using `thread::park_timeout` rather than `thread::sleep`:

```
fn main() {
    let num_done = AtomicUsize::new(0);

    let main_thread = thread::current();

    thread::scope(|s| {
        // A background thread to process all 100 items.
        s.spawn(|| {
            for i in 0..100 {
                process_item(i); // Assuming this takes some time.
                num_done.store(i + 1, Relaxed);
                main_thread.unpark(); // Wake up the main thread.
            }
        });

        // The main thread shows status updates.
        loop {
            let n = num_done.load(Relaxed);
            if n == 100 { break; }
            println!("Working.. {n}/100 done");
            thread::park_timeout(Duration::from_secs(1));
        }
    });

    println!("Done!");
}
```

Not much has changed. We've obtained a handle to the main thread through `thread::current()`, which is now used by the background thread to unpark the main thread after every status update. The main thread now uses `park_timeout` rather than `sleep`, so that it can be interrupted.

Now, any status updates are immediately reported to the user, while still repeating the last update every second to show that the program is still running.

Example: Lazy Initialization

The last example before we move on to more advanced atomic operations is about *lazy initialization*.

Imagine there is a value `x`, which we are reading from a file, obtaining from the operating system, or calculating in some other way, that we expect to be constant during a run of the program. Maybe `x` is the version of the operating system, or the total amount of memory, or the 400th digit of tau. It doesn't really matter for this example.

Since we don't expect it to change, we can request or calculate it only the first time we need it, and remember the result. The first thread that needs it will have to calculate the value, but it can store it in an atomic `static` to make it available for all threads, including itself if it needs it again later.

Let's take a look at an example of this. To keep things simple, we'll assume `x` is never zero, so that we can use zero as a placeholder before it has been calculated.

```
use std::sync::atomic::AtomicU64;

fn get_x() -> u64 {
    static X: AtomicU64 = AtomicU64::new(0);
    let mut x = X.load(Relaxed);
    if x == 0 {
        x = calculate_x();
        X.store(x, Relaxed);
    }
    x
}
```

The first thread to call `get_x()` will check the static `X` and see it is still zero, calculate its value, and store the result back in the static to make it available for future use. Later, any call to `get_x()` will see that the value in the static is nonzero, and return it immediately without calculating it again.

However, if a second thread calls `get_x()` while the first one is still calculating `x`, the second thread will also see a zero and also calculate `x` in parallel. One of the threads will end up overwriting the result of the other, depending on which one finishes first. This is called a *race*. Not a *data race*, which is undefined behavior and impossible in Rust without using `unsafe`, but still a race with an unpredictable winner.

Since we expect `x` to be constant, it doesn't matter who wins the race, as the result will be the same regardless. Depending on how much time we expect `calculate_x()` to take, this might be a very good or very bad strategy.

If `calculate_x()` is expected to take a long time, it's better if threads wait while the first thread is still initializing `X`, to avoid unnecessarily wasting processor time. You could implement this using a condition variable or thread parking ("Waiting: Parking and Condition Variables" in Chapter 1),

but that quickly gets too complicated for a small example. The Rust standard library provides exactly this functionality through `std::sync::Once` and `std::sync::OnceLock`, so there's usually no need to implement these yourself.

Fetch-and-Modify Operations

Now that we've seen a few use cases for the basic `load` and `store` operations, let's move on to more interesting operations: the *fetch-and-modify* operations. These operations modify the atomic variable, but also load (fetch) the original value, as a single atomic operation.

The most commonly used ones are `fetch_add` and `fetch_sub`, which perform addition and subtraction, respectively. Some of the other available operations are `fetch_or` and `fetch_and` for bitwise operations, and `fetch_max` and `fetch_min` which can be used to keep a running maximum or minimum.

Their function signatures are as follows, using `AtomicI32` as an example:

```
impl AtomicI32 {
    pub fn fetch_add(&self, v: i32, ordering: Ordering) -> i32;
    pub fn fetch_sub(&self, v: i32, ordering: Ordering) -> i32;
    pub fn fetch_or(&self, v: i32, ordering: Ordering) -> i32;
    pub fn fetch_and(&self, v: i32, ordering: Ordering) -> i32;
    pub fn fetch_nand(&self, v: i32, ordering: Ordering) -> i32;
    pub fn fetch_xor(&self, v: i32, ordering: Ordering) -> i32;
    pub fn fetch_max(&self, v: i32, ordering: Ordering) -> i32;
    pub fn fetch_min(&self, v: i32, ordering: Ordering) -> i32;
    pub fn swap(&self, v: i32, ordering: Ordering) -> i32; // "fetch_store"
}
```

The one outlier is the operation that simply stores a new value, regardless of the old value. Instead of `fetch_store`, it has been called `swap`.

Here's a quick demonstration showing how `fetch_add` returns the value before the operation:

```
use std::sync::atomic::AtomicI32;

let a = AtomicI32::new(100);
let b = a.fetch_add(23, Relaxed);
let c = a.load(Relaxed);

assert_eq!(b, 100);
assert_eq!(c, 123);
```

The `fetch_add` operation incremented `a` from 100 to 123, but returned to us the old value of 100. Any next operation will see the value of 123.

The return value from these operations is not always relevant. If you only need the operation to be applied to the atomic value, but are not interested in the value itself, it's perfectly fine to simply ignore the return value.

An important thing to keep in mind is that `fetch_add` and `fetch_sub` implement *wrapping* behavior for overflows. Incrementing a value past the maximum representable value will wrap around and result in the minimum representable value. This is different than the behavior of the plus and minus operators on regular integers, which will panic in debug mode on overflow.

In "Compare-and-Exchange Operations", we'll see how to do atomic addition with overflow checking.

But first, let's see some real-world use cases of these methods.

Example: Progress Reporting from Multiple Threads

In "Example: Progress Reporting", we used an `AtomicUsize` to report the progress of a background thread. If we had split the work over, for example, four threads with each processing 25 items, we'd need to know the progress from all four threads.

We could use a separate `AtomicUsize` for each thread and load them all in the main thread and sum them up, but an easier solution is to use a single `AtomicUsize` to track the total number of processed items over all threads.

To make that work, we can no longer use the `store` method, as that would overwrite the progress from other threads. Instead, we can use an atomic add operation to increment the counter after every processed item.

Let's update the example from "Example: Progress Reporting" to split the work over four threads:

```
fn main() {
    let num_done = &AtomicUsize::new(0);

    thread::scope(|s| {
        // Four background threads to process all 100 items, 25 each.
        for t in 0..4 {
            s.spawn(move || {
                for i in 0..25 {
                    process_item(t * 25 + i); // Assuming this takes some time.
                    num_done.fetch_add(1, Relaxed);
                }
            });
        }

        // The main thread shows status updates, every second.
        loop {
            let n = num_done.load(Relaxed);
            if n == 100 { break; }
            println!("Working.. {n}/100 done");
            thread::sleep(Duration::from_secs(1));
        }
    });
```

```
    println!("Done!");
}
```

A few things have changed. Most importantly, we now spawn four background threads rather than one, and use `fetch_add` instead of `store` to modify the `num_done` atomic variable.

More subtly, we now use a `move` closure for the background threads, and `num_done` is now a reference. This is not related to our use of `fetch_add`, but rather to how we spawn four threads in a loop. This closure captures `t` to know which of the four threads it is, and thus whether to start at item 0, 25, 50, or 75. Without the `move` keyword, the closure would try to capture `t` by reference. That isn't allowed, as it only exists briefly during the loop.

As a `move` closure, it moves (or copies) its captures rather than borrowing them, giving it a copy of `t`. Because it also captures `num_done`, we've changed that variable to be a reference, since we still want to borrow that same `AtomicUsize`. Note that the atomic types do not implement the `Copy` trait, so we'd have gotten an error if we had tried to move one into more than one thread.

Closure capture subtleties aside, the change to use `fetch_add` here is very simple. We don't know in which order the threads will increment `num_done`, but as the addition is atomic, we don't have to worry about anything and can be sure it will be exactly 100 when all threads are done.

Example: Statistics

Continuing with this concept of reporting what other threads are doing through atomics, let's extend our example to also collect and report some statistics on the time it takes to process an item.

Next to `num_done`, we're adding two atomic variables, `total_time` and `max_time`, to keep track of the amount of time spent processing items. We'll use these to report the average and peak processing times.

```
fn main() {
    let num_done = &AtomicUsize::new(0);
    let total_time = &AtomicU64::new(0);
    let max_time = &AtomicU64::new(0);

    thread::scope(|s| {
        // Four background threads to process all 100 items, 25 each.
        for t in 0..4 {
            s.spawn(move || {
                for i in 0..25 {
                    let start = Instant::now();
                    process_item(t * 25 + i); // Assuming this takes some time.
                    let time_taken = start.elapsed().as_micros() as u64;
                    num_done.fetch_add(1, Relaxed);
                    total_time.fetch_add(time_taken, Relaxed);
                    max_time.fetch_max(time_taken, Relaxed);
                }
            });
```

```
        }

        // The main thread shows status updates, every second.
        loop {
            let total_time = Duration::from_micros(total_time.load(Relaxed));
            let max_time = Duration::from_micros(max_time.load(Relaxed));
            let n = num_done.load(Relaxed);
            if n == 100 { break; }
            if n == 0 {
                println!("Working.. nothing done yet.");
            } else {
                println!(
                    "Working.. {n}/100 done, {:?} average, {:?} peak",
                    total_time / n as u32,
                    max_time,
                );
            }
            thread::sleep(Duration::from_secs(1));
        }
    });

    println!("Done!");
}
```

The background threads now use `Instant::now()` and `Instant::elapsed()` to measure the time they spend in `process_item()`. An atomic add operation is used to add the number of microseconds to `total_time`, and an atomic max operation is used to keep track of the highest measurement in `max_time`.

The main thread divides the total time by the number of processed items to obtain the average processing time, which it then reports together with the peak time from `max_time`.

Since the three atomic variables are updated separately, it is possible for the main thread to load the values after a thread has incremented `num_done`, but before it has updated `total_time`, resulting in an underestimate of the average. More subtly, because the `Relaxed` memory ordering gives no guarantees about the relative order of operations as seen from another thread, it might even briefly see a new updated value of `total_time`, while still seeing an old value of `num_done`, resulting in an overestimate of the average.

Neither of these is a big issue in our example. The worst that can happen is that an inaccurate average is briefly reported to the user.

If we want to avoid this, we can put the three statistics inside a `Mutex`. Then we'd briefly lock the mutex while updating the three numbers, which no longer have to be atomic by themselves. This effectively turns the three updates into a single atomic operation, at the cost of locking and unlocking a mutex, and potentially temporarily blocking threads.

Example: ID Allocation

Let's move on to a use case where we actually need the return value from `fetch_add`.

Suppose we need some function, `allocate_new_id()`, that gives a new unique number every time it is called. We might use these numbers to identify tasks or other things in our program; things that need to be uniquely identified by something small that can be easily stored and passed around between threads, such as an integer.

Implementing this function turns out to be trivial using `fetch_add`:

```
use std::sync::atomic::AtomicU32;

fn allocate_new_id() -> u32 {
    static NEXT_ID: AtomicU32 = AtomicU32::new(0);
    NEXT_ID.fetch_add(1, Relaxed)
}
```

We simply keep track of the *next* number to give out, and increment it every time we load it. The first caller will get a 0, the second a 1, and so on.

The only problem here is the wrapping behavior on overflow. The 4,294,967,296th call will overflow the 32-bit integer, such that the next call will return 0 again.

Whether this is a problem depends on the use case: how likely is it to be called this often, and what's the worst that can happen if the numbers are not unique? While this might seem like a huge number, modern computers can easily execute our function that many times within seconds. If memory safety is dependent on these numbers being unique, our implementation above is not acceptable.

To solve this, we can attempt to make the function panic if it is called too many times, like this:

```
// This version is problematic.
fn allocate_new_id() -> u32 {
    static NEXT_ID: AtomicU32 = AtomicU32::new(0);
    let id = NEXT_ID.fetch_add(1, Relaxed);
    assert!(id < 1000, "too many IDs!");
    id
}
```

Now, the `assert` statement will panic after a thousand calls. However, this happens *after* the atomic add operation already happened, meaning that `NEXT_ID` has already been incremented to 1001 when we panic. If another thread then calls the function, it'll increment it to 1002 before panicking, and so on. Although it might take significantly longer, we'll run into the same problem after 4,294,966,296 panics when `NEXT_ID` will overflow to zero again.

There are three common solutions to this problem. The first one is to not panic but instead completely abort the process on overflow. The `std::process::abort` function will abort the entire process, ruling out the possibility of anything continuing to call our function. While aborting the process might take a brief moment in which the function can still be called by other threads, the chance of that happening billions of times before the program is truly aborted is negligible.

This is, in fact, how the overflow check in `Arc::clone()` in the standard library is implemented, in case you somehow manage to clone it `isize::MAX` times. That'd take hundreds of years on a 64-bit computer, but is achievable in seconds if `isize` is only 32 bits.

A second way to deal with the overflow is to use `fetch_sub` to decrement the counter again before panicking, like this:

```
fn allocate_new_id() -> u32 {
    static NEXT_ID: AtomicU32 = AtomicU32::new(0);
    let id = NEXT_ID.fetch_add(1, Relaxed);
    if id >= 1000 {
        NEXT_ID.fetch_sub(1, Relaxed);
        panic!("too many IDs!");
    }
    id
}
```

It's still possible for the counter to very briefly be incremented beyond 1000 when multiple threads execute this function at the same time, but it is limited by the number of active threads. It's reasonable to assume there will never be billions of active threads at once, especially not all simultaneously executing the same function in the brief moment between `fetch_add` and `fetch_sub`.

This is how overflows are handled for the number of running threads in the standard library's `thread::scope` implementation.

The third way of handling overflows is arguably the only truly correct one, as it prevents the addition from happening at all if it would overflow. However, we cannot implement that with the atomic operations we've seen so far. For this, we'll need compare-and-exchange operations, which we'll explore next.

Compare-and-Exchange Operations

The most advanced and flexible atomic operation is the *compare-and-exchange* operation. This operation checks if the atomic value is equal to a given value, and only if that is the case does it replace it with a new value, all atomically as a single operation. It will return the previous value and tell us whether it replaced it or not.

Its signature is a bit more complicated than the ones we've seen so far. Using `AtomicI32` as an example, it looks like this:

```
impl AtomicI32 {
    pub fn compare_exchange(
        &self,
        expected: i32,
        new: i32,
        success_order: Ordering,
        failure_order: Ordering
    ) -> Result<i32, i32>;
```

```
}
```

Ignoring memory ordering for a moment, it is basically identical to the following implementation, except it all happens as a single, indivisible atomic operation:

```
impl AtomicI32 {
    pub fn compare_exchange(&self, expected: i32, new: i32) -> Result<i32, i32> {
        // In reality, the load, comparison and store,
        // all happen as a single atomic operation.
        let v = self.load();
        if v == expected {
            // Value is as expected.
            // Replace it and report success.
            self.store(new);
            Ok(v)
        } else {
            // The value was not as expected.
            // Leave it untouched and report failure.
            Err(v)
        }
    }
}
```

Using this, we can load a value from an atomic variable, perform any calculation we like, and then only store the newly calculated value if the atomic variable didn't change in the meantime. If we put this in a loop to retry if it did change, we could use this to implement all the other atomic operations, making this the most general one.

To demonstrate, let's increment an `AtomicU32` by one without using `fetch_add`, just to see how `compare_exchange` is used in practice:

```
fn increment(a: &AtomicU32) {
    let mut current = a.load(Relaxed); // (1)
    loop {
        let new = current + 1; // (2)
        match a.compare_exchange(current, new, Relaxed, Relaxed) { // (3)
            Ok(_) => return, // (4)
            Err(v) => current = v, // (5)
        }
    }
}
```

> **Note**

If the atomic variable changes from some value `A` to `B` and then back to `A` after the `load` operation, but before the `compare_exchange` operation, it would still succeed, even though the atomic variable was changed (and changed back) in the meantime. In many cases, as with our `increment` example,

this is not a problem. However, there are certain algorithms, often involving atomic pointers, for which this can be a problem. This is known as the *ABA problem*.

In addition to `compare_exchange`, there is a similar method named `compare_exchange_weak`. The difference is that the weak version may still sometimes leave the value untouched and return an `Err`, even though the atomic value matched the expected value. On some platforms, this method can be implemented more efficiently and should be preferred in cases where the consequences of a spurious compare-and-exchange failure are insignificant, such as in our `increment` function above. In Chapter 7, we'll dive into the low-level details to find out why the weak version can be more efficient.

Example: ID Allocation Without Overflow

Now, back to our overflow problem in `allocate_new_id()` from "Example: ID Allocation".

To stop incrementing `NEXT_ID` beyond a certain limit to prevent overflows, we can use `compare_exchange` to implement atomic addition with an upper bound. Using that idea, let's make a version of `allocate_new_id` that always handles overflow correctly, even in practically impossible situations:

```
fn allocate_new_id() -> u32 {
    static NEXT_ID: AtomicU32 = AtomicU32::new(0);
    let mut id = NEXT_ID.load(Relaxed);
    loop {
        assert!(id < 1000, "too many IDs!");
        match NEXT_ID.compare_exchange_weak(id, id + 1, Relaxed, Relaxed) {
            Ok(_) => return id,
            Err(v) => id = v,
        }
    }
}
```

Now we check and panic *before* modifying `NEXT_ID`, guaranteeing it will never be incremented beyond 1000, making overflow impossible. We can now raise the upper limit from 1000 to `u32::MAX` if we want, without having to worry about edge cases in which it might get incremented beyond the limit.

Fetch-Update

The atomic types have a convenience method called `fetch_update` for the compare-and-exchange loop pattern. It's equivalent to a `load` operation followed by a loop that repeats a calculation and `compare_exchange_weak`, just like what we did above.

Using it, we could implement our `allocate_new_id` function with a one-liner:

```
    NEXT_ID.fetch_update(Relaxed, Relaxed,
        |n| n.checked_add(1)).expect("too many IDs!")
```

Check out the method's documentation for details.

We'll not use the `fetch_update` method in this book, so we can focus on the individual atomic operations.

Example: Lazy One-Time Initialization

In "Example: Lazy Initialization", we looked at an example of lazy initialization of a constant value. We made a function that lazily initializes a value on the first call, but reuses it on later calls. When multiple threads run the function concurrently during the first call, more than one thread might execute the initialization, and they will overwrite each other's results in an unpredictable order.

This is fine for values that we expect to be constant, or when we don't care about changing values. However, there are also use cases where such a value gets initialized to a different value each time, even though we need every invocation of the function within a single run of the program to return the same value.

For example, imagine a function `get_key()` that returns a randomly generated key that's only generated once per run of the program. It might be an encryption key used for communication with the program, which needs to be unique every time the program is run, but stays constant within a process.

This means we cannot simply use a `store` operation after generating a key, since that might overwrite a key generated by another thread just moments ago, resulting in two threads using different keys. Instead, we can use `compare_exchange` to make sure we only store the key if no other thread has already done so, and otherwise throw our key away and use the stored key instead.

Here's an implementation of this idea:

```
fn get_key() -> u64 {
    static KEY: AtomicU64 = AtomicU64::new(0);
    let key = KEY.load(Relaxed);
    if key == 0 {
        let new_key = generate_random_key(); // (1)
        match KEY.compare_exchange(0, new_key, Relaxed, Relaxed) { // (2)
            Ok(_) => new_key, // (3)
            Err(k) => k, // (4)
        }
    } else {
        key
    }
}
```

This is a good example of a situation where `compare_exchange` is more appropriate than its weak variant. We don't run our compare-and-exchange operation in a loop, and we don't want to return zero if the operation spuriously fails.

As mentioned in "Example: Lazy Initialization", if `generate_random_key()` takes a lot of time, it might make more sense to block threads during initialization, to avoid potentially spending time

generating keys that will not be used. The Rust standard library provides such functionality through `std::sync::Once` and `std::sync::OnceLock`.

Summary

- Atomic operations are indivisible; they have either fully completed, or they haven't happened yet.
- Atomic operations in Rust are done through the atomic types in `std::sync::atomic`, such as `AtomicI32`.
- Not all atomic types are available on all platforms.
- The relative ordering of atomic operations is tricky when multiple variables are involved. More in Chapter 3.
- Simple loads and stores are nice for very basic inter-thread communication, like stop flags and status reporting.
- Lazy initialization can be done as a *race*, without causing a *data race*.
- Fetch-and-modify operations allow for a small set of basic atomic modifications that are especially useful when multiple threads are modifying the same atomic variable.
- Atomic addition and subtraction silently wrap around on overflow.
- Compare-and-exchange operations are the most flexible and general, and a building block for making any other atomic operation.
- A *weak* compare-and-exchange operation can be slightly more efficient.

Complete Runnable Examples

The following complete, tested programs illustrate the primitives and patterns developed throughout this chapter.

Example: `ch2-01-stop-flag.rs` (01 Stop Flag)

```
use std::sync::atomic::AtomicBool;
use std::sync::atomic::Ordering::Relaxed;
use std::thread;
use std::time::Duration;

fn main() {
    static STOP: AtomicBool = AtomicBool::new(false);
```

```
    // Spawn a thread to do the work.
    let background_thread = thread::spawn(|| {
        while !STOP.load(Relaxed) {
            some_work();
        }
    });

    // Use the main thread to listen for user input.
    for line in std::io::stdin().lines() {
        match line.unwrap().as_str() {
            "help" => println!("commands: help, stop"),
            "stop" => break,
            cmd => println!("unknown command: {cmd:?}"),
        }
    }

    // Inform the background thread it needs to stop.
    STOP.store(true, Relaxed);

    // Wait until the background thread finishes.
    background_thread.join().unwrap();
}

fn some_work() {
    thread::sleep(Duration::from_millis(100));
}
```

Example: `ch2-02-progress-reporting.rs` (02 Progress Reporting)

```
use std::sync::atomic::AtomicUsize;
use std::sync::atomic::Ordering::Relaxed;
use std::thread;
use std::time::Duration;

fn main() {
    let num_done = AtomicUsize::new(0);

    thread::scope(|s| {
        // A background thread to process all 100 items.
        s.spawn(|| {
            for i in 0..100 {
                process_item(i); // Assuming this takes some time.
                num_done.store(i + 1, Relaxed);
            }
        });
```

```
        // The main thread shows status updates, every second.
        loop {
            let n = num_done.load(Relaxed);
            if n == 100 { break; }
            println!("Working.. {n}/100 done");
            thread::sleep(Duration::from_secs(1));
        }
    });

    println!("Done!");
}

fn process_item(_: usize) {
    thread::sleep(Duration::from_millis(37));
}
```

Example: `ch2-03-progress-reporting-unpark.rs` (03 Progress Reporting Unpark)

```
use std::sync::atomic::AtomicUsize;
use std::sync::atomic::Ordering::Relaxed;
use std::thread;
use std::time::Duration;

fn main() {
    let num_done = AtomicUsize::new(0);

    let main_thread = thread::current();

    thread::scope(|s| {
        // A background thread to process all 100 items.
        s.spawn(|| {
            for i in 0..100 {
                process_item(i); // Assuming this takes some time.
                num_done.store(i + 1, Relaxed);
                main_thread.unpark(); // Wake up the main thread.
            }
        });

        // The main thread shows status updates.
        loop {
            let n = num_done.load(Relaxed);
            if n == 100 { break; }
            println!("Working.. {n}/100 done");
            thread::park_timeout(Duration::from_secs(1));
        }
```

```
    });

    println!("Done!");
}

fn process_item(_: usize) {
    thread::sleep(Duration::from_millis(37));
}
```

Example: `ch2-04-lazy-init.rs` (04 Lazy Init)

```
use std::sync::atomic::AtomicU64;
use std::sync::atomic::Ordering::Relaxed;
use std::thread;
use std::time::Duration;

fn get_x() -> u64 {
    static X: AtomicU64 = AtomicU64::new(0);
    let mut x = X.load(Relaxed);
    if x == 0 {
        x = calculate_x();
        X.store(x, Relaxed);
    }
    x
}

fn calculate_x() -> u64 {
    thread::sleep(Duration::from_secs(1));
    123
}

fn main() {
    dbg!(get_x());
    dbg!(get_x());
}
```

Example: `ch2-05-fetch-add.rs` (05 Fetch Add)

```
use std::sync::atomic::AtomicI32;
use std::sync::atomic::Ordering::Relaxed;

fn main() {
    let a = AtomicI32::new(100);
    let b = a.fetch_add(23, Relaxed);
```

```
    let c = a.load(Relaxed);

    assert_eq!(b, 100);
    assert_eq!(c, 123);
}
```

Example: `ch2-06-progress-reporting-multiple-threads.rs` (06 Progress Reporting Multiple Threads)

```
use std::sync::atomic::AtomicUsize;
use std::sync::atomic::Ordering::Relaxed;
use std::thread;
use std::time::Duration;

fn main() {
    let num_done = &AtomicUsize::new(0);

    thread::scope(|s| {
        // Four background threads to process all 100 items, 25 each.
        for t in 0..4 {
            s.spawn(move || {
                for i in 0..25 {
                    process_item(t * 25 + i); // Assuming this takes some time.
                    num_done.fetch_add(1, Relaxed);
                }
            });
        }

        // The main thread shows status updates, every second.
        loop {
            let n = num_done.load(Relaxed);
            if n == 100 { break; }
            println!("Working.. {n}/100 done");
            thread::sleep(Duration::from_secs(1));
        }
    });

    println!("Done!");
}

fn process_item(_: usize) {
    thread::sleep(Duration::from_millis(123));
}
```

Example: `ch2-07-statistics.rs` **(07 Statistics)**

```rust
use std::sync::atomic::AtomicUsize;
use std::sync::atomic::AtomicU64;
use std::sync::atomic::Ordering::Relaxed;
use std::thread;
use std::time::Duration;
use std::time::Instant;

fn main() {
    let num_done = &AtomicUsize::new(0);
    let total_time = &AtomicU64::new(0);
    let max_time = &AtomicU64::new(0);

    thread::scope(|s| {
        // Four background threads to process all 100 items, 25 each.
        for t in 0..4 {
            s.spawn(move || {
                for i in 0..25 {
                    let start = Instant::now();
                    process_item(t * 25 + i); // Assuming this takes some time.
                    let time_taken = start.elapsed().as_micros() as u64;
                    num_done.fetch_add(1, Relaxed);
                    total_time.fetch_add(time_taken, Relaxed);
                    max_time.fetch_max(time_taken, Relaxed);
                }
            });
        }

        // The main thread shows status updates, every second.
        loop {
            let total_time = Duration::from_micros(total_time.load(Relaxed));
            let max_time = Duration::from_micros(max_time.load(Relaxed));
            let n = num_done.load(Relaxed);
            if n == 100 { break; }
            if n == 0 {
                println!("Working.. nothing done yet.");
            } else {
                println!(
                    "Working.. {n}/100 done, {:?} average, {:?} peak",
                    total_time / n as u32,
                    max_time,
                );
            }
            thread::sleep(Duration::from_secs(1));
        }
    });
```

```
    println!("Done!");
}

fn process_item(_: usize) {
    thread::sleep(Duration::from_millis(123));
}
```

Example: `ch2-08-id-allocation.rs` (08 Id Allocation)

```
use std::sync::atomic::AtomicU32;
use std::sync::atomic::Ordering::Relaxed;

// This version is problematic.
fn allocate_new_id() -> u32 {
    static NEXT_ID: AtomicU32 = AtomicU32::new(0);
    NEXT_ID.fetch_add(1, Relaxed)
}

fn main() {
    dbg!(allocate_new_id()); // 0
    dbg!(allocate_new_id()); // 1
    dbg!(allocate_new_id()); // 2

    println!("overflowing the counter... (this might take a minute)");

    for _ in 3..=u32::MAX {
        allocate_new_id();
    }

    println!("overflowed!");

    dbg!(allocate_new_id()); //  This will produce zero again.
}
```

Example: `ch2-09-id-allocation-panic.rs` (09 Id Allocation Panic)

```
use std::sync::atomic::AtomicU32;
use std::sync::atomic::Ordering::Relaxed;

// This version is problematic.
fn allocate_new_id() -> u32 {
    static NEXT_ID: AtomicU32 = AtomicU32::new(0);
    let id = NEXT_ID.fetch_add(1, Relaxed);
```

```
    assert!(id < 1000, "too many IDs!");
    id
}

fn main() {
    dbg!(allocate_new_id()); // This will produce a zero.

    for _ in 1..1000 {
        allocate_new_id(); // 1 through 999.
    }

    println!("overflowing the counter... (this might take a few hours)");

    std::panic::set_hook(Box::new(|_| {}));

    for _ in 1000..=u32::MAX {
        let _ = std::panic::catch_unwind(|| allocate_new_id());
    }

    println!("overflowed!");

    dbg!(allocate_new_id()); //  This will produce zero again.
}
```

Example: `ch2-10-id-allocation-subtract-before-panic.rs` (10 Id Allocation Subtract Before Panic)

```
use std::sync::atomic::AtomicU32;
use std::sync::atomic::Ordering::Relaxed;

fn allocate_new_id() -> u32 {
    static NEXT_ID: AtomicU32 = AtomicU32::new(0);
    let id = NEXT_ID.fetch_add(1, Relaxed);
    if id >= 1000 {
        NEXT_ID.fetch_sub(1, Relaxed);
        panic!("too many IDs!");
    }
    id
}

fn main() {
    dbg!(allocate_new_id());
    dbg!(allocate_new_id());
    dbg!(allocate_new_id());
    // TODO
```

```
}
```

Example: `ch2-11-increment-with-compare-exchange.rs` (11 Increment With Compare Exchange)

```
use std::sync::atomic::AtomicU32;
use std::sync::atomic::Ordering::Relaxed;

fn increment(a: &AtomicU32) {
    let mut current = a.load(Relaxed);
    loop {
        let new = current + 1;
        match a.compare_exchange(current, new, Relaxed, Relaxed) {
            Ok(_) => return,
            Err(v) => current = v,
        }
    }
}

fn main() {
    let a = AtomicU32::new(0);
    increment(&a);
    increment(&a);
    assert_eq!(a.into_inner(), 2);
}
```

Example: `ch2-12-id-allocation-without-overflow.rs` (12 Id Allocation Without Overflow)

```
use std::sync::atomic::AtomicU32;
use std::sync::atomic::Ordering::Relaxed;

fn allocate_new_id() -> u32 {
    static NEXT_ID: AtomicU32 = AtomicU32::new(0);
    let mut id = NEXT_ID.load(Relaxed);
    loop {
        assert!(id < 1000, "too many IDs!");
        match NEXT_ID.compare_exchange_weak(id, id + 1, Relaxed, Relaxed) {
            Ok(_) => return id,
            Err(v) => id = v,
        }
    }
}
```

```
fn main() {
    dbg!(allocate_new_id());
    dbg!(allocate_new_id());
    dbg!(allocate_new_id());
    // TODO
}
```

Example: `ch2-13-lazy-one-time-init.rs` (13 Lazy One Time Init)

```
use std::sync::atomic::AtomicU64;
use std::sync::atomic::Ordering::Relaxed;

fn get_key() -> u64 {
    static KEY: AtomicU64 = AtomicU64::new(0);
    let key = KEY.load(Relaxed);
    if key == 0 {
        let new_key = generate_random_key();
        match KEY.compare_exchange(0, new_key, Relaxed, Relaxed) {
            Ok(_) => new_key,
            Err(k) => k,
        }
    } else {
        key
    }
}

fn generate_random_key() -> u64 {
    123
    // TODO
}

fn main() {
    dbg!(get_key());
    dbg!(get_key());
    dbg!(get_key());
}
```

Memory Ordering

Memory Ordering

In Chapter 2, we briefly touched upon the concept of *memory ordering*. In this chapter, we'll dive into this topic and explore all the available memory ordering options, and, most importantly, when to use which one.

Reordering and Optimizations

Processors and compilers perform all sorts of tricks to make your programs run as fast as possible. A processor might determine that two particular consecutive instructions in your program will not affect each other, and execute them *out of order* if that is faster, for example. While one instruction is briefly blocked on fetching some data from main memory, several of the following instructions might be executed and finished before the first instruction finishes, as long as that wouldn't change the behavior of your program. Similarly, a compiler might decide to reorder or rewrite parts of your program if it has reason to believe that might result in faster execution. But, again, only if that wouldn't change the behavior of your program.

Let's take a look at the following function as an example:

```
fn f(a: &mut i32, b: &mut i32) {
    *a += 1;
    *b += 1;
    *a += 1;
}
```

Here, the compiler will most certainly understand that the order of these operations does not matter, since nothing happens between these three addition operations that depends on the value of `*a` or `*b`. (Assuming overflow checking is disabled.) Because of that, it might reorder the second and third operations, and then merge the first two into a single addition:

```
fn f(a: &mut i32, b: &mut i32) {
    *a += 2;
    *b += 1;
}
```

Later, while executing this function of the optimized compiled program, a processor might for a variety of reasons end up executing the second addition before the first addition, possibly because `*b` was available in a cache, while `*a` had to be fetched from the main memory.

Regardless of these optimizations, the result stays the same: `*a` is incremented by two and `*b` is incremented by one. The order in which they were incremented is entirely invisible to the rest of your program.

The logic for verifying that a specific reordering or other optimization won't affect the behavior of your program does not take other threads into account. In our example above, that's perfectly fine, as the unique references (`&mut i32`) guarantee that nothing else can possibly access the values, making other threads irrelevant. The only situation where this is a problem is when mutating data that's shared between threads. Or, in other words, when working with atomics. This is why we have to explicitly tell the compiler and processor what they can and can't do with our atomic operations, since their usual logic ignores interactions between threads and might allow for optimizations that do change the result of your program.

The interesting question is *how* we tell them. If we wanted to precisely spell out exactly what is and isn't acceptable, concurrent programming might become exceedingly verbose and error-prone, and maybe even architecture-specific:

```
let x = a.fetch_add(1,
    Dear compiler and processor,
    Feel free to reorder this with operations on b,
    but if there's another thread concurrently executing f,
    please don't reorder this with operations on c!
    Also, processor, don't forget to flush your store buffer!
    If b is zero, though, it doesn't matter.
    In that case, feel free to do whatever is fastest.
    Thanks~ <3
);
```

Instead, we can only pick from a small set of options, represented by the `std::sync::atomic::Ordering` enum, which every atomic operation takes as an argument. The set of available options is very limited, but has been carefully picked to fit most use cases well. The orderings are very abstract and do not directly reflect the actual compiler and processor mechanisms involved, such as instruction reordering. This makes it possible for your concurrent code to be architecture-independent and future-proof. It allows for verification without knowing the details of every single current and future processor and compiler version.

The available orderings in Rust are:

- Relaxed ordering: `Ordering::Relaxed`
- Release and acquire ordering: `Ordering::{Release, Acquire, AcqRel}`
- Sequentially consistent ordering: `Ordering::SeqCst`

In C++, there is also something called *consume ordering*, which has been purposely omitted from Rust, but is nonetheless interesting to discuss as well.

The Memory Model

The different memory ordering options have a strict formal definition to make sure we know exactly what we're allowed to assume, and for compiler writers to know exactly what guarantees they need to provide to us. To decouple this from the details of specific processor architectures, memory ordering is defined in terms of an abstract *memory model*.

Rust's memory model, which is mostly copied from C++, doesn't match any existing processor architecture, but instead is an abstract model with a strict set of rules that attempt to represent the greatest common denominator of all current and future architectures, while also giving the compiler enough freedom to make useful assumptions while analyzing and optimizing programs.

We've already seen a part of the memory model in action in "Borrowing and Data Races" in Chapter 1, where we talked about how data races result in undefined behavior. Rust's memory model allows for concurrent atomic stores, but considers concurrent non-atomic stores to the same variable to be a data race, resulting in undefined behavior.

On most processor architectures, however, there is actually no difference between an atomic store and a regular non-atomic store, as we'll see in Chapter 7. One could argue that the memory model is more restrictive than necessary, but these strict rules make it easier to reason about a program, both for the compiler and the programmer, and they leave space for future developments.

Happens-Before Relationship

The memory model defines the order in which operations happen in terms of *happens-before relationships*. This means that as an abstract model, it doesn't talk about machine instructions, caches, buffers, timing, instruction reordering, compiler optimizations, and so on, but instead only defines situations where one thing is guaranteed to happen before another thing, and leaves the order of everything else undefined.

The basic happens-before rule is that everything that happens within the same thread happens in order. If a thread is executing `f(); g();`, then `f()` *happens-before* `g()`.

Between threads, however, happens-before relationships only occur in a few specific cases, such as when spawning and joining a thread, unlocking and locking a mutex, and through atomic operations that use non-relaxed memory ordering. `Relaxed` memory ordering is the most basic (and most performant) memory ordering that, by itself, never results in any cross-thread happens-before relationships.

To explore what that means, let's take a look at the following example where we assume `a` and `b` are concurrently executed by different threads:

```
static X: AtomicI32 = AtomicI32::new(0);
static Y: AtomicI32 = AtomicI32::new(0);

fn a() {
    X.store(10, Relaxed); // (1)
    Y.store(20, Relaxed); // (2)
}

fn b() {
    let y = Y.load(Relaxed); // (3)
    let x = X.load(Relaxed); // (4)
    println!("{x} {y}");
}
```

As mentioned above, the basic happens-before rule is that everything that happens within the same thread happens in order. In this case: (1) happens-before (2), and (3) happens-before (4), as shown in Figure 3-1. Since we use relaxed memory ordering, there are no other happens-before relationships in our example.

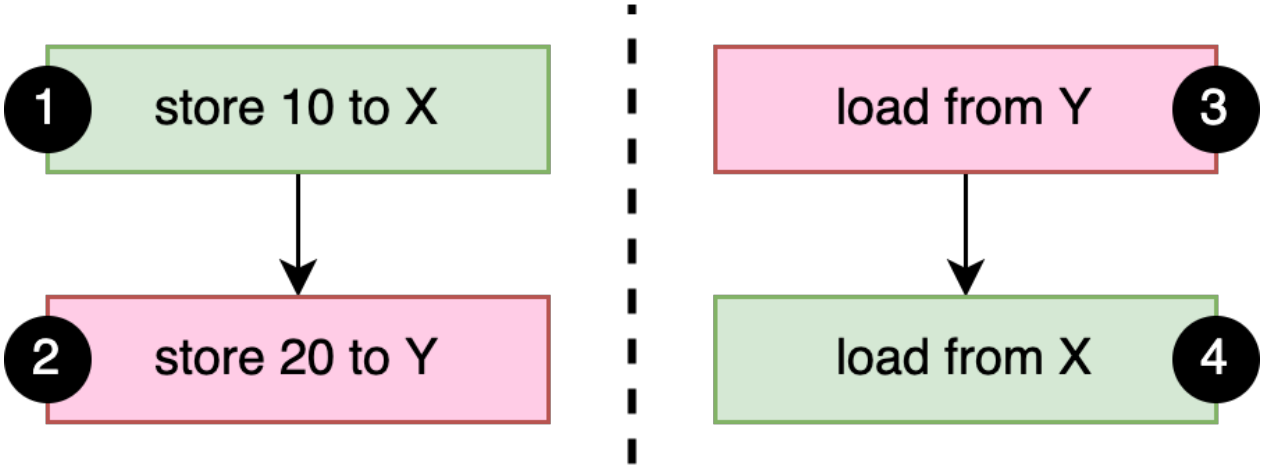


Figure 1: Figure 3-1. The happens-before relationships between atomic operations in the example code.

If either of `a` or `b` completes before the other starts, the output will be `0 0` or `10 20`. If `a` and `b` run concurrently, it's easy to see how the output can be `10 0`. One way this can happen is if the operations run in this order: (3) (1) (2) (4).

More interestingly, the output can also be `0 20`, even though there is no possible globally consistent order of the four operations that would result in this outcome. When (3) is executed, there is no happens-before relationship with (2), which means it could load either 0 or 20. When (4) is executed, there is no happens-before relationship with (1), which means it could load either 0 or 10. Given this, the output `0 20` is a valid outcome.

The important and counter-intuitive thing to understand is that operation (3) loading the value 20 does *not* result in a happens-before relationship with (2), even though that value is the one stored by (2). Our intuitive understanding of the concept of "before" breaks down when things don't necessarily happen in a globally consistent order, such as when instruction reordering is involved.

A more practical and intuitive, but less formal, understanding is that from the perspective of the thread executing `b`, operations (1) and (2) might appear to happen in the opposite order.

Spawning and Joining

Spawning a thread creates a happens-before relationship between what happened before the `spawn()` call, and the new thread. Similarly, joining a thread creates a happens-before relationship between the joined thread and what happens after the `join()` call.

To demonstrate, the assertion in the following example cannot fail:

```
static X: AtomicI32 = AtomicI32::new(0);

fn main() {
    X.store(1, Relaxed);
    let t = thread::spawn(f);
    X.store(2, Relaxed);
    t.join().unwrap();
    X.store(3, Relaxed);
}

fn f() {
    let x = X.load(Relaxed);
    assert!(x == 1 || x == 2);
}
```

Because of the happens-before relationships formed by the join and spawn operations, we know for sure that the load from `X` happens after the first store, but before the last store, as visualized in Figure 3-2. However, whether it observes the value before or after the second store is unpredictable. In other words, it could load either 1 or 2, but not 0 or 3.

Relaxed Ordering

While atomic operations using relaxed memory ordering do not provide any happens-before relationship, they do guarantee a *total modification order* of each individual atomic variable. This means that all modifications *of the same atomic variable* happen in an order that is the same from the perspective of every single thread.

To demonstrate what that means, consider the following example where we assume `a` and `b` are concurrently executed by different threads:

```
static X: AtomicI32 = AtomicI32::new(0);

fn a() {
    X.fetch_add(5, Relaxed);
    X.fetch_add(10, Relaxed);
}

fn b() {
    let a = X.load(Relaxed);
    let b = X.load(Relaxed);
    let c = X.load(Relaxed);
    let d = X.load(Relaxed);
    println!("{a} {b} {c} {d}");
}
```

In this example, only one thread modifies `X`, which makes it easy to see that there's only one possible order of modification of `X`: 0→5→15. It starts at zero, then becomes five, and is finally changed to

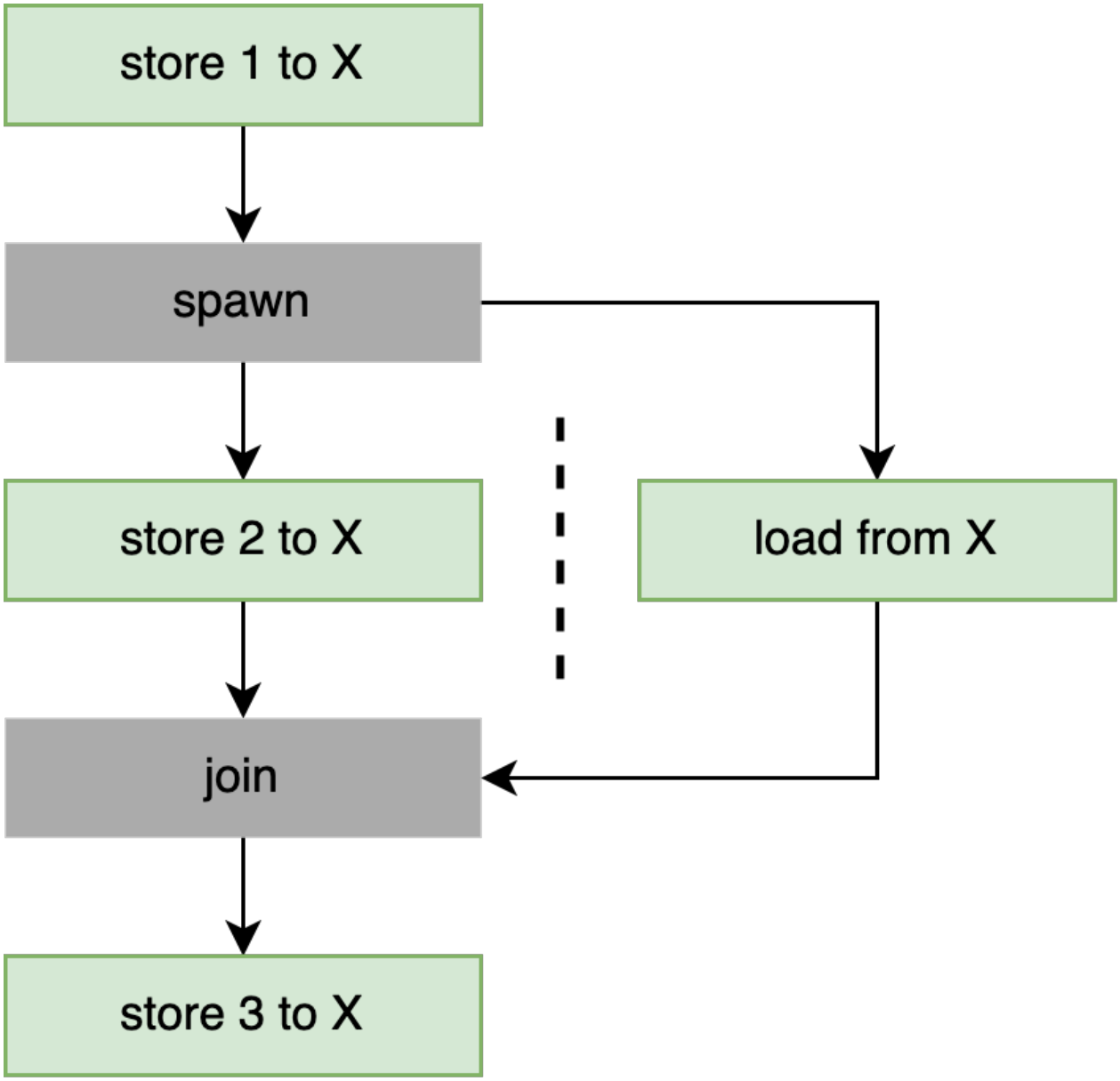


Figure 2: Figure 3-2. The happens-before relationships between the spawn, join, store, and load operations in the example code.

fifteen. Threads cannot observe any values from `X` that are inconsistent with this total modification order. This means that `"0 0 0 0"`, `"0 0 5 15"`, and `"0 15 15 15"` are some of the possible results from the print statement in the other thread, while an output of `"0 5 0 15"` or `"0 0 10 15"` is impossible.

Even if there's more than one possible order of modification for an atomic variable, all threads will agree on a single order.

Let's replace `a` by two separate functions, `a1` and `a2`, which we assume are each executed by a separate thread:

```
fn a1() {
    X.fetch_add(5, Relaxed);
}

fn a2() {
    X.fetch_add(10, Relaxed);
}
```

Assuming these are the only threads modifying `X`, there are now two possible modification orders: either 0→5→15, or 0→10→15, depending on which `fetch_add` operation executes first. Whichever happens, all threads observe the same order. So, even if we have hundreds of additional threads all running our `b()` function, we know that if one of them prints a `10`, the order must be 0→10→15 and none of them can possibly print a `5`. And vice versa.

In Chapter 2, we saw several examples of use cases where this total modification order guarantee for individual variables is enough, making relaxed memory ordering sufficient. However, if we try anything more advanced beyond those examples, we'll quickly see we need something stronger than relaxed memory ordering.

Out-of-Thin-Air Values

The lack of ordering guarantees around relaxed memory ordering can lead to some theoretical complications when operations depend on each other in a cyclic way.

To demonstrate, here's a contrived example where two threads load a value from one atomic and store it in the other:

```
static X: AtomicI32 = AtomicI32::new(0);
static Y: AtomicI32 = AtomicI32::new(0);

fn main() {
    let a = thread::spawn(|| {
        let x = X.load(Relaxed);
        Y.store(x, Relaxed);
    });
    let b = thread::spawn(|| {
        let y = Y.load(Relaxed);
```

```
        X.store(y, Relaxed);
    });
    a.join().unwrap();
    b.join().unwrap();
    assert_eq!(X.load(Relaxed), 0); // Might fail?
    assert_eq!(Y.load(Relaxed), 0); // Might fail?
}
```

It seems easy to conclude that the values of `X` and `Y` will never be anything other than zero, since the store operations only store values that were loaded from these same atomics, which are only ever zero.

If we strictly follow the theoretical memory model, however, we have to come to terms with our cyclic reasoning, and come to the scary conclusion that we might be wrong. In fact, the memory model technically allows for an outcome where both `X` and `Y` are 37 in the end, or any other value, making the assertions fail.

Due to the lack of ordering guarantees, the load operations of these two threads might *both* see the result of the store operation of the other thread, allowing for a cycle in the order of operations: we store 37 in `Y` because we loaded 37 from `X`, which was stored to `X` because we loaded 37 from `Y`, which is the value we stored in `Y`.

Fortunately, the possibility of such *out-of-thin-air* values is universally considered to be a bug in the theoretical model, and not something you need to take into account in practice. The theoretical problem of how to formalize relaxed memory ordering without the model allowing for such anomalies is an unsolved one. While this is an eyesore for formal verification that keeps many theoreticians up at night, the rest of us can relax in blissful ignorance knowing that this does not happen in practice.

Release and Acquire Ordering

Release and *acquire* memory ordering are used in a pair to form a happens-before relationship between threads. `Release` memory ordering applies to store operations, while `Acquire` memory ordering applies to load operations.

A happens-before relationship is formed when an acquire-load operation observes the result of a release-store operation. In this case, the store and everything before it happened before the load and everything after it.

When using `Acquire` for a fetch-and-modify or compare-and-exchange operation, it applies only to the part of the operation that loads the value. Similarly, `Release` applies only to the store part of an operation. `AcqRel` is used to represent the combination of `Acquire` and `Release`, which causes both the load to use acquire ordering and the store to use release ordering.

Let's go over an example to see how that's used in practice. In the following example, we send a 64-bit integer from a spawned thread to the main thread. We use an extra atomic boolean to indicate to the main thread that the integer has been stored and is ready to be read.

```
use std::sync::atomic::Ordering::{Acquire, Release};

static DATA: AtomicU64 = AtomicU64::new(0);
static READY: AtomicBool = AtomicBool::new(false);

fn main() {
    thread::spawn(|| {
        DATA.store(123, Relaxed);
        READY.store(true, Release); // Everything from before this store ..
    });
    while !READY.load(Acquire) { // .. is visible after this loads `true`.
        thread::sleep(Duration::from_millis(100));
        println!("waiting...");
    }
    println!("{}", DATA.load(Relaxed));
}
```

When the spawned thread is done storing the data, it uses a release-store to set the `READY` flag to `true`. When the main thread observes this through its acquire-load operation, a happens-before relationship is established between those two operations, as shown in Figure 3-3. At that point, we know for sure that everything that happened before the release-store to `READY` is visible to everything that happens after the acquire-load. Specifically, when the main thread loads from `DATA`, we know for sure it will load the value stored by the background thread. There's only one possible outcome this program can print on its last line: `123`.

If we had used relaxed memory ordering for all operations in this example, the main thread could have seen `READY` flip to `true`, while still loading a zero from `DATA` afterwards.

> **Note**

The names "release" and "acquire" are based on their most basic use case: one thread *releases* data by atomically storing some value to an atomic variable, and another thread *acquires* it by atomically loading that value. This is exactly what happens when we unlock (release) a mutex and subsequently lock (acquire) it on another thread.

In our example, the happens-before relationship from the `READY` flag guarantees that the store and load operations of `DATA` cannot happen concurrently. This means that we don't actually need those operations to be atomic.

However, if we simply try to use a regular non-atomic type for our data variable, the compiler will refuse our program, since Rust's type system doesn't allow us to mutate those from one thread when another thread is also borrowing them. The type system does not magically understand the happens-before relationship we've created here. Some `unsafe` code is necessary to promise to the compiler that we've thought about this carefully and we're sure we're not breaking any rules, as follows:

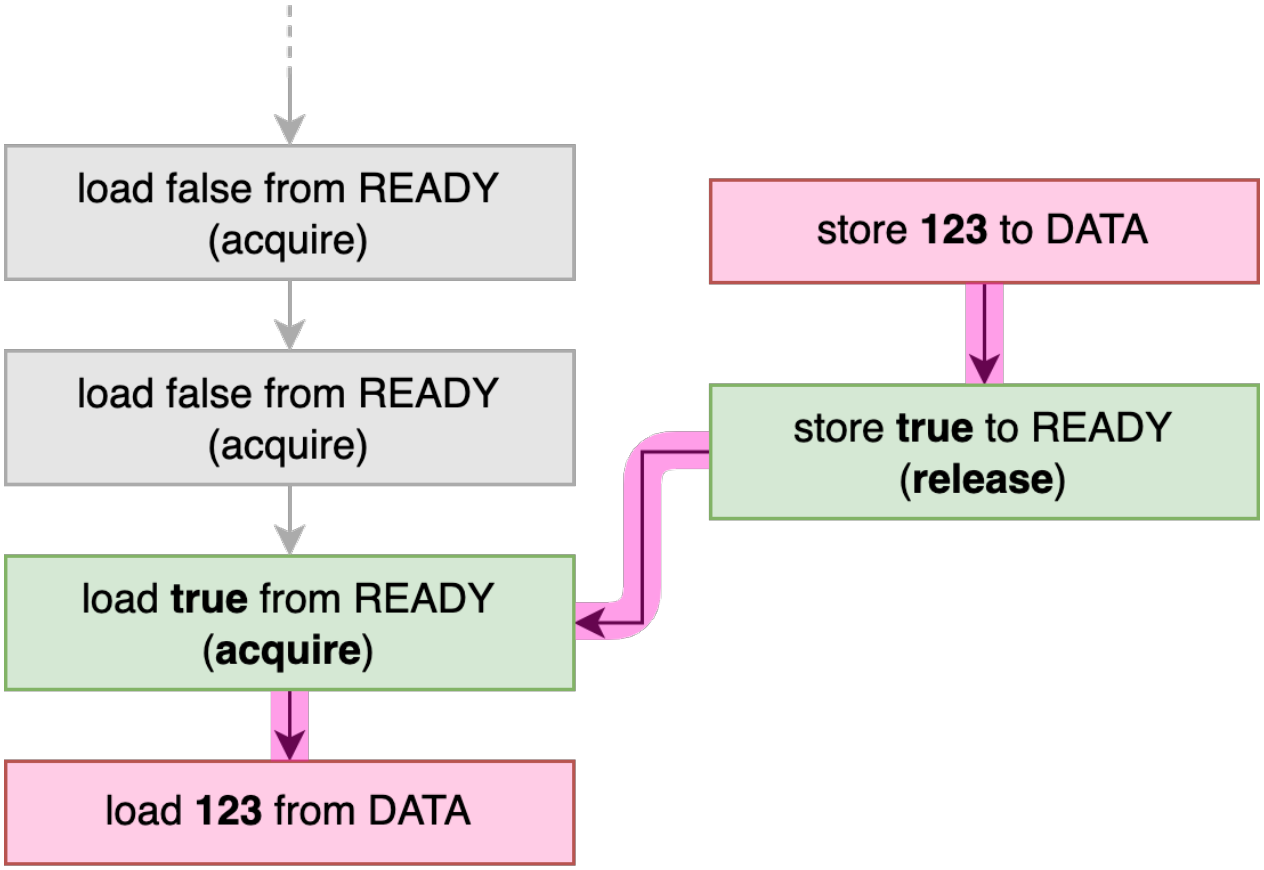


Figure 3: Figure 3-3. The happens-before relationships between atomic operations in the example code, showing the cross-thread relationship formed by the acquire and release operations.

```rust
static mut DATA: u64 = 0;
static READY: AtomicBool = AtomicBool::new(false);

fn main() {
    thread::spawn(|| {
        // Safety: Nothing else is accessing DATA,
        // because we haven't set the READY flag yet.
        unsafe { DATA = 123 };
        READY.store(true, Release); // Everything from before this store ..
    });
    while !READY.load(Acquire) { // .. is visible after this loads `true`.
        thread::sleep(Duration::from_millis(100));
        println!("waiting...");
    }
    // Safety: Nothing is mutating DATA, because READY is set.
    println!("{}", unsafe { DATA });
}
```

More Formally

A happens-before relationship is formed when an acquire-load operation observes the result of a release-store operation. But what does that mean?

Imagine that two threads both release-store a seven into the same atomic variable, and a third thread loads a seven from that variable. Does the third thread now have a happens-before relationship with the first thread or the second one? That depends on "which seven" it loaded: the one from thread one or the one from thread two. (Or perhaps an unrelated seven.) This leads us to the conclusion that even though seven equals seven, there is something different about the two sevens from the two threads.

The way to think about this is in terms of the *total modification order* that we talked about in "Relaxed Ordering": the ordered list of all modifications that happen to an atomic variable. Even if the same value is written to the same variable more than once, each of these operations represents a separate event in the total modification order of that variable. When we load a value, the value loaded matches a specific point on this per-variable "timeline," which tells us which operation we might be synchronizing with.

For example, if the total modification order of the atomic is

1. Initialized at 0
2. *Release*-store 7 (from thread two)
3. Relaxed-store 6
4. *Release*-store 7 (from thread one)

then acquire-loading a 7 would synchronize with either the release-store from the second event, or the release-store from the last event. However, if we have previously (in terms of happens-before

relationships) seen a 6, we know we're seeing the last 7, not the first one, meaning we now have a happens-before relationship with thread one, and not with thread two.

There is one extra detail, which is that a release-stored value may be modified by any number of fetch-and-modify and compare-and-exchange operations, while still resulting in a happens-before relationship with an acquire-load that reads the final result.

For example, imagine an atomic variable with the following total modification order:

1. Initialized at 0
2. *Release*-store 7
3. Relaxed-fetch-and-add 1, changing 7 to 8
4. *Release*-fetch-and-add 1, changing 8 to 9
5. *Release*-store 7
6. Relaxed-swap 10, changing 7 to 10

Now, if we acquire-load a 9 from this variable, we not only establish a happens-before relationship with the fourth operation (that stored this value), but also with the second operation (which stored a 7), even though the third operation used relaxed memory ordering.

Similarly, if we acquire-load a 10 from this variable, which was written by a relaxed operation, we still establish a happens-before relationship with the fifth operation (which stored a 7). Because that was just a regular store operation (not a fetch-and-modify or compare-and-exchange operation), it breaks the chain: we don't establish a happens-before relationship with any of the other operations.

Example: Locking

Mutexes are the most common use case for release and acquire ordering (see "Locking: Mutexes and RwLocks" in Chapter 1). When locking, they use an atomic operation to check if it was unlocked, using acquire ordering, while also (atomically) changing the state to "locked." When unlocking, they set the state back to "unlocked" using release ordering. This means that there will be a happens-before relationship between unlocking a mutex and subsequently locking it.

Here's a demonstration of this pattern:

```
static mut DATA: String = String::new();
static LOCKED: AtomicBool = AtomicBool::new(false);

fn f() {
    if LOCKED.compare_exchange(false, true, Acquire, Relaxed).is_ok() {
        // Safety: We hold the exclusive lock, so nothing else is accessing DATA.
        unsafe { DATA.push('!') };
        LOCKED.store(false, Release);
    }
}

fn main() {
```

```
    thread::scope(|s| {
        for _ in 0..100 {
            s.spawn(f);
        }
    });
}
```

As we've briefly seen in "Compare-and-Exchange Operations" in Chapter 2, compare-and-exchange operations take two memory ordering arguments: one for the case where the comparison succeeded and the store happened, and one for the case where the comparison failed and the store did not happen. In `f`, we attempt to change `LOCKED` from `false` to `true`, and only access `DATA` if that succeeds. So, we only care about the success memory ordering. If the `compare_exchange` operation fails, that must be because `LOCKED` was already set to `true`, in which case `f` doesn't do anything. This matches the `try_lock` operation on a regular mutex.

> **Note**

An observant reader might have already noticed that the compare-and-exchange operation could also have been a swap operation, since swapping `true` for `true` when already locked doesn't change the correctness of the code:

```
// This also works.
if LOCKED.swap(true, Acquire) == false {
    …
}
```

Thanks to the acquire and release memory ordering, we know for sure that no two threads can concurrently access `DATA`. As visualized in Figure 3-4, any previous access to `DATA` happened-before the subsequent release-store of `false` to `LOCKED`, which in turn happened-before the next acquire-compare-and-exchange (or acquire-swap) operation that changed that `false` to `true`, which happened-before the next access to `DATA`.

In Chapter 4 we'll turn this concept into a reusable type: a spin lock.

Example: Lazy Initialization with Indirection

In "Example: Lazy One-Time Initialization" in Chapter 2, we implemented lazy initialization of a global variable, using a compare-and-exchange operation to handle situations where multiple threads race to initialize the value concurrently. Because the value was a nonzero 64-bit integer, we were able to use an `AtomicU64` to store it, using zero as the placeholder before initializing it.

To do the same for a much larger data type that does not fit in a single atomic variable, we need to look for an alternative.

For this example, let's say we want to maintain the non-blocking behavior, so that threads never wait for another thread, but instead race and take the value from the first thread to complete

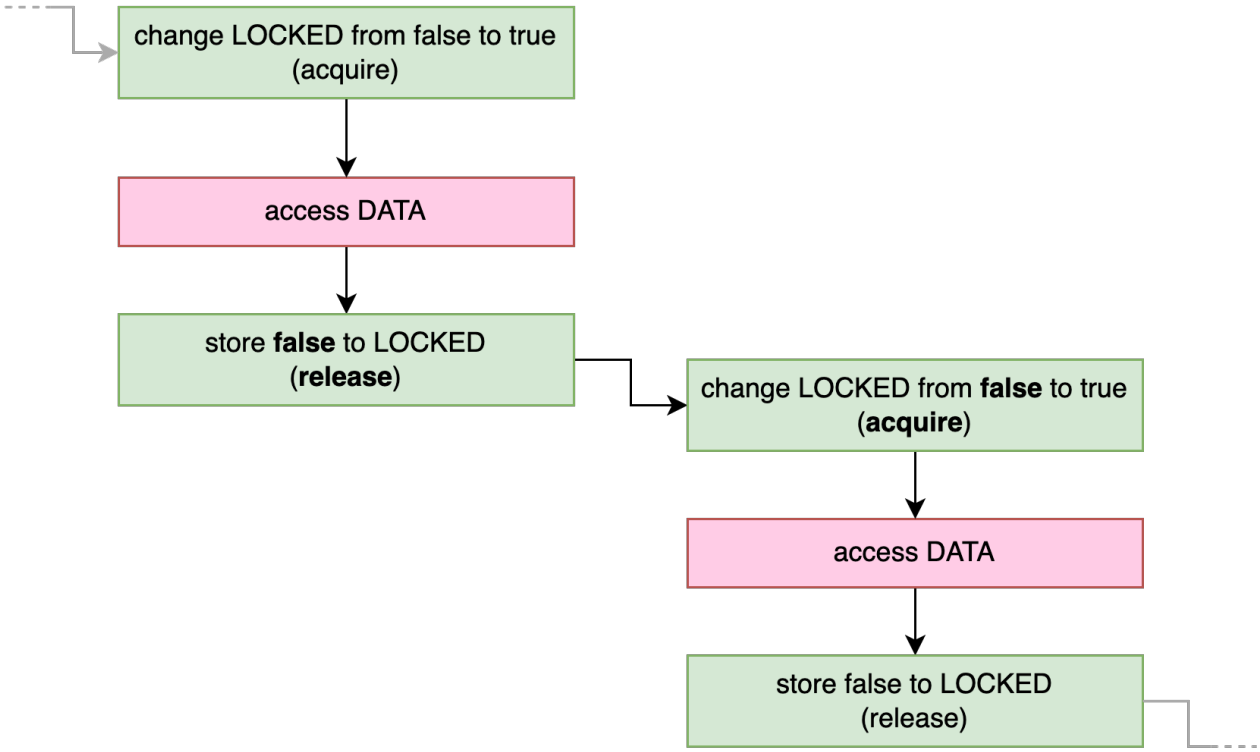


Figure 4: Figure 3-4. The happens-before relationships between atomic operations in the locking example, showing two threads locking and unlocking in sequence.

initialization. This means we still need to be able to go from ”uninitialized” to ”fully initialized” in a single atomic operation.

As the fundamental theorem of software engineering tells us, every problem in computer science can be solved by adding another layer of indirection, and this problem is no different. Since we can't fit the data into a single atomic variable, we can instead use an atomic variable to store *a pointer* to the data.

An `AtomicPtr<T>` is the atomic version of a `*mut T`: a pointer to `T`. We can use a null pointer as the placeholder for the initial state, and use a compare-and-exchange operation to atomically replace it with a pointer to a newly allocated, fully initialized `T`, which can then be read by the other threads.

Since we're not only sharing the atomic variable containing the pointer, but also the data it points to, we can no longer use relaxed memory ordering like in Chapter 2. We need to make sure that allocating and initializing the data does not race with reading it. In other words, we need to use release and acquire ordering on the store and load operations, to make sure the compiler and processor won't break our code by—for example—reordering the store of the pointer and the initialization of the data itself.

This leads us to the following implementation, for some arbitrary data type called `Data`:

```
use std::sync::atomic::AtomicPtr;

fn get_data() -> &'static Data {
    static PTR: AtomicPtr<Data> = AtomicPtr::new(std::ptr::null_mut());

    let mut p = PTR.load(Acquire);

    if p.is_null() {
        p = Box::into_raw(Box::new(generate_data()));
        if let Err(e) = PTR.compare_exchange(
            std::ptr::null_mut(), p, Release, Acquire
        ) {
            // Safety: p comes from Box::into_raw right above,
            // and wasn't shared with any other thread.
            drop(unsafe { Box::from_raw(p) });
            p = e;
        }
    }

    // Safety: p is not null and points to a properly initialized value.
    unsafe { &*p }
}
```

If the pointer we acquire-load from `PTR` is non-null, we assume it points to the already initialized data, and construct a reference to that data.

If it's still null, however, we generate new data and store it in a new allocation using `Box::new`. We then turn this `Box` into a raw pointer using `Box::into_raw`, so we can attempt to store it into `PTR` using a compare-and-exchange operation. If another thread wins the initialization race, `compare_exchange` fails as `PTR` is no longer null. If that happens, we turn our raw pointer back into a `Box` to deallocate it using `drop`, avoiding a memory leak, and continue with the pointer that the other thread stored in `PTR`.

The safety comment on the final `unsafe` block states our assumption that the data it points to has already been initialized. Note how this includes an assumption about the order in which things happened. To make sure our assumption holds, we use release and acquire memory ordering to make sure initializing the data has actually *happened-before* creating a reference to it.

We load a potentially non-null (i.e., initialized) pointer in two places: through the `load` operation and through the `compare_exchange` operation when it fails. So, as explained above, we need to use `Acquire` for both the `load` memory ordering and the `compare_exchange` failure memory ordering, to be able to synchronize with the operation that stores the pointer. This store happens when the `compare_exchange` operation succeeds, so we must use `Release` as its success ordering.

Figure 3-5 shows a visualization of the operations and happens-before relationships for a situation in which three threads call `get_data()`. In this situation, threads A and B both observe a null pointer and both attempt to initialize the atomic pointer. Thread A wins that race, causing thread B's `compare_exchange` operation to fail. Thread C only observes the atomic pointer after it has

been initialized by thread A. The end result is that all three threads end up using the box that was allocated by thread A.

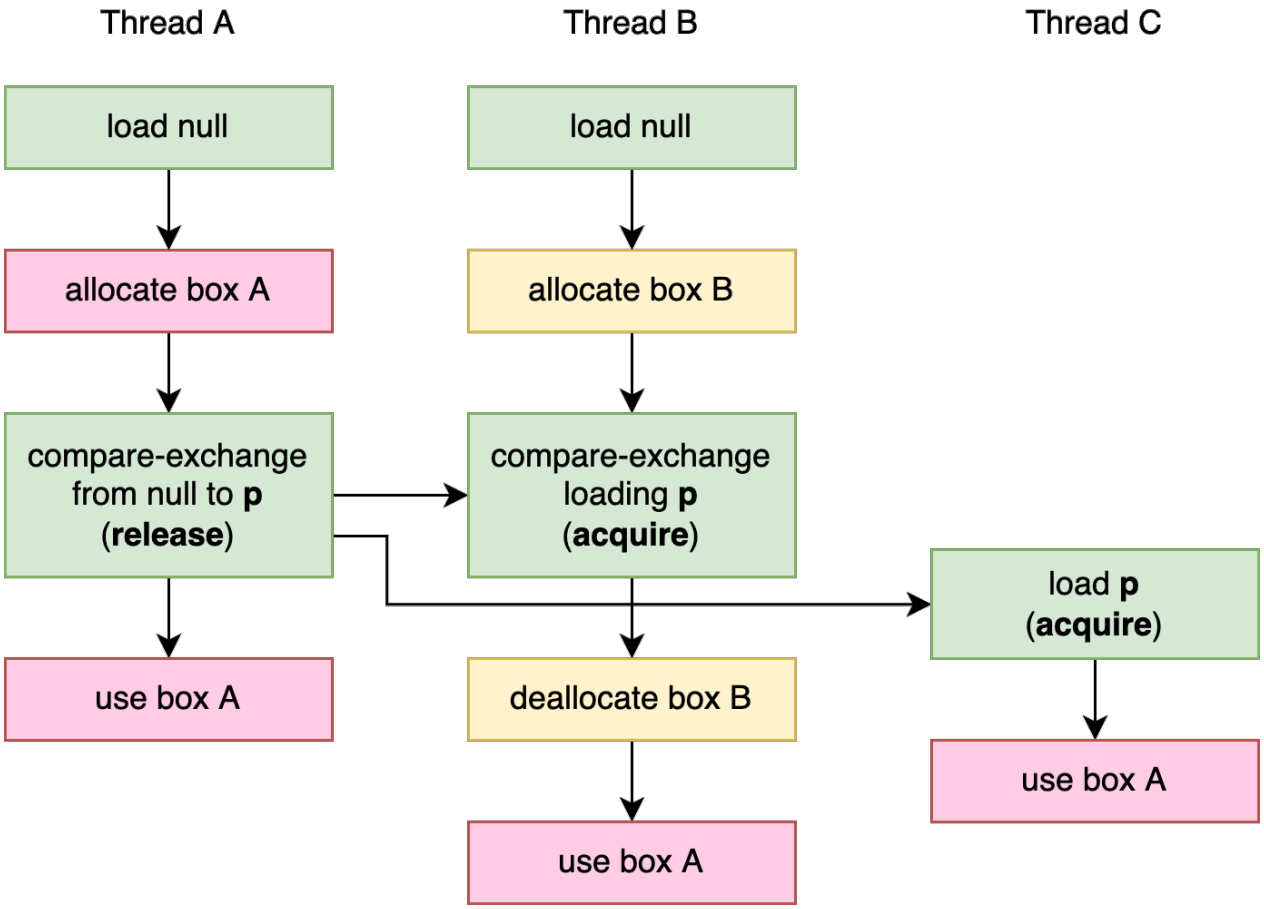


Figure 5: Figure 3-5. The operations and happens-before relationships among three threads calling get_data().

Consume Ordering

Let's take a closer look at the memory ordering in our last example. If we leave the strict memory model aside and think of it in more practical terms, we could say that the release ordering prevents the initialization of the data from being reordered with the store operation that shares the pointer with the other threads. This is important, since otherwise other threads might be able to see the data before it's fully initialized.

Similarly, we could explain the acquire ordering as preventing reordering that would cause the data to be accessed before the pointer is loaded. One might reasonably wonder, however, if that makes any sense in practice. How could the data be accessed before its address is known? We might conclude that something weaker than acquire ordering might suffice. And we would be right: this weaker ordering is called *consume ordering*.

Consume ordering is basically a lightweight—more efficient—variant of acquire ordering, whose synchronizing effects are limited to things that *depend on* the loaded value.

What that means is that if you consume-load a release-stored value `x` from an atomic variable, then, basically, that store happened before the evaluation of *dependent* expressions like `*x`, `array[x]` or `table.lookup(x + 1)`, but *not* necessarily before independent operations, like reading another variable that we don't need the value of `x` for.

Now there's good news and bad news.

The good news is that—on all modern processor architectures—consume ordering is achieved with the exact same instructions as relaxed ordering. In other words, consume ordering can be "free," which—at least on some platforms—is not the case for acquire ordering.

The bad news is that no compiler actually implements consume ordering.

As it turns out, not only is this concept of a "dependent" evaluation hard to define, it's even harder to keep such dependencies intact while transforming and optimizing a program. For example, a compiler might be able to optimize `x + 2 - x` to just `2`, effectively dropping the dependency on `x`. More subtle variations of this issue can happen with more realistic expressions like `array[x]`, if the compiler is able to make any logical deductions about the possible values of `x` or the array's elements. The issue gets even more complicated when taking control flow into account, like `if` statements or function calls.

Because of this, compilers upgrade consume ordering to acquire ordering, just to be safe. The C++20 standard even explicitly discourages the use of consume ordering, noting that an implementation other than just acquire ordering turned out to be infeasible.

It's possible that a workable definition and implementation of consume ordering might be found in the future. Until that time arrives, however, Rust does not expose `Ordering::Consume`.

Sequentially Consistent Ordering

The strongest memory ordering is *sequentially consistent* ordering: `Ordering::SeqCst`. It includes all the guarantees of acquire ordering (for loads) and release ordering (for stores), and *also* guarantees a globally consistent order of operations.

This means that every single operation using `SeqCst` ordering within a program is part of a single total order that all threads agree on. This total order is consistent with the total modification order of each individual variable.

Since it is strictly stronger than acquire and release memory ordering, a sequentially consistent load or store can take the place of an acquire-load or release-store in a release-acquire pair to form a happens-before relationship. In other words, an acquire-load can not only form a happens-before relationship with a release-store, but also with a sequentially consistent store, and similarly the other way around.

> **i** Note

Only when both sides of a happens-before relationship use `SeqCst` ordering is it guaranteed to be consistent with the single total order of `SeqCst` operations.

While it might seem like the easiest memory ordering to reason about, `SeqCst` ordering is almost never necessary in practice. In nearly all cases, regular acquire and release ordering suffice.

Here's an example that depends on sequentially consistent ordered operations:

```
use std::sync::atomic::Ordering::SeqCst;

static A: AtomicBool = AtomicBool::new(false);
static B: AtomicBool = AtomicBool::new(false);

static mut S: String = String::new();

fn main() {
    let a = thread::spawn(|| {
        A.store(true, SeqCst);
        if !B.load(SeqCst) {
            unsafe { S.push('!') };
        }
    });

    let b = thread::spawn(|| {
        B.store(true, SeqCst);
        if !A.load(SeqCst) {
            unsafe { S.push('!') };
        }
    });

    a.join().unwrap();
    b.join().unwrap();
}
```

Both threads first set their own atomic boolean to `true` to warn the other thread that they are about to access `S`, and then check the other's atomic boolean to see if they can safely access `S` without causing a data race.

If both store operations happen before either of the load operations, it's possible that neither thread ends up accessing `S`. However, it's impossible for *both* threads to access `S` and cause undefined behavior, since the sequentially consistent ordering guarantees only one of them can win the race. In every possible single total order, the first operation will be a store operation, which prevents the other thread from accessing `S`.

Virtually all real-world uses of `SeqCst` involve a similar pattern of a store that must be globally visible before a subsequent load on the same thread. For these situations, a potentially more efficient alternative is to instead use relaxed operations in combination with a `SeqCst` *fence*, which we'll explore next.

Fences

In addition to operations on atomic variables, there is one more thing that we can apply a memory ordering to: atomic fences.

The `std::sync::atomic::fence` function represents an *atomic fence* and is either a *release fence* (`Release`), an *acquire fence* (`Acquire`), or both (`AcqRel` or `SeqCst`). A `SeqCst` fence additionally also takes part in the sequentially consistent total order.

An atomic fence allows you to separate the memory ordering from the atomic operation. This can be useful if you want to apply a memory ordering to multiple operations, or if you only want to apply it conditionally.

In essence, a release-store can be split into a release fence followed by a (relaxed) store, and an acquire-load can be split into a (relaxed) load followed by an acquire fence:

The store of a release-acquire relationship,

can be substituted by a release fence followed by a relaxed store:

The load of a release-acquire relationship,

can be substituted by a relaxed load followed by an acquire fence:

Using a separate fence can result in an extra processor instruction, though, which can be slightly less efficient.

More importantly, unlike a release-store or an acquire-load, a fence is not tied to any single atomic variable. This means that a single fence can be used for multiple variables at once.

Formally, a release fence can take the place of a release operation in a happens-before relationship if that release fence is followed (on the same thread) by any atomic operation that stores a value observed by the acquire operation we're synchronizing with. Similarly, an acquire fence can take the place of any acquire operation if that acquire fence is preceded (on the same thread) by any atomic operation that loads a value stored by the release operation.

Putting this together, it means that a happens-before relationship is created between a release fence and an acquire fence if *any* store after the release fence is observed by *any* load before the acquire fence.

For example, suppose we have one thread executing a release fence followed by three atomic store operations to different variables, and another thread executing three load operations from those same variables followed by an acquire fence, as follows:

Thread 1:

Thread 2:

In this situation, if *any* of the load operations on thread 2 loads the value from the corresponding store operation of thread 1, the release fence on thread 1 happens-before the acquire fence on thread 2.

A fence does not have to directly precede or follow the atomic operations. Anything else can happen in between, including control flow. This can be used to make the fence conditional, similar to how compare-and-exchange operations have both a success and a failure ordering.

For example, if we load a pointer from an atomic variable using acquire memory ordering, we could use a fence to apply the acquire ordering only when the pointer is not null:

Using an acquire-load:

Using a conditional acquire fence:

This can be beneficial if the pointer is often expected to be null, to avoid acquire memory ordering when not necessary.

Let's take a look at a more complicated use case of release and acquire fences:

```
use std::sync::atomic::fence;

static mut DATA: [u64; 10] = [0; 10];

const ATOMIC_FALSE: AtomicBool = AtomicBool::new(false);
static READY: [AtomicBool; 10] = [ATOMIC_FALSE; 10];

fn main() {
    for i in 0..10 {
        thread::spawn(move || {
            let data = some_calculation(i);
            unsafe { DATA[i] = data };
            READY[i].store(true, Release);
        });
    }
    thread::sleep(Duration::from_millis(500));
    let ready: [bool; 10] = std::array::from_fn(|i| READY[i].load(Relaxed));
    if ready.contains(&true) {
        fence(Acquire);
        for i in 0..10 {
            if ready[i] {
                println!("data{i} = {}", unsafe { DATA[i] });
            }
        }
    }
}
```

> **Note**

`std::array::from_fn` is an easy way to execute something a certain number of times and collect the results into an array.

In this example, 10 threads do some calculations and store their results in a (non-atomic) shared variable. Each thread sets an atomic boolean to indicate that the data is ready to be read by the main thread, using a normal release-store. The main thread waits for half a second, checks all 10 booleans to see which threads are done, and prints whichever results are ready.

Instead of using 10 acquire-load operations to read the booleans, the main thread uses relaxed operations and a single acquire fence. It executes the fence before reading the data, but only if there is data to be read.

While in this particular example it might be completely unnecessary to put any effort into such optimization, this pattern for saving the overhead of additional acquire operations can be important when building highly efficient concurrent data structures.

A `SeqCst` fence is both a release fence and an acquire fence (just like `AcqRel`), but also part of the single total order of sequentially consistent operations. However, only the fence is part of the total order, but not necessarily the atomic operations before or after it. This means that unlike a release or acquire operation, a sequentially consistent operation cannot be split into a relaxed operation and a memory fence.

Compiler Fences

In addition to a regular atomic fence, the Rust standard library also provides a *compiler fence*: `std::sync::atomic::compiler_fence`. Its signature is identical to that of the regular `fence()` we discussed above, but its effects are restricted to just the compiler. Unlike a regular atomic fence, it does not prevent the processor from, for example, reordering instructions. In the vast majority of use cases for fences, a compiler fence does not suffice.

A potential use case might arise when implementing a Unix *signal handler*, or an *interrupt* on embedded systems. These are mechanisms that can suddenly interrupt a thread to temporarily execute an unrelated function on the same processor core. Because it happens on the same processor core, the usual ways in which the processor might affect memory ordering don't apply. (More on that in Chapter 7.) In this case, a compiler fence might suffice, potentially saving an instruction and hopefully increasing performance.

Another use case involves *process-wide memory barriers*. This technique falls outside the scope of Rust's memory model and is only supported on some operating systems: on Linux through the `membarrier` syscall, and on Windows using the `FlushProcessWriteBuffers` function. It effectively allows a thread to forcefully inject a (sequentially consistent) atomic fence into all concurrently running threads. This allows us to replace two matching fences with a lightweight compiler fence on one side and a heavyweight process-wide barrier on the other side. If the code on the side of the lightweight fence is executed much more often, this can improve overall performance. (See the documentation of the `membarrier` crate on crates.io for more details and a cross-platform way to use such barriers in Rust.)

A compiler fence can also be an interesting tool for exploring the effect of the processor on memory ordering. In "An Experiment" in Chapter 7, we'll break our code on purpose by replacing a regular fence with a compiler fence. This will let us experience the subtle but potentially disastrous effects of the processor when using the wrong memory ordering.

Common Misconceptions

There are a lot of misconceptions around memory ordering. Before we end this chapter, let's go over the most common ones.

Myth: I need strong memory ordering to make sure changes are "immediately" visible.

A common misunderstanding is that using a weak memory ordering like `Relaxed` means that changes to an atomic variable might never arrive at another thread, or only after a significant delay. The name "relaxed" might make it sound like nothing happens until something forces some part of the hardware to wake up and do what it should've done instead of relaxing.

The truth is that the memory model doesn't say anything about timing at all. It only defines in which order certain things happen, not how long you might have to wait for them. A hypothetical computer in which it takes years to get data from one thread to another is quite unusable, but can perfectly satisfy the memory model.

In real life, memory ordering is about things like reordering instructions, which usually happen at nanosecond scale. Stronger memory ordering does not make your data travel faster; it might even slow your program down.

Myth: Disabling optimization means I don't need to care about memory ordering.

Both the compiler and the processor play a role in making things happen in a different order than we might expect. Disabling compiler optimization does not disable every possible transformation in the compiler, and does not disable the processor features that result in instruction reordering and similar potentially problematic behavior.

Myth: Using a processor that doesn't reorder instructions means I don't need to care about memory ordering.

Some simple processors, such as those in small microcontrollers, have only one core and only ever execute one instruction at a time, all in order. However, while it's true that on such devices there's a significantly lower chance of an incorrect memory ordering resulting in actual issues, it's still possible for the compiler to make invalid assumptions based on incorrect memory ordering, breaking your code. Besides that, it's also important to realize that even when a processor does not execute instructions out of order, it might still have other features that can be relevant for memory ordering.

Myth: Relaxed operations are free.

Whether this is true depends on your definition of "free." It's true that `Relaxed` is the most efficient memory ordering and that it can be significantly faster than the others. It's even true that on all modern platforms, relaxed load and store operations compile down to the same processor instructions as non-atomic reads and writes.

If an atomic variable is only used by a single thread, any difference in speed with a non-atomic variable will most likely be because of the compiler having more freedom and being more effective at optimizing non-atomic operations. (Compilers tend to avoid most types of optimizations for atomic variables.)

However, accessing the same memory from multiple threads is usually significantly slower than accessing it from a single thread. A thread that continuously writes to an atomic variable will likely experience a noticeable slowdown when other threads start repeatedly reading the variable, since the processor cores and their caches now have to start collaborating.

We'll explore this effect in Chapter 7.

Myth: Sequentially consistent memory ordering is a great default and is always correct.

Putting aside performance concerns, sequentially consistent memory ordering is often seen as the perfect type of memory ordering to default to, because of its strong guarantees. It's true that *if* any other memory ordering is correct, `SeqCst` is also correct. This might make it sound like `SeqCst` is always correct. However, it's entirely possible that a concurrent algorithm is simply incorrect, regardless of memory ordering.

More importantly, when reading code, `SeqCst` basically tells the reader: "this operation depends on the total order of every single `SeqCst` operation in the program," which is an incredibly far-reaching claim. The same code would likely be easier to review and verify if it used weaker memory ordering instead, if possible. For example, `Release` effectively tells the reader: "this relates to an acquire operation on the same variable," which involves far fewer considerations when forming an understanding of the code.

It is advisable to see `SeqCst` as a warning sign. Seeing it in the wild often means that either something complicated is going on, or simply that the author did not take the time to analyze their memory ordering related assumptions, both of which are reasons for extra scrutiny.

Myth: Sequentially consistent memory ordering can be used for a "release-load" or an "acquire-store."

While `SeqCst` can stand in for `Acquire` or `Release`, it is not a way to somehow create an acquire-store or release-load. Those remain nonexistent. Release only applies to store operations, and acquire only to load operations.

For example, a `Release`-store does not form any release-acquire relationship with a `SeqCst`-store. If you need them to be part of a globally consistent order, both operations will have to use `SeqCst`.

Summary

- There might not be a global consistent order of all atomic operations, as things can appear to happen in a different order from different threads.
- However, each individual atomic variable has its own *total modification order*, regardless of memory ordering, which all threads agree on.
- The order of operations is formally defined through *happens-before* relationships.
- Within a single thread, there is a happens-before relationship between every single operation.
- Spawning a thread happens-before everything the spawned thread does.
- Everything a thread does happens-before joining that thread.
- Unlocking a mutex happens-before locking that mutex again.
- Acquire-loading the value from a release-store establishes a happens-before relationship. This value may be modified by any number of fetch-and-modify and compare-and-exchange operations.
- A consume-load would be a lightweight version of an acquire-load, if it existed.

- Sequentially consistent ordering results in a globally consistent order of operations, but is almost never necessary and can make code review more complicated.
- Fences allow you to combine the memory ordering of multiple operations or apply a memory ordering conditionally.

Complete Runnable Examples

The following complete, tested programs illustrate the primitives and patterns developed throughout this chapter.

Example: `ch3-01-relaxed.rs` (01 Relaxed)

```
use std::sync::atomic::AtomicI32;
use std::sync::atomic::Ordering::Relaxed;
use std::thread;

static X: AtomicI32 = AtomicI32::new(0);
static Y: AtomicI32 = AtomicI32::new(0);

fn a() {
    X.store(10, Relaxed);
    Y.store(20, Relaxed);
}

fn b() {
    let y = Y.load(Relaxed);
    let x = X.load(Relaxed);
    println!("{x} {y}");
}

fn main() {
    thread::scope(|s| {
        s.spawn(a);
        s.spawn(b);
    });
}
```

Example: `ch3-02-spawn-join.rs` **(02 Spawn Join)**

```
use std::sync::atomic::AtomicI32;
use std::sync::atomic::Ordering::Relaxed;
use std::thread;

static X: AtomicI32 = AtomicI32::new(0);

fn main() {
    X.store(1, Relaxed);
    let t = thread::spawn(f);
    X.store(2, Relaxed);
    t.join().unwrap();
    X.store(3, Relaxed);
}

fn f() {
    let x = X.load(Relaxed);
    assert!(x == 1 || x == 2);
}
```

Example: `ch3-03-total-modification-order.rs` **(03 Total Modification Order)**

```
use std::sync::atomic::AtomicI32;
use std::sync::atomic::Ordering::Relaxed;
use std::thread;

static X: AtomicI32 = AtomicI32::new(0);

fn a() {
    X.fetch_add(5, Relaxed);
    X.fetch_add(10, Relaxed);
}

fn b() {
    let a = X.load(Relaxed);
    let b = X.load(Relaxed);
    let c = X.load(Relaxed);
    let d = X.load(Relaxed);
    println!("{a} {b} {c} {d}");
}

fn main() {
    thread::scope(|s| {
        s.spawn(a);
```

```
        s.spawn(b);
    });
}
```

Example: `ch3-04-total-modification-order-2.rs` **(04 Total Modification Order 2)**

```
use std::sync::atomic::AtomicI32;
use std::sync::atomic::Ordering::Relaxed;
use std::thread;

static X: AtomicI32 = AtomicI32::new(0);

fn a1() {
    X.fetch_add(5, Relaxed);
}

fn a2() {
    X.fetch_add(10, Relaxed);
}

fn b() {
    let a = X.load(Relaxed);
    let b = X.load(Relaxed);
    let c = X.load(Relaxed);
    let d = X.load(Relaxed);
    println!("{a} {b} {c} {d}");
}

fn main() {
    thread::scope(|s| {
        s.spawn(a1);
        s.spawn(a2);
        s.spawn(b);
    });
}
```

Example: `ch3-05-out-of-thin-air.rs` **(05 Out Of Thin Air)**

```
use std::sync::atomic::AtomicI32;
use std::sync::atomic::Ordering::Relaxed;
use std::thread;

static X: AtomicI32 = AtomicI32::new(0);
static Y: AtomicI32 = AtomicI32::new(0);
```

```
fn main() {
    let a = thread::spawn(|| {
        let x = X.load(Relaxed);
        Y.store(x, Relaxed);
    });
    let b = thread::spawn(|| {
        let y = Y.load(Relaxed);
        X.store(y, Relaxed);
    });
    a.join().unwrap();
    b.join().unwrap();
    assert_eq!(X.load(Relaxed), 0); // Might fail?
    assert_eq!(Y.load(Relaxed), 0); // Might fail?
}
```

Example: `ch3-06-release-acquire.rs` (06 Release Acquire)

```
use std::sync::atomic::AtomicBool;
use std::sync::atomic::AtomicU64;
use std::sync::atomic::Ordering::Relaxed;
use std::sync::atomic::Ordering::{Acquire, Release};
use std::thread;
use std::time::Duration;

static DATA: AtomicU64 = AtomicU64::new(0);
static READY: AtomicBool = AtomicBool::new(false);

fn main() {
    thread::spawn(|| {
        DATA.store(123, Relaxed);
        READY.store(true, Release); // Everything from before this store ..
    });
    while !READY.load(Acquire) { // .. is visible after this loads `true`.
        thread::sleep(Duration::from_millis(100));
        println!("waiting...");
    }
    println!("{}", DATA.load(Relaxed));
}
```

Example: `ch3-07-release-acquire-unsafe.rs` (07 Release Acquire Unsafe)

```
use std::sync::atomic::AtomicBool;
use std::sync::atomic::Ordering::{Acquire, Release};
use std::thread;
use std::time::Duration;

static mut DATA: u64 = 0;
static READY: AtomicBool = AtomicBool::new(false);

fn main() {
    thread::spawn(|| {
        // Safety: Nothing else is accessing DATA,
        // because we haven't set the READY flag yet.
        unsafe { DATA = 123 };
        READY.store(true, Release); // Everything from before this store ..
    });
    while !READY.load(Acquire) { // .. is visible after this loads `true`.
        thread::sleep(Duration::from_millis(100));
        println!("waiting...");
    }
    // Safety: Nothing is mutating DATA, because READY is set.
    println!("{}", unsafe { DATA });
}
```

Example: `ch3-08-lock.rs` (08 Lock)

```
use std::sync::atomic::AtomicBool;
use std::sync::atomic::Ordering::{Acquire, Relaxed, Release};
use std::thread;

static mut DATA: String = String::new();
static LOCKED: AtomicBool = AtomicBool::new(false);

fn f() {
    if LOCKED.compare_exchange(false, true, Acquire, Relaxed).is_ok() {
        // Safety: We hold the exclusive lock, so nothing else is accessing DATA.
        unsafe { DATA.push('!') };
        LOCKED.store(false, Release);
    }
}

fn main() {
    thread::scope(|s| {
        for _ in 0..100 {
```

```
            s.spawn(f);
        }
    });
    // DATA now contains at least one exclamation mark (and maybe more).
    assert!(unsafe { DATA.len() } > 0);
    assert!(unsafe { DATA.chars().all(|c| c == '!') });
}
```

Example: `ch3-09-lazy-init-box.rs` (09 Lazy Init Box)

```
use std::sync::atomic::AtomicPtr;
use std::sync::atomic::Ordering::{Acquire, Release};

fn get_data() -> &'static Data {
    static PTR: AtomicPtr<Data> = AtomicPtr::new(std::ptr::null_mut());

    let mut p = PTR.load(Acquire);

    if p.is_null() {
        p = Box::into_raw(Box::new(generate_data()));
        if let Err(e) = PTR.compare_exchange(
            std::ptr::null_mut(), p, Release, Acquire
        ) {
            // Safety: p comes from Box::into_raw right above,
            // and wasn't shared with any other thread.
            drop(unsafe { Box::from_raw(p) });
            p = e;
        }
    }

    // Safety: p is not null and points to a properly initialized value.
    unsafe { &*p }
}

struct Data([u8; 100]);

fn generate_data() -> Data {
    Data([123; 100])
}

fn main() {
    println!("{:p}", get_data());
    println!("{:p}", get_data()); // Same address as before.
}
```

Example: `ch3-10-seqcst.rs` (10 Seqcst)

```
use std::sync::atomic::AtomicBool;
use std::sync::atomic::Ordering::SeqCst;
use std::thread;

static A: AtomicBool = AtomicBool::new(false);
static B: AtomicBool = AtomicBool::new(false);

static mut S: String = String::new();

fn main() {
    let a = thread::spawn(|| {
        A.store(true, SeqCst);
        if !B.load(SeqCst) {
            unsafe { S.push('!') };
        }
    });

    let b = thread::spawn(|| {
        B.store(true, SeqCst);
        if !A.load(SeqCst) {
            unsafe { S.push('!') };
        }
    });

    a.join().unwrap();
    b.join().unwrap();
}
```

Example: `ch3-11-fence.rs` (11 Fence)

```
use std::sync::atomic::fence;
use std::sync::atomic::AtomicBool;
use std::sync::atomic::Ordering::{Acquire, Relaxed, Release};
use std::thread;
use std::time::Duration;

static mut DATA: [u64; 10] = [0; 10];

const ATOMIC_FALSE: AtomicBool = AtomicBool::new(false);
static READY: [AtomicBool; 10] = [ATOMIC_FALSE; 10];

fn main() {
    for i in 0..10 {
```

```
        thread::spawn(move || {
            let data = some_calculation(i);
            unsafe { DATA[i] = data };
            READY[i].store(true, Release);
        });
    }
    thread::sleep(Duration::from_millis(500));
    let ready: [bool; 10] = std::array::from_fn(|i| READY[i].load(Relaxed));
    if ready.contains(&true) {
        fence(Acquire);
        for i in 0..10 {
            if ready[i] {
                println!("data{i} = {}", unsafe { DATA[i] });
            }
        }
    }
}

fn some_calculation(i: usize) -> u64 {
    thread::sleep(Duration::from_millis(400 + i as u64 % 3 * 100));
    123
}
```

Building Our Own Spin Lock

Building Our Own Spin Lock

Locking a regular mutex (see "Locking: Mutexes and RwLocks" in Chapter 1) will put your thread to sleep when the mutex is already locked. This avoids wasting resources while waiting for the lock to be released. If a lock is only ever held for very brief moments and the threads locking it can run in parallel on different processor cores, it might be better for the threads to repeatedly try to lock it without actually going to sleep.

A spin lock is a mutex that does exactly that. Attempting to lock an already locked mutex will result in *busy-looping* or *spinning*: repeatedly trying over and over again until it finally succeeds. This can waste processor cycles, but can sometimes result in lower latency when locking.

> **Note**

Many real-world implementations of mutexes, including `std::sync::Mutex` on some platforms, briefly behave like a spin lock before asking the operating system to put a thread to sleep. This is an attempt to combine the best of both worlds, although it depends entirely on the specific use case whether this behavior is beneficial or not.

In this chapter, we'll build our own `SpinLock` type, applying what we've learned in Chapters 2 and 3, and see how we can use Rust's type system to provide a safe and useful interface to the user of our `SpinLock`.

A Minimal Implementation

Let's implement such a spin lock from scratch.

The most minimal version is pretty simple, as follows:

```
pub struct SpinLock {
    locked: AtomicBool,
}
```

All we need is a single boolean that indicates whether it is locked or not. We use an *atomic* boolean, since we want more than one thread to be able to interact with it simultaneously.

Then all we need is a constructor function, and the `lock` and `unlock` methods:

```
impl SpinLock {
    pub const fn new() -> Self {
        Self { locked: AtomicBool::new(false) }
```

```
    }

    pub fn lock(&self) {
        while self.locked.swap(true, Acquire) {
            std::hint::spin_loop();
        }
    }

    pub fn unlock(&self) {
        self.locked.store(false, Release);
    }
}
```

The `locked` boolean starts at `false`, the `lock` swaps that for `true` and keeps trying if it was already `true`, and the `unlock` method just sets it back to `false`.

> **Note**

Instead of using a swap operation, we could also have used a compare-and-exchange operation to atomically check if the boolean is `false` and set it to `true` if that's the case:

```
    while self.locked.compare_exchange_weak(
            false, true, Acquire, Relaxed).is_err()
```

It's a bit more verbose, but depending on your tastes this might be easier to follow, as it more clearly captures the concept of an operation that can fail or succeed. It might also result in slightly different instructions, however, as we'll see in Chapter 7.

Within the `while` loop, we use a *spin loop hint*, which tells the processor that we're spinning while waiting for something to change. On most major platforms, this hint results in a special instruction that causes the processor core to optimize its behavior for such a situation. For example, it might temporarily slow down or prioritize other useful things it can do. Unlike blocking operations such as `thread::sleep` or `thread::park`, however, a spin loop hint does not cause the operating system to be called to put your thread to sleep in favor of another thread.

> **Note**

In general, it's a good idea to include such a hint in a spin loop. Depending on the situation, it might even be good to execute this hint several times before attempting to access the atomic variable again. If you care about the last few nanoseconds of performance and want to find the optimal strategy, you'll have to benchmark your specific use case. Unfortunately, the conclusions of such benchmarks can be highly dependent on the hardware, as we'll see in Chapter 7.

We use acquire and release memory ordering to make sure that every `unlock()` call establishes a happens-before relationship with the `lock()` calls that follow. In other words, to make sure that after locking it, we can safely assume that whatever happened during the last time it was locked

has already happened. This is the most classic use case of acquire and release ordering: acquiring and releasing a lock.

Figure 4-1 visualizes a situation where our `SpinLock` is used to protect access to some shared data, with two threads concurrently attempting to acquire the lock. Note how the unlock operation on the first thread forms a happens-before relationship with the lock operation on the second thread, which makes sure the threads cannot access the data concurrently.

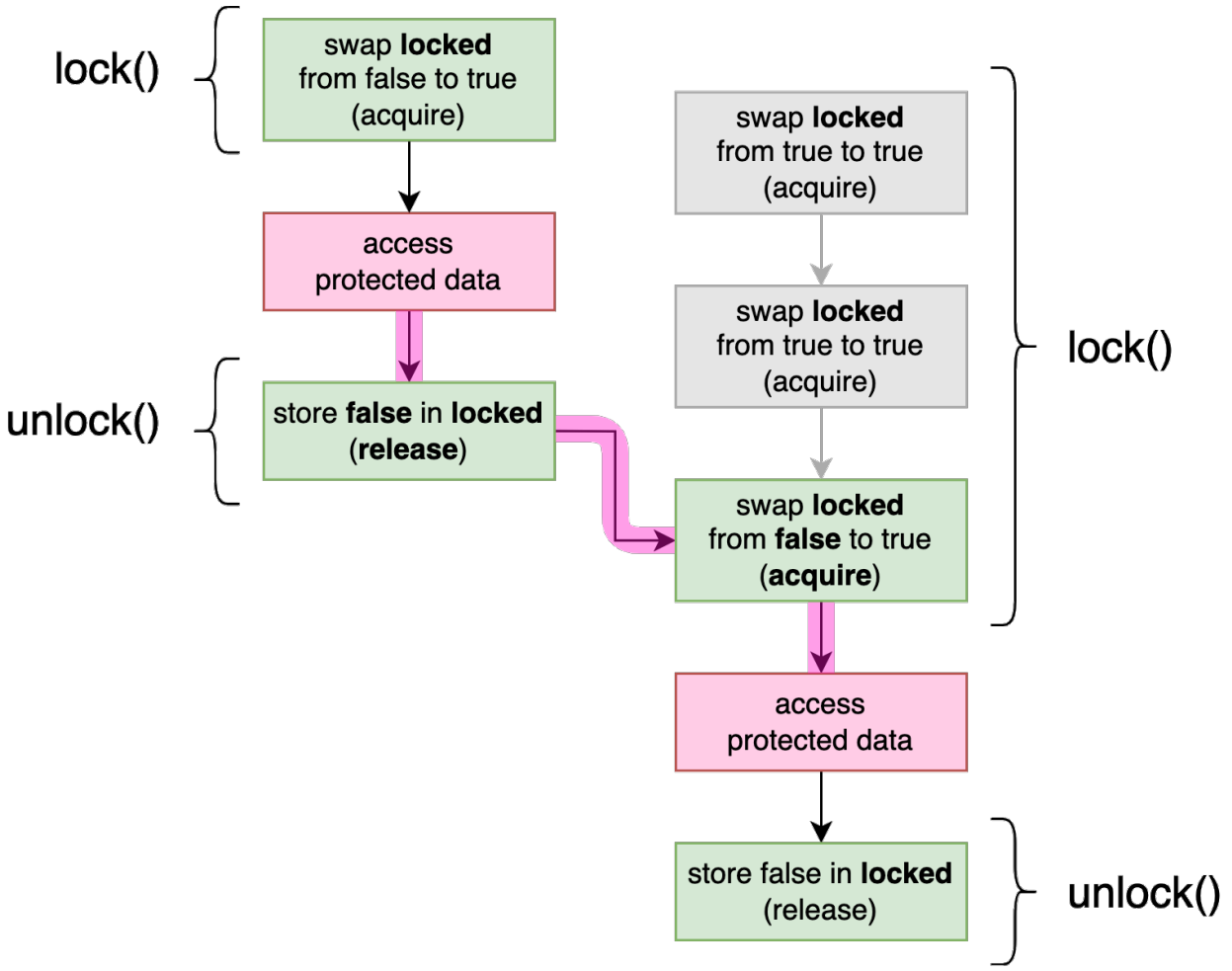


Figure 1: Figure 4-1. The happens-before relationships between two threads using our SpinLock to protect access to some shared data.

An Unsafe Spin Lock

Our `SpinLock` type above has a fully safe interface since, by itself, it doesn't cause any undefined behavior when misused. In most use cases, however, it will be used to protect mutations to a shared variable, which means the user will still have to use unsafe, unchecked code.

To provide an easier interface, we can change the `lock` method to give us an exclusive reference (`&mut T`) to the data protected by the lock, since in most use cases, it's the lock operation that guarantees that it's safe to assume exclusive access.

To be able to do that, we have to change the type to be generic over the type of data it protects and add a field to hold that data. Since this data can be mutated (or accessed exclusively) even though the spin lock itself is shared, we need to use interior mutability (see "Interior Mutability" in Chapter 1), for which we'll use an `UnsafeCell`:

```
use std::cell::UnsafeCell;

pub struct SpinLock<T> {
    locked: AtomicBool,
    value: UnsafeCell<T>,
}
```

As a precaution, `UnsafeCell` does not implement `Sync`, which means that our type is now no longer shareable between threads, making it rather useless. To fix that, we need to promise to the compiler that it is actually safe for our type to be shared between threads. However, since the lock can be used to send values of type `T` from one thread to another, we must limit this promise to types that are safe to send between threads. So, we (unsafely) implement `Sync` for `SpinLock<T>` for all `T` that implement `Send`, like this:

```
unsafe impl<T> Sync for SpinLock<T> where T: Send {}
```

Note that we don't need to require that `T` is `Sync`, because our `SpinLock<T>` will only allow one thread at a time to access the `T` it protects. Only if we were to give multiple threads access at once, like a reader-writer lock does for readers, would we (additionally) need to require `T: Sync`.

Next, our `new` function now needs to take a value of type `T` to initialize that `UnsafeCell` with:

```
impl<T> SpinLock<T> {
    pub const fn new(value: T) -> Self {
        Self {
            locked: AtomicBool::new(false),
            value: UnsafeCell::new(value),
        }
    }

    …
}
```

And then we get to the interesting part: `lock` and `unlock`. The reason we are doing all this, is to be able to return a `&mut T` from `lock()`, such that the user isn't required to write unsafe, unchecked code when using our lock to protect their data. This means that we now have to use unsafe code on our side, within the implementation of `lock`. An `UnsafeCell` can give us a raw pointer to its contents (`*mut T`) through its `get()` method, which we can convert to a reference within an `unsafe` block, as follows:

```
    pub fn lock(&self) -> &mut T {
        while self.locked.swap(true, Acquire) {
            std::hint::spin_loop();
```

```
        }
        unsafe { &mut *self.value.get() }
    }
```

Since the function signature of `lock` contains a reference both in its input and output, the lifetimes of the `&self` and `&mut T` have been elided and assumed to be identical. (See "Lifetime Elision" in "Chapter 10: Generic Types, Traits, and Lifetimes" of the Rust Book.) We can make the lifetimes explicit by writing them out manually, like this:

```
pub fn lock<'a>(&'a self) -> &'a mut T { … }
```

This clearly shows that the lifetime of the returned reference is the same as that of `&self`. This means that we have claimed that the returned reference is valid as long as the lock itself exists.

If we pretend `unlock()` doesn't exist, this would be a perfectly safe and sound interface. A `SpinLock` can be locked, resulting in a `&mut T`, and can then never be locked again, which guarantees this exclusive reference is indeed exclusive.

If we try to add the `unlock()` method back, however, we would need a way to limit the lifetime of the returned reference until the next call to `unlock()`. If the compiler understood English, perhaps this would work:

```
pub fn lock<'a>(&self) -> &'a mut T
where
    'a ends at the next call to unlock() on self,
    even if that's done by another thread.
    Oh, and it also ends when self is dropped, of course.
    (Thanks!)
{ … }
```

Unfortunately, that's not valid Rust. Instead of trying to explain this limitation to the compiler, we'll have to explain it to the user. To shift the responsibility to the user, we mark the `unlock` function as `unsafe`, and leave a note for them explaining what they need to do to keep things sound:

```
/// Safety: The &mut T from lock() must be gone!
/// (And no cheating by keeping references to fields of that T around!)
pub unsafe fn unlock(&self) {
    self.locked.store(false, Release);
}
```

A Safe Interface Using a Lock Guard

To be able to provide a fully safe interface, we need to tie the unlocking operation to the end of the `&mut T`. We can do that by wrapping this reference in our own type that behaves like a reference, but also implements the `Drop` trait to do something when it is dropped.

Such a type is often called a *guard*, as it effectively guards the state of the lock, and stays responsible for that state until it is dropped.

Our `Guard` type will simply contain a reference to the `SpinLock`, so that it can both access its `UnsafeCell` and reset the `AtomicBool` later:

```
pub struct Guard<T> {
    lock: &SpinLock<T>,
}
```

If we try to compile this, however, the compiler tells us:

```
error[E0106]: missing lifetime specifier
   --> src/lib.rs
    |
    |         lock: &SpinLock<T>,
    |               ^ expected named lifetime parameter
    |
help: consider introducing a named lifetime parameter
    |
    ~     pub struct Guard<'a, T> {
    |                        ^^^
    ~         lock: &'a SpinLock<T>,
    |                ^^
    |
```

Apparently, this is not a place where lifetimes can be elided. We have to make explicit that the reference has a limited lifetime, exactly as the compiler suggests:

```
pub struct Guard<'a, T> {
    lock: &'a SpinLock<T>,
}
```

This guarantees the `Guard` cannot outlive the `SpinLock`.

Next, we change the `lock` method on our `SpinLock` to return a `Guard`:

```
    pub fn lock(&self) -> Guard<T> {
        while self.locked.swap(true, Acquire) {
            std::hint::spin_loop();
        }
        Guard { lock: self }
    }
```

Our `Guard` type has no constructor and its field is private, so this is the only way the user can obtain a `Guard`. Therefore, we can safely assume that the existence of a `Guard` means that the `SpinLock` has been locked.

To make `Guard<T>` behave like an (exclusive) reference, transparently giving access to the `T`, we have to implement the special `Deref` and `DerefMut` traits as follows:

```rust
use std::ops::{Deref, DerefMut};

impl<T> Deref for Guard<'_, T> {
    type Target = T;
    fn deref(&self) -> &T {
        // Safety: The very existence of this Guard
        // guarantees we've exclusively locked the lock.
        unsafe { &*self.lock.value.get() }
    }
}

impl<T> DerefMut for Guard<'_, T> {
    fn deref_mut(&mut self) -> &mut T {
        // Safety: The very existence of this Guard
        // guarantees we've exclusively locked the lock.
        unsafe { &mut *self.lock.value.get() }
    }
}
```

If we don't manually implement `Send` and `Sync` for our `Guard` type, the compiler will do so automatically based on its fields. Since its only field is a `&SpinLock<T>`, the compiler will conclude that a `Guard<T>` can safely be shared between threads if a `SpinLock<T>` can. However, because of our (unsafe) `Deref` implementation above, our `Guard<T>` actually behaves like an (exclusive) reference to the value (`T`), not to the lock (`SpinLock<T>`).

So, we need to make sure our `Guard` is only `Sync` if `T` is `Sync` (and `Send` if `T` is `Send`). Otherwise, we would accidentally allow multiple threads to share a single `Guard<T>` and concurrently access the same `T`, even when `T` is not `Sync`.

There are several ways to do this. The easiest is to add our own implementation of `Send` and `Sync` with the right bounds, as follows:

```rust
unsafe impl<T> Send for Guard<'_, T> where T: Send {}
unsafe impl<T> Sync for Guard<'_, T> where T: Sync {}
```

> **Note**

Note that the `Send` implementation is technically unnecessary, since a `&SpinLock<T>` is also `Send` only if `T` is `Send`.

An alternative to adding these manual trait implementations is to add a `PhantomData<&mut T>` field to `Guard`, to inform the compiler it behaves like it has an exclusive reference to a `T`.

Another possibility is to replace the `&SpinLock<T>` field of the `Guard` by separate `&AtomicBool` and `&mut T` fields, to refer to the `locked` and `value` fields of the `SpinLock` separately. That way, the compiler can see the `&mut T` and will correctly implement `Send` and `Sync` for `Guard` based on that.

As the final step, we implement `Drop` for `Guard`, allowing us to completely remove the unsafe `unlock` method:

```
impl<T> Drop for Guard<'_, T> {
    fn drop(&mut self) {
        self.lock.locked.store(false, Release);
    }
}
```

And just like that, through the magic of `Drop` and Rust's type system, we gave our `SpinLock` type a fully safe (and useful) interface.

Let's try it out:

```
fn main() {
    let x = SpinLock::new(Vec::new());
    thread::scope(|s| {
        s.spawn(|| x.lock().push(1));
        s.spawn(|| {
            let mut g = x.lock();
            g.push(2);
            g.push(2);
        });
    });
    let g = x.lock();
    assert!(g.as_slice() == [1, 2, 2] || g.as_slice() == [2, 2, 1]);
}
```

The program above demonstrates how easy our `SpinLock` is to use. Thanks to `Deref` and `DerefMut`, we can directly call the `Vec::push` method on the guard. And thanks to `Drop`, we don't need to worry about unlocking.

Explicitly unlocking is also possible, by calling `drop(g)` to drop the guard. If you try to unlock too early, you'll see the guard doing its job through a compiler error. For example, if you insert `drop(g);` between the two `push(2)` lines, the second `push` will not compile, since you've already dropped `g` at that point:

```
error[E0382]: borrow of moved value: `g`
   --> src/lib.rs
    |
    |     drop(g);
    |          - value moved here
    |     g.push(2);
    |     ^^^^^^^^^ value borrowed here after move
```

Thanks to Rust's type system, we can rest assured that mistakes like this are caught before we can even run our program.

Summary

- A *spin lock* is a mutex that busy-loops, or spins, while waiting.
- Spinning can reduce latency, but can also be a waste of clock cycles and reduce performance.
- A *spin loop hint*, `std::hint::spin_loop()`, can be used to inform the processor of a spin loop, which might increase its efficiency.
- A `SpinLock<T>` can be implemented with just an `AtomicBool` and an `UnsafeCell<T>`, the latter of which is necessary for *interior mutability* (see "Interior Mutability" in Chapter 1).
- A *happens-before relationship* between unlock and lock operations is necessary to prevent a *data race*, which would result in *undefined behavior*.
- *Acquire and release memory ordering* are a perfect fit for this use case.
- When making unchecked assumptions necessary to avoid undefined behavior, the responsibility can be shifted to the caller by making the function `unsafe`.
- The `Deref` and `DerefMut` traits can be used to make a type behave like a reference, transparently providing access to another object.
- The `Drop` trait can be used to do something when an object is dropped, such as when it goes out of scope, or when it is passed to `drop()`.
- A *lock guard* is a useful design pattern of a special type that's used to represent (safe) access to a locked lock. Such a type usually behaves similarly to a reference, thanks to the `Deref` traits, and implements automatic unlocking through the `Drop` trait.

Complete Runnable Examples

The following complete, tested programs illustrate the primitives and patterns developed throughout this chapter.

Example: `s1_minimal.rs` (S1 Minimal)

```
use std::sync::atomic::AtomicBool;
use std::sync::atomic::Ordering::{Acquire, Release};

pub struct SpinLock {
    locked: AtomicBool,
}

impl SpinLock {
    pub const fn new() -> Self {
        Self { locked: AtomicBool::new(false) }
```

```
    }

    pub fn lock(&self) {
        while self.locked.swap(true, Acquire) {
            std::hint::spin_loop();
        }
    }

    pub fn unlock(&self) {
        self.locked.store(false, Release);
    }
}
```

Example: `s2_unsafe.rs` (S2 Unsafe)

```
use std::cell::UnsafeCell;
use std::sync::atomic::AtomicBool;
use std::sync::atomic::Ordering::{Acquire, Release};

pub struct SpinLock<T> {
    locked: AtomicBool,
    value: UnsafeCell<T>,
}

unsafe impl<T> Sync for SpinLock<T> where T: Send {}

impl<T> SpinLock<T> {
    pub const fn new(value: T) -> Self {
        Self {
            locked: AtomicBool::new(false),
            value: UnsafeCell::new(value),
        }
    }

    pub fn lock(&self) -> &mut T {
        while self.locked.swap(true, Acquire) {
            std::hint::spin_loop();
        }
        unsafe { &mut *self.value.get() }
    }

    /// Safety: The &mut T from lock() must be gone!
    /// (And no cheating by keeping reference to fields of that T around!)
    pub unsafe fn unlock(&self) {
        self.locked.store(false, Release);
```

```
    }
}
```

Example: `s3_guard.rs` (S3 Guard)

```
use std::ops::{Deref, DerefMut};
use std::cell::UnsafeCell;
use std::sync::atomic::AtomicBool;
use std::sync::atomic::Ordering::{Acquire, Release};

pub struct SpinLock<T> {
    locked: AtomicBool,
    value: UnsafeCell<T>,
}

unsafe impl<T> Sync for SpinLock<T> where T: Send {}

pub struct Guard<'a, T> {
    lock: &'a SpinLock<T>,
}

unsafe impl<T> Sync for Guard<'_, T> where T: Sync {}

impl<T> SpinLock<T> {
    pub const fn new(value: T) -> Self {
        Self {
            locked: AtomicBool::new(false),
            value: UnsafeCell::new(value),
        }
    }

    pub fn lock(&self) -> Guard<T> {
        while self.locked.swap(true, Acquire) {
            std::hint::spin_loop();
        }
        Guard { lock: self }
    }
}

impl<T> Deref for Guard<'_, T> {
    type Target = T;
    fn deref(&self) -> &T {
        // Safety: The very existence of this Guard
        // guarantees we've exclusively locked the lock.
        unsafe { &*self.lock.value.get() }
```

```
    }
}

impl<T> DerefMut for Guard<'_, T> {
    fn deref_mut(&mut self) -> &mut T {
        // Safety: The very existence of this Guard
        // guarantees we've exclusively locked the lock.
        unsafe { &mut *self.lock.value.get() }
    }
}

impl<T> Drop for Guard<'_, T> {
    fn drop(&mut self) {
        self.lock.locked.store(false, Release);
    }
}

#[test]
fn main() {
    use std::thread;
    let x = SpinLock::new(Vec::new());
    thread::scope(|s| {
        s.spawn(|| x.lock().push(1));
        s.spawn(|| {
            let mut g = x.lock();
            g.push(2);
            g.push(2);
        });
    });
    let g = x.lock();
    assert!(g.as_slice() == [1, 2, 2] || g.as_slice() == [2, 2, 1]);
}
```

Building Our Own Channels

Building Our Own Channels

Channels can be used to send data between threads, and they come in many variants. Some channels can only be used between exactly one sender and one receiver, while others can send from any number of threads, or even allow multiple receivers. Some channels are blocking, meaning that receiving (and sometimes sending) is a blocking operation, making your thread sleep until the operation can be completed. Some channels are optimized for throughput, while others are optimized for low latency.

The variations are endless, and there is no one-size-fits-all version that fits all use cases.

In this chapter, we'll implement a few relatively simple channels to not only explore some more applications of atomics, but also to learn more about how our requirements and assumptions can be captured in Rust's type system.

A Simple Mutex-Based Channel

A basic channel implementation does not require any knowledge of atomics. We can take a `VecDeque`, which is basically a `Vec` that allows for efficient adding and removing of elements on both ends, and protect it with a `Mutex` to allow multiple threads to access it. We then use the `VecDeque` as a queue of data, often called *messages*, that's been sent but not yet received. Any thread that wants to send a message can simply add it to the back of the queue, and any thread that wants to receive a message just has to remove one from the front of the queue.

There's just one more thing to add, which is used to make the receive operation blocking: a `Condvar` (see "Condition Variables" in Chapter 1) to notify waiting receivers of a new message.

An implementation of this can be quite short and relatively straightforward, as shown below:

```
pub struct Channel<T> {
    queue: Mutex<VecDeque<T>>,
    item_ready: Condvar,
}

impl<T> Channel<T> {
    pub fn new() -> Self {
        Self {
            queue: Mutex::new(VecDeque::new()),
            item_ready: Condvar::new(),
        }
    }
```

```
    pub fn send(&self, message: T) {
        self.queue.lock().unwrap().push_back(message);
        self.item_ready.notify_one();
    }

    pub fn receive(&self) -> T {
        let mut b = self.queue.lock().unwrap();
        loop {
            if let Some(message) = b.pop_front() {
                return message;
            }
            b = self.item_ready.wait(b).unwrap();
        }
    }
}
```

Note how we didn't have to use any atomics or unsafe code and didn't have to think about the `Send` or `Sync` traits. The compiler understands the interface of `Mutex` and what guarantees that type provides, and will implicitly understand that if both `Mutex<T>` and `Condvar` can safely be shared between threads, so can our `Channel<T>`.

Our `send` function locks the mutex to push the new message to the back of the queue, and directly notifies one potentially waiting receiver after unlocking the queue, by using the condition variable.

The `receive` function also locks the mutex to pop the next message from the front of the queue, but will use the condition variable to wait if there's no message available yet.

> **Note**

Remember that the `Condvar::wait` method will unlock the `Mutex` while waiting and relock it before returning. So, our `receive` function will not keep the mutex locked while waiting.

While this channel is very flexible in usage, as it allows any number of sending and receiving threads, its implementation can be far from optimal in many situations. Even if there are plenty of messages ready to be received, any send or receive operation will briefly block any other send or receive operation, since they all have to lock the same mutex. If `VecDeque::push_back` has to grow the capacity of the `VecDeque`, all sending and receiving threads will have to wait for that one thread to finish the reallocation, which might be unacceptable in some situations.

Another property which might be undesirable is that this channel's queue might grow without bounds. Nothing is stopping senders from continuously sending new messages at a higher rate than receivers are processing them.

An Unsafe One-Shot Channel

The variety of use cases for channels is virtually endless. However, in the rest of this chapter, we'll focus on a specific type of use case: sending exactly one message from one thread to another. A

channel designed for such a use case is often called a *one-shot channel*.

We could take our `Mutex<VecDeque>` based implementation from above and replace the `VecDeque` with an `Option`, effectively reducing the capacity of the queue to exactly one message. It would avoid allocation, but would still have some of the same downsides of using a `Mutex`. We can avoid this by building our own one-shot channel from scratch using atomics.

First, let's build a minimal implementation of a one-shot channel without putting much thought into its interface. Later in this chapter, we'll explore ways to improve its interface, and how to team up with Rust's type system to provide users of our channel a pleasant experience.

The tools we need to start with are basically the same as we used for our `SpinLock<T>` (from Chapter 4): an `UnsafeCell` for storage and an `AtomicBool` to indicate its state. In this case, we use the atomic boolean to indicate whether the message is ready for consumption.

Before a message is sent, the channel is "empty" and does not contain any message of type `T` yet. We could use an `Option<T>` inside the cell to allow for the absence of a `T`. However, that could waste valuable space in memory, since our atomic boolean already tells us whether there is a message or not. Instead, we can use a `std::mem::MaybeUninit<T>`, which is essentially the bare-bones unsafe version of `Option<T>`: it requires its user to manually keep track of whether it has been initialized or not, and almost its entire interface is unsafe, as it can't perform its own checks.

Putting that all together, we start our first attempt with this struct definition:

```
use std::mem::MaybeUninit;

pub struct Channel<T> {
    message: UnsafeCell<MaybeUninit<T>>,
    ready: AtomicBool,
}
```

Just like for our `SpinLock<T>`, we need to tell the compiler that our channel is safe to share between threads, or at least as long as `T` is `Send`:

```
unsafe impl<T> Sync for Channel<T> where T: Send {}
```

A new channel is empty, with `ready` set to `false`, and `message` left uninitialized:

```
impl<T> Channel<T> {
    pub const fn new() -> Self {
        Self {
            message: UnsafeCell::new(MaybeUninit::uninit()),
            ready: AtomicBool::new(false),
        }
    }

    …
}
```

To send a message, it first needs to be stored in the cell, after which we can release it to the receiver by setting the `ready` flag to `true`. Attempting to do this more than once would be dangerous, since after setting the `ready` flag, the receiver might read the message at any point, which could race with a second attempt to send a message. For now, we make this the responsibility of the user by making the method `unsafe` and leaving a note for them:

```
    /// Safety: Only call this once!
    pub unsafe fn send(&self, message: T) {
        (*self.message.get()).write(message);
        self.ready.store(true, Release);
    }
```

In the snippet above, we use the `UnsafeCell::get` method to obtain a pointer to the `MaybeUninit<T>`, and unsafely dereference that to call `MaybeUninit::write` to initialize it. This could result in undefined behavior when misused, but we've punted that responsibility over to the caller.

For the memory ordering, we need to use release ordering, since the atomic store effectively releases the message to the receiver. This makes sure the initialization of the message will be finished from the perspective of the receiving thread if it loads `true` from `self.ready` with acquire ordering.

For receiving, we'll not bother with providing a blocking interface for now. Instead, we'll provide two methods: one to check whether a message is available, and another to receive it. We'll leave it to the user of our channel to use something like thread parking ("Thread Parking" in Chapter 1) if they want to block.

These are the last two methods to complete this version of our channel:

```
    pub fn is_ready(&self) -> bool {
        self.ready.load(Acquire)
    }

    /// Safety: Only call this once,
    /// and only after is_ready() returns true!
    pub unsafe fn receive(&self) -> T {
        (*self.message.get()).assume_init_read()
    }
```

While the `is_ready` method can always be called safely, the `receive` method uses `MaybeUninit::assume_init_rea` which unsafely assumes it has already been initialized and that it isn't being used to produce multiple copies of non-`Copy` objects. Just like for `send`, we simply make that our user's problem by making the function itself `unsafe`.

The result is a technically usable channel, but one that is unwieldy and generally disappointing. If held right, it does exactly what it is supposed to do, but there are many subtle ways to misuse it.

Calling `send` more than once might result in a data race, since the second sender will be overwriting the data while the receiver might be trying to read the first message. Even if receiving was properly synchronized, calling `send` from multiple threads might result in two threads attempting to concurrently write to the cell, again resulting in a data race. Also, calling `receive` more than

once results in two copies of the message, even if `T` does not implement `Copy` and thus cannot safely be copied.

A more subtle issue is the lack of a `Drop` implementation for our channel. The `MaybeUninit` type does not track whether it has been initialized or not, and will therefore not automatically drop its contents when dropped. This means that if a message is sent but never received, the message will never be dropped. This is not unsound, but it's still something to avoid. While leaking is universally considered safe in Rust, it's generally only acceptable as a consequence of another leak. For example, leaking a `Vec` also leaks its contents, but regular usage of a `Vec` does not result in any leaks.

Since we made the user responsible for everything, it's only a matter of time before this results in an unfortunate accident.

Safety Through Runtime Checks

To allow for a safer interface, we can add some checks to make misuse result in a panic with a clear message, which is much preferable to undefined behavior.

Let's start with the issue of calling `receive` before a message is ready. This one is simple to handle, as all we have to do is make the `receive` method validate the `ready` flag before attempting to read a message:

```
    /// Panics if no message is available yet.
    ///
    /// Tip: Use `is_ready` to check first.
    ///
    /// Safety: Only call this once!
    pub unsafe fn receive(&self) -> T {
        if !self.ready.load(Acquire) {
            panic!("no message available!");
        }
        (*self.message.get()).assume_init_read()
    }
```

The function is still unsafe, as the user is still responsible for not calling this function more than once, but failing to check `is_ready()` first no longer results in undefined behavior.

Since we now have an acquire-load of the `ready` flag inside the `receive` method providing the necessary synchronization, we can lower the memory ordering of the load in `is_ready` to `Relaxed`, since that one is now only used for indicative purposes:

```
    pub fn is_ready(&self) -> bool {
        self.ready.load(Relaxed)
    }
```

> **Note**

Remember that the *total modification order* (see "Relaxed Ordering" in Chapter 3) on `ready` guarantees that after `is_ready` loads `true` from it, `receive` will also see `true`. There is no possibility of `is_ready` returning `true` and `receive()` still panicking, regardless of the memory ordering used in `is_ready`.

The next issue to address is what happens when calling `receive` more than once. We can easily make that result in a panic as well by setting the `ready` flag back to `false` in our `receive` method, like this:

```
    /// Panics if no message is available yet,
    /// or if the message was already consumed.
    ///
    /// Tip: Use `is_ready` to check first.
    pub fn receive(&self) -> T {
        if !self.ready.swap(false, Acquire) {
            panic!("no message available!");
        }
        // Safety: We've just checked (and reset) the ready flag.
        unsafe { (*self.message.get()).assume_init_read() }
    }
```

We've simply changed the `load` for a `swap` to `false`, and suddenly the `receive` method is fully safe to call in any condition. The function is no longer marked as `unsafe`. Instead of making the user responsible for everything, we now take responsibility for the unsafe code, resulting in less stress for our user.

For `send`, things are slightly more complicated. To prevent multiple `send` calls from accessing the cell at the same time, we need to know whether another `send` call has already started. The `ready` flag only tells us whether another `send` call has already finished, so that won't suffice.

Let's add a second flag, named `in_use`, to indicate whether the channel has been taken in use:

```
pub struct Channel<T> {
    message: UnsafeCell<MaybeUninit<T>>,
    in_use: AtomicBool, // New!
    ready: AtomicBool,
}

impl<T> Channel<T> {
    pub const fn new() -> Self {
        Self {
            message: UnsafeCell::new(MaybeUninit::uninit()),
            in_use: AtomicBool::new(false), // New!
            ready: AtomicBool::new(false),
        }
```

```
    }

    …
}
```

Now all we need to do is set `in_use` to `true` in the `send` method before accessing the cell and panic if it was already set by another call:

```
    /// Panics when trying to send more than one message.
    pub fn send(&self, message: T) {
        if self.in_use.swap(true, Relaxed) {
            panic!("can't send more than one message!");
        }
        unsafe { (*self.message.get()).write(message) };
        self.ready.store(true, Release);
    }
```

We can use relaxed memory ordering for the atomic swap operation, because the *total modification order* (see "Relaxed Ordering" in Chapter 3) of `in_use` guarantees that there will only be a single swap operation on `in_use` that will return `false`, which is the only case in which `send` will attempt to access the cell.

We now have a fully safe interface, though there is still one problem left. The last remaining issue occurs when sending a message that's never received: it will never be dropped. While this does not result in undefined behavior and is allowed in safe code, it's definitely something to avoid.

Since we reset the `ready` flag in the `receive` method, fixing this is easy: the `ready` flag indicates whether there's a not-yet-received message in the cell that needs to be dropped.

In the `Drop` implementation of our `Channel`, we don't need to use an atomic operation to check the atomic `ready` flag, because an object can only be dropped if it is fully owned by whichever thread is dropping it, with no outstanding borrows. This means we can use the `AtomicBool::get_mut` method, which takes an exclusive reference (`&mut self`), proving that atomic access is unnecessary. The same holds for `UnsafeCell`, through `UnsafeCell::get_mut`.

Using that, here's the final piece of our fully safe and non-leaking channel:

```
impl<T> Drop for Channel<T> {
    fn drop(&mut self) {
        if *self.ready.get_mut() {
            unsafe { self.message.get_mut().assume_init_drop() }
        }
    }
}
```

Let's try it out!

Since our `Channel` doesn't provide a blocking interface (yet), we'll manually use thread parking to wait on a message. The receiving thread will `park()` itself as long as there's no message ready, and the sending thread will `unpark()` the receiver once it has sent something.

Here's a complete test program, sending the string literal `"hello world!"` through our `Channel` from a second thread back to the main thread:

```
fn main() {
    let channel = Channel::new();
    let t = thread::current();
    thread::scope(|s| {
        s.spawn(|| {
            channel.send("hello world!");
            t.unpark();
        });
        while !channel.is_ready() {
            thread::park();
        }
        assert_eq!(channel.receive(), "hello world!");
    });
}
```

This program compiles, runs, and exits cleanly, showing that our `Channel` works as it should.

If we duplicate the `send` line, we can also see one of our safety checks in action, producing the following panic message when the program is run:

```
thread '<unnamed>' panicked at src/main.rs:
can't send more than one message!
```

While a panicking program isn't great, it's far better for a program to reliably panic than to get anywhere near the potential horrors of undefined behavior.

Using a Single Atomic for the Channel State

In case you can't get enough of implementing channels, here's a subtle variation that can save one byte of memory.

Instead of using two separate atomic booleans to represent the state of the channel, we instead use a single `AtomicU8` to represent all four states. Rather than atomically swapping booleans, we'll have to use `compare_exchange` to atomically check if the channel is in the expected state and change it to another state.

```
const EMPTY: u8 = 0;
const WRITING: u8 = 1;
const READY: u8 = 2;
const READING: u8 = 3;

pub struct Channel<T> {
    message: UnsafeCell<MaybeUninit<T>>,
    state: AtomicU8,
}
```

```
unsafe impl<T: Send> Sync for Channel<T> {}

impl<T> Channel<T> {
    pub const fn new() -> Self {
        Self {
            message: UnsafeCell::new(MaybeUninit::uninit()),
            state: AtomicU8::new(EMPTY),
        }
    }

    pub fn send(&self, message: T) {
        if self.state.compare_exchange(
            EMPTY, WRITING, Relaxed, Relaxed
        ).is_err() {
            panic!("can't send more than one message!");
        }
        unsafe { (*self.message.get()).write(message) };
        self.state.store(READY, Release);
    }

    pub fn is_ready(&self) -> bool {
        self.state.load(Relaxed) == READY
    }

    pub fn receive(&self) -> T {
        if self.state.compare_exchange(
            READY, READING, Acquire, Relaxed
        ).is_err() {
            panic!("no message available!");
        }
        unsafe { (*self.message.get()).assume_init_read() }
    }
}

impl<T> Drop for Channel<T> {
    fn drop(&mut self) {
        if *self.state.get_mut() == READY {
            unsafe { self.message.get_mut().assume_init_drop() }
        }
    }
}
```

Safety Through Types

While we've successfully protected users of our `Channel` from undefined behavior, they still risk a panic if they accidentally use it incorrectly. Ideally, the compiler would check correct usage and point out misuse before the program is even run.

Let's take a look at the issue of calling `send` or `receive` more than once.

To prevent a function from being called more than once, we can let it take an argument *by value*, which—for non-`Copy` types—will consume the object. After an object is consumed, or moved, it's gone from the caller, preventing it from being used another time.

By representing the ability to call `send` or `receive` each as a separate (non-`Copy`) type, and consuming that object when performing the operation, we can make sure each can only happen once.

This brings us to the following interface design, where instead of a single `Channel` type, a channel is represented by a pair of a `Sender` and a `Receiver`, which each have a method that takes `self` by value:

```
pub fn channel<T>() -> (Sender<T>, Receiver<T>) { … }

pub struct Sender<T> { … }
pub struct Receiver<T> { … }

impl<T> Sender<T> {
    pub fn send(self, message: T) { … }
}

impl<T> Receiver<T> {
    pub fn is_ready(&self) -> bool { … }
    pub fn receive(self) -> T { … }
}
```

The user can create a channel by calling `channel()`, which will give them one `Sender` and one `Receiver`. They can freely pass each of these objects around, move them to another thread, and so on. However, they cannot end up with multiple copies of either of them, guaranteeing that `send` and `receive` can each only be called once.

To implement this, we need to find a place for our `UnsafeCell` and `AtomicBool`. Previously, we just had a single struct with those fields, but now we have two separate structs, each of which could outlive the other.

Since the sender and receiver will need to share the ownership of those variables, we'll use an `Arc` ("Reference Counting" in Chapter 1) to provide us with a reference-counted shared allocation, in which we store the shared `Channel` object. As shown below, the `Channel` type does not have to be public, as its existence is just an implementation detail irrelevant to the user.

```
pub struct Sender<T> {
    channel: Arc<Channel<T>>,
}
```

```
pub struct Receiver<T> {
    channel: Arc<Channel<T>>,
}

struct Channel<T> { // no longer `pub`
    message: UnsafeCell<MaybeUninit<T>>,
    ready: AtomicBool,
}

unsafe impl<T> Sync for Channel<T> where T: Send {}
```

Just like before, we implement `Sync` for `Channel<T>` on the condition that `T` is `Send`, to allow it to be used across threads.

Note how we no longer need the `in_use` atomic boolean like we did in our previous channel implementation. It was only used by `send` to check that it hadn't been called more than once, which is now statically guaranteed through the type system.

The `channel` function to create a channel and sender-receiver pair looks similar to the `Channel::new` function we had previously, except it wraps the `Channel` in an `Arc`, and wraps that `Arc` and a clone of it in the `Sender` and `Receiver` types:

```
pub fn channel<T>() -> (Sender<T>, Receiver<T>) {
    let a = Arc::new(Channel {
        message: UnsafeCell::new(MaybeUninit::uninit()),
        ready: AtomicBool::new(false),
    });
    (Sender { channel: a.clone() }, Receiver { channel: a })
}
```

The `send`, `is_ready`, and `receive` methods are basically identical to the ones we implemented before, with a few differences:

- They are now moved to their respective type, such that only the (one single) sender can send, and only the (one single) receiver can receive.
- `send` and `receive` now take `self` by value rather than by reference, to make sure they can each only be called once.
- `send` can no longer panic, as its precondition (only being called once) is now statically guaranteed.

So, they now look like this:

```
impl<T> Sender<T> {
    /// This never panics. :)
    pub fn send(self, message: T) {
        unsafe { (*self.channel.message.get()).write(message) };
        self.channel.ready.store(true, Release);
```

```
    }
}

impl<T> Receiver<T> {
    pub fn is_ready(&self) -> bool {
        self.channel.ready.load(Relaxed)
    }

    pub fn receive(self) -> T {
        if !self.channel.ready.swap(false, Acquire) {
            panic!("no message available!");
        }
        unsafe { (*self.channel.message.get()).assume_init_read() }
    }
}
```

The `receive` function can still panic, since the user might still call it before `is_ready()` returns `true`. It also still uses `swap` to set the `ready` flag back to `false` (instead of just `load`), so that the `Drop` implementation of `Channel` knows whether there's an unread message that needs to be dropped.

That `Drop` implementation is exactly the same as the one we implemented before:

```
impl<T> Drop for Channel<T> {
    fn drop(&mut self) {
        if *self.ready.get_mut() {
            unsafe { self.message.get_mut().assume_init_drop() }
        }
    }
}
```

The `Drop` implementation of `Arc<Channel<T>>` will decrement the reference counter of the allocation when either the `Sender<T>` or `Receiver<T>` is dropped. When dropping the second one, that counter reaches zero, and the `Channel<T>` itself is dropped. That will invoke our `Drop` implementation above, where we get to drop the message if one was sent but not received.

Let's try it out:

```
fn main() {
    thread::scope(|s| {
        let (sender, receiver) = channel();
        let t = thread::current();
        s.spawn(move || {
            sender.send("hello world!");
            t.unpark();
        });
        while !receiver.is_ready() {
            thread::park();
```

```
        }
        assert_eq!(receiver.receive(), "hello world!");
    });
}
```

It's a bit inconvenient that we still have to manually use thread parking to wait on a message, but we'll deal with that problem later.

Our goal, for now, was to make at least one form of misuse impossible at compile time. Unlike last time, attempting to send twice does not result in a program that panics, but instead does not result in a valid program at all. If we add another `send` call to the working program above, the compiler now catches the issue and patiently informs us of our mistake:

```
error[E0382]: use of moved value: `sender`
  --> src/main.rs
   |
   |             sender.send("hello world!");
   |                    --------------------
   |                     `sender` moved due to this method call
   |
   |             sender.send("second message");
   |             ^^^^^^ value used here after move
   |
note: this function takes ownership of the receiver `self`, which moves `sender`
  --> src/lib.rs
   |
   |     pub fn send(self, message: T) {
   |                 ^^^^
   = note: move occurs because `sender` has type `Sender<&str>`,
           which does not implement the `Copy` trait
```

Depending on the situation, it can be extremely tricky to design an interface that catches mistakes at compile time. If the situation does lend itself to such an interface, it can result not only in more convenience for the user, but also in a reduced number of runtime checks for things that are now statically guaranteed. We no longer needed the `in_use` flag, and removed the `swap` and check from the `send` method, for example.

Unfortunately, new problems may arise that could lead to more runtime overhead. In this case, the problem was the split ownership, for which we had to reach for an `Arc` and pay the cost of an allocation.

Having to make trade-offs between safety, convenience, flexibility, simplicity, and performance is unfortunate, but sometimes unavoidable. Rust generally strives to make it easy to excel at all of these, but sometimes makes you trade a bit of one to maximize another.

Borrowing to Avoid Allocation

The `Arc`-based channel implementation we just designed is very convenient to use—at the cost of some performance, since it has to allocate memory. If we want to optimize for efficiency, we can trade some convenience for performance by making the user responsible for the shared `Channel` object. Instead of taking care of the allocation and ownership of the `Channel` behind the scenes, we can force the user to create a `Channel` that can be *borrowed* by the `Sender` and `Receiver`. That way, they can choose to simply put that `Channel` in a local variable, avoiding the overhead of allocating memory.

We will also have to trade in some simplicity, since we will now have to deal with borrowing and lifetimes.

So, the three types will now look as follows, with `Channel` public again, and `Sender` and `Receiver` borrowing it for a certain lifetime.

```
pub struct Channel<T> {
    message: UnsafeCell<MaybeUninit<T>>,
    ready: AtomicBool,
}

unsafe impl<T> Sync for Channel<T> where T: Send {}

pub struct Sender<'a, T> {
    channel: &'a Channel<T>,
}

pub struct Receiver<'a, T> {
    channel: &'a Channel<T>,
}
```

Instead of a `channel()` function to create a `(Sender, Receiver)` pair, we move back to the `Channel::new` we had earlier in this chapter, allowing the user to create such an object as a local variable.

In addition, we need a way for the user to create a `Sender` and `Receiver` object that will borrow the `Channel`. This will need to be an exclusive borrow (`&mut Channel`), to make sure there can't be multiple senders or receivers for the same channel. By providing both the `Sender` and the `Receiver` at the same time, we can *split* the exclusive borrow into two shared borrows, such that both the sender and receiver can reference the channel, while preventing anything else from touching the channel.

This leads us to the following implementation:

```
impl<T> Channel<T> {
    pub const fn new() -> Self {
        Self {
            message: UnsafeCell::new(MaybeUninit::uninit()),
            ready: AtomicBool::new(false),
```

```
        }
    }

    pub fn split<'a>(&'a mut self) -> (Sender<'a, T>, Receiver<'a, T>) {
        *self = Self::new();
        (Sender { channel: self }, Receiver { channel: self })
    }
}
```

The `split` method, with its somewhat complicated signature, warrants a closer look. It exclusively borrows `self` through an exclusive reference, but it splits that into two shared references, wrapped in the `Sender` and `Receiver` types. The `'a` lifetime makes it clear that both of those objects borrow something with a limited lifetime; in this case, the `Channel` itself. Since the `Channel` is exclusively borrowed, the caller will not be able to borrow or move it as long as the `Sender` or `Receiver` object exists.

Once those objects both cease to exist, however, the mutable borrow expires and the compiler happily lets the `Channel` object be borrowed again by a second call to `split()`. While we can assume `split()` cannot be called again while the `Sender` and `Receiver` still exist, we cannot prevent a second call to `split()` after those objects are dropped or forgotten. We need to make sure we don't accidentally create a new `Sender` or `Receiver` object for a channel that already has its `ready` flag set, since that would break the assumptions that prevent undefined behavior.

By overwriting `*self` with a new empty channel in `split()`, we make sure it's in the expected state when creating the `Sender` and `Receiver` objects. This also invokes the `Drop` implementation on the old `*self`, which will take care of dropping a message that was previously sent but not received.

> **Note**

Since the lifetime in the signature of `split` comes from `self`, it can be elided. The signature of `split` in the snippet above is identical to this less verbose version:

```
pub fn split(&mut self) -> (Sender<T>, Receiver<T>) { … }
```

While this version doesn't explicitly show that the returned objects borrow `self`, the compiler still checks correct usage of the lifetime exactly the same as it does with the more verbose version.

The remaining methods and `Drop` implementation are the same as in our `Arc`-based implementation, except for an additional `'_` lifetime argument to the `Sender` and `Receiver` types. (If you forget those, the compiler will helpfully suggest adding them.)

For completeness, here's the remaining code:

```
impl<T> Sender<'_, T> {
    pub fn send(self, message: T) {
        unsafe { (*self.channel.message.get()).write(message) };
        self.channel.ready.store(true, Release);
    }
```

```
}

impl<T> Receiver<'_, T> {
    pub fn is_ready(&self) -> bool {
        self.channel.ready.load(Relaxed)
    }

    pub fn receive(self) -> T {
        if !self.channel.ready.swap(false, Acquire) {
            panic!("no message available!");
        }
        unsafe { (*self.channel.message.get()).assume_init_read() }
    }
}

impl<T> Drop for Channel<T> {
    fn drop(&mut self) {
        if *self.ready.get_mut() {
            unsafe { self.message.get_mut().assume_init_drop() }
        }
    }
}
```

Let's test it!

```
fn main() {
    let mut channel = Channel::new();
    thread::scope(|s| {
        let (sender, receiver) = channel.split();
        let t = thread::current();
        s.spawn(move || {
            sender.send("hello world!");
            t.unpark();
        });
        while !receiver.is_ready() {
            thread::park();
        }
        assert_eq!(receiver.receive(), "hello world!");
    });
}
```

The reduction in convenience compared to the `Arc`-based version is quite minimal: we only needed one more line to manually create a `Channel` object. Note, however, how the channel has to be created before the scope, to prove to the compiler that its existence will outlast both the sender and receiver.

To see the compiler's borrow checker in action, try adding a second call to `channel.split()` in various places. You'll see that calling it a second time within the thread scope results in an error,

while calling it after the scope is acceptable. Even calling `split()` before the scope is fine, as long as you stop using the returned `Sender` and `Receiver` before the scope starts.

Blocking

Let's finally deal with the last remaining major inconvenience of our `Channel`, the lack of a blocking interface. We've already used thread parking every time we tested a new variant of our channel. It shouldn't be too hard to integrate that pattern into the channel itself.

To be able to unpark the receiver, the sender needs to know which thread to unpark. The `std::thread::Thread` type represents a handle to a thread and is exactly what we need for calling `unpark()`. We'll store the handle to the receiving thread inside the `Sender` object, as follows:

```
use std::thread::Thread;

pub struct Sender<'a, T> {
    channel: &'a Channel<T>,
    receiving_thread: Thread, // New!
}
```

This handle would refer to the wrong thread, however, if the `Receiver` object is sent between threads. The `Sender` would be unaware of that and would still refer to the thread that originally held the `Receiver`.

We can handle that problem by making the `Receiver` a bit more restrictive, by not allowing it to be sent between threads anymore. As discussed in "Thread Safety: Send and Sync" in Chapter 1, we can use the special `PhantomData` marker type to add this restriction to our struct. A `PhantomData<*const ()>` does the job, since a raw pointer, such as `*const ()`, does not implement `Send`:

```
pub struct Receiver<'a, T> {
    channel: &'a Channel<T>,
    _no_send: PhantomData<*const ()>, // New!
}
```

Next, we'll have to modify the `Channel::split` method to fill in the new fields, like this:

```
    pub fn split<'a>(&'a mut self) -> (Sender<'a, T>, Receiver<'a, T>) {
        *self = Self::new();
        (
            Sender {
                channel: self,
                receiving_thread: thread::current(), // New!
            },
            Receiver {
                channel: self,
                _no_send: PhantomData, // New!
```

```
            }
        )
    }
```

We use the handle to the current thread for the `receiving_thread` field, since the `Receiver` object we return will stay on the current thread.

The `send` method doesn't change much, as shown below. We only have to call `unpark()` on the `receiving_thread` to wake up the receiver in case it is waiting:

```
impl<T> Sender<'_, T> {
    pub fn send(self, message: T) {
        unsafe { (*self.channel.message.get()).write(message) };
        self.channel.ready.store(true, Release);
        self.receiving_thread.unpark(); // New!
    }
}
```

The `receive` function undergoes a slightly larger change. The new version won't panic if there's no message yet, but will instead patiently wait for a message using `thread::park()` and try again, as many times as necessary.

```
impl<T> Receiver<'_, T> {
    pub fn receive(self) -> T {
        while !self.channel.ready.swap(false, Acquire) {
            thread::park();
        }
        unsafe { (*self.channel.message.get()).assume_init_read() }
    }
}
```

> **Note**

Remember that `thread::park()` might return spuriously. (Or because something other than our `send` method called `unpark()`.) This means that we cannot assume that the `ready` flag has been set when `park()` returns. So, we need to use a loop to check the flag again after getting unparked.

The `Channel<T>` struct, its `Sync` implementation, its `new` function, and its `Drop` implementation remain unchanged.

Let's try it out!

```
fn main() {
    let mut channel = Channel::new();
    thread::scope(|s| {
        let (sender, receiver) = channel.split();
        s.spawn(move || {
```

```
            sender.send("hello world!");
        });
        assert_eq!(receiver.receive(), "hello world!");
    });
}
```

Clearly, this `Channel` is more convenient to use than the last one, at least in this simple test program. We've had to pay for this convenience by trading in some flexibility: only the thread that calls `split()` may call `receive()`. If you swap the `send` and `receive` lines, this program will no longer compile. Depending on the use case, that might be entirely fine, useful, or very inconvenient.

There are a number of ways to address that issue, many of which will cost us some additional complexity and affect performance. In general, the number of variations and trade-offs we can continue to explore is virtually endless.

We could easily spend an unhealthy number of hours implementing another twenty variants of a one-shot channel, each with slightly different properties, for every imaginable use case and more. While that might sound like lots of fun, we should probably avoid that rabbit hole and end this chapter before things get out of hand.

Summary

- A *channel* is used to send *messages* between threads.
- A simple and flexible, but potentially inefficient, channel is relatively easy to implement with just a `Mutex` and a `Condvar`.
- A *one-shot channel* is a channel designed to send only one message.
- The `MaybeUninit<T>` type can be used to represent a potentially not-yet-initialized `T`. Its interface is mostly unsafe, making its user responsible for tracking whether it has been initialized, not duplicating non-`Copy` data, and dropping its contents if necessary.
- Not dropping objects (also called *leaking* or *forgetting*) is safe, but frowned upon when done without good reason.
- Panicking is an important tool for creating a safe interface.
- Taking a non-`Copy` object by value can be used to prevent something from being done more than once.
- Exclusively borrowing and splitting borrows can be a powerful tool for forcing correctness.
- We can make sure an object stays on the same thread by making sure its type does not implement `Send`, which can be achieved with the `PhantomData` marker type.
- Every design and implementation decision involves a trade-off and can best be made with a specific use case in mind.
- Designing something without a use case can be fun and educational, but can turn out to be an endless task.

Complete Runnable Examples

The following complete, tested programs illustrate the primitives and patterns developed throughout this chapter.

Example: `s1_simple.rs` (S1 Simple)

```
use std::collections::VecDeque;
use std::sync::Condvar;
use std::sync::Mutex;

pub struct Channel<T> {
    queue: Mutex<VecDeque<T>>,
    item_ready: Condvar,
}

impl<T> Channel<T> {
    pub fn new() -> Self {
        Self {
            queue: Mutex::new(VecDeque::new()),
            item_ready: Condvar::new(),
        }
    }

    pub fn send(&self, message: T) {
        self.queue.lock().unwrap().push_back(message);
        self.item_ready.notify_one();
    }

    pub fn receive(&self) -> T {
        let mut b = self.queue.lock().unwrap();
        loop {
            if let Some(message) = b.pop_front() {
                return message;
            }
            b = self.item_ready.wait(b).unwrap();
        }
    }
}
```

Example: `s2_unsafe.rs` (S2 Unsafe)

```
use std::cell::UnsafeCell;
use std::mem::MaybeUninit;
use std::sync::atomic::AtomicBool;
use std::sync::atomic::Ordering::{Acquire, Release};

pub struct Channel<T> {
    message: UnsafeCell<MaybeUninit<T>>,
    ready: AtomicBool,
}

unsafe impl<T> Sync for Channel<T> where T: Send {}

impl<T> Channel<T> {
    pub const fn new() -> Self {
        Self {
            message: UnsafeCell::new(MaybeUninit::uninit()),
            ready: AtomicBool::new(false),
        }
    }

    /// Safety: Only call this once!
    pub unsafe fn send(&self, message: T) {
        (*self.message.get()).write(message);
        self.ready.store(true, Release);
    }

    pub fn is_ready(&self) -> bool {
        self.ready.load(Acquire)
    }

    /// Safety: Only call this once,
    /// and only after is_ready() returns true!
    pub unsafe fn receive(&self) -> T {
        (*self.message.get()).assume_init_read()
    }
}
```

Example: `s3_checks.rs` (S3 Checks)

```
use std::cell::UnsafeCell;
use std::mem::MaybeUninit;
use std::sync::atomic::AtomicBool;
use std::sync::atomic::Ordering::{Acquire, Relaxed, Release};
```

```
pub struct Channel<T> {
    message: UnsafeCell<MaybeUninit<T>>,
    in_use: AtomicBool,
    ready: AtomicBool,
}

unsafe impl<T> Sync for Channel<T> where T: Send {}

impl<T> Channel<T> {
    pub const fn new() -> Self {
        Self {
            message: UnsafeCell::new(MaybeUninit::uninit()),
            in_use: AtomicBool::new(false),
            ready: AtomicBool::new(false),
        }
    }

    /// Panics when trying to send more than one message.
    pub fn send(&self, message: T) {
        if self.in_use.swap(true, Relaxed) {
            panic!("can't send more than one message!");
        }
        unsafe { (*self.message.get()).write(message) };
        self.ready.store(true, Release);
    }

    pub fn is_ready(&self) -> bool {
        self.ready.load(Relaxed)
    }

    /// Panics if no message is available yet,
    /// or if the message was already consumed.
    ///
    /// Tip: Use `is_ready` to check first.
    pub fn receive(&self) -> T {
        if !self.ready.swap(false, Acquire) {
            panic!("no message available!");
        }
        // Safety: We've just checked (and reset) the ready flag.
        unsafe { (*self.message.get()).assume_init_read() }
    }
}

impl<T> Drop for Channel<T> {
    fn drop(&mut self) {
        if *self.ready.get_mut() {
```

```
            unsafe { self.message.get_mut().assume_init_drop() }
        }
    }
}

#[test]
fn main() {
    use std::thread;
    let channel = Channel::new();
    let t = thread::current();
    thread::scope(|s| {
        s.spawn(|| {
            channel.send("hello world!");
            t.unpark();
        });
        while !channel.is_ready() {
            thread::park();
        }
        assert_eq!(channel.receive(), "hello world!");
    });
}
```

Example: `s3_single_atomic.rs` (S3 Single Atomic)

```
use std::cell::UnsafeCell;
use std::mem::MaybeUninit;
use std::sync::atomic::AtomicU8;
use std::sync::atomic::Ordering::{Acquire, Relaxed, Release};

const EMPTY: u8 = 0;
const WRITING: u8 = 1;
const READY: u8 = 2;
const READING: u8 = 3;

pub struct Channel<T> {
    message: UnsafeCell<MaybeUninit<T>>,
    state: AtomicU8,
}

unsafe impl<T: Send> Sync for Channel<T> {}

impl<T> Channel<T> {
    pub const fn new() -> Self {
        Self {
            message: UnsafeCell::new(MaybeUninit::uninit()),
```

```
            state: AtomicU8::new(EMPTY),
        }
    }

    pub fn send(&self, message: T) {
        if self.state.compare_exchange(
            EMPTY, WRITING, Relaxed, Relaxed
        ).is_err() {
            panic!("can't send more than one message!");
        }
        unsafe { (*self.message.get()).write(message) };
        self.state.store(READY, Release);
    }

    pub fn is_ready(&self) -> bool {
        self.state.load(Relaxed) == READY
    }

    pub fn receive(&self) -> T {
        if self.state.compare_exchange(
            READY, READING, Acquire, Relaxed
        ).is_err() {
            panic!("no message available!");
        }
        unsafe { (*self.message.get()).assume_init_read() }
    }
}

impl<T> Drop for Channel<T> {
    fn drop(&mut self) {
        if *self.state.get_mut() == READY {
            unsafe { self.message.get_mut().assume_init_drop() }
        }
    }
}

#[test]
fn main() {
    use std::thread;
    let channel = Channel::new();
    let t = thread::current();
    thread::scope(|s| {
        s.spawn(|| {
            channel.send("hello world!");
            t.unpark();
        });
```

```
        while !channel.is_ready() {
            thread::park();
        }
        assert_eq!(channel.receive(), "hello world!");
    });
}
```

Example: `s4_types.rs` (S4 Types)

```
use std::cell::UnsafeCell;
use std::mem::MaybeUninit;
use std::sync::atomic::AtomicBool;
use std::sync::atomic::Ordering::{Acquire, Relaxed, Release};
use std::sync::Arc;

pub struct Sender<T> {
    channel: Arc<Channel<T>>,
}

pub struct Receiver<T> {
    channel: Arc<Channel<T>>,
}

pub fn channel<T>() -> (Sender<T>, Receiver<T>) {
    let a = Arc::new(Channel {
        message: UnsafeCell::new(MaybeUninit::uninit()),
        ready: AtomicBool::new(false),
    });
    (Sender { channel: a.clone() }, Receiver { channel: a })
}

struct Channel<T> {
    message: UnsafeCell<MaybeUninit<T>>,
    ready: AtomicBool,
}

impl<T> Sender<T> {
    pub fn send(self, message: T) {
        unsafe { (*self.channel.message.get()).write(message) };
        self.channel.ready.store(true, Release);
    }
}

impl<T> Receiver<T> {
    pub fn is_ready(&self) -> bool {
```

```
        self.channel.ready.load(Relaxed)
    }

    pub fn receive(self) -> T {
        if !self.channel.ready.swap(false, Acquire) {
            panic!("no message available!");
        }
        unsafe { (*self.channel.message.get()).assume_init_read() }
    }
}

unsafe impl<T> Sync for Channel<T> where T: Send {}

impl<T> Drop for Channel<T> {
    fn drop(&mut self) {
        if *self.ready.get_mut() {
            unsafe { self.message.get_mut().assume_init_drop() }
        }
    }
}

#[test]
fn main() {
    use std::thread;
    thread::scope(|s| {
        let (sender, receiver) = channel();
        let t = thread::current();
        s.spawn(move || {
            sender.send("hello world!");
            t.unpark();
        });
        while !receiver.is_ready() {
            thread::park();
        }
        assert_eq!(receiver.receive(), "hello world!");
    });
}
```

Example: `s5_borrowing.rs` (S5 Borrowing)

```
use std::cell::UnsafeCell;
use std::mem::MaybeUninit;
use std::sync::atomic::AtomicBool;
use std::sync::atomic::Ordering::{Acquire, Relaxed, Release};
```

```
pub struct Channel<T> {
    message: UnsafeCell<MaybeUninit<T>>,
    ready: AtomicBool,
}

unsafe impl<T> Sync for Channel<T> where T: Send {}

pub struct Sender<'a, T> {
    channel: &'a Channel<T>,
}

pub struct Receiver<'a, T> {
    channel: &'a Channel<T>,
}

impl<T> Channel<T> {
    pub const fn new() -> Self {
        Self {
            message: UnsafeCell::new(MaybeUninit::uninit()),
            ready: AtomicBool::new(false),
        }
    }

    pub fn split<'a>(&'a mut self) -> (Sender<'a, T>, Receiver<'a, T>) {
        *self = Self::new();
        (Sender { channel: self }, Receiver { channel: self })
    }
}

impl<T> Sender<'_, T> {
    pub fn send(self, message: T) {
        unsafe { (*self.channel.message.get()).write(message) };
        self.channel.ready.store(true, Release);
    }
}

impl<T> Receiver<'_, T> {
    pub fn is_ready(&self) -> bool {
        self.channel.ready.load(Relaxed)
    }

    pub fn receive(self) -> T {
        if !self.channel.ready.swap(false, Acquire) {
            panic!("no message available!");
        }
        unsafe { (*self.channel.message.get()).assume_init_read() }
    }
```

```
}

impl<T> Drop for Channel<T> {
    fn drop(&mut self) {
        if *self.ready.get_mut() {
            unsafe { self.message.get_mut().assume_init_drop() }
        }
    }
}

#[test]
fn main() {
    use std::thread;
    let mut channel = Channel::new();
    thread::scope(|s| {
        let (sender, receiver) = channel.split();
        let t = thread::current();
        s.spawn(move || {
            sender.send("hello world!");
            t.unpark();
        });
        while !receiver.is_ready() {
            thread::park();
        }
        assert_eq!(receiver.receive(), "hello world!");
    });
}
```

Example: `s6_blocking.rs` (S6 Blocking)

```
use std::cell::UnsafeCell;
use std::marker::PhantomData;
use std::mem::MaybeUninit;
use std::sync::atomic::AtomicBool;
use std::sync::atomic::Ordering::{Acquire, Release};
use std::thread;
use std::thread::Thread;

pub struct Channel<T> {
    message: UnsafeCell<MaybeUninit<T>>,
    ready: AtomicBool,
}

unsafe impl<T> Sync for Channel<T> where T: Send {}
```

```rust
pub struct Sender<'a, T> {
    channel: &'a Channel<T>,
    receiving_thread: Thread, // New!
}

pub struct Receiver<'a, T> {
    channel: &'a Channel<T>,
    _no_send: PhantomData<*const ()>, // New!
}

impl<T> Channel<T> {
    pub const fn new() -> Self {
        Self {
            message: UnsafeCell::new(MaybeUninit::uninit()),
            ready: AtomicBool::new(false),
        }
    }

    pub fn split<'a>(&'a mut self) -> (Sender<'a, T>, Receiver<'a, T>) {
        *self = Self::new();
        (
            Sender {
                channel: self,
                receiving_thread: thread::current(), // New!
            },
            Receiver {
                channel: self,
                _no_send: PhantomData, // New!
            }
        )
    }
}

impl<T> Sender<'_, T> {
    pub fn send(self, message: T) {
        unsafe { (*self.channel.message.get()).write(message) };
        self.channel.ready.store(true, Release);
        self.receiving_thread.unpark(); // New!
    }
}

impl<T> Receiver<'_, T> {
    pub fn receive(self) -> T {
        while !self.channel.ready.swap(false, Acquire) {
            thread::park();
        }
```

```
        unsafe { (*self.channel.message.get()).assume_init_read() }
    }
}

impl<T> Drop for Channel<T> {
    fn drop(&mut self) {
        if *self.ready.get_mut() {
            unsafe { self.message.get_mut().assume_init_drop() }
        }
    }
}

#[test]
fn main() {
    let mut channel = Channel::new();
    thread::scope(|s| {
        let (sender, receiver) = channel.split();
        s.spawn(move || {
            sender.send("hello world!");
        });
        assert_eq!(receiver.receive(), "hello world!");
    });
}
```

Building Our Own Arc

Building Our Own Arc

In "Reference Counting" in Chapter 1, we saw the `std::sync::Arc<T>` type that allows for shared ownership through reference counting. The `Arc::new` function creates a new allocation, just like `Box::new`. However, unlike a `Box`, a cloned `Arc` will share the original allocation, without creating a new one. The shared allocation will only be dropped once the `Arc` and all its clones are dropped.

The memory ordering considerations involved in an implementation of this type can get quite interesting. In this chapter, we'll put more of the theory to practice by implementing our own `Arc<T>`. We'll start with a basic version, then extend it to support *weak pointers* for cyclic structures, and finish the chapter with an optimized version that's nearly identical to the implementation in the standard library.

Basic Reference Counting

Our first version will use a single `AtomicUsize` to count the number of `Arc` objects that share an allocation. Let's start with a struct that holds this counter and the `T` object:

```
struct ArcData<T> {
    ref_count: AtomicUsize,
    data: T,
}
```

Note that this struct is not public. It's an internal implementation detail of our `Arc` implementation.

Next is the `Arc<T>` struct itself, which is effectively just a pointer to a (shared) `ArcData<T>` object.

It might be tempting to make it a wrapper for a `Box<ArcData<T>>`, using a standard `Box` to handle the allocation of the `ArcData<T>`. However, a `Box` represents exclusive ownership, not shared ownership. We can't use a reference either, because we're not just borrowing the data owned by something else, and its lifetime ("until the last clone of this `Arc` is dropped") is not directly representable with a Rust lifetime.

Instead, we'll have to resort to using a pointer and handle allocation and the concept of ownership manually. Instead of a `*mut T` or `*const T`, we'll use a `std::ptr::NonNull<T>`, which represents a pointer to `T` that is never null. That way, an `Option<Arc<T>>` will be the same size as an `Arc<T>`, using the null pointer representation for `None`.

```
use std::ptr::NonNull;

pub struct Arc<T> {
```

```
    ptr: NonNull<ArcData<T>>,
}
```

With a reference or a `Box`, the compiler automatically understands for which `T` it should make your struct `Send` or `Sync`. When using a raw pointer or `NonNull`, however, it'll conservatively assume it's never `Send` or `Sync` unless we explicitly tell it otherwise.

Sending an `Arc<T>` across threads results in a `T` object being shared, requiring `T` to be `Sync`. Similarly, sending an `Arc<T>` across threads could result in another thread dropping that `T`, effectively transferring it to the other thread, requiring `T` to be `Send`. In other words, `Arc<T>` should be `Send` if and only if `T` is both `Send` and `Sync`. The exact same holds for `Sync`, since a shared `&Arc<T>` can be cloned into a new `Arc<T>`.

```
unsafe impl<T: Send + Sync> Send for Arc<T> {}
unsafe impl<T: Send + Sync> Sync for Arc<T> {}
```

For `Arc<T>::new`, we'll have to create a new allocation with an `ArcData<T>` with a reference count of one. We'll use `Box::new` to create a new allocation, `Box::leak` to give up our exclusive ownership of this allocation, and `NonNull::from` to turn it into a pointer:

```
impl<T> Arc<T> {
    pub fn new(data: T) -> Arc<T> {
        Arc {
            ptr: NonNull::from(Box::leak(Box::new(ArcData {
                ref_count: AtomicUsize::new(1),
                data,
            }))),
        }
    }

    …
}
```

We know the pointer will always point to a valid `ArcData<T>` as long as the `Arc` object exists. However, this is not something the compiler knows or checks for us, so accessing the `ArcData` through the pointer requires unsafe code. We'll add a private helper function to get from the `Arc` to the `ArcData`, since this is something we'll have to do several times:

```
    fn data(&self) -> &ArcData<T> {
        unsafe { self.ptr.as_ref() }
    }
```

Using that, we can now implement the `Deref` trait to make our `Arc<T>` transparently behave like a reference to a `T`:

```
impl<T> Deref for Arc<T> {
    type Target = T;
```

```
    fn deref(&self) -> &T {
        &self.data().data
    }
}
```

Note that we don't implement `DerefMut`. Since an `Arc<T>` represents shared ownership, we can't unconditionally provide a `&mut T`.

Next: the `Clone` implementation. The cloned `Arc` will use the same pointer, after incrementing the reference counter:

```
impl<T> Clone for Arc<T> {
    fn clone(&self) -> Self {
        // TODO: Handle overflows.
        self.data().ref_count.fetch_add(1, Relaxed);
        Arc {
            ptr: self.ptr,
        }
    }
}
```

We can use `Relaxed` memory ordering to increment the reference counter, since there are no operations on other variables that need to strictly happen before or after this atomic operation. We already had access to the contained `T` before this operation (through the original `Arc`), and that remains unchanged afterwards (but now through at least two `Arc` objects).

An `Arc` would need to be cloned many times before the counter has any chance of overflowing, but running `std::mem::forget(arc.clone())` in a loop can make it happen. We can use any of the techniques discussed in "Example: ID Allocation" in Chapter 2 and "Example: ID Allocation Without Overflow" in Chapter 2 to handle this issue.

To keep things as efficient as possible in the normal (non-overflowing) case, we'll keep the original `fetch_add` and simply abort the whole process if we get uncomfortably close to overflowing:

```
        if self.data().ref_count.fetch_add(1, Relaxed) > usize::MAX / 2 {
            std::process::abort();
        }
```

> **Note**

Aborting the process is not instant, leaving for some time during which another thread can also call `Arc::clone`, incrementing the reference counter further. Therefore, just checking for `usize::MAX - 1` would not suffice. However, using `usize::MAX / 2` as the limit works fine: assuming every thread takes at least a few bytes of space in memory, it's impossible for `usize::MAX / 2` threads to exist concurrently.

Just like we increment the counter when cloning, we need to decrement it when dropping an `Arc`. The thread that sees the counter go from one to zero knows it dropped the last `Arc<T>`, and is

responsible for dropping and deallocating the `ArcData<T>`.

We'll use `Box::from_raw` to reclaim exclusive ownership of the allocation, and then drop it right away using `drop()`:

```
impl<T> Drop for Arc<T> {
    fn drop(&mut self) {
        // TODO: Memory ordering.
        if self.data().ref_count.fetch_sub(1, …) == 1 {
            unsafe {
                drop(Box::from_raw(self.ptr.as_ptr()));
            }
        }
    }
}
```

For this operation, we can't use `Relaxed` ordering, since we need to make sure that nothing is still accessing the data when we drop it. In other words, every single drop of one of the former `Arc` clones must have happened before the final drop. So, the final `fetch_sub` must establish a happens-before relationship with every previous `fetch_sub` operation, which we can do using release and acquire ordering: decrementing it from, for example, two to one effectively "releases" the data, while decrementing it from one to zero "acquires" ownership of it.

We could use `AcqRel` memory ordering to cover both cases, but only the final decrement to zero needs `Acquire`, while the others only need `Release`. For efficiency, we'll use only `Release` for the `fetch_sub` operation and a separate `Acquire` fence only when necessary:

```
        if self.data().ref_count.fetch_sub(1, Release) == 1 {
            fence(Acquire);
            unsafe {
                drop(Box::from_raw(self.ptr.as_ptr()));
            }
        }
```

Testing It

To test that our `Arc` is behaving as intended, we can write a unit test that creates an `Arc` containing a special object that lets us know when it gets dropped:

```
#[test]
fn test() {
    static NUM_DROPS: AtomicUsize = AtomicUsize::new(0);

    struct DetectDrop;

    impl Drop for DetectDrop {
        fn drop(&mut self) {
```

```
            NUM_DROPS.fetch_add(1, Relaxed);
        }
    }

    // Create two Arcs sharing an object containing a string
    // and a DetectDrop, to detect when it's dropped.
    let x = Arc::new(("hello", DetectDrop));
    let y = x.clone();

    // Send x to another thread, and use it there.
    let t = std::thread::spawn(move || {
        assert_eq!(x.0, "hello");
    });

    // In parallel, y should still be usable here.
    assert_eq!(y.0, "hello");

    // Wait for the thread to finish.
    t.join().unwrap();

    // One Arc, x, should be dropped by now.
    // We still have y, so the object shouldn't have been dropped yet.
    assert_eq!(NUM_DROPS.load(Relaxed), 0);

    // Drop the remaining `Arc`.
    drop(y);

    // Now that `y` is dropped too,
    // the object should've been dropped.
    assert_eq!(NUM_DROPS.load(Relaxed), 1);
}
```

This compiles and runs fine, so it seems our `Arc` is behaving as intended! While this is encouraging, it doesn't prove that the implementation is fully correct. It's advisable to use a long stress test involving many threads to gain more confidence.

Miri

It can also be very useful to run tests using Miri. Miri is an experimental but very useful and powerful tool to check unsafe code for various forms of undefined behavior.

Miri is an interpreter for the Rust compiler's mid-level intermediate representation. This means that it runs your code not by compiling it to native processor instructions, but instead by interpreting it at a point when information like types and lifetimes are still available. Because of this, Miri runs programs significantly slower than when they are compiled and run normally, but is able to detect many mistakes that would result in undefined behavior.

It includes experimental support for detecting data races, which allows it to detect memory ordering problems.

For more details and a guide on how to use Miri, see its GitHub page.

Mutation

As mentioned before, we can't implement `DerefMut` for our `Arc`. We can't unconditionally promise exclusive access (`&mut T`) to the data, because it might be accessed through other `Arc` objects.

However, what we can do is to allow it conditionally. We can make a method that only gives out a `&mut T` if the reference counter is one, proving that there's no other `Arc` object that could be used to access the same data.

This function, which we'll call `get_mut`, will have to take a `&mut Self` to make sure nothing else can use this same `Arc` to access the `T`. Knowing that there's only one `Arc` would be meaningless if that one `Arc` can still be shared.

We'll need to use acquire memory ordering to make sure that threads that previously owned a clone of the `Arc` are no longer accessing the data. We need to establish a happens-before relationship with every single `drop` that led to the reference counter being one.

This only matters when the reference counter is actually one; if it's higher, we'll not provide a `&mut T`, and the memory ordering is irrelevant. So, we can use a relaxed load, followed by a conditional acquire fence, as follows:

```
pub fn get_mut(arc: &mut Self) -> Option<&mut T> {
    if arc.data().ref_count.load(Relaxed) == 1 {
        fence(Acquire);
        // Safety: Nothing else can access the data, since
        // there's only one Arc, to which we have exclusive access.
        unsafe { Some(&mut arc.ptr.as_mut().data) }
    } else {
        None
    }
}
```

This function does not take a `self` argument, but takes a regular argument (named `arc`) instead. This means it can only be called as `Arc::get_mut(&mut a)`, and not as `a.get_mut()`. This is advisable for types that implement `Deref`, to avoid ambiguity with a similarly named method on the underlying `T`.

The returned mutable reference implicitly borrows the lifetime from the argument, meaning that nothing can use the original `Arc` as long as the returned `&mut T` is still around, allowing for safe mutation.

When the lifetime of the `&mut T` expires, the `Arc` can be used and shared with other threads again. One might wonder whether we need to worry about memory ordering for threads accessing the data afterwards. However, that's the responsibility of whatever mechanism is used for sharing the `Arc` (or a new clone of it) with another thread. (For example, a mutex, a channel, or spawning a new thread.)

Weak Pointers

Reference counting can be very useful when representing structures in memory consisting of multiple objects. For example, every node in a tree structure could contain an `Arc` to each of its child nodes. That way, when a node is dropped, its child nodes that are no longer in use are all (recursively) dropped as well.

This breaks down for *cyclic structures*, however. If a child node also contains an `Arc` to its parent node, neither will be dropped since there's always at least one `Arc` that still refers to it.

The standard library's `Arc` comes with a solution for that problem: `Weak<T>`. A `Weak<T>`, also called a *weak pointer*, behaves a bit like an `Arc<T>`, but does not prevent an object from getting dropped. A `T` can be shared between several `Arc<T>` and `Weak<T>` objects, but when all `Arc<T>` objects are gone, the `T` is dropped, regardless of whether there are any `Weak<T>` objects left.

This means that a `Weak<T>` can exist without a `T`, and thus cannot provide a `&T` unconditionally, like an `Arc<T>` can. However, to access the `T` given a `Weak<T>`, it can be *upgraded* to an `Arc<T>` through its `upgrade()` method. This method returns an `Option<Arc<T>>`, returning `None` if the `T` has already been dropped.

In an `Arc`-based structure, `Weak` can be used to break cycles. For example, child nodes in a tree structure could use `Weak` rather than `Arc` for their parent node. Then, dropping of a parent node is not prevented through the existence of its child nodes.

Let's implement this.

Just like before, when the number of `Arc` objects reaches zero, we can drop the contained `T` object. However, we can't drop and deallocate the `ArcData` yet, since there might still be weak pointers referencing it. Only once the last `Weak` pointer is also gone can we drop and deallocate the `ArcData`.

So, we'll use two counters: one for "the number of things that reference the `T`," and another for "the number of things that reference the `ArcData<T>`." In other words, the first counter is the same as before: it counts `Arc` objects, while the second counter counts both `Arc` and `Weak` objects.

We also need something that allows us to drop the contained object (`T`) while the `ArcData<T>` is still in use by the weak pointers. We'll use an `Option<T>` so we can use `None` for when the data is dropped, and wrap that in an `UnsafeCell` for *interior mutability* ("Interior Mutability" in Chapter 1), to allow that to happen when the `ArcData<T>` isn't exclusively owned:

```
struct ArcData<T> {
    /// Number of `Arc`s.
    data_ref_count: AtomicUsize,
    /// Number of `Arc`s and `Weak`s combined.
    alloc_ref_count: AtomicUsize,
    /// The data. `None` if there are only weak pointers left.
    data: UnsafeCell<Option<T>>,
}
```

If we think of a `Weak<T>` as an object responsible for keeping an `ArcData<T>` alive, it can make sense to implement `Arc<T>` as a struct containing a `Weak<T>`, since an `Arc<T>` needs to do the same, and more.

```
pub struct Arc<T> {
    weak: Weak<T>,
}

pub struct Weak<T> {
    ptr: NonNull<ArcData<T>>,
}

unsafe impl<T: Sync + Send> Send for Weak<T> {}
unsafe impl<T: Sync + Send> Sync for Weak<T> {}
```

The `new` function is mostly the same as before, except it now has two counters to initialize at once:

```
impl<T> Arc<T> {
    pub fn new(data: T) -> Arc<T> {
        Arc {
            weak: Weak {
                ptr: NonNull::from(Box::leak(Box::new(ArcData {
                    alloc_ref_count: AtomicUsize::new(1),
                    data_ref_count: AtomicUsize::new(1),
                    data: UnsafeCell::new(Some(data)),
                }))),
            },
        }
    }

    …
}
```

Just like before, we assume that the `ptr` field always points at a valid `ArcData<T>`. This time, we'll encode that assumption as a private `data()` helper method on `Weak<T>`:

```
impl<T> Weak<T> {
    fn data(&self) -> &ArcData<T> {
        unsafe { self.ptr.as_ref() }
    }

    …
}
```

In the `Deref` implementation for `Arc<T>`, we now have to use `UnsafeCell::get()` to get a pointer to the contents of the cell, and use unsafe code to promise it can safely be shared at this point. We also need `as_ref().unwrap()` to get a reference into the `Option<T>`. We don't have to worry about this panicking, since the `Option` will only be `None` when there are no `Arc` objects left.

```
impl<T> Deref for Arc<T> {
    type Target = T;

    fn deref(&self) -> &T {
        let ptr = self.weak.data().data.get();
        // Safety: Since there's an Arc to the data,
        // the data exists and may be shared.
        unsafe { (*ptr).as_ref().unwrap() }
    }
}
```

The `Clone` implementation for `Weak<T>` is quite straightforward; it's pretty much identical to our previous `Clone` implementation for `Arc<T>`:

```
impl<T> Clone for Weak<T> {
    fn clone(&self) -> Self {
        if self.data().alloc_ref_count.fetch_add(1, Relaxed) > usize::MAX / 2 {
            std::process::abort();
        }
        Weak { ptr: self.ptr }
    }
}
```

In the `Clone` implementation for our new `Arc<T>`, we need to increment both counters. We'll simply use `self.weak.clone()` to reuse the code above for the first counter, so we only have to manually increment the second counter:

```
impl<T> Clone for Arc<T> {
    fn clone(&self) -> Self {
        let weak = self.weak.clone();
        if weak.data().data_ref_count.fetch_add(1, Relaxed) > usize::MAX / 2 {
            std::process::abort();
        }
        Arc { weak }
    }
}
```

Dropping a `Weak` should decrement its counter and drop and deallocate the `ArcData` when the counter goes from one to zero. This is identical to what the `Drop` implementation of our previous `Arc` did.

```
impl<T> Drop for Weak<T> {
    fn drop(&mut self) {
        if self.data().alloc_ref_count.fetch_sub(1, Release) == 1 {
            fence(Acquire);
            unsafe {
                drop(Box::from_raw(self.ptr.as_ptr()));
```

```
            }
        }
    }
}
```

Dropping an `Arc` should decrement both counters. Note that one of these is already automatically taken care of since every `Arc` contains a `Weak`, such that dropping an `Arc` will also result in dropping a `Weak`. We only have to take care of the other counter:

```
impl<T> Drop for Arc<T> {
    fn drop(&mut self) {
        if self.weak.data().data_ref_count.fetch_sub(1, Release) == 1 {
            fence(Acquire);
            let ptr = self.weak.data().data.get();
            // Safety: The data reference counter is zero,
            // so nothing will access it.
            unsafe {
                (*ptr) = None;
            }
        }
    }
}
```

> **Note**

Dropping an object in Rust will first run its `Drop::drop` function (if it implements `Drop`), and then drop all of its fields, one by one, recursively.

The check in the `get_mut` method remains mostly unchanged, except it now needs to take weak pointers into account. It might seem like it could ignore weak pointers when checking for exclusivity, but a `Weak<T>` can be upgraded to an `Arc<T>` at any time. So, `get_mut` will have to check that there are no other `Arc<T>` or `Weak<T>` pointers before it can give out a `&mut T`:

```
impl<T> Arc<T> {
    …

    pub fn get_mut(arc: &mut Self) -> Option<&mut T> {
        if arc.weak.data().alloc_ref_count.load(Relaxed) == 1 {
            fence(Acquire);
            // Safety: Nothing else can access the data, since
            // there's only one Arc, to which we have exclusive access,
            // and no Weak pointers.
            let arcdata = unsafe { arc.weak.ptr.as_mut() };
            let option = arcdata.data.get_mut();
            // We know the data is still available since we
            // have an Arc to it, so this won't panic.
```

```
            let data = option.as_mut().unwrap();
            Some(data)
        } else {
            None
        }
    }

    …
}
```

Next up: upgrading a weak pointer. Upgrading a `Weak` to an `Arc` is only possible when the data still exists. If there are only weak pointers left, there's no data left that can be shared through an `Arc`. So, we'll have to increase the `Arc` counter, but can only do so if it isn't already zero. We'll use a compare-and-exchange loop ("Compare-and-Exchange Operations" in Chapter 2) to do this.

Just like before, relaxed memory ordering is fine for incrementing a reference counter. There are no operations on other variables that need to strictly happen before or after this atomic operation.

```
impl<T> Weak<T> {
    …

    pub fn upgrade(&self) -> Option<Arc<T>> {
        let mut n = self.data().data_ref_count.load(Relaxed);
        loop {
            if n == 0 {
                return None;
            }
            assert!(n <= usize::MAX / 2);
            if let Err(e) =
                self.data()
                    .data_ref_count
                    .compare_exchange_weak(n, n + 1, Relaxed, Relaxed)
            {
                n = e;
                continue;
            }
            return Some(Arc { weak: self.clone() });
        }
    }
}
```

The opposite, getting a `Weak<T>` from an `Arc<T>`, is much simpler:

```
impl<T> Arc<T> {
    …

    pub fn downgrade(arc: &Self) -> Weak<T> {
```

```
            arc.weak.clone()
        }
    }
```

Testing It

To quickly test our creation, we'll modify our previous unit test to use weak pointers and verify that they can be upgraded when expected:

```
#[test]
fn test() {
    static NUM_DROPS: AtomicUsize = AtomicUsize::new(0);

    struct DetectDrop;

    impl Drop for DetectDrop {
        fn drop(&mut self) {
            NUM_DROPS.fetch_add(1, Relaxed);
        }
    }

    // Create an Arc with two weak pointers.
    let x = Arc::new(("hello", DetectDrop));
    let y = Arc::downgrade(&x);
    let z = Arc::downgrade(&x);

    let t = std::thread::spawn(move || {
        // Weak pointer should be upgradable at this point.
        let y = y.upgrade().unwrap();
        assert_eq!(y.0, "hello");
    });
    assert_eq!(x.0, "hello");
    t.join().unwrap();

    // The data shouldn't be dropped yet,
    // and the weak pointer should be upgradable.
    assert_eq!(NUM_DROPS.load(Relaxed), 0);
    assert!(z.upgrade().is_some());

    drop(x);

    // Now, the data should be dropped, and the
    // weak pointer should no longer be upgradable.
    assert_eq!(NUM_DROPS.load(Relaxed), 1);
    assert!(z.upgrade().is_none());
}
```

This also compiles and runs without problems, which leaves us with a very usable handmade `Arc` implementation.

Optimizing

While weak pointers can be useful, the `Arc` type is often used without any weak pointers. A downside of our last implementation is that cloning and dropping an `Arc` now both take two atomic operations each, as they have to increment or decrement both counters. This makes all `Arc` users "pay" for the cost of weak pointers, even when they are not using them.

It might seem like the solution is to count `Arc<T>` and `Weak<T>` pointers separately, but then we wouldn't be able to atomically check that both counters are zero. To understand how that's a problem, imagine we have a thread executing the following annoying function:

```
fn annoying(mut arc: Arc<Something>) {
    loop {
        let weak = Arc::downgrade(&arc);
        drop(arc);
        println!("I have no Arc!"); // (1)
        arc = weak.upgrade().unwrap();
        drop(weak);
        println!("I have no Weak!"); // (2)
    }
}
```

This thread continuously downgrades and upgrades an `Arc`, such that it repeatedly cycles through moments where it holds no `Arc` (1), and moments where it holds no `Weak` (2). If we check both counters to see if there are any threads still using the allocation, this thread might be able to hide its existence if we are unlucky and check the `Arc` counter during its first print statement (1), but check the `Weak` counter during its second print statement (2).

In our last implementation, we solved this by counting every `Arc` also as a `Weak`. A more subtle way of solving this is to count all `Arc` pointers combined as one single `Weak` pointer. That way, the weak pointer counter (`alloc_ref_count`) never reaches zero as long as there is still at least one `Arc` object around, just like in our last implementation, but cloning an `Arc` doesn't need to touch that counter at all. Only dropping the very last `Arc` will decrement the weak pointer counter too.

Let's try that.

This time, we can't just implement `Arc<T>` as a wrapper around `Weak<T>`, so both will wrap a non-null pointer to the allocation:

```
pub struct Arc<T> {
    ptr: NonNull<ArcData<T>>,
}

unsafe impl<T: Sync + Send> Send for Arc<T> {}
unsafe impl<T: Sync + Send> Sync for Arc<T> {}
```

```
pub struct Weak<T> {
    ptr: NonNull<ArcData<T>>,
}

unsafe impl<T: Sync + Send> Send for Weak<T> {}
unsafe impl<T: Sync + Send> Sync for Weak<T> {}
```

Since we're optimizing our implementation, we might as well make `ArcData<T>` slightly smaller by using a `std::mem::ManuallyDrop<T>` instead of an `Option<T>`. We used an `Option<T>` to be able to replace a `Some(T)` by `None` when dropping the data, but we don't actually need a separate `None` state to tell us the data is gone, since the existence or absence of `Arc<T>` already tells us that. A `ManuallyDrop<T>` takes the exact same amount of space as a `T`, but allows us to manually drop it at any point by using an unsafe call to `ManuallyDrop::drop()`:

```
use std::mem::ManuallyDrop;

struct ArcData<T> {
    /// Number of `Arc`s.
    data_ref_count: AtomicUsize,
    /// Number of `Weak`s, plus one if there are any `Arc`s.
    alloc_ref_count: AtomicUsize,
    /// The data. Dropped if there are only weak pointers left.
    data: UnsafeCell<ManuallyDrop<T>>,
}
```

The `Arc::new` function remains almost unchanged, initializing both counters at once like before, but now using `ManuallyDrop::new()` instead of `Some()`:

```
impl<T> Arc<T> {
    pub fn new(data: T) -> Arc<T> {
        Arc {
            ptr: NonNull::from(Box::leak(Box::new(ArcData {
                alloc_ref_count: AtomicUsize::new(1),
                data_ref_count: AtomicUsize::new(1),
                data: UnsafeCell::new(ManuallyDrop::new(data)),
            }))),
        }
    }

    …
}
```

The `Deref` implementation can no longer make use of the private `data` method on the `Weak` type, so we'll add the same private helper function on `Arc<T>`:

```
impl<T> Arc<T> {
    …

    fn data(&self) -> &ArcData<T> {
        unsafe { self.ptr.as_ref() }
    }

    …
}

impl<T> Deref for Arc<T> {
    type Target = T;

    fn deref(&self) -> &T {
        // Safety: Since there's an Arc to the data,
        // the data exists and may be shared.
        unsafe { &*self.data().data.get() }
    }
}
```

The `Clone` and `Drop` implementations for `Weak<T>` remain exactly the same as for our last implementation. Here they are for completeness, including the private `Weak::data` helper function:

```
impl<T> Weak<T> {
    fn data(&self) -> &ArcData<T> {
        unsafe { self.ptr.as_ref() }
    }

    …
}

impl<T> Clone for Weak<T> {
    fn clone(&self) -> Self {
        if self.data().alloc_ref_count.fetch_add(1, Relaxed) > usize::MAX / 2 {
            std::process::abort();
        }
        Weak { ptr: self.ptr }
    }
}

impl<T> Drop for Weak<T> {
    fn drop(&mut self) {
        if self.data().alloc_ref_count.fetch_sub(1, Release) == 1 {
            fence(Acquire);
            unsafe {
                drop(Box::from_raw(self.ptr.as_ptr()));
            }
```

```
        }
    }
}
```

And now we get to the part that this new optimized implementation was all about—cloning an `Arc<T>` now needs to touch only one counter:

```
impl<T> Clone for Arc<T> {
    fn clone(&self) -> Self {
        if self.data().data_ref_count.fetch_add(1, Relaxed) > usize::MAX / 2 {
            std::process::abort();
        }
        Arc { ptr: self.ptr }
    }
}
```

Similarly, dropping an `Arc<T>` now needs to decrement only one counter, except for the last drop that sees that counter go from one to zero. In that case, the weak pointer counter also needs to be decremented, such that it can reach zero once there are no weak pointers left. We do this by simply creating a `Weak<T>` out of thin air and immediately dropping it:

```
impl<T> Drop for Arc<T> {
    fn drop(&mut self) {
        if self.data().data_ref_count.fetch_sub(1, Release) == 1 {
            fence(Acquire);
            // Safety: The data reference counter is zero,
            // so nothing will access the data anymore.
            unsafe {
                ManuallyDrop::drop(&mut *self.data().data.get());
            }
            // Now that there are no `Arc<T>`s left,
            // drop the implicit weak pointer that represented all `Arc<T>`s.
            drop(Weak { ptr: self.ptr });
        }
    }
}
```

The `upgrade` method on `Weak<T>` remains mostly the same, except it no longer clones a weak pointer, since it doesn't need to increment the weak pointer counter anymore. Upgrading only succeeds if there is already at least one `Arc<T>` to the allocation, which means that `Arc`s are already accounted for in the weak pointer counter.

```
impl<T> Weak<T> {
    …

    pub fn upgrade(&self) -> Option<Arc<T>> {
        let mut n = self.data().data_ref_count.load(Relaxed);
```

```
            loop {
                if n == 0 {
                    return None;
                }
                assert!(n <= usize::MAX / 2);
                if let Err(e) =
                    self.data()
                        .data_ref_count
                        .compare_exchange_weak(n, n + 1, Relaxed, Relaxed)
                {
                    n = e;
                    continue;
                }
                return Some(Arc { ptr: self.ptr });
            }
        }
    }
```

So far the differences between this and our previous implementation are very minimal. Where things get tricky, though, is with the last two methods we still need to implement: `downgrade` and `get_mut`.

Unlike before, the `get_mut` method now needs to check if both counters are set to one to be able to determine whether there's only one `Arc<T>` and no `Weak<T>` left, since a weak pointer counter of one can now represent multiple `Arc<T>` pointers. Reading the counters are two separate operations that happen at (slightly) different times, so we have to be very careful to not miss any concurrent downgrades, such as in the example case we saw at the start of "Optimizing".

If we first check that `data_ref_count` is one, then we could miss a subsequent `upgrade()` before we check the other counter. But, if we first check that `alloc_ref_count` is one, then we could miss a subsequent `downgrade()` before we check the other counter.

A way out of this dilemma is to briefly block the `downgrade()` operation by "locking" the weak pointer counter. To do that, we don't need anything like a mutex. We can use a special value, like `usize::MAX`, to represent a special "locked" state of the weak pointer counter. It'll only be locked very briefly, only to load the other counter, so the `downgrade` method could just spin until it's unlocked, in the unlikely situation it runs at the exact same moment as `get_mut`.

So, in `get_mut` we'll first have to check if `alloc_ref_count` is one and at the same time replace it by `usize::MAX`, if it was indeed one. That's a job for `compare_exchange`.

Then we'll have to check if the other counter is also one, after which we can immediately unlock the weak pointer counter. If the second counter is also one, we know we have exclusive access to the allocation and the data, such that we can return a `&mut T`.

```
    pub fn get_mut(arc: &mut Self) -> Option<&mut T> {
        // Acquire matches Weak::drop's Release decrement, to make sure any
        // upgraded pointers are visible in the next data_ref_count.load.
        if arc.data().alloc_ref_count.compare_exchange(
```

```
            1, usize::MAX, Acquire, Relaxed
        ).is_err() {
            return None;
        }
        let is_unique = arc.data().data_ref_count.load(Relaxed) == 1;
        // Release matches Acquire increment in `downgrade`, to make sure any
        // changes to the data_ref_count that come after `downgrade` don't
        // change the is_unique result above.
        arc.data().alloc_ref_count.store(1, Release);
        if !is_unique {
            return None;
        }
        // Acquire to match Arc::drop's Release decrement, to make sure nothing
        // else is accessing the data.
        fence(Acquire);
        unsafe { Some(&mut *arc.data().data.get()) }
    }
```

As you might have expected, the locking operation (the `compare_exchange`) will have to use `Acquire` memory ordering, and the unlocking operation (the `store`) will have to use `Release` memory ordering.

If we had used `Relaxed` for the `compare_exchange` instead, it would have been possible for the subsequent `load` from `data_ref_count` to not see the new value of a freshly upgraded `Weak` pointer, even though the `compare_exchange` had already confirmed that every `Weak` pointer had been dropped.

If we had used `Relaxed` for the `store`, it would have been possible for the preceding `load` to observe the result of a future `Arc::drop` for an `Arc` that can still be downgraded.

The acquire fence is the same as before: it synchronizes with the release-decrement operation in `Arc::drop` to make sure every access through former `Arc` clones has happened before the new exclusive access.

The last piece of the puzzle is the `downgrade` method, which will have to check for the special `usize::MAX` value to see if the weak pointer counter is locked, and spin until it is unlocked. Just like in the `upgrade` implementation, we'll use a compare-and-exchange loop to check for the special value and overflow before incrementing the counter:

```
    pub fn downgrade(arc: &Self) -> Weak<T> {
        let mut n = arc.data().alloc_ref_count.load(Relaxed);
        loop {
            if n == usize::MAX {
                std::hint::spin_loop();
                n = arc.data().alloc_ref_count.load(Relaxed);
                continue;
            }
            assert!(n <= usize::MAX / 2);
            // Acquire synchronizes with get_mut's release-store.
            if let Err(e) =
```

```
                arc.data()
                    .alloc_ref_count
                    .compare_exchange_weak(n, n + 1, Acquire, Relaxed)
            {
                n = e;
                continue;
            }
            return Weak { ptr: arc.ptr };
        }
    }
```

We use acquire memory ordering for `compare_exchange_weak`, which synchronizes with the release-store in the `get_mut` function. Otherwise, it would be possible for the effect of a subsequent `Arc::drop` to be visible to a thread running `get_mut` before it unlocks the counter.

In other words, the acquire compare-and-exchange operation here effectively "locks" `get_mut`, preventing it from succeeding. It can be "unlocked" again by a later `Weak::drop` that decrements the counter back to one, using release memory ordering.

> **Note**

The optimized implementation of `Arc<T>` and `Weak<T>` that we just made is nearly identical to the one included in the Rust standard library.

If we run the exact same test as before ("Testing It"), we see that this optimized implementation also compiles and passes our tests.

> **Note**

If you feel that getting the memory ordering decisions right for this optimized implementation was difficult, don't worry. Many concurrent data structures are simpler to implement correctly than this one. This `Arc` implementation is included in this chapter specifically because of its tricky subtleties around memory ordering.

Summary

- `Arc<T>` provides shared ownership of a reference-counted allocation.
- By checking if the reference counter is exactly one, an `Arc<T>` can conditionally provide exclusive access (`&mut T`).
- Incrementing the atomic reference counter can be done using a relaxed operation, but the final decrement must synchronize with all previous decrements.
- A *weak pointer* (`Weak<T>`) can be used to avoid cycles.
- The `NonNull<T>` type represents a pointer to `T` that is never null.

- The `ManuallyDrop<T>` type can be used to manually decide, using `unsafe` code, when to drop a `T`.
- As soon as more than one atomic variable is involved, things get more complicated.
- Implementing an ad hoc (spin) lock can sometimes be a valid strategy for operating on multiple atomic variables at once.

Complete Runnable Examples

The following complete, tested programs illustrate the primitives and patterns developed throughout this chapter.

Example: `s1_basic.rs` (S1 Basic)

```
use std::ops::Deref;
use std::sync::atomic::AtomicUsize;
use std::sync::atomic::fence;
use std::sync::atomic::Ordering::{Acquire, Relaxed, Release};
use std::ptr::NonNull;

struct ArcData<T> {
    ref_count: AtomicUsize,
    data: T,
}

pub struct Arc<T> {
    ptr: NonNull<ArcData<T>>,
}

unsafe impl<T: Send + Sync> Send for Arc<T> {}
unsafe impl<T: Send + Sync> Sync for Arc<T> {}

impl<T> Arc<T> {
    pub fn new(data: T) -> Arc<T> {
        Arc {
            ptr: NonNull::from(Box::leak(Box::new(ArcData {
                ref_count: AtomicUsize::new(1),
                data,
            }))),
        }
    }

    fn data(&self) -> &ArcData<T> {
```

```
        unsafe { self.ptr.as_ref() }
    }

    pub fn get_mut(arc: &mut Self) -> Option<&mut T> {
        if arc.data().ref_count.load(Relaxed) == 1 {
            fence(Acquire);
            // Safety: Nothing else can access the data, since
            // there's only one Arc, to which we have exclusive access.
            unsafe { Some(&mut arc.ptr.as_mut().data) }
        } else {
            None
        }
    }
}

impl<T> Deref for Arc<T> {
    type Target = T;

    fn deref(&self) -> &T {
        &self.data().data
    }
}

impl<T> Clone for Arc<T> {
    fn clone(&self) -> Self {
        if self.data().ref_count.fetch_add(1, Relaxed) > usize::MAX / 2 {
            std::process::abort();
        }
        Arc {
            ptr: self.ptr,
        }
    }
}

impl<T> Drop for Arc<T> {
    fn drop(&mut self) {
        if self.data().ref_count.fetch_sub(1, Release) == 1 {
            fence(Acquire);
            unsafe {
                drop(Box::from_raw(self.ptr.as_ptr()));
            }
        }
    }
}

#[test]
fn test() {
```

```
    static NUM_DROPS: AtomicUsize = AtomicUsize::new(0);

    struct DetectDrop;

    impl Drop for DetectDrop {
        fn drop(&mut self) {
            NUM_DROPS.fetch_add(1, Relaxed);
        }
    }

    // Create two Arcs sharing an object containing a string
    // and a DetectDrop, to detect when it's dropped.
    let x = Arc::new(("hello", DetectDrop));
    let y = x.clone();

    // Send x to another thread, and use it there.
    let t = std::thread::spawn(move || {
        assert_eq!(x.0, "hello");
    });

    // In parallel, y should still be usable here.
    assert_eq!(y.0, "hello");

    // Wait for the thread to finish.
    t.join().unwrap();

    // One Arc, x, should be dropped by now.
    // We still have y, so the object shouldn't have been dropped yet.
    assert_eq!(NUM_DROPS.load(Relaxed), 0);

    // Drop the remaining `Arc`.
    drop(y);

    // Now that `y` is dropped too,
    // the object should've been dropped.
    assert_eq!(NUM_DROPS.load(Relaxed), 1);
}
```

Example: `s2_weak.rs` (S2 Weak)

```
use std::cell::UnsafeCell;
use std::ops::Deref;
use std::sync::atomic::AtomicUsize;
use std::sync::atomic::fence;
use std::sync::atomic::Ordering::{Acquire, Relaxed, Release};
```

```rust
use std::ptr::NonNull;

struct ArcData<T> {
    /// Number of `Arc`s.
    data_ref_count: AtomicUsize,
    /// Number of `Arc`s and `Weak`s combined.
    alloc_ref_count: AtomicUsize,
    /// The data. `None` if there's only weak pointers left.
    data: UnsafeCell<Option<T>>,
}

pub struct Arc<T> {
    weak: Weak<T>,
}

pub struct Weak<T> {
    ptr: NonNull<ArcData<T>>,
}

unsafe impl<T: Sync + Send> Send for Weak<T> {}
unsafe impl<T: Sync + Send> Sync for Weak<T> {}

impl<T> Arc<T> {
    pub fn new(data: T) -> Arc<T> {
        Arc {
            weak: Weak {
                ptr: NonNull::from(Box::leak(Box::new(ArcData {
                    alloc_ref_count: AtomicUsize::new(1),
                    data_ref_count: AtomicUsize::new(1),
                    data: UnsafeCell::new(Some(data)),
                }))),
            },
        }
    }

    pub fn get_mut(arc: &mut Self) -> Option<&mut T> {
        if arc.weak.data().alloc_ref_count.load(Relaxed) == 1 {
            fence(Acquire);
            // Safety: Nothing else can access the data, since
            // there's only one Arc, to which we have exclusive access,
            // and no Weak pointers.
            let arcdata = unsafe { arc.weak.ptr.as_mut() };
            let option = arcdata.data.get_mut();
            // We know the data is still available since we
            // have an Arc to it, so this won't panic.
            let data = option.as_mut().unwrap();
            Some(data)
```

```
        } else {
            None
        }
    }

    pub fn downgrade(arc: &Self) -> Weak<T> {
        arc.weak.clone()
    }
}

impl<T> Weak<T> {
    fn data(&self) -> &ArcData<T> {
        unsafe { self.ptr.as_ref() }
    }

    pub fn upgrade(&self) -> Option<Arc<T>> {
        let mut n = self.data().data_ref_count.load(Relaxed);
        loop {
            if n == 0 {
                return None;
            }
            assert!(n <= usize::MAX / 2);
            if let Err(e) =
                self.data()
                    .data_ref_count
                    .compare_exchange_weak(n, n + 1, Relaxed, Relaxed)
            {
                n = e;
                continue;
            }
            return Some(Arc { weak: self.clone() });
        }
    }
}

impl<T> Deref for Arc<T> {
    type Target = T;

    fn deref(&self) -> &T {
        let ptr = self.weak.data().data.get();
        // Safety: Since there's an Arc to the data,
        // the data exists and may be shared.
        unsafe { (*ptr).as_ref().unwrap() }
    }
}

impl<T> Clone for Weak<T> {
```

```
    fn clone(&self) -> Self {
        if self.data().alloc_ref_count.fetch_add(1, Relaxed) > usize::MAX / 2 {
            std::process::abort();
        }
        Weak { ptr: self.ptr }
    }
}

impl<T> Clone for Arc<T> {
    fn clone(&self) -> Self {
        let weak = self.weak.clone();
        if weak.data().data_ref_count.fetch_add(1, Relaxed) > usize::MAX / 2 {
            std::process::abort();
        }
        Arc { weak }
    }
}

impl<T> Drop for Weak<T> {
    fn drop(&mut self) {
        if self.data().alloc_ref_count.fetch_sub(1, Release) == 1 {
            fence(Acquire);
            unsafe {
                drop(Box::from_raw(self.ptr.as_ptr()));
            }
        }
    }
}

impl<T> Drop for Arc<T> {
    fn drop(&mut self) {
        if self.weak.data().data_ref_count.fetch_sub(1, Release) == 1 {
            fence(Acquire);
            let ptr = self.weak.data().data.get();
            // Safety: The data reference counter is zero,
            // so nothing will access it.
            unsafe {
                (*ptr) = None;
            }
        }
    }
}

#[test]
fn test() {
    static NUM_DROPS: AtomicUsize = AtomicUsize::new(0);
```

```
    struct DetectDrop;

    impl Drop for DetectDrop {
        fn drop(&mut self) {
            NUM_DROPS.fetch_add(1, Relaxed);
        }
    }

    // Create an Arc with two weak pointers.
    let x = Arc::new(("hello", DetectDrop));
    let y = Arc::downgrade(&x);
    let z = Arc::downgrade(&x);

    let t = std::thread::spawn(move || {
        // Weak pointer should be upgradable at this point.
        let y = y.upgrade().unwrap();
        assert_eq!(y.0, "hello");
    });
    assert_eq!(x.0, "hello");
    t.join().unwrap();

    // The data shouldn't be dropped yet,
    // and the weak pointer should be upgradable.
    assert_eq!(NUM_DROPS.load(Relaxed), 0);
    assert!(z.upgrade().is_some());

    drop(x);

    // Now, the data should be dropped, and the
    // weak pointer should no longer be upgradable.
    assert_eq!(NUM_DROPS.load(Relaxed), 1);
    assert!(z.upgrade().is_none());
}
```

Example: `s3_optimized.rs` (S3 Optimized)

```
use std::mem::ManuallyDrop;
use std::cell::UnsafeCell;
use std::ops::Deref;
use std::sync::atomic::AtomicUsize;
use std::sync::atomic::fence;
use std::sync::atomic::Ordering::{Acquire, Relaxed, Release};
use std::ptr::NonNull;

pub struct Arc<T> {
```

```
    ptr: NonNull<ArcData<T>>,
}

unsafe impl<T: Sync + Send> Send for Arc<T> {}
unsafe impl<T: Sync + Send> Sync for Arc<T> {}

pub struct Weak<T> {
    ptr: NonNull<ArcData<T>>,
}

unsafe impl<T: Sync + Send> Send for Weak<T> {}
unsafe impl<T: Sync + Send> Sync for Weak<T> {}

struct ArcData<T> {
    /// Number of `Arc`s.
    data_ref_count: AtomicUsize,
    /// Number of `Weak`s, plus one if there are any `Arc`s.
    alloc_ref_count: AtomicUsize,
    /// The data. Dropped if there are only weak pointers left.
    data: UnsafeCell<ManuallyDrop<T>>,
}

impl<T> Arc<T> {
    pub fn new(data: T) -> Arc<T> {
        Arc {
            ptr: NonNull::from(Box::leak(Box::new(ArcData {
                alloc_ref_count: AtomicUsize::new(1),
                data_ref_count: AtomicUsize::new(1),
                data: UnsafeCell::new(ManuallyDrop::new(data)),
            }))),
        }
    }

    fn data(&self) -> &ArcData<T> {
        unsafe { self.ptr.as_ref() }
    }

    pub fn get_mut(arc: &mut Self) -> Option<&mut T> {
        // Acquire matches Weak::drop's Release decrement, to make sure any
        // upgraded pointers are visible in the next data_ref_count.load.
        if arc.data().alloc_ref_count.compare_exchange(
            1, usize::MAX, Acquire, Relaxed
        ).is_err() {
            return None;
        }
        let is_unique = arc.data().data_ref_count.load(Relaxed) == 1;
        // Release matches Acquire increment in `downgrade`, to make sure any
```

```
        // changes to the data_ref_count that come after `downgrade` don't
        // change the is_unique result above.
        arc.data().alloc_ref_count.store(1, Release);
        if !is_unique {
            return None;
        }
        // Acquire to match Arc::drop's Release decrement, to make sure nothing
        // else is accessing the data.
        fence(Acquire);
        unsafe { Some(&mut *arc.data().data.get()) }
    }

    pub fn downgrade(arc: &Self) -> Weak<T> {
        let mut n = arc.data().alloc_ref_count.load(Relaxed);
        loop {
            if n == usize::MAX {
                std::hint::spin_loop();
                n = arc.data().alloc_ref_count.load(Relaxed);
                continue;
            }
            assert!(n <= usize::MAX / 2);
            // Acquire synchronises with get_mut's release-store.
            if let Err(e) =
                arc.data()
                    .alloc_ref_count
                    .compare_exchange_weak(n, n + 1, Acquire, Relaxed)
            {
                n = e;
                continue;
            }
            return Weak { ptr: arc.ptr };
        }
    }
}

impl<T> Deref for Arc<T> {
    type Target = T;

    fn deref(&self) -> &T {
        // Safety: Since there's an Arc to the data,
        // the data exists and may be shared.
        unsafe { &*self.data().data.get() }
    }
}

impl<T> Weak<T> {
    fn data(&self) -> &ArcData<T> {
```

```
        unsafe { self.ptr.as_ref() }
    }

    pub fn upgrade(&self) -> Option<Arc<T>> {
        let mut n = self.data().data_ref_count.load(Relaxed);
        loop {
            if n == 0 {
                return None;
            }
            assert!(n <= usize::MAX / 2);
            if let Err(e) =
                self.data()
                    .data_ref_count
                    .compare_exchange_weak(n, n + 1, Relaxed, Relaxed)
            {
                n = e;
                continue;
            }
            return Some(Arc { ptr: self.ptr });
        }
    }
}

impl<T> Clone for Weak<T> {
    fn clone(&self) -> Self {
        if self.data().alloc_ref_count.fetch_add(1, Relaxed) > usize::MAX / 2 {
            std::process::abort();
        }
        Weak { ptr: self.ptr }
    }
}

impl<T> Drop for Weak<T> {
    fn drop(&mut self) {
        if self.data().alloc_ref_count.fetch_sub(1, Release) == 1 {
            fence(Acquire);
            unsafe {
                drop(Box::from_raw(self.ptr.as_ptr()));
            }
        }
    }
}

impl<T> Clone for Arc<T> {
    fn clone(&self) -> Self {
        if self.data().data_ref_count.fetch_add(1, Relaxed) > usize::MAX / 2 {
            std::process::abort();
```

```
        }
        Arc { ptr: self.ptr }
    }
}

impl<T> Drop for Arc<T> {
    fn drop(&mut self) {
        if self.data().data_ref_count.fetch_sub(1, Release) == 1 {
            fence(Acquire);
            // Safety: The data reference counter is zero,
            // so nothing will access the data anymore.
            unsafe {
                ManuallyDrop::drop(&mut *self.data().data.get());
            }
            // Now that there's no `Arc<T>`s left,
            // drop the implicit weak pointer that represented all `Arc<T>`s.
            drop(Weak { ptr: self.ptr });
        }
    }
}

#[test]
fn test() {
    static NUM_DROPS: AtomicUsize = AtomicUsize::new(0);

    struct DetectDrop;

    impl Drop for DetectDrop {
        fn drop(&mut self) {
            NUM_DROPS.fetch_add(1, Relaxed);
        }
    }

    // Create an Arc with two weak pointers.
    let x = Arc::new(("hello", DetectDrop));
    let y = Arc::downgrade(&x);
    let z = Arc::downgrade(&x);

    let t = std::thread::spawn(move || {
        // Weak pointer should be upgradable at this point.
        let y = y.upgrade().unwrap();
        assert_eq!(y.0, "hello");
    });
    assert_eq!(x.0, "hello");
    t.join().unwrap();
```

```
    // The data shouldn't be dropped yet,
    // and the weak pointer should be upgradable.
    assert_eq!(NUM_DROPS.load(Relaxed), 0);
    assert!(z.upgrade().is_some());

    drop(x);

    // Now, the data should be dropped, and the
    // weak pointer should no longer be upgradable.
    assert_eq!(NUM_DROPS.load(Relaxed), 1);
    assert!(z.upgrade().is_none());
}
```

Understanding the Processor

Understanding the Processor

While the theory from Chapters 2 and 3 is all we need to write correct concurrent code, it can additionally be very useful to develop an approximate understanding of what goes on in practice at the processor level. In this chapter, we'll explore the machine instructions that atomic operations compile down to, how different processor architectures differ, why a weak version of `compare_exchange` exists, what memory ordering means at the lowest level of individual instructions, and how caching relates to it all.

The goal of this chapter is not to understand every relevant detail of every single processor architecture. That would take many bookshelves full of books, many of which have probably not been written or are not publicly available. Instead, the goal of this chapter is to develop a general idea of how atomics work at the processor level, to be able to make more informed decisions when implementing and optimizing code involving atomics. And, of course, to simply satisfy our curiosity about what goes on behind the scenes—taking a break from all the abstract theory.

To make things as concrete as possible, we'll focus on two specific processor architectures:

x86-64:
The 64-bit version of the x86 architecture implemented by Intel and AMD processors used in the majority of laptops, desktops, servers, and some game consoles. While the originally 16-bit x86 architecture and its very popular 32-bit extension were developed by Intel, the 64-bit version that we now call x86-64 was initially an extension developed by AMD, often referred to as AMD64. Intel also developed its own 64-bit architecture, IA-64, but ended up adopting AMD's more popular x86 extension instead (under the names IA-32e, EM64T, and later Intel 64).

ARM64:
The 64-bit version of the ARM architecture used by nearly all modern mobile devices, high-performance embedded systems, and also increasingly in recent laptops and desktops. It is also known as AArch64 and was introduced as part of ARMv8. Earlier (32-bit) versions of ARM, which are similar in many ways, are used in an even wider variety of applications. Many popular microcontrollers in every kind of embedded system imaginable, from cars to electronic COVID tests, are based on ARMv6 and ARMv7.

These two architectures are unalike in many ways. Most importantly, they take different approaches to atomics. Understanding how atomics work on both of them provides us with a more general understanding that is transferable to many other architectures.

Processor Instructions

We can develop an approximate understanding of how things work at the processor level by taking a close look at the output of the compiler, the exact instructions that the processor will execute.

Brief Introduction to Assembly

When compiling software written in any compiled language like Rust or C, your code gets translated into *machine instructions* that can be executed by the processor that will eventually run your program. These instructions are highly specific to the processor architecture you're compiling your program for.

These instructions, also called *machine code*, are encoded in binary form, which is quite unreadable to us humans. *Assembly* is the human-readable representation of these instructions. Every instruction is represented by one line of text, usually starting with a single word or acronym to identify the instruction, followed by its arguments or operands. An *assembler* converts the text representation to binary representation, and a *disassembler* does the opposite.

After compiling from a language like Rust, most of the structure of the original source code is gone. Depending on optimization level, functions and function calls might still be recognizable. However, types such as structs or enums have been reduced to bytes and addresses, and loops and conditionals have been reduced to a flat structure with basic jump or branch instructions.

Here's an example of what a snippet of assembly for a small part of a program might look like, for some made-up architecture:

```
ldr x, 1234 // load from memory address 1234 into x
li y, 0     // set y to zero
inc x       // increment x
add y, x    // add x to y
mul x, 3    // multiply x by 3
cmp y, 10   // compare y to 10
jne -5      // jump five instructions back if not equal
str 1234, x // store x to memory address 1234
```

In this example, `x` and `y` are names of *registers*. Registers are part of the processor, not of the main memory, and usually hold a single integer or memory address. On 64-bit architectures, they are generally 64 bits in size. The number of registers varies per architecture, but is usually very limited. Registers are basically used as a temporary scratchpad in calculations, a place to keep intermediate results before storing things back to memory.

Constants that refer to specific memory addresses, such as `1234` and `-5` in the example above, are often replaced with more human-readable *labels*. The assembler will automatically replace them with the actual address when converting assembly to binary machine code.

Using labels, the previous example might look like this instead:

```
          ldr x, SOME_VAR
          li y, 0
my_loop:  inc x
          add y, x
          mul x, 3
          cmp y, 10
          jne my_loop
```

```
        str SOME_VAR, x
```

Since the names of the labels are only part of the assembly, but not of the binary machine code, a disassembler will not know what labels were originally used and will most likely just use meaningless generated names like `label1` and `var2`.

A full course on assembly for all the different architectures falls outside the scope of this book, but is not a prerequisite for reading this chapter. A very general understanding is more than enough to understand the examples, as we'll only be reading assembly, not writing it. The relevant instructions in each example will be explained in enough detail to be able to follow along with no prior experience with assembly.

To look at the exact machine code that the Rust compiler produces, we have several options. We could compile our code as usual, and then use a disassembler (such as `objdump`) to turn the produced binary file back into assembly. Using the debug information the compiler produces as part of the compilation process, the disassembler can produce labels that correspond to the original function names of the Rust source code. A downside of this method is that you need a disassembler that supports the specific processor architecture you're compiling for. While the Rust compiler supports many architectures, many disassemblers only support the one architecture that they were compiled for.

A more direct option is to ask the compiler to produce assembly instead of a binary by using the `--emit=asm` flag to `rustc`. A downside of this method is that the produced output contains a lot of irrelevant lines, containing information for the assembler and debug tools that we don't need.

There are great tools such as `cargo-show-asm` that integrate with `cargo` and automate the process of compiling your crate with the right flags, finding the relevant assembly for the function you're interested in, and highlighting the relevant lines containing the actual instructions.

For relatively small snippets, the easiest and most recommended way is to use a web service like the excellent Compiler Explorer by Matt Godbolt. This website allows you to write code in a number of languages, including Rust, and directly see corresponding compiled assembly using the selected compiler version. It even uses coloring to show which lines of Rust correspond to which lines of assembly, as far as such a correspondence still exists after optimization.

Since we want to look at the assembly for different architectures, we'll need to specify an exact target for the Rust compiler to compile to. We'll use `x86_64-unknown-linux-musl` for x86-64 and `aarch64-unknown-linux-musl` for ARM64. These are already supported directly in Compiler Explorer. If you're compiling locally, for example using `cargo-show-asm` or the other methods mentioned above, you'll need to make sure you've installed the Rust standard library for these targets, which is usually done using `rustup target add`.

In all cases, the target to compile for is selected using the `--target` compiler flag. For example, `--target=aarch64-unknown-linux-musl`. If you don't specify any target, it'll automatically pick the platform you're currently on. (Or, in the case of Compiler Explorer, the platform that it is hosted on, which is currently `x86_64-unknown-linux-gnu`.)

In addition, it's advisable to enable the `-O` flag to enable optimization (or `--release` when using Cargo), as that will enable optimization and disable overflow checking, which can significantly reduce the amount of produced assembly for the small functions we'll be looking at.

To try it out, let's look at the assembly for x86-64 and ARM64 for the following function:

```
pub fn add_ten(num: &mut i32) {
    *num += 10;
}
```

Using `-O --target=aarch64-unknown-linux-musl` as the compiler flags with any of the methods described above, we'll get something like the following assembly output for ARM64:

```
add_ten:
    ldr w8, [x0]
    add w8, w8, #10
    str w8, [x0]
    ret
```

The `x0` register contains the argument to our function, `num`, the address of the `i32` to increment by ten. First, the `ldr` instruction loads the 32-bit value from that memory address into the `w8` register. Then, the `add` instruction adds ten to `w8` and stores the result back into `w8`. And afterwards, the `str` instruction stores the `w8` register back into the same memory address. Finally, the `ret` instruction marks the end of the function and causes the processor to jump back and continue with the function that called `add_ten`.

If we compile the exact same code for `x86_64-unknown-linux-musl`, we'll get something like this instead:

```
add_ten:
    add dword ptr [rdi], 10
    ret
```

This time, a register called `rdi` is used for the `num` argument. More interestingly, on x86-64, a single `add` instruction can do what takes three instructions on ARM64: loading, incrementing, and storing a value.

This is usually the case on a *complex instruction set computer* (CISC) architecture, such as x86. Instructions on such an architecture often have many variants, for example to operate on a register or to directly operate on memory of certain size. (The `dword` in the assembly specifies a 32-bit operation.)

In contrast, a *reduced instruction set computer* (RISC) architecture, like ARM, usually has a simpler set of instructions with very few variants. Most instructions can only operate on registers, and loading and storing to memory takes a separate instruction. This allows for a simpler processor, which can result in a reduction in cost or sometimes higher performance.

This difference is especially relevant for atomic fetch-and-modify instructions, as we'll see momentarily.

> **Note**

While compilers are generally pretty smart, they don't always generate the most optimal assembly, especially when atomic operations are involved. If you're experimenting and find cases where you

feel confused by a seemingly needless complexity in the assembly, that often just means there are more optimization opportunities for a future version of the compiler.

Load and Store

Before we dive into anything more advanced, let's first look at the instructions used for the most basic atomic operations: load and store.

A regular non-atomic store through a `&mut i32` takes just a single instruction on both x86-64 and ARM64, as shown below:

Rust source

Compiled x86-64

Compiled ARM64

On x86-64, the very versatile `mov` instruction is used to copy ("move") data from one place to another; in this case, from a zero constant to memory. On ARM64, the `str` (store register) instruction is used to store a 32-bit register into memory. In this case, the special `wzr` register is used, which always contains zero.

If we change the code to instead use a relaxed atomic store to an `AtomicI32`, we get:

Rust source

Compiled x86-64

Compiled ARM64

Perhaps somewhat surprisingly, the assembly is identical to the non-atomic version. As it turns out, the `mov` and `str` instructions were already atomic. They either happened, or they didn't happen at all. Apparently, any difference between `&mut i32` and `&AtomicI32` here is only relevant for the compiler checks and optimizations, but is meaningless for the processor—at least for relaxed store operations on these two architectures.

The same thing happens when we look at relaxed load operations:

Rust source

Compiled x86-64

Compiled ARM64

On x86-64 the `mov` instruction is used again, this time to copy from memory into the 32-bit `eax` register. On ARM64, the `ldr` (load register) instruction is used to load the value from memory into the `w0` register.

> **Note**

The 32-bit `eax` and `w0` registers are used for passing back a 32-bit return value of a function. (For 64-bit values, the 64-bit `rax` and `x0` registers are used.)

While the processor apparently does not differentiate between atomic and non-atomic stores and loads, we cannot safely ignore the difference in our Rust code. If we use a `&mut i32`, the Rust compiler may assume that no other thread can concurrently access the same `i32`, and might decide to transform or optimize the code in such a way that a store operation no longer results in a single corresponding store instruction. For example, it would be perfectly correct, although somewhat unusual, for a non-atomic 32-bit load or store to happen with two separate 16-bit instructions.

Read-Modify-Write Operations

Things get far more interesting for *read-modify-write operations* such as addition. As discussed earlier in this chapter, a non-atomic read-modify-write operation usually compiles to three separate instructions (read, modify, and write) on a RISC architecture like ARM64, but can often be done in a single instruction on a CISC architecture like x86-64. This short example demonstrates that:

Rust source

Compiled x86-64

Compiled ARM64

Before we even look at the corresponding atomic operation, we can reasonably assume that this time we will see a difference between the non-atomic and atomic versions. The ARM64 version here is clearly not atomic, as loading and storing happens in separate steps.

While not directly obvious from the assembly itself, the x86-64 version is not atomic. The `add` instruction will be split by the processor into several *microinstructions* behind the scenes, with separate steps for loading the value and storing the result. This would be irrelevant on a single-core computer, as switching a processor core between threads generally only happens between instructions. However, when multiple cores are executing instructions in parallel, we can no longer assume instructions all happen atomically without considering the multiple steps involved in executing a single instruction.

x86 lock prefix

To support multi-core systems, Intel introduced an instruction prefix called `lock`. It is used as a modifier to instructions like `add` to make their operation atomic.

The `lock` prefix originally caused the processor to temporarily block all other cores from accessing memory for the duration of the instruction. While this is a simple and effective way to make something appear as atomic to the other cores, it can be quite inefficient to stop the world for every atomic operation. Newer processors have a much more advanced implementation of the `lock` prefix, which doesn't stop other cores from operating on unrelated memory, and allows cores to do useful things while waiting for a certain piece of memory to become available.

The `lock` prefix can only be applied to a very limited number of instructions, including `add`, `sub`, `and`, `not`, `or`, and `xor`, which are all very useful operations to be able to do atomically. The `xchg` (exchange) instruction, which corresponds to the atomic swap operation, has an implicit lock prefix: it behaves like `lock xchg` regardless of the `lock` prefix.

Let's see `lock add` in action by changing our last example to operate on an `AtomicI32`:

Rust source

Compiled x86-64

As expected, the only difference with the non-atomic version is the `lock` prefix.

In the example above, we ignore the return value from `fetch_add`, the value of `x` before the operation. However, if we use that value, the `add` instruction no longer suffices. The `add` instruction can provide a little bit of useful information to next instructions, such as whether the updated value was zero or negative, but it does not provide the full (original or updated) value. Instead, another instruction can be used: `xadd` ("exchange and add"), which puts the originally loaded value into a register.

We can see it in action by making a small modification to our code to return the value that `fetch_add` returns:

Rust source

Compiled x86-64

Instead of a constant 10, a register containing the value 10 is now used instead. The `xadd` instruction will reuse that register to store the old value.

Unfortunately, other than `xadd` and `xchg`, none of the other lock-prefixable instructions, like `sub`, `and`, and `or`, have such a variant. For example, there is no `xsub` instruction. For subtraction that's no issue, as `xadd` can be used with a negative value. For `and` and `or`, however, there's no such alternative.

For `and`, `or`, and `xor` operations that affect only a single bit, such as `fetch_or(1)` or `fetch_and(!1)`, it's possible to use the `bts` (bit test and set), `btr` (bit test and reset), and `btc` (bit test and complement) instructions. These instructions also allow a `lock` prefix, change only a single bit, and make the previous value of that one bit available for an instruction that follows, such as a conditional jump.

When these operations affect more than one bit, they cannot be represented by a single x86-64 instruction. Similarly, the `fetch_max` and `fetch_min` operations also have no corresponding x86-64 instruction. For these operations, we need a different strategy than a simple `lock` prefix.

x86 compare-and-exchange instruction

In "Compare-and-Exchange Operations" in Chapter 2, we saw how any atomic fetch-and-modify operation can be implemented as a compare-and-exchange loop. This is exactly what the compiler will use for operations that cannot be represented by a single x86-64 instruction, since this architecture does include a (lock-prefixable) `cmpxchg` (compare and exchange) instruction.

We can see this in action by changing our last example from `fetch_add` to `fetch_or`:

Rust source

Compiled x86-64

The first `mov` instruction loads the value from the atomic variable into the `eax` register. The following `mov` and `or` instructions copy that value into `ecx` and apply the binary `or` operation, such that `eax` contains the old value and `ecx` the new value. The `cmpxchg` instruction afterwards behaves exactly like the `compare_exchange` method in Rust. Its first argument is the memory address on which

to operate (the atomic variable), the second argument (`ecx`) is the new value, the expected value is implicitly taken from `eax`, and the return value is implicitly stored in `eax`. It also sets a status flag that a subsequent instruction can use to conditionally branch based on whether the operation succeeded or not. In this case, a `jne` (jump if not equal) instruction is used to jump back to the `.L1` label to try again on failure.

Here's what the equivalent compare-and-exchange loop looks like in Rust, just like we saw in "Compare-and-Exchange Operations" in Chapter 2:

```
pub fn a(x: &AtomicI32) -> i32 {
    let mut current = x.load(Relaxed);
    loop {
        let new = current | 10;
        match x.compare_exchange(current, new, Relaxed, Relaxed) {
            Ok(v) => return v,
            Err(v) => current = v,
        }
    }
}
```

Compiling this code results in the exact same assembly as the `fetch_or` version. This shows that, at least on x86-64, they are indeed equivalent in every way.

> **Note**

On x86-64, there is no difference between `compare_exchange` and `compare_exchange_weak`. Both compile down to a `lock cmpxchg` instruction.

Load-Linked and Store-Conditional Instructions

The closest thing to a compare-and-exchange loop on a RISC architecture is a *load-linked/store-conditional* (LL/SC) loop. It involves two special instructions that come in a pair: a load-linked instruction, which mostly behaves like a regular load instruction, and a store-conditional instruction, which mostly behaves like a regular store instruction. They are used in a pair, with both instructions targeting the same memory address. The key difference to the regular load and store instructions is that the store is conditional: it refuses to store to memory if any other thread has overwritten that memory since the load-linked instruction.

These two instructions allow us to load a value from memory, modify it, and store the new value back only if nobody has overwritten the value since we loaded it. If that fails, we can simply retry. Once it succeeds, we can safely pretend the whole operation was atomic, since it didn't get disrupted.

The key to making these instructions feasible and efficient to implement is twofold: (1) only one memory address (per core) can be tracked at a time, and (2) the store-conditional is allowed to have false negatives, meaning that it may fail to store even though nothing has changed that particular piece of memory.

This makes it possible to be less precise when tracking changes to memory, at the cost of perhaps a few extra cycles through an LL/SC loop. Access to memory could be tracked not per byte, but per chunk of 64 bytes, or per kilobyte, or even the entire memory as a whole. Less accurate memory tracking results in more unnecessary cycles through LL/SC loops, significantly reducing performance, but also reducing implementation complexity.

Taking things to the extreme, a basic, hypothetical single-core system could use a strategy where it does not track writes to memory at all. Instead, it could track interrupts or context switches, the events that can cause the processor to switch to another thread. If, in a system without any parallelism, no such event happened, it could safely assume no other thread could have touched the memory. If any such event happened, it could just assume the worst, refuse the store, and hope for better luck in the next iteration of the loop.

ARM load-exclusive and store-exclusive

On ARM64, or at least in the first version of ARMv8, no atomic fetch-and-modify or compare-and-exchange operation can be represented by a single instruction. True to its RISC nature, the load and store steps are separate from the calculation and comparison.

ARM64's load-linked and store-conditional instructions are called `ldxr` (load exclusive register) and `stxr` (store exclusive register). In addition, the `clrex` (clear exclusive) instruction can be used as an alternative to `stxr` to stop tracking writes to memory without storing anything.

To see them in action, let's see what happens when we do an atomic addition on ARM64:

Rust source

Compiled ARM64

We get something that looks quite similar to the non-atomic version we got before (in "Read-Modify-Write Operations"): a load instruction, an add instruction, and a store instruction. The load and store instructions have been replaced by their "exclusive" LL/SC version, and a new `cbnz` (compare and branch on nonzero) instruction appeared. The `stxr` instruction stores a zero in `w10` if it succeeded, or a one if it didn't. The `cbnz` instruction uses this to restart the whole operation if it failed.

Note that unlike with `lock add` on x86-64, we don't need to do anything special to retrieve the old value. In the example above, the old value will still be available in register `w8` after the operation succeeds, so there's no need for a specialized instruction like `xadd`.

This LL/SC pattern is quite flexible: it doesn't just work for a limited set of operations like `add` and `or`, but for virtually any operation. We can just as easily implement an atomic `fetch_divide` or `fetch_shift_left` by putting the corresponding instruction(s) between the `ldxr` and `stxr` instructions. However, if there are too many instructions between them, there's an increasingly high chance of disruption resulting in extra cycles. Generally, the compiler will attempt to keep the number of instructions in an LL/SC pattern as small as possible, to avoid LL/SC loops that would rarely—or even never—succeed and thus could spin forever.

ARMv8.1 Atomic Instructions

A later version of ARM64, part of ARMv8.1, also includes new CISC-style instructions for common atomic operations. For example, the new `ldadd` (load and add) instruction is equivalent to an atomic `fetch_add` operation, without the need for an LL/SC loop. It even includes instructions for operations like `fetch_max`, which don't exist on x86-64.

It also includes a `cas` (compare and swap) instruction corresponding to `compare_exchange`. When this instruction is used, there's no difference between `compare_exchange` and `compare_exchange_weak`, just like on x86-64.

While the LL/SC pattern is quite flexible and nicely fits the general RISC pattern, these new instructions can be more performant, as they can be easier to optimize for with specialized hardware.

Compare-and-exchange on ARM

The `compare_exchange` operation maps quite nicely onto this LL/SC pattern by using a conditional branch instruction to skip the store instruction if the comparison failed. Let's look at the generated assembly:

Rust source

Compiled ARM64

> **Note**

Note that a `compare_exchange_weak` operation is normally used in a loop that repeats if the comparison fails. For this example, however, we only call it once and ignore its return value, which shows us the relevant assembly without distractions.

The `ldxr` instruction loads the value, which is then immediately compared with the `cmp` (compare) instruction to the expected value of 5. The `b.ne` (branch if not equal) instruction will cause a jump to the `.L1` label if the value was not as expected, at which point the `clrex` instruction is used to abort the LL/SC pattern. If the value was five, the flow continues through the `mov` and `stxr` instructions to store the new value of six in memory, but only if nothing has overwritten the five in the meantime.

Remember that `stxr` is allowed to have false negatives; it might fail here even if the five wasn't overwritten. That's okay, because we're using `compare_exchange_weak`, which is allowed to have false negatives too. In fact, this is the reason why a weak version of `compare_exchange` exists.

If we replace `compare_exchange_weak` with `compare_exchange`, we get nearly identical assembly, except for an extra branch to restart the operation if it failed:

Rust source

Compiled ARM64

As expected, there's now an extra `cbnz` (compare and branch on nonzero) instruction to restart the LL/SC loop on failure. Additionally, the `mov` instruction has been moved out of the loop, to keep the loop as short as possible.

Optimization of Compare-and-Exchange Loops

As we saw in "x86 compare-and-exchange instruction", a `fetch_or` operation and the equivalent `compare_exchange` loop compile down to the exact same instructions on x86-64. One might expect the same to happen on ARM, at least with `compare_exchange_weak`, as the load and weak compare-and-exchange operations could directly be mapped to the LL/SC instructions.

Unfortunately, this is currently (as of Rust 1.66.0) not what happens.

While this might change in the future as the compiler is always improving, it's quite hard for a compiler to safely turn a manually written compare-and-exchange loop into the corresponding LL/SC loop. One of the reasons is that there's a limit on the number and type of instructions that can be put between the `ldxr` and `stxr` instructions, which is not something that the compiler is designed to keep in mind while applying other optimizations. At the time when patterns like a compare-and-exchange loop are still recognizable, the exact instructions that an expression will compile down to are not known yet, making this a very tricky optimization to implement for the general case.

So, at least until we get even smarter compilers, it's advisable to use the dedicated fetch-and-modify methods rather than a compare-and-exchange loop, if possible.

Caching

Reading and writing memory is slow, and can easily cost as much time as executing tens or hundreds of instructions. This is why all performant processors implement *caching*, to avoid interacting with the relatively slow memory as much as possible. The exact implementation details of memory caches in modern processors are complex, partially proprietary, and, most importantly, mostly irrelevant to us when writing software. After all, the name *cache* comes from the French word *caché*, meaning *hidden*. Nevertheless, understanding the basic principles behind how most processors implement caching can be extremely useful when optimizing software for performance. (Not that we need an excuse to learn more about an interesting topic, of course.)

Except for very small microcontrollers, virtually all modern processors use caching. Such a processor never interacts directly with main memory, but instead routes every single read and write request through its cache. If an instruction needs to read something from memory, the processor will ask its cache for that data. If it is already cached, the cache will quickly respond with the cached data, avoiding interacting with main memory. Otherwise, it'll have to take the slow path, where the cache might have to ask the main memory for a copy of the relevant data. Once the main memory responds, not only will the cache finally respond to the original read request, but it will also remember the data, such that it can respond more quickly the next time this data is requested. If the cache becomes full, it makes space by dropping some old data it deems least likely to be useful.

When an instruction wants to write something to memory, the cache could decide to hold on to the modified data without writing it to main memory. Any subsequent read requests for the same memory address will then get a copy of the modified data, ignoring the outdated data in main memory. It would only actually write the data back to main memory when the modified data needs to be dropped from the cache to make space.

In most processor architectures, the cache reads and writes memory in blocks of 64 bytes, even if only a single byte was requested. These blocks are often called *cache lines*. By caching the entire 64-byte block that surrounds the requested byte, any subsequent instructions that need to access any of the other bytes in that block will not have to wait for main memory.

Cache Coherence

In modern processors, there is usually more than one layer of caching. The first cache, or *level one (L1) cache*, is the smallest and fastest. Instead of talking to the main memory, it talks to the level two (L2) cache, which is much larger, but slower. The L2 cache might be the one to talk to main memory, or there might be yet another, larger and slower, L3 cache—perhaps even an L4 cache.

Adding extra layers doesn't change much about how they work; each layer can operate independently. Where things get interesting, however, is when there are multiple processor cores that each have their own cache. In a multi-core system, each processor core usually has its own L1 cache, while the L2 or L3 caches are often shared with some or all of the other cores.

A naive caching implementation would break down under these conditions, as the cache can no longer assume it controls all interactions with the next layer. If one cache would accept a write and mark some cache line as modified without informing the other caches, the state of the caches could become inconsistent. Not only would the modified data not be available to the other cores until the cache writes the data down to the next level(s), it could end up conflicting with different modifications cached in other caches.

To solve this problem, a *cache coherence protocol* is used. Such a protocol defines how exactly the caches operate and communicate with each other to keep everything in a consistent state. The exact protocol used varies per architecture, processor model, and even per cache level.

We'll discuss two basic cache coherence protocols. Modern processors use many variations of these.

The write-through protocol

In caches that implement the *write-through cache coherence protocol*, writes are not cached but immediately sent through to the next layer. The other caches are connected to the next layer through the same shared communication channel, which means that they can observe the other caches' communications to the next layer. When a cache observes a write for an address it currently has cached, it immediately either drops or updates its own cache line to keep everything consistent.

Using this protocol, caches never contain any cache lines in a modified state. While this simplifies things significantly, it nullifies any benefits of caching for writes. When optimizing just for reading, this can be a great choice.

The MESI protocol

The *MESI cache coherence protocol* is named after the four possible states it defines for a cache line: modified, exclusive, shared and invalid. Modified (M) is used for cache lines that contain data that has been modified but not yet written to memory (or the next level cache). Exclusive (E) is used for cache lines that contain unmodified data that's not cached in any other cache (at the same

level). Shared (S) is used for unmodified cache lines that might also appear in one or more of the other (same level) caches. Invalid (I) is used for unused (empty or dropped) cache lines, which do not contain any useful data.

Caches that use this protocol communicate with all the other caches at the same level. They send each other updates and requests to make it possible for them to stay consistent with each other.

When a cache gets a request for an address it has not yet cached, also called a *cache miss*, it does not immediately request it from the next layer. Instead, it first asks the other caches (at the same level) if any of them have this cache line available. If none of them have it, the cache will continue to request the address from the (slower) next layer, and mark the resulting new cache line as exclusive (E). When this cache line is then modified by a write operation, the cache can change the state to modified (M) without informing the others, since it knows none of the others have the same cache line cached.

When requesting a cache line that's already available in any of the other caches, the result is a shared (S) cache line, obtained directly from the other cache(s). If the cache line was in the modified (M) state, it'll first be written (or *flushed*) to the next layer, before changing it to shared (S) and sharing it. If it was in the exclusive (E) state, it'll be changed to shared (S) immediately.

If the cache wants exclusive rather than shared access (for example, because it is going to modify the data right after), the other cache(s) will not keep the cache line in shared (S) state, but instead drop it entirely by changing it to invalid (I). In this case, the result is an exclusive (E) cache line.

If a cache needs exclusive access to a cache line it already has available in the shared (S) state, it simply tells the others to drop the cache line before upgrading it to exclusive (E).

There are several variations of this protocol. For example, the *MOESI* protocol adds an extra state to allow sharing of modified data without immediately writing it to the next layer, and the *MESIF* protocol uses an extra state to decide which cache responds to a request for a shared cache line that's available in multiple caches. Modern processors often use more elaborate and proprietary cache coherence protocols.

Impact on Performance

While caching is mostly hidden from us, caching behavior can have significant effects on the performance of our atomic operations. Let's try to measure some of those effects.

Measuring the speed of a single atomic operation is very tricky, since they are extremely fast. To be able to get some useful numbers, we'll have to repeat an operation, say, a billion times, and measure how long that takes in total. For example, we could try to measure the time it takes for a billion load operations like this:

```
static A: AtomicU64 = AtomicU64::new(0);

fn main() {
    let start = Instant::now();
    for _ in 0..1_000_000_000 {
        A.load(Relaxed);
    }
```

```
    println!("{:?}", start.elapsed());
}
```

Unfortunately, this does not work as expected.

When running this with optimizations turned on (e.g., with `cargo run --release` or `rustc -O`), we'll see an unreasonably low measured time. What happened is that the compiler was smart enough to understand that we're not using the loaded values, so it decided to completely optimize the "unnecessary" loop away.

To avoid this, we can use the special `std::hint::black_box` function. This function takes an argument of any type, which it just returns without doing anything. What makes this function special is that the compiler will try its best not to assume anything about what the function does; it treats it like a "black box" that could do anything.

We can use this to avoid certain optimizations that would render a benchmark useless. In this case, we can pass the result of the load operation to `black_box()` to stop any optimizations that assume we don't actually need the loaded values. That's not enough, though, since the compiler could still assume that `A` is always zero, making the load operations unnecessary. To avoid that, we can pass a reference to `A` to `black_box()` at the start, such that the compiler may no longer assume there's only one thread that accesses `A`. After all, it must assume that `black_box(&A)` might have spawned an extra thread that interacts with `A`.

Let's try that out:

```
use std::hint::black_box;

static A: AtomicU64 = AtomicU64::new(0);

fn main() {
    black_box(&A); // New!
    let start = Instant::now();
    for _ in 0..1_000_000_000 {
        black_box(A.load(Relaxed)); // New!
    }
    println!("{:?}", start.elapsed());
}
```

The output fluctuates a bit when running this multiple times, but on a not-so-recent x86-64 computer, it seems to give a result of about 300 milliseconds.

To see any caching effects, we'll spawn a background thread that interacts with the atomic variable. That way, we can see if it affects the load operations of the main thread or not.

First, let's try that with just load operations on the background thread, as follows:

```
static A: AtomicU64 = AtomicU64::new(0);

fn main() {
    black_box(&A);
```

```
    thread::spawn(|| { // New!
        loop {
            black_box(A.load(Relaxed));
        }
    });

    let start = Instant::now();
    for _ in 0..1_000_000_000 {
        black_box(A.load(Relaxed));
    }
    println!("{:?}", start.elapsed());
}
```

Note that we're not measuring the performance of the operations on the background thread. We're still only measuring how long it takes for the main thread to perform a billion load operations.

Running this program results in similar measurements as before: it fluctuates a bit around 300 milliseconds when tested on the same x86-64 computer. The background thread has no significant effect on the main thread. They presumably each run on a separate processor core, but the caches of *both* cores contain a copy of `A`, allowing for very fast access.

Now let's change the background thread to perform store operations instead:

```
static A: AtomicU64 = AtomicU64::new(0);

fn main() {
    black_box(&A);
    thread::spawn(|| {
        loop {
            A.store(0, Relaxed); // New!
        }
    });
    let start = Instant::now();
    for _ in 0..1_000_000_000 {
        black_box(A.load(Relaxed));
    }
    println!("{:?}", start.elapsed());
}
```

This time, we do see a significant difference. Running this program on the same x86-64 machine now results in an output that fluctuates around three whole seconds, almost ten times as much as before. More recent computers will show a less significant but still very measurable difference. For example, it went from 350 milliseconds to 500 milliseconds on a recent Apple M1 processor, and from 250 milliseconds to 650 milliseconds on a very recent x86-64 AMD processor.

This behavior matches our understanding of cache coherence protocols: a store operation requires exclusive access to a cache line, which slows down subsequent load operations on other cores that no longer share the cache line.

Failing Compare-and-Exchange Operations

Interestingly, on most processor architectures, the same effect we saw with store operations also happens when the background thread performs only compare-and-exchange operations, even if they all fail.

To try that out, we can replace the store operation (of the background thread) with a call to `compare_exchange` that will never succeed:

```
    …
        loop {
            // Never succeeds, because A is never 10.
            black_box(A.compare_exchange(10, 20, Relaxed, Relaxed).is_ok());
        }
    …
```

Because `A` is always zero, this `compare_exchange` operation will never succeed. It'll load the current value of `A`, but never update it to a new value.

One might reasonably expect this to behave the same as a load operation, since it does not modify the atomic variable. However, on most processor architectures, the instruction(s) of `compare_exchange` will claim exclusive access of the relevant cache line regardless of whether the comparison succeeds or not.

This means that it can be beneficial to not use `compare_exchange` (or `swap`) in a spin loop like we did for our `SpinLock` in Chapter 4, but instead use a `load` operation first to check if the lock has been unlocked. That way, we avoid unnecessarily claiming exclusive access to the relevant cache line.

Since caching happens per cache line, not per individual byte or variable, we should be able to see the same effect using adjacent variables rather than the same one. To try this out, let's use three atomic variables instead of one, have the main thread use only the middle variable, and make the background thread use only the other two, as follows:

```
static A: [AtomicU64; 3] = [
    AtomicU64::new(0),
    AtomicU64::new(0),
    AtomicU64::new(0),
];

fn main() {
    black_box(&A);
    thread::spawn(|| {
        loop {
            A[0].store(0, Relaxed);
            A[2].store(0, Relaxed);
        }
    });
    let start = Instant::now();
    for _ in 0..1_000_000_000 {
```

```
        black_box(A[1].load(Relaxed));
    }
    println!("{:?}", start.elapsed());
}
```

Running this produces similar results as before: it takes several seconds on that same x86-64 computer. Even though `A[0]`, `A[1]`, and `A[2]` are each used by only one thread, we still see the same effects as if we're using the same variable on both threads. The reason is that `A[1]` shares a cache line with either or both of the others. The processor core running the background thread repeatedly claims exclusive access to the cache line(s) containing `A[0]` and `A[2]`, which also contains `A[1]`, slowing down "unrelated" operations on `A[1]`. This effect is called *false sharing*.

We can avoid this by spacing the atomic variables further apart, so they each get their own cache line. As mentioned before, 64 bytes is a reasonable guess for the size of a cache line, so let's try wrapping our atomics in a 64-byte aligned struct, as follows:

```
#[repr(align(64))] // This struct must be 64-byte aligned.
struct Aligned(AtomicU64);

static A: [Aligned; 3] = [
    Aligned(AtomicU64::new(0)),
    Aligned(AtomicU64::new(0)),
    Aligned(AtomicU64::new(0)),
];

fn main() {
    black_box(&A);
    thread::spawn(|| {
        loop {
            A[0].0.store(1, Relaxed);
            A[2].0.store(1, Relaxed);
        }
    });
    let start = Instant::now();
    for _ in 0..1_000_000_000 {
        black_box(A[1].0.load(Relaxed));
    }
    println!("{:?}", start.elapsed());
}
```

The `#[repr(align)]` attribute enables us to tell the compiler the (minimal) alignment of our type, in bytes. Since an `AtomicU64` is only 8 bytes, this will add 56 bytes of padding to our `Aligned` struct.

Running this program no longer gives slow results. Instead, we get the same results as when we had no background thread at all: about 300 milliseconds when run on the same x86-64 computer as before.

> **Note**

Depending on the type of processor you're trying this on, you might need to use 128-byte alignment to see the same effect.

The experiment above shows that it can be advisable not to put unrelated atomic variables close to each other. For example, a dense array of small mutexes might not always perform as well as an alternative structure that keeps the mutexes spaced further apart.

On the other hand, when multiple (atomic) variables are related and often accessed in quick succession, it can be good to put them close together. For example, our `SpinLock<T>` from Chapter 4 stores the `T` right next to the `AtomicBool`, which means it's likely that the cache line containing the `AtomicBool` will also contain the `T`, such that a claim for (exclusive) access to one also includes the other. Whether this is beneficial depends entirely on the situation.

Reordering

Consistent caching, for example through the MESI protocol we explored earlier in this chapter, generally does not affect correctness of a program, even when multiple threads are involved. The only observable differences caused by consistent caching come down to differences in timing. However, modern processors implement many more optimizations that can have a big impact on correctness, at least when multiple threads are involved.

At the start of Chapter 3, we briefly discussed *instruction reordering*, how both the compiler and the processor can change the order of instructions. Focusing just on the processor, here are some examples of various ways in which instructions, or their effects, might happen *out of order*:

Store buffers
Since writes can be slow, even with caching, processor cores often include a *store buffer*. Write operations to memory can be stored in this store buffer, which is very quick, to allow the processor to immediately continue with the instructions that follow. Then, in the background, the write operation is completed by writing to the (L1) cache, which can be significantly slower. This way, the processor does not have to wait while the cache coherence protocol jumps into action to get exclusive access to the relevant cache line.

As long as special care is taken to handle subsequent read operations from the same memory address, this is entirely invisible for instructions running as part of the same thread, on the same processor core. However, for a brief moment, the write operation is not yet visible to the other cores, resulting in an inconsistent view of what the memory looks like from different threads running on different cores.

Invalidation queues
Regardless of the exact coherency protocol, caches that operate in parallel need to process invalidation requests: instructions to drop a specific cache line because it's about to be modified and become invalid. As a performance optimization, it's common for such requests not to be processed immediately, but to be queued for (slightly) later processing instead. When such invalidation queues are in use, the caches are no longer always consistent, as cache lines might be briefly outdated before they are dropped. However, this has no impact on a single-threaded program, other than speeding it up.

The only impact is the visibility of write operations from other cores, which might now appear as (very slightly) delayed.

Pipelining
Another very common processor feature that significantly improves performance is *pipelining*: executing consecutive instructions in parallel, if possible. Before an instruction finishes executing, the processor might already start executing the next one. Modern processors can often start the execution of quite a few instructions in series while the first one is still in progress.

If each instruction operates on the result from the previous one, this doesn't help much; they each still need to wait on the result of the one before it. But when an instruction can be executed independently of the previous one, it might even finish first. For example, an instruction that just increments a register might finish very quickly, while a previously started instruction might still be waiting on reading something from memory, or some other slow operation.

While this doesn't affect a single-threaded program (other than speed), interaction with other cores might happen out of order when an instruction that operates on memory finishes executing before a preceding one does.

There are many ways in which a modern processor might end up executing instructions in an entirely different order than expected. There are many proprietary techniques involved, some of which become public only when a subtle mistake is found that can be exploited by malicious software. When they work as expected, however, they all have one thing in common: they do not affect single-threaded programs, other than timing, but can cause interaction with other cores to appear to happen in an inconsistent order.

Processor architectures that allow for memory operations to be reordered also provide a way to prevent this through special instructions. These instructions might, for example, force the processor to flush its store buffer, or to finish any pipelined instructions, before continuing. Sometimes, these instructions only prevent a certain type of reordering. For example, there might be an instruction to prevent store operations from being reordered with respect to each other, while still allowing load operations to be reordered. Which types of reordering might happen, and how they can be prevented, depends on the processor architecture.

Memory Ordering

When performing any atomic operation in a language like Rust or C, we specify a memory ordering to inform the compiler of our ordering requirements. The compiler will generate the right instructions for the processor to prevent it from reordering instructions in ways that would break the rules.

Which types of instruction reordering are allowed depends on the kind of memory operation. For non-atomic and relaxed atomic operations, any type of reordering is acceptable. At the other extreme, sequentially consistent atomic operations don't allow for any type of reordering at all.

An acquire operation may not get reordered with any memory operations that follow, while a release operation may not be reordered with any memory operations that precede it. Otherwise, some mutex-protected data might be accessed before acquiring—or after releasing—its mutex, resulting in a data race.

Other-Multi-Copy Atomicity

The ways in which the order of memory operations is affected on some processor architectures, such as those one might find in graphics cards, cannot always be explained by instruction reordering. The effect of two consecutive store operations on one core might become visible in the same order on a second core, but in opposite order on a third core. This could happen, for example, because of inconsistent caching or shared store buffers. This effect can't be explained by the instructions on the first core being reordered, as that doesn't explain the inconsistency between the second and third core's observations.

The theoretical memory model we discussed in Chapter 3 leaves space for such processor architectures by not requiring a globally consistent order for anything but sequentially consistent atomic operations.

The architectures we're focusing on in this chapter, x86-64 and ARM64, are *other-multi-copy atomic*, which means that write operations, once they are visible to any core, become visible to all cores at the same time. For other-multi-copy atomic architectures, memory ordering is only a matter of instruction reordering.

Some architectures, such as ARM64, are called *weakly ordered*, as they allow the processor to freely reorder any memory operation. On the other hand, *strongly ordered* architectures, such as x86-64, are very restrictive about which memory operations may be reordered.

x86-64: Strongly Ordered

On an x86-64 processor, a load operation will never appear to have happened after a memory operation that follows. Similarly, this architecture doesn't allow for a store operation to appear to have happened before a preceding memory operation. The only kind of reordering you might see on x86-64 is a store operation getting delayed until after a later load operation.

> **Note**

Because of the reordering restrictions of the x86-64 architecture, it is often described as a *strongly ordered* architecture, although some prefer to reserve this term for architectures that preserve the order of *all* memory operations.

These restrictions satisfy all the needs of acquire-loads (because a load is never reordered with a later operation), and of release-stores (because a store is never reordered with an earlier operation). This means that on x86-64, we get release and acquire semantics "for free": release and acquire operations are identical to relaxed operations.

We can verify this by seeing what happens to a few of the snippets from "Load and Store" and "x86 lock prefix" when we change `Relaxed` to `Release`, `Acquire`, or `AcqRel`:

Rust source

Compiled x86-64

As expected, the assembly is identical, even though we specified a stronger memory ordering.

We can conclude that on x86-64, ignoring potential compiler optimizations, acquire and release operations are just as cheap as relaxed operations. Or, perhaps more accurately, that relaxed operations are just as expensive as acquire and release operations.

Let's check out what happens for `SeqCst`:

Rust source

Compiled x86-64

The `load` and `fetch_add` operations still result in the same assembly as before, but the assembly for `store` changed completely. The `xor` instruction looks a bit out of place, but is just a common way to set the `eax` register to zero by `xor`'ing it with itself, which always results in zero. A `mov eax, 0` instruction would've worked as well, but takes a bit more space.

The interesting part is the `xchg` instruction, which is normally used for a swap operation: a store operation that also retrieves the old value.

A regular `mov` instruction like before wouldn't suffice for a `SeqCst` store, because it would allow reordering it with a later load operation, breaking the globally consistent order. By changing it to an operation that also performs a load, even though we don't care about the value it loads, we get the additional guarantee of our instruction not getting reordered with later memory operations, solving the issue.

> **Note**

A `SeqCst` load operation can still be a regular `mov`, exactly because `SeqCst` stores are upgraded to `xchg`. `SeqCst` operations guarantee a globally consistent order only with other `SeqCst` operations. The `mov` from a `SeqCst` load might still be reordered with the `mov` of an earlier non-`SeqCst` store operation, but that's perfectly fine.

On x86-64, a store operation is the only atomic operation for which there's a difference between `SeqCst` and weaker memory ordering. In other words, x86-64 `SeqCst` operations other than stores are just as cheap as `Release`, `Acquire`, `AcqRel`, and even `Relaxed` operations. Or, if you prefer, x86-64 makes `Relaxed` operations other than stores as expensive as `SeqCst` operations.

ARM64: Weakly Ordered

On a *weakly ordered* architecture such as ARM64, all memory operations can potentially be reordered with each other. This means that unlike x86-64, acquire and release operations will not be identical to relaxed operations.

Let's take a look at what happens on ARM64 for `Release`, `Acquire`, and `AcqRel`:

Rust source

Compiled ARM64

The changes, compared to the `Relaxed` versions we saw earlier, are very subtle:

(1) `str` (store register) is now `stlr` (store-release register).

(2) `ldr` (load register) is now `ldar` (load-acquire register).

(3) `ldxr` (load exclusive register) is now `ldaxr` (load-acquire exclusive register).

(4) `stxr` (store exclusive register) is now `stlxr` (store-release exclusive register).

As this shows, ARM64 has special versions of its load and store instructions for acquire and release ordering. Unlike an `ldr` or `ldxr` instruction, an `ldar` or `ldaxr` instruction will never be reordered with any later memory operation. Similarly, unlike an `str` or `stxr` instruction, an `stlr` or `stlxr` instruction will never be reordered with any earlier memory operation.

> **Note**

A fetch-and-modify operation using only `Release` or `Acquire` ordering instead of `AcqRel` uses only one of the `stlxr` and `ldaxr` instructions, respectively, paired with a regular `ldxr` or `stxr` instruction.

In addition to the restrictions required for release and acquire semantics, none of the special acquire and release instructions is ever reordered with any other of these special instructions, making them also suitable for `SeqCst`.

As demonstrated below, upgrading to `SeqCst` results in the exact same assembly as before:

Rust source

Compiled ARM64

This means that on ARM64, sequentially consistent operations are exactly as cheap as acquire and release operations. Or, rather, that ARM64 `Acquire`, `Release`, and `AcqRel` operations are as expensive as `SeqCst`. Unlike x86-64, however, `Relaxed` operations are relatively cheap, as they don't result in stronger ordering guarantees than necessary.

ARMv8.1 Atomic Release and Acquire Instructions

As discussed in "ARMv8.1 Atomic Instructions", the ARMv8.1 version of ARM64 includes CISC-style instructions for atomic operations such as `ldadd` (load and add) as an alternative to an `ldxr`/`stxr` loop.

Just like the load and store operations have special versions with acquire and release semantics, these instructions also have variants for stronger memory ordering. Because these instructions involve both a load and a store, they each have three additional variants: one for release (`-l`), one for acquire (`-a`), and one for combined release and acquire (`-al`) semantics.

For example, for `ldadd`, there is also `ldaddl`, `ldadda`, and `ldaddal`. Similarly, the `cas` instruction comes with the `casl`, `casa`, and `casal` variants.

Just like for the load and store instructions, the combined release and acquire (`-al`) variants also suffice for `SeqCst` operations.

An Experiment

An unfortunate consequence of the popularity of strongly ordered architectures is that certain classes of memory ordering bugs can easily stay undiscovered. Using `Relaxed` where `Acquire` or `Release` is necessary is incorrect, but could accidentally end up working just fine in practice on x86-64, assuming the compiler doesn't reorder your atomic operations.

> **Note**

Remember that it's not only the processor that can cause things to happen out of order. The compiler is also allowed to reorder the instructions it produces, as long as it takes the memory ordering constraints into account.

In practice, compilers tend to be very conservative about optimizations involving atomic operations, but that might very well change in the future.

This means one can easily write incorrect concurrent code that (accidentally) works perfectly fine on x86-64, but might break down when compiled for and run on an ARM64 processor.

Let's try to do exactly that.

We'll create a spin lock–protected counter, but change all the memory orderings to `Relaxed`. Let's not bother with creating a custom type or unsafe code. Instead, let's just use an `AtomicBool` for the lock and an `AtomicUsize` for the counter.

To be sure the compiler won't be the one to reorder our operations, we'll use the `std::sync::atomic::compiler_fe` function to inform the compiler of the operations that should have been `Acquire` or `Release`, without telling the processor.

We'll make four threads repeatedly lock, increment the counter, and unlock—a million times each. Putting that all together, we end up with the following code:

```
fn main() {
    let locked = AtomicBool::new(false);
    let counter = AtomicUsize::new(0);

    thread::scope(|s| {
        // Spawn four threads, that each iterate a million times.
        for _ in 0..4 {
            s.spawn(|| for _ in 0..1_000_000 {
                // Acquire the lock, using the wrong memory ordering.
                while locked.swap(true, Relaxed) {}
                compiler_fence(Acquire);

                // Non-atomically increment the counter, while holding the lock.
                let old = counter.load(Relaxed);
                let new = old + 1;
                counter.store(new, Relaxed);
```

```
                // Release the lock, using the wrong memory ordering.
                compiler_fence(Release);
                locked.store(false, Relaxed);
            });
        }
    });

    println!("{}", counter.into_inner());
}
```

If the lock works properly, we'd expect the final value of the counter to be exactly four million. Note how incrementing the counter happens in a non-atomic way, with a separate `load` and `store` rather than a single `fetch_add`, to make sure that any problems with the spin lock could result in missed increments and thus a lower total value of the counter.

Running this program a few times on a computer with an x86-64 processor gives:

```
4000000
4000000
4000000
```

As expected, we get release and acquire semantics for "free," and our mistake does not cause any issues.

Trying this on an Android phone from 2021 and a Raspberry Pi 3 model B, which both use an ARM64 processor, results in the same output:

```
4000000
4000000
4000000
```

This suggests that not all ARM64 processors make use of all forms of their instruction reordering, although we can't assume much based on this experiment.

When trying this out on a 2021 Apple iMac, which contains an ARM64-based Apple M1 processor, we get something different:

```
3988255
3982153
3984205
```

Our previously hidden mistake suddenly turned into an actual issue—an issue that is only visible on a weakly ordered system. The counter is only off by about 0.4%, showing how subtle such an issue can be. In a real-life scenario, an issue like this might stay undiscovered for a very long time.

> **Note**

Don't forget to enable optimization (with `cargo run --release` or `rustc -O`) when trying to replicate the results above. Without optimization, the same code often results in many more instructions, which can hide the subtle effects of instruction reordering.

Memory Fences

There is one type of memory ordering related instruction we haven't seen yet: memory fences. A *memory fence* or *memory barrier* instruction is used to represent a `std::sync::atomic::fence`, which we discussed in "Fences" in Chapter 3.

As we've seen before, memory ordering on x86-64 and ARM64 is all about instruction reordering. A fence instruction prevents certain types of instructions from being reordered past it.

An acquire fence must prevent preceding load operations from getting reordered with any memory operations that follow. Similarly, a release fence must prevent subsequent store operations from getting reordered with any preceding memory operations. A sequentially consistent fence must prevent all memory operations that precede it from being reordered with memory operations after the fence.

On x86-64, the basic memory ordering semantics already satisfy the needs of acquire and release fences. This architecture doesn't allow the types of reordering that these fences prevent, regardless.

Let's dive right in and see what instructions the four different fences compile to on both x86-64 and ARM64:

Rust source

Compiled x86-64

Compiled ARM64

Unsurprisingly, release and acquire fences on x86-64 do not result in any instruction. We get release and acquire semantics "for free" on this architecture. Only a `SeqCst` fence results in an `mfence` (memory fence) instruction. This instruction makes sure that all memory operations before it have been completed before continuing.

On ARM64, the equivalent instruction is `dmb ish` (data memory barrier, inner shared domain). Unlike on x86-64, it is used for `Release` and `AcqRel` as well, since this architecture doesn't implicitly provide acquire and release semantics. For `Acquire`, a slightly less impactful variant is used: `dmb ishld`. This variant only waits for load operations to complete, but freely allows preceding store operations to be reordered past it.

Similar to what we saw before with the atomic operations, we see that x86-64 gives us release and acquire fences "for free," while on ARM64, sequentially consistent fences come at the same cost as release fences.

Summary

- On x86-64 and ARM64, relaxed load and store operations are identical to their non-atomic equivalents.
- The common atomic fetch-and-modify and compare-and-exchange operations on x86-64 (and ARM64 since ARMv8.1) have their own instructions.
- On x86-64, an atomic operation for which there is no equivalent instruction compiles down to a compare-and-exchange loop.
- On ARM64, any atomic operation can be represented by a load-linked/store-conditional loop: a loop that automatically restarts if the attempted memory operation was disrupted.
- Caches operate on cache lines, which are often 64 bytes in size.
- Caches are kept consistent with a cache coherence protocol, such as write-through or MESI.
- Padding, for example through `#[repr(align(64))]`, can be useful for improving performance by preventing *false sharing*.
- A load operation can be significantly cheaper than a failed compare-and-exchange operation, in part because the latter often demands exclusive access to a cache line.
- Instruction reordering is invisible within a single-threaded program.
- On most architectures, including x86-64 and ARM64, memory ordering is about preventing certain types of instruction reordering.
- On x86-64, every memory operation has acquire and release semantics, making it exactly as cheap or expensive as a relaxed operation. Everything other than stores and fences also has sequentially consistent semantics at no extra cost.
- On ARM64, acquire and release semantics are not as cheap as relaxed operations, but do include sequentially consistent semantics at no extra cost.

A summary of the assembly instructions we've seen in this chapter can be found in Figure 7-1.

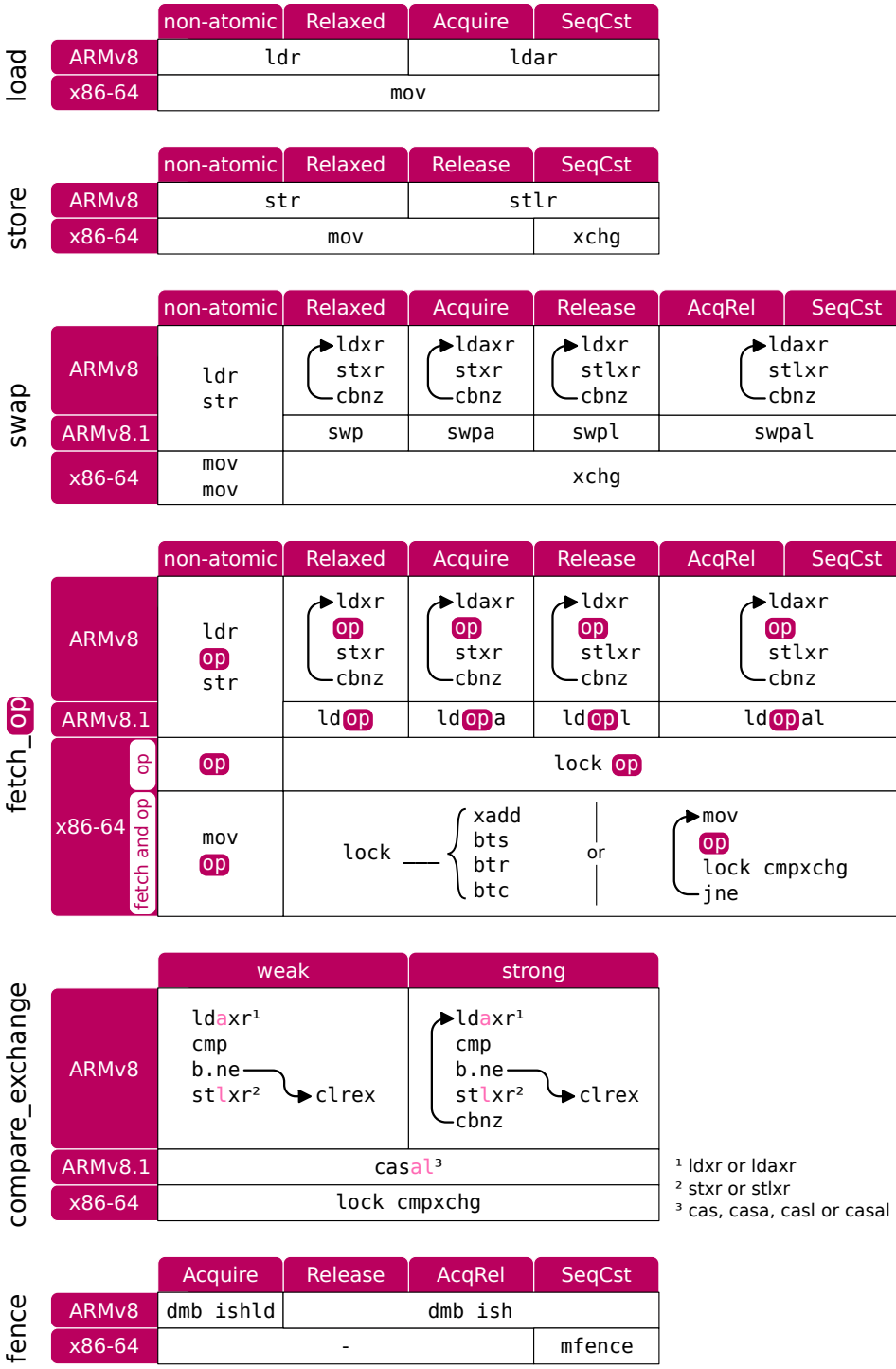


Figure 1: Figure 7-1. An overview of the instructions that the various atomic operations compile down to on ARM64 and x86-64 for each memory ordering.

Operating System Primitives

Operating System Primitives

So far, we've mostly focused on non-blocking operations. If we want to implement something like a mutex or condition variable, something that can wait for another thread to unlock or notify it, we need a way to efficiently block the current thread.

As we saw in Chapter 4, we can do this ourselves without the help of the operating system by *spinning*, repeatedly trying something over and over again, which can easily waste a lot of processor time. If we want to block efficiently, however, we need the help of the operating system's kernel.

The kernel, or more specifically the *scheduler* part of it, is responsible for deciding which process or thread gets to run when, for how long, and on which processor core. While a thread is waiting for something to happen, the kernel can stop giving it any processor time, prioritizing other threads that can make better use of this scarce resource.

We'll need a way to inform the kernel that we're waiting for something and ask it to put our thread to sleep until something relevant happens.

Interfacing with the Kernel

The way things are communicated with the kernel depends heavily on the operating system, and often even its version. Usually, the details of how this works are hidden behind one or more libraries that handle this for us. For example, using the Rust standard library, we can just call `File::open()` to open a file, without having to know any details about the operating system's kernel interface. Similarly, using the C standard library, `libc`, one can call the standard `fopen()` function to open a file. Calling such a function will eventually result in a call into the operating system's kernel, also known as a *syscall*, which is usually done through a specialized processor instruction. (On some architectures, that instruction is literally called `syscall`.)

Programs are generally expected, sometimes even required, to not make any syscalls directly, but to make use of higher-level libraries that ship with the operating system. On Unix systems, such as those based on Linux, `libc` takes this special role of providing the standard interface to the kernel.

The "Portable Operating System Interface" standard, more commonly known as the POSIX standard, includes additional requirements for `libc` on Unix systems. For example, next to the `fopen()` function from the C standard, POSIX additionally requires the existence of the lower-level `open()` and `openat()` functions for opening files, which often correspond directly to a syscall. Because of the special status of `libc` on Unix systems, programs written in languages other than C usually still use `libc` for all their interactions with the kernel.

Rust software, including the standard library, often makes use of `libc` through the identically named `libc` crate.

For Linux specifically, the syscall interface is guaranteed to be stable, allowing us to make syscalls directly, without using `libc`. While that's not the most common or most advised route, it is slowly becoming more popular.

However, on macOS, also a Unix operating system that follows the POSIX standard, the syscall interface of the kernel is not stable, and we're not supposed to use it directly. The only stable interfaces that programs are allowed to use are provided through the libraries that ship with the system, such as `libc`, `libc++`, and various other libraries for C, C++, Objective-C, and Swift, Apple's programming languages of choice.

Windows does not follow the POSIX standard. It does not ship with an extended `libc` that serves as the main interface to the kernel, but instead ships with a separate set of libraries, such as `kernel32.dll`, that provide Windows-specific functions, such as `CreateFileW` for opening files. Just like on macOS, we're not supposed to make use of undocumented lower-level functions or make syscalls directly.

Through their libraries, operating systems provide us with synchronization primitives that need to interact with the kernel, such as mutexes and condition variables. Which part of their implementation is part of such a library or part of the kernel varies heavily per operating system. For example, sometimes the mutex lock and unlock operations correspond directly with a kernel syscall, whereas on other systems, the library handles most of the operations and will only perform a syscall when a thread needs to be blocked or woken up. (The latter tends to be more efficient, as making a syscall can be slow.)

POSIX

As part of the POSIX Threads extensions, better known as pthreads, POSIX specifies data types and functions for concurrency. While technically specified as part of a separate system library, `libpthread`, this functionality is nowadays often included in `libc` directly.

Next to functionality like spawning and joining threads (`pthread_create` and `pthread_join`), pthread provides the most common synchronization primitives: mutexes (`pthread_mutex_t`), reader-writer locks (`pthread_rwlock_t`), and condition variables (`pthread_cond_t`).

`pthread_mutex_t`
Pthread's mutex must be initialized by calling `pthread_mutex_init()` and destroyed with `pthread_mutex_destroy()`. The initialization function takes an argument of type `pthread_mutexattr_t` that can be used to configure some of the properties of the mutex.

One of those properties is its behavior on *recursive locking*, which happens when the same thread that already holds a lock attempts to lock it again. This results in undefined behavior when using the default setting (`PTHREAD_MUTEX_DEFAULT`), but it can also be configured to result in an error (`PTHREAD_MUTEX_ERRORCHECK`), in a deadlock (`PTHREAD_MUTEX_NORMAL`), or in a successful second lock (`PTHREAD_MUTEX_RECURSIVE`).

These mutexes are locked through `pthread_mutex_lock()` or `pthread_mutex_trylock()`, and unlocked through `pthread_mutex_unlock()`. Additionally, unlike Rust's standard mutex, they also support locking with a time limit, through `pthread_mutex_timedlock()`.

A `pthread_mutex_t` can be statically initialized without a call to `pthread_mutex_init()` by assigning it the value `PTHREAD_MUTEX_INITIALIZER`. However, this is only possible for a mutex with default settings.

`pthread_rwlock_t`
Pthread's reader-writer lock is initialized and destroyed through `pthread_rwlock_init()` and `pthread_rwlock_destroy()`. Similar to a mutex, a default `pthread_rwlock_t` can also be initialized statically with `PTHREAD_RWLOCK_INITIALIZER`.

A pthread reader-writer lock has significantly fewer properties that can be configured through its initialization function, compared to a pthread mutex. Most notably, attempting to recursively write-lock it will always result in a deadlock.

Attempts to recursively acquire additional read locks, however, are guaranteed to succeed, even if there are writers waiting. This effectively rules out any efficient implementation that prioritizes writers over readers, which is why most pthread implementations prioritize readers.

Its interface is nearly identical to that of `pthread_mutex_t`, including support for time limits, except each locking function comes in two variants: one for readers (`pthread_rwlock_rdlock`), and one for writers (`pthread_rwlock_wrlock`). Perhaps surprisingly, there is only one unlock function (`pthread_rwlock_unlock`) that's used for unlocking a lock of either kind.

`pthread_cond_t`
A pthread condition variable is used together with a pthread mutex. It is initialized and destroyed through `pthread_cond_init` and `pthread_cond_destroy`, and has a few attributes that can be configured. Most notably, we can configure whether time limits will use a monotonic clock (like Rust's `Instant`) or real-time clock (like Rust's `SystemTime`, somtimes called "wall-clock time"). A condition variable with default settings, such as one statically initialized by `PTHREAD_COND_INITIALIZER`, uses the real-time clock.

Waiting for such a condition variable is done through `pthread_cond_wait()` or, with a time limit, `pthread_cond_timedwait()`. Waking up a waiting thread is done by calling `pthread_cond_signal()`, or, to wake all waiting threads at once, `pthread_cond_broadcast()`.

The remaining synchronization primitives provided by pthread are barriers (`pthread_barrier_t`), spin locks (`pthread_spinlock_t`), and one-time initialization (`pthread_once_t`), which we will not discuss.

Wrapping in Rust

It might seem like we could easily expose these pthread synchronization primitives to Rust by conveniently wrapping their C type (through the `libc` crate) in a Rust struct, like this:

```
pub struct Mutex {
    m: libc::pthread_mutex_t,
}
```

However, there are a few issues with that, as this pthread type was designed for C, not for Rust.

First of all, Rust has rules about mutability and borrowing, which normally don't allow for mutations to something when shared. Since functions like `pthread_mutex_lock` will most likely mutate the

mutex, we'll need interior mutability to make sure that's acceptable. So, we'll have to wrap it in an `UnsafeCell`:

```
pub struct Mutex {
    m: UnsafeCell<libc::pthread_mutex_t>,
}
```

A much bigger problem is related to *moving*.

In Rust, we move objects around all the time. For example, by returning an object from a function, passing it as an argument, or simply assigning it to a new place. Everything that we own (and that isn't borrowed by anything else), we can freely move into a new place.

In C, however, that's not universally true. It's quite common for a type in C to depend on its memory address staying constant. For example, it might contain a pointer that points into itself, or store a pointer to itself in some global data structure. In that case, moving it to a new place could result in undefined behavior.

The pthread types we discussed do not guarantee they are *movable*, which becomes quite a problem in Rust. Even a simple idiomatic `Mutex::new()` function is a problem: it would return a mutex object, which would move it into a new place in memory.

Since a user could always move around any mutex object they own, we either need to make them promise they won't do that, by making the interface `unsafe`; or we need to take away their ownership and hide everything behind a wrapper (something that `std::pin::Pin` can be used for). Neither of these are good solutions, as they impact the interface of our mutex type, making it very error-prone and/or inconvenient to use.

A solution to this problem is to wrap the mutex in a `Box`. By putting the pthread mutex in its own allocation, it stays in the same location in memory, even if its owner is moved around.

```
pub struct Mutex {
    m: Box<UnsafeCell<libc::pthread_mutex_t>>,
}
```

> **Note**

This is how `std::sync::Mutex` was implemented on all Unix platforms before Rust 1.62.

The downside of this approach is the overhead: every mutex now gets its own allocation, adding significant overhead to creating, destroying, and using the mutex. Another downside is that it prevents the `new` function from being `const`, which gets in the way of having a `static` mutex.

Even if `pthread_mutex_t` was movable, a `const fn new` could only initialize it with default settings, which results in undefined behavior when locking recursively. There is no way to design a safe interface that prevents locking recursively, so this means we'd need to make the `lock` function `unsafe` to make the user promise they won't do that.

A problem that remains with our `Box` approach occurs when dropping a locked mutex. It might seem that, with the right design, it would not be possible to drop a `Mutex` while locked, since it's

impossible to drop it while it's still borrowed by a `MutexGuard`. The `MutexGuard` would have to be dropped first, unlocking the `Mutex`. However, in Rust, it's safe to forget (or leak) an object, without dropping it. This means one could write something like this:

```
fn main() {
    let m = Mutex::new(..);

    let guard = m.lock(); // Lock it ..
    std::mem::forget(guard); // .. but don't unlock it.
}
```

In the example above, `m` will be dropped at the end of the scope while it is still locked. This is fine, according to the Rust compiler, because the guard has been leaked and can't be used anymore.

However, pthread specifies that calling `pthread_mutex_destroy()` on a locked mutex is not guaranteed to work and might result in undefined behavior. One work-around is to first attempt to lock (and unlock) the pthread mutex when dropping our `Mutex`, and panic (or leak the `Box`) when it is already locked, but that adds even more overhead.

These issues don't just apply to `pthread_mutex_t`, but to the other types we discussed as well. Overall, the design of the pthread synchronization primitives is fine for C, but just not a great fit for Rust.

Linux

On Linux systems, the pthread synchronization primitives are all implemented using the *futex* syscall. Its name comes from "fast user-space mutex," as the original motivation for adding this syscall was to allow libraries (like pthread implementations) to include a fast and efficient mutex implementation. It's much more flexible than that though and can be used to build many different synchronization tools.

The futex syscall was added to the Linux kernel in 2003 and has seen several improvements and extensions since then. Some other operating systems have since also added similar functionality, most notably Windows 8 in 2012 with the addition of `WaitOnAddress` (which we'll discuss a bit later, in "Windows"). In 2020, the C++ language even added support for basic futex-like operations to its standard library, with the addition of the `atomic_wait` and `atomic_notify` functions.

Futex

On Linux, `SYS_futex` is a syscall that implements various operations that all operate on a 32-bit atomic integer. The main two operations are `FUTEX_WAIT` and `FUTEX_WAKE`. The wait operation puts a thread to sleep, and a wake operation on the same atomic variable wakes the thread up again.

These operations don't store anything in the atomic integer. Instead, the kernel remembers which threads are waiting on which memory address to allow a wake operation to wake up the right threads.

In "Waiting: Parking and Condition Variables" in Chapter 1, we saw how other mechanisms for blocking and waking up threads need a way to make sure that wake operations don't get lost in a race. For thread parking, that issue is solved by making the `unpark()` operation also apply to future `park()` operations as well. And for condition variables, that's taken care of by the mutex that is used together with the condition variable.

For the futex wait and wake operations, another mechanism is used. The wait operation takes an argument which specifies the value we expect the atomic variable to have and will refuse to block if it doesn't match. The wait operation behaves atomically with respect to the wake operation, meaning that no wake signal can get lost between the check of the expected value and the moment it actually goes to sleep.

If we make sure that the value of the atomic variable is changed right before a wake operation, we can be sure that a thread that's about to start waiting will not go to sleep, such that potentially missing the futex wake operation no longer matters.

Let's go over a minimal example to see this in practice.

First, we need to be able to invoke these syscalls. We can use the `syscall` function from the `libc` crate to do so, and wrap each of them in a convenient Rust function, as follows:

```
#[cfg(not(target_os = "linux"))]
compile_error!("Linux only. Sorry!");

pub fn wait(a: &AtomicU32, expected: u32) {
    // Refer to the futex (2) man page for the syscall signature.
    unsafe {
        libc::syscall(
            libc::SYS_futex, // The futex syscall.
            a as *const AtomicU32, // The atomic to operate on.
            libc::FUTEX_WAIT, // The futex operation.
            expected, // The expected value.
            std::ptr::null::<libc::timespec>(), // No timeout.
        );
    }
}

pub fn wake_one(a: &AtomicU32) {
    // Refer to the futex (2) man page for the syscall signature.
    unsafe {
        libc::syscall(
            libc::SYS_futex, // The futex syscall.
            a as *const AtomicU32, // The atomic to operate on.
            libc::FUTEX_WAKE, // The futex operation.
            1, // The number of threads to wake up.
        );
    }
}
```

Now, as a usage example, let's use these to make a thread wait for another. We'll use an atomic variable that we initialize at zero, which the main thread will futex-wait on. A second thread will change the variable to one, and then run a futex wake operation on it to wake up the main thread.

Just like thread parking and waiting on a condition variable, a futex wait operation can *spuriously* wake up, even when nothing happened. Therefore, it is most commonly used in a loop, repeating it if the condition we're waiting for isn't met yet.

Let's take a look at the example below:

```
fn main() {
    let a = AtomicU32::new(0);

    thread::scope(|s| {
        s.spawn(|| {
            thread::sleep(Duration::from_secs(3));
            a.store(1, Relaxed); // (1)
            wake_one(&a); // (2)
        });

        println!("Waiting...");
        while a.load(Relaxed) == 0 { // (3)
            wait(&a, 0); // (4)
        }
        println!("Done!");
    });
}
```

An important observation to make here is that the `wait` call is avoided entirely if `a` was already set to one before the `while` loop. In a similar fashion, if the main thread had also stored in the atomic variable whether it started waiting for the signal (by setting it to a value other than zero or one), the signaling thread could skip the futex wake operation if the main thread hadn't started waiting yet. This is what makes futex-based synchronization primitives so fast: since we manage the state ourselves, we don't need to rely on the kernel, except when we actually need to block.

> **Note**

Since Rust 1.48, the standard library's thread parking functions on Linux are implemented like this. They use one atomic variable per thread, with three possible states: zero for the idle and initial state, one for "unparked but not yet parked," and minus one for "parked but not yet unparked."

In Chapter 9, we'll implement mutexes, condition variables, and reader-writer locks using these operations.

Futex Operations

Next to the wait and wake operations, the futex syscall supports several other operations. In this section, we'll briefly discuss every operation supported by this syscall.

The first argument to the futex is always a pointer to the 32-bit atomic variable to operate on. The second argument is a constant representing the operation, such as `FUTEX_WAIT`, to which up to two flags can be added: `FUTEX_PRIVATE_FLAG` and/or `FUTEX_CLOCK_REALTIME`, which we will discuss below. The remaining arguments depend on the operation and are described for each of the operations below.

`FUTEX_WAIT`
This operation takes two additional arguments: the value the atomic variable is expected to have and a pointer to a `timespec` representing the maximum time to wait.

If the value of the atomic variable matches the expected value, the wait operation blocks until woken up by one of the wake operations, or until the duration specified by the `timespec` has passed. If the pointer to the `timespec` is null, there is no time limit. Additionally, the wait operation might spuriously wake up and return without a corresponding wake operation, before the time limit is reached.

The check and blocking operation happens as a single atomic operation with respect to other futex operations, meaning that no wake signals can get lost between them.

The duration specified by the `timespec` represents a duration on the monotonic clock (like Rust's `Instant`) by default. By adding the `FUTEX_CLOCK_REALTIME` flag, the real-time clock (like Rust's `SystemTime`) is used instead.

The return value indicates whether the expected value matched and whether the timeout was reached or not.

`FUTEX_WAKE`
This operation takes one additional argument: the number of threads to wake up, as an `i32`.

This wakes up as many threads as specified that are blocked in a wait operation on the same atomic variable. (Or fewer if there are not that many waiting threads.) Most commonly, this argument is either one to wake up just one thread, or `i32::MAX` to wake up all threads.

The number of awoken threads is returned.

`FUTEX_WAIT_BITSET`
This operation takes four additional arguments: the value the atomic variable is expected to have, a pointer to a `timespec` representing the maximum time to wait, a pointer that's ignored, and a 32-bit "bitset" (a `u32`).

This operation behaves the same as `FUTEX_WAIT`, with two differences.

The first difference is that it takes a `bitset` argument that can be used to wait for only specific wake operations, rather than all wake operations on the same atomic variable. A `FUTEX_WAKE` operation is never ignored, but a signal from a `FUTEX_WAKE_BITSET` operation is ignored if the wait bitset and the wake bitset do not have any 1-bit in common.

For example, a `FUTEX_WAKE_BITSET` operation with a bitset of `0b0101` will wake up a `FUTEX_WAIT_BITSET` operation with a bitset of `0b1100`, but not one with a bitset of `0b0010`.

This might be useful when implementing something like a reader-writer lock, to wake up a writer without waking up any readers. However, note that using two separate atomic variables can be more efficient than using a single one for two different kinds of waiters, since the kernel will keep a single list of waiters per atomic variable.

The other difference with `FUTEX_WAIT` is that the `timespec` is used as an absolute timestamp, rather than a duration. Because of this, `FUTEX_WAIT_BITSET` is often used with a bitset of `u32::MAX` (all bits set), effectively turning it into a regular `FUTEX_WAIT` operation, but with an absolute timestamp for the time limit.

`FUTEX_WAKE_BITSET`
This operation takes four additional arguments: the number of threads to wake up, two pointers that are ignored, and a 32-bit "bitset" (a `u32`).

This operation is identical to `FUTEX_WAKE`, except it does not wake up `FUTEX_WAIT_BITSET` operations with a bitset that does not overlap. (See `FUTEX_WAIT_BITSET` above.)

With a bitset of `u32::MAX` (all bits set), this is identical to `FUTEX_WAKE`.

`FUTEX_REQUEUE`
This operation takes three additional arguments: the number of threads to wake up (an `i32`), the number of threads to requeue (an `i32`), and the address of a secondary atomic variable.

This operation wakes up a given number of waiting threads, and then *requeues* a given number of remaining waiting threads to instead wait on another atomic variable.

Waiting threads that are requeued continue to wait, but are no longer affected by wake operations on the primary atomic variable. Instead, they are now woken up by wake operations on the secondary atomic variable.

This can be useful for implementing something like the "notify all" operation of a condition variable. Instead of waking up all threads, which will subsequently try to lock a mutex, most likely making all but one thread wait for that mutex right afterwards, we could only wake up a single thread and requeue all the others to directly wait for the mutex without waking them up first.

Just like with the `FUTEX_WAKE` operation, the value of `i32::MAX` can be used to requeue all waiting threads. (Specifying a value of `i32::MAX` for the number of threads to wake up is not very useful, since that will make this operation equivalent to `FUTEX_WAKE`.)

The number of awoken threads is returned.

`FUTEX_CMP_REQUEUE`
This operation takes four additional arguments: the number of threads to wake up (an `i32`), the number of threads to requeue (an `i32`), the address of a secondary atomic variable, and the value the primary atomic variable is expected to have.

This operation is nearly identical to `FUTEX_REQUEUE`, except it refuses to operate if the value of the primary atomic variable does not match the expected value. The check of the value and the requeuing operation happens atomically with respect to other futex operations.

Unlike `FUTEX_REQUEUE`, this returns the sum of the number of awoken and requeued threads.

`FUTEX_WAKE_OP`
This operation takes four additional arguments: the number of threads to wake up on the primary atomic variable (an `i32`), the number of threads to potentially wake up on the second atomic variable (an `i32`), the address of a secondary atomic variable, and a 32-bit value encoding both an operation and a comparison to be made.

This is a very specialized operation that modifies the secondary atomic variable, wakes a number of threads waiting on the primary atomic variable, checks if the previous value of the atomic variable matches a given condition, and if so, also wakes a number of threads on a secondary atomic variable.

In other words, it is identical to the following code, except the entire operation behaves atomically with respect to other futex operations:

```
let old = atomic2.fetch_update(Relaxed, Relaxed, some_operation);
wake(atomic1, N);
if some_condition(old) {
    wake(atomic2, M);
}
```

The modifying operation to perform and the condition to check are both specified by the last argument to the syscall, encoded in its 32 bits. The operation can be one of the following: assignment, addition, binary `or`, binary `and-not`, and binary `xor`, with either a 12-bit argument or a 32-bit argument that's a power of two. The comparison can be chosen from `==`, `!=`, `<`, `<=`, `>`, and `>=`, with a 12-bit argument.

See the `futex(2)` Linux man page for details on the encoding of this argument, or use the `linux-futex` crate on crates.io, which includes a convenient way to construct this argument.

This operation returns the total number of awoken threads.

At first glance this may seem like a flexible operation with many use cases. However, it was designed for just one specific use case in GNU `libc` where two threads had to be woken up from two separate atomic variables. That specific case has been replaced by a different implementation that no longer makes use of `FUTEX_WAKE_OP`.

The `FUTEX_PRIVATE_FLAG` can be added to any of these operations to enable a possible optimization if all relevant futex operations on the same atomic variable(s) come from threads of the same process, which is usually the case. To make use of it, every relevant futex operation must include this same flag. By allowing the kernel to assume there will be no interactions with other processes, it can skip some otherwise potentially expensive steps in performing futex operations, improving performance.

In addition to Linux, NetBSD also supports all the futex operations described above. OpenBSD also has a futex syscall, but only supports the `FUTEX_WAIT`, `FUTEX_WAKE`, and `FUTEX_REQUEUE` operations. FreeBSD does not have a native futex syscall, but does include a syscall called `_umtx_op`, which includes functionality nearly identical to `FUTEX_WAIT` and `FUTEX_WAKE`: `UMTX_OP_WAIT` (for 64-bit atomics), `UMTX_OP_WAIT_UINT` (for 32-bit atomics), and `UMTX_OP_WAKE`. Windows also includes functions that behave very similarly to the futex wait and wake operations, which we will discuss later in this chapter.

New Futex Operations

As of Linux 5.16, released in 2022, there is an additional futex syscall: `futex_waitv`. This new syscall allows waiting for more than one futex at once, by providing it a list of atomic variables

(and their expected values) to wait on. A thread blocked on `futex_waitv` can be woken up by a wake operation on any of the specified variables.

This new syscall also leaves space for future extensions. For example, it's possible to specify the size of the atomic variable to wait on. While the initial implementation only supports 32-bit atomics, just like the original `futex` syscall, this might be extended in the future to include 8-bit, 16-bit, and 64-bit atomics.

Priority Inheritance Futex Operations

Priority inversion is a problem that occurs when a high-priority thread is blocked on a lock held by a low-priority thread. The high-priority thread effectively has its priority "inverted," since it now has to wait for the low-priority thread to release the lock before it can make progress.

A solution to this problem is *priority inheritance*, in which the lock-holding thread inherits the priority of the highest-priority thread that is waiting for it, temporarily increasing the priority of the low-priority thread while it holds the lock.

In addition to the seven futex operations we discussed before, there are six priority inheriting futex operations specifically designed for implementing *priority inheriting* locks.

The general futex operations we discussed before do not have any requirements for the exact contents of the atomic variable. We got to choose ourselves what the 32 bits represent. However, for a priority inheriting mutex, the kernel needs to be able to understand whether the mutex is locked, and if so, which thread has locked it.

To avoid having to make a syscall on every state change, the priority inheritance futex operations specify the exact contents of the 32-bit atomic variable, so the kernel can understand it: the highest bit represents whether there are any threads waiting to lock the mutex, and the lowest 30 bits contain the thread ID (the Linux `tid`, not the Rust `ThreadId`) of the thread that holds the lock, or zero when unlocked.

As an extra feature, the kernel will set the second highest bit if the thread that holds the lock terminates without unlocking it, but only if there are any waiters. This allows for the mutex to be *robust*: a term used to describe a mutex that can gracefully handle a situation where its "owning" thread unexpectedly terminates.

The priority inheriting futex operations have a one to one correspondence to the standard mutex operations: `FUTEX_LOCK_PI` for locking, `FUTEX_UNLOCK_PI` for unlocking, and `FUTEX_TRYLOCK_PI` for locking without blocking. Additionally, the `FUTEX_CMP_REQUEUE_PI` and `FUTEX_WAIT_REQUEUE_PI` operations can be used to implement a condition variable that pairs with a priority inheriting mutex.

We'll not discuss these operations in detail. See the `futex(2)` Linux man page or the `linux-futex` crate on crates.io for their details.

macOS

The kernel that's part of macOS supports various useful low-level concurrency related syscalls. However, just like most operating systems, the kernel interface is not considered stable, and we're not supposed to use it directly.

The only way software should interact with the macOS kernel is through the libraries that ship with the system. These libraries include its standard library implementations for C (libc), C++ (libc++), Objective-C, and Swift.

As a POSIX-compliant Unix system, the macOS C library includes a full pthread implementation. The standard locks in other languages tend to use pthread's primitives under the hood.

Pthread's locks tend to be relatively slow on macOS compared to the equivalent on other operating systems. One of the reasons is that the locks on macOS behave as *fair locks* by default. This means that when several threads attempt to lock the same mutex, they are served in order of arrival, like a perfect queue. While fairness can be a desirable property, it can significantly reduce performance, especially under high contention.

os_unfair_lock

Next to the pthread primitives, macOS 10.12 introduced a new lightweight platform-specific mutex, which is not fair: `os_unfair_lock`. It is just 32 bits in size, initialized statically with the `OS_UNFAIR_LOCK_INIT` constant, and does not require destruction. It can be locked through `os_unfair_lock_lock()` (blocking) or `os_unfair_lock_trylock()` (non-blocking), and is unlocked through `os_unfair_lock_unlock()`.

Unfortunately, it does not come with a condition variable, nor does it have a reader-writer variant.

Windows

The Windows operating system ships with a set of libraries that together form the *Windows API*, often called the "Win32 API" (even on 64-bit systems). This forms a layer on top of the "Native API": the largely undocumented interface with the kernel, which we're not supposed to use directly.

The Windows API is made available to Rust programs through Microsoft's official `windows` and `windows-sys` crates, which are available on crates.io.

Heavyweight Kernel Objects

Many of the older synchronization primitives available on Windows are managed fully by the kernel, making them quite heavyweight, and giving them similar properties as other kernel-managed objects, such as files. They can be used by multiple processes, they can be named and located by their name, and they support fine-grained permissions, similar to files. For example, it's possible to allow a process to wait on some object, without allowing it to send signals through it to wake others.

These heavyweight kernel-managed synchronization objects include `Mutex` (which can be locked and unlocked), `Event` (which can be signalled and waited for), and `WaitableTimer` (which can be automatically signalled after a chosen time, or periodically). Creating such an object results in a `HANDLE`, just like opening a file, that can be easily passed around and used with the regular `HANDLE` functions; most notably the family of wait functions. These functions allow us to wait for one or more objects of various types, including the heavyweight synchronization primitives, processes, threads, and various forms of I/O.

Lighter-Weight Objects

A lighter-weight synchronization primitive included in the Windows API is the "critical section."

The term *critical section* refers to a part of a program, a "section" of code, that may not be entered concurrently by more than one thread. The mechanism for protecting a critical section is often called a mutex. In this case, however, Microsoft used the name "critical section" for the mechanism, quite possibly because the name "mutex" was already taken by the heavyweight `Mutex` object discussed above.

A Windows `CRITICAL_SECTION` is effectively a *recursive mutex*, except it uses the terms "enter" and "leave" rather than "lock" and "unlock." As a recursive mutex, it is designed to only protect against other threads. It allows the same thread to lock (or "enter") it more than once, requiring it to also unlock (leave) it the same number of times.

This is something to keep in mind when wrapping this type in Rust. Successfully locking (entering) a `CRITICAL_SECTION` shouldn't result in an exclusive reference (`&mut T`) to data protected by it. Otherwise, a thread could use this to create two exclusive references to the same data, which immediately results in undefined behavior.

A `CRITICAL_SECTION` is initialized using the `InitializeCriticalSection()` function, destroyed with `DeleteCriticalSection()`, and may not be moved. It is locked through `EnterCriticalSection()` or `TryEnterCriticalSection()`, and unlocked with `LeaveCriticalSection()`.

> **Note**

Until Rust 1.51, `std::sync::Mutex` on Windows XP was based on a (`Box`-allocated) `CRITICAL_SECTION` object. (Rust 1.51 dropped support for Windows XP.)

Slim reader-writer locks

As of Windows Vista (and Windows Server 2008), the Windows API includes a much nicer locking primitive that's very lightweight: the *slim reader-writer lock*, or *SRW lock* for short.

The `SRWLOCK` type is just one pointer in size, can be statically initialized with `SRWLOCK_INIT`, and does not require destruction. While not in use (borrowed), we're even allowed to move it, making it an excellent candidate for being wrapped in a Rust type.

It provides exclusive (writer) locking and unlocking through `AcquireSRWLockExclusive()`, `TryAcquireSRWLockExclusive()`, and `ReleaseSRWLockExclusive()`, and provides shared

(reader) locking and unlocking through `AcquireSRWLockShared()`, `TryAcquireSRWLockShared()`, and `ReleaseSRWLockShared()`. It is often used as a regular mutex, simply by ignoring the shared (reader) locking functions.

An SRW lock prioritizes neither writers nor readers. While not guaranteed, it attempts to serve all lock requests in order, as far as possible without reducing performance. One must not attempt to acquire a second shared (reader) lock on a thread that already holds one. Doing so could lead to a permanent deadlock if the operation gets queued behind an exclusive (writer) lock operation of another thread, which would be blocked because of the first shared (reader) lock that the first thread already holds.

The SRW lock was introduced to the Windows API together with the condition variable. Similar to an SRW lock, a `CONDITION_VARIABLE` is just one pointer in size, can be initialized statically with `CONDITION_VARIABLE_INIT`, and does not require destruction. We're also allowed to move it, as long as it isn't in use (borrowed).

This condition variable can not only be used together with an SRW lock, through `SleepConditionVariableSRW`, but also with a critical section, through `SleepConditionVariableCS`.

Waking up waiting threads is done either through `WakeConditionVariable` to wake a single thread, or `WakeAllConditionVariable` to wake all waiting threads.

> **Note**

Originally, Windows SRW locks and condition variables used in the standard library were wrapped in a `Box` to avoid moving the objects. Microsoft didn't document the movability guarantees until we requested that in 2020. Since then, as of Rust 1.49, `std::sync::Mutex`, `std::sync::RwLock`, and `std::sync::Condvar` on Windows Vista and later directly wrap an `SRWLOCK` or `CONDITION_VARIABLE`, without any allocations.

Address-Based Waiting

Windows 8 (and Windows Server 2012) introduced a new, more flexible, type of synchronization functionality that is very similar to the Linux `FUTEX_WAIT` and `FUTEX_WAKE` operations we discussed earlier in this chapter.

The `WaitOnAddress` function can operate on an 8-bit, 16-bit, 32-bit, or 64-bit atomic variable. It takes four arguments: the address of the atomic variable, the address of a variable that holds the expected value, the size of the atomic variable (in bytes), and the maximum number of milliseconds to wait before giving up (or `u32::MAX` for an infinite timeout).

Just like the `FUTEX_WAIT` operation, it compares the value of the atomic variable with the expected value, and goes to sleep if they match, waiting for a corresponding wake operation. The check and sleep operation happens atomically with respect to wake operations, meaning no wake signal can get lost in between.

Waking a thread that's waiting on `WaitOnAddress` is done through either `WakeByAddressSingle` to wake a single thread, or `WakeByAddressAll` to wake all waiting threads. These two functions take just a single argument: the address of the atomic variable, which was also passed to `WaitOnAddress`.

Some, but not all, of the synchronization primitives of the Windows API are implemented using these functions. More importantly, they are a great building block for building our own primitives, which we will do in Chapter 9.

Summary

- A *syscall* is a call into the operating system's kernel and is relatively slow compared to a regular function call.
- Usually, programs don't make syscalls directly, but instead go through the operating system's libraries (e.g., `libc`) to interface with the kernel. On many operating systems, this is the only supported way of interfacing with the kernel.
- The `libc` crate gives Rust code access to `libc`.
- On POSIX systems, `libc` includes more than what's required by the C standard to comply with the POSIX standard.
- The POSIX standard includes *pthreads*, a library with concurrency primitives such as `pthread_mutex_t`.
- Pthread types are designed for C, not for Rust. For example, they are not movable, which can be a problem.
- Linux has a *futex* syscall supporting several waiting and waking operations on an `AtomicU32`. The wait operation verifies the expected value of the atomic, which is used to avoid missing notifications.
- In addition to pthreads, macOS also provides `os_unfair_lock` as a lightweight locking primitive.
- Windows heavyweight concurrency primitives always require interacting with the kernel, but can be passed between processes and used with the standard Windows waiting functions.
- Windows lightweight concurrency primitives include a "slim" reader-writer lock (*SRW lock*) and a condition variable. These are easily wrapped in Rust, as they are movable.
- Windows also provides basic futex-like functionality, through `WaitOnAddress` and `WakeByAddress`.

Complete Runnable Examples

The following complete, tested programs illustrate the primitives and patterns developed throughout this chapter.

Example: `ch8-01-futex.rs` (01 Futex)

```rust
#![cfg(target_os = "linux")]

use std::sync::atomic::AtomicU32;
use std::sync::atomic::Ordering::Relaxed;
use std::thread;
use std::time::Duration;

pub fn wait(a: &AtomicU32, expected: u32) {
    // Refer to the futex (2) man page for the syscall signature.
    unsafe {
        libc::syscall(
            libc::SYS_futex, // The futex syscall.
            a as *const AtomicU32, // The atomic to operate on.
            libc::FUTEX_WAIT, // The futex operation.
            expected, // The expected value.
            std::ptr::null::<libc::timespec>(), // No timeout.
        );
    }
}

pub fn wake_one(a: &AtomicU32) {
    // Refer to the futex (2) man page for the syscall signature.
    unsafe {
        libc::syscall(
            libc::SYS_futex, // The futex syscall.
            a as *const AtomicU32, // The atomic to operate on.
            libc::FUTEX_WAKE, // The futex operation.
            1, // The number of threads to wake up.
        );
    }
}

fn main() {
    let a = AtomicU32::new(0);

    thread::scope(|s| {
        s.spawn(|| {
            thread::sleep(Duration::from_secs(3));
            a.store(1, Relaxed);
            wake_one(&a);
        });

        println!("Waiting...");
        while a.load(Relaxed) == 0 {
```

```
            wait(&a, 0);
        }
        println!("Done!");
    });
}
```

Building Our Own Locks

Building Our Own Locks

In this chapter, we'll build our own mutex, condition variable, and reader-writer lock. For each of them, we'll start with a very basic version, and then extend it to make it more efficient.

Since we're not going to use the lock types from the standard library (which would be cheating), we're going to have to use the tools from Chapter 8 to be able to make threads wait without busy-looping. However, as we saw in that chapter, the available tools the operating system provides vary a lot per platform, making it hard to build something that works cross-platform.

Fortunately, most modern operating systems support futex-like functionality, or at least the wake and wait operations. As we saw in Chapter 8, Linux has supported them since 2003 with the `futex` syscall, Windows since 2012 with the `WaitOnAddress` family of functions, FreeBSD since 2016 as part of the `_umtx_op` syscall, and so on.

The most notable exception is macOS. While its kernel does support these operations, they are not exposed through any stable, publicly usable C function that we can use. However, macOS does ship with a recent version of libc++, an implementation of the C++ standard library. This library includes support for C++20, which is the version of C++ that comes with built-in support for very basic atomic wait and wake operations (like `std::atomic<T>::wait()`). While it's somewhat tricky to make use of that from Rust for a variety of reasons, it is certainly possible, giving us access to basic futex-like wait and wake functionality on macOS as well.

We'll not dive into the dirty details, but instead make use of the `atomic-wait` crate from crates.io to provide us with the building blocks for our locking primitives. This crate provides just three functions: `wait()`, `wake_one()`, and `wake_all()`. It implements these for all the major platforms, using the various platform-specific implementations we've discussed above. This means we no longer have to think about any platform-specific details, as long as we stick to these three functions.

These functions behave like the identically named ones we implemented in "Futex" in Chapter 8 for Linux, but let's quickly recall how they work:

`wait(&AtomicU32, u32)`
This function is used to wait until the atomic variable no longer contains the given value. It blocks if the value stored in the atomic variable is equal to the given value. When another thread modifies the value of the atomic variable, that thread needs to call one of the wake functions below, on the same atomic variable, to wake up the waiting thread from its sleep.

This function might return spuriously, without a corresponding wake operation. So make sure to check the value of the atomic variable after it returns, and repeat the `wait()` call if necessary.

`wake_one(&AtomicU32)`
This wakes up a single thread that's currently blocked on `wait()` on the same atomic variable. Use this right after modifying the atomic variable, to inform one waiting thread of the change.

`wake_all(&AtomicU32)`
This wakes up all threads that are currently blocked on `wait()` on the same atomic variable. Use this right after modifying the atomic variable, to inform the waiting threads of the change.

Only 32-bit atomics are supported, because that's the only size that's supported on all major platforms.

> **Note**

In "Futex" in Chapter 8, we discussed a very minimal example that shows how these functions are used in practice. If you've forgotten, make sure you check out that example before continuing.

To use the `atomic-wait` crate, add `atomic-wait = "1"` to the `[dependencies]` section in your `Cargo.toml`; or run `cargo add atomic-wait@1`, which will do that for you. The three functions are defined in the root of the crate and can be imported with `use atomic_wait::{wait, wake_one, wake_all};`.

> **Note**

There might be later versions of this crate available by the time you're reading this, but only major version 1 is made for this chapter. Later versions might not have a compatible interface.

Now that we have our basic building blocks ready, let's get started.

Mutex

We'll take our `SpinLock<T>` type from Chapter 4 as our reference while building our `Mutex<T>`. The parts not involved in blocking, such as the design of the guard type, will remain unchanged.

Let's start with the type definition. We'll have to make one change compared to the spin lock: instead of an `AtomicBool` set to `false` or `true`, we'll use an `AtomicU32` set to zero or one, so we can use it with the atomic wait and wake functions.

```
pub struct Mutex<T> {
    /// 0: unlocked
    /// 1: locked
    state: AtomicU32,
    value: UnsafeCell<T>,
}
```

Just like for the spin lock, we need to promise that a `Mutex<T>` can be shared between threads, even though it contains a scary `UnsafeCell`:

```
unsafe impl<T> Sync for Mutex<T> where T: Send {}
```

We'll also add a `MutexGuard` type that implements the `Deref` traits to provide a fully safe locking interface, like we did in "A Safe Interface Using a Lock Guard" in Chapter 4:

```
pub struct MutexGuard<'a, T> {
    mutex: &'a Mutex<T>,
}

unsafe impl<T> Send for MutexGuard<'_, T> where T: Send {}
unsafe impl<T> Sync for MutexGuard<'_, T> where T: Sync {}

impl<T> Deref for MutexGuard<'_, T> {
    type Target = T;
    fn deref(&self) -> &T {
        unsafe { &*self.mutex.value.get() }
    }
}

impl<T> DerefMut for MutexGuard<'_, T> {
    fn deref_mut(&mut self) -> &mut T {
        unsafe { &mut *self.mutex.value.get() }
    }
}
```

> **Note**

For the design and operation of a lock guard type, see "A Safe Interface Using a Lock Guard" in Chapter 4.

Let's also get the `Mutex::new` function out of the way before we move on to the interesting part.

```
impl<T> Mutex<T> {
    pub const fn new(value: T) -> Self {
        Self {
            state: AtomicU32::new(0), // unlocked state
            value: UnsafeCell::new(value),
        }
    }

    …
}
```

Now that we have all that out of the way, there are two remaining pieces left: locking (`Mutex::lock()`) and unlocking (`Drop for MutexGuard<T>`).

The lock function we implemented for our spin lock uses an atomic swap operation to attempt to obtain the lock, returning if it successfully changed the state from "unlocked" to "locked." If unsuccessful, it immediately tries again.

To lock our mutex we'll do almost the same, except we use `wait()` to wait before trying again:

```
    pub fn lock(&self) -> MutexGuard<T> {
        // Set the state to 1: locked.
        while self.state.swap(1, Acquire) == 1 {
            // If it was already locked..
            // .. wait, unless the state is no longer 1.
            wait(&self.state, 1);
        }
        MutexGuard { mutex: self }
    }
```

> **Note**

For the memory ordering, the same reasoning applies as with our spin lock. Refer back to Chapter 4 for the details.

Note how the `wait()` function will only block if the state is still set to 1 (locked) at the time we call it, such that we don't have to worry about the possibility of missing a wake-up call between the swap and the wait calls.

The `Drop` implementation of the guard type is responsible for unlocking the mutex. Unlocking our spin lock was easy: just set the state back to `false` (unlocked). For our mutex, however, that won't suffice. If there's a thread waiting to lock the mutex, it won't know that the mutex has been unlocked unless we notify it using a wake operation. If we don't wake it up, it will most likely stay asleep forever. (Maybe it is lucky and is spuriously woken up at the right time, but let's not count on that.)

So, we'll not only set the state back to 0 (unlocked), but also call `wake_one()` right afterwards:

```
impl<T> Drop for MutexGuard<'_, T> {
    fn drop(&mut self) {
        // Set the state back to 0: unlocked.
        self.mutex.state.store(0, Release);
        // Wake up one of the waiting threads, if any.
        wake_one(&self.mutex.state);
    }
}
```

Waking one thread is enough, because even if there are multiple threads waiting, only one of them will be able to claim the lock. The next thread to lock it will wake up another thread when it's done with the lock, and so on. Waking up more than one thread at once will just set those threads up for disappointment, wasting valuable processor time when all but one of them realize their chance at locking has been snatched away by another lucky thread, before they go back to sleep again.

Note that there is no guarantee that the one thread that we wake up will be able to grab the lock. Another thread might still grab the lock right before it gets the chance.

An important observation to make here is how this mutex implementation would still be technically correct (that is, memory safe) without the wait and wake functions. Because the `wait()` operation can spuriously wake up, we can't make any assumptions about when it returns. We still have to

manage the state of our locking primitives ourselves. If we were to remove the wait and wake function calls, our mutex would be basically identical to our spin lock.

In general, the atomic wait and wake functions never play a factor in correctness, from a memory safety perspective. They are only a (very serious) optimization to avoid busy-looping. This doesn't mean that an unusably inefficient lock would be considered "correct" by any practical standards, but this insight can be helpful when trying to reason about `unsafe` Rust code.

Lock API

If you're planning to take on implementing Rust locks as a new hobby, you might quickly get bored with the boilerplate code involved in providing a safe interface. That is, the `UnsafeCell`, the `Sync` implementation, the guard type, the `Deref` implementations, and so on.

The `lock_api` crate on crates.io can be used to automatically take care of all these things. You'll only have to make a type that represents the lock state, and provide (unsafe) lock and unlock functions through the (unsafe) `lock_api::RawMutex` trait. In return, the `lock_api::Mutex` type will provide you with a fully safe and ergonomic mutex type, including a mutex guard, based on your lock implementation.

Avoiding Syscalls

By far the slowest part of our mutex are the wait and wake, since those (can) result in a *syscall*, a call into the operating system's kernel. Talking with the kernel like that is quite an involved process that tends to be quite slow, especially compared to atomic operations. So, for a performant mutex implementation, we should try to avoid wait and wake calls as much as possible.

Luckily, we're already halfway there. Because the `while` loop in our locking function checks the state before the `wait()` call, the wait operation is skipped entirely in the situation where we don't need to wait, when the mutex wasn't locked. We do, however, unconditionally call the `wake_one()` function when unlocking.

We can skip the `wake_one()` if we know there are no other threads waiting. To know whether there are waiting threads, we need to keep track of this information ourselves.

We can do this by splitting the "locked" state into two separate states: "locked without waiters" and "locked with waiter(s)." We'll use the values 1 and 2 for that, and update our documentation comment of the `state` field in the struct definition:

```
pub struct Mutex<T> {
    /// 0: unlocked
    /// 1: locked, no other threads waiting
    /// 2: locked, other threads waiting
    state: AtomicU32,
    value: UnsafeCell<T>,
}
```

Now, for an unlocked mutex, our lock function still needs to set the state to 1 to lock it. However, if it was already locked, our lock function now needs to set the state to 2 before going to sleep, so that the unlock function can tell there's a waiting thread.

To do this, we'll first use a compare-and-exchange function to attempt to change the state from 0 to 1. If that succeeds, we've locked the mutex, and we know there are no other waiters, since the mutex wasn't locked before. If it fails, that must be because the mutex is currently locked (in state 1 or 2). In that case, we'll use an atomic swap operation to set it to 2. If that swap operation returns an old value of 1 or 2, that means the mutex was indeed still locked, and only then do we use `wait()` to block until it changes. If the swap operation returns 0, that means we've successfully locked the mutex by changing its state from 0 to 2.

```
    pub fn lock(&self) -> MutexGuard<T> {
        if self.state.compare_exchange(0, 1, Acquire, Relaxed).is_err() {
            while self.state.swap(2, Acquire) != 0 {
                wait(&self.state, 2);
            }
        }
        MutexGuard { mutex: self }
    }
```

Now, our unlock function can make use of the new information by skipping the `wake_one()` call when it's unnecessary. Instead of just storing a 0 to unlock the mutex, we'll now use a swap operation so we can check its previous value. Only if that value was 2, will we continue to wake up a thread:

```
impl<T> Drop for MutexGuard<'_, T> {
    fn drop(&mut self) {
        if self.mutex.state.swap(0, Release) == 2 {
            wake_one(&self.mutex.state);
        }
    }
}
```

Note that after setting the state back to zero, it no longer indicates whether there are any waiting threads. The thread that's woken up is responsible for setting the state back to 2, to make sure any other waiting threads are not forgotten. This is why the compare-and-exchange operation is not part of the `while` loop in our lock function.

This does mean that for every time a thread had to `wait()` while locking, it will also call `wake_one()` when unlocking, even when that's not necessary. However, what's most important is that in the *uncontended case*, the ideal situation where threads are not attempting to acquire the lock simultaneously, both the `wait()` and `wake_one()` calls are entirely avoided.

Figure 9-1 visualizes the operations and happens-before relationships in a situation where two threads concurrently attempt to lock our `Mutex`. The first thread locks the mutex by changing the state from 0 to 1. At that point, the second thread will not be able to acquire the lock and therefore goes to sleep after changing the state from 1 to 2. When the first thread unlocks the `Mutex`, it swaps the state back to 0. Because it was 2, indicating a waiting thread, it calls `wake_one()` to

wake up the second thread. Note how we do not depend on any happens-before relationship between the wake and wait operations. While it's likely that the wake operation is the one responsible for waking up the waiting thread, the happens-before relationship is established through the acquire swap operation observing the value stored by the release swap operation.

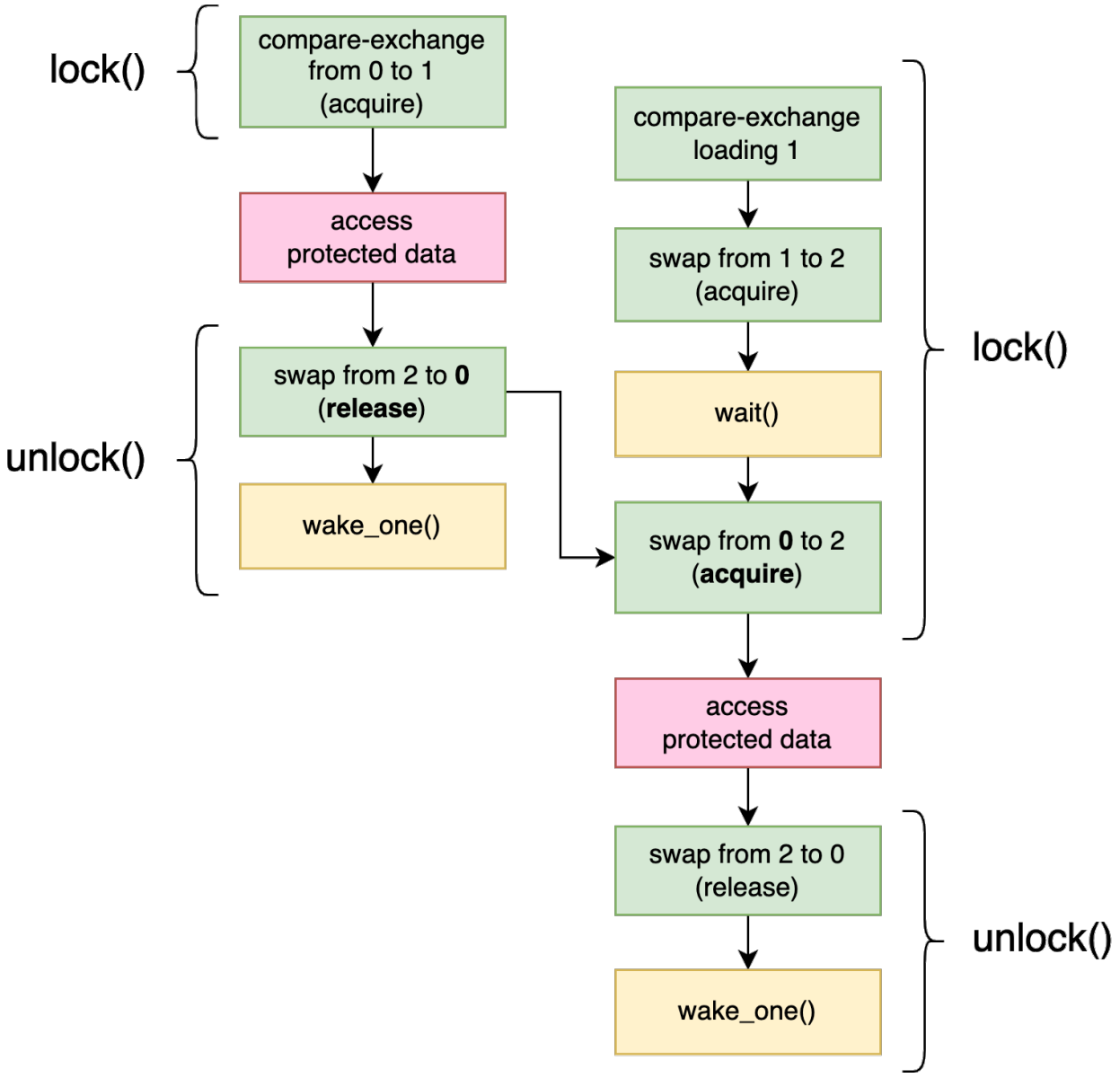


Figure 1: Figure 9-1. The happens-before relationships between two threads concurrently attempting to lock our Mutex.

Optimizing Further

At this point, it might seem like there's not much else we could optimize further. In the uncontended case, we perform zero syscalls, and all that's left are just two very simple atomic operations.

The only way to avoid the wait and wake operations is to go back to our spin lock implementation.

While spinning is usually very inefficient, it at least does avoid the potential overhead of a syscall. The only situation where spinning can be more efficient is when waiting for only a very short time.

For locking a mutex, that happens only in situations where the thread that currently holds the lock is running in parallel on a different processor core and will keep the lock only very briefly. This is, however, a very common case.

We can try to combine the best of both approaches by spinning for a very short amount of time before resorting to calling `wait()`. That way, if the lock is released very quickly, we don't need to call `wait()` at all, but we still avoid consuming an unreasonable amount of processor time that other threads could make better use of.

Implementing this only requires changes to our lock function.

To keep things as performant as possible in the uncontended case, we'll keep the original compare-and-exchange operation at the start of the lock function. We'll leave spinning waiting to a separate function.

```
impl<T> Mutex<T> {
    …

    pub fn lock(&self) -> MutexGuard<T> {
        if self.state.compare_exchange(0, 1, Acquire, Relaxed).is_err() {
            // The lock was already locked. :(
            lock_contended(&self.state);
        }
        MutexGuard { mutex: self }
    }
}

fn lock_contended(state: &AtomicU32) {
    …
}
```

In `lock_contended`, we could simply repeat the same compare-and-exchange operation a few hundred times before continuing to the wait loop. However, a compare-and-exchange operation generally attempts to get exclusive access to the relevant cache line (see "The MESI protocol" in Chapter 7), which can be more expensive than a simple load operation when executed repeatedly.

With that in mind, we come to the following `lock_contended` implementation:

```
fn lock_contended(state: &AtomicU32) {
    let mut spin_count = 0;

    while state.load(Relaxed) == 1 && spin_count < 100 {
        spin_count += 1;
        std::hint::spin_loop();
    }

    if state.compare_exchange(0, 1, Acquire, Relaxed).is_ok() {
```

```
            return;
        }

        while state.swap(2, Acquire) != 0 {
            wait(state, 2);
        }
    }
```

First, we spin up to 100 times, making use of a *spin loop hint* like we did in Chapter 4. We only spin as long as the mutex is locked and has no waiters. If another thread is already waiting, it means it gave up spinning because it took too long, which can be an indication that spinning will likely not be very useful for this thread either.

> **Note**

The spin duration of a hundred cycles is chosen mostly arbitrarily. The time an iteration takes and the duration of a syscall (which we're trying to avoid) depend heavily on the platform. Extensive benchmarking can help with choosing the right number, but unfortunately there's not a single correct answer.

The Linux implementation of `std::sync::Mutex` in the Rust standard library, at least the one in Rust 1.66.0, uses a spin count of 100.

After the lock state has changed, we try once more to lock it by setting it to 1, before we give up and start waiting. As we discussed before, after we call `wait()` we can no longer lock the mutex by setting its state to 1, since that might result in other waiters being forgotten.

Cold and Inline Attributes

You could add the `#[cold]` attribute to the `lock_contended` function definition to help the compiler understand that this function is not called in the common (uncontended) case, which can help with optimizations for the `lock` method.

Additionally, you could add the `#[inline]` attribute to the `Mutex` and `MutexGuard` methods to inform the compiler that it might be a good idea to *inline* them: to put the resulting instructions directly at the place where the method is called. Whether that increases performance is hard to say in general, but for very small functions like these, it usually does.

Benchmarking

Testing the performance of a mutex implementation is hard. It's easy to write a benchmark test and get some numbers, but it's very hard to get any meaningful numbers.

Optimizing a mutex implementation to perform very well in a specific benchmark test is relatively easy, but not very useful. After all, the point is to make something that performs well in real-world programs, not just in test programs.

We'll attempt to write two simple benchmark tests showing that our optimizations at least had some positive effect on some use cases, but keep in mind that any conclusions won't necessarily hold up in different scenarios.

For our first test, we'll create a `Mutex` and lock and unlock it a few million times, all on the same thread, measuring the total time it takes. This is a test for the trivial uncontended scenario, where there are never any threads that need to be woken up. Hopefully, this will show us a significant difference between our 2-state and 3-state versions.

```
fn main() {
    let m = Mutex::new(0);
    std::hint::black_box(&m);
    let start = Instant::now();
    for _ in 0..5_000_000 {
        *m.lock() += 1;
    }
    let duration = start.elapsed();
    println!("locked {} times in {:?}", *m.lock(), duration);
}
```

> **Note**

We use `std::hint::black_box` (like we did in "Impact on Performance" in Chapter 7) to force the compiler to assume there might be more code that accesses the mutex, preventing it from optimizing away the loop or locking operations.

Results will vary heavily depending on hardware and operating system. Trying this on one particular Linux computer with a recent AMD processor results in a total time of about 400 milliseconds for our unoptimized 2-state mutex, and about 40 milliseconds for our more optimized 3-state mutex. A factor ten improvement! On another Linux computer with an older Intel processor, the difference is even bigger: about 1800 milliseconds versus 60 milliseconds. This confirms that the addition of the third state can indeed be a very significant optimization.

Running this on a computer that runs macOS, however, produces completely different results: about 50 milliseconds for both versions, showing that it's all highly platform-dependent.

As it turns out, the implementation of libc++'s `std::atomic<T>::wake()`, which we use on macOS, already performs its own bookkeeping, independent from the kernel, to avoid unnecessary syscalls. The same holds for `WakeByAddressSingle()` on Windows.

Avoiding a call to those functions can still result in slightly better performance, since their implementation is far from trivial, especially because they can't store any information in the atomic variable itself. However, if we'd only be targeting only these operating systems, we should question whether adding a third state to our mutex was really worth the effort.

To see if our spinning optimization made any positive difference, we need a different benchmark test: one with lots of contention, with multiple threads repeatedly trying to lock an already locked mutex.

Let's try a scenario where four threads all concurrently attempt to lock and unlock the mutex a few million times:

```
fn main() {
    let m = Mutex::new(0);
    std::hint::black_box(&m);
    let start = Instant::now();
    thread::scope(|s| {
        for _ in 0..4 {
            s.spawn(|| {
                for _ in 0..5_000_000 {
                    *m.lock() += 1;
                }
            });
        }
    });
    let duration = start.elapsed();
    println!("locked {} times in {:?}", *m.lock(), duration);
}
```

Note that this is an extreme and unrealistic scenario. The mutex is only kept for an extremely short time (only to increment an integer), and the threads will immediately attempt to lock the mutex again after unlocking. A different scenario will most likely result in very different results.

Let's run this benchmark on the same two Linux computers as before. On the one with the older Intel processor, this results in about 900 milliseconds for the version of our mutex that doesn't spin, and about 750 milliseconds when using the spinning version. An improvement! On the computer with the newer AMD processor, however, we get opposite results: about 650 milliseconds without spinning and about 800 milliseconds with.

In conclusion, the answer as to whether spinning actually increases performance is, unfortunately, "it depends," even when looking at just one scenario.

Condition Variable

Let's move on to something more fun: implementing a condition variable.

As we saw in "Condition Variables" in Chapter 1, a condition variable is used together with a mutex to wait until the mutex-protected data matches some condition. It has a wait method that unlocks a mutex, waits for a signal, and locks the same mutex again. Signals are sent by other threads, usually right after modifying the mutex-protected data, to either one waiting thread (often called "notify one" or "signal") or all waiting threads (often called "notify all" or "broadcast").

While a condition variable attempts to keep a waiting thread asleep until it is signalled, it is possible for a waiting thread to be woken up spuriously, without a corresponding signal. The condition variable's wait operation will still relock the mutex before returning, though.

Notice how this interface is nearly identical to our futex-like `wait()`, `wake_one()`, and `wake_all()` functions. The main difference is the mechanism used to prevent lost signals. A condition variable

will start "listening" to signals before unlocking the mutex to not miss any signals right after, while our futex-style `wait()` function relies on a check of the state of the atomic variable to make sure waiting is still a good idea.

This leads to the following minimal implementation idea for a condition variable: if we make sure that every notification changes an atomic variable (like a counter), then all our `Condvar::wait()` method needs to do is check the value of that variable before unlocking the mutex, and pass it to the futex-style `wait()` function after unlocking it. That way, it will not go to sleep if any notification signal arrived since unlocking the mutex.

Let's try that out!

We start with a `Condvar` struct that just contains a single `AtomicU32`, which we initialize at zero:

```
pub struct Condvar {
    counter: AtomicU32,
}

impl Condvar {
    pub const fn new() -> Self {
        Self { counter: AtomicU32::new(0) }
    }

    …
}
```

The notify methods are simple. They just need to change the counter and use the corresponding wake operation to notify any waiting thread(s):

```
    pub fn notify_one(&self) {
        self.counter.fetch_add(1, Relaxed);
        wake_one(&self.counter);
    }

    pub fn notify_all(&self) {
        self.counter.fetch_add(1, Relaxed);
        wake_all(&self.counter);
    }
```

(We'll discuss the memory ordering in a moment.)

The wait method will take a `MutexGuard`, since that represents proof of a locked mutex. It will also return a `MutexGuard`, since it'll make sure the mutex is locked again before returning.

As we sketched out above, the method will first check the current value of the counter before unlocking the mutex. After unlocking the mutex, it should only wait if the counter hasn't changed, to make sure we didn't miss any signals. Here's what that looks like as code:

```
    pub fn wait<'a, T>(&self, guard: MutexGuard<'a, T>) -> MutexGuard<'a, T> {
        let counter_value = self.counter.load(Relaxed);

        // Unlock the mutex by dropping the guard,
        // but remember the mutex so we can lock it again later.
        let mutex = guard.mutex;
        drop(guard);

        // Wait, but only if the counter hasn't changed since unlocking.
        wait(&self.counter, counter_value);

        mutex.lock()
    }
```

> **Note**

This makes use of the private `mutex` field of the `MutexGuard`. Privacy in Rust is based on modules, so if you're defining this in a different module than the `MutexGuard`, you'll need to mark the `mutex` field of the `MutexGuard` as, for example, `pub(crate)` to make it available to other modules in the crate.

Before we celebrate our success in finishing our condition variable, let's think for a second about memory ordering.

While the mutex is locked, no other thread can change the mutex-protected data. Therefore, we don't need to worry about notifications from before we unlock the mutex, since, as long as we hold the mutex locked, nothing can happen to the data that would make us change our mind about wanting to go to sleep and wait.

The only situation we're interested in is when, after we release the mutex, another thread comes along and locks the mutex, changes the protected data, and signals us (hopefully after unlocking the mutex).

In this situation, there's a happens-before relationship between unlocking the mutex in `Condvar::wait()` and locking the mutex in the notifying thread. This happens-before relationship is what guarantees that our relaxed load, which happens before unlocking, will observe the value *before* the notification's relaxed increment operation, which happens after locking.

We don't know whether the `wait()` operation will see the value before or after incrementing, since there's nothing that guarantees any ordering at that point. However, that doesn't matter, since `wait()` behaves atomically with respect to corresponding wake operations. Either it sees the new value, in which case it does not go to sleep at all, or it sees the old value, in which case it goes to sleep and will be woken up by the corresponding `wake_one()` or `wake_all()` call from the notification.

Figure 9-2 shows the operations and happens-before relationships for a situation in which one thread uses `Condvar::wait()` to wait for some mutex-protected data to change and gets woken up by a second thread that modifies the data and calls `Condvar::notify_one()`. Note how the first load operation is guaranteed to observe the value before it gets incremented, thanks to the unlock and lock operations.

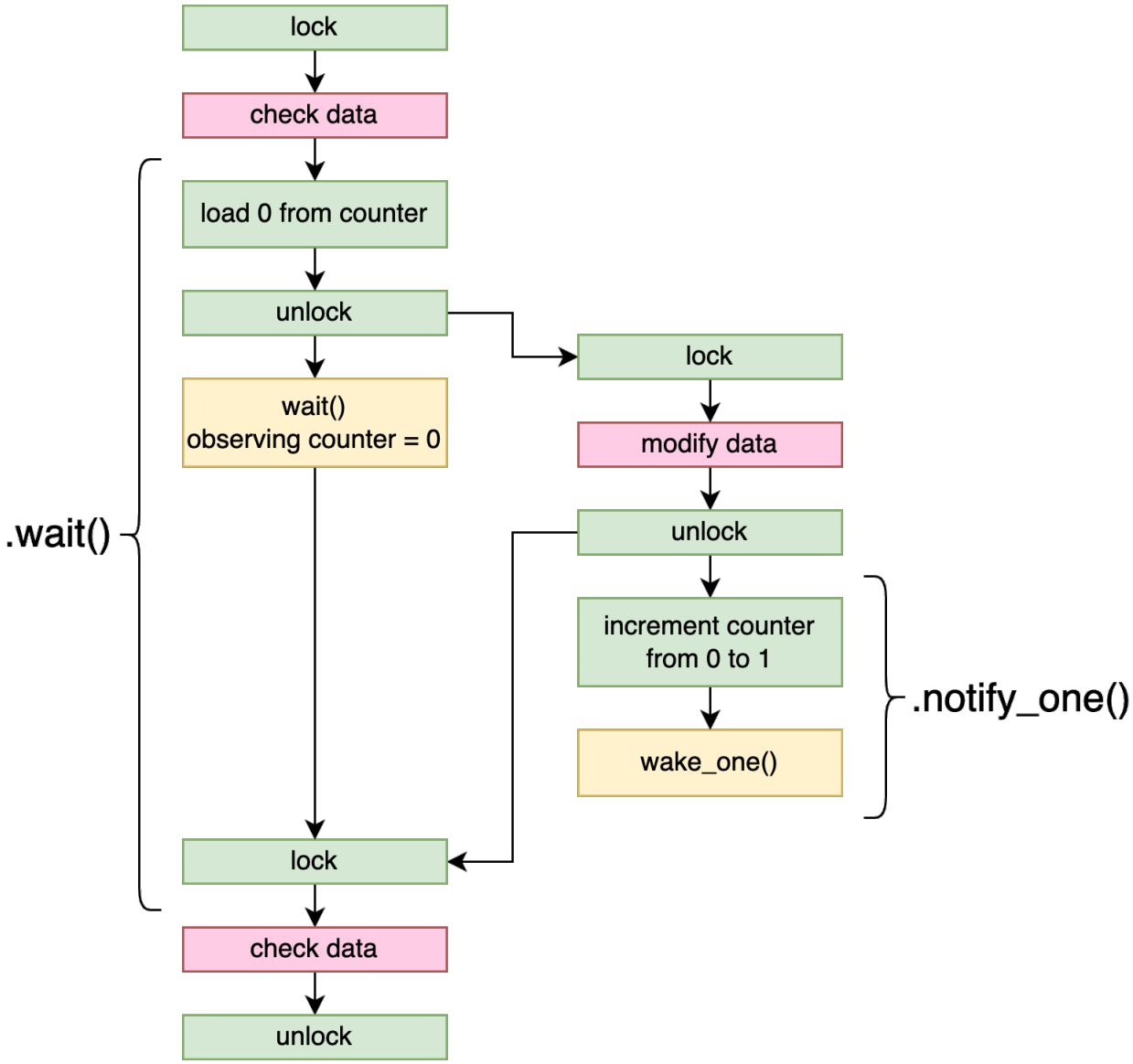


Figure 2: Figure 9-2. The operations and happens-before relationships of one thead using Condvar::wait() that

We should also consider what happens if the counter overflows.

The actual value of the counter doesn't matter as long as it is different after each notification. Unfortunately, after a bit more than four billion notifications, the counter will overflow and restart at zero, going back to values that have been used before. Technically, it is possible for our `Condvar::wait()` implementation to go to sleep when it shouldn't: if it misses exactly 4,294,967,296 notifications (or any multiple of that), it will overflow the counter all the way around to the value it had before.

It's entirely reasonable to consider the chance of that happening to be negligible. In contrast to what we did in our mutex locking method, we don't recheck the state and repeat the `wait()` call after waking up here, so we only need to worry about an overflow round-trip happening in the moment between the relaxed load of the counter and the `wait()` call. If a thread can be interrupted for so long that it allows for (exactly) that many notifications to happen, something has probably already gone terribly wrong, and the program has already turned unresponsive. At that point, one might reasonably argue, a microscopic additional risk of a thread staying asleep no longer matters.

> **Note**

On platforms that support futex-style waiting with a time limit, the risk of overflowing can be mitigated by using a timeout for the wait operation of a few seconds. Sending four billion notifications will take significantly longer, at which point the risk of a few additional seconds will have very little impact. This completely removes any risk of the program locking up due to a waiting thread wrongly staying asleep forever.

Let's see if it works!

```
#[test]
fn test_condvar() {
    let mutex = Mutex::new(0);
    let condvar = Condvar::new();

    let mut wakeups = 0;

    thread::scope(|s| {
        s.spawn(|| {
            thread::sleep(Duration::from_secs(1));
            *mutex.lock() = 123;
            condvar.notify_one();
        });

        let mut m = mutex.lock();
        while *m < 100 {
            m = condvar.wait(m);
            wakeups += 1;
        }

        assert_eq!(*m, 123);
```

```
    });

    // Check that the main thread actually did wait (not busy-loop),
    // while still allowing for a few spurious wake ups.
    assert!(wakeups < 10);
}
```

We count the number of times the condition variable returns from its wait method, to make sure it actually goes to sleep. If that number would be very high, it would indicate that we are accidentally spin-looping instead. It's important to test this, since a condition variable that never sleeps still results in "correct" behavior, but would effectively turn the waiting loop into a spin loop.

If we run this test, we see that it compiles and passes just fine, confirming that our condition variable did actually put the main thread to sleep. Of course, this doesn't prove that its implementation is correct. A long stress test that involves many threads, ideally run on a computer with a weakly ordered processor architecture, can be used to gain more confidence if necessary.

Avoiding Syscalls

As we realized in "Mutex: Avoiding Syscalls", optimizing a locking primitive is mainly about avoiding unnecessary wait and wake operations.

In the case of our condition variable, there is not much use in trying to avoid the `wait()` call in our `Condvar::wait()` implementation. By the time a thread decides to wait on a condition variable, it has already checked that the thing it's waiting for hasn't happened yet, and it needs to go to sleep. If the wait wasn't necessary, it wouldn't have called `Condvar::wait()` at all.

We can, however, avoid the `wake_one()` and `wake_all()` calls if there are no waiting threads, similar to what we did for our `Mutex`.

A simple way to do this is to keep track of the number of waiting threads. Our wait method will need to increment it before waiting, and decrement it when it's done. Then our notify methods can skip sending their signal if that number is zero.

So, we add a new field to our `Condvar` struct to track the number of active waiters:

```
pub struct Condvar {
    counter: AtomicU32,
    num_waiters: AtomicUsize, // New!
}

impl Condvar {
    pub const fn new() -> Self {
        Self {
            counter: AtomicU32::new(0),
            num_waiters: AtomicUsize::new(0), // New!
        }
    }
```

```
    …
}
```

By using an `AtomicUsize` for `num_waiters`, we don't have to worry about it overflowing. A `usize` is big enough to count every byte in memory, so if we assume that every active thread takes up at least a single byte of memory, it's definitely big enough to count any number of concurrently existing threads.

Next, we update our notification functions to not do anything if there are no waiters:

```
pub fn notify_one(&self) {
    if self.num_waiters.load(Relaxed) > 0 { // New!
        self.counter.fetch_add(1, Relaxed);
        wake_one(&self.counter);
    }
}

pub fn notify_all(&self) {
    if self.num_waiters.load(Relaxed) > 0 { // New!
        self.counter.fetch_add(1, Relaxed);
        wake_all(&self.counter);
    }
}
```

(We'll discuss the memory ordering in a moment.)

And finally, most importantly, we increment it at the start of our `wait` method and decrement as soon as it wakes up:

```
pub fn wait<'a, T>(&self, guard: MutexGuard<'a, T>) -> MutexGuard<'a, T> {
    self.num_waiters.fetch_add(1, Relaxed); // New!

    let counter_value = self.counter.load(Relaxed);

    let mutex = guard.mutex;
    drop(guard);

    wait(&self.counter, counter_value);

    self.num_waiters.fetch_sub(1, Relaxed); // New!

    mutex.lock()
}
```

We should again ask ourselves carefully if relaxed memory ordering is enough for all these atomic operations.

A new potential risk we've introduced is one of the notify methods observing a zero in `num_waiters`, skipping its wake operation, while there was actually a thread to wake up. This can happen when

a notify method observes the value either before the increment operation or after the decrement operation.

Just as with the relaxed load from the counter, the fact that the waiter still holds the mutex locked while incrementing `num_waiters` makes sure that any load of `num_waiters` that happens after unlocking the mutex will not see a value from before it was incremented.

We also don't need to worry about the notifying thread observing the decremented value "too soon," because once the decrementing operation is executed, perhaps after a spurious wake-up, the waiting thread no longer needs to be woken up anyway.

In other words, the happens-before relationship that the mutex establishes still provides all the guarantees we need.

Avoiding Spurious Wake-ups

Another way in which we could optimize our condition variable is by avoiding spuriously waking up. Every time a thread is woken up, it'll try to lock the mutex, potentially competing with other threads, which can have a big impact on performance.

It's quite uncommon for the underlying `wait()` operation to spuriously wake up, but our condition variable implementation easily allows for `notify_one()` to cause more than one thread to stop waiting. If a thread is in the process of going to sleep, has just loaded the counter value, but hasn't gone to sleep yet, a call to `notify_one()` will prevent that thread from going to sleep due to the updated counter, but it'll also cause a second thread to wake up because of the `wake_one()` operation that follows. Both of those threads will then compete to lock the mutex, wasting valuable processor time.

This might sound like a rarely occurring situation, but this can actually happen quite easily, due to how the mutex ends up synchronizing the threads. A thread that will call `notify_one()` on the condition variable will most likely lock and unlock the mutex right before, to change something about the data that the waiting thread is waiting for. This means that as soon as the `Condvar::wait()` method unlocks the mutex, that might immediately unblock a notifying thread that was waiting for the mutex. At that point the two threads are racing: the waiting thread to go to sleep, and the notifying thread to lock and unlock the mutex and notify the condition variable. If the notifying thread wins that race, the waiting thread will not go to sleep because of the incremented counter, but the notifying thread will still call `wake_one()`. This is exactly the problematic situation described above, where it might unnecessarily wake up an extra waiting thread.

A relatively straightforward solution would be to keep track of the number of threads that are allowed to wake up (that is, return from `Condvar::wait()`). The `notify_one` method would increase it by one, and the `wait` method would attempt to decrease it by one if it's not zero. If the counter is at zero, it could go (back) to sleep, instead of attempting to relock the mutex and returning. (Notifying all threads could be done by adding another counter specifically for `notify_all` that is never decremented.)

This approach works, but comes with a new and more subtle issue: a notification might wake up a thread that hasn't even called `Condvar::wait()` yet, including itself. A call to `Condvar::notify_one()` would increment the number of threads that should be woken up and use `wake_one()` to wake up one waiting thread. Then, if another (or even the same) thread calls `Condvar::wait()` afterwards, before the thread that was already waiting has a chance to

wake up, the newly waiting thread could see that there's one notification pending and claim it by decrementing the counter to zero, returning immediately. The first thread that was waiting would then go back to sleep, since another thread already took the notification.

Depending on the use case, this might be perfectly fine, or it might be a big problem, causing some threads to never make progress.

> **Note**

GNU libc's `pthread_cond_t` implementation used to suffer from this issue. After much discussion on whether or not this was allowed by the POSIX specification, the issue was eventually resolved with the release of GNU libc 2.25 in 2017, which included a completely new condition variable implementation.

In many situations where a condition variable is used, it's perfectly fine for a waiter to snatch away an earlier notification. However, when implementing a condition variable for general use rather than a specific kind of use case, this behavior might be unacceptable.

Again, we must come to the conclusion that the answer to whether we should use an optimized approach is, unsurprisingly, "it depends."

> **Note**

There are ways to avoid this problem while still avoiding spurious wake-ups, but those are significantly more complicated than other approaches.

The solution used by GNU libc's new condition variable involves categorizing waiters into two groups, only allowing the first group to consume notifications and swapping the groups around when the first one has no waiters left.

A downside of this approach is not only the complexity of the algorithm, but also that it significantly increases the size of the condition variable type, since it now needs to keep track of much more information.

Thundering Herd Problem

Another performance problem that one might encounter when using a condition variable occurs when using `notify_all()` to wake up many threads waiting for the same thing.

The problem is that, after waking up, all those threads will immediately try to lock the same mutex. Most likely, only one thread will succeed, and all the others will have to go back to sleep. This resource-wasting problem of many threads all rushing to claim the same resource is referred to as the *thundering herd problem*.

It's not unreasonable to argue that `Condvar::notify_all()` is fundamentally an anti-pattern not worth optimizing for. A condition variable's purpose is to unlock a mutex and relock it when notified, so perhaps notifying more than one thread at once will never lead to anything good.

Even so, if we want to optimize for this situation, we can do so on operating systems that support a futex-like *requeuing* operation, like `FUTEX_REQUEUE` on Linux. (See "Futex Operations" in Chapter 8.)

Instead of waking up many threads of which all but one will immediately go back to sleep once they realize the lock has already been taken, we can *requeue* all but one thread such that their futex wait operations no longer wait for the condition variable's counter, but start waiting for the mutex state instead.

Requeueing a waiting thread doesn't wake it up. In fact, the thread won't even know that it has been requeued. Unfortunately, this can lead to some very subtle pitfalls.

For example, remember how a three-state mutex always must be locked to the right state ("locked with waiters") after waking up, to make sure other waiters aren't forgotten about? This means we should no longer use the regular mutex lock method in our `Condvar::wait()` implementation, which might set the mutex to the wrong state.

A requeuing condition variable implementation would need to store a pointer to the mutex used by the waiting threads. Otherwise, the notify methods wouldn't know which atomic variable (the mutex state) to requeue the waiting threads to. This is why a condition variable generally does not allow two threads to wait for different mutexes. Even though many condition variable implementations do not make use of requeuing, it can be useful to keep open the possibility for a future version to do so.

Reader-Writer Lock

It's time to implement a reader-writer lock!

Recall how, unlike a mutex, a reader-writer lock supports two types of locking: read-locking and write-locking, sometimes called shared locking and exclusive locking. Write-locking behaves identically to locking a mutex, only allowing one lock at a time, while read-locking allows for multiple readers to hold a lock at once. In other words, it closely matches how exclusive references (`&mut T`) and shared references (`&T`) work in Rust, allowing only one exclusive reference, or any number of shared references, to be active at the same time.

For our mutex, we needed to track only whether it was locked or not. For our reader-writer lock, however, we also need to know how many (reader) locks are currently held, to make sure write-locking only happens after all readers have released their locks.

Let's start with a `RwLock` struct that uses a single `AtomicU32` as its state. We'll use it to represent the number of currently acquired read locks, such that a value of zero means it's unlocked. To represent the write-locked state, let's use a special value of `u32::MAX`.

```
pub struct RwLock<T> {
    /// The number of readers, or u32::MAX if write-locked.
    state: AtomicU32,
    value: UnsafeCell<T>,
}
```

For our `Mutex<T>`, we had to restrict its `Sync` implementation to types `T` that implement `Send`, to make sure it can't be used to send, for example, an `Rc` to another thread. For our new `RwLock<T>`, we additionally need to require that `T` also implements `Sync`, because multiple readers will be able to access the data at once:

```
unsafe impl<T> Sync for RwLock<T> where T: Send + Sync {}
```

Because our `RwLock` can be locked in two different ways, we'll have two separate lock functions, each with its own type of guard:

```
impl<T> RwLock<T> {
    pub const fn new(value: T) -> Self {
        Self {
            state: AtomicU32::new(0), // Unlocked.
            value: UnsafeCell::new(value),
        }
    }

    pub fn read(&self) -> ReadGuard<T> {
        …
    }

    pub fn write(&self) -> WriteGuard<T> {
        …
    }
}

pub struct ReadGuard<'a, T> {
    rwlock: &'a RwLock<T>,
}

pub struct WriteGuard<'a, T> {
    rwlock: &'a RwLock<T>,
}
```

The write guard should behave like an exclusive reference (`&mut T`), which we do by implementing both `Deref` and `DerefMut` for it:

```
impl<T> Deref for WriteGuard<'_, T> {
    type Target = T;
    fn deref(&self) -> &T {
        unsafe { &*self.rwlock.value.get() }
    }
}

impl<T> DerefMut for WriteGuard<'_, T> {
    fn deref_mut(&mut self) -> &mut T {
```

```
        unsafe { &mut *self.rwlock.value.get() }
    }
}
```

However, the read guard should only implement `Deref`, not `DerefMut`, because it doesn't have exclusive access to the data, making it behave like a shared reference (`&T`):

```
impl<T> Deref for ReadGuard<'_, T> {
    type Target = T;
    fn deref(&self) -> &T {
        unsafe { &*self.rwlock.value.get() }
    }
}
```

Now that we got all that boilerplate code out of the way, let's get to the interesting parts: locking and unlocking.

To read-lock our `RwLock`, we must increment the state by one, but only if it wasn't already write-locked. We'll use a compare-and-exchange loop ("Compare-and-Exchange Operations" in Chapter 2) to do so. In case the state is `u32::MAX`, meaning the `RwLock` is write-locked, we'll use a `wait()` operation to sleep and retry later.

```
    pub fn read(&self) -> ReadGuard<T> {
        let mut s = self.state.load(Relaxed);
        loop {
            if s < u32::MAX {
                assert!(s < u32::MAX - 1, "too many readers");
                match self.state.compare_exchange_weak(
                    s, s + 1, Acquire, Relaxed
                ) {
                    Ok(_) => return ReadGuard { rwlock: self },
                    Err(e) => s = e,
                }
            }
            if s == u32::MAX {
                wait(&self.state, u32::MAX);
                s = self.state.load(Relaxed);
            }
        }
    }
```

Write-locking is easier; we just need to change the state from zero to `u32::MAX`, or `wait()` if it was already locked:

```
    pub fn write(&self) -> WriteGuard<T> {
        while let Err(s) = self.state.compare_exchange(
            0, u32::MAX, Acquire, Relaxed
```

```
        ) {
            // Wait while already locked.
            wait(&self.state, s);
        }
        WriteGuard { rwlock: self }
    }
```

Note how the exact state value of a locked `RwLock` varies, but the `wait()` operation expects us to give it an exact value to compare the state with. This is why we use the return value from the compare-and-exchange operation for the `wait()` operation.

Unlocking a reader involves decrementing the state by one. The reader that ends up unlocking the `RwLock`, the one that changes the state from one to zero, is responsible for waking up a waiting writer, if any.

Waking up just one thread is enough, because we know there cannot be any waiting readers at this point. There would simply be no reason for a reader to be waiting on a read-locked `RwLock`.

```
impl<T> Drop for ReadGuard<'_, T> {
    fn drop(&mut self) {
        if self.rwlock.state.fetch_sub(1, Release) == 1 {
            // Wake up a waiting writer, if any.
            wake_one(&self.rwlock.state);
        }
    }
}
```

A writer must reset the state to zero to unlock, after which it should wake either one waiting writer or all waiting readers.

We don't know whether readers or writers are waiting, nor do we have a way to wake up only a writer or only the readers. So, we'll just wake all threads:

```
impl<T> Drop for WriteGuard<'_, T> {
    fn drop(&mut self) {
        self.rwlock.state.store(0, Release);
        // Wake up all waiting readers and writers.
        wake_all(&self.rwlock.state);
    }
}
```

And that's it! We've built a very simple but perfectly usable reader-writer lock.

Time to fix some issues.

Avoiding Busy-Looping Writers

One issue with our implementation is that write-locking might result in an accidental busy-loop.

If we have an `RwLock` with a lot of readers repeatedly locking and unlocking it, the lock state might be continuously in flux, rapidly going up and down. For our `write` method, this results in a high chance of the lock state changing between the compare-and-exchange operation and the subsequent `wait()` operation, especially if the `wait()` operation is directly implemented as a (relatively slow) syscall. This means that the `wait()` operation will often return immediately, even though the lock was never unlocked; it just had a different number of readers than expected.

A solution can be to use a different `AtomicU32` for the writers to wait on, and only change the value of that atomic when we actually want to wake up a writer.

Let's try that, by adding a new `writer_wake_counter` field to our `RwLock`:

```
pub struct RwLock<T> {
    /// The number of readers, or u32::MAX if write-locked.
    state: AtomicU32,
    /// Incremented to wake up writers.
    writer_wake_counter: AtomicU32, // New!
    value: UnsafeCell<T>,
}

impl<T> RwLock<T> {
    pub const fn new(value: T) -> Self {
        Self {
            state: AtomicU32::new(0),
            writer_wake_counter: AtomicU32::new(0), // New!
            value: UnsafeCell::new(value),
        }
    }

    …
}
```

The `read` method remains unchanged, but the `write` method now needs to wait for the new atomic variable instead. To make sure we don't miss any notifications between seeing that the `RwLock` is read-locked and actually going to sleep, we'll use a pattern similar to the one we used for implementing our condition variable: check the `writer_wake_counter` before checking if we still want to sleep:

```
    pub fn write(&self) -> WriteGuard<T> {
        while self.state.compare_exchange(
            0, u32::MAX, Acquire, Relaxed
        ).is_err() {
            let w = self.writer_wake_counter.load(Acquire);
            if self.state.load(Relaxed) != 0 {
                // Wait if the RwLock is still locked, but only if
                // there have been no wake signals since we checked.
                wait(&self.writer_wake_counter, w);
            }
```

```
        }
        WriteGuard { rwlock: self }
    }
```

The acquire-load operation of `writer_wake_counter` will form a happens-before relationship with a release-increment operation that's executed right after unlocking the state, before waking up a waiting writer:

```
impl<T> Drop for ReadGuard<'_, T> {
    fn drop(&mut self) {
        if self.rwlock.state.fetch_sub(1, Release) == 1 {
            self.rwlock.writer_wake_counter.fetch_add(1, Release); // New!
            wake_one(&self.rwlock.writer_wake_counter); // Changed!
        }
    }
}
```

The happens-before relationship makes sure that the `write` method cannot observe the incremented `writer_wake_counter` value while still seeing the not-yet-decremented `state` value afterwards. Otherwise, the write-locking thread might conclude the `RwLock` is still locked while having missed the wake-up call.

As before, write-unlocking should wake either one waiting writer or all waiting readers. Since we still don't know whether there are writers or readers waiting, we have to wake both one waiting writer (through `wake_one`) and all waiting readers (using `wake_all`):

```
impl<T> Drop for WriteGuard<'_, T> {
    fn drop(&mut self) {
        self.rwlock.state.store(0, Release);
        self.rwlock.writer_wake_counter.fetch_add(1, Release); // New!
        wake_one(&self.rwlock.writer_wake_counter); // New!
        wake_all(&self.rwlock.state);
    }
}
```

> **Note**

On some operating systems, the operation behind the wake operations returns the number of threads it woke up. It might indicate a lower number than the actual number of awoken threads (due to spuriously awoken threads), but its return value can still be useful as an optimization.

In the drop implementation above, for example, we could skip the `wake_all()` call if the `wake_one()` operation would indicate it actually woke up a thread.

Avoiding Writer Starvation

A common use case for an `RwLock` is a situation with many frequent readers, but very few, often only one, infrequent writer. For example, one thread might be responsible for reading out some sensor input or periodically downloading some new data that many other threads need to use.

In such a situation, we can quickly run into an issue called *writer starvation*: a situation where the writer(s) never get a chance to lock the `RwLock` because there are always readers around to keep the `RwLock` read-locked.

One solution to this problem is to prevent any new readers from acquiring a lock when there is a writer waiting, even when the `RwLock` is still read-locked. That way, all new readers will have to wait until the writer has had its turn, making sure that readers will get access to the latest data that the writer wanted to share.

Let's implement this.

To do this, we need to keep track of whether there are any waiting writers. To make space for this information in the state variable, we can multiply the reader count by 2, and add 1 for situations where there is a writer waiting. This means that a state of 6 or 7 both represent a situation with three active read locks: 6 without a waiting writer, and 7 with a waiting writer.

If we keep `u32::MAX`, which is an odd number, as the write-locked state, then readers will have to wait if the state is odd, but are free to acquire a read lock by incrementing it by two if the state is even.

```
pub struct RwLock<T> {
    /// The number of read locks times two, plus one if there's a writer waiting.
    /// u32::MAX if write locked.
    ///
    /// This means that readers may acquire the lock when
    /// the state is even, but need to block when odd.
    state: AtomicU32,
    /// Incremented to wake up writers.
    writer_wake_counter: AtomicU32,
    value: UnsafeCell<T>,
}
```

We'll have to change the two `if` statements in our `read` method to no longer compare the state against `u32::MAX`, but instead check whether the state is even or odd. We also need to make sure we lock by incrementing by two instead of one.

```
    pub fn read(&self) -> ReadGuard<T> {
        let mut s = self.state.load(Relaxed);
        loop {
            if s % 2 == 0 { // Even.
                assert!(s < u32::MAX - 2, "too many readers");
                match self.state.compare_exchange_weak(
                    s, s + 2, Acquire, Relaxed
                ) {
```

```
                    Ok(_) => return ReadGuard { rwlock: self },
                    Err(e) => s = e,
                }
            }
            if s % 2 == 1 { // Odd.
                wait(&self.state, s);
                s = self.state.load(Relaxed);
            }
        }
    }
```

Our `write` method has to undergo bigger changes. We'll use a compare-and-exchange loop, just like our `read` method above. If the state is 0 or 1, which means the `RwLock` is unlocked, we'll attempt to change the state to `u32::MAX` to write-lock it. Otherwise, we'll have to wait. Before doing so, however, we need to make sure the state is odd, to stop new readers from acquiring the lock. After making sure the state is odd, we wait for the `writer_wake_counter` variable, while making sure that the lock hasn't been unlocked in the meantime.

In code, that looks like this:

```
    pub fn write(&self) -> WriteGuard<T> {
        let mut s = self.state.load(Relaxed);
        loop {
            // Try to lock if unlocked.
            if s <= 1 {
                match self.state.compare_exchange(
                    s, u32::MAX, Acquire, Relaxed
                ) {
                    Ok(_) => return WriteGuard { rwlock: self },
                    Err(e) => { s = e; continue; }
                }
            }
            // Block new readers, by making sure the state is odd.
            if s % 2 == 0 {
                match self.state.compare_exchange(
                    s, s + 1, Relaxed, Relaxed
                ) {
                    Ok(_) => {}
                    Err(e) => { s = e; continue; }
                }
            }
            // Wait, if it's still locked
            let w = self.writer_wake_counter.load(Acquire);
            s = self.state.load(Relaxed);
            if s >= 2 {
                wait(&self.writer_wake_counter, w);
                s = self.state.load(Relaxed);
```

```
            }
        }
    }
```

Since we now track whether there are any waiting writers, read-unlocking can now skip the `wake_one()` call when unnecessary:

```
impl<T> Drop for ReadGuard<'_, T> {
    fn drop(&mut self) {
        // Decrement the state by 2 to remove one read-lock.
        if self.rwlock.state.fetch_sub(2, Release) == 3 {
            // If we decremented from 3 to 1, that means
            // the RwLock is now unlocked _and_ there is
            // a waiting writer, which we wake up.
            self.rwlock.writer_wake_counter.fetch_add(1, Release);
            wake_one(&self.rwlock.writer_wake_counter);
        }
    }
}
```

While write-locked (a state of `u32::MAX`), we do not track any information on whether any thread is waiting. So, we have no new information to use for write-unlocking, which will remain identical:

```
impl<T> Drop for WriteGuard<'_, T> {
    fn drop(&mut self) {
        self.rwlock.state.store(0, Release);
        self.rwlock.writer_wake_counter.fetch_add(1, Release);
        wake_one(&self.rwlock.writer_wake_counter);
        wake_all(&self.rwlock.state);
    }
}
```

For a reader-writer lock that's optimized for the "frequent reading and infrequent writing" use case, this would be quite acceptable, since write-locking (and therefore write-unlocking) happens infrequently.

For a more general purpose reader-writer lock, however, it is definitely worth optimizing further, to bring the performance of write-locking and -unlocking near the performance of an efficient 3-state mutex. This is left as a fun exercise for the reader.

Summary

- The `atomic-wait` crate provides basic futex-like functionality that works on (recent versions of) all major operating systems.
- A minimal mutex implementation only needs two states, like our `SpinLock` from Chapter 4.

- A more efficient mutex tracks whether there are any waiting threads, so it can avoid an unnecessary wake operation.
- Spinning before going to sleep might in some cases be beneficial, but it depends heavily on the situation, operating system, and hardware.
- A minimal condition variable only needs a notification counter, which `Condvar::wait` will have to check both before and after unlocking the mutex.
- A condition variable could track the number of waiting threads to avoid unnecessary wake operations.
- Avoiding spuriously waking up from `Condvar::wait` can be tricky, requiring extra bookkeeping.
- A minimal reader-writer lock requires only an atomic counter as its state.
- An additional atomic variable can be used to wake writers independently from readers.
- To avoid writer starvation, extra state is required to prioritize a waiting writer over new readers.

Complete Runnable Examples

The following complete, tested programs illustrate the primitives and patterns developed throughout this chapter.

Example: `mutex_1.rs` (Mutex 1)

```
use atomic_wait::{wait, wake_one};
use std::cell::UnsafeCell;
use std::ops::{Deref, DerefMut};
use std::sync::atomic::AtomicU32;
use std::sync::atomic::Ordering::{Acquire, Release};

pub struct Mutex<T> {
    /// 0: unlocked
    /// 1: locked
    state: AtomicU32,
    value: UnsafeCell<T>,
}

unsafe impl<T> Sync for Mutex<T> where T: Send {}

pub struct MutexGuard<'a, T> {
    mutex: &'a Mutex<T>,
```

```
}

unsafe impl<T> Sync for MutexGuard<'_, T> where T: Sync {}

impl<T> Deref for MutexGuard<'_, T> {
    type Target = T;
    fn deref(&self) -> &T {
        unsafe { &*self.mutex.value.get() }
    }
}

impl<T> DerefMut for MutexGuard<'_, T> {
    fn deref_mut(&mut self) -> &mut T {
        unsafe { &mut *self.mutex.value.get() }
    }
}

impl<T> Mutex<T> {
    pub const fn new(value: T) -> Self {
        Self {
            state: AtomicU32::new(0), // unlocked state
            value: UnsafeCell::new(value),
        }
    }

    pub fn lock(&self) -> MutexGuard<T> {
        // Set the state to 1: locked.
        while self.state.swap(1, Acquire) == 1 {
            // If it was already locked..
            // .. wait, unless the state is no longer 1.
            wait(&self.state, 1);
        }
        MutexGuard { mutex: self }
    }
}

impl<T> Drop for MutexGuard<'_, T> {
    fn drop(&mut self) {
        // Set the state back to 0: unlocked.
        self.mutex.state.store(0, Release);
        // Wake up one of the waiting threads, if any.
        wake_one(&self.mutex.state);
    }
}
```

Example: `mutex_2.rs` (Mutex 2)

```
use atomic_wait::{wait, wake_one};
use std::cell::UnsafeCell;
use std::ops::{Deref, DerefMut};
use std::sync::atomic::AtomicU32;
use std::sync::atomic::Ordering::{Acquire, Relaxed, Release};

pub struct Mutex<T> {
    /// 0: unlocked
    /// 1: locked, no other threads waiting
    /// 2: locked, other threads waiting
    state: AtomicU32,
    value: UnsafeCell<T>,
}

unsafe impl<T> Sync for Mutex<T> where T: Send {}

pub struct MutexGuard<'a, T> {
    mutex: &'a Mutex<T>,
}

unsafe impl<T> Sync for MutexGuard<'_, T> where T: Sync {}

impl<T> Deref for MutexGuard<'_, T> {
    type Target = T;
    fn deref(&self) -> &T {
        unsafe { &*self.mutex.value.get() }
    }
}

impl<T> DerefMut for MutexGuard<'_, T> {
    fn deref_mut(&mut self) -> &mut T {
        unsafe { &mut *self.mutex.value.get() }
    }
}

impl<T> Mutex<T> {
    pub const fn new(value: T) -> Self {
        Self {
            state: AtomicU32::new(0), // unlocked state
            value: UnsafeCell::new(value),
        }
    }

    pub fn lock(&self) -> MutexGuard<T> {
```

```
        if self.state.compare_exchange(0, 1, Acquire, Relaxed).is_err() {
            while self.state.swap(2, Acquire) != 0 {
                wait(&self.state, 2);
            }
        }
        MutexGuard { mutex: self }
    }
}

impl<T> Drop for MutexGuard<'_, T> {
    fn drop(&mut self) {
        if self.mutex.state.swap(0, Release) == 2 {
            wake_one(&self.mutex.state);
        }
    }
}
```

Example: `mutex_3.rs` (Mutex 3)

```
use atomic_wait::{wait, wake_one};
use std::cell::UnsafeCell;
use std::ops::{Deref, DerefMut};
use std::sync::atomic::AtomicU32;
use std::sync::atomic::Ordering::{Acquire, Relaxed, Release};

pub struct Mutex<T> {
    /// 0: unlocked
    /// 1: locked, no other threads waiting
    /// 2: locked, other threads waiting
    state: AtomicU32,
    value: UnsafeCell<T>,
}

unsafe impl<T> Sync for Mutex<T> where T: Send {}

pub struct MutexGuard<'a, T> {
    pub(crate) mutex: &'a Mutex<T>,
}

unsafe impl<T> Sync for MutexGuard<'_, T> where T: Sync {}

impl<T> Deref for MutexGuard<'_, T> {
    type Target = T;
    fn deref(&self) -> &T {
        unsafe { &*self.mutex.value.get() }
```

```
    }
}

impl<T> DerefMut for MutexGuard<'_, T> {
    fn deref_mut(&mut self) -> &mut T {
        unsafe { &mut *self.mutex.value.get() }
    }
}

impl<T> Mutex<T> {
    pub const fn new(value: T) -> Self {
        Self {
            state: AtomicU32::new(0), // unlocked state
            value: UnsafeCell::new(value),
        }
    }

    pub fn lock(&self) -> MutexGuard<T> {
        if self.state.compare_exchange(0, 1, Acquire, Relaxed).is_err() {
            // The lock was already locked. :(
            lock_contended(&self.state);
        }
        MutexGuard { mutex: self }
    }
}

fn lock_contended(state: &AtomicU32) {
    let mut spin_count = 0;

    while state.load(Relaxed) == 1 && spin_count < 100 {
        spin_count += 1;
        std::hint::spin_loop();
    }

    if state.compare_exchange(0, 1, Acquire, Relaxed).is_ok() {
        return;
    }

    while state.swap(2, Acquire) != 0 {
        wait(state, 2);
    }
}

impl<T> Drop for MutexGuard<'_, T> {
    fn drop(&mut self) {
        if self.mutex.state.swap(0, Release) == 2 {
            wake_one(&self.mutex.state);
```

```
        }
    }
}

// TODO (bench)
#[test]
fn main() {
    use std::time::Instant;
    let m = Mutex::new(0);
    std::hint::black_box(&m);
    let start = Instant::now();
    for _ in 0..5_000_000 {
        *m.lock() += 1;
    }
    let duration = start.elapsed();
    println!("locked {} times in {:?}", *m.lock(), duration);
}

// TODO (bench)
#[test]
fn main2() {
    use std::thread;
    use std::time::Instant;
    let m = Mutex::new(0);
    std::hint::black_box(&m);
    let start = Instant::now();
    thread::scope(|s| {
        for _ in 0..4 {
            s.spawn(|| {
                for _ in 0..5_000_000 {
                    *m.lock() += 1;
                }
            });
        }
    });
    let duration = start.elapsed();
    println!("locked {} times in {:?}", *m.lock(), duration);
}
```

Example: `condvar_1.rs` (Condvar 1)

```
use atomic_wait::{wait, wake_all, wake_one};
use std::sync::atomic::AtomicU32;
use std::sync::atomic::Ordering::Relaxed;
use super::mutex_3::MutexGuard;
```

```rust
pub struct Condvar {
    counter: AtomicU32,
}

impl Condvar {
    pub const fn new() -> Self {
        Self { counter: AtomicU32::new(0) }
    }

    pub fn notify_one(&self) {
        self.counter.fetch_add(1, Relaxed);
        wake_one(&self.counter);
    }

    pub fn notify_all(&self) {
        self.counter.fetch_add(1, Relaxed);
        wake_all(&self.counter);
    }

    pub fn wait<'a, T>(&self, guard: MutexGuard<'a, T>) -> MutexGuard<'a, T> {
        let counter_value = self.counter.load(Relaxed);

        // Unlock the mutex by dropping the guard,
        // but remember the mutex so we can lock it again later.
        let mutex = guard.mutex;
        drop(guard);

        // Wait, but only if the counter hasn't changed since unlocking.
        wait(&self.counter, counter_value);

        mutex.lock()
    }
}

#[test]
fn test_condvar() {
    use super::mutex_3::Mutex;
    use std::thread;
    use std::time::Duration;

    let mutex = Mutex::new(0);
    let condvar = Condvar::new();

    let mut wakeups = 0;

    thread::scope(|s| {
```

```
        s.spawn(|| {
            thread::sleep(Duration::from_secs(1));
            *mutex.lock() = 123;
            condvar.notify_one();
        });

        let mut m = mutex.lock();
        while *m < 100 {
            m = condvar.wait(m);
            wakeups += 1;
        }

        assert_eq!(*m, 123);
    });

    // Check that the main thread actually did wait (not busy-loop),
    // while still allowing for a few spurious wake ups.
    assert!(wakeups < 10);
}
```

Example: `condvar_2.rs` (Condvar 2)

```
use atomic_wait::{wait, wake_all, wake_one};
use std::sync::atomic::AtomicU32;
use std::sync::atomic::AtomicUsize;
use std::sync::atomic::Ordering::Relaxed;
use super::mutex_3::MutexGuard;

pub struct Condvar {
    counter: AtomicU32,
    num_waiters: AtomicUsize,
}

impl Condvar {
    pub const fn new() -> Self {
        Self {
            counter: AtomicU32::new(0),
            num_waiters: AtomicUsize::new(0),
        }
    }

    pub fn notify_one(&self) {
        if self.num_waiters.load(Relaxed) > 0 {
            self.counter.fetch_add(1, Relaxed);
            wake_one(&self.counter);
```

```
        }
    }

    pub fn notify_all(&self) {
        if self.num_waiters.load(Relaxed) > 0 {
            self.counter.fetch_add(1, Relaxed);
            wake_all(&self.counter);
        }
    }

    pub fn wait<'a, T>(&self, guard: MutexGuard<'a, T>) -> MutexGuard<'a, T> {
        self.num_waiters.fetch_add(1, Relaxed);

        let counter_value = self.counter.load(Relaxed);

        let mutex = guard.mutex;
        drop(guard);

        wait(&self.counter, counter_value);

        self.num_waiters.fetch_sub(1, Relaxed);

        mutex.lock()
    }
}

#[test]
fn test_condvar() {
    use super::mutex_3::Mutex;
    use std::thread;
    use std::time::Duration;

    let mutex = Mutex::new(0);
    let condvar = Condvar::new();

    let mut wakeups = 0;

    thread::scope(|s| {
        s.spawn(|| {
            thread::sleep(Duration::from_secs(1));
            *mutex.lock() = 123;
            condvar.notify_one();
        });

        let mut m = mutex.lock();
        while *m < 100 {
            m = condvar.wait(m);
```

```
            wakeups += 1;
        }

        assert_eq!(*m, 123);
    });

    // Check that the main thread actually did wait (not busy-loop),
    // while still allowing for a few spurious wake ups.
    assert!(wakeups < 10);
}
```

Example: `rwlock_1.rs` (Rwlock 1)

```
use atomic_wait::{wait, wake_all, wake_one};
use std::cell::UnsafeCell;
use std::ops::{Deref, DerefMut};
use std::sync::atomic::AtomicU32;
use std::sync::atomic::Ordering::{Acquire, Relaxed, Release};

pub struct RwLock<T> {
    /// The number of readers, or u32::MAX if write-locked.
    state: AtomicU32,
    value: UnsafeCell<T>,
}

unsafe impl<T> Sync for RwLock<T> where T: Send + Sync {}

impl<T> RwLock<T> {
    pub const fn new(value: T) -> Self {
        Self {
            state: AtomicU32::new(0), // Unlocked.
            value: UnsafeCell::new(value),
        }
    }

    pub fn read(&self) -> ReadGuard<T> {
        let mut s = self.state.load(Relaxed);
        loop {
            if s < u32::MAX {
                assert!(s < u32::MAX - 1, "too many readers");
                match self.state.compare_exchange_weak(
                    s, s + 1, Acquire, Relaxed
                ) {
                    Ok(_) => return ReadGuard { rwlock: self },
                    Err(e) => s = e,
```

```
                }
            }
            if s == u32::MAX {
                wait(&self.state, u32::MAX);
                s = self.state.load(Relaxed);
            }
        }
    }

    pub fn write(&self) -> WriteGuard<T> {
        while let Err(s) = self.state.compare_exchange(
            0, u32::MAX, Acquire, Relaxed
        ) {
            // Wait while already locked.
            wait(&self.state, s);
        }
        WriteGuard { rwlock: self }
    }
}

pub struct ReadGuard<'a, T> {
    rwlock: &'a RwLock<T>,
}

pub struct WriteGuard<'a, T> {
    rwlock: &'a RwLock<T>,
}

impl<T> Deref for WriteGuard<'_, T> {
    type Target = T;
    fn deref(&self) -> &T {
        unsafe { &*self.rwlock.value.get() }
    }
}

impl<T> DerefMut for WriteGuard<'_, T> {
    fn deref_mut(&mut self) -> &mut T {
        unsafe { &mut *self.rwlock.value.get() }
    }
}

impl<T> Deref for ReadGuard<'_, T> {
    type Target = T;
    fn deref(&self) -> &T {
        unsafe { &*self.rwlock.value.get() }
    }
}
```

```
impl<T> Drop for ReadGuard<'_, T> {
    fn drop(&mut self) {
        if self.rwlock.state.fetch_sub(1, Release) == 1 {
            // Wake up a waiting writer, if any.
            wake_one(&self.rwlock.state);
        }
    }
}

impl<T> Drop for WriteGuard<'_, T> {
    fn drop(&mut self) {
        self.rwlock.state.store(0, Release);
        // Wake up all waiting readers and writers.
        wake_all(&self.rwlock.state);
    }
}
```

Example: `rwlock_2.rs` (Rwlock 2)

```
use atomic_wait::{wait, wake_all, wake_one};
use std::cell::UnsafeCell;
use std::ops::{Deref, DerefMut};
use std::sync::atomic::AtomicU32;
use std::sync::atomic::Ordering::{Acquire, Relaxed, Release};

pub struct RwLock<T> {
    /// The number of readers, or u32::MAX if write-locked.
    state: AtomicU32,
    /// Incremented to wake up writers.
    writer_wake_counter: AtomicU32,
    value: UnsafeCell<T>,
}

unsafe impl<T> Sync for RwLock<T> where T: Send + Sync {}

impl<T> RwLock<T> {
    pub const fn new(value: T) -> Self {
        Self {
            state: AtomicU32::new(0),
            writer_wake_counter: AtomicU32::new(0),
            value: UnsafeCell::new(value),
        }
    }

    pub fn read(&self) -> ReadGuard<T> {
```

```
        let mut s = self.state.load(Relaxed);
        loop {
            if s < u32::MAX {
                assert!(s < u32::MAX - 1, "too many readers");
                match self.state.compare_exchange_weak(
                    s, s + 1, Acquire, Relaxed
                ) {
                    Ok(_) => return ReadGuard { rwlock: self },
                    Err(e) => s = e,
                }
            }
            if s == u32::MAX {
                wait(&self.state, u32::MAX);
                s = self.state.load(Relaxed);
            }
        }
    }

    pub fn write(&self) -> WriteGuard<T> {
        while self.state.compare_exchange(
            0, u32::MAX, Acquire, Relaxed
        ).is_err() {
            let w = self.writer_wake_counter.load(Acquire);
            if self.state.load(Relaxed) != 0 {
                // Wait if the RwLock is still locked, but only if
                // there have been no wake signals since we checked.
                wait(&self.writer_wake_counter, w);
            }
        }
        WriteGuard { rwlock: self }
    }
}

pub struct ReadGuard<'a, T> {
    rwlock: &'a RwLock<T>,
}

pub struct WriteGuard<'a, T> {
    rwlock: &'a RwLock<T>,
}

impl<T> Deref for WriteGuard<'_, T> {
    type Target = T;
    fn deref(&self) -> &T {
        unsafe { &*self.rwlock.value.get() }
    }
}
```

```
impl<T> DerefMut for WriteGuard<'_, T> {
    fn deref_mut(&mut self) -> &mut T {
        unsafe { &mut *self.rwlock.value.get() }
    }
}

impl<T> Deref for ReadGuard<'_, T> {
    type Target = T;
    fn deref(&self) -> &T {
        unsafe { &*self.rwlock.value.get() }
    }
}

impl<T> Drop for ReadGuard<'_, T> {
    fn drop(&mut self) {
        if self.rwlock.state.fetch_sub(1, Release) == 1 {
            self.rwlock.writer_wake_counter.fetch_add(1, Release); // New!
            wake_one(&self.rwlock.writer_wake_counter); // Changed!
        }
    }
}

impl<T> Drop for WriteGuard<'_, T> {
    fn drop(&mut self) {
        self.rwlock.state.store(0, Release);
        self.rwlock.writer_wake_counter.fetch_add(1, Release); // New!
        wake_one(&self.rwlock.writer_wake_counter); // New!
        wake_all(&self.rwlock.state);
    }
}
```

Example: `rwlock_3.rs` (Rwlock 3)

```
use atomic_wait::{wait, wake_all, wake_one};
use std::cell::UnsafeCell;
use std::ops::{Deref, DerefMut};
use std::sync::atomic::AtomicU32;
use std::sync::atomic::Ordering::{Acquire, Relaxed, Release};

pub struct RwLock<T> {
    /// The number of read locks times two, plus one if there's a writer waiting.
    /// u32::MAX if write locked.
    ///
    /// This means that readers may acquire the lock when
    /// the state is even, but need to block when odd.
```

```
    state: AtomicU32,
    /// Incremented to wake up writers.
    writer_wake_counter: AtomicU32,
    value: UnsafeCell<T>,
}

unsafe impl<T> Sync for RwLock<T> where T: Send + Sync {}

impl<T> RwLock<T> {
    pub const fn new(value: T) -> Self {
        Self {
            state: AtomicU32::new(0),
            writer_wake_counter: AtomicU32::new(0),
            value: UnsafeCell::new(value),
        }
    }

    pub fn read(&self) -> ReadGuard<T> {
        let mut s = self.state.load(Relaxed);
        loop {
            if s % 2 == 0 { // Even.
                assert!(s < u32::MAX - 2, "too many readers");
                match self.state.compare_exchange_weak(
                    s, s + 2, Acquire, Relaxed
                ) {
                    Ok(_) => return ReadGuard { rwlock: self },
                    Err(e) => s = e,
                }
            }
            if s % 2 == 1 { // Odd.
                wait(&self.state, s);
                s = self.state.load(Relaxed);
            }
        }
    }

    pub fn write(&self) -> WriteGuard<T> {
        let mut s = self.state.load(Relaxed);
        loop {
            // Try to lock if unlocked.
            if s <= 1 {
                match self.state.compare_exchange(
                    s, u32::MAX, Acquire, Relaxed
                ) {
                    Ok(_) => return WriteGuard { rwlock: self },
                    Err(e) => { s = e; continue; }
                }
```

```
            }
            // Block new readers, by making sure the state is odd.
            if s % 2 == 0 {
                match self.state.compare_exchange(
                    s, s + 1, Relaxed, Relaxed
                ) {
                    Ok(_) => {}
                    Err(e) => { s = e; continue; }
                }
            }
            // Wait, if it's still locked
            let w = self.writer_wake_counter.load(Acquire);
            s = self.state.load(Relaxed);
            if s >= 2 {
                wait(&self.writer_wake_counter, w);
                s = self.state.load(Relaxed);
            }
        }
    }
}

pub struct ReadGuard<'a, T> {
    rwlock: &'a RwLock<T>,
}

pub struct WriteGuard<'a, T> {
    rwlock: &'a RwLock<T>,
}

impl<T> Deref for WriteGuard<'_, T> {
    type Target = T;
    fn deref(&self) -> &T {
        unsafe { &*self.rwlock.value.get() }
    }
}

impl<T> DerefMut for WriteGuard<'_, T> {
    fn deref_mut(&mut self) -> &mut T {
        unsafe { &mut *self.rwlock.value.get() }
    }
}

impl<T> Deref for ReadGuard<'_, T> {
    type Target = T;
    fn deref(&self) -> &T {
        unsafe { &*self.rwlock.value.get() }
    }
```

```
}

impl<T> Drop for ReadGuard<'_, T> {
    fn drop(&mut self) {
        // Decrement the state by 2 to remove one read-lock.
        if self.rwlock.state.fetch_sub(2, Release) == 3 {
            // If we decremented from 3 to 1, that means
            // the RwLock is now unlocked _and_ there is
            // a waiting writer, which we wake up.
            self.rwlock.writer_wake_counter.fetch_add(1, Release);
            wake_one(&self.rwlock.writer_wake_counter);
        }
    }
}

impl<T> Drop for WriteGuard<'_, T> {
    fn drop(&mut self) {
        self.rwlock.state.store(0, Release);
        self.rwlock.writer_wake_counter.fetch_add(1, Release);
        wake_one(&self.rwlock.writer_wake_counter);
        wake_all(&self.rwlock.state);
    }
}
```

Ideas and Inspiration

Ideas and Inspiration

There are an infinite number of concurrency related topics, algorithms, data structures, anecdotes, and other potential chapters that could be part of this book. However, we've arrived at the final chapter and it's almost time for us to part ways, hopefully leaving you with an excited feeling of new possibilities and ready to apply new knowledge and skills in practice.

This final chapter's purpose is to provide inspiration for your own creations and future work by showing you some ideas that you can study, explore, and build on your own.

Semaphore

A *semaphore* is effectively just a counter with two operations: *signal* (also called *up* or *V*) and *wait* (also called *down* or *P*). The signal operation increments the counter up to a certain maximum, while a wait operation decrements the counter. If the counter is zero, a wait operation will block and wait for a matching signal operation, preventing the counter from ever becoming negative. It is a flexible tool that can be used to implement other synchronization primitives.

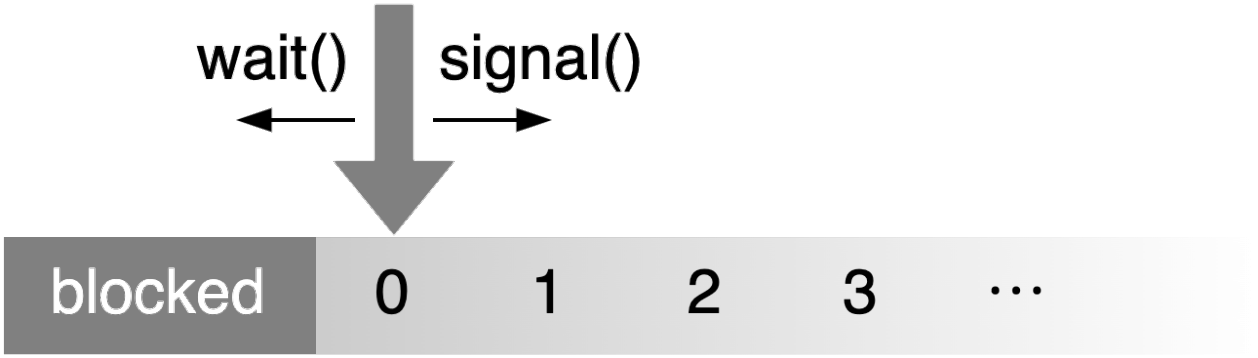


Figure 1: Visualization of a semaphore. A seemingly infinite strip with integers starting at zero, with the word

A semaphore can be implemented as a combination of a `Mutex<u32>` for the counter and a `Condvar` for wait operations to wait for. However, there are several ways to implement it more efficiently. Most notably, on platforms that support futex-like operations ("Futex" in Chapter 8), it can be implemented more efficiently as a single `AtomicU32` (or even `AtomicU8`).

A semaphore with a maximum value of one is sometimes called a *binary semaphore*, and can be used as a building block with which to build other primitives. For example, it can be used as a mutex by initializing the counter at one, using the wait operation for locking, and the signal operation for unlocking. By initializing it at zero, it can also be used for signaling, like a condition variable.

For example, the standard `park()` and `unpark()` functions in `std::thread` can be implemented as wait and signal operations on a binary semaphore associated with the thread.

> **Note**

Note how a mutex can be implemented using a semaphore, while a semaphore can be implemented using a mutex (and a condition variable). It's advisable to avoid using a mutex-based semaphore to implement a semaphore-based mutex, and the other way around.

Further reading:

- Wikipedia article on semaphores
- Stanford University course notes on semaphores

RCU

If you want to allow multiple threads to (mostly) read and (sometimes) mutate some data, you can use an `RwLock`. When this data is just a single integer, you can use an atomic variable (such as `AtomicU32`) to avoid locking, which is more efficient. However, for larger chunks of data, like a struct with many fields, there's no available atomic type that allows for lock-free atomic operations on the entire object.

Just like every other problem in computer science, this problem can be solved by adding a layer of indirection. Instead of the struct itself, you can use an atomic variable to store a pointer to it. This still doesn't allow you to modify the struct as a whole atomically, but it does allow you to replace the entire struct atomically, which is nearly as good.

This pattern is often called *RCU*, which stands for "read, copy, update," the steps necessary to replace the data. After reading the pointer, the struct can be copied into a new allocation that can be modified without worrying about other threads. When ready, the atomic pointer can be updated using a compare-and-exchange operation ("Compare-and-Exchange Operations" in Chapter 2), which will only succeed if no other thread has replaced the data in the meantime.

The most interesting part about the RCU pattern is the last step, which does not have a letter in the acronym: deallocating the old data. After a successful update, other threads might still be reading the old copy, if they read the pointer before the update. You'll have to wait for all those threads to be done before the old copy can be deallocated.

There are many possible solutions for this issue, including reference counting (like `Arc`), leaking memory (ignoring the problem), garbage collection, hazard pointers (a way for threads to tell the others what pointers they are currently using), and quiescent state tracking (waiting for each thread to reach a point at which it is definitely not using any pointers). The last one can be extremely efficient in certain conditions.

Many data structures in the Linux kernel are RCU based, and there are many interesting talks and articles about their implementation details that can provide a great deal of inspiration.

Further reading:

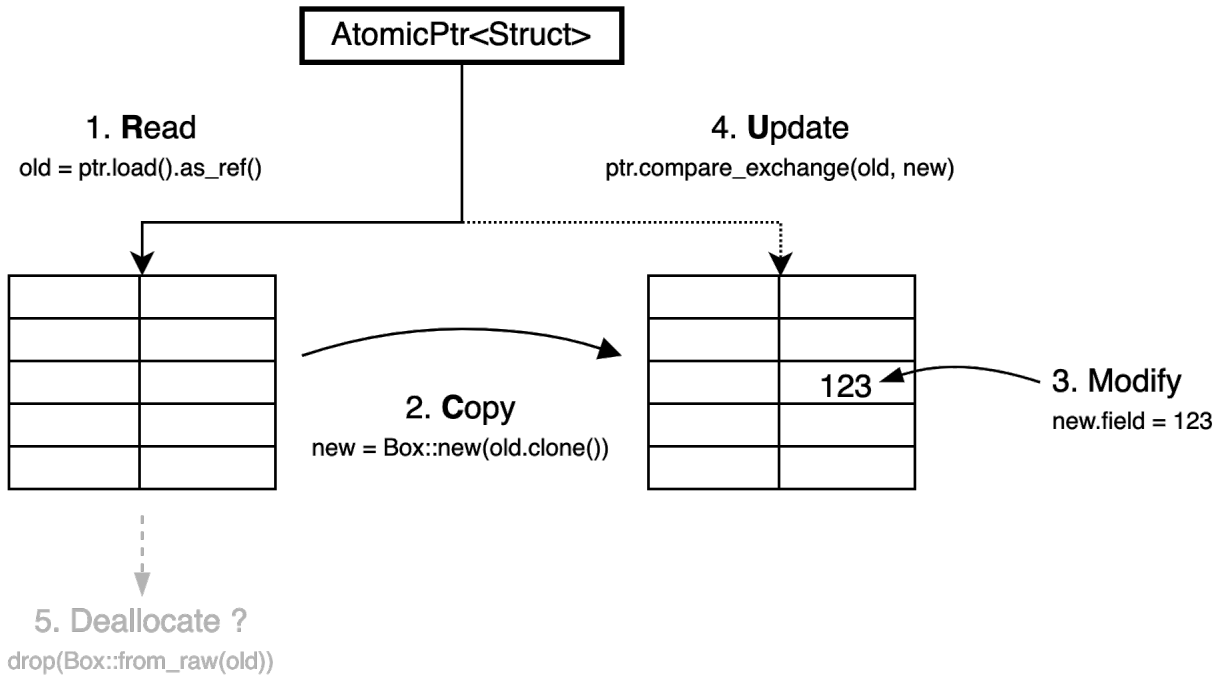


Figure 2: Visualization of the RCU pattern showing five steps. Step 1, read, shows an arrow from an atomic pointer to a struct it points at. Step 2, copy, copies the entire struct into a new allocation. Step 3, modify, modifies a single field of the copy. Step 4, update, is a compare-exchange operation to update the atomic pointer to point at the newly allocated copy. Step 5, deallocate, deallocates the original struct, but this step is grayed out and has a question mark.

- Wikipedia article on the read-copy-update pattern
- LWN article "What is RCU, Fundamentally?"

Lock-Free Linked List

Expanding on the basic RCU pattern, you can add an atomic pointer to the struct to point to the next one, to turn it into a *linked list*. This allows for threads to atomically add or remove elements on this list, without having to copy the entire list for every update.

To insert a new element at the start of the list, you only have to allocate that element and point its pointer at the first element in the list, and then atomically update the initial pointer to point to your newly allocated element.

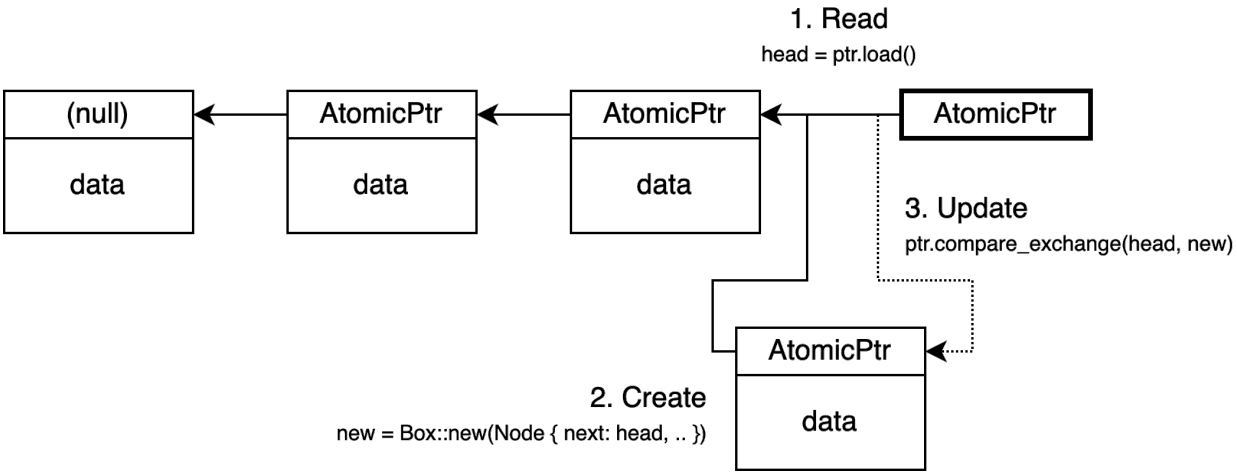


Figure 3: Visualization of a singly linked list with three nodes, that is getting a fourth node inserted at the head in three steps. Step 1, read, shows an arrow from an atomic pointer to the first node. Step 2, create, is the allocation and creation of a new node that points at the first node from step 1. Step 3, update, is a compare-exchange operation to update the first atomic pointer to point at the newly created node.

Similarly, removing an element can be done by atomically updating the pointer before it to point to the element after it. However, when multiple writers are involved, care must be taken to handle concurrent insertion or removal operations on neighboring elements. Otherwise, you might accidentally also remove a concurrently newly inserted element, or undo the removal of a concurrently removed element.

> **Note**

To keep things simple, you can use a regular mutex to avoid concurrent mutations. That way, reading is still a lock-free operation, but you don't have to worry about handling concurrent mutation.

After detaching an element from the linked list, you'll run into the same issue as before: waiting until you can deallocate it (or otherwise claim ownership). The same solutions we discussed for the basic RCU pattern can work in this case as well.

In general, you can build a wide variety of elaborate lock-free data structures based on compare-and-exchange operations on atomic pointers, but you'll always need a good strategy for deallocating or otherwise reclaiming ownership of the allocations.

Further reading:

- Wikipedia article on non-blocking linked lists
- LWN article "Using RCU for Linked Lists—A Case Study"

Queue-Based Locks

For most standard locking primitives, the operating system's kernel keeps track of the threads that are blocked on it, and is responsible for picking one to wake up when asked to do so. An interesting alternative is to implement a mutex (or other locking primitive), by manually keeping track of the queue of waiting threads.

Such a mutex could be implemented as a single `AtomicPtr` that can point to a (list of) waiting threads.

Each element in this list needs to contain something that can be used to wake up the corresponding thread, such as a `std::thread::Thread` object. Some unused bits of the atomic pointer can be used to store the state of the mutex itself, and whatever is necessary for managing the state of the queue.

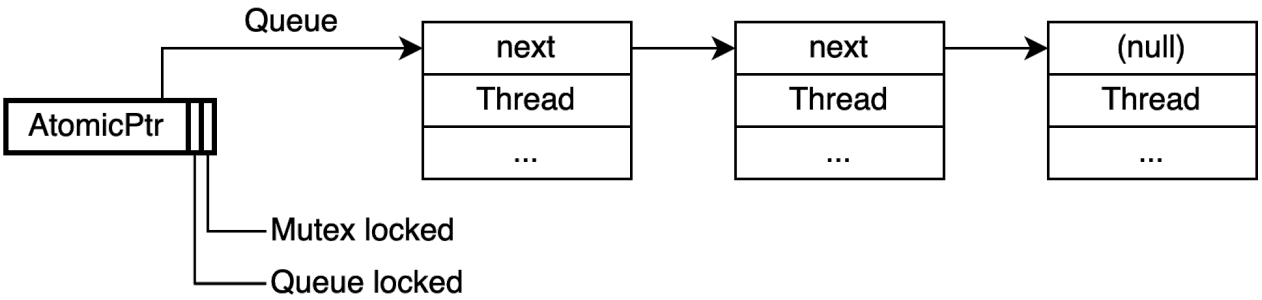


Figure 4: Visualization of a queue based lock. The lock is an atomic pointer, of which the last two bits are labeled

There are many variations possible. The queue could be protected by its own lock bit or it could be implemented as a (partially) lock-free structure. The elements don't have to be allocated on the heap, but could be local variables of the threads that are waiting. The queue could be a doubly-linked list with not only pointers to the next element, but also to the previous element. The first element could also include a pointer to the last element to allow efficiently appending an element at the end.

This pattern allows for implementing efficient locking primitives using only something that can be used to block and wake up a single thread, such as thread parking.

Windows SRW locks ("Slim reader-writer locks" in Chapter 8) are implemented using this pattern.

Further reading:

- Notes on the implementation of Windows SRW locks
- A Rust implementation of queue-based locks

Parking Lot–Based Locks

To make a highly efficient mutex that's as small as possible, you can build upon the queue-based locks idea by moving the queue into a global data structure, leaving only one or two bits inside the mutex itself. This way, the mutex only needs to be a single byte. You could even put it in some unused bits of a pointer, allowing for very fine-grained locking at almost no extra cost.

The global data structure can be a `HashMap` that maps memory addresses to a queue of threads waiting on the mutex at that address. This global data structure is often called a *parking lot*, since it's a collection of parked threads.

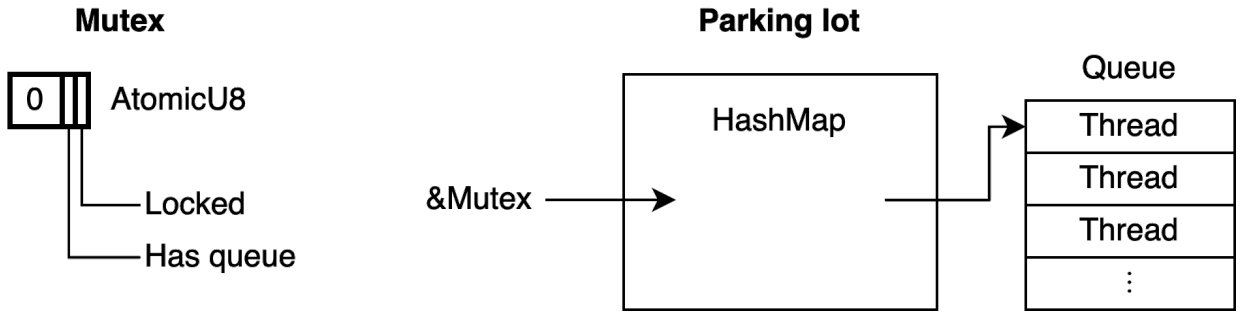


Figure 5: Visualization of a mutex and parking lot. The mutex is an AtomicU8 of which the last two bits are labeled

The pattern can be generalized by not only tracking queues for mutexes, but also for condition variables and other primitives. By tracking a queue for any atomic variable, this effectively provides a way to implement futex-like functionality on platforms that don't natively support that.

This pattern is most well known from its 2015 implementation in WebKit, where it was used for locking JavaScript objects. Its implementation inspired other implementations, such as the popular `parking_lot` Rust crate.

Further reading:

- WebKit blog post, "Locking in WebKit"
- Documentation of the `parking_lot` crate

Sequence Lock

A sequence lock is another solution to the problem of atomically updating (larger) data without using traditional (blocking) locks. It uses an atomic counter that is odd while the data is being updated, and even when the data is ready to be read.

The writing thread will have to increment the counter from even to odd before mutating the data, after which it has to increment the counter again to leave it at a (different) even value.

Any reading thread can, at any point and without blocking, read the data by reading the counter both before and after. If the two values from the counter are equal and even, there was no concurrent mutation, meaning you read a valid copy of the data. Otherwise, you might have read data that was concurrently being modified, in which case you should just try again.

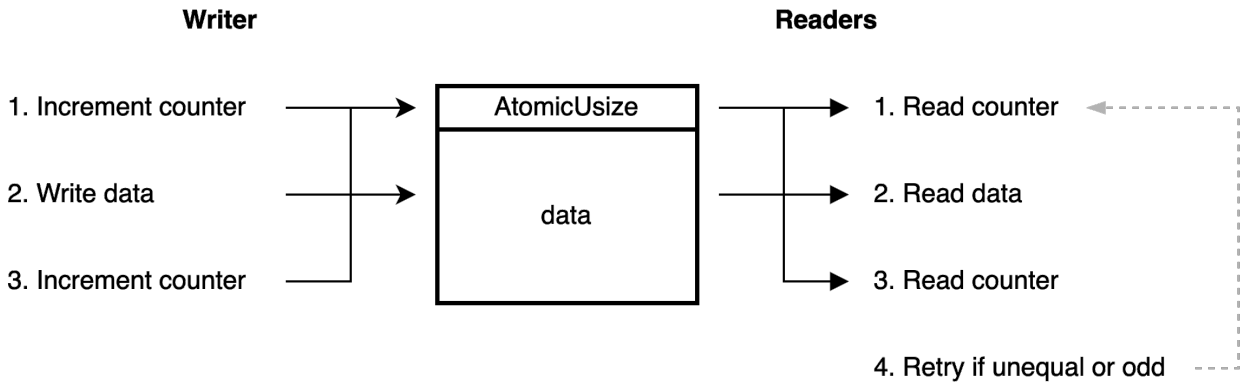


Figure 6: Visualization of a sequence lock, an AtomicUsize followed by a larger chunk of data. A writer has three steps: increment counter, write data, increment counter again. Readers have four steps: read counter, read data, read counter, and retry if unequal or odd.

This is a great pattern for making data available to other threads, without the possibility of the reading threads blocking the writing thread. It is often used in operating system kernels and many embedded systems. Since the readers need only read access to the memory and no pointers are involved, this can be a great data structure to safely use in shared memory, between processes, without needing to trust the readers. For example, the Linux kernel uses this pattern to very efficiently provide timestamps to processes by providing them with read-only access to (shared) memory.

An interesting question is how this fits into the memory model. Concurrent non-atomic reads and writes to the same data result in undefined behavior, even if the read data is ignored. This means that, technically speaking, both reading and writing the data should be done using only atomic operations, even though the entire read or write does not have to be a single atomic operation.

Further reading:

- Wikipedia article on Linux's Seqlock
- Rust RFC 3301, `AtomicPerByte`
- Documentation of the `seqlock` crate

Teaching Materials

It can be great fun to spend many hours—or years—inventing new concurrent data structures and designing ergonomic Rust implementations of them. If you're looking for something else to do with

your knowledge on Rust, atomics, locks, concurrent data structures, and concurrency in general, it can also be very fulfilling to create new teaching materials to share your knowledge with others.

There is a great lack of accessible resources aimed at those new to these topics. Rust has played a significant role in making systems programming more accessible to everyone, but many programmers still shy away from low-level concurrency. Atomics are often thought of as a somewhat mystical topic that's best left to a very small group of experts, which is a shame.

I hope this very book makes a significant difference, but there is so much space for more books, blog posts, articles, video courses, conference talks, and other materials about Rust concurrency.

~

I'm excited to see what you create.

Good luck.